ПОШАГОВЫЙ ГАЙД ВКР

Новый вводный блок: как из идеи получить скелет, архитектуру и порядок реализации

Этот блок добавлен перед прежним учебником. Его задача - закрыть самый трудный провал: не строку кода объяснить, а показать, как вообще рождается структура программы. Сначала идея превращается в сущности, потом сущности в структуры данных, потом структуры в модули, потом модули в функции, а уже после этого пишется код.

0. Универсальный путь от идеи к программе

Когда проект сложный, нельзя начинать с `main.c` и сразу писать код. Сначала надо построить мысленный каркас. Каркас отвечает на вопросы: что приходит на вход, что должно выйти, какие промежуточные модели нужны, какие проверки обязательны, какие модули отвечают за отдельные слои, как проверить каждый слой отдельно.

```
идея проекта
  -> цель и пользователь
  -> входные данные
  -> выходные данные
  -> ограничения и опасности
  -> сущности предметной области
  -> структуры данных C
  -> модули
  -> функции внутри модулей
  -> порядок реализации
  -> тесты и критерии готовности
```

Вопрос	Что должен ответить разработчик	Во что это превращается в C
Зачем программа нужна?	какую проблему решает и для кого	README, usage, CLI options
Что приходит на вход?	файл, аргументы, режимы, лимиты	`Args`, `Buf`, `DynamicReport` options
Что должно выйти?	отчет, таблица, verdict, код завершения	JSON/HTML/CSV, console output
Какие сущности есть?	PE-образ, секция, функция, unwind-info, finding	`PeImage`, `PeSection`, `RuntimeFunction`, `UnwindSummary`
Какие границы опасны?	размер файла, SizeOfImage, RVA, section ranges	bounds checks, OOB findings
Какие слои независимы?	байты, PE, unwind, эвристики, отчет	отдельные `.c/.h` модули
Как проверять?	unit, regression, fuzz, CLI contract	`tests/unit`, Python tests, mutation fuzz

1. Как идея ВКР раскладывается на программу

Исходная идея звучит примерно так: сделать инструмент, который анализирует Windows x64 PE-файлы и объясняет странности в EH/Unwind metadata. Для программиста это еще не архитектура. Архитектура появляется, когда мы переводим эту фразу в конкретные входы, выходы, слои и структуры данных.

Фраза из идеи	Инженерный смысл	Артефакт в проекте
Windows x64 PE-файл	вход недоверенный, бинарный, с адресами и таблицами	`Buf`, `pe_parse`, `PeImage`
EH/Unwind metadata	нужно читать exception directory, .pdata/.xdata	`RuntimeFunction`, `UnwindSummary`
анализирует странности	сырые факты надо превратить в признаки	`heuristics_analyze`, `HeuristicFinding`
объясняет	нужен отчет, понятный человеку и тестам	`report_write_json/html/csv`
статический и динамический режим	часть работает по файлу, часть через Windows Debug API	`dynamic_collect_windows`
подготовка к сдаче	должны быть тесты, воспроизводимость, версии	`app_info`, tests, schema, release hygiene

Что является одной записью в этом проекте

В бытовых проектах одной записью может быть расход или контакт. В ВКР одной записи мало: проект многоуровневый. Поэтому здесь несколько типов записей, и каждый тип соответствует своему слою анализа.

Слой	Одна запись слоя	Структура
PE section table	одна секция PE	`PeSection`
PE image model	один разобранный образ	`PeImage`
Function table	одна запись функции	`RuntimeFunction`
Unwind info	одна сводка unwind-info	`UnwindSummary`
Heuristics	один признак/finding	`HeuristicFinding`
Dynamic	одно runtime-событие	`DynamicEvent`
Report	один итог анализа	`HeuristicReport`, `DynamicReport`, `ReportMetadata`

2. Конкретный скелет ВКР

Скелет проекта - это не полный код. Это схема, которая показывает, кто кому передает данные и почему модули расположены именно так. Если этот скелет понятен, потом большой код перестает быть хаосом: каждая функция занимает свое место.

```
eh_unwind_analyzer
  main.c
    -> читает аргументы
    -> запускает pipeline
    -> управляет ресурсами и кодом возврата

  util.c / util.h
    -> безопасно читает файл в Buf
    -> дает StrBuf для отчетов и ошибок
    -> считает SHA-256

  pe.c / pe.h
    -> превращает байты PE в PeImage
    -> проверяет DOS/NT headers, sections, SizeOfImage, exception directory
    -> умеет переводить RVA в file offset

  unwind.c / unwind.h
    -> читает RUNTIME_FUNCTION
    -> разбирает UNWIND_INFO
    -> строит UnwindSummary

  heuristics.c / heuristics.h
    -> превращает PE/unwind факты в 27 findings
    -> считает score, suspicion, confidence, verdict

  dynamic_win.c / dynamic.h
    -> при необходимости запускает Windows runtime validation
    -> собирает DynamicEvent и DynamicReport

  report.c / report.h
    -> пишет brief, CSV, JSON, HTML
    -> делает результат понятным человеку и проверяемым тестами
```

Модуль	Что принимает	Что отдает	Чего не должен делать
util	путь к файлу, строки	`Buf`, `StrBuf`, hash	не должен знать PE-формат
pe	`Buf`	`PeImage`	не должен считать подозрительность
unwind	`Buf`, `PeImage`	`RuntimeFunction[]`, `UnwindSummary[]`	не должен решать verdict
heuristics	`PeImage`, runtime functions, summaries	`HeuristicReport`	не должен писать HTML/JSON напрямую
dynamic	путь, static model, options	`DynamicReport`	не должен менять статический parser
report	все готовые модели	console/CSV/JSON/HTML	не должен заново парсить PE
main	CLI arguments	exit code	не должен содержать всю бизнес-логику

3. Путь данных: от файла до verdict

Самый надежный способ понять ВКР - проследить движение данных. Входной PE-файл постепенно превращается в более высокоуровневые модели. Каждый следующий слой доверяет не сырым байтам, а результату предыдущего слоя.

```
PE file on disk
  -> Buf: просто байты + размер
  -> PeImage: заголовки, секции, ImageBase, SizeOfImage, exception directory
  -> RuntimeFunction[]: таблица функций из .pdata
  -> UnwindSummary[]: разобранные UNWIND_INFO из .xdata/.rdata
  -> HeuristicReport: findings, score, confidence, verdict
  -> DynamicReport: runtime-события, если включен dynamic mode
  -> Report files: JSON, HTML, CSV, console summary
```

Шаг pipeline	Главный вопрос	Если ответ плохой
read file	можно ли безопасно загрузить файл?	ошибка запуска, анализа нет
parse PE	это PE32+ x64 и секции согласованы?	PE findings или parse error
exception directory	где .pdata и читается ли она?	`pe_exception_directory_oob`
runtime functions	валидны ли Begin/End/UnwindInfoAddress?	`invalid_ranges`, `rf_oob`, `unwind_rva_oob`
unwind info	можно ли разобрать opcode/flags/handler?	`unwind_parse_fail`, `unwind_anomalies`
placement	metadata и handler лежат в разумных секциях?	`unwind_writable_section`, `handler_nonexec_section`
reuse/dominance	нет ли слишком сильного повторного использования?	`unwind_reuse_*`, `handler_dominance`
correlation	усиливают ли признаки друг друга?	combo findings
verdict	какой итог и насколько ему верить?	`score`, `confidence`, `requires_manual_review`

4. Как из сущностей получаются структуры C

Структура C появляется не потому, что так захотелось. Она появляется, когда у предметной сущности есть несколько полей, которые всегда должны ходить вместе. Например, секция PE - это не только имя. У нее есть virtual address, virtual size, raw pointer, raw size и characteristics. Значит, нужна `PeSection`.

Сущность	Почему это отдельная структура	Ключевые поля
`Buf`	все чтение должно знать указатель на байты и размер	data, size
`PeSection`	секция имеет имя, raw-границы, virtual-границы и права	name, virtual_address, raw_ptr, raw_size, characteristics
`PeImage`	это итоговая модель PE-образа	machine, image_base, size_of_image, exception_rva, sections, counters
`RuntimeFunction`	одна строка .pdata описывает диапазон функции	begin_rva, end_rva, unwind_rva
`UnwindSummary`	сырой UNWIND_INFO надо сжать до удобной сводки	flags, has_handler, handler_rva, stack, anomalies
`HeuristicFinding`	finding должен быть стабильным элементом отчета	code, family, severity, message, representative_rva
`HeuristicReport`	нужен один контейнер для всех признаков и verdict	score, confidence, counts, findings, mitigations
`DynamicReport`	runtime validation дает много событий и счетчиков	events, trace, handler hits, VEH/VCH/UEF counters

Правило проектирования структур

- Если несколько значений описывают один объект - делай `struct`.
- Если объект выделяет память внутри себя - рядом нужна функция `free`.
- Если структура является результатом парсинга - в ней должны быть флаги валидности и счетчики ошибок.
- Если поле участвует в отчете - оно должно быть стабильным, понятным и тестируемым.
- Если поле является адресом PE - в имени лучше явно писать `rva`, `raw`, `offset`, чтобы не смешать координатные системы.

5. Как скелет обрастает мясом: порядок реализации

Большой проект нельзя написать за один проход. Его выращивают вертикальными слоями: сначала минимальный скелет, потом чтение файла, потом PE, потом unwind, потом эвристики, потом отчеты, потом dynamic и тесты. На каждом этапе должна быть проверка, иначе дальше ты строишь поверх ошибки.

Итерация	Что добавляем	Новые структуры/функции	Как проверить
0. Empty CLI	программа запускается, показывает usage/version	`main`, `Args`, `app_info`	`--version`, запуск без аргументов
1. File bytes	чтение файла в память	`Buf`, `buf_init_from_file`, `sha256_hex`	прочитать любой exe, вывести размер/hash
2. PE identity	DOS/NT headers, x64 checks	`pe_parse` начало	отсеять не-PE и не-x64
3. Sections	section table, RVA/raw mapping	`PeSection`, `pe_rva_to_fileoff`	проверить секции и OOB
4. Exception directory	найти .pdata	exception_rva/size	пустая/битая таблица дает понятный result
5. Function table	прочитать RUNTIME_FUNCTION[]	`unwind_read_runtime_functions`	количество функций совпадает с размером directory
6. Unwind parser	разобрать UNWIND_INFO	`UnwindSummary`, `unwind_parse`	unit tests на opcode/flags
7. Findings	счетчики и 27 признаков	`HeuristicReport`, `add_finding`	synthetic samples дают ожидаемые findings
8. Reports	JSON/HTML/CSV	`report_write_*`	schema/semantic tests
9. Dynamic	Windows Debug API evidence	`DynamicReport`, `DynamicEvent`	dynamic samples VEH/VCH/UEF
10. Release	чистота, документация, упаковка	tests, docs, CPack	ctest, release hygiene

6. Как программировать один модуль

Чтобы не теряться, каждый модуль можно писать по одному рецепту. Сначала публичный контракт в `.h`, потом реализация в `.c`, потом маленький тест. Так проект не превращается в один огромный `main.c`.

```
module.h
  1. include guard / #pragma once
  2. нужные стандартные заголовки
  3. структуры, которые видят другие модули
  4. публичные функции
  5. комментарий: кто владеет памятью

module.c
  1. include "module.h"
  2. private helpers: static functions
  3. основная публичная функция
  4. cleanup/free function
  5. понятные сообщения ошибок через StrBuf

test
  1. нормальный случай
  2. пустой/битый вход
  3. граница размера
  4. проверка, что ошибка не приводит к crash
```

Модуль	Минимальный первый вариант	Потом наращиваем
util	прочитать файл и проверить bounds	StrBuf, atomic write, SHA-256
pe	проверить MZ/PE/x64	sections, alignments, exception directory
unwind	прочитать runtime functions	opcode parser, chain, ABI warnings
heuristics	один finding	27 findings, weights, confidence, mitigations
report	brief print	CSV, JSON schema, HTML, Russian explanations
dynamic	инициализация empty report	debug loop, breakpoints, events, trace
main	parse input path	all options, child args, cleanup, exit codes

7. Общий C-скелет любой полезной программы

Когда ты не понимаешь, с чего начать программу, начни не с алгоритма, а с жизненного цикла данных. Почти любая консольная программа на C имеет одну и ту же основу: разобрать аргументы, получить входные данные, превратить вход в удобную модель, выполнить анализ или действие, сохранить результат и освободить память. Это и есть скелет. Алгоритмы потом вставляются внутрь отдельных шагов.

```
int main(int argc, char **argv) {
    Args args = args_parse(argc, argv);
    if (!args.ok) {
        print_usage();
        return 2;
    }

    Buf input = {0};
    if (!buf_read_file(args.input_path, &input)) {
        print_error("cannot read input file");
        return 1;
    }

    Model model = {0};
    if (!model_parse(&input, &model)) {
        print_error("cannot parse input");
        buf_free(&input);
        return 1;
    }

    Analysis analysis = {0};
    analysis_run(&model, &analysis);

    int report_rc = report_write(&analysis, args.output_path);

    analysis_free(&analysis);
    model_free(&model);
    buf_free(&input);

    return report_rc == 0 ? 0 : 1;
}
```

Часть скелета	Что она значит технически	Что она значит логически
`argc/argv`	массив строк, переданный программе операционной системой	пользователь говорит программе, что делать и с каким файлом
`Args`	структура с разобранными параметрами	мы превращаем хаотичные строки командной строки в понятный договор
`Buf`	указатель на байты и размер	входной файл становится контролируемым объектом, у которого есть границы
`Model`	структуры предметной области	сырые байты превращаются в смысл: PE-образ, секции, функции, unwind
`Analysis`	результаты проверок, счетчики, findings	программа отвечает на вопрос: что в объекте нормально, а что подозрительно
`report_write`	запись JSON/HTML/CSV/console	результат становится пригодным для человека, тестов и комиссии
`free`	освобождение памяти в обратном порядке	программа не оставляет мусор и не прячет ошибки владения памятью

Как этим пользоваться при любом новом проекте

- Сначала напиши такой скелет с пустыми структурами и заглушками функций, чтобы проект компилировался.
- Потом замени `Model` на реальные сущности твоей предметной области.
- Потом наполни `model_parse`: именно там вход превращается в структуру данных.
- Потом наполни `analysis_run`: именно там появляются правила, проверки и выводы.
- Потом напиши отчеты и тесты, потому что без них ты не докажешь, что программа делает то, что обещает.

8. Конкретный скелет main.c для ВКР

Для ВКР общий скелет превращается в конкретную цепочку: файл читается как `Buf`, затем разбирается как `PeImage`, потом из exception directory читается таблица `RuntimeFunction`, потом каждая функция получает `UnwindSummary`, после этого `heuristics.c` строит 27 признаков и verdict, а `report.c` записывает результат. Ниже не финальный исходник, а логический каркас, который объясняет, куда потом вставляется настоящий код из учебника.

```
int main(int argc, char **argv) {
    AppOptions opt = {0};
    Buf file = {0};
    PeImage pe = {0};
    RuntimeFunction *rf = NULL;
    size_t rf_count = 0;
    HeuristicReport static_report = {0};
    DynamicReport dynamic_report = {0};
    StrBuf errors = {0};

    if (!parse_args(argc, argv, &opt)) {
        print_usage();
        return EXIT_USAGE;
    }

    if (!buf_read_file(opt.input_path, &file, &errors)) {
        report_input_error(&errors);
        goto fail;
    }

    if (!pe_parse(&file, &pe, &errors)) {
        report_parse_error(&errors);
        goto fail;
    }

    if (!unwind_read_runtime_functions_ex(&pe, &rf, &rf_count, &errors)) {
        note_exception_directory_problem(&static_report, &errors);
    }

    heuristics_analyze(&pe, rf, rf_count, &static_report);

    if (opt.dynamic_enabled) {
        dynamic_collect_windows(&opt, &dynamic_report, &errors);
    }

    if (!report_write_all(&opt, &pe, &static_report, &dynamic_report, &errors)) {
        goto fail;
    }

    free(rf);
    dynamic_report_free(&dynamic_report);
    heuristic_report_free(&static_report);
    pe_free(&pe);
    buf_free(&file);
    strbuf_free(&errors);
    return EXIT_OK;

fail:
    free(rf);
    dynamic_report_free(&dynamic_report);
    heuristic_report_free(&static_report);
    pe_free(&pe);
    buf_free(&file);
    strbuf_free(&errors);
    return EXIT_ERROR;
}
```

Строка/блок	Зачем нужен в ВКР	Почему без него проект разваливается
`AppOptions opt`	хранит режимы запуска: input, output, JSON/HTML, dynamic	иначе настройки размазываются по всему `main.c`
`Buf file`	безопасная оболочка над байтами PE-файла	нельзя парсить недоверенный файл без размера и bounds checks
`PeImage pe`	координатная система PE: headers, sections, ImageBase, SizeOfImage	без нее нельзя понять, куда указывает RVA
`RuntimeFunction *rf`	массив записей .pdata	без него нет карты функций и нечего связывать с unwind info
`HeuristicReport`	контейнер findings, score, confidence, verdict	признаки должны быть собраны в одном проверяемом результате
`DynamicReport`	доказательства runtime-слоя, если включен dynamic mode	динамический анализ нельзя смешивать с парсером PE
`goto fail`	единая точка аварийной очистки	в C легко забыть освободить память при раннем выходе

Главная мысль этого скелета

`main.c` не должен знать все детали PE и unwind. Его задача - вести поток данных сверху вниз: прочитал, разобрал, проанализировал, записал отчет, освободил. Если в `main.c` начинает появляться ручной разбор секций, opcode или весов признаков, значит архитектура ломается: детали надо переносить в `pe.c`, `unwind.c` или `heuristics.c`.

9. Почему признаки появляются из архитектуры, а не из воздуха

27 признаков не надо воспринимать как случайный список. Они появляются из слоев архитектуры. Каждый слой имеет свой контракт. Если контракт нарушен, появляется finding. Именно поэтому сначала важен скелет: без него непонятно, откуда вообще берутся признаки.

Архитектурный слой	Контракт	Тип findings
PE section table	raw ranges внутри файла, virtual ranges внутри SizeOfImage	`pe_section_*`, `pe_exception_directory_oob`
Runtime function table	Begin < End, адреса внутри образа, таблица упорядочена	`invalid_ranges`, `rf_oob`, `rf_unsorted`, `rf_overlaps`
Unwind info parser	UNWIND_INFO читается, opcode/operand имеют смысл	`unwind_parse_fail`, `unwind_anomalies`
Metadata placement	unwind metadata обычно data, handler обычно executable	`unwind_writable_section`, `handler_nonexec_section`
Reuse/dominance	повторное использование не должно выглядеть как массовая подмена	`unwind_reuse_*`, `handler_dominance`
Correlation	несколько слоев одновременно усиливают подозрение	`*_combo` findings

Формула одного finding

```
сырое наблюдение
  -> счетчик или флаг в структуре
  -> условие в heuristics_analyze
  -> add_score(...)
  -> add_finding(code, family, severity, message)
  -> annotate_finding(location, affected_count)
  -> JSON/HTML/CSV report
```

10. Если писать ВКР с нуля: с чего начать завтра

Ниже не теория, а практический маршрут. Если ты хочешь повторить проект руками, иди сверху вниз. Не переходи к следующему пункту, пока предыдущий не компилируется и не проверен.

Шаг	Что написать	Что понять	Готово, если
1	пустой `main.c` с usage/version	CLI - входная дверь проекта	запуск без аргументов понятен
2	`util.h/c`: `Buf`, чтение файла	недоверенный бинарник читается как bytes+size	битый путь дает ошибку, нормальный файл читается
3	MZ/PE/x64 checks	формат начинается с координатной системы	не-PE не проходит
4	section table	raw != virtual, есть file bounds и image bounds	секции выводятся и проверяются
5	exception directory	DataDirectory[3] указывает на .pdata	таблица найдена или понятно отсутствует
6	runtime functions	одна запись = begin/end/unwind	видно число функций
7	unwind parser	UNWIND_INFO описывает выход из кадра	есть summary по каждой функции
8	первые 5 findings	finding = нарушение контракта слоя	синтетический файл дает expected finding
9	score/confidence/verdict	оценка отделена от факта	verdict меняется от набора findings
10	JSON/HTML/CSV	результат должен быть проверяемым и читаемым	тесты читают JSON
11	dynamic optional	runtime evidence отдельно от static parser	на Windows собирается DynamicReport
12	tests/release/docs	проект готовится к сдаче	ctest проходит, мусора нет

Контрольные вопросы перед переходом к старому гайду

- Можешь ли ты объяснить, почему `Buf` ниже `PeImage`, а не наоборот?
- Понимаешь ли ты разницу между raw offset, RVA и virtual address?
- Можешь ли ты сказать, какая структура соответствует одному слою анализа?
- Понимаешь ли ты, почему `heuristics.c` не должен сам читать PE-файл?
- Можешь ли ты объяснить путь одного finding от счетчика до HTML/JSON отчета?
- Понимаешь ли ты, почему dynamic-слой отдельный и Windows-only?

После этого блока начинается прежний подробный учебник с полным кодом и построчным разбором. Теперь его надо читать уже не как стену текста, а как заполнение скелета, который описан выше.

ПОШАГОВЫЙ ГАЙД ВКР

Учебник с нуля по C-проекту EH/Unwind Analyzer: код, теория, архитектура и нормальный построчный разбор

Эта версия специально пересобрана без заглушек. В разборе нет формальных фраз вида «маленький шаг алгоритма». Каждая непустая строка кода объясняется по делу: что она делает технически, зачем она нужна логически и как связана с задачей анализа PE/EH/Unwind.

Источник кода: `C:\Users\Templar\Desktop\вкр\ВКР ПРОЕКТ ОКОНЧЕННЫЙ\ip_full`. Сам проект этим генератором не изменяется: PDF только читает файлы и собирает учебник.

Паспорт проекта

- Название: EH/Unwind Analyzer.
- Язык ядра: C17.
- Предмет: анализ Windows x64 PE-файлов, exception directory, RUNTIME_FUNCTION, UNWIND_INFO и handler metadata.
- Результат работы: console summary, CSV по функциям, JSON для автоматической проверки, HTML для ручного анализа.
- Главная инженерная сложность: безопасный разбор недоверенного бинарника, несколько адресных систем, эвристическая оценка и проверяемый отчет.

Архитектура проекта

Проект устроен слоями. Нижние слои не знают о verdict и score: они только безопасно читают байты и строят модель PE. Верхние слои не читают файл вручную: они используют готовые структуры PeImage, RuntimeFunction, UnwindSummary и DynamicReport. Так архитектура остается проверяемой: если finding появился, можно проследить, из какого слоя пришло доказательство.

```
input PE file
  -> util.c: bytes, bounds, strings, SHA-256
  -> pe.c: DOS header, NT header, section table, exception directory
  -> unwind.c: RUNTIME_FUNCTION array, UNWIND_INFO, handlers, anomalies
  -> heuristics.c: 27 findings, weights, thresholds, correlation, confidence, verdict
  -> dynamic_win.c: optional Windows runtime validation
  -> report.c: brief, CSV, JSON, HTML
  -> main.c: command line lifecycle and exit codes
```

Шесть аналитических слоев признаков

Слой	Что проверяет	Что дает проекту
1. Machine point	RIP, инструкция, регистры, тип доступа	понимание непосредственной причины сбоя или поведения
2. PE image	модуль, секция, ImageBase, RVA, SizeOfImage	правильную координатную систему
3. Function table	RUNTIME_FUNCTION Begin/End/UnwindInfoAddress	карту функций и границы кадров
4. Unwind info	version, flags, opcode, stack alloc, saved registers	возможность корректно выйти из кадра
5. Handler layer	EHANDLER/UHANDLER, HandlerRVA, handler data	логику обработки исключения
6. Stack walking result	цепочку кадров и обрывы unwind	итоговую историю выполнения

27 признаков проекта

Ниже не отдельная новая логика, а карта того, что потом реализует `heuristics.c`. Признаки выбраны так, чтобы покрыть: согласованность PE-формата, таблицу функций, декодирование unwind, размещение metadata, reuse/dominance и корреляционные сочетания.

Finding	Что означает простыми словами
`pe_section_raw_oob`	секция на диске выходит за размер файла; это ломает базовый контракт PE-загрузчика
`pe_section_virtual_oob`	секция в памяти выходит за SizeOfImage; это ошибка виртуальной координатной системы образа
`pe_section_raw_overlap`	raw-диапазоны секций пересекаются; один участок файла заявлен сразу несколькими секциями
`pe_section_virtual_overlap`	виртуальные диапазоны секций пересекаются; RVA попадает в несколько секций одновременно
`pe_section_misaligned`	section table нарушает выравнивание или порядок, поэтому адресация становится ненадежной
`pe_exception_directory_oob`	каталог исключений невозможно безопасно прочитать как таблицу RUNTIME_FUNCTION
`no_runtime_functions`	у исполняемого x64-образа нет .pdata; это бывает легально, но требует ручной проверки
`invalid_ranges`	у RUNTIME_FUNCTION EndAddress не больше BeginAddress, значит диапазон функции невозможен
`rf_oob`	Begin/End функции выходят за SizeOfImage

Finding	Что означает простыми словами
unwind_rva_oob	UnwindInfoAddress отсутствует или выходит за SizeOfImage
rf_unsorted	таблица функций не отсортирована по BeginAddress, хотя Windows ожидает упорядоченный поиск
rf_overlaps	диапазоны функций пересекаются после сортировки
duplicate_begin	несколько функций начинаются с одного RVA
unwind_parse_fail	UNWIND_INFO не удалось разобрать безопасно
unwind_anomalies	UNWIND_INFO разобран, но содержит странные opcode, version, alignment или chain-связи
unwind_reuse_strong	слишком много функций ссылаются на один UNWIND_INFO
handler_dominance	слишком много функций используют один handler
unwind_reuse_moderate	умеренное повторное использование UNWIND_INFO требует проверки контекста
unwind_in_exec_section	UNWIND_INFO лежит в исполняемой секции, что нетипично для xdata
unwind_unusual_section	UNWIND_INFO лежит в секции с нетипичным именем/назначением
unwind_writable_section	UNWIND_INFO лежит в writable-секции и может быть изменен во время выполнения
handler_nonexec_section	handler указывает в неисполняемую область
handler_writable_section	handler лежит в writable-секции

Finding	Что означает простыми словами
placement_exec_nonexec_handler_combo	сочетание исполняемой unwind-секции и неисполняемого handler усиливает подозрение
placement_integrity_combo	проблемы размещения совпали с проблемами целостности unwind/table
reuse_dominance_combo	повторное использование unwind совпало с доминированием handler
writable_metadata_handler_combo	и unwind metadata, и handler связаны с writable-областями

1. Каркас проекта и сборка

На этом шаге ты создаешь не алгоритм анализа, а инженерный каркас: как проект собирается, какие файлы входят в ядро, как включаются тесты, где задаются режимы debug/release.

Что сначала выучить в C и теории

- что такое единица сборки C, статическая библиотека и исполняемый файл
- как CMake связывает исходники, include-папку, тесты и флаги компилятора
- почему предупреждения компилятора в C лучше считать ошибками

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

CMakeLists.txt

Назначение файла: главный сценарий сборки: описывает библиотеку ядра, исполняемый файл, тесты, флаги компилятора и упаковку.

Код, который пишем на этом шаге

```
cmake_minimum_required(VERSION 3.23)

project(
    eh_unwind_analyzer
    VERSION 1.0.0
    DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"
    LANGUAGES C
)

include(CTest)

option(EHUA_WARNINGS_AS_ERRORS "Treat compiler warnings as errors" ON)
option(EHUA_ENABLE_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)
option(EHUA_ENABLE_COVERAGE "Enable GCC/Clang coverage instrumentation" OFF)
option(EHUA_BUILD_FUZZER "Build the Clang libFuzzer parser harness" OFF)

if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)
    message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")
endif()

if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)
    message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")
endif()

function(ehua_apply_warnings target)
    if(MSVC)
        target_compile_options(${target} PRIVATE /W4 /permissive- /utf-8)
        target_compile_definitions(${target} PRIVATE _CRT_SECURE_NO_WARNINGS=1)
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE /WX)
        endif()
    else()
        target_compile_options(
            ${target}
            PRIVATE
                -Wall
                -Wextra
                -Wpedantic
                -Wconversion
                -Wshadow
                -Wformat=2
        )
        if(CMAKE_C_COMPILER_ID STREQUAL "GNU")
            target_compile_options(${target} PRIVATE -finput-charset=UTF-8 -fexec-charset=UTF-8)
        endif()
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE -Werror)
        endif()
    endif()
endfunction()

function(ehua_apply_instrumentation target)
    if(EHUA_ENABLE_SANITIZERS)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_SANITIZERS is currently supported with GCC or Clang")
        endif()
        target_compile_options(${target} PRIVATE -fsanitize=address,undefined -fno-omit-frame-pointer)
        target_link_options(${target} PRIVATE -fsanitize=address,undefined)
    elseif(EHUA_ENABLE_COVERAGE)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_COVERAGE is currently supported with GCC or Clang")
        endif()
        target_compile_options(${target} PRIVATE --coverage -O0 -g)
        target_link_options(${target} PRIVATE --coverage)
    endif()
endfunction()

set(EHUA_CORE_SOURCES
    src/app_info.c
    src/dynamic_win.c
    src/heuristics.c
    src/pe.c
    src/report.c
    src/unwind.c
    src/util.c
```

```
)

add_library(ehua_core STATIC ${EHUA_CORE_SOURCES})
target_include_directories(
    ehua_core
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
)
target_compile_features(ehua_core PUBLIC c_std_17)

if(WIN32)
    target_compile_definitions(ehua_core PUBLIC WIN32_LEAN_AND_MEAN=1)
endif()

if(MINGW)
    target_compile_definitions(ehua_core PUBLIC __USE_MINGW_ANSI_STDIO=1)
endif()
ehua_apply_warnings(ehua_core)

add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
ehua_apply_warnings(eh_unwind_analyzer)
ehua_apply_instrumentation(ehua_core)
ehua_apply_instrumentation(eh_unwind_analyzer)

if(EHUA_BUILD_FUZZER)
    if(NOT CMAKE_C_COMPILER_ID MATCHES "Clang")
        message(FATAL_ERROR "EHUA_BUILD_FUZZER requires Clang")
    endif()
    include(CheckCSourceCompiles)
    set(EHUA_SAVED_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS}")
    set(EHUA_SAVED_REQUIRED_LINK_OPTIONS "${CMAKE_REQUIRED_LINK_OPTIONS}")
    set(CMAKE_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS} -fsanitize=fuzzer,address,undefined")
    set(CMAKE_REQUIRED_LINK_OPTIONS -fsanitize=fuzzer,address,undefined)
    check_c_source_compiles(
        "#include <stddef.h>
         #include <stdint.h>
         int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
             return (data != 0 && size != 0) ? 0 : 0;
         }"
        EHUA_LIBFUZZER_AVAILABLE
    )
    set(CMAKE_REQUIRED_FLAGS "${EHUA_SAVED_REQUIRED_FLAGS}")
    set(CMAKE_REQUIRED_LINK_OPTIONS "${EHUA_SAVED_REQUIRED_LINK_OPTIONS}")
    if(NOT EHUA_LIBFUZZER_AVAILABLE)
        message(FATAL_ERROR
            "Clang is present, but its libFuzzer runtime is unavailable. "
            "Install a full LLVM/Clang toolchain or leave EHUA_BUILD_FUZZER=OFF.")
    endif()
    add_executable(
        ehua_parser_fuzzer
        tests/fuzz/fuzz_parser.c
        src/heuristics.c
        src/pe.c
        src/unwind.c
        src/util.c
    )
    target_include_directories(ehua_parser_fuzzer PRIVATE "${CMAKE_CURRENT_SOURCE_DIR}/include")
    target_compile_features(ehua_parser_fuzzer PRIVATE c_std_17)
    ehua_apply_warnings(ehua_parser_fuzzer)
    target_compile_options(
        ehua_parser_fuzzer
        PRIVATE
            -fsanitize=fuzzer,address,undefined
            -fno-omit-frame-pointer
    )
    target_link_options(ehua_parser_fuzzer PRIVATE -fsanitize=fuzzer,address,undefined)
endif()

if(BUILD_TESTING)
    function(ehua_add_unit_test target source)
        add_executable(${target} ${source})
        target_link_libraries(${target} PRIVATE ehua_core)
        if(MSVC)
            target_compile_options(${target} PRIVATE /UNDEBUG)
        else()
            target_compile_options(${target} PRIVATE -UNDEBUG)
        endif()
        ehua_apply_warnings(${target})
        ehua_apply_instrumentation(${target})
        set(unit_workdir "${CMAKE_CURRENT_BINARY_DIR}/tests/unit/${target}")
        file(MAKE_DIRECTORY "${unit_workdir}")
        add_test(NAME ${target} COMMAND ${target})
        set_tests_properties(
            ${target}
            PROPERTIES
                WORKING_DIRECTORY "${unit_workdir}"
                TIMEOUT 30
        )
    endfunction()

    ehua_add_unit_test(unit_unwind tests/unit/unit_unwind.c)
    ehua_add_unit_test(unit_heuristics tests/unit/unit_heuristics.c)
    ehua_add_unit_test(unit_heuristics_mitigations tests/unit/unit_heuristics_mitigations.c)
    ehua_add_unit_test(unit_heuristics_context tests/unit/unit_heuristics_context.c)
    ehua_add_unit_test(unit_heuristics_combos tests/unit/unit_heuristics_combos.c)
    ehua_add_unit_test(unit_core_edges tests/unit/unit_core_edges.c)
    ehua_add_unit_test(unit_report_json tests/unit/unit_report_json.c)
    ehua_add_unit_test(unit_report_html tests/unit/unit_report_html.c)
    ehua_add_unit_test(unit_matrix_report tests/unit/unit_matrix_report.c)

    find_package(Python3 COMPONENTS Interpreter)
    if(Python3_Interpreter_FOUND)
        add_test(
            NAME regression_samples
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/run_regression_samples.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(regression_samples PROPERTIES TIMEOUT 120)
        add_test(
            NAME dynamic_validator
            COMMAND
```

```
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_dynamic_validator.py"
        )
        set_tests_properties(dynamic_validator PROPERTIES TIMEOUT 30)
        add_test(
            NAME report_semantics
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_report_semantics.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(report_semantics PROPERTIES TIMEOUT 30)
        add_test(
            NAME release_metadata
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_metadata.py"
        )
        set_tests_properties(release_metadata PROPERTIES TIMEOUT 30)
        add_test(
            NAME release_hygiene
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_hygiene.py"
        )
        set_tests_properties(release_hygiene PROPERTIES TIMEOUT 30)
        add_test(
            NAME cli_contract
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_cli_contract.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(cli_contract PROPERTIES TIMEOUT 60)
        add_test(
            NAME mutation_fuzz
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/run_mutation_fuzz.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
                --cases
                120
        )
        set_tests_properties(
            mutation_fuzz
            PROPERTIES
                TIMEOUT 180
                LABELS "fuzz"
        )
        set(EHUA_PYTHON_TEST_ENV
            "PYTHONDONTWRITEBYTECODE=1"
            "PYTHONPATH=${CMAKE_BINARY_DIR}/pydeps"
        )
        set_tests_properties(
            regression_samples
            dynamic_validator
            report_semantics
            release_metadata
            release_hygiene
            cli_contract
            mutation_fuzz
            PROPERTIES
                ENVIRONMENT "${EHUA_PYTHON_TEST_ENV}"
        )
    endif()
endif()

install(TARGETS eh_unwind_analyzer RUNTIME DESTINATION bin)
install(
    DIRECTORY docs/
    DESTINATION share/eh_unwind_analyzer/docs
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
    PATTERN "__pycache__" EXCLUDE
    PATTERN "*.pyc" EXCLUDE
)
install(
    DIRECTORY ghidra/
    DESTINATION share/eh_unwind_analyzer/ghidra
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
    PATTERN "__pycache__" EXCLUDE
    PATTERN "*.pyc" EXCLUDE
)
install(
    DIRECTORY examples/
    DESTINATION share/eh_unwind_analyzer/examples
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
)
install(
    FILES
        README.md
        LICENSE
        CHANGELOG.md
        SECURITY.md
        CONTRIBUTING.md
    DESTINATION share/eh_unwind_analyzer
)

set(CPACK_PACKAGE_NAME "EH-Unwind-Analyzer")
set(CPACK_PACKAGE_VENDOR "EH/Unwind Analyzer project")
set(CPACK_PACKAGE_VERSION "1.0.0")
set(CPACK_PACKAGE_FILE_NAME "eh-unwind-analyzer-${CPACK_PACKAGE_VERSION}-${CMAKE_SYSTEM_NAME}-${CMAKE_SYSTEM_PROCESSOR}")
set(CPACK_GENERATOR "ZIP")
include(CPack)
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`cmake_minimum_required(VERSION 3.23)`	Технически задает минимальную версию CMake. Логически проект фиксирует набор возможностей сборки, на который можно рассчитывать.

Стр.	Код	Нормальное объяснение
3	`project(`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
4	`  eh_unwind_analyzer`	Технически это команда или аргумент CMake `eh_unwind_analyzer`. Логически она участвует в описании сборки проекта.
5	`  VERSION 1.0.0`	Технически указывает версию проекта. Логически эта версия попадает в release/packaging и помогает защите говорить о конкретном выпуске.
6	`DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"`	Технически задает описание проекта. Логически оно фиксирует назначение: статический и динамический анализ EH/Unwind metadata Windows x64.
7	`  LANGUAGES C`	Технически ограничивает проект языком C. Логически это подтверждает, что ядро ВКР написано на C, без C++ ABI и runtime.
8	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
10	`include(CTest)`	Технически подключает стандартный модуль CMake. Логически это включает поддержку тестов или проверок сборки.
12	`option(EHUA_WARNINGS_AS_ERRORS "Treat compiler warnings as errors" ON)`	Технически объявляет настраиваемый флаг сборки. Логически пользователь может включать warnings-as-errors, sanitizers, coverage или fuzzing без изменения кода.
13	`option(EHUA_ENABLE_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)`	Технически объявляет настраиваемый флаг сборки. Логически пользователь может включать warnings-as-errors, sanitizers, coverage или fuzzing без изменения кода.
14	`option(EHUA_ENABLE_COVERAGE "Enable GCC/Clang coverage instrumentation" OFF)`	Технически объявляет настраиваемый флаг сборки. Логически пользователь может включать warnings-as-errors, sanitizers, coverage или fuzzing без изменения кода.
15	`option(EHUA_BUILD_FUZZER "Build the Clang libFuzzer parser harness" OFF)`	Технически объявляет настраиваемый флаг сборки. Логически пользователь может включать warnings-as-errors, sanitizers, coverage или fuzzing без изменения кода.
17	`if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)`	Технически начинает условную ветку `if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
18	`message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")`	Технически это команда или аргумент CMake `message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")`. Логически она участвует в описании сборки проекта.
19	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
21	`if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)`	Технически начинает условную ветку `if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
22	`message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")`	Технически это команда или аргумент CMake `message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")`. Логически она участвует в описании сборки проекта.
23	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
25	`function(ehua_apply_warnings target)`	Технически объявляет CMake-функцию. Логически повторяющиеся настройки предупреждений/тестов оформлены в одно место.
26	`  if(MSVC)`	Технически начинает условную ветку `if(MSVC)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
27	`target_compile_options(${target} PRIVATE /W4 /permissive- /utf-8)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
28	`target_compile_definitions(${target} PRIVATE _CRT_SECURE_NO_WARNINGS=1)`	Технически добавляет макросы компиляции. Логически код получает платформенные настройки вроде WIN32_LEAN_AND_MEAN или безопасного printf в MinGW.
29	`  if(EHUA_WARNINGS_AS_ERRORS)`	Технически начинает условную ветку `if(EHUA_WARNINGS_AS_ERRORS)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
30	`  target_compile_options(${target} PRIVATE /WX)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
31	`  endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
32	`  else()`	Технически задает альтернативную ветку условия CMake. Логически это отдельный путь для другого компилятора или режима сборки.
33	`  target_compile_options(`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
34	`  ${target}`	Технически это команда или аргумент CMake `${target}`. Логически она участвует в описании сборки проекта.
35	`  PRIVATE`	Технически это команда или аргумент CMake `PRIVATE`. Логически она участвует в описании сборки проекта.
36	`    -Wall`	Технически это команда или аргумент CMake `-Wall`. Логически она участвует в описании сборки проекта.
37	`    -Wextra`	Технически это команда или аргумент CMake `-Wextra`. Логически она участвует в описании сборки проекта.
38	`    -Wpedantic`	Технически это команда или аргумент CMake `-Wpedantic`. Логически она участвует в описании сборки проекта.
39	`    -Wconversion`	Технически это команда или аргумент CMake `-Wconversion`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
40	`-Wshadow`	Технически это команда или аргумент CMake `-Wshadow`. Логически она участвует в описании сборки проекта.
41	`-Wformat=2`	Технически это команда или аргумент CMake `-Wformat=2`. Логически она участвует в описании сборки проекта.
42	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
43	`if(CMAKE_C_COMPILER_ID STREQUAL "GNU")`	Технически начинает условную ветку `if(CMAKE_C_COMPILER_ID STREQUAL "GNU")`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
44	`target_compile_options(${target} PRIVATE -finput-charset=UTF-8 -fexec-charset=UTF-8)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
45	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
46	`if(EHUA_WARNINGS_AS_ERRORS)`	Технически начинает условную ветку `if(EHUA_WARNINGS_AS_ERRORS)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
47	`target_compile_options(${target} PRIVATE -Werror)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
48	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
49	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
50	`endfunction()`	Технически завершает CMake-функцию. Логически после этой строки функцию можно вызывать для разных target.
52	`function(ehua_apply_instrumentation target)`	Технически объявляет CMake-функцию. Логически повторяющиеся настройки предупреждений/тестов оформлены в одно место.
53	`if(EHUA_ENABLE_SANITIZERS)`	Технически начинает условную ветку `if(EHUA_ENABLE_SANITIZERS)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
54	`if(MSVC)`	Технически начинает условную ветку `if(MSVC)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
55	`message(FATAL_ERROR "EHUA_ENABLE_SANITIZERS is currently supported with GCC or Clang")`	Технически это команда или аргумент CMake `message(FATAL_ERROR "EHUA_ENABLE_SANITIZERS is currently supported with GCC or Clang")`. Логически она участвует в описании сборки проекта.
56	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
57	`target_compile_options(${target} PRIVATE -fsanitize=address,undefined -fno-omit-frame-pointer)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
58	`target_link_options(${target} PRIVATE -fsanitize=address,undefined)`	Технически добавляет флаги линковщика. Логически sanitizers/coverage должны работать и на этапе связывания.
59	`elseif(EHUA_ENABLE_COVERAGE)`	Технически задает альтернативную ветку условия CMake. Логически это отдельный путь для другого компилятора или режима сборки.
60	`if(MSVC)`	Технически начинает условную ветку `if(MSVC)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
61	`message(FATAL_ERROR "EHUA_ENABLE_COVERAGE is currently supported with GCC or Clang")`	Технически это команда или аргумент CMake `message(FATAL_ERROR "EHUA_ENABLE_COVERAGE is currently supported with GCC or Clang")`. Логически она участвует в описании сборки проекта.
62	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
63	`target_compile_options(${target} PRIVATE --coverage -O0 -g)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
64	`target_link_options(${target} PRIVATE --coverage)`	Технически добавляет флаги линковщика. Логически sanitizers/coverage должны работать и на этапе связывания.
65	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
66	`endfunction()`	Технически завершает CMake-функцию. Логически после этой строки функцию можно вызывать для разных target.
68	`set(EHUA_CORE_SOURCES`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
69	`src/app_info.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
70	`src/dynamic_win.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
71	`src/heuristics.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
72	`src/pe.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
73	`src/report.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.

Стр.	Код	Нормальное объяснение
74	src/unwind.c	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.

Стр.	Код	Нормальное объяснение
75	src/util.c	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
76	)	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
78	add_library(ehua_core STATIC ${EHUA_CORE_SOURCES})	Технически создает статическую библиотеку ядра. Логически основная логика отделена от CLI и может тестироваться отдельно.
79	target_include_directories(	Технически добавляет include-папку. Логически исходники смогут подключать заголовки проекта через #include.
80	ehua_core	Технически это команда или аргумент CMake `ehua_core`. Логически она участвует в описании сборки проекта.
81	PUBLIC	Технически это команда или аргумент CMake `PUBLIC`. Логически она участвует в описании сборки проекта.
82	$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>	Технически это команда или аргумент CMake `$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>`. Логически она участвует в описании сборки проекта.
83	$<INSTALL_INTERFACE:include>	Технически это команда или аргумент CMake `$<INSTALL_INTERFACE:include>`. Логически она участвует в описании сборки проекта.
84	)	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
85	target_compile_features(ehua_core PUBLIC c_std_17)	Технически требует стандарт C17. Логически проект фиксирует современный, но переносимый уровень языка C.
87	if(WIN32)	Технически начинает условную ветку `if(WIN32)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
88	target_compile_definitions(ehua_core PUBLIC WIN32_LEAN_AND_MEAN=1)	Технически добавляет макросы компиляции. Логически код получает платформенные настройки вроде WIN32_LEAN_AND_MEAN или безопасного printf в MinGW.
89	endif()	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
91	if(MINGW)	Технически начинает условную ветку `if(MINGW)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
92	target_compile_definitions(ehua_core PUBLIC __USE_MINGW_ANSI_STDIO=1)	Технически добавляет макросы компиляции. Логически код получает платформенные настройки вроде WIN32_LEAN_AND_MEAN или безопасного printf в MinGW.
93	endif()	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
94	ehua_apply_warnings(ehua_core)	Технически это команда или аргумент CMake `ehua_apply_warnings(ehua_core)`. Логически она участвует в описании сборки проекта.
96	add_executable(eh_unwind_analyzer src/main.c)	Технически создает исполняемый файл. Логически это либо основной analyzer, либо отдельный unit-test/fuzzer.
97	target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)	Технически связывает executable с библиотекой. Логически main.c получает доступ к PE/unwind/heuristics/report ядру.
98	ehua_apply_warnings(eh_unwind_analyzer)	Технически это команда или аргумент CMake `ehua_apply_warnings(eh_unwind_analyzer)`. Логически она участвует в описании сборки проекта.
99	ehua_apply_instrumentation(ehua_core)	Технически это команда или аргумент CMake `ehua_apply_instrumentation(ehua_core)`. Логически она участвует в описании сборки проекта.
100	ehua_apply_instrumentation(eh_unwind_analyzer)	Технически это команда или аргумент CMake `ehua_apply_instrumentation(eh_unwind_analyzer)`. Логически она участвует в описании сборки проекта.
102	if(EHUA_BUILD_FUZZER)	Технически начинает условную ветку `if(EHUA_BUILD_FUZZER)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
103	if(NOT CMAKE_C_COMPILER_ID MATCHES "Clang")	Технически начинает условную ветку `if(NOT CMAKE_C_COMPILER_ID MATCHES "Clang")`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
104	message(FATAL_ERROR "EHUA_BUILD_FUZZER requires Clang")	Технически это команда или аргумент CMake `message(FATAL_ERROR "EHUA_BUILD_FUZZER requires Clang")`. Логически она участвует в описании сборки проекта.
105	endif()	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
106	include(CheckCSourceCompiles)	Технически подключает стандартный модуль CMake. Логически это включает поддержку тестов или проверок сборки.
107	set(EHUA_SAVED_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS}")	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
108	set(EHUA_SAVED_REQUIRED_LINK_OPTIONS " ${CMAKE_REQUIRED_LINK_OPTIONS}")	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
109	set(CMAKE_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS} -fsanitize=fuzzer,address,undefined")	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
110	set(CMAKE_REQUIRED_LINK_OPTIONS -fsanitize=fuzzer,address,undefined)	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.

Стр.	Код	Нормальное объяснение
111	`check_c_source_compiles(`	Технически это команда или аргумент CMake `check_c_source_compiles(`. Логически она участвует в описании сборки проекта.
112	`"#include <stddef.h>`	Технически это команда или аргумент CMake `"#include <stddef.h>`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
113	`#include <stdint.h>`	Технически это команда или аргумент CMake `#include <stdint.h>`. Логически она участвует в описании сборки проекта.
114	`int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {`	Технически это команда или аргумент CMake `int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {`. Логически она участвует в описании сборки проекта.
115	`return (data != 0 && size != 0) ? 0 : 0;`	Технически это команда или аргумент CMake `return (data != 0 && size != 0) ? 0 : 0;`. Логически она участвует в описании сборки проекта.
116	`}"`	Технически это команда или аргумент CMake `}"`. Логически она участвует в описании сборки проекта.
117	`EHUA_LIBFUZZER_AVAILABLE`	Технически это команда или аргумент CMake `EHUA_LIBFUZZER_AVAILABLE`. Логически она участвует в описании сборки проекта.
118	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
119	`set(CMAKE_REQUIRED_FLAGS "${EHUA_SAVED_REQUIRED_FLAGS}")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
120	`set(CMAKE_REQUIRED_LINK_OPTIONS "${EHUA_SAVED_REQUIRED_LINK_OPTIONS}")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
121	`if(NOT EHUA_LIBFUZZER_AVAILABLE)`	Технически начинает условную ветку `if(NOT EHUA_LIBFUZZER_AVAILABLE)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
122	`message(FATAL_ERROR`	Технически это команда или аргумент CMake `message(FATAL_ERROR`. Логически она участвует в описании сборки проекта.
123	`"Clang is present, but its libFuzzer runtime is unavailable. "`	Технически это команда или аргумент CMake `"Clang is present, but its libFuzzer runtime is unavailable. "`. Логически она участвует в описании сборки проекта.
124	`"Install a full LLVM/Clang toolchain or leave EHUA_BUILD_FUZZER=OFF.")`	Технически это команда или аргумент CMake `"Install a full LLVM/Clang toolchain or leave EHUA_BUILD_FUZZER=OFF.")`. Логически она участвует в описании сборки проекта.
125	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
126	`add_executable(`	Технически создает исполняемый файл. Логически это либо основной analyzer, либо отдельный unit-test/fuzzer.
127	`ehua_parser_fuzzer`	Технически это команда или аргумент CMake `ehua_parser_fuzzer`. Логически она участвует в описании сборки проекта.
128	`tests/fuzz/fuzz_parser.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
129	`src/heuristics.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
130	`src/pe.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
131	`src/unwind.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
132	`src/util.c`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
133	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
134	`target_include_directories(ehua_parser_fuzzer PRIVATE "${CMAKE_CURRENT_SOURCE_DIR}/include")`	Технически добавляет include-папку. Логически исходники смогут подключать заголовки проекта через #include.
135	`target_compile_features(ehua_parser_fuzzer PRIVATE c_std_17)`	Технически требует стандарт C17. Логически проект фиксирует современный, но переносимый уровень языка C.
136	`ehua_apply_warnings(ehua_parser_fuzzer)`	Технически это команда или аргумент CMake `ehua_apply_warnings(ehua_parser_fuzzer)`. Логически она участвует в описании сборки проекта.
137	`target_compile_options(`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
138	`ehua_parser_fuzzer`	Технически это команда или аргумент CMake `ehua_parser_fuzzer`. Логически она участвует в описании сборки проекта.
139	`PRIVATE`	Технически это команда или аргумент CMake `PRIVATE`. Логически она участвует в описании сборки проекта.
140	`-fsanitize=fuzzer,address,undefined`	Технически это команда или аргумент CMake `-fsanitize=fuzzer,address,undefined`. Логически она участвует в описании сборки проекта.
141	`-fno-omit-frame-pointer`	Технически это команда или аргумент CMake `-fno-omit-frame-pointer`. Логически она участвует в описании сборки проекта.
142	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.

Стр.	Код	Нормальное объяснение
143	`target_link_options(ehua_parser_fuzzer PRIVATE -fsanitize=fuzzer,address,undefined)`	Технически добавляет флаги линковщика. Логически sanitizers/coverage должны работать и на этапе связывания.
144	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
146	`if(BUILD_TESTING)`	Технически начинает условную ветку `if(BUILD_TESTING)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.

Стр.	Код	Нормальное объяснение
147	`function(ehua_add_unit_test target source)`	Технически объявляет CMake-функцию. Логически повторяющиеся настройки предупреждений/тестов оформлены в одно место.
148	`add_executable(${target} ${source})`	Технически создает исполняемый файл. Логически это либо основной analyzer, либо отдельный unit-test/fuzzer.
149	`target_link_libraries(${target} PRIVATE ehua_core)`	Технически связывает executable с библиотекой. Логически main.c получает доступ к PE/unwind/heuristics/report ядру.
150	`if(MSVC)`	Технически начинает условную ветку `if(MSVC)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
151	`target_compile_options(${target} PRIVATE /UNDEBUG)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
152	`else()`	Технически задает альтернативную ветку условия CMake. Логически это отдельный путь для другого компилятора или режима сборки.
153	`target_compile_options(${target} PRIVATE -UNDEBUG)`	Технически добавляет флаги компилятора к target. Логически это повышает строгость C-кода и помогает ловить ошибки до запуска.
154	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
155	`ehua_apply_warnings(${target})`	Технически это команда или аргумент CMake `ehua_apply_warnings(${target})`. Логически она участвует в описании сборки проекта.
156	`ehua_apply_instrumentation(${target})`	Технически это команда или аргумент CMake `ehua_apply_instrumentation(${target})`. Логически она участвует в описании сборки проекта.
157	`set(unit_workdir "${CMAKE_CURRENT_BINARY_DIR}/tests/unit/${target}")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
158	`file(MAKE_DIRECTORY "${unit_workdir}")`	Технически это команда или аргумент CMake `file(MAKE_DIRECTORY "${unit_workdir}")`. Логически она участвует в описании сборки проекта.
159	`add_test(NAME ${target} COMMAND ${target})`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
160	`set_tests_properties(`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
161	`${target}`	Технически это команда или аргумент CMake `${target}`. Логически она участвует в описании сборки проекта.
162	`PROPERTIES`	Технически это команда или аргумент CMake `PROPERTIES`. Логически она участвует в описании сборки проекта.
163	`WORKING_DIRECTORY "${unit_workdir}"`	Технически это команда или аргумент CMake `WORKING_DIRECTORY "${unit_workdir}"`. Логически она участвует в описании сборки проекта.
164	`TIMEOUT 30`	Технически это команда или аргумент CMake `TIMEOUT 30`. Логически она участвует в описании сборки проекта.
165	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
166	`endfunction()`	Технически завершает CMake-функцию. Логически после этой строки функцию можно вызывать для разных target.
168	`ehua_add_unit_test(unit_unwind tests/unit/unit_unwind.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_unwind tests/unit/unit_unwind.c)`. Логически она участвует в описании сборки проекта.
169	`ehua_add_unit_test(unit_heuristics tests/unit/unit_heuristics.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_heuristics tests/unit/unit_heuristics.c)`. Логически она участвует в описании сборки проекта.
170	`ehua_add_unit_test(unit_heuristics_mitigations tests/unit/unit_heuristics_mitigations.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_heuristics_mitigations tests/unit/unit_heuristics_mitigations.c)`. Логически она участвует в описании сборки проекта.
171	`ehua_add_unit_test(unit_heuristics_context tests/unit/unit_heuristics_context.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_heuristics_context tests/unit/unit_heuristics_context.c)`. Логически она участвует в описании сборки проекта.
172	`ehua_add_unit_test(unit_heuristics_combos tests/unit/unit_heuristics_combos.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_heuristics_combos tests/unit/unit_heuristics_combos.c)`. Логически она участвует в описании сборки проекта.
173	`ehua_add_unit_test(unit_core_edges tests/unit/unit_core_edges.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_core_edges tests/unit/unit_core_edges.c)`. Логически она участвует в описании сборки проекта.
174	`ehua_add_unit_test(unit_report_json tests/unit/unit_report_json.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_report_json tests/unit/unit_report_json.c)`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
175	`ehua_add_unit_test(unit_report_html tests/unit/unit_report_html.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_report_html tests/unit/unit_report_html.c)`. Логически она участвует в описании сборки проекта.
176	`ehua_add_unit_test(unit_matrix_report tests/unit/unit_matrix_report.c)`	Технически это команда или аргумент CMake `ehua_add_unit_test(unit_matrix_report tests/unit/unit_matrix_report.c)`. Логически она участвует в описании сборки проекта.
178	`find_package(Python3 COMPONENTS Interpreter)`	Технически это команда или аргумент CMake `find_package(Python3 COMPONENTS Interpreter)`. Логически она участвует в описании сборки проекта.
179	`if(Python3_Interpreter_FOUND)`	Технически начинает условную ветку `if(Python3_Interpreter_FOUND)`. Логически CMake выбирает поведение под платформу, компилятор или режим проверки.
180	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
181	`NAME regression_samples`	Технически это команда или аргумент CMake `NAME regression_samples`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
182	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
183	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
184	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/run_regression_samples.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/run_regression_samples.py"`. Логически она участвует в описании сборки проекта.
185	`"$<TARGET_FILE:eh_unwind_analyzer>"`	Технически это команда или аргумент CMake `"$<TARGET_FILE:eh_unwind_analyzer>"`. Логически она участвует в описании сборки проекта.
186	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
187	`set_tests_properties(regression_samples PROPERTIES TIMEOUT 120)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
188	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
189	`NAME dynamic_validator`	Технически это команда или аргумент CMake `NAME dynamic_validator`. Логически она участвует в описании сборки проекта.
190	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
191	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
192	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_dynamic_validator.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_dynamic_validator.py"`. Логически она участвует в описании сборки проекта.
193	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
194	`set_tests_properties(dynamic_validator PROPERTIES TIMEOUT 30)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
195	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
196	`NAME report_semantics`	Технически это команда или аргумент CMake `NAME report_semantics`. Логически она участвует в описании сборки проекта.
197	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
198	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
199	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_report_semantics.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_report_semantics.py"`. Логически она участвует в описании сборки проекта.
200	`"$<TARGET_FILE:eh_unwind_analyzer>"`	Технически это команда или аргумент CMake `"$<TARGET_FILE:eh_unwind_analyzer>"`. Логически она участвует в описании сборки проекта.
201	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
202	`set_tests_properties(report_semantics PROPERTIES TIMEOUT 30)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
203	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
204	`NAME release_metadata`	Технически это команда или аргумент CMake `NAME release_metadata`. Логически она участвует в описании сборки проекта.
205	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
206	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
207	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_metadata.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_metadata.py"`. Логически она участвует в описании сборки проекта.
208	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
209	`set_tests_properties(release_metadata PROPERTIES TIMEOUT 30)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
210	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
211	`NAME release_hygiene`	Технически это команда или аргумент CMake `NAME release_hygiene`. Логически она участвует в описании сборки проекта.
212	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
213	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
214	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_hygiene.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_hygiene.py"`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
215	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
216	`set_tests_properties(release_hygiene PROPERTIES TIMEOUT 30)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
217	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
218	`NAME cli_contract`	Технически это команда или аргумент CMake `NAME cli_contract`. Логически она участвует в описании сборки проекта.
219	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
220	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
221	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_cli_contract.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/test_cli_contract.py"`. Логически она участвует в описании сборки проекта.
222	`"$<TARGET_FILE:eh_unwind_analyzer>"`	Технически это команда или аргумент CMake `"$<TARGET_FILE:eh_unwind_analyzer>"`. Логически она участвует в описании сборки проекта.
223	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
224	`set_tests_properties(cli_contract PROPERTIES TIMEOUT 60)`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
225	`add_test(`	Технически регистрирует тест в CTest. Логически проверка становится частью стандартного прогона `ctest`.
226	`NAME mutation_fuzz`	Технически это команда или аргумент CMake `NAME mutation_fuzz`. Логически она участвует в описании сборки проекта.
227	`COMMAND`	Технически это команда или аргумент CMake `COMMAND`. Логически она участвует в описании сборки проекта.
228	`"${Python3_EXECUTABLE}"`	Технически это команда или аргумент CMake `"${Python3_EXECUTABLE}"`. Логически она участвует в описании сборки проекта.
229	`"${CMAKE_CURRENT_SOURCE_DIR}/tests/run_mutation_fuzz.py"`	Технически это команда или аргумент CMake `"${CMAKE_CURRENT_SOURCE_DIR}/tests/run_mutation_fuzz.py"`. Логически она участвует в описании сборки проекта.
230	`"$<TARGET_FILE:eh_unwind_analyzer>"`	Технически это команда или аргумент CMake `"$<TARGET_FILE:eh_unwind_analyzer>"`. Логически она участвует в описании сборки проекта.
231	`--cases`	Технически это команда или аргумент CMake `--cases`. Логически она участвует в описании сборки проекта.
232	`120`	Технически это команда или аргумент CMake `120`. Логически она участвует в описании сборки проекта.
233	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
234	`set_tests_properties(`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
235	`mutation_fuzz`	Технически это команда или аргумент CMake `mutation_fuzz`. Логически она участвует в описании сборки проекта.
236	`PROPERTIES`	Технически это команда или аргумент CMake `PROPERTIES`. Логически она участвует в описании сборки проекта.
237	`TIMEOUT 180`	Технически это команда или аргумент CMake `TIMEOUT 180`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
238	`  LABELS "fuzz"`	Технически это команда или аргумент CMake `LABELS "fuzz"`. Логически она участвует в описании сборки проекта.
239	`  )`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
240	`  set(EHUA_PYTHON_TEST_ENV`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
241	`  "PYTHONDONTWRITEBYTECODE=1"`	Технически это команда или аргумент CMake `"PYTHONDONTWRITEBYTECODE=1"`. Логически она участвует в описании сборки проекта.
242	`  "PYTHONPATH=${CMAKE_BINARY_DIR}/pydeps"`	Технически это команда или аргумент CMake `"PYTHONPATH=${CMAKE_BINARY_DIR}/pydeps"`. Логически она участвует в описании сборки проекта.
243	`  )`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
244	`  set_tests_properties(`	Технически задает параметры теста. Логически фиксируются рабочая папка, timeout и окружение.
245	`  regression_samples`	Технически это команда или аргумент CMake `regression_samples`. Логически она участвует в описании сборки проекта.
246	`  dynamic_validator`	Технически это команда или аргумент CMake `dynamic_validator`. Логически она участвует в описании сборки проекта.
247	`  report_semantics`	Технически это команда или аргумент CMake `report_semantics`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
248	`  release_metadata`	Технически это команда или аргумент CMake `release_metadata`. Логически она участвует в описании сборки проекта.
249	`  release_hygiene`	Технически это команда или аргумент CMake `release_hygiene`. Логически она участвует в описании сборки проекта.
250	`  cli_contract`	Технически это команда или аргумент CMake `cli_contract`. Логически она участвует в описании сборки проекта.
251	`  mutation_fuzz`	Технически это команда или аргумент CMake `mutation_fuzz`. Логически она участвует в описании сборки проекта.
252	`  PROPERTIES`	Технически это команда или аргумент CMake `PROPERTIES`. Логически она участвует в описании сборки проекта.
253	`  ENVIRONMENT "${EHUA_PYTHON_TEST_ENV}"`	Технически это команда или аргумент CMake `ENVIRONMENT "${EHUA_PYTHON_TEST_ENV}"`. Логически она участвует в описании сборки проекта.
254	`  )`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
255	`  endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
256	`endif()`	Технически закрывает условие CMake. Логически следующие команды снова выполняются независимо от этой проверки.
258	`install(TARGETS eh_unwind_analyzer RUNTIME DESTINATION bin)`	Технически описывает установку файлов. Логически release-пакет получает exe, docs, examples и служебные материалы.
259	`install(`	Технически описывает установку файлов. Логически release-пакет получает exe, docs, examples и служебные материалы.
260	`  DIRECTORY docs/`	Технически это команда или аргумент CMake `DIRECTORY docs/`. Логически она участвует в описании сборки проекта.
261	`  DESTINATION share/eh_unwind_analyzer/docs`	Технически это команда или аргумент CMake `DESTINATION share/eh_unwind_analyzer/docs`. Логически она участвует в описании сборки проекта.
262	`  PATTERN ".DS_Store" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN ".DS_Store" EXCLUDE`. Логически она участвует в описании сборки проекта.
263	`  PATTERN "._*" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "._*" EXCLUDE`. Логически она участвует в описании сборки проекта.
264	`  PATTERN "__pycache__" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "__pycache__" EXCLUDE`. Логически она участвует в описании сборки проекта.
265	`  PATTERN "*.pyc" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "*.pyc" EXCLUDE`. Логически она участвует в описании сборки проекта.
266	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
267	`install(`	Технически описывает установку файлов. Логически release-пакет получает exe, docs, examples и служебные материалы.
268	`  DIRECTORY ghidra/`	Технически это команда или аргумент CMake `DIRECTORY ghidra/`. Логически она участвует в описании сборки проекта.
269	`  DESTINATION share/eh_unwind_analyzer/ghidra`	Технически это команда или аргумент CMake `DESTINATION share/eh_unwind_analyzer/ghidra`. Логически она участвует в описании сборки проекта.
270	`  PATTERN ".DS_Store" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN ".DS_Store" EXCLUDE`. Логически она участвует в описании сборки проекта.

Стр.	Код	Нормальное объяснение
271	`  PATTERN "._*" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "._*" EXCLUDE`. Логически она участвует в описании сборки проекта.
272	`  PATTERN "__pycache__" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "__pycache__" EXCLUDE`. Логически она участвует в описании сборки проекта.
273	`  PATTERN "*.pyc" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "*.pyc" EXCLUDE`. Логически она участвует в описании сборки проекта.
274	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
275	`install(`	Технически описывает установку файлов. Логически release-пакет получает exe, docs, examples и служебные материалы.
276	`  DIRECTORY examples/`	Технически это команда или аргумент CMake `DIRECTORY examples/`. Логически она участвует в описании сборки проекта.
277	`  DESTINATION share/eh_unwind_analyzer/examples`	Технически это команда или аргумент CMake `DESTINATION share/eh_unwind_analyzer/examples`. Логически она участвует в описании сборки проекта.
278	`  PATTERN ".DS_Store" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN ".DS_Store" EXCLUDE`. Логически она участвует в описании сборки проекта.
279	`  PATTERN "._*" EXCLUDE`	Технически это команда или аргумент CMake `PATTERN "._*" EXCLUDE`. Логически она участвует в описании сборки проекта.
280	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
281	`install(`	Технически описывает установку файлов. Логически release-пакет получает exe, docs, examples и служебные материалы.

Стр.	Код	Нормальное объяснение
282	`  FILES`	Технически это команда или аргумент CMake `FILES`. Логически она участвует в описании сборки проекта.
283	`  README.md`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
284	`  LICENSE`	Технически это команда или аргумент CMake `LICENSE`. Логически она участвует в описании сборки проекта.
285	`  CHANGELOG.md`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
286	`  SECURITY.md`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
287	`  CONTRIBUTING.md`	Технически это элемент списка файлов. Логически CMake включает этот исходник, документ или ресурс в сборку/установку.
288	`  DESTINATION share/eh_unwind_analyzer`	Технически это команда или аргумент CMake `DESTINATION share/eh_unwind_analyzer`. Логически она участвует в описании сборки проекта.
289	`)`	Технически эта строка относится к объявлению проекта CMake. Логически здесь задаются имя, версия, описание и язык C.
291	`set(CPACK_PACKAGE_NAME "EH-Unwind-Analyzer")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
292	`set(CPACK_PACKAGE_VENDOR "EH/Unwind Analyzer project")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
293	`set(CPACK_PACKAGE_VERSION "1.0.0")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
294	`set(CPACK_PACKAGE_FILE_NAME "eh-unwind-analyzer-${CPACK_PACKAGE_VERSION}-${CMAKE_SYSTEM_NAME}-${CMAKE_SYSTEM_PROCESSOR}")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
295	`set(CPACK_GENERATOR "ZIP")`	Технически создает или изменяет переменную CMake. Логически так собираются списки исходников, параметры packaging или окружение тестов.
296	`include(CPack)`	Технически подключает стандартный модуль CMake. Логически это включает поддержку тестов или проверок сборки.

Разобрано непустых строк: 278. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

CMakePresets.json

Назначение файла: набор готовых профилей CMake для debug/release и разных генераторов.

Код, который пишем на этом шаге

```
 1 | {
 2 |   "version": 6,
 3 |   "cmakeMinimumRequired": {
 4 |     "major": 3,
 5 |     "minor": 23,
 6 |     "patch": 0
 7 |   },
 8 |   "configurePresets": [
 9 |     {
10 |       "name": "base",
11 |       "hidden": true,
12 |       "generator": "Ninja",
13 |       "binaryDir": "${sourceDir}/build/${presetName}",
```

```
    "cacheVariables": {
      "BUILD_TESTING": "ON",
      "CMAKE_EXPORT_COMPILE_COMMANDS": "ON",
      "EHUA_WARNINGS_AS_ERRORS": "ON"
    }
  },
  {
    "name": "debug",
    "displayName": "Debug",
    "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "Debug"
    }
  },
  {
    "name": "release",
    "displayName": "Release",
    "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "Release"
    }
  },
  {
    "name": "sanitize",
    "displayName": "ASan + UBSan",
    "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "Debug",
      "EHUA_ENABLE_SANITIZERS": "ON"
    }
  },
  {
    "name": "coverage",
    "displayName": "Coverage",
    "inherits": "base",
    "cacheVariables": {
      "CMAKE_BUILD_TYPE": "Debug",
      "EHUA_ENABLE_COVERAGE": "ON"
    }
  }
 ],
 "buildPresets": [
  {
    "name": "debug",
    "configurePreset": "debug"
  },
  {
    "name": "release",
    "configurePreset": "release"
  },
  {
    "name": "sanitize",
    "configurePreset": "sanitize"
  },
  {
    "name": "coverage",
    "configurePreset": "coverage"
  }
 ],
 "testPresets": [
  {
    "name": "debug",
    "configurePreset": "debug",
    "output": {
      "outputOnFailure": true
    }
  },
  {
    "name": "release",
    "configurePreset": "release",
    "output": {
      "outputOnFailure": true
    }
  },
  {
    "name": "sanitize",
    "configurePreset": "sanitize",
    "output": {
      "outputOnFailure": true
    }
  },
  {
    "name": "coverage",
    "configurePreset": "coverage",
    "output": {
      "outputOnFailure": true
    }
  }
 ],
 "packagePresets": [
  {
    "name": "release",
    "configurePreset": "release",
    "generators": [
      "ZIP"
    ]
  }
 ]
}
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
2	`"version": 6,`	Технически задает JSON-поле `version`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
3	`"cmakeMinimumRequired": {`	Технически задает JSON-поле `cmakeMinimumRequired`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.

Стр.	Код	Нормальное объяснение
4	`"major": 3,`	Технически задает JSON-поле `major`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
5	`"minor": 23,`	Технически задает JSON-поле `minor`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
6	`"patch": 0`	Технически задает JSON-поле `patch`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
7	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
8	`"configurePresets": [`	Технически задает JSON-поле `configurePresets`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
9	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
10	`"name": "base",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
11	`"hidden": true,`	Технически задает JSON-поле `hidden`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
12	`"generator": "Ninja",`	Технически задает JSON-поле `generator`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
13	`"binaryDir": "${sourceDir}/build/${presetName}",`	Технически задает JSON-поле `binaryDir`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
14	`"cacheVariables": {`	Технически задает JSON-поле `cacheVariables`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
15	`"BUILD_TESTING": "ON",`	Технически задает JSON-поле `BUILD_TESTING`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
16	`"CMAKE_EXPORT_COMPILE_COMMANDS": "ON",`	Технически задает JSON-поле `CMAKE_EXPORT_COMPILE_COMMANDS`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
17	`"EHUA_WARNINGS_AS_ERRORS": "ON"`	Технически задает JSON-поле `EHUA_WARNINGS_AS_ERRORS`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
18	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
19	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
20	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
21	`"name": "debug",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
22	`"displayName": "Debug",`	Технически задает JSON-поле `displayName`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
23	`"inherits": "base",`	Технически задает JSON-поле `inherits`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
24	`"cacheVariables": {`	Технически задает JSON-поле `cacheVariables`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
25	`"CMAKE_BUILD_TYPE": "Debug"`	Технически задает JSON-поле `CMAKE_BUILD_TYPE`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
26	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
27	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
28	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
29	`"name": "release",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
30	`"displayName": "Release",`	Технически задает JSON-поле `displayName`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
31	`"inherits": "base",`	Технически задает JSON-поле `inherits`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
32	`"cacheVariables": {`	Технически задает JSON-поле `cacheVariables`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
33	`"CMAKE_BUILD_TYPE": "Release"`	Технически задает JSON-поле `CMAKE_BUILD_TYPE`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.

Стр.	Код	Нормальное объяснение
34	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
35	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.

Стр.	Код	Нормальное объяснение
36	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
37	`"name": "sanitize",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
38	`"displayName": "ASan + UBSan",`	Технически задает JSON-поле `displayName`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
39	`"inherits": "base",`	Технически задает JSON-поле `inherits`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
40	`"cacheVariables": {`	Технически задает JSON-поле `cacheVariables`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
41	`"CMAKE_BUILD_TYPE": "Debug",`	Технически задает JSON-поле `CMAKE_BUILD_TYPE`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
42	`"EHUA_ENABLE_SANITIZERS": "ON"`	Технически задает JSON-поле `EHUA_ENABLE_SANITIZERS`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
43	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
44	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
45	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
46	`"name": "coverage",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
47	`"displayName": "Coverage",`	Технически задает JSON-поле `displayName`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
48	`"inherits": "base",`	Технически задает JSON-поле `inherits`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
49	`"cacheVariables": {`	Технически задает JSON-поле `cacheVariables`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
50	`"CMAKE_BUILD_TYPE": "Debug",`	Технически задает JSON-поле `CMAKE_BUILD_TYPE`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
51	`"EHUA_ENABLE_COVERAGE": "ON"`	Технически задает JSON-поле `EHUA_ENABLE_COVERAGE`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
52	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
53	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
54	`],`	Технически это часть JSON-конфигурации `],`. Логически она помогает запускать сборку одинаковыми командами.
55	`"buildPresets": [`	Технически задает JSON-поле `buildPresets`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
56	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
57	`"name": "debug",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
58	`"configurePreset": "debug"`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
59	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
60	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
61	`"name": "release",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
62	`"configurePreset": "release"`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
63	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
64	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
65	`"name": "sanitize",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
66	`"configurePreset": "sanitize"`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.

Стр.	Код	Нормальное объяснение
67	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.

Стр.	Код	Нормальное объяснение
68	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
69	`"name": "coverage",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
70	`"configurePreset": "coverage"`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
71	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
72	`],`	Технически это часть JSON-конфигурации `],`. Логически она помогает запускать сборку одинаковыми командами.
73	`"testPresets": [`	Технически задает JSON-поле `testPresets`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
74	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
75	`"name": "debug",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
76	`"configurePreset": "debug",`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
77	`"output": {`	Технически задает JSON-поле `output`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
78	`"outputOnFailure": true`	Технически задает JSON-поле `outputOnFailure`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
79	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
80	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
81	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
82	`"name": "release",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
83	`"configurePreset": "release",`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
84	`"output": {`	Технически задает JSON-поле `output`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
85	`"outputOnFailure": true`	Технически задает JSON-поле `outputOnFailure`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
86	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
87	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
88	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
89	`"name": "sanitize",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
90	`"configurePreset": "sanitize",`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
91	`"output": {`	Технически задает JSON-поле `output`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
92	`"outputOnFailure": true`	Технически задает JSON-поле `outputOnFailure`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
93	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
94	`},`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
95	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
96	`"name": "coverage",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
97	`"configurePreset": "coverage",`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
98	`"output": {`	Технически задает JSON-поле `output`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
99	`"outputOnFailure": true`	Технически задает JSON-поле `outputOnFailure`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.

Стр.	Код	Нормальное объяснение
100	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
101	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
102	`],`	Технически это часть JSON-конфигурации `],`. Логически она помогает запускать сборку одинаковыми командами.
103	`"packagePresets": [`	Технически задает JSON-поле `packagePresets`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
104	`{`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
105	`"name": "release",`	Технически задает JSON-поле `name`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
106	`"configurePreset": "release",`	Технически задает JSON-поле `configurePreset`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
107	`"generators": [`	Технически задает JSON-поле `generators`. Логически это параметр CMake preset: генератор, buildDir, cacheVariables или имя конфигурации.
108	`"ZIP"`	Технически это часть JSON-конфигурации `"ZIP"`. Логически она помогает запускать сборку одинаковыми командами.
109	`]`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
110	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
111	`]`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.
112	`}`	Технически строка открывает или закрывает JSON-объект/массив. Логически она группирует настройки preset в корректный JSON.

Разобрано непустых строк: 112. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

requirements-dev.txt

Назначение файла: Python-зависимости для разработки и тестов.

Код, который пишем на этом шаге

```
1 | jsonschema==4.26.0
2 | gcovr==8.6
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`jsonschema==4.26.0`	Технически строка объявляет зависимость `jsonschema==4.26.0`. Логически она нужна Python-тестам или валидации, но не является частью C-ядра.
2	`gcovr==8.6`	Технически строка объявляет зависимость `gcovr==8.6`. Логически она нужна Python-тестам или валидации, но не является частью C-ядра.

Разобрано непустых строк: 2. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

requirements-validation.txt

Назначение файла: Python-зависимости для валидации отчетов и проверочных скриптов.

Код, который пишем на этом шаге

```
1 | jsonschema==4.26.0
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`jsonschema==4.26.0`	Технически строка объявляет зависимость `jsonschema==4.26.0`. Логически она нужна Python-тестам или валидации, но не является частью C-ядра.

Разобрано непустых строк: 1. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

2. Версия, платформа и метаданные сборки

Этот модуль нужен, чтобы отчет был воспроизводимым: по нему видно, какой версией анализатора, каким компилятором и в каком режиме был получен результат.

Что сначала выучить в C и теории

- заголовочные файлы и объявления функций
- условная компиляция через #if/#ifdef
- работа со строками времени и буферами фиксированного размера

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/app_info.h

Назначение файла: публичный интерфейс метаданных версии и сборки.

Код, который пишем на этом шаге

```
 1 | #pragma once
 2 |
 3 | #include <stdbool.h>
 4 | #include <stddef.h>
 5 |
 6 | #ifdef __cplusplus
 7 | extern "C" {
 8 | #endif
 9 |
10 | #define EHUA_VERSION "1.0.0"
11 | #define EHUA_REPORT_SCHEMA_VERSION "1.1.0"
12 |
13 | const char* app_version(void);
14 | const char* app_report_schema_version(void);
15 | const char* app_build_compiler(void);
16 | const char* app_build_platform(void);
17 | const char* app_build_mode(void);
18 |
19 | /* UTC timestamp in RFC 3339 form: YYYY-MM-DDTHH:MM:SSZ. */
20 | bool app_utc_timestamp(char* out, size_t out_size);
21 |
22 | #ifdef __cplusplus
23 | }
24 | #endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
3	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
4	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
6	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
7	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
8	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
10	`#define EHUA_VERSION "1.0.0"`	Технически объявляет макрос `EHUA_VERSION` со значением `"1.0.0"`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
11	`#define EHUA_REPORT_SCHEMA_VERSION "1.1.0"`	Технически объявляет макрос `EHUA_REPORT_SCHEMA_VERSION` со значением `"1.1.0"`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
13	`const char* app_version(void);`	Технически объявляет прототип функции `app_version` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и возвращает версию анализатора, чтобы отчет можно было связать с конкретным выпуском.
14	`const char* app_report_schema_version(void);`	Технически объявляет прототип функции `app_report_schema_version` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и возвращает версию схемы отчета, чтобы тесты и внешние инструменты понимали формат JSON.
15	`const char* app_build_compiler(void);`	Технически объявляет прототип функции `app_build_compiler` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и определяет компилятор, которым собран бинарник анализатора.
16	`const char* app_build_platform(void);`	Технически объявляет прототип функции `app_build_platform` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и определяет платформу сборки: Windows, macOS, Linux или другое окружение.

Стр.	Код	Нормальное объяснение
17	`const char* app_build_mode(void);`	Технически объявляет прототип функции `app_build_mode` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и отдает режим сборки debug/release по макросу NDEBUG.
19	`/* UTC timestamp in RFC 3339 form: YYYY-MM-DDTHH:MM:SSZ. */`	Технически это комментарий исходника. Логически он объясняет разработчику: UTC timestamp in RFC 3339 form: YYYY-MM-DDTHH:MM:SSZ..
20	`bool app_utc_timestamp(char* out, size_t out_size);`	Технически объявляет прототип функции `app_utc_timestamp` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и формирует UTC-время для метаданных отчета.
22	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
23	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
24	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 18. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/app_info.c

Назначение файла: реализация версии, платформы, режима сборки и UTC timestamp.

Код, который пишем на этом шаге

```
 1 | #include "app_info.h"
 2 |
 3 | #include <stdio.h>
 4 | #include <time.h>
 5 |
 6 | const char* app_version(void) {
 7 |     return EHUA_VERSION;
 8 | }
 9 |
10 | const char* app_report_schema_version(void) {
11 |     return EHUA_REPORT_SCHEMA_VERSION;
12 | }
13 |
14 | const char* app_build_compiler(void) {
15 | #if defined(_MSC_VER)
16 |     static char compiler[48];
17 |     if (compiler[0] == '\0') {
18 |         (void)snprintf(compiler, sizeof(compiler), "MSVC %d", _MSC_VER);
19 |     }
20 |     return compiler;
21 | #elif defined(__clang__)
22 |     return "Clang " __clang_version__;
23 | #elif defined(__GNUC__)
24 |     return "GCC " __VERSION__;
25 | #else
26 |     return "unknown";
27 | #endif
28 | }
29 |
30 | const char* app_build_platform(void) {
31 | #if defined(_WIN64)
32 |     return "windows-x64";
33 | #elif defined(_WIN32)
34 |     return "windows-x86";
35 | #elif defined(__APPLE__) && defined(__aarch64__)
36 |     return "macos-arm64";
37 | #elif defined(__APPLE__) && defined(__x86_64__)
38 |     return "macos-x64";
39 | #elif defined(__linux__) && defined(__x86_64__)
40 |     return "linux-x64";
41 | #elif defined(__linux__) && defined(__aarch64__)
42 |     return "linux-arm64";
43 | #else
44 |     return "unknown";
45 | #endif
46 | }
47 |
48 | const char* app_build_mode(void) {
49 | #if defined(NDEBUG)
50 |     return "release";
51 | #else
52 |     return "debug";
53 | #endif
54 | }
55 |
56 | bool app_utc_timestamp(char* out, size_t out_size) {
57 |     if (!out || out_size < 21) return false;
58 |     time_t now = time(NULL);
59 |     if (now == (time_t)-1) return false;
60 |
61 |     struct tm utc;
62 | #if defined(_WIN32)
63 |     if (gmtime_s(&utc, &now) != 0) return false;
64 | #else
65 |     if (!gmtime_r(&now, &utc)) return false;
66 | #endif
67 |     return strftime(out, out_size, "%Y-%m-%dT%H:%M:%SZ", &utc) == 20;
68 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "app_info.h"`	Технически подключает `app_info.h`. Логически это дает модулю доступ к: публичный интерфейс метаданных версии и сборки.
3	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.
4	`#include <time.h>`	Технически подключает `time.h`. Логически это дает модулю доступ к: работа со временем сборки/отчета.
6	`const char* app_version(void) {`	Технически объявляет/начинает функцию `app_version`; она доступная другим модулям через заголовок или линковку. Логически функция возвращает версию анализатора, чтобы отчет можно было связать с конкретным выпуском.
7	`return EHUA_VERSION;`	Технически возвращает выражение `EHUA_VERSION`. Логически именно это значение становится результатом функции `app_version` для вызывающего кода.
8	`}`	Технически завершает область видимости внутри `app_version`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
10	`const char* app_report_schema_version(void) {`	Технически объявляет/начинает функцию `app_report_schema_version`; она доступная другим модулям через заголовок или линковку. Логически функция возвращает версию схемы отчета, чтобы тесты и внешние инструменты понимали формат JSON.
11	`return EHUA_REPORT_SCHEMA_VERSION;`	Технически возвращает выражение `EHUA_REPORT_SCHEMA_VERSION`. Логически именно это значение становится результатом функции `app_report_schema_version` для вызывающего кода.
12	`}`	Технически завершает область видимости внутри `app_report_schema_version`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
14	`const char* app_build_compiler(void) {`	Технически объявляет/начинает функцию `app_build_compiler`; она доступная другим модулям через заголовок или линковку. Логически функция определяет компилятор, которым собран бинарник анализатора.
15	`#if defined(_MSC_VER)`	Технически включает код только при условии `#if defined(_MSC_VER)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
16	`static char compiler[48];`	Технически объявляет `compiler` в функции `app_build_compiler`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
17	`if (compiler[0] == '\0') {`	Технически проверяет условие `compiler[0] == '\0'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
18	`(void)snprintf(compiler, sizeof(compiler), "MSVC %d", _MSC_VER);`	Технически вызывает `snprintf`. Логически это действие нужно в функции `app_build_compiler`: формирует строку с ограничением размера буфера.
19	`}`	Технически завершает область видимости внутри `app_build_compiler`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
20	`return compiler;`	Технически возвращает выражение `compiler`. Логически именно это значение становится результатом функции `app_build_compiler` для вызывающего кода.
21	`#elif defined(__clang__)`	Технически включает код только при условии `#elif defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
22	`return "Clang " __clang_version__;`	Технически возвращает выражение `"Clang " __clang_version__`. Логически именно это значение становится результатом функции `app_build_compiler` для вызывающего кода.
23	`#elif defined(__GNUC__)`	Технически включает код только при условии `#elif defined(__GNUC__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
24	`return "GCC " __VERSION__;`	Технически возвращает выражение `"GCC " __VERSION__`. Логически именно это значение становится результатом функции `app_build_compiler` для вызывающего кода.
25	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
26	`return "unknown";`	Технически возвращает выражение `"unknown"`. Логически именно это значение становится результатом функции `app_build_compiler` для вызывающего кода.
27	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
28	`}`	Технически завершает область видимости внутри `app_build_compiler`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
30	`const char* app_build_platform(void) {`	Технически объявляет/начинает функцию `app_build_platform`; она доступная другим модулям через заголовок или линковку. Логически функция определяет платформу сборки: Windows, macOS, Linux или другое окружение.
31	`#if defined(_WIN64)`	Технически включает код только при условии `#if defined(_WIN64)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
32	`return "windows-x64";`	Технически возвращает выражение `"windows-x64"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.
33	`#elif defined(_WIN32)`	Технически включает код только при условии `#elif defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
34	`return "windows-x86";`	Технически возвращает выражение `"windows-x86"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.

Стр.	Код	Нормальное объяснение
35	`#elif defined(__APPLE__) && defined(__aarch64__)`	Технически включает код только при условии `#elif defined(__APPLE__) && defined(__aarch64__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
36	`return "macos-arm64";`	Технически возвращает выражение `"macos-arm64"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.
37	`#elif defined(__APPLE__) && defined(__x86_64__)`	Технически включает код только при условии `#elif defined(__APPLE__) && defined(__x86_64__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
38	`return "macos-x64";`	Технически возвращает выражение `"macos-x64"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.

Стр.	Код	Нормальное объяснение
39	`#elif defined(__linux__) && defined(__x86_64__)`	Технически включает код только при условии `#elif defined(__linux__) && defined(__x86_64__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
40	`return "linux-x64";`	Технически возвращает выражение `"linux-x64"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.
41	`#elif defined(__linux__) && defined(__aarch64__)`	Технически включает код только при условии `#elif defined(__linux__) && defined(__aarch64__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
42	`return "linux-arm64";`	Технически возвращает выражение `"linux-arm64"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.
43	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
44	`return "unknown";`	Технически возвращает выражение `"unknown"`. Логически именно это значение становится результатом функции `app_build_platform` для вызывающего кода.
45	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
46	`}`	Технически завершает область видимости внутри `app_build_platform`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
48	`const char* app_build_mode(void) {`	Технически объявляет/начинает функцию `app_build_mode`; она доступная другим модулям через заголовок или линковку. Логически функция отдает режим сборки debug/release по макросу NDEBUG.
49	`#if defined(NDEBUG)`	Технически включает код только при условии `#if defined(NDEBUG)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
50	`return "release";`	Технически возвращает выражение `"release"`. Логически именно это значение становится результатом функции `app_build_mode` для вызывающего кода.
51	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
52	`return "debug";`	Технически возвращает выражение `"debug"`. Логически именно это значение становится результатом функции `app_build_mode` для вызывающего кода.
53	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
54	`}`	Технически завершает область видимости внутри `app_build_mode`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
56	`bool app_utc_timestamp(char* out, size_t out_size) {`	Технически объявляет/начинает функцию `app_utc_timestamp`; она доступная другим модулям через заголовок или линковку. Логически функция формирует UTC-время для метаданных отчета.
57	`if (!out || out_size < 21) return false;`	Технически проверяет условие `!out || out_size < 21`; логически это защита от NULL-указателя и работы с неинициализированными данными.
58	`time_t now = time(NULL);`	Технически присваивает `time_t now` значение `time(NULL)`. Логически это обновляет `time_t now`: хранит промежуточное значение, которое дальше используется в текущей функции.
59	`if (now == (time_t)-1) return false;`	Технически проверяет условие `now == (time_t)-1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
61	`struct tm utc;`	Технически начинает описание структуры `utc`. Логически структура собирает связанные поля в одну модель данных проекта.
62	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
63	`if (gmtime_s(&utc, &now) != 0) return false;`	Технически проверяет условие `gmtime_s(&utc, &now) != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
64	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
65	`if (!gmtime_r(&now, &utc)) return false;`	Технически проверяет условие `!gmtime_r(&now, &utc)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
66	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Стр.	Код	Нормальное объяснение
67	`return strftime(out, out_size, "%Y-%m-%dT%H:%M:%SZ", &utc) == 20;`	Технически возвращает выражение `strftime(out, out_size, "%Y-%m-%dT%H:%M:%SZ", &utc) == 20`. Логически именно это значение становится результатом функции `app_utc_timestamp` для вызывающего кода.
68	`}`	Технически завершает область видимости внутри `app_utc_timestamp`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 60. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

3. Безопасное чтение файла, строки и SHA-256

Это фундамент проекта. PE-файл нельзя читать как обычную структуру C без проверок: вход может быть битым, коротким или специально поврежденным.

Что сначала выучить в C и теории

- указатели, владение памятью и освобождение malloc/free
- проверка границ перед чтением байтов
- динамический строковый буфер и varargs
- побайтовые операции и SHA-256 как хеш входного бинарника

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/util.h

Назначение файла: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.

Код, который пишем на этом шаге

```
#pragma once
/*
  util.h - small helpers for safe parsing and dynamic arrays
  Project: EH/Unwind Analyzer (PE x64)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
void buf_free_owned(uint8_t* owned_mem);

bool buf_read(const Buf* b, size_t off, void* out, size_t len);
bool buf_read_u8 (const Buf* b, size_t off, uint8_t*  out);
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);

static inline uint16_t le16(const uint8_t* p) {
    return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));
}
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24));
}
static inline uint64_t le64(const uint8_t* p) {
    return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);
}

typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;

#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(gnu_printf, format_index, first_arg)))
#elif defined(__GNUC__) || defined(__clang__)
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(printf, format_index, first_arg)))
#else
#define EHUA_PRINTF_FORMAT(format_index, first_arg)
#endif

bool sb_init(StrBuf* sb, size_t initial_cap);
void sb_free(StrBuf* sb);
bool sb_appendf(StrBuf* sb, const char* fmt, ...)
    EHUA_PRINTF_FORMAT(2, 3);
bool sb_append(StrBuf* sb, const char* s);
bool sb_appendn(StrBuf* sb, const char* s, size_t n);
bool sb_ok(const StrBuf* sb);
bool file_write_atomic(const char* path, const char* data, size_t len);

/* SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL. */
bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);

#undef EHUA_PRINTF_FORMAT

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`util.h - small helpers for safe parsing and dynamic arrays`	Технически эта строка является частью C-конструкции `util.h - small helpers for safe parsing and dynamic arrays`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
4	`Project: EH/Unwind Analyzer (PE x64)`	Технически объявляет/начинает функцию `Analyzer`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
5	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
7	`#include <stdint.h>`	Технически подключает `stdint.h`. Логически это дает модулю доступ к: точные типы uint8_t/uint32_t/uint64_t, без которых нельзя надежно читать бинарный формат.
8	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
9	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
11	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
12	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
13	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
15	`typedef struct Buf {`	Технически начинает описание структуры `Buf`. Логически структура собирает связанные поля в одну модель данных проекта.
16	`const uint8_t* data;`	Технически объявляет поле `data` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
17	`size_t size;`	Технически объявляет поле `size` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
18	`} Buf;`	Технически завершает typedef и дает типу имя `Buf`. Логически дальше код может объявлять переменные этого типа без `struct`.
20	`bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);`	Технически объявляет прототип функции `buf_init_from_file` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и загружает весь файл в память и создает безопасную обертку Buf.
21	`void buf_free_owned(uint8_t* owned_mem);`	Технически объявляет прототип функции `buf_free_owned` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и освобождает память, которой владеет Buf.
23	`bool buf_read(const Buf* b, size_t off, void* out, size_t len);`	Технически объявляет прототип функции `buf_read` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и безопасно копирует участок входного файла после проверки границ.
24	`bool buf_read_u8 (const Buf* b, size_t off, uint8_t* out);`	Технически объявляет прототип функции `buf_read_u8` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и читает один байт из входного буфера.
25	`bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);`	Технически объявляет прототип функции `buf_read_u16` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и читает 16-битное little-endian поле PE.
26	`bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);`	Технически объявляет прототип функции `buf_read_u32` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и читает 32-битное little-endian поле PE.
27	`bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);`	Технически объявляет прототип функции `buf_read_u64` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и читает 64-битное little-endian поле PE.
29	`static inline uint16_t le16(const uint8_t* p) {`	Технически объявляет/начинает функцию `le16`; она внутренняя для файла. Логически функция собирает 16-битное число из байтов little-endian.
30	`return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));`	Технически возвращает выражение `(uint16_t)(p[0] | ((uint16_t)p[1] << 8))`. Логически именно это значение становится результатом функции `le16` для вызывающего кода.
31	`}`	Технически завершает область видимости внутри `le16`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
32	`static inline uint32_t le32(const uint8_t* p) {`	Технически объявляет/начинает функцию `le32`; она внутренняя для файла. Логически функция собирает 32-битное число из байтов little-endian.
33	`return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24));`	Технически возвращает выражение `(uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24))`. Логически именно это значение становится результатом функции `le32` для вызывающего кода.
34	`}`	Технически завершает область видимости внутри `le32`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
35	`static inline uint64_t le64(const uint8_t* p) {`	Технически объявляет/начинает функцию `le64`; она внутренняя для файла. Логически функция собирает 64-битное число из байтов little-endian.
36	`return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);`	Технически возвращает выражение `(uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32)`. Логически именно это значение становится результатом функции `le64` для вызывающего кода.
37	`}`	Технически завершает область видимости внутри `le64`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
39	`typedef struct StrBuf {`	Технически начинает описание структуры `StrBuf`. Логически структура собирает связанные поля в одну модель данных проекта.
40	`char* s;`	Технически объявляет поле `s` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
41	`size_t len;`	Технически объявляет поле `len` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
42	`size_t cap;`	Технически объявляет поле `cap` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
43	`bool failed;`	Технически объявляет поле `failed` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
44	`} StrBuf;`	Технически завершает typedef и дает типу имя `StrBuf`. Логически дальше код может объявлять переменные этого типа без `struct`.
46	`#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))`	Технически включает код только при условии `#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
47	`#define EHUA_PRINTF_FORMAT(format_index, first_arg) \`	Технически объявляет макрос `EHUA_PRINTF_FORMAT` со значением `(format_index, first_arg) \`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
48	`__attribute__((format(gnu_printf, format_index, first_arg)))`	Технически объявляет/начинает функцию `__attribute__`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
49	`#elif defined(__GNUC__) || defined(__clang__)`	Технически включает код только при условии `#elif defined(__GNUC__) || defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
50	`#define EHUA_PRINTF_FORMAT(format_index, first_arg) \`	Технически объявляет макрос `EHUA_PRINTF_FORMAT` со значением `(format_index, first_arg) \`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
51	`__attribute__((format(printf, format_index, first_arg)))`	Технически объявляет/начинает функцию `__attribute__`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
52	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
53	`#define EHUA_PRINTF_FORMAT(format_index, first_arg)`	Технически объявляет макрос `EHUA_PRINTF_FORMAT` со значением `(format_index, first_arg)`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
54	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
56	`bool sb_init(StrBuf* sb, size_t initial_cap);`	Технически объявляет прототип функции `sb_init` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и инициализирует динамический строковый буфер.
57	`void sb_free(StrBuf* sb);`	Технически объявляет прототип функции `sb_free` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и освобождает память динамического строкового буфера.
58	`bool sb_appendf(StrBuf* sb, const char* fmt, ...)`	Технически объявляет/начинает функцию `sb_appendf`; она доступная другим модулям через заголовок или линковку. Логически функция добавляет форматированный текст в StrBuf.
59	`EHUA_PRINTF_FORMAT(2, 3);`	Технически вызывает `EHUA_PRINTF_FORMAT`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `EHUA_PRINTF_FORMAT` для выполнения части алгоритма.
60	`bool sb_append(StrBuf* sb, const char* s);`	Технически объявляет прототип функции `sb_append` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет C-строку в StrBuf.
61	`bool sb_appendn(StrBuf* sb, const char* s, size_t n);`	Технически объявляет прототип функции `sb_appendn` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет ровно n байтов текста в StrBuf.
62	`bool sb_ok(const StrBuf* sb);`	Технически объявляет прототип функции `sb_ok` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и проверяет, что StrBuf не перешел в состояние ошибки.
63	`bool file_write_atomic(const char* path, const char* data, size_t len);`	Технически объявляет прототип функции `file_write_atomic` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и записывает файл через временное имя, чтобы не оставлять битый отчет при сбое.
65	`/* SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL. */`	Технически это комментарий исходника. Логически он объясняет разработчику: SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL..

Стр.	Код	Нормальное объяснение
66	`bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);`	Технически объявляет прототип функции `sha256_hex` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и считает SHA-256 входного бинарника для воспроизводимости отчета.
68	`#undef EHUA_PRINTF_FORMAT`	Технически это директива препроцессора `#undef EHUA_PRINTF_FORMAT`. Логически она меняет код еще до компиляции C.
70	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
71	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
72	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 60. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/util.c

Назначение файла: реализация файлового чтения, StrBuf, атомарной записи и SHA-256.

Код, который пишем на этом шаге

```
#include "util.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

#if defined(_WIN32)
#include <windows.h>
#else
#include <unistd.h>
#endif

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem) {
    if (!out || !path || !owned_mem) return false;
    *owned_mem = NULL;
    FILE* f = fopen(path, "rb");
    if (!f) return false;

    if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return false; }
    long sz = ftell(f);
    if (sz < 0) { fclose(f); return false; }
    if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); return false; }

    size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
    uint8_t* mem = (uint8_t*)malloc(alloc_size);
    if (!mem) { fclose(f); return false; }

    size_t rd = fread(mem, 1, (size_t)sz, f);
    fclose(f);
    if (rd != (size_t)sz) { free(mem); return false; }

    out->data = mem;
    out->size = (size_t)sz;
    *owned_mem = mem;
    return true;
}

void buf_free_owned(uint8_t* owned_mem) {
    free(owned_mem);
}

bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
}

static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
```

```
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}

bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
        sb->failed = true;
        return false;
    }
    sb->s[0] = 0;
    return true;
}

void sb_free(StrBuf* sb) {
    if (!sb) return;
    free(sb->s);
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
}

bool sb_appendn(StrBuf* sb, const char* s, size_t n) {
    if (!sb) return false;
    if (sb->failed || (!s && n != 0)) {
        sb->failed = true;
        return false;
    }
    if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {
        sb->failed = true;
        return false;
    }
    if (!sb_reserve(sb, sb->len + n + 1)) return false;
    if (n != 0) memcpy(sb->s + sb->len, s, n);
    sb->len += n;
    sb->s[sb->len] = 0;
    return true;
}

bool sb_append(StrBuf* sb, const char* s) {
    if (!sb) return false;
    if (!s) {
        sb->failed = true;
        return false;
    }
    return sb_appendn(sb, s, strlen(s));
}

bool sb_appendf(StrBuf* sb, const char* fmt, ...) {
    if (!sb) return false;
    if (sb->failed || !fmt) {
        sb->failed = true;
        return false;
    }
    va_list ap;
    va_start(ap, fmt);
    va_list ap2;
    va_copy(ap2, ap);

#if defined(__GNUC__) || defined(__clang__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wformat-nonliteral"
#endif
    int need = vsnprintf(NULL, 0, fmt, ap);
    va_end(ap);
    if (need < 0) {
        va_end(ap2);
        sb->failed = true;
        return false;
    }

    if (sb->len == SIZE_MAX || (size_t)need > SIZE_MAX - sb->len - 1u) {
        va_end(ap2);
        sb->failed = true;
        return false;
    }
    if (!sb_reserve(sb, sb->len + (size_t)need + 1u)) { va_end(ap2); return false; }
    int wrote = vsnprintf(sb->s + sb->len, sb->cap - sb->len, fmt, ap2);
#if defined(__GNUC__) || defined(__clang__)
#pragma GCC diagnostic pop
#endif
    va_end(ap2);
    if (wrote != need) {
        sb->failed = true;
        return false;
    }
    sb->len += (size_t)wrote;
    return true;
}

bool sb_ok(const StrBuf* sb) {
    return sb && !sb->failed && sb->s && sb->len < sb->cap;
}

bool file_write_atomic(const char* path, const char* data, size_t len) {
    if (!path || (!data && len != 0)) return false;
    size_t path_len = strlen(path);
    if (path_len > SIZE_MAX - 48u) return false;

    size_t temp_size = path_len + 48u;
```

```
    char* temp_path = (char*)malloc(temp_size);
    if (!temp_path) return false;
#if defined(_WIN32)
    unsigned long process_id = (unsigned long)GetCurrentProcessId();
#else
    unsigned long process_id = (unsigned long)getpid();
#endif
    int written = snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id);
    if (written < 0 || (size_t)written >= temp_size) {
        free(temp_path);
        return false;
    }

    FILE* f = fopen(temp_path, "wb");
    if (!f) {
        free(temp_path);
        return false;
    }
    size_t wr = len ? fwrite(data, 1, len, f) : 0u;
    bool ok = wr == len && ferror(f) == 0 && fflush(f) == 0;
    if (fclose(f) != 0) ok = false;

    if (ok) {
#if defined(_WIN32)
        ok = MoveFileExA(
                 temp_path,
                 path,
                 MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;
#else
        ok = rename(temp_path, path) == 0;
#endif
    }
    if (!ok) (void)remove(temp_path);
    free(temp_path);
    return ok;
}

typedef struct Sha256Ctx {
    uint32_t state[8];
    uint64_t bit_count;
    uint8_t block[64];
    size_t block_len;
} Sha256Ctx;

static uint32_t sha256_rotr(uint32_t value, unsigned count) {
    return (value >> count) | (value << (32u - count));
}

static void sha256_transform(Sha256Ctx* ctx, const uint8_t block[64]) {
    static const uint32_t k[64] = {
        0x428a2f98u,0x71374491u,0xb5c0fbcfu,0xe9b5dba5u,0x3956c25bu,0x59f111f1u,0x923f82a4u,0xab1c5ed5u,
        0xd807aa98u,0x12835b01u,0x243185beu,0x550c7dc3u,0x72be5d74u,0x80deb1feu,0x9bdc06a7u,0xc19bf174u,
        0xe49b69c1u,0xefbe4786u,0x0fc19dc6u,0x240ca1ccu,0x2de92c6fu,0x4a7484aau,0x5cb0a9dcu,0x76f988dau,
        0x983e5152u,0xa831c66du,0xb00327c8u,0xbf597fc7u,0xc6e00bf3u,0xd5a79147u,0x06ca6351u,0x14292967u,
        0x27b70a85u,0x2e1b2138u,0x4d2c6dfcu,0x53380d13u,0x650a7354u,0x766a0abbu,0x81c2c92eu,0x92722c85u,
        0xa2bfe8a1u,0xa81a664bu,0xc24b8b70u,0xc76c51a3u,0xd192e819u,0xd6990624u,0xf40e3585u,0x106aa070u,
        0x19a4c116u,0x1e376c08u,0x2748774cu,0x34b0bcb5u,0x391c0cb3u,0x4ed8aa4au,0x5b9cca4fu,0x682e6ff3u,
        0x748f82eeu,0x78a5636fu,0x84c87814u,0x8cc70208u,0x90befffau,0xa4506cebu,0xbef9a3f7u,0xc67178f2u
    };
    uint32_t w[64];
    for (size_t i = 0; i < 16; i++) {
        size_t j = i * 4;
        w[i] = ((uint32_t)block[j] << 24) |
               ((uint32_t)block[j + 1] << 16) |
               ((uint32_t)block[j + 2] << 8) |
               (uint32_t)block[j + 3];
    }
    for (size_t i = 16; i < 64; i++) {
        uint32_t s0 = sha256_rotr(w[i - 15], 7) ^ sha256_rotr(w[i - 15], 18) ^ (w[i - 15] >> 3);
        uint32_t s1 = sha256_rotr(w[i - 2], 17) ^ sha256_rotr(w[i - 2], 19) ^ (w[i - 2] >> 10);
        w[i] = w[i - 16] + s0 + w[i - 7] + s1;
    }

    uint32_t a = ctx->state[0], b = ctx->state[1], c = ctx->state[2], d = ctx->state[3];
    uint32_t e = ctx->state[4], f = ctx->state[5], g = ctx->state[6], h = ctx->state[7];
    for (size_t i = 0; i < 64; i++) {
        uint32_t s1 = sha256_rotr(e, 6) ^ sha256_rotr(e, 11) ^ sha256_rotr(e, 25);
        uint32_t ch = (e & f) ^ ((~e) & g);
        uint32_t t1 = h + s1 + ch + k[i] + w[i];
        uint32_t s0 = sha256_rotr(a, 2) ^ sha256_rotr(a, 13) ^ sha256_rotr(a, 22);
        uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
        uint32_t t2 = s0 + maj;
        h = g; g = f; f = e; e = d + t1;
        d = c; c = b; b = a; a = t1 + t2;
    }
    ctx->state[0] += a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d;
    ctx->state[4] += e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h;
}

static void sha256_init(Sha256Ctx* ctx) {
    static const uint32_t initial[8] = {
        0x6a09e667u,0xbb67ae85u,0x3c6ef372u,0xa54ff53au,
        0x510e527fu,0x9b05688cu,0x1f83d9abu,0x5be0cd19u
    };
    memcpy(ctx->state, initial, sizeof(initial));
    ctx->bit_count = 0;
    ctx->block_len = 0;
}

static void sha256_update(Sha256Ctx* ctx, const uint8_t* data, size_t size) {
    if (size == 0) return;
    ctx->bit_count += (uint64_t)size * 8u;
    while (size > 0) {
        size_t available = 64u - ctx->block_len;
        size_t take = size < available ? size : available;
        memcpy(ctx->block + ctx->block_len, data, take);
        ctx->block_len += take;
        data += take;
        size -= take;
        if (ctx->block_len == 64u) {
            sha256_transform(ctx, ctx->block);
            ctx->block_len = 0;
        }
    }
}
```

```
311 |
312 | static void sha256_final(Sha256Ctx* ctx, uint8_t digest[32]) {
313 |     ctx->block[ctx->block_len++] = 0x80;
314 |     if (ctx->block_len > 56u) {
315 |         while (ctx->block_len < 64u) ctx->block[ctx->block_len++] = 0;
316 |         sha256_transform(ctx, ctx->block);
317 |         ctx->block_len = 0;
318 |     }
319 |     while (ctx->block_len < 56u) ctx->block[ctx->block_len++] = 0;
320 |     for (int i = 7; i >= 0; i--) {
321 |         ctx->block[ctx->block_len++] = (uint8_t)(ctx->bit_count >> (unsigned)(i * 8));
322 |     }
323 |     sha256_transform(ctx, ctx->block);
324 |
325 |     for (size_t i = 0; i < 8; i++) {
326 |         digest[i * 4] = (uint8_t)(ctx->state[i] >> 24);
327 |         digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);
328 |         digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);
329 |         digest[i * 4 + 3] = (uint8_t)ctx->state[i];
330 |     }
331 | }
332 |
333 | bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]) {
334 |     static const char digits[] = "0123456789abcdef";
335 |     if (!out_hex || (size > 0 && !data)) return false;
336 |     if ((uint64_t)size > UINT64_MAX / 8u) return false;
337 |     Sha256Ctx ctx;
338 |     uint8_t digest[32];
339 |     sha256_init(&ctx);
340 |     sha256_update(&ctx, data, size);
341 |     sha256_final(&ctx, digest);
342 |     for (size_t i = 0; i < sizeof(digest); i++) {
343 |         out_hex[i * 2] = digits[digest[i] >> 4];
344 |         out_hex[i * 2 + 1] = digits[digest[i] & 0x0fu];
345 |     }
346 |     out_hex[64] = '\0';
347 |     return true;
348 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
2	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.
3	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
4	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
5	`#include <stdarg.h>`	Технически подключает `stdarg.h`. Логически это дает модулю доступ к: va_list для функций вида appendf/printf.
7	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
8	`#include <windows.h>`	Технически подключает `windows.h`. Логически это дает модулю доступ к: WinAPI для динамической проверки под Windows.
9	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
10	`#include <unistd.h>`	Технически подключает `unistd.h`. Логически это дает модулю доступ к: объявления, которые нужны этому модулю.
11	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
13	`bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem) {`	Технически объявляет/начинает функцию `buf_init_from_file`; она доступная другим модулям через заголовок или линковку. Логически функция загружает весь файл в память и создает безопасную обертку Buf.
14	`if (!out || !path || !owned_mem) return false;`	Технически проверяет условие `!out || !path || !owned_mem`; логически это защита от NULL-указателя и работы с неинициализированными данными.
15	`*owned_mem = NULL;`	Технически это комментарий исходника. Логически он объясняет разработчику: owned_mem = NULL;.
16	`FILE* f = fopen(path, "rb");`	Технически присваивает `FILE* f` значение `fopen(path, "rb")`. Логически это обновляет `FILE* f`: хранит промежуточное значение, которое дальше используется в текущей функции.
17	`if (!f) return false;`	Технически проверяет условие `!f`; логически это защита от NULL-указателя и работы с неинициализированными данными.
19	`if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return false; }`	Технически проверяет условие `fseek(f, 0, SEEK_END) != 0) { fclose(f`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
20	`long sz = ftell(f);`	Технически присваивает `long sz` значение `ftell(f)`. Логически это обновляет `long sz`: хранит промежуточное значение, которое дальше используется в текущей функции.
21	`if (sz < 0) { fclose(f); return false; }`	Технически проверяет условие `sz < 0) { fclose(f`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
22	`if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); return false; }`	Технически проверяет условие `fseek(f, 0, SEEK_SET) != 0) { fclose(f`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
24	`size_t alloc_size = sz > 0 ? (size_t)sz : 1u;`	Технически присваивает `size_t alloc_size` значение `sz > 0 ? (size_t)sz : 1u`. Логически это обновляет `size_t alloc_size`: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
25	`uint8_t* mem = (uint8_t*)malloc(alloc_size);`	Технически присваивает `uint8_t* mem` значение `(uint8_t*)malloc(alloc_size)`. Логически это обновляет `uint8_t* mem`: хранит промежуточное значение, которое дальше используется в текущей функции.
26	`if (!mem) { fclose(f); return false; }`	Технически проверяет условие `!mem) { fclose(f`; логически это защита от NULL-указателя и работы с неинициализированными данными.
28	`size_t rd = fread(mem, 1, (size_t)sz, f);`	Технически присваивает `size_t rd` значение `fread(mem, 1, (size_t)sz, f)`. Логически это обновляет `size_t rd`: хранит размер или количество, поэтому используется в проверках границ.
29	`fclose(f);`	Технически вызывает `fclose`. Логически это действие нужно в функции `buf_init_from_file`: закрывает файловый дескриптор.
30	`if (rd != (size_t)sz) { free(mem); return false; }`	Технически проверяет условие `rd != (size_t)sz) { free(mem`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
32	`out->data = mem;`	Технически присваивает `out->data` значение `mem`. Логически это обновляет `data`: хранит промежуточное значение, которое дальше используется в текущей функции.
33	`out->size = (size_t)sz;`	Технически присваивает `out->size` значение `(size_t)sz`. Логически это обновляет `size`: хранит размер или количество, поэтому используется в проверках границ.
34	`*owned_mem = mem;`	Технически это комментарий исходника. Логически он объясняет разработчику: owned_mem = mem;.
35	`return true;`	Технически возвращает `true` из функции `buf_init_from_file`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
36	`}`	Технически завершает область видимости внутри `buf_init_from_file`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
38	`void buf_free_owned(uint8_t* owned_mem) {`	Технически объявляет/начинает функцию `buf_free_owned`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает память, которой владеет Buf.
39	`free(owned_mem);`	Технически вызывает `free`. Логически это действие нужно в функции `buf_free_owned`: освобождает память, чтобы в C не было утечки.
40	`}`	Технически завершает область видимости внутри `buf_free_owned`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
42	`bool buf_read(const Buf* b, size_t off, void* out, size_t len) {`	Технически объявляет/начинает функцию `buf_read`; она доступная другим модулям через заголовок или линковку. Логически функция безопасно копирует участок входного файла после проверки границ.
43	`if (!b || !out) return false;`	Технически проверяет условие `!b || !out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
44	`if (off > b->size) return false;`	Технически проверяет условие `off > b->size`; логически это защита от чтения за пределами файла на диске.
45	`if (len > b->size - off) return false;`	Технически проверяет условие `len > b->size - off`; логически это защита от чтения за пределами файла на диске.
46	`memcpy(out, b->data + off, len);`	Технически вызывает `memcpy`. Логически это действие нужно в функции `buf_read`: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
47	`return true;`	Технически возвращает `true` из функции `buf_read`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
48	`}`	Технически завершает область видимости внутри `buf_read`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
50	`bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {`	Технически объявляет/начинает функцию `buf_read_u8`; она доступная другим модулям через заголовок или линковку. Логически функция читает один байт из входного буфера.
51	`return buf_read(b, off, out, 1);`	Технически объявляет прототип функции `buf_read` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и безопасно копирует участок входного файла после проверки границ.
52	`}`	Технически завершает область видимости внутри `buf_read_u8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
53	`bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {`	Технически объявляет/начинает функцию `buf_read_u16`; она доступная другим модулям через заголовок или линковку. Логически функция читает 16-битное little-endian поле PE.
54	`uint8_t tmp[2];`	Технически объявляет `tmp` в функции `buf_read_u16`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
55	`if (!buf_read(b, off, tmp, 2)) return false;`	Технически проверяет условие `!buf_read(b, off, tmp, 2)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
56	`*out = le16(tmp);`	Технически это комментарий исходника. Логически он объясняет разработчику: out = le16(tmp);.
57	`return true;`	Технически возвращает `true` из функции `buf_read_u16`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.

Стр.	Код	Нормальное объяснение
58	`}`	Технически завершает область видимости внутри `buf_read_u16`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
59	`bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {`	Технически объявляет/начинает функцию `buf_read_u32`; она доступная другим модулям через заголовок или линковку. Логически функция читает 32-битное little-endian поле PE.
60	`uint8_t tmp[4];`	Технически объявляет `tmp` в функции `buf_read_u32`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
61	`if (!buf_read(b, off, tmp, 4)) return false;`	Технически проверяет условие `!buf_read(b, off, tmp, 4)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
62	`*out = le32(tmp);`	Технически это комментарий исходника. Логически он объясняет разработчику: out = le32(tmp);.
63	`return true;`	Технически возвращает `true` из функции `buf_read_u32`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
64	`}`	Технически завершает область видимости внутри `buf_read_u32`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
65	`bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {`	Технически объявляет/начинает функцию `buf_read_u64`; она доступная другим модулям через заголовок или линковку. Логически функция читает 64-битное little-endian поле PE.
66	`uint8_t tmp[8];`	Технически объявляет `tmp` в функции `buf_read_u64`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
67	`if (!buf_read(b, off, tmp, 8)) return false;`	Технически проверяет условие `!buf_read(b, off, tmp, 8)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
68	`*out = le64(tmp);`	Технически это комментарий исходника. Логически он объясняет разработчику: out = le64(tmp);.
69	`return true;`	Технически возвращает `true` из функции `buf_read_u64`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
70	`}`	Технически завершает область видимости внутри `buf_read_u64`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
72	`static bool sb_reserve(StrBuf* sb, size_t need_cap) {`	Технически объявляет/начинает функцию `sb_reserve`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
73	`if (!sb || sb->failed) return false;`	Технически проверяет условие `!sb || sb->failed`; логически это защита от NULL-указателя и работы с неинициализированными данными.
74	`if (need_cap <= sb->cap) return true;`	Технически проверяет условие `need_cap <= sb->cap`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
75	`size_t newcap = sb->cap ? sb->cap : 128;`	Технически объявляет `newcap` в функции `sb_reserve` и сразу задает начальное значение `sb->cap ? sb->cap : 128`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
76	`while (newcap < need_cap) {`	Технически начинает цикл while с условием `newcap < need_cap`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.

Стр.	Код	Нормальное объяснение
77	`size_t increment = newcap / 2u + 64u;`	Технически объявляет `increment` в функции `sb_reserve` и сразу задает начальное значение `newcap / 2u + 64u`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
78	`if (increment > SIZE_MAX - newcap) {`	Технически проверяет условие `increment > SIZE_MAX - newcap`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
79	`newcap = need_cap;`	Технически присваивает `newcap` значение `need_cap`. Логически это обновляет `newcap`: хранит промежуточное значение, которое дальше используется в текущей функции.
80	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
81	`}`	Технически завершает область видимости внутри `sb_reserve`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
82	`newcap += increment;`	Технически изменяет `newcap` оператором += на основе `increment`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
83	`}`	Технически завершает область видимости внутри `sb_reserve`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
84	`char* p = (char*)realloc(sb->s, newcap);`	Технически присваивает `char* p` значение `(char*)realloc(sb->s, newcap)`. Логически это обновляет `char* p`: хранит промежуточное значение, которое дальше используется в текущей функции.
85	`if (!p) {`	Технически проверяет условие `!p`; логически это защита от NULL-указателя и работы с неинициализированными данными.
86	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
87	`return false;`	Технически возвращает `false` из функции `sb_reserve`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.

Стр.	Код	Нормальное объяснение
88	`}`	Технически завершает область видимости внутри `sb_reserve`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
89	`sb->s = p;`	Технически присваивает `sb->s` значение `p`. Логически это обновляет `s`: хранит промежуточное значение, которое дальше используется в текущей функции.
90	`sb->cap = newcap;`	Технически присваивает `sb->cap` значение `newcap`. Логически это обновляет `cap`: хранит промежуточное значение, которое дальше используется в текущей функции.
91	`return true;`	Технически возвращает `true` из функции `sb_reserve`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
92	`}`	Технически завершает область видимости внутри `sb_reserve`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
94	`bool sb_init(StrBuf* sb, size_t initial_cap) {`	Технически объявляет/начинает функцию `sb_init`; она доступная другим модулям через заголовок или линковку. Логически функция инициализирует динамический строковый буфер.
95	`if (!sb) return false;`	Технически проверяет условие `!sb`; логически это защита от NULL-указателя и работы с неинициализированными данными.
96	`sb->s = NULL;`	Технически присваивает `sb->s` значение `NULL`. Логически это обновляет `s`: хранит промежуточное значение, которое дальше используется в текущей функции.
97	`sb->len = 0;`	Технически присваивает `sb->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
98	`sb->cap = 0;`	Технически присваивает `sb->cap` значение `0`. Логически это обновляет `cap`: хранит промежуточное значение, которое дальше используется в текущей функции.
99	`sb->failed = false;`	Технически присваивает `sb->failed` значение `false`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
100	`if (initial_cap == 0) initial_cap = 256;`	Технически проверяет условие `initial_cap == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
101	`if (!sb_reserve(sb, initial_cap) || !sb->s) {`	Технически проверяет условие `!sb_reserve(sb, initial_cap) || !sb->s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
102	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
103	`return false;`	Технически возвращает `false` из функции `sb_init`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
104	`}`	Технически завершает область видимости внутри `sb_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
105	`sb->s[0] = 0;`	Технически присваивает `sb->s[0]` значение `0`. Логически это обновляет `s[0]`: хранит промежуточное значение, которое дальше используется в текущей функции.
106	`return true;`	Технически возвращает `true` из функции `sb_init`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
107	`}`	Технически завершает область видимости внутри `sb_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
109	`void sb_free(StrBuf* sb) {`	Технически объявляет/начинает функцию `sb_free`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает память динамического строкового буфера.
110	`if (!sb) return;`	Технически проверяет условие `!sb`; логически это защита от NULL-указателя и работы с неинициализированными данными.
111	`free(sb->s);`	Технически вызывает `free`. Логически это действие нужно в функции `sb_free`: освобождает память, чтобы в C не было утечки.

Стр.	Код	Нормальное объяснение
112	`sb->s = NULL;`	Технически присваивает `sb->s` значение `NULL`. Логически это обновляет `s`: хранит промежуточное значение, которое дальше используется в текущей функции.
113	`sb->len = 0;`	Технически присваивает `sb->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
114	`sb->cap = 0;`	Технически присваивает `sb->cap` значение `0`. Логически это обновляет `cap`: хранит промежуточное значение, которое дальше используется в текущей функции.
115	`sb->failed = false;`	Технически присваивает `sb->failed` значение `false`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
116	`}`	Технически завершает область видимости внутри `sb_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
118	`bool sb_appendn(StrBuf* sb, const char* s, size_t n) {`	Технически объявляет/начинает функцию `sb_appendn`; она доступная другим модулям через заголовок или линковку. Логически функция добавляет ровно n байтов текста в StrBuf.
119	`if (!sb) return false;`	Технически проверяет условие `!sb`; логически это защита от NULL-указателя и работы с неинициализированными данными.
120	`if (sb->failed || (!s && n != 0)) {`	Технически проверяет условие `sb->failed || (!s && n != 0)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
121	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
122	`return false;`	Технически возвращает `false` из функции `sb_appendn`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
123	`}`	Технически завершает область видимости внутри `sb_appendn`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
124	`if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {`	Технически проверяет условие `sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
125	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
126	`return false;`	Технически возвращает `false` из функции `sb_appendn`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
127	`}`	Технически завершает область видимости внутри `sb_appendn`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
128	`if (!sb_reserve(sb, sb->len + n + 1)) return false;`	Технически проверяет условие `!sb_reserve(sb, sb->len + n + 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
129	`if (n != 0) memcpy(sb->s + sb->len, s, n);`	Технически проверяет условие `n != 0) memcpy(sb->s + sb->len, s, n`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
130	`sb->len += n;`	Технически изменяет `sb->len` оператором `+=` на основе `n`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
131	`sb->s[sb->len] = 0;`	Технически присваивает `sb->s[sb->len]` значение `0`. Логически это обновляет `len]`: хранит промежуточное значение, которое дальше используется в текущей функции.
132	`return true;`	Технически возвращает `true` из функции `sb_appendn`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
133	`}`	Технически завершает область видимости внутри `sb_appendn`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
135	`bool sb_append(StrBuf* sb, const char* s) {`	Технически объявляет/начинает функцию `sb_append`; она доступная другим модулям через заголовок или линковку. Логически функция добавляет C-строку в StrBuf.
136	`if (!sb) return false;`	Технически проверяет условие `!sb`; логически это защита от NULL-указателя и работы с неинициализированными данными.
137	`if (!s) {`	Технически проверяет условие `!s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
138	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
139	`return false;`	Технически возвращает `false` из функции `sb_append`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
140	`}`	Технически завершает область видимости внутри `sb_append`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
141	`return sb_appendn(sb, s, strlen(s));`	Технически объявляет прототип функции `sb_appendn` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет ровно n байтов текста в StrBuf.
142	`}`	Технически завершает область видимости внутри `sb_append`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
144	`bool sb_appendf(StrBuf* sb, const char* fmt, ...) {`	Технически объявляет/начинает функцию `sb_appendf`; она доступная другим модулям через заголовок или линковку. Логически функция добавляет форматированный текст в StrBuf.
145	`if (!sb) return false;`	Технически проверяет условие `!sb`; логически это защита от NULL-указателя и работы с неинициализированными данными.
146	`if (sb->failed || !fmt) {`	Технически проверяет условие `sb->failed || !fmt`; логически это защита от NULL-указателя и работы с неинициализированными данными.
147	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
148	`return false;`	Технически возвращает `false` из функции `sb_appendf`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
149	`}`	Технически завершает область видимости внутри `sb_appendf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
150	`va_list ap;`	Технически выполняет оператор `va_list ap;`. Логически в функции `sb_appendf` он меняет состояние, проверяемые данные или подготовку отчета.
151	`va_start(ap, fmt);`	Технически вызывает `va_start`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_start` для выполнения части алгоритма.
152	`va_list ap2;`	Технически выполняет оператор `va_list ap2;`. Логически в функции `sb_appendf` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
153	`va_copy(ap2, ap);`	Технически вызывает `va_copy`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_copy` для выполнения части алгоритма.
155	`#if defined(__GNUC__) || defined(__clang__)`	Технически включает код только при условии `#if defined(__GNUC__) || defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
156	`#pragma GCC diagnostic push`	Технически это директива препроцессора `#pragma GCC diagnostic push`. Логически она меняет код еще до компиляции C.
157	`#pragma GCC diagnostic ignored "-Wformat-nonliteral"`	Технически это директива препроцессора `#pragma GCC diagnostic ignored "-Wformat-nonliteral"`. Логически она меняет код еще до компиляции C.
158	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
159	`int need = vsnprintf(NULL, 0, fmt, ap);`	Технически присваивает `int need` значение `vsnprintf(NULL, 0, fmt, ap)`. Логически это обновляет `int need`: хранит промежуточное значение, которое дальше используется в текущей функции.
160	`va_end(ap);`	Технически вызывает `va_end`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_end` для выполнения части алгоритма.
161	`if (need < 0) {`	Технически проверяет условие `need < 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
162	`va_end(ap2);`	Технически вызывает `va_end`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_end` для выполнения части алгоритма.
163	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
164	`return false;`	Технически возвращает `false` из функции `sb_appendf`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
165	`}`	Технически завершает область видимости внутри `sb_appendf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
167	`if (sb->len == SIZE_MAX || (size_t)need > SIZE_MAX - sb->len - 1u) {`	Технически проверяет условие `sb->len == SIZE_MAX || (size_t)need > SIZE_MAX - sb->len - 1u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
168	`va_end(ap2);`	Технически вызывает `va_end`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_end` для выполнения части алгоритма.
169	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
170	`return false;`	Технически возвращает `false` из функции `sb_appendf`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
171	`}`	Технически завершает область видимости внутри `sb_appendf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
172	`if (!sb_reserve(sb, sb->len + (size_t)need + 1u)) { va_end(ap2); return false; }`	Технически проверяет условие `!sb_reserve(sb, sb->len + (size_t)need + 1u)) { va_end(ap2;` логически это защита от NULL-указателя и работы с неинициализированными данными.
173	`int wrote = vsnprintf(sb->s + sb->len, sb->cap - sb->len, fmt, ap2);`	Технически присваивает `int wrote` значение `vsnprintf(sb->s + sb->len, sb->cap - sb->len, fmt, ap2)`. Логически это обновляет `int wrote`: хранит промежуточное значение, которое дальше используется в текущей функции.
174	`#if defined(__GNUC__) || defined(__clang__)`	Технически включает код только при условии `#if defined(__GNUC__) || defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
175	`#pragma GCC diagnostic pop`	Технически это директива препроцессора `#pragma GCC diagnostic pop`. Логически она меняет код еще до компиляции C.
176	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
177	`va_end(ap2);`	Технически вызывает `va_end`. Логически это действие нужно в функции `sb_appendf`: вызывает функцию `va_end` для выполнения части алгоритма.
178	`if (wrote != need) {`	Технически проверяет условие `wrote != need`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
179	`sb->failed = true;`	Технически присваивает `sb->failed` значение `true`. Логически это обновляет `failed`: хранит промежуточное значение, которое дальше используется в текущей функции.
180	`return false;`	Технически возвращает `false` из функции `sb_appendf`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
181	`}`	Технически завершает область видимости внутри `sb_appendf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
182	`sb->len += (size_t)wrote;`	Технически изменяет `sb->len` оператором `+=` на основе `(size_t)wrote`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.

Стр.	Код	Нормальное объяснение
183	`return true;`	Технически возвращает `true` из функции `sb_appendf`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.

Стр.	Код	Нормальное объяснение
184	`}`	Технически завершает область видимости внутри `sb_appendf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
186	`bool sb_ok(const StrBuf* sb) {`	Технически объявляет/начинает функцию `sb_ok`; она доступная другим модулям через заголовок или линковку. Логически функция проверяет, что StrBuf не перешел в состояние ошибки.
187	`return sb && !sb->failed && sb->s && sb->len < sb->cap;`	Технически возвращает выражение `sb && !sb->failed && sb->s && sb->len < sb->cap`. Логически именно это значение становится результатом функции `sb_ok` для вызывающего кода.
188	`}`	Технически завершает область видимости внутри `sb_ok`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
190	`bool file_write_atomic(const char* path, const char* data, size_t len) {`	Технически объявляет/начинает функцию `file_write_atomic`; она доступная другим модулям через заголовок или линковку. Логически функция записывает файл через временное имя, чтобы не оставлять битый отчет при сбое.
191	`if (!path || (!data && len != 0)) return false;`	Технически проверяет условие `!path || (!data && len != 0)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
192	`size_t path_len = strlen(path);`	Технически присваивает `size_t path_len` значение `strlen(path)`. Логически это обновляет `size_t path_len`: хранит размер или количество, поэтому используется в проверках границ.
193	`if (path_len > SIZE_MAX - 48u) return false;`	Технически проверяет условие `path_len > SIZE_MAX - 48u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
195	`size_t temp_size = path_len + 48u;`	Технически объявляет `temp_size` в функции `file_write_atomic` и сразу задает начальное значение `path_len + 48u`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
196	`char* temp_path = (char*)malloc(temp_size);`	Технически присваивает `char* temp_path` значение `(char*)malloc(temp_size)`. Логически это обновляет `char* temp_path`: хранит промежуточное значение, которое дальше используется в текущей функции.
197	`if (!temp_path) return false;`	Технически проверяет условие `!temp_path`; логически это защита от NULL-указателя и работы с неинициализированными данными.
198	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
199	`unsigned long process_id = (unsigned long)GetCurrentProcessId();`	Технически присваивает `unsigned long process_id` значение `(unsigned long)GetCurrentProcessId()`. Логически это обновляет `unsigned long process_id`: хранит промежуточное значение, которое дальше используется в текущей функции.
200	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
201	`unsigned long process_id = (unsigned long)getpid();`	Технически присваивает `unsigned long process_id` значение `(unsigned long)getpid()`. Логически это обновляет `unsigned long process_id`: хранит промежуточное значение, которое дальше используется в текущей функции.
202	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
203	`int written = snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id);`	Технически присваивает `int written` значение `snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id)`. Логически это обновляет `int written`: хранит промежуточное значение, которое дальше используется в текущей функции.
204	`if (written < 0 || (size_t)written >= temp_size) {`	Технически проверяет условие `written < 0 || (size_t)written >= temp_size`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
205	`free(temp_path);`	Технически вызывает `free`. Логически это действие нужно в функции `file_write_atomic`: освобождает память, чтобы в C не было утечки.
206	`return false;`	Технически возвращает `false` из функции `file_write_atomic`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
207	`}`	Технически завершает область видимости внутри `file_write_atomic`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
209	`FILE* f = fopen(temp_path, "wb");`	Технически присваивает `FILE* f` значение `fopen(temp_path, "wb")`. Логически это обновляет `FILE* f`: хранит промежуточное значение, которое дальше используется в текущей функции.
210	`if (!f) {`	Технически проверяет условие `!f`; логически это защита от NULL-указателя и работы с неинициализированными данными.
211	`free(temp_path);`	Технически вызывает `free`. Логически это действие нужно в функции `file_write_atomic`: освобождает память, чтобы в C не было утечки.
212	`return false;`	Технически возвращает `false` из функции `file_write_atomic`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
213	`}`	Технически завершает область видимости внутри `file_write_atomic`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
214	`size_t wr = len ? fwrite(data, 1, len, f) : 0u;`	Технически присваивает `size_t wr` значение `len ? fwrite(data, 1, len, f) : 0u`. Логически это обновляет `size_t wr`: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
215	`bool ok = wr == len && ferror(f) == 0 && fflush(f) == 0;`	Технически присваивает `bool ok` значение `wr == len && ferror(f) == 0 && fflush(f) == 0`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
216	`if (fclose(f) != 0) ok = false;`	Технически проверяет условие `fclose(f) != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
218	`if (ok) {`	Технически проверяет условие `ok`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
219	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
220	`ok = MoveFileExA(`	Технически присваивает `ok` значение `MoveFileExA(`. Логически это обновляет `ok`: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.

Стр.	Код	Нормальное объяснение
221	`temp_path,`	Технически эта строка является частью C-конструкции `temp_path,`. Логически она уточняет структуру блока функции `file_write_atomic` и влияет на то, какие данные будут обработаны.
222	`path,`	Технически эта строка является частью C-конструкции `path,`. Логически она уточняет структуру блока функции `file_write_atomic` и влияет на то, какие данные будут обработаны.
223	`MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;`	Технически выполняет битовую операцию `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
224	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
225	`ok = rename(temp_path, path) == 0;`	Технически присваивает `ok` значение `rename(temp_path, path) == 0`. Логически это обновляет `ok`: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
226	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
227	`}`	Технически завершает область видимости внутри `file_write_atomic`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
228	`if (!ok) (void)remove(temp_path);`	Технически проверяет условие `!ok) (void)remove(temp_path`; логически это защита от NULL-указателя и работы с неинициализированными данными.
229	`free(temp_path);`	Технически вызывает `free`. Логически это действие нужно в функции `file_write_atomic`: освобождает память, чтобы в C не было утечки.
230	`return ok;`	Технически возвращает выражение `ok`. Логически именно это значение становится результатом функции `file_write_atomic` для вызывающего кода.
231	`}`	Технически завершает область видимости внутри `file_write_atomic`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
233	`typedef struct Sha256Ctx {`	Технически начинает описание структуры `Sha256Ctx`. Логически структура собирает связанные поля в одну модель данных проекта.
234	`uint32_t state[8];`	Технически объявляет поле `state` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
235	`uint64_t bit_count;`	Технически объявляет поле `bit_count` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
236	`uint8_t block[64];`	Технически объявляет поле `block` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
237	`size_t block_len;`	Технически объявляет поле `block_len` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
238	`} Sha256Ctx;`	Технически завершает typedef и дает типу имя `Sha256Ctx`. Логически дальше код может объявлять переменные этого типа без `struct`.
240	`static uint32_t sha256_rotr(uint32_t value, unsigned count) {`	Технически объявляет/начинает функцию `sha256_rotr`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
241	`return (value >> count) | (value << (32u - count));`	Технически возвращает выражение `(value >> count) | (value << (32u - count))`. Логически именно это значение становится результатом функции `sha256_rotr` для вызывающего кода.
242	`}`	Технически завершает область видимости внутри `sha256_rotr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
244	`static void sha256_transform(Sha256Ctx* ctx, const uint8_t block[64]) {`	Технически объявляет/начинает функцию `sha256_transform`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
245	`static const uint32_t k[64] = {`	Технически присваивает `static const uint32_t k[64]` значение `{`. Логически это обновляет `static const uint32_t k[64]`: хранит промежуточное значение, которое дальше используется в текущей функции.
246	`0x428a2f98u,0x71374491u,0xb5c0fbcfu,0xe9b5dba5u,0x3956c25bu,0x59f111f1u,0x923f82a4u,0xab1c5ed5u,`	Технически эта строка является частью C-конструкции `0x428a2f98u,0x71374491u,0xb5c0fbcfu,0xe9b5dba5u,0x3956c25bu,0x59f111f1u,0x923f82a4u,0xab1c5ed5u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
247	`0xd807aa98u,0x12835b01u,0x243185beu,0x550c7dc3u,0x72be5d74u,0x80deb1feu,0x9bdc06a7u,0xc19bf174u,`	Технически эта строка является частью C-конструкции `0xd807aa98u,0x12835b01u,0x243185beu,0x550c7dc3u,0x72be5d74u,0x80deb1feu,0x9bdc06a7u,0xc19bf174u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
248	`0xe49b69c1u,0xefbe4786u,0x0fc19dc6u,0x240ca1ccu,0x2de92c6fu,0x4a7484aau,0x5cb0a9dcu,0x76f988dau,`	Технически эта строка является частью C-конструкции `0xe49b69c1u,0xefbe4786u,0x0fc19dc6u,0x240ca1ccu,0x2de92c6fu,0x4a7484aau,0x5cb0a9dcu,0x76f988dau,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
249	`0x983e5152u,0xa831c66du,0xb00327c8u,0xbf597fc7u,0xc6e00bf3u,0xd5a79147u,0x06ca6351u,0x14292967u,`	Технически эта строка является частью C-конструкции `0x983e5152u,0xa831c66du,0xb00327c8u,0xbf597fc7u,0xc6e00bf3u,0xd5a79147u,0x06ca6351u,0x14292967u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
250	`0x27b70a85u,0x2e1b2138u,0x4d2c6dfcu,0x53380d13u,0x650a7354u,0x766a0abbu,0x81c2c92eu,0x92722c85u,`	Технически эта строка является частью C-конструкции `0x27b70a85u,0x2e1b2138u,0x4d2c6dfcu,0x53380d13u,0x650a7354u,0x766a0abbu,0x81c2c92eu,0x92722c85u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
251	`0xa2bfe8a1u,0xa81a664bu,0xc24b8b70u,0xc76c51a3u,0xd192e819u,0xd6990624u,0xf40e3585u,0x106aa070u,`	Технически эта строка является частью C-конструкции `0xa2bfe8a1u,0xa81a664bu,0xc24b8b70u,0xc76c51a3u,0xd192e819u,0xd6990624u,0xf40e3585u,0x106aa070u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
252	`0x19a4c116u,0x1e376c08u,0x2748774cu,0x34b0bcb5u,0x391c0cb3u,0x4ed8aa4au,0x5b9cca4fu,0x682e6ff3u,`	Технически эта строка является частью C-конструкции `0x19a4c116u,0x1e376c08u,0x2748774cu,0x34b0bcb5u,0x391c0cb3u,0x4ed8aa4au,0x5b9cca4fu,0x682e6ff3u,`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
253	`0x748f82eeu,0x78a5636fu,0x84c87814u,0x8cc70208u,0x90befffau,0xa4506cebu,0xbef9a3f7u,0xc67178f2u`	Технически эта строка является частью C-конструкции `0x748f82eeu,0x78a5636fu,0x84c87814u,0x8cc70208u,0x90befffau,0xa4506cebu,0xbef9a3f7u,0xc67178f2u`. Логически она уточняет структуру блока функции `sha256_transform` и влияет на то, какие данные будут обработаны.
254	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
255	`uint32_t w[64];`	Технически объявляет `w` в функции `sha256_transform`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
256	`for (size_t i = 0; i < 16; i++) {`	Технически начинает цикл `size_t i = 0; i < 16; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
257	`size_t j = i * 4;`	Технически объявляет `j` в функции `sha256_transform` и сразу задает начальное значение `i * 4`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
258	`w[i] = ((uint32_t)block[j] << 24) |`	Технически присваивает `w[i]` значение `((uint32_t)block[j] << 24) |`. Логически это обновляет `w[i]`: хранит промежуточное значение, которое дальше используется в текущей функции.
259	`((uint32_t)block[j + 1] << 16) |`	Технически выполняет битовую операцию `((uint32_t)block[j + 1] << 16) |`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
260	`((uint32_t)block[j + 2] << 8) |`	Технически выполняет битовую операцию `((uint32_t)block[j + 2] << 8) |`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
261	`(uint32_t)block[j + 3];`	Технически выполняет оператор `(uint32_t)block[j + 3];`. Логически в функции `sha256_transform` он меняет состояние, проверяемые данные или подготовку отчета.
262	`}`	Технически завершает область видимости внутри `sha256_transform`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
263	`for (size_t i = 16; i < 64; i++) {`	Технически начинает цикл `size_t i = 16; i < 64; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
264	`uint32_t s0 = sha256_rotr(w[i - 15], 7) ^ sha256_rotr(w[i - 15], 18) ^ (w[i - 15] >> 3);`	Технически присваивает `uint32_t s0` значение `sha256_rotr(w[i - 15], 7) ^ sha256_rotr(w[i - 15], 18) ^ (w[i - 15] >> 3)`. Логически это обновляет `uint32_t s0`: хранит промежуточное значение, которое дальше используется в текущей функции.
265	`uint32_t s1 = sha256_rotr(w[i - 2], 17) ^ sha256_rotr(w[i - 2], 19) ^ (w[i - 2] >> 10);`	Технически присваивает `uint32_t s1` значение `sha256_rotr(w[i - 2], 17) ^ sha256_rotr(w[i - 2], 19) ^ (w[i - 2] >> 10)`. Логически это обновляет `uint32_t s1`: хранит промежуточное значение, которое дальше используется в текущей функции.
266	`w[i] = w[i - 16] + s0 + w[i - 7] + s1;`	Технически присваивает `w[i]` значение `w[i - 16] + s0 + w[i - 7] + s1`. Логически это обновляет `w[i]`: хранит промежуточное значение, которое дальше используется в текущей функции.
267	`}`	Технически завершает область видимости внутри `sha256_transform`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
269	`uint32_t a = ctx->state[0], b = ctx->state[1], c = ctx->state[2], d = ctx->state[3];`	Технически объявляет `a` в функции `sha256_transform` и сразу задает начальное значение `ctx->state[0], b = ctx->state[1], c = ctx->state[2], d = ctx->state[3]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
270	`uint32_t e = ctx->state[4], f = ctx->state[5], g = ctx->state[6], h = ctx->state[7];`	Технически объявляет e в функции sha256_transform и сразу задает начальное значение ctx->state[4], f = ctx->state[5], g = ctx->state[6], h = ctx->state[7]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
271	`for (size_t i = 0; i < 64; i++) {`	Технически начинает цикл size_t i = 0; i < 64; i++. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
272	`uint32_t s1 = sha256_rotr(e, 6) ^ sha256_rotr(e, 11) ^ sha256_rotr(e, 25);`	Технически присваивает uint32_t s1 значение sha256_rotr(e, 6) ^ sha256_rotr(e, 11) ^ sha256_rotr(e, 25). Логически это обновляет uint32_t s1: хранит промежуточное значение, которое дальше используется в текущей функции.
273	`uint32_t ch = (e & f) ^ ((~e) & g);`	Технически присваивает uint32_t ch значение (e & f) ^ ((~e) & g). Логически это обновляет uint32_t ch: хранит промежуточное значение, которое дальше используется в текущей функции.
274	`uint32_t t1 = h + s1 + ch + k[i] + w[i];`	Технически объявляет t1 в функции sha256_transform и сразу задает начальное значение h + s1 + ch + k[i] + w[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
275	`uint32_t s0 = sha256_rotr(a, 2) ^ sha256_rotr(a, 13) ^ sha256_rotr(a, 22);`	Технически присваивает uint32_t s0 значение sha256_rotr(a, 2) ^ sha256_rotr(a, 13) ^ sha256_rotr(a, 22). Логически это обновляет uint32_t s0: хранит промежуточное значение, которое дальше используется в текущей функции.
276	`uint32_t maj = (a & b) ^ (a & c) ^ (b & c);`	Технически присваивает uint32_t maj значение (a & b) ^ (a & c) ^ (b & c). Логически это обновляет uint32_t maj: хранит промежуточное значение, которое дальше используется в текущей функции.
277	`uint32_t t2 = s0 + maj;`	Технически объявляет t2 в функции sha256_transform и сразу задает начальное значение s0 + maj. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
278	`h = g; g = f; f = e; e = d + t1;`	Технически присваивает h значение g; g = f; f = e; e = d + t1. Логически это обновляет h: хранит промежуточное значение, которое дальше используется в текущей функции.
279	`d = c; c = b; b = a; a = t1 + t2;`	Технически присваивает d значение c; c = b; b = a; a = t1 + t2. Логически это обновляет d: хранит промежуточное значение, которое дальше используется в текущей функции.
280	`}`	Технически завершает область видимости внутри sha256_transform. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
281	`ctx->state[0] += a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d;`	Технически изменяет ctx->state[0] оператором += на основе a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
282	`ctx->state[4] += e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h;`	Технически изменяет ctx->state[4] оператором += на основе e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
283	`}`	Технически завершает область видимости внутри sha256_transform. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
285	`static void sha256_init(Sha256Ctx* ctx) {`	Технически объявляет/начинает функцию sha256_init; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
286	`static const uint32_t initial[8] = {`	Технически присваивает static const uint32_t initial[8] значение {. Логически это обновляет static const uint32_t initial[8]: хранит промежуточное значение, которое дальше используется в текущей функции.
287	`0x6a09e667u,0xbb67ae85u,0x3c6ef372u,0xa54ff53au,`	Технически эта строка является частью C-конструкции 0x6a09e667u,0xbb67ae85u,0x3c6ef372u,0xa54ff53au,. Логически она уточняет структуру блока функции sha256_init и влияет на то, какие данные будут обработаны.
288	`0x510e527fu,0x9b05688cu,0x1f83d9abu,0x5be0cd19u`	Технически эта строка является частью C-конструкции 0x510e527fu,0x9b05688cu,0x1f83d9abu,0x5be0cd19u. Логически она уточняет структуру блока функции sha256_init и влияет на то, какие данные будут обработаны.
289	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
290	`memcpy(ctx->state, initial, sizeof(initial));`	Технически вызывает memcpy. Логически это действие нужно в функции sha256_init: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
291	`ctx->bit_count = 0;`	Технически присваивает ctx->bit_count значение 0. Логически это обновляет bit_count: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
292	`ctx->block_len = 0;`	Технически присваивает ctx->block_len значение 0. Логически это обновляет block_len: хранит размер или количество, поэтому используется в проверках границ.
293	`}`	Технически завершает область видимости внутри sha256_init. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
295	`static void sha256_update(Sha256Ctx* ctx, const uint8_t* data, size_t size) {`	Технически объявляет/начинает функцию sha256_update; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
296	`if (size == 0) return;`	Технически проверяет условие size == 0; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
297	`ctx->bit_count += (uint64_t)size * 8u;`	Технически изменяет `ctx->bit_count` оператором += на основе `(uint64_t)size * 8u`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
298	`while (size > 0) {`	Технически начинает цикл while с условием `size > 0`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
299	`size_t available = 64u - ctx->block_len;`	Технически объявляет `available` в функции `sha256_update` и сразу задает начальное значение `64u - ctx->block_len`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
300	`size_t take = size < available ? size : available;`	Технически объявляет `take` в функции `sha256_update` и сразу задает начальное значение `size < available ? size : available`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
301	`memcpy(ctx->block + ctx->block_len, data, take);`	Технически вызывает `memcpy`. Логически это действие нужно в функции `sha256_update`: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
302	`ctx->block_len += take;`	Технически изменяет `ctx->block_len` оператором += на основе `take`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
303	`data += take;`	Технически изменяет `data` оператором += на основе `take`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
304	`size -= take;`	Технически изменяет `size` оператором -= на основе `take`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
305	`if (ctx->block_len == 64u) {`	Технически проверяет условие `ctx->block_len == 64u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
306	`sha256_transform(ctx, ctx->block);`	Технически вызывает `sha256_transform`. Логически это действие нужно в функции `sha256_update`: вызывает функцию `sha256_transform` для выполнения части алгоритма.
307	`ctx->block_len = 0;`	Технически присваивает `ctx->block_len` значение `0`. Логически это обновляет `block_len`: хранит размер или количество, поэтому используется в проверках границ.
308	`}`	Технически завершает область видимости внутри `sha256_update`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
309	`}`	Технически завершает область видимости внутри `sha256_update`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
310	`}`	Технически завершает область видимости внутри `sha256_update`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
312	`static void sha256_final(Sha256Ctx* ctx, uint8_t digest[32]) {`	Технически объявляет/начинает функцию `sha256_final`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
313	`ctx->block[ctx->block_len++] = 0x80;`	Технически присваивает `ctx->block[ctx->block_len++]` значение `0x80`. Логически это обновляет `block_len++]`: хранит промежуточное значение, которое дальше используется в текущей функции.
314	`if (ctx->block_len > 56u) {`	Технически проверяет условие `ctx->block_len > 56u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
315	`while (ctx->block_len < 64u) ctx->block[ctx->block_len++] = 0;`	Технически начинает цикл while с условием `ctx->block_len < 64u`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
316	`sha256_transform(ctx, ctx->block);`	Технически вызывает `sha256_transform`. Логически это действие нужно в функции `sha256_final`: вызывает функцию `sha256_transform` для выполнения части алгоритма.
317	`ctx->block_len = 0;`	Технически присваивает `ctx->block_len` значение `0`. Логически это обновляет `block_len`: хранит размер или количество, поэтому используется в проверках границ.
318	`}`	Технически завершает область видимости внутри `sha256_final`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
319	`while (ctx->block_len < 56u) ctx->block[ctx->block_len++] = 0;`	Технически начинает цикл while с условием `ctx->block_len < 56u`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
320	`for (int i = 7; i >= 0; i--) {`	Технически начинает цикл `int i = 7; i >= 0; i--`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
321	`ctx->block[ctx->block_len++] = (uint8_t)(ctx->bit_count >> (unsigned)(i * 8));`	Технически присваивает `ctx->block[ctx->block_len++]` значение `(uint8_t)(ctx->bit_count >> (unsigned)(i * 8))`. Логически это обновляет `block_len++]`: хранит промежуточное значение, которое дальше используется в текущей функции.
322	`}`	Технически завершает область видимости внутри `sha256_final`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
323	`sha256_transform(ctx, ctx->block);`	Технически вызывает `sha256_transform`. Логически это действие нужно в функции `sha256_final`: вызывает функцию `sha256_transform` для выполнения части алгоритма.
325	`for (size_t i = 0; i < 8; i++) {`	Технически начинает цикл `size_t i = 0; i < 8; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
326	`digest[i * 4] = (uint8_t)(ctx->state[i] >> 24);`	Технически присваивает `digest[i * 4]` значение `(uint8_t)(ctx->state[i] >> 24)`. Логически это обновляет `digest[i * 4]`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
327	`digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);`	Технически присваивает `digest[i * 4 + 1]` значение `(uint8_t)(ctx->state[i] >> 16)`. Логически это обновляет `digest[i * 4 + 1]`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
328	`digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);`	Технически присваивает `digest[i * 4 + 2]` значение `(uint8_t)(ctx->state[i] >> 8)`. Логически это обновляет `digest[i * 4 + 2]`: хранит промежуточное значение, которое дальше используется в текущей функции.
329	`digest[i * 4 + 3] = (uint8_t)ctx->state[i];`	Технически присваивает `digest[i * 4 + 3]` значение `(uint8_t)ctx->state[i]`. Логически это обновляет `digest[i * 4 + 3]`: хранит промежуточное значение, которое дальше используется в текущей функции.
330	`}`	Технически завершает область видимости внутри `sha256_final`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
331	`}`	Технически завершает область видимости внутри `sha256_final`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
333	`bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]) {`	Технически объявляет/начинает функцию `sha256_hex`; она доступная другим модулям через заголовок или линковку. Логически функция считает SHA-256 входного бинарника для воспроизводимости отчета.
334	`static const char digits[] = "0123456789abcdef";`	Технически объявляет `digits` в функции `sha256_hex` и сразу задает начальное значение `"0123456789abcdef"`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
335	`if (!out_hex || (size > 0 && !data)) return false;`	Технически проверяет условие `!out_hex || (size > 0 && !data)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
336	`if ((uint64_t)size > UINT64_MAX / 8u) return false;`	Технически проверяет условие `(uint64_t)size > UINT64_MAX / 8u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
337	`Sha256Ctx ctx;`	Технически выполняет оператор `Sha256Ctx ctx;`. Логически в функции `sha256_hex` он меняет состояние, проверяемые данные или подготовку отчета.
338	`uint8_t digest[32];`	Технически объявляет `digest` в функции `sha256_hex`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
339	`sha256_init(&ctx);`	Технически вызывает `sha256_init`. Логически это действие нужно в функции `sha256_hex`: вызывает функцию `sha256_init` для выполнения части алгоритма.
340	`sha256_update(&ctx, data, size);`	Технически вызывает `sha256_update`. Логически это действие нужно в функции `sha256_hex`: вызывает функцию `sha256_update` для выполнения части алгоритма.
341	`sha256_final(&ctx, digest);`	Технически вызывает `sha256_final`. Логически это действие нужно в функции `sha256_hex`: вызывает функцию `sha256_final` для выполнения части алгоритма.
342	`for (size_t i = 0; i < sizeof(digest); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(digest); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
343	`out_hex[i * 2] = digits[digest[i] >> 4];`	Технически присваивает `out_hex[i * 2]` значение `digits[digest[i] >> 4]`. Логически это обновляет `out_hex[i * 2]`: является выходным параметром, куда функция записывает результат.
344	`out_hex[i * 2 + 1] = digits[digest[i] & 0x0fu];`	Технически присваивает `out_hex[i * 2 + 1]` значение `digits[digest[i] & 0x0fu]`. Логически это обновляет `out_hex[i * 2 + 1]`: является выходным параметром, куда функция записывает результат.
345	`}`	Технически завершает область видимости внутри `sha256_hex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
346	`out_hex[64] = '\0';`	Технически присваивает `out_hex[64]` значение `'\0'`. Логически это обновляет `out_hex[64]`: является выходным параметром, куда функция записывает результат.
347	`return true;`	Технически возвращает `true` из функции `sha256_hex`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
348	`}`	Технически завершает область видимости внутри `sha256_hex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 317. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

4. Разбор PE-образа

Здесь проект превращает массив байтов в модель PE-файла: секции, размеры, выравнивание, ImageBase и адрес каталога исключений.

Что сначала выучить в C и теории

- структуры как модель бинарного формата
- little-endian поля PE
- RVA, file offset, SizeOfImage, SizeOfHeaders
- проверка согласованности section table и exception directory

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/pe.h

Назначение файла: структуры и функции PE-парсера.

Код, который пишем на этом шаге

```
#pragma once
/*
  pe.h - minimal PE32+/x64 parser (no Windows headers)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

#define PE_DOS_MAGIC 0x5A4D      /* 'MZ' */
#define PE_NT_MAGIC  0x00004550  /* 'PE\0\0' */

#define IMAGE_FILE_MACHINE_AMD64 0x8664
#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b

#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3

typedef struct PeSection {
    char     name[9]; /* null-terminated */
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;

typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections; /* owned */

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
void pe_free(PeImage* img);

/* Map RVA -> file offset. Returns false if not mappable. */
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);

/* Find section containing RVA (or NULL). */
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`pe.h - minimal PE32+/x64 parser (no Windows headers)`	Технически объявляет/начинает функцию `parser`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
4	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
6	`#include <stdint.h>`	Технически подключает `stdint.h`. Логически это дает модулю доступ к: точные типы uint8_t/uint32_t/uint64_t, без которых нельзя надежно читать бинарный формат.
7	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
8	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
9	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
11	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
12	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
13	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
15	`#define PE_DOS_MAGIC 0x5A4D /* 'MZ' */`	Технически объявляет макрос `PE_DOS_MAGIC` со значением `0x5A4D /* 'MZ' */`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
16	`#define PE_NT_MAGIC 0x00004550 /* 'PE\0\0' */`	Технически объявляет макрос `PE_NT_MAGIC` со значением `0x00004550 /* 'PE\0\0' */`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
18	`#define IMAGE_FILE_MACHINE_AMD64 0x8664`	Технически объявляет макрос `IMAGE_FILE_MACHINE_AMD64` со значением `0x8664`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
19	`#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b`	Технически объявляет макрос `IMAGE_NT_OPTIONAL_HDR64_MAGIC` со значением `0x20b`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
21	`#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16`	Технически объявляет макрос `IMAGE_NUMBEROF_DIRECTORY_ENTRIES` со значением `16`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
22	`#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3`	Технически объявляет макрос `IMAGE_DIRECTORY_ENTRY_EXCEPTION` со значением `3`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
24	`typedef struct PeSection {`	Технически начинает описание структуры `PeSection`. Логически структура собирает связанные поля в одну модель данных проекта.
25	`char name[9]; /* null-terminated */`	Технически объявляет поле `name` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
26	`uint32_t virtual_address;`	Технически объявляет поле `virtual_address` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
27	`uint32_t virtual_size;`	Технически объявляет поле `virtual_size` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
28	`uint32_t raw_ptr;`	Технически объявляет поле `raw_ptr` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
29	`uint32_t raw_size;`	Технически объявляет поле `raw_size` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
30	`uint32_t characteristics;`	Технически объявляет поле `characteristics` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
31	`} PeSection;`	Технически завершает typedef и дает типу имя `PeSection`. Логически дальше код может объявлять переменные этого типа без `struct`.
33	`typedef struct PeImage {`	Технически начинает описание структуры `PeImage`. Логически структура собирает связанные поля в одну модель данных проекта.
34	`bool ok;`	Технически объявляет поле ok внутри структуры. Логически это поле: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
35	`bool is_pe64;`	Технически объявляет поле `is_pe64` внутри структуры. Логически это поле: признак PE32+ x64, потому что Windows x64 unwind работает именно с этой моделью.
36	`uint16_t machine;`	Технически объявляет поле `machine` внутри структуры. Логически это поле: тип архитектуры из COFF header; нужен, чтобы отсеять не-x64 бинарники.
37	`uint64_t image_base;`	Технически объявляет поле `image_base` внутри структуры. Логически это поле: базовый адрес образа, относительно которого Windows строит виртуальные адреса.
38	`uint32_t size_of_image;`	Технически объявляет поле `size_of_image` внутри структуры. Логически это поле: верхняя виртуальная граница PE-образа; выход за нее считается ошибкой координат.
39	`uint32_t size_of_headers;`	Технически объявляет поле `size_of_headers` внутри структуры. Логически это поле: размер заголовков на диске; нужен для проверки секций и raw offsets.
40	`uint32_t section_alignment;`	Технически объявляет поле `section_alignment` внутри структуры. Логически это поле: выравнивание секций в памяти, установленное PE optional header.

Стр.	Код	Нормальное объяснение
41	`uint32_t file_alignment;`	Технически объявляет поле `file_alignment` внутри структуры. Логически это поле: выравнивание секций на диске, установленное PE optional header.
42	`uint32_t exception_rva;`	Технически объявляет поле `exception_rva` внутри структуры. Логически это поле: RVA каталога исключений, то есть начало таблицы .pdata.
43	`uint32_t exception_size;`	Технически объявляет поле `exception_size` внутри структуры. Логически это поле: размер exception directory, по которому считается число RUNTIME_FUNCTION.
44	`size_t file_size;`	Технически объявляет поле `file_size` внутри структуры. Логически это поле: размер входного файла; главная граница для raw-чтения.
45	`size_t num_sections;`	Технически объявляет поле `num_sections` внутри структуры. Логически это поле: количество записей section table.
46	`PeSection* sections; /* owned */`	Технически объявляет поле `sections` внутри структуры. Логически это поле: массив секций, которым владеет PeImage.
48	`size_t section_raw_oob;`	Технически объявляет поле `section_raw_oob` внутри структуры. Логически это поле: счетчик секций, raw-диапазон которых выходит за размер файла.
49	`size_t section_virtual_oob;`	Технически объявляет поле `section_virtual_oob` внутри структуры. Логически это поле: счетчик секций, виртуальный диапазон которых выходит за SizeOfImage.
50	`size_t section_raw_overlaps;`	Технически объявляет поле `section_raw_overlaps` внутри структуры. Логически это поле: счетчик пересечений секций на диске.
51	`size_t section_virtual_overlaps;`	Технически объявляет поле `section_virtual_overlaps` внутри структуры. Логически это поле: счетчик пересечений секций в памяти.
52	`size_t section_misaligned;`	Технически объявляет поле `section_misaligned` внутри структуры. Логически это поле: счетчик нарушений выравнивания и базовых инвариантов section table.
53	`bool exception_directory_oob;`	Технически объявляет `exception_directory_oob` в функции `глобальной области файла`. Логически это место для данных: флаг битого или нечитаемого exception directory.
54	`} PeImage;`	Технически завершает typedef и дает типу имя `PeImage`. Логически дальше код может объявлять переменные этого типа без `struct`.
56	`bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);`	Технически объявляет прототип функции `pe_parse` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и читает PE-заголовки, секции и exception directory из входного файла.
57	`void pe_free(PeImage* img);`	Технически объявляет прототип функции `pe_free` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и освобождает память, выделенную PE-парсером.
59	`/* Map RVA -> file offset. Returns false if not mappable. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Map RVA -> file offset. Returns false if not mappable..
60	`bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);`	Технически объявляет прототип функции `pe_rva_to_fileoff` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и переводит RVA из виртуальной координатной системы PE в смещение файла на диске.
62	`/* Find section containing RVA (or NULL). */`	Технически это комментарий исходника. Логически он объясняет разработчику: Find section containing RVA (or NULL)..
63	`const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);`	Технически объявляет прототип функции `pe_find_section_for_rva` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и находит секцию, внутри которой лежит заданный RVA.
65	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
66	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
67	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 55. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/pe.c

Назначение файла: разбор DOS/NT headers, секций, data directory и преобразование RVA в file offset.

Код, который пишем на этом шаге

```
 1 | #include "pe.h"
 2 | #include <stdlib.h>
 3 | #include <string.h>
 4 |
 5 | static uint32_t u32_max(uint32_t a, uint32_t b) { return a > b ? a : b; }
 6 |
 7 | static bool is_power_of_two_u32(uint32_t value) {
 8 |     return value != 0 && (value & (value - 1u)) == 0;
 9 | }
10 |
11 | static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {
12 |     uint16_t mz = 0;
13 |     if (!buf_read_u16(b, 0, &mz)) return false;
14 |     if (mz != PE_DOS_MAGIC) return false;
15 |     uint32_t lfanew = 0;
16 |     if (!buf_read_u32(b, 0x3c, &lfanew)) return false;
17 |     *out_lfanew = lfanew;
18 |     return true;
19 | }
```

```
 20 |
 21 | static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
 22 |     uint64_t t = (uint64_t)a + (uint64_t)b;
 23 |     if (t > 0xffffffffu) return false;
 24 |     *out = (uint32_t)t;
 25 |     return true;
 26 | }
 27 |
 28 | static bool ranges_overlap_u64(uint64_t a_begin,
 29 |                                uint64_t a_size,
 30 |                                uint64_t b_begin,
 31 |                                uint64_t b_size) {
 32 |     if (a_size == 0 || b_size == 0) return false;
 33 |     return a_begin < b_begin + b_size && b_begin < a_begin + a_size;
 34 | }
 35 |
 36 | bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
 37 |     if (!out || !b || !err) return false;
 38 |     memset(out, 0, sizeof(*out));
 39 |
 40 |     uint32_t lfanew = 0;
 41 |     if (!read_dos_lfanew(b, &lfanew)) {
 42 |         sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
 43 |         return false;
 44 |     }
 45 |     if ((size_t)lfanew + 4 + 20 > b->size) {
 46 |         sb_append(err, "PE header offset out of range.\n");
 47 |         return false;
 48 |     }
 49 |
 50 |     uint32_t sig = 0;
 51 |     if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
 52 |         sb_append(err, "Invalid PE signature.\n");
 53 |         return false;
 54 |     }
 55 |
 56 |     size_t filehdr = (size_t)lfanew + 4;
 57 |     uint16_t machine = 0;
 58 |     uint16_t nsects = 0;
 59 |     uint16_t opt_size = 0;
 60 |     if (!buf_read_u16(b, filehdr + 0, &machine) ||
 61 |         !buf_read_u16(b, filehdr + 2, &nsects) ||
 62 |         !buf_read_u16(b, filehdr + 16, &opt_size)) {
 63 |         sb_append(err, "Failed to read COFF header.\n");
 64 |         return false;
 65 |     }
 66 |
 67 |     if (machine != IMAGE_FILE_MACHINE_AMD64) {
 68 |         sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
 69 |         return false;
 70 |     }
 71 |
 72 |     size_t opthdr = filehdr + 20;
 73 |     if (opthdr + opt_size > b->size) {
 74 |         sb_append(err, "Optional header out of range.\n");
 75 |         return false;
 76 |     }
 77 |     if (opt_size < 0x70u) {
 78 |         sb_append(err, "PE32+ optional header is too small for required fields.\n");
 79 |         return false;
 80 |     }
 81 |
 82 |     uint16_t magic = 0;
 83 |     if (!buf_read_u16(b, opthdr + 0, &magic)) {
 84 |         sb_append(err, "Failed to read optional header magic.\n");
 85 |         return false;
 86 |     }
 87 |     if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
 88 |         sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic);
 89 |         return false;
 90 |     }
 91 |
 92 |     uint64_t image_base = 0;
 93 |     uint32_t size_of_image = 0;
 94 |     uint32_t size_of_headers = 0;
 95 |     uint32_t section_alignment = 0;
 96 |     uint32_t file_alignment = 0;
 97 |     uint32_t number_of_rva_and_sizes = 0;
 98 |
 99 |     /* PE32+ offsets */
100 |     if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
101 |         !buf_read_u32(b, opthdr + 0x20, &section_alignment) ||
102 |         !buf_read_u32(b, opthdr + 0x24, &file_alignment) ||
103 |         !buf_read_u32(b, opthdr + 0x38, &size_of_image) ||
104 |         !buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||
105 |         !buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {
106 |         sb_append(err, "Failed to read key fields from optional header.\n");
107 |         return false;
108 |     }
109 |     if (size_of_image == 0) {
110 |         sb_append(err, "SizeOfImage is zero.\n");
111 |         return false;
112 |     }
113 |     if (size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size) {
114 |         sb_append(err, "SizeOfHeaders is zero or exceeds the file size.\n");
115 |         return false;
116 |     }
117 |     if (section_alignment == 0 || file_alignment == 0) {
118 |         sb_append(err, "SectionAlignment/FileAlignment must be non-zero.\n");
119 |         return false;
120 |     }
121 |     if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {
122 |         sb_append(err, "DataDirectory too small.\n");
123 |         return false;
124 |     }
125 |
126 |     /* DataDirectory starts at 0x70 */
127 |     size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;
128 |     if (dd_rel + 8u > opt_size) {
129 |         sb_append(err, "Exception data directory lies outside the optional header.\n");
130 |         return false;
131 |     }
132 |     size_t dd = opthdr + dd_rel;
133 |     uint32_t exc_rva = 0, exc_sz = 0;
134 |     if (!buf_read_u32(b, dd + 0, &exc_rva) || !buf_read_u32(b, dd + 4, &exc_sz)) {
```

```
        sb_append(err, "Failed to read exception directory.\n");
        return false;
    }

    size_t secthdr = opthdr + opt_size;
    if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) {
        sb_append(err, "Section header size overflow.\n");
        return false;
    }
    size_t needed = secthdr + (size_t)nsects * 40u;
    if (needed > b->size) {
        sb_append(err, "Section headers out of range.\n");
        return false;
    }

    PeSection* secs = (PeSection*)calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection));
    if (!secs) {
        sb_append(err, "Out of memory.\n");
        return false;
    }

    for (uint16_t i = 0; i < nsects; i++) {
        size_t sh = secthdr + (size_t)i * 40;
        uint8_t name8[8];
        if (!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section name.\n"); return false; }
        memcpy(secs[i].name, name8, 8);
        secs[i].name[8] = 0;

        if (!buf_read_u32(b, sh + 8, &secs[i].virtual_size) ||
            !buf_read_u32(b, sh + 12, &secs[i].virtual_address) ||
            !buf_read_u32(b, sh + 16, &secs[i].raw_size) ||
            !buf_read_u32(b, sh + 20, &secs[i].raw_ptr) ||
            !buf_read_u32(b, sh + 36, &secs[i].characteristics)) {
            free(secs);
            sb_append(err, "Failed to read section header.\n");
            return false;
        }
    }

    size_t raw_oob = 0;
    size_t virtual_oob = 0;
    size_t raw_overlaps = 0;
    size_t virtual_overlaps = 0;
    size_t misaligned = 0;

    bool compact_alignment =
        section_alignment < 0x1000u && file_alignment == section_alignment;
    bool file_alignment_valid =
        is_power_of_two_u32(file_alignment) &&
        ((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||
         compact_alignment);
    if (!file_alignment_valid) misaligned++;
    if (section_alignment < file_alignment) misaligned++;
    if (section_alignment < 0x1000u && file_alignment != section_alignment) {
        misaligned++;
    }
    if ((size_of_image % section_alignment) != 0) misaligned++;
    if ((size_of_headers % file_alignment) != 0 ||
        (uint64_t)size_of_headers < (uint64_t)needed) {
        misaligned++;
    }
    if ((image_base & 0xffffu) != 0) misaligned++;

    for (size_t i = 0; i < (size_t)nsects; i++) {
        const PeSection* s = &secs[i];
        uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;
        uint64_t virtual_span = (uint64_t)u32_max(s->virtual_size, s->raw_size);
        uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;
        if (s->raw_size > 0 && raw_end > (uint64_t)b->size) raw_oob++;
        if (virtual_span > 0 && virtual_end > (uint64_t)size_of_image) virtual_oob++;
        if ((s->virtual_address % section_alignment) != 0 ||
            (s->raw_size > 0 &&
             ((s->raw_ptr % file_alignment) != 0 ||
              (s->raw_size % file_alignment) != 0))) {
            misaligned++;
        }
        if (i > 0 && s->virtual_address <= secs[i - 1].virtual_address) {
            misaligned++;
        }

        for (size_t j = 0; j < i; j++) {
            const PeSection* p = &secs[j];
            if (ranges_overlap_u64(s->raw_ptr, s->raw_size, p->raw_ptr, p->raw_size)) {
                raw_overlaps++;
            }
            if (ranges_overlap_u64(
                    s->virtual_address,
                    u32_max(s->virtual_size, s->raw_size),
                    p->virtual_address,
                    u32_max(p->virtual_size, p->raw_size))) {
                virtual_overlaps++;
            }
        }
    }

    bool exception_directory_oob = false;
    if (exc_rva != 0 || exc_sz != 0) {
        uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;
        exception_directory_oob =
            exc_rva == 0 || exc_sz == 0 || exc_end > (uint64_t)size_of_image;
    }

    out->ok = true;
    out->is_pe64 = true;
    out->machine = machine;
    out->image_base = image_base;
    out->size_of_image = size_of_image;
    out->size_of_headers = size_of_headers;
    out->section_alignment = section_alignment;
    out->file_alignment = file_alignment;
    out->exception_rva = exc_rva;
    out->exception_size = exc_sz;
    out->file_size = b->size;
    out->num_sections = nsects;
    out->sections = secs;
```

```
250 |     out->section_raw_oob = raw_oob;
251 |     out->section_virtual_oob = virtual_oob;
252 |     out->section_raw_overlaps = raw_overlaps;
253 |     out->section_virtual_overlaps = virtual_overlaps;
254 |     out->section_misaligned = misaligned;
255 |     out->exception_directory_oob = exception_directory_oob;
256 |     return true;
257 | }
258 |
259 | void pe_free(PeImage* img) {
260 |     if (!img) return;
261 |     free(img->sections);
262 |     memset(img, 0, sizeof(*img));
263 | }
264 |
265 | const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva) {
266 |     if (!img || !img->sections) return NULL;
267 |     for (size_t i = 0; i < img->num_sections; i++) {
268 |         const PeSection* s = &img->sections[i];
269 |         uint32_t size = u32_max(s->virtual_size, s->raw_size);
270 |         uint64_t start = (uint64_t)s->virtual_address;
271 |         uint64_t end = start + (uint64_t)size;
272 |         if ((uint64_t)rva >= start && (uint64_t)rva < end) return s;
273 |     }
274 |     return NULL;
275 | }
276 |
277 | bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {
278 |     if (!img || !out_off) return false;
279 |     const PeSection* s = pe_find_section_for_rva(img, rva);
280 |     if (s) {
281 |         uint32_t delta = rva - s->virtual_address;
282 |         if (delta >= s->raw_size) {
283 |             /* RVA points into virtual tail (zero-fill) - not present in file */
284 |             return false;
285 |         }
286 |         uint32_t off = 0;
287 |         if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
288 |         if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;
289 |         *out_off = off;
290 |         return true;
291 |     }
292 |
293 |     /* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */
294 |     if (img->size_of_headers > 0 &&
295 |         rva < img->size_of_headers &&
296 |         (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {
297 |         *out_off = rva;
298 |         return true;
299 |     }
300 |     return false;
301 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.
2	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
3	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
5	`static uint32_t u32_max(uint32_t a, uint32_t b) { return a > b ? a : b; }`	Технически вызывает `u32_max`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `u32_max` для выполнения части алгоритма.
7	`static bool is_power_of_two_u32(uint32_t value) {`	Технически объявляет/начинает функцию `is_power_of_two_u32`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
8	`return value != 0 && (value & (value - 1u)) == 0;`	Технически возвращает выражение `value != 0 && (value & (value - 1u)) == 0`. Логически именно это значение становится результатом функции `is_power_of_two_u32` для вызывающего кода.
9	`}`	Технически завершает область видимости внутри `is_power_of_two_u32`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
11	`static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {`	Технически объявляет/начинает функцию `read_dos_lfanew`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
12	`uint16_t mz = 0;`	Технически объявляет `mz` в функции `read_dos_lfanew` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
13	`if (!buf_read_u16(b, 0, &mz)) return false;`	Технически проверяет условие `!buf_read_u16(b, 0, &mz)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
14	`if (mz != PE_DOS_MAGIC) return false;`	Технически проверяет условие `mz != PE_DOS_MAGIC`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
15	`uint32_t lfanew = 0;`	Технически объявляет `lfanew` в функции `read_dos_lfanew` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
16	`if (!buf_read_u32(b, 0x3c, &lfanew)) return false;`	Технически проверяет условие `!buf_read_u32(b, 0x3c, &lfanew)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
17	`*out_lfanew = lfanew;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_lfanew = lfanew;.
18	`return true;`	Технически возвращает `true` из функции `read_dos_lfanew`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.

Стр.	Код	Нормальное объяснение
19	`}`	Технически завершает область видимости внутри `read_dos_lfanew`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
21	`static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {`	Технически объявляет/начинает функцию `safe_add_u32`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
22	`uint64_t t = (uint64_t)a + (uint64_t)b;`	Технически присваивает `uint64_t t` значение `(uint64_t)a + (uint64_t)b`. Логически это обновляет `uint64_t t`: хранит промежуточное значение, которое дальше используется в текущей функции.
23	`if (t > 0xffffffffu) return false;`	Технически проверяет условие `t > 0xffffffffu`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
24	`*out = (uint32_t)t;`	Технически это комментарий исходника. Логически он объясняет разработчику: out = (uint32_t)t;.
25	`return true;`	Технически возвращает `true` из функции `safe_add_u32`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
26	`}`	Технически завершает область видимости внутри `safe_add_u32`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
28	`static bool ranges_overlap_u64(uint64_t a_begin,`	Технически вызывает `ranges_overlap_u64`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `ranges_overlap_u64` для выполнения части алгоритма.
29	`uint64_t a_size,`	Технически эта строка является частью C-конструкции `uint64_t a_size,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
30	`uint64_t b_begin,`	Технически эта строка является частью C-конструкции `uint64_t b_begin,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
31	`uint64_t b_size) {`	Технически эта строка является частью C-конструкции `uint64_t b_size) {`. Логически она уточняет структуру блока функции `ranges_overlap_u64` и влияет на то, какие данные будут обработаны.
32	`if (a_size == 0 || b_size == 0) return false;`	Технически проверяет условие `a_size == 0 || b_size == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
33	`return a_begin < b_begin + b_size && b_begin < a_begin + a_size;`	Технически возвращает выражение `a_begin < b_begin + b_size && b_begin < a_begin + a_size`. Логически именно это значение становится результатом функции `ranges_overlap_u64` для вызывающего кода.
34	`}`	Технически завершает область видимости внутри `ranges_overlap_u64`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
36	`bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {`	Технически объявляет/начинает функцию `pe_parse`; она доступная другим модулям через заголовок или линковку. Логически функция читает PE-заголовки, секции и exception directory из входного файла.
37	`if (!out || !b || !err) return false;`	Технически проверяет условие `!out || !b || !err`; логически это защита от NULL-указателя и работы с неинициализированными данными.
38	`memset(out, 0, sizeof(*out));`	Технически вызывает `memset`. Логически это действие нужно в функции `pe_parse`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
40	`uint32_t lfanew = 0;`	Технически объявляет `lfanew` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
41	`if (!read_dos_lfanew(b, &lfanew)) {`	Технически проверяет условие `!read_dos_lfanew(b, &lfanew)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
42	`sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
43	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
44	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
45	`if ((size_t)lfanew + 4 + 20 > b->size) {`	Технически проверяет условие `(size_t)lfanew + 4 + 20 > b->size`; логически это защита от чтения за пределами файла на диске.
46	`sb_append(err, "PE header offset out of range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
47	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
48	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
50	`uint32_t sig = 0;`	Технически объявляет `sig` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
51	`if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {`	Технически проверяет условие `!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC`; логически это защита от NULL-указателя и работы с неинициализированными данными.
52	`sb_append(err, "Invalid PE signature.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
53	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
54	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
56	`size_t filehdr = (size_t)lfanew + 4;`	Технически присваивает `size_t filehdr` значение `(size_t)lfanew + 4`. Логически это обновляет `size_t filehdr`: хранит размер или количество, поэтому используется в проверках границ.
57	`uint16_t machine = 0;`	Технически объявляет `machine` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: тип архитектуры из COFF header; нужен, чтобы отсеять не-x64 бинарники.
58	`uint16_t nsects = 0;`	Технически объявляет `nsects` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
59	`uint16_t opt_size = 0;`	Технически объявляет `opt_size` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
60	`if (!buf_read_u16(b, filehdr + 0, &machine) ||`	Технически проверяет условие `!buf_read_u16(b, filehdr + 0, &machine`; логически это защита от NULL-указателя и работы с неинициализированными данными.
61	`!buf_read_u16(b, filehdr + 2, &nsects) ||`	Технически вызывает `buf_read_u16`. Логически это действие нужно в функции `pe_parse`: читает 16-битное little-endian поле из PE.
62	`!buf_read_u16(b, filehdr + 16, &opt_size)) {`	Технически объявляет/начинает функцию `buf_read_u16`; она доступная другим модулям через заголовок или линковку. Логически функция читает 16-битное little-endian поле PE.
63	`sb_append(err, "Failed to read COFF header.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
64	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
65	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
67	`if (machine != IMAGE_FILE_MACHINE_AMD64) {`	Технически проверяет условие `machine != IMAGE_FILE_MACHINE_AMD64`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
68	`sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `pe_parse`: добавляет форматированный текст в динамический буфер.
69	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
70	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
72	`size_t opthdr = filehdr + 20;`	Технически объявляет `opthdr` в функции `pe_parse` и сразу задает начальное значение `filehdr + 20`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
73	`if (opthdr + opt_size > b->size) {`	Технически проверяет условие `opthdr + opt_size > b->size`; логически это защита от чтения за пределами файла на диске.
74	`sb_append(err, "Optional header out of range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
75	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
76	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
77	`if (opt_size < 0x70u) {`	Технически проверяет условие `opt_size < 0x70u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
78	`sb_append(err, "PE32+ optional header is too small for required fields.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
79	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
80	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
82	`uint16_t magic = 0;`	Технически объявляет `magic` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
83	`if (!buf_read_u16(b, opthdr + 0, &magic)) {`	Технически проверяет условие `!buf_read_u16(b, opthdr + 0, &magic)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
84	`sb_append(err, "Failed to read optional header magic.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
85	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
86	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
87	`if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {`	Технически проверяет условие `magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
88	`sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `pe_parse`: добавляет форматированный текст в динамический буфер.
89	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
90	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
92	`uint64_t image_base = 0;`	Технически объявляет `image_base` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: базовый адрес образа, относительно которого Windows строит виртуальные адреса.
93	`uint32_t size_of_image = 0;`	Технически объявляет `size_of_image` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: верхняя виртуальная граница PE-образа; выход за нее считается ошибкой координат.
94	`uint32_t size_of_headers = 0;`	Технически объявляет `size_of_headers` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: размер заголовков на диске; нужен для проверки секций и raw offsets.
95	`uint32_t section_alignment = 0;`	Технически объявляет `section_alignment` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: выравнивание секций в памяти, установленное PE optional header.
96	`uint32_t file_alignment = 0;`	Технически объявляет `file_alignment` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: выравнивание секций на диске, установленное PE optional header.
97	`uint32_t number_of_rva_and_sizes = 0;`	Технически объявляет `number_of_rva_and_sizes` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
99	`/* PE32+ offsets */`	Технически это комментарий исходника. Логически он объясняет разработчику: PE32+ offsets.
100	`if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||`	Технически проверяет условие `!buf_read_u64(b, opthdr + 0x18, &image_base`; логически это защита от NULL-указателя и работы с неинициализированными данными.
101	`!buf_read_u32(b, opthdr + 0x20, &section_alignment) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
102	`!buf_read_u32(b, opthdr + 0x24, &file_alignment) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
103	`!buf_read_u32(b, opthdr + 0x38, &size_of_image) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
104	`!buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
105	`!buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {`	Технически объявляет/начинает функцию `buf_read_u32`; она доступная другим модулям через заголовок или линковку. Логически функция читает 32-битное little-endian поле PE.
106	`sb_append(err, "Failed to read key fields from optional header.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
107	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
108	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
109	`if (size_of_image == 0) {`	Технически проверяет условие `size_of_image == 0`; логически это защита от выхода RVA/диапазона за SizeOfImage.
110	`sb_append(err, "SizeOfImage is zero.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
111	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
112	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
113	`if (size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size) {`	Технически проверяет условие `size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size`; логически это защита от чтения за пределами файла на диске.

Стр.	Код	Нормальное объяснение
114	`sb_append(err, "SizeOfHeaders is zero or exceeds the file size.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
115	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
116	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
117	`if (section_alignment == 0 || file_alignment == 0) {`	Технически проверяет условие `section_alignment == 0 || file_alignment == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
118	`sb_append(err, "SectionAlignment/FileAlignment must be non-zero.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
119	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
120	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
121	`if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {`	Технически проверяет условие `number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
122	`sb_append(err, "DataDirectory too small.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
123	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
124	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
126	`/* DataDirectory starts at 0x70 */`	Технически это комментарий исходника. Логически он объясняет разработчику: DataDirectory starts at 0x70.
127	`size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;`	Технически объявляет `dd_rel` в функции `pe_parse` и сразу задает начальное значение `0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
128	`if (dd_rel + 8u > opt_size) {`	Технически проверяет условие `dd_rel + 8u > opt_size`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
129	`sb_append(err, "Exception data directory lies outside the optional header.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
130	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
131	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
132	`size_t dd = opthdr + dd_rel;`	Технически объявляет `dd` в функции `pe_parse` и сразу задает начальное значение `opthdr + dd_rel`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
133	`uint32_t exc_rva = 0, exc_sz = 0;`	Технически объявляет `exc_rva` в функции `pe_parse` и сразу задает начальное значение `0, exc_sz = 0`. Логически это место для данных: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
134	`if (!buf_read_u32(b, dd + 0, &exc_rva) || !buf_read_u32(b, dd + 4, &exc_sz)) {`	Технически проверяет условие `!buf_read_u32(b, dd + 0, &exc_rva) || !buf_read_u32(b, dd + 4, &exc_sz)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
135	`sb_append(err, "Failed to read exception directory.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
136	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
137	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
139	`size_t secthdr = opthdr + opt_size;`	Технически объявляет `secthdr` в функции `pe_parse` и сразу задает начальное значение `opthdr + opt_size`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
140	`if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) {`	Технически проверяет условие `(size_t)nsects > (SIZE_MAX - secthdr) / 40u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
141	`sb_append(err, "Section header size overflow.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
142	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.

Стр.	Код	Нормальное объяснение
143	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
144	`size_t needed = secthdr + (size_t)nsects * 40u;`	Технически присваивает `size_t needed` значение `secthdr + (size_t)nsects * 40u`. Логически это обновляет `size_t needed`: хранит размер или количество, поэтому используется в проверках границ.
145	`if (needed > b->size) {`	Технически проверяет условие `needed > b->size`; логически это защита от чтения за пределами файла на диске.
146	`sb_append(err, "Section headers out of range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
147	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
148	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
150	`PeSection* secs = (PeSection*)calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection));`	Технически присваивает `PeSection* secs` значение `(PeSection*)calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection))`. Логически это обновляет `PeSection* secs`: хранит промежуточное значение, которое дальше используется в текущей функции.
151	`if (!secs) {`	Технически проверяет условие `!secs`; логически это защита от NULL-указателя и работы с неинициализированными данными.
152	`sb_append(err, "Out of memory.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
153	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
154	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
156	`for (uint16_t i = 0; i < nsects; i++) {`	Технически начинает цикл `uint16_t i = 0; i < nsects; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
157	`size_t sh = secthdr + (size_t)i * 40;`	Технически присваивает `size_t sh` значение `secthdr + (size_t)i * 40`. Логически это обновляет `size_t sh`: хранит размер или количество, поэтому используется в проверках границ.
158	`uint8_t name8[8];`	Технически объявляет `name8` в функции `pe_parse`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
159	`if (!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section name.\n"); return false; }`	Технически проверяет условие `!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section name.\n"`; логически это защита от NULL-указателя и работы с неинициализированными данными.
160	`memcpy(secs[i].name, name8, 8);`	Технически вызывает `memcpy`. Логически это действие нужно в функции `pe_parse`: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
161	`secs[i].name[8] = 0;`	Технически присваивает `secs[i].name[8]` значение `0`. Логически это обновляет `name[8]`: хранит промежуточное значение, которое дальше используется в текущей функции.
163	`if (!buf_read_u32(b, sh + 8, &secs[i].virtual_size) ||`	Технически проверяет условие `!buf_read_u32(b, sh + 8, &secs[i].virtual_size`; логически это защита от NULL-указателя и работы с неинициализированными данными.
164	`!buf_read_u32(b, sh + 12, &secs[i].virtual_address) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
165	`!buf_read_u32(b, sh + 16, &secs[i].raw_size) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
166	`!buf_read_u32(b, sh + 20, &secs[i].raw_ptr) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `pe_parse`: читает 32-битное little-endian поле из PE.
167	`!buf_read_u32(b, sh + 36, &secs[i].characteristics)) {`	Технически объявляет/начинает функцию `buf_read_u32`; она доступная другим модулям через заголовок или линковку. Логически функция читает 32-битное little-endian поле PE.
168	`free(secs);`	Технически вызывает `free`. Логически это действие нужно в функции `pe_parse`: освобождает память, чтобы в C не было утечки.
169	`sb_append(err, "Failed to read section header.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `pe_parse`: добавляет текст в динамический буфер.
170	`return false;`	Технически возвращает `false` из функции `pe_parse`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
171	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
172	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
174	`size_t raw_oob = 0;`	Технически объявляет `raw_oob` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
175	`size_t virtual_oob = 0;`	Технически объявляет `virtual_oob` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
176	`size_t raw_overlaps = 0;`	Технически объявляет `raw_overlaps` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
177	`size_t virtual_overlaps = 0;`	Технически объявляет `virtual_overlaps` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
178	`size_t misaligned = 0;`	Технически объявляет `misaligned` в функции `pe_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
180	`bool compact_alignment =`	Технически присваивает `bool compact_alignment` значение `. Логически это обновляет` bool compact_alignment`: хранит промежуточное значение, которое дальше используется в текущей функции.
181	`section_alignment < 0x1000u && file_alignment == section_alignment;`	Технически выполняет битовую операцию `section_alignment < 0x1000u && file_alignment == section_alignment;`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
182	`bool file_alignment_valid =`	Технически присваивает `bool file_alignment_valid` значение `. Логически это обновляет` bool file_alignment_valid`: хранит промежуточное значение, которое дальше используется в текущей функции.
183	`is_power_of_two_u32(file_alignment) &&`	Технически вызывает `is_power_of_two_u32`. Логически это действие нужно в функции `pe_parse`: вызывает функцию `is_power_of_two_u32` для выполнения части алгоритма.
184	`((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||`	Технически выполняет битовую операцию `((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
185	`compact_alignment);`	Технически выполняет оператор `compact_alignment);`. Логически в функции `pe_parse` он меняет состояние, проверяемые данные или подготовку отчета.
186	`if (!file_alignment_valid) misaligned++;`	Технически проверяет условие `!file_alignment_valid`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
187	`if (section_alignment < file_alignment) misaligned++;`	Технически проверяет условие `section_alignment < file_alignment`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
188	`if (section_alignment < 0x1000u && file_alignment != section_alignment) {`	Технически проверяет условие `section_alignment < 0x1000u && file_alignment != section_alignment`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
189	`misaligned++;`	Технически увеличивает `misaligned` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
190	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
191	`if ((size_of_image % section_alignment) != 0) misaligned++;`	Технически проверяет условие `(size_of_image % section_alignment) != 0`; логически это защита от выхода RVA/диапазона за SizeOfImage.
192	`if ((size_of_headers % file_alignment) != 0 ||`	Технически проверяет условие `(size_of_headers % file_alignment`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
193	`(uint64_t)size_of_headers < (uint64_t)needed) {`	Технически эта строка является частью С-конструкции `(uint64_t)size_of_headers < (uint64_t)needed) {`. Логически она уточняет структуру блока функции `pe_parse` и влияет на то, какие данные будут обработаны.
194	`misaligned++;`	Технически увеличивает `misaligned` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
195	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
196	`if ((image_base & 0xffffu) != 0) misaligned++;`	Технически проверяет условие `(image_base & 0xffffu) != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
198	`for (size_t i = 0; i < (size_t)nsects; i++) {`	Технически начинает цикл `size_t i = 0; i < (size_t)nsects; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
199	`const PeSection* s = &secs[i];`	Технически объявляет `s` в функции `pe_parse` и сразу задает начальное значение `&secs[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
200	`uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;`	Технически присваивает `uint64_t raw_end` значение `(uint64_t)s->raw_ptr + (uint64_t)s->raw_size`. Логически это обновляет `uint64_t raw_end`: хранит промежуточное значение, которое дальше используется в текущей функции.
201	`uint64_t virtual_span = (uint64_t)u32_max(s->virtual_size, s->raw_size);`	Технически присваивает `uint64_t virtual_span` значение `(uint64_t)u32_max(s->virtual_size, s->raw_size)`. Логически это обновляет `uint64_t virtual_span`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
202	`uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;`	Технически присваивает `uint64_t virtual_end` значение `(uint64_t)s->virtual_address + virtual_span`. Логически это обновляет `uint64_t virtual_end`: хранит промежуточное значение, которое дальше используется в текущей функции.
203	`if (s->raw_size > 0 && raw_end > (uint64_t)b->size) raw_oob++;`	Технически проверяет условие `s->raw_size > 0 && raw_end > (uint64_t)b->size`; логически это защита от чтения за пределами файла на диске.
204	`if (virtual_span > 0 && virtual_end > (uint64_t)size_of_image) virtual_oob++;`	Технически проверяет условие `virtual_span > 0 && virtual_end > (uint64_t)size_of_image`; логически это защита от выхода RVA/диапазона за SizeOfImage.
205	`if ((s->virtual_address % section_alignment) != 0 ||`	Технически проверяет условие `(s->virtual_address % section_alignment`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
206	`(s->raw_size > 0 &&`	Технически выполняет битовую операцию `(s->raw_size > 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
207	`((s->raw_ptr % file_alignment) != 0 ||`	Технически выполняет битовую операцию `((s->raw_ptr % file_alignment) != 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
208	`(s->raw_size % file_alignment) != 0))) {`	Технически эта строка является частью C-конструкции `(s->raw_size % file_alignment) != 0))) {`. Логически она уточняет структуру блока функции `pe_parse` и влияет на то, какие данные будут обработаны.
209	`misaligned++;`	Технически увеличивает `misaligned` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
210	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
211	`if (i > 0 && s->virtual_address <= secs[i - 1].virtual_address) {`	Технически проверяет условие `i > 0 && s->virtual_address <= secs[i - 1].virtual_address`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
212	`misaligned++;`	Технически увеличивает `misaligned` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
213	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
215	`for (size_t j = 0; j < i; j++) {`	Технически начинает цикл `size_t j = 0; j < i; j++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
216	`const PeSection* p = &secs[j];`	Технически объявляет `p` в функции `pe_parse` и сразу задает начальное значение `&secs[j]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
217	`if (ranges_overlap_u64(s->raw_ptr, s->raw_size, p->raw_ptr, p->raw_size)) {`	Технически проверяет условие `ranges_overlap_u64(s->raw_ptr, s->raw_size, p->raw_ptr, p->raw_size)`; логически это защита от чтения за пределами файла на диске.
218	`raw_overlaps++;`	Технически увеличивает `raw_overlaps` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
219	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
220	`if (ranges_overlap_u64(`	Технически проверяет условие `if (ranges_overlap_u64(`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
221	`s->virtual_address,`	Технически эта строка является частью C-конструкции `s->virtual_address,`. Логически она уточняет структуру блока функции `pe_parse` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
222	`u32_max(s->virtual_size, s->raw_size),`	Технически вызывает `u32_max`. Логически это действие нужно в функции `pe_parse`: вызывает функцию `u32_max` для выполнения части алгоритма.
223	`p->virtual_address,`	Технически эта строка является частью C-конструкции `p->virtual_address,`. Логически она уточняет структуру блока функции `pe_parse` и влияет на то, какие данные будут обработаны.
224	`u32_max(p->virtual_size, p->raw_size))) {`	Технически объявляет/начинает функцию `u32_max`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
225	`virtual_overlaps++;`	Технически увеличивает `virtual_overlaps` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
226	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
227	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
228	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
230	`bool exception_directory_oob = false;`	Технически объявляет `exception_directory_oob` в функции `pe_parse` и сразу задает начальное значение `false`. Логически это место для данных: флаг битого или нечитаемого exception directory.

Стр.	Код	Нормальное объяснение
231	`if (exc_rva != 0 || exc_sz != 0) {`	Технически проверяет условие `exc_rva != 0 || exc_sz != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
232	`uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;`	Технически присваивает `uint64_t exc_end` значение `(uint64_t)exc_rva + (uint64_t)exc_sz`. Логически это обновляет `uint64_t exc_end`: хранит промежуточное значение, которое дальше используется в текущей функции.
233	`exception_directory_oob =`	Технически присваивает `exception_directory_oob` значение `` `. Логически это обновляет exception_directory_oob` ``: флаг битого или нечитаемого exception directory.
234	`exc_rva == 0 || exc_sz == 0 || exc_end > (uint64_t)size_of_image;`	Технически выполняет битовую операцию `exc_rva == 0 || exc_sz == 0 || exc_end > (uint64_t)size_of_image;`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
235	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
237	`out->ok = true;`	Технически присваивает `out->ok` значение `true`. Логически это обновляет `ok`: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
238	`out->is_pe64 = true;`	Технически присваивает `out->is_pe64` значение `true`. Логически это обновляет `is_pe64`: признак PE32+ x64, потому что Windows x64 unwind работает именно с этой моделью.
239	`out->machine = machine;`	Технически присваивает `out->machine` значение `machine`. Логически это обновляет `machine`: тип архитектуры из COFF header; нужен, чтобы отсеять не-x64 бинарники.
240	`out->image_base = image_base;`	Технически присваивает `out->image_base` значение `image_base`. Логически это обновляет `image_base`: базовый адрес образа, относительно которого Windows строит виртуальные адреса.
241	`out->size_of_image = size_of_image;`	Технически присваивает `out->size_of_image` значение `size_of_image`. Логически это обновляет `size_of_image`: верхняя виртуальная граница PE-образа; выход за нее считается ошибкой координат.
242	`out->size_of_headers = size_of_headers;`	Технически присваивает `out->size_of_headers` значение `size_of_headers`. Логически это обновляет `size_of_headers`: размер заголовков на диске; нужен для проверки секций и raw offsets.
243	`out->section_alignment = section_alignment;`	Технически присваивает `out->section_alignment` значение `section_alignment`. Логически это обновляет `section_alignment`: выравнивание секций в памяти, установленное PE optional header.
244	`out->file_alignment = file_alignment;`	Технически присваивает `out->file_alignment` значение `file_alignment`. Логически это обновляет `file_alignment`: выравнивание секций на диске, установленное PE optional header.
245	`out->exception_rva = exc_rva;`	Технически присваивает `out->exception_rva` значение `exc_rva`. Логически это обновляет `exception_rva`: RVA каталога исключений, то есть начало таблицы .pdata.
246	`out->exception_size = exc_sz;`	Технически присваивает `out->exception_size` значение `exc_sz`. Логически это обновляет `exception_size`: размер exception directory, по которому считается число RUNTIME_FUNCTION.
247	`out->file_size = b->size;`	Технически присваивает `out->file_size` значение `b->size`. Логически это обновляет `file_size`: размер входного файла; главная граница для raw-чтения.
248	`out->num_sections = nsects;`	Технически присваивает `out->num_sections` значение `nsects`. Логически это обновляет `num_sections`: количество записей section table.
249	`out->sections = secs;`	Технически присваивает `out->sections` значение `secs`. Логически это обновляет `sections`: массив секций, которым владеет PeImage.
250	`out->section_raw_oob = raw_oob;`	Технически присваивает `out->section_raw_oob` значение `raw_oob`. Логически это обновляет `section_raw_oob`: счетчик секций, raw-диапазон которых выходит за размер файла.
251	`out->section_virtual_oob = virtual_oob;`	Технически присваивает `out->section_virtual_oob` значение `virtual_oob`. Логически это обновляет `section_virtual_oob`: счетчик секций, виртуальный диапазон которых выходит за SizeOfImage.
252	`out->section_raw_overlaps = raw_overlaps;`	Технически присваивает `out->section_raw_overlaps` значение `raw_overlaps`. Логически это обновляет `section_raw_overlaps`: счетчик пересечений секций на диске.
253	`out->section_virtual_overlaps = virtual_overlaps;`	Технически присваивает `out->section_virtual_overlaps` значение `virtual_overlaps`. Логически это обновляет `section_virtual_overlaps`: счетчик пересечений секций в памяти.
254	`out->section_misaligned = misaligned;`	Технически присваивает `out->section_misaligned` значение `misaligned`. Логически это обновляет `section_misaligned`: счетчик нарушений выравнивания и базовых инвариантов section table.
255	`out->exception_directory_oob = exception_directory_oob;`	Технически присваивает `out->exception_directory_oob` значение `exception_directory_oob`. Логически это обновляет `exception_directory_oob`: флаг битого или нечитаемого exception directory.
256	`return true;`	Технически возвращает `true` из функции `pe_parse`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.

Стр.	Код	Нормальное объяснение
257	`}`	Технически завершает область видимости внутри `pe_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
259	`void pe_free(PeImage* img) {`	Технически объявляет/начинает функцию `pe_free`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает память, выделенную PE-парсером.

Стр.	Код	Нормальное объяснение
260	`if (!img) return;`	Технически проверяет условие `!img`; логически это защита от NULL-указателя и работы с неинициализированными данными.
261	`free(img->sections);`	Технически вызывает `free`. Логически это действие нужно в функции `pe_free`: освобождает память, чтобы в C не было утечки.
262	`memset(img, 0, sizeof(*img));`	Технически вызывает `memset`. Логически это действие нужно в функции `pe_free`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
263	`}`	Технически завершает область видимости внутри `pe_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
265	`const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva) {`	Технически объявляет/начинает функцию `pe_find_section_for_rva`; она доступная другим модулям через заголовок или линковку. Логически функция находит секцию, внутри которой лежит заданный RVA.
266	`if (!img || !img->sections) return NULL;`	Технически проверяет условие `!img || !img->sections`; логически это защита от NULL-указателя и работы с неинициализированными данными.
267	`for (size_t i = 0; i < img->num_sections; i++) {`	Технически начинает цикл `size_t i = 0; i < img->num_sections; i++`. Логически он проходит по секциям PE-файла, чтобы проверить каждую запись, а не только первую.
268	`const PeSection* s = &img->sections[i];`	Технически объявляет `s` в функции `pe_find_section_for_rva` и сразу задает начальное значение `&img->sections[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
269	`uint32_t size = u32_max(s->virtual_size, s->raw_size);`	Технически присваивает `uint32_t size` значение `u32_max(s->virtual_size, s->raw_size)`. Логически это обновляет `uint32_t size`: хранит промежуточное значение, которое дальше используется в текущей функции.
270	`uint64_t start = (uint64_t)s->virtual_address;`	Технически присваивает `uint64_t start` значение `(uint64_t)s->virtual_address`. Логически это обновляет `uint64_t start`: хранит промежуточное значение, которое дальше используется в текущей функции.
271	`uint64_t end = start + (uint64_t)size;`	Технически присваивает `uint64_t end` значение `start + (uint64_t)size`. Логически это обновляет `uint64_t end`: хранит промежуточное значение, которое дальше используется в текущей функции.
272	`if ((uint64_t)rva >= start && (uint64_t)rva < end) return s;`	Технически проверяет условие `(uint64_t)rva >= start && (uint64_t)rva < end`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
273	`}`	Технически завершает область видимости внутри `pe_find_section_for_rva`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
274	`return NULL;`	Технически возвращает выражение `NULL`. Логически именно это значение становится результатом функции `pe_find_section_for_rva` для вызывающего кода.
275	`}`	Технически завершает область видимости внутри `pe_find_section_for_rva`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
277	`bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {`	Технически объявляет/начинает функцию `pe_rva_to_fileoff`; она доступная другим модулям через заголовок или линковку. Логически функция переводит RVA из виртуальной координатной системы PE в смещение файла на диске.
278	`if (!img || !out_off) return false;`	Технически проверяет условие `!img || !out_off`; логически это защита от NULL-указателя и работы с неинициализированными данными.
279	`const PeSection* s = pe_find_section_for_rva(img, rva);`	Технически присваивает `const PeSection* s` значение `pe_find_section_for_rva(img, rva)`. Логически это обновляет `const PeSection* s`: хранит промежуточное значение, которое дальше используется в текущей функции.
280	`if (s) {`	Технически проверяет условие `s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
281	`uint32_t delta = rva - s->virtual_address;`	Технически объявляет `delta` в функции `pe_rva_to_fileoff` и сразу задает начальное значение `rva - s->virtual_address`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
282	`if (delta >= s->raw_size) {`	Технически проверяет условие `delta >= s->raw_size`; логически это защита от чтения за пределами файла на диске.
283	`/* RVA points into virtual tail (zero-fill) - not present in file */`	Технически это комментарий исходника. Логически он объясняет разработчику: RVA points into virtual tail (zero-fill) - not present in file.
284	`return false;`	Технически возвращает `false` из функции `pe_rva_to_fileoff`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
285	`}`	Технически завершает область видимости внутри `pe_rva_to_fileoff`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
286	`uint32_t off = 0;`	Технически объявляет `off` в функции `pe_rva_to_fileoff` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
287	`if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;`	Технически проверяет условие `!safe_add_u32(s->raw_ptr, delta, &off)`; логически это защита от чтения за пределами файла на диске.
288	`if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;`	Технически проверяет условие `img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size`; логически это защита от чтения за пределами файла на диске.
289	`*out_off = off;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_off = off;.

Стр.	Код	Нормальное объяснение
290	`  return true;`	Технически возвращает `true` из функции `pe_rva_to_fileoff`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
291	`  }`	Технически завершает область видимости внутри `pe_rva_to_fileoff`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
293	`/* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */`	Технически это комментарий исходника. Логически он объясняет разработчику: RVAs inside SizeOfHeaders map directly to the corresponding file offset..

Стр.	Код	Нормальное объяснение
294	`  if (img->size_of_headers > 0 &&`	Технически проверяет условие `if (img->size_of_headers > 0 &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
295	`  rva < img->size_of_headers &&`	Технически выполняет битовую операцию `rva < img->size_of_headers &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
296	`  (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {`	Технически выполняет битовую операцию `(img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
297	`  *out_off = rva;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_off = rva;.
298	`  return true;`	Технически возвращает `true` из функции `pe_rva_to_fileoff`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
299	`  }`	Технически завершает область видимости внутри `pe_rva_to_fileoff`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
300	`  return false;`	Технически возвращает `false` из функции `pe_rva_to_fileoff`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
301	`}`	Технически завершает область видимости внутри `pe_rva_to_fileoff`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 272. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

5. RUNTIME_FUNCTION и UNWIND_INFO

Этот слой читает .pdata/.xdata и извлекает то, что потом станет сердцем findings: диапазоны функций, адрес unwind-info, handler flags, обработчики и аномалии.

Что сначала выучить в C и теории

- битовые маски и флаги
- табличные записи Windows x64 exception handling
- проверка opcode/operand в UNWIND_INFO
- почему unwind-коды должны соответствовать ABI

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/unwind.h

Назначение файла: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.

Код, который пишем на этом шаге

```c
#pragma once
/*
  unwind.h - x64 UNWIND_INFO parsing (PE exception directory)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;

typedef struct UnwindSummary {
    bool     valid;
    uint8_t  version;
    uint8_t  flags;
    uint8_t  prolog_size;
    uint8_t  count_codes;
    uint8_t  frame_reg;
    uint8_t  frame_off; /* scaled by 16 */
    bool     has_handler;
    uint32_t handler_rva;

    bool     has_chain;
    RuntimeFunction chain;

    uint32_t fixed_stack_bytes;     /* estimated fixed stack frame delta: push+alloc+machframe */
    uint32_t pushed_nonvol_count;
    uint16_t saved_nonvol_mask;     /* bits 0..15 => reg numbers */
    uint16_t saved_xmm_mask;        /* bits 0..15 => XMM reg numbers */

    uint32_t anomalies;             /* bitmask of internal anomalies */
    char     parse_error[160];       /* non-empty when unwind_parse returns false */
} UnwindSummary;

enum UnwindFlags {
    UNW_FLAG_EHANDLER  = 0x01,
    UNW_FLAG_UHANDLER  = 0x02,
    UNW_FLAG_CHAININFO = 0x04
};

/* anomalies flags (for heuristic analysis / diagnostics) */
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
    UA_CHAIN_INVALID         = 1u << 11,
    UA_ABI_REGISTER_CLASS    = 1u << 12,
    UA_ABI_RESERVED_OPINFO   = 1u << 13,
    UA_ABI_PUSH_ORDER        = 1u << 14,
    UA_ABI_CHAIN_RESTRICTION = 1u << 15,
    UA_ABI_SAVE_OFFSET       = 1u << 16
};

typedef struct UnwindParseOpts {
    int  max_chain_depth; /* default 8 */
    bool follow_chain;    /* compute + annotate chain info */
```

```
    bool strict_abi;      /* flag formal Microsoft x64 unwind ABI violations */
} UnwindParseOpts;

typedef enum RuntimeFunctionReadResult {
    RUNTIME_FUNCTION_READ_OK = 0,
    RUNTIME_FUNCTION_READ_MALFORMED,
    RUNTIME_FUNCTION_READ_OOM
} RuntimeFunctionReadResult;

/* Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops. */
bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err);

/* Parse exception directory (RUNTIME_FUNCTION table) into array.
   Caller owns *out (malloc). */
RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
                                  const Buf* b,
                                  const PeImage* img,
                                  RuntimeFunction** out,
                                  size_t* out_count,
                                  StrBuf* err);

bool unwind_read_runtime_functions(const Buf* b,
                                  const PeImage* img,
                                  RuntimeFunction** out,
                                  size_t* out_count,
                                  StrBuf* err);

/* Pretty helpers */
const char* unwind_reg_name(uint8_t reg);
void unwind_flags_to_string(uint8_t flags, char out[64]);

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`unwind.h - x64 UNWIND_INFO parsing (PE exception directory)`	Технически объявляет/начинает функцию `parsing`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
4	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
6	`#include <stdint.h>`	Технически подключает `stdint.h`. Логически это дает модулю доступ к: точные типы uint8_t/uint32_t/uint64_t, без которых нельзя надежно читать бинарный формат.
7	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
8	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
9	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
10	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.
12	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
13	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
14	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
16	`typedef struct RuntimeFunction {`	Технически начинает описание структуры `RuntimeFunction`. Логически структура собирает связанные поля в одну модель данных проекта.
17	`uint32_t begin_rva;`	Технически объявляет поле `begin_rva` внутри структуры. Логически это поле: начало функции в координатах RVA.
18	`uint32_t end_rva;`	Технически объявляет поле `end_rva` внутри структуры. Логически это поле: конец функции в координатах RVA.
19	`uint32_t unwind_rva;`	Технически объявляет поле `unwind_rva` внутри структуры. Логически это поле: RVA структуры UNWIND_INFO для этой функции.
20	`} RuntimeFunction;`	Технически завершает typedef и дает типу имя `RuntimeFunction`. Логически дальше код может объявлять переменные этого типа без `struct`.
22	`typedef struct UnwindSummary {`	Технически начинает описание структуры `UnwindSummary`. Логически структура собирает связанные поля в одну модель данных проекта.
23	`bool valid;`	Технически объявляет поле `valid` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
24	`uint8_t version;`	Технически объявляет поле `version` внутри структуры. Логически это поле: версия UNWIND_INFO; для Windows x64 ожидается версия 1.
25	`uint8_t flags;`	Технически объявляет поле `flags` внутри структуры. Логически это поле: флаги UNWIND_INFO, включая наличие exception/termination handler.

Стр.	Код	Нормальное объяснение
26	`uint8_t prolog_size;`	Технически объявляет поле `prolog_size` внутри структуры. Логически это поле: длина пролога функции в байтах.
27	`uint8_t count_codes;`	Технически объявляет поле `count_codes` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
28	`uint8_t frame_reg;`	Технически объявляет поле `frame_reg` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
29	`uint8_t frame_off; /* scaled by 16 */`	Технически объявляет поле `frame_off` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
30	`bool has_handler;`	Технически объявляет поле `has_handler` внутри структуры. Логически это поле: показывает, есть ли EHANDLER/UHANDLER.
31	`uint32_t handler_rva;`	Технически объявляет поле `handler_rva` внутри структуры. Логически это поле: адрес language-specific handler, если он указан в UNWIND_INFO.
33	`bool has_chain;`	Технически объявляет поле `has_chain` внутри структуры. Логически это поле: хранит булев признак, от которого зависит ветка алгоритма.
34	`RuntimeFunction chain;`	Технически объявляет поле `chain` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
36	`uint32_t fixed_stack_bytes; /* estimated fixed stack frame delta: push+alloc+machframe */`	Технически объявляет поле `fixed_stack_bytes` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
37	`uint32_t pushed_nonvol_count;`	Технически объявляет поле `pushed_nonvol_count` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
38	`uint16_t saved_nonvol_mask; /* bits 0..15 => reg numbers */`	Технически объявляет поле `saved_nonvol_mask` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
39	`uint16_t saved_xmm_mask; /* bits 0..15 => XMM reg numbers */`	Технически объявляет поле `saved_xmm_mask` внутри структуры. Логически это поле: битовая маска сохраненных nonvolatile XMM регистров.

Стр.	Код	Нормальное объяснение
41	`uint32_t anomalies; /* bitmask of internal anomalies */`	Технически объявляет поле `anomalies` внутри структуры. Логически это поле: битовая маска обнаруженных проблем декодирования unwind.
42	`char parse_error[160]; /* non-empty when unwind_parse returns false */`	Технически эта строка является частью C-конструкции `char parse_error[160]; /* non-empty when unwind_parse returns false */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
43	`} UnwindSummary;`	Технически завершает typedef и дает типу имя `UnwindSummary`. Логически дальше код может объявлять переменные этого типа без `struct`.
45	`enum UnwindFlags {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
46	`UNW_FLAG_EHANDLER = 0x01,`	Технически присваивает `UNW_FLAG_EHANDLER` значение `0x01,`. Логически это обновляет `UNW_FLAG_EHANDLER`: хранит промежуточное значение, которое дальше используется в текущей функции.
47	`UNW_FLAG_UHANDLER = 0x02,`	Технически присваивает `UNW_FLAG_UHANDLER` значение `0x02,`. Логически это обновляет `UNW_FLAG_UHANDLER`: хранит промежуточное значение, которое дальше используется в текущей функции.
48	`UNW_FLAG_CHAININFO = 0x04`	Технически присваивает `UNW_FLAG_CHAININFO` значение `0x04`. Логически это обновляет `UNW_FLAG_CHAININFO`: хранит промежуточное значение, которое дальше используется в текущей функции.
49	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
51	`/* anomalies flags (for heuristic analysis / diagnostics) */`	Технически это комментарий исходника. Логически он объясняет разработчику: anomalies flags (for heuristic analysis / diagnostics).
52	`enum UnwindAnom {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
53	`UA_VERSION_UNKNOWN = 1u << 0,`	Технически присваивает `UA_VERSION_UNKNOWN` значение `1u << 0,`. Логически это обновляет `UA_VERSION_UNKNOWN`: хранит промежуточное значение, которое дальше используется в текущей функции.
54	`UA_TRUNCATED = 1u << 1,`	Технически присваивает `UA_TRUNCATED` значение `1u << 1,`. Логически это обновляет `UA_TRUNCATED`: хранит промежуточное значение, которое дальше используется в текущей функции.
55	`UA_BAD_OPCODE = 1u << 2,`	Технически присваивает `UA_BAD_OPCODE` значение `1u << 2,`. Логически это обновляет `UA_BAD_OPCODE`: хранит промежуточное значение, которое дальше используется в текущей функции.
56	`UA_BAD_OPERAND = 1u << 3,`	Технически присваивает `UA_BAD_OPERAND` значение `1u << 3,`. Логически это обновляет `UA_BAD_OPERAND`: хранит промежуточное значение, которое дальше используется в текущей функции.
57	`UA_CODEOFFSET_OUTSIDE = 1u << 4,`	Технически присваивает `UA_CODEOFFSET_OUTSIDE` значение `1u << 4,`. Логически это обновляет `UA_CODEOFFSET_OUTSIDE`: хранит промежуточное значение, которое дальше используется в текущей функции.
58	`UA_CHAIN_LOOP = 1u << 5,`	Технически присваивает `UA_CHAIN_LOOP` значение `1u << 5,`. Логически это обновляет `UA_CHAIN_LOOP`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
59	`UA_CHAIN_DEPTH_EXCEEDED = 1u << 6,`	Технически присваивает `UA_CHAIN_DEPTH_EXCEEDED` значение `1u << 6,`. Логически это обновляет `UA_CHAIN_DEPTH_EXCEEDED`: хранит промежуточное значение, которое дальше используется в текущей функции.
60	`UA_HANDLER_OOB = 1u << 7,`	Технически присваивает `UA_HANDLER_OOB` значение `1u << 7,`. Логически это обновляет `UA_HANDLER_OOB`: хранит промежуточное значение, которое дальше используется в текущей функции.
61	`UA_CODEOFFSET_ORDER = 1u << 8,`	Технически присваивает `UA_CODEOFFSET_ORDER` значение `1u << 8,`. Логически это обновляет `UA_CODEOFFSET_ORDER`: хранит промежуточное значение, которое дальше используется в текущей функции.
62	`UA_OPERAND_SANITY = 1u << 9,`	Технически присваивает `UA_OPERAND_SANITY` значение `1u << 9,`. Логически это обновляет `UA_OPERAND_SANITY`: хранит промежуточное значение, которое дальше используется в текущей функции.
63	`UA_ALIGNMENT = 1u << 10,`	Технически присваивает `UA_ALIGNMENT` значение `1u << 10,`. Логически это обновляет `UA_ALIGNMENT`: хранит промежуточное значение, которое дальше используется в текущей функции.
64	`UA_CHAIN_INVALID = 1u << 11,`	Технически присваивает `UA_CHAIN_INVALID` значение `1u << 11,`. Логически это обновляет `UA_CHAIN_INVALID`: хранит промежуточное значение, которое дальше используется в текущей функции.
65	`UA_ABI_REGISTER_CLASS = 1u << 12,`	Технически присваивает `UA_ABI_REGISTER_CLASS` значение `1u << 12,`. Логически это обновляет `UA_ABI_REGISTER_CLASS`: хранит промежуточное значение, которое дальше используется в текущей функции.
66	`UA_ABI_RESERVED_OPINFO = 1u << 13,`	Технически присваивает `UA_ABI_RESERVED_OPINFO` значение `1u << 13,`. Логически это обновляет `UA_ABI_RESERVED_OPINFO`: хранит промежуточное значение, которое дальше используется в текущей функции.
67	`UA_ABI_PUSH_ORDER = 1u << 14,`	Технически присваивает `UA_ABI_PUSH_ORDER` значение `1u << 14,`. Логически это обновляет `UA_ABI_PUSH_ORDER`: хранит промежуточное значение, которое дальше используется в текущей функции.
68	`UA_ABI_CHAIN_RESTRICTION = 1u << 15,`	Технически присваивает `UA_ABI_CHAIN_RESTRICTION` значение `1u << 15,`. Логически это обновляет `UA_ABI_CHAIN_RESTRICTION`: хранит промежуточное значение, которое дальше используется в текущей функции.
69	`UA_ABI_SAVE_OFFSET = 1u << 16`	Технически присваивает `UA_ABI_SAVE_OFFSET` значение `1u << 16`. Логически это обновляет `UA_ABI_SAVE_OFFSET`: хранит промежуточное значение, которое дальше используется в текущей функции.
70	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
72	`typedef struct UnwindParseOpts {`	Технически начинает описание структуры `UnwindParseOpts`. Логически структура собирает связанные поля в одну модель данных проекта.
73	`int max_chain_depth; /* default 8 */`	Технически объявляет поле `max_chain_depth` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
74	`bool follow_chain; /* compute + annotate chain info */`	Технически объявляет поле `follow_chain` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
75	`bool strict_abi; /* flag formal Microsoft x64 unwind ABI violations */`	Технически объявляет поле `strict_abi` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
76	`} UnwindParseOpts;`	Технически завершает typedef и дает типу имя `UnwindParseOpts`. Логически дальше код может объявлять переменные этого типа без `struct`.

Стр.	Код	Нормальное объяснение
78	`typedef enum RuntimeFunctionReadResult {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
79	`RUNTIME_FUNCTION_READ_OK = 0,`	Технически присваивает `RUNTIME_FUNCTION_READ_OK` значение `0,`. Логически это обновляет `RUNTIME_FUNCTION_READ_OK`: хранит промежуточное значение, которое дальше используется в текущей функции.
80	`RUNTIME_FUNCTION_READ_MALFORMED,`	Технически эта строка является частью C-конструкции `RUNTIME_FUNCTION_READ_MALFORMED,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
81	`RUNTIME_FUNCTION_READ_OOM`	Технически объявляет поле `RUNTIME_FUNCTION_READ_OOM` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
82	`} RuntimeFunctionReadResult;`	Технически завершает typedef и дает типу имя `RuntimeFunctionReadResult`. Логически дальше код может объявлять переменные этого типа без `struct`.
84	`/* Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops..
85	`bool unwind_parse(const Buf* b,`	Технически вызывает `unwind_parse`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `unwind_parse` для выполнения части алгоритма.
86	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
87	`uint32_t unwind_rva,`	Технически эта строка является частью C-конструкции `uint32_t unwind_rva,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
88	`  const UnwindParseOpts* opts,`	Технически эта строка является частью C-конструкции `const UnwindParseOpts* opts,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
89	`  UnwindSummary* out,`	Технически эта строка является частью C-конструкции `UnwindSummary* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
90	`  StrBuf* ops_text,`	Технически эта строка является частью C-конструкции `StrBuf* ops_text,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
91	`  StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
93	`/* Parse exception directory (RUNTIME_FUNCTION table) into array.`	Технически это комментарий исходника. Логически он объясняет разработчику: Parse exception directory (RUNTIME_FUNCTION table) into array..
94	`  Caller owns *out (malloc). */`	Технически вызывает `out`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `out` для выполнения части алгоритма.
95	`RuntimeFunctionReadResult unwind_read_runtime_functions_ex(`	Технически вызывает `unwind_read_runtime_functions_ex`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `unwind_read_runtime_functions_ex` для выполнения части алгоритма.
96	`  const Buf* b,`	Технически эта строка является частью C-конструкции `const Buf* b,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
97	`  const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
98	`  RuntimeFunction** out,`	Технически эта строка является частью C-конструкции `RuntimeFunction** out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
99	`  size_t* out_count,`	Технически эта строка является частью C-конструкции `size_t* out_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
100	`  StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
102	`bool unwind_read_runtime_functions(const Buf* b,`	Технически вызывает `unwind_read_runtime_functions`. Логически это действие нужно в функции `глобальной области файла`: читает массив RUNTIME_FUNCTION из exception directory.
103	`  const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
104	`  RuntimeFunction** out,`	Технически эта строка является частью C-конструкции `RuntimeFunction** out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
105	`  size_t* out_count,`	Технически эта строка является частью C-конструкции `size_t* out_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
106	`  StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
108	`/* Pretty helpers */`	Технически это комментарий исходника. Логически он объясняет разработчику: Pretty helpers.
109	`const char* unwind_reg_name(uint8_t reg);`	Технически объявляет прототип функции `unwind_reg_name` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и переводит номер регистра из UNWIND_INFO в понятное имя.
110	`void unwind_flags_to_string(uint8_t flags, char out[64]);`	Технически объявляет прототип функции `unwind_flags_to_string` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и переводит флаги UNWIND_INFO в текст.
112	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
113	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
114	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 98. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/unwind.c

Назначение файла: декодер таблицы функций и UNWIND_INFO Windows x64.

Код, который пишем на этом шаге

```
#include "unwind.h"
#include <stdlib.h>
#include <string.h>

const char* unwind_reg_name(uint8_t reg) {
    /* mapping per MS x64 unwind encoding */
    static const char* names[16] = {
        "RAX","RCX","RDX","RBX","RSP","RBP","RSI","RDI",
        "R8","R9","R10","R11","R12","R13","R14","R15"
    };
    if (reg < 16) return names[reg];
    return "REG?";
}

void unwind_flags_to_string(uint8_t flags, char out[64]) {
    out[0] = 0;
    bool first = true;
    if (flags & UNW_FLAG_EHANDLER) { strcat(out, "EHANDLER"); first = false; }
    if (flags & UNW_FLAG_UHANDLER) { strcat(out, first ? "UHANDLER" : "|UHANDLER"); first = false; }
    if (flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO"); first = false; }
    if (first) strcpy(out, "0");
}

static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 + opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
}

static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
                                    uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {
        sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);
        return false;
    }
    if ((size_t)off + 4 > b->size) {
        sb_append(err, "UnwindInfo header out of file range.\n");
        return false;
    }
    if (!buf_read(b, off, hdr, 4)) {
        sb_append(err, "Failed to read UnwindInfo header bytes.\n");
        return false;
    }
    *out_fileoff = off;
    return true;
}

static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));
    return true;
}

static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi) {
    return (uint32_t)lo | ((uint32_t)hi << 16);
}

static bool is_unwindable_gpr(uint8_t reg) {
    /* Real MSVC, Rust and OpenSSL records also describe volatile registers. */
    return reg < 16 && reg != 4;
}

static bool is_unwindable_xmm(uint8_t reg) {
    return reg < 16;
}

static bool is_abi_nonvolatile_gpr(uint8_t reg) {
    return reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12;
}

static bool is_abi_nonvolatile_xmm(uint8_t reg) {
    return reg >= 6 && reg < 16;
}

static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies) {
    if (!total || !anomalies) return;
    if (UINT32_MAX - *total < amount) {
        *total = UINT32_MAX;
        *anomalies |= UA_OPERAND_SANITY;
        return;
    }
    *total += amount;
}

```

```
static bool alloc_large_mode0_suspicious(uint16_t scaled8) {
    uint32_t size = (uint32_t)scaled8 * 8u;
    return size == 0 || size <= 128u || (size % 8u) != 0;
}

static bool alloc_large_mode1_suspicious(uint32_t size) {
    return size == 0 || (size % 8u) != 0 || size < (512u * 1024u);
}

static bool save_nonvol_far_suspicious(uint32_t off) {
    return off == 0 || (off % 8u) != 0 || off < (512u * 1024u);
}

static bool save_xmm_far_suspicious(uint32_t off) {
    return off == 0 || (off % 16u) != 0 || off < (1024u * 1024u);
}

static bool parse_chain_rf(const Buf* b, const PeImage* img, size_t fileoff_after_codes,
                           RuntimeFunction* out_chain) {
    uint32_t a=0, c=0, u=0;
    if (!buf_read_u32(b, fileoff_after_codes + 0, &a)) return false;
    if (!buf_read_u32(b, fileoff_after_codes + 4, &c)) return false;
    if (!buf_read_u32(b, fileoff_after_codes + 8, &u)) return false;
    out_chain->begin_rva = a;
    out_chain->end_rva = c;
    out_chain->unwind_rva = u;
    (void)img;
    return true;
}

static bool unwind_parse_fail(UnwindSummary* out,
                              StrBuf* err,
                              size_t err_start,
                              const char* fallback) {
    if (!out) return false;
    out->valid = false;

    const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";
    if (err && err->s && err->len > err_start) {
        message = err->s + err_start;
    } else if (err && fallback) {
        sb_appendf(err, "%s\n", fallback);
    }

    size_t n = strlen(message);
    while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;
    if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;
    memcpy(out->parse_error, message, n);
    out->parse_error[n] = '\0';
    return false;
}

bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err) {
    if (!b || !img || !out || !err) return false;
    memset(out, 0, sizeof(*out));
    size_t err_start = err->len;
    if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }

    int max_depth = 8;
    bool follow_chain = true;
    bool strict_abi = false;
    if (opts) {
        if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;
        follow_chain = opts->follow_chain;
        strict_abi = opts->strict_abi;
    }
    if (max_depth > 16) max_depth = 16;

    uint8_t hdr[4];
    uint32_t uoff = 0;
    if ((unwind_rva & 3u) != 0) {
        out->anomalies |= UA_ALIGNMENT;
    }
    if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {
        out->anomalies |= UA_TRUNCATED;
        return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");
    }

    uint8_t ver = (uint8_t)(hdr[0] & 0x7);
    uint8_t flags = (uint8_t)(hdr[0] >> 3);
    uint8_t prolog = hdr[1];
    uint8_t count = hdr[2];
    uint8_t fr = (uint8_t)(hdr[3] & 0x0f);
    uint8_t foff = (uint8_t)(hdr[3] >> 4);

    out->version = ver;
    out->flags = flags;
    out->prolog_size = prolog;
    out->count_codes = count;
    out->frame_reg = fr;
    out->frame_off = foff;

    if (!(ver == 1 || ver == 2)) {
        out->anomalies |= UA_VERSION_UNKNOWN;
        return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");
    }
    if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr == 0 && foff != 0) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (fr != 0 && !is_unwindable_gpr(fr)) {
        out->anomalies |= UA_BAD_OPERAND;
    }
    if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {
```

```
228 |         out->anomalies |= UA_ABI_REGISTER_CLASS;
229 |     }
230 |
231 |     /* total slots stored on disk (aligned to even number) */
232 |     uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);
233 |     size_t codes_off = (size_t)uoff + 4;
234 |     size_t codes_bytes = (size_t)slots_even * 2u;
235 |     if (codes_off + codes_bytes > b->size) {
236 |         out->anomalies |= UA_TRUNCATED;
237 |         sb_append(err, "Unwind codes array out of file range.\n");
238 |         return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file range.");
239 |     }
240 |
241 |     uint32_t fixed_stack = 0;
242 |     uint32_t pushed = 0;
243 |     uint16_t nonvol_mask = 0;
244 |     uint16_t xmm_mask = 0;
245 |     bool have_prev_code_offset = false;
246 |     uint8_t prev_code_offset = 0;
247 |     bool saw_set_fpreg = false;
248 |     bool saw_push_nonvol = false;
249 |     bool v2_seen_epilog = false;
250 |
251 |     /* parse slots 0..count-1 as unwind code starts */
252 |     for (uint32_t i = 0; i < count; ) {
253 |         uint16_t slot = 0;
254 |         if (!read_u16_slot(b, codes_off + (size_t)i * 2, &slot)) {
255 |             out->anomalies |= UA_TRUNCATED;
256 |             sb_append(err, "Failed to read unwind slot.\n");
257 |             return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");
258 |         }
259 |
260 |         uint8_t code_offset = (uint8_t)(slot & 0xff);
261 |         uint8_t op_byte = (uint8_t)(slot >> 8);
262 |         uint8_t op = (uint8_t)(op_byte & 0x0f);
263 |         uint8_t opinfo = (uint8_t)(op_byte >> 4);
264 |
265 |         uint8_t slots = 0;
266 |         if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {
267 |             out->anomalies |= UA_BAD_OPCODE;
268 |             /* try to continue safely as 1 slot */
269 |             slots = 1;
270 |         }
271 |         if (i + slots > count) {
272 |             out->anomalies |= UA_TRUNCATED;
273 |             sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");
274 |             return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond CountOfCodes.");
275 |         }
276 |         bool v2_descriptor = ver == 2 && (op == 6 || op == 7);
277 |         if (!v2_descriptor) {
278 |             if (code_offset > prolog) {
279 |                 out->anomalies |= UA_CODEOFFSET_OUTSIDE;
280 |             }
281 |             if (have_prev_code_offset && code_offset > prev_code_offset) {
282 |                 out->anomalies |= UA_CODEOFFSET_ORDER;
283 |             }
284 |             have_prev_code_offset = true;
285 |             prev_code_offset = code_offset;
286 |         }
287 |         if (strict_abi && !v2_descriptor) {
288 |             if (saw_push_nonvol && op != 0 && op != 10) {
289 |                 out->anomalies |= UA_ABI_PUSH_ORDER;
290 |             }
291 |             if (op == 0) saw_push_nonvol = true;
292 |             if ((flags & UNW_FLAG_CHAININFO) && op != 4) {
293 |                 out->anomalies |= UA_ABI_CHAIN_RESTRICTION;
294 |             }
295 |         }
296 |
297 |         if (ops_text) {
298 |             /* print one-line per op start */
299 |             switch (op) {
300 |                 case 0: sb_appendf(ops_text, "  @%02x PUSH_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;
301 |                 case 1: sb_appendf(ops_text, "  @%02x ALLOC_LARGE mode=%u\n", code_offset, (unsigned)opinfo); break;
302 |                 case 2: sb_appendf(ops_text, "  @%02x ALLOC_SMALL %u\n", code_offset, (unsigned)(opinfo*8u + 8u)); break;
303 |                 case 3: sb_appendf(ops_text, "  @%02x SET_FPREG %s + %u\n", code_offset, unwind_reg_name(fr), (unsigned)(foff*16u));
    break;
304 |                 case 4: sb_appendf(ops_text, "  @%02x SAVE_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;
305 |                 case 5: sb_appendf(ops_text, "  @%02x SAVE_NONVOL_FAR %s\n", code_offset, unwind_reg_name(opinfo)); break;
306 |                 case 6:
307 |                     if (ver == 2) {
308 |                         if (!v2_seen_epilog) {
309 |                             if ((opinfo & 1u) == 0u) {
310 |                                 uint16_t epilog_slot = 0;
311 |                                 if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,
312 |                                                   &epilog_slot)) {
313 |                                     unsigned epilog_offset =
314 |                                         (unsigned)(epilog_slot & 0xffu) |
315 |                                         ((unsigned)(epilog_slot >> 12) << 8);
316 |                                     sb_appendf(ops_text,
317 |                                                "  EPILOG size=%u offset_from_end=%u flags=%u (v2)\n",
318 |                                                (unsigned)code_offset, epilog_offset,
319 |                                                (unsigned)opinfo);
320 |                                 }
321 |                             } else {
322 |                                 sb_appendf(ops_text, "  EPILOG size=%u flags=%u (v2)\n",
323 |                                            (unsigned)code_offset, (unsigned)opinfo);
324 |                             }
325 |                         } else {
326 |                             unsigned epilog_offset =
327 |                                 (unsigned)code_offset | ((unsigned)opinfo << 8);
328 |                             sb_appendf(ops_text, "  EPILOG offset_from_end=%u (v2)\n",
329 |                                        epilog_offset);
330 |                         }
331 |                     } else {
332 |                         sb_appendf(ops_text, "  @%02x SAVE_XMM (undoc/v1)\n", code_offset);
333 |                     }
334 |                     break;
335 |                 case 7:
336 |                     if (ver == 2) sb_appendf(ops_text, "  @%02x SPARE (v2)\n", code_offset);
337 |                     else sb_appendf(ops_text, "  @%02x SAVE_XMM_FAR (undoc/v1)\n", code_offset);
338 |                     break;
339 |                 case 8: sb_appendf(ops_text, "  @%02x SAVE_XMM128 XMM%u\n", code_offset, (unsigned)opinfo); break;
340 |                 case 9: sb_appendf(ops_text, "  @%02x SAVE_XMM128_FAR XMM%u\n", code_offset, (unsigned)opinfo); break;
341 |                 case 10: sb_appendf(ops_text, "  @%02x PUSH_MACHFRAME err=%u\n", code_offset, (unsigned)opinfo); break;
```

```
                default: sb_appendf(ops_text, "  @%02x OP_%u info=%u\n", code_offset, (unsigned)op, (unsigned)opinfo); break;
            }
        }

        /* compute effects */
        switch (op) {
            case 0: /* push nonvol */
                add_fixed_stack(&fixed_stack, 8, &out->anomalies);
                pushed++;
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            case 1: { /* alloc large */
                if (opinfo == 0) {
                    uint16_t scaled = 0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");
                    }
                    add_fixed_stack(&fixed_stack, (uint32_t)scaled * 8u, &out->anomalies);
                    if (alloc_large_mode0_suspicious(scaled)) out->anomalies |= UA_OPERAND_SANITY;
                    if (ops_text) sb_appendf(ops_text, "       size=%u\n", (unsigned)((uint32_t)scaled * 8u));
                } else if (opinfo == 1) {
                    uint16_t lo=0, hi=0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                        !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");
                    }
                    uint32_t size = read_u32_from_slots(lo, hi);
                    add_fixed_stack(&fixed_stack, size, &out->anomalies);
                    if (alloc_large_mode1_suspicious(size)) out->anomalies |= UA_OPERAND_SANITY;
                    if (ops_text) sb_appendf(ops_text, "       size=%u\n", (unsigned)size);
                } else {
                    out->anomalies |= UA_BAD_OPERAND;
                }
                break;
            }
            case 2: /* alloc small */
                add_fixed_stack(&fixed_stack, (uint32_t)opinfo * 8u + 8u, &out->anomalies);
                break;
            case 3: /* set frame reg */
                if (fr == 0 || saw_set_fpreg) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && opinfo != 0) {
                    out->anomalies |= UA_ABI_RESERVED_OPINFO;
                }
                saw_set_fpreg = true;
                /* no fixed stack delta */
                break;
            case 4: { /* save nonvol */
                uint16_t scaled = 0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL operand.");
                }
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && scaled == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 5: { /* save nonvol far */
                uint16_t lo=0, hi=0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                    !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL_FAR operand.");
                }
                uint32_t save_offset = read_u32_from_slots(lo, hi);
                if (save_nonvol_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && save_offset == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 6: { /* save xmm (v1, undocumented encoding) or epilog (v2) */
                if (ver != 2) {
                    uint16_t scaled = 0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM operand.");
                    }
                    if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                    if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                        out->anomalies |= UA_ABI_REGISTER_CLASS;
                    }
                    if (strict_abi && scaled == 0) {
                        out->anomalies |= UA_ABI_SAVE_OFFSET;
                    }
                    if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                }
                break;
            }
            case 7: { /* save xmm far (v1, undocumented encoding) or spare (v2) */
                if (ver != 2) {
                    uint16_t lo=0, hi=0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                        !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM_FAR operand.");
                    }
                    uint32_t save_offset = read_u32_from_slots(lo, hi);
                    if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
```

```
                    if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                    if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                        out->anomalies |= UA_ABI_REGISTER_CLASS;
                    }
                    if (strict_abi && save_offset == 0) {
                        out->anomalies |= UA_ABI_SAVE_OFFSET;
                    }
                    if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                }
                break;
            }
            case 8: { /* save xmm128 */
                uint16_t scaled = 0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128 operand.");
                }
                if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && scaled == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 9: { /* save xmm128 far */
                uint16_t lo=0, hi=0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                    !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128_FAR operand.");
                }
                uint32_t save_offset = read_u32_from_slots(lo, hi);
                if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
                if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && save_offset == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 10: /* push machframe */
                if (opinfo > 1) out->anomalies |= UA_BAD_OPERAND;
                add_fixed_stack(&fixed_stack, (opinfo == 0) ? 40u : 48u, &out->anomalies);
                break;
            default:
                /* ignore */
                break;
        }

        if (ver == 2 && op == 6) v2_seen_epilog = true;
        i += slots;
    }
    if (fr != 0 && !saw_set_fpreg && !(flags & UNW_FLAG_CHAININFO)) {
        out->anomalies |= UA_BAD_OPERAND;
    }

    /* tail (handler or chained info) */
    size_t after_codes = codes_off + codes_bytes;

    if (flags & UNW_FLAG_CHAININFO) {
        out->has_chain = true;
        RuntimeFunction chain = {0};
        if (!parse_chain_rf(b, img, after_codes, &chain)) {
            out->anomalies |= UA_TRUNCATED;
            return unwind_parse_fail(out, err, err_start, "Truncated CHAININFO RUNTIME_FUNCTION.");
        } else {
            out->chain = chain;
            if (chain.end_rva <= chain.begin_rva ||
                chain.begin_rva >= img->size_of_image ||
                chain.end_rva > img->size_of_image ||
                chain.unwind_rva == 0 ||
                chain.unwind_rva >= img->size_of_image) {
                out->anomalies |= UA_CHAIN_INVALID;
            }
        }
    }

    if (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)) {
        out->has_handler = true;
        uint32_t handler_rva = 0;
        if (!buf_read_u32(b, after_codes + 0, &handler_rva)) {
            out->anomalies |= UA_TRUNCATED;
            return unwind_parse_fail(out, err, err_start, "Truncated exception handler RVA.");
        } else {
            out->handler_rva = handler_rva;
            if (handler_rva >= img->size_of_image) {
                out->anomalies |= UA_HANDLER_OOB;
            }
        }
    }

    out->fixed_stack_bytes = fixed_stack;
    out->pushed_nonvol_count = pushed;
    out->saved_nonvol_mask = nonvol_mask;
    out->saved_xmm_mask = xmm_mask;

    /* follow chain (lightly): detect loops and depth. */
    if (follow_chain && out->has_chain) {
        uint32_t seen[16]; /* depth max 16 */
        int seen_n = 0;
        uint32_t cur = out->chain.unwind_rva;
        for (int depth = 0; depth < max_depth; depth++) {
            bool loop = false;
            for (int j = 0; j < seen_n; j++) if (seen[j] == cur) loop = true;
            if (loop) { out->anomalies |= UA_CHAIN_LOOP; break; }
            if (seen_n < (int)(sizeof(seen)/sizeof(seen[0]))) seen[seen_n++] = cur;

            UnwindSummary tmp;
            StrBuf tmp_ops = {0};
```

```
572 |             StrBuf tmp_err = {0};
573 |             sb_init(&tmp_err, 256);
574 |             sb_init(&tmp_ops, 256);
575 |             UnwindParseOpts o2 = {
576 |                 .max_chain_depth = 0,
577 |                 .follow_chain = false,
578 |                 .strict_abi = strict_abi
579 |             };
580 |             if (!unwind_parse(b, img, cur, &o2, &tmp, NULL, &tmp_err)) {
581 |                 out->anomalies |= UA_TRUNCATED;
582 |                 sb_free(&tmp_err);
583 |                 sb_free(&tmp_ops);
584 |                 break;
585 |             }
586 |             /* combine fixed stack heuristically */
587 |             add_fixed_stack(&out->fixed_stack_bytes, tmp.fixed_stack_bytes, &out->anomalies);
588 |             out->pushed_nonvol_count += tmp.pushed_nonvol_count;
589 |             out->saved_nonvol_mask |= tmp.saved_nonvol_mask;
590 |             out->saved_xmm_mask |= tmp.saved_xmm_mask;
591 |             if (tmp.frame_reg != out->frame_reg || tmp.frame_off != out->frame_off) {
592 |                 out->anomalies |= UA_BAD_OPERAND;
593 |             }
594 |             if (!out->has_handler && tmp.has_handler) {
595 |                 out->has_handler = true;
596 |                 out->handler_rva = tmp.handler_rva;
597 |             }
598 |             sb_free(&tmp_err);
599 |             sb_free(&tmp_ops);
600 |
601 |             if (!tmp.has_chain) break;
602 |             cur = tmp.chain.unwind_rva;
603 |             if (depth == max_depth-1) out->anomalies |= UA_CHAIN_DEPTH_EXCEEDED;
604 |         }
605 |     }
606 |
607 |     out->valid = true;
608 |     return true;
609 | }
610 |
611 | RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
612 |                                 const Buf* b,
613 |                                 const PeImage* img,
614 |                                 RuntimeFunction** out_rf,
615 |                                 size_t* out_count,
616 |                                 StrBuf* err) {
617 |     if (!b || !img || !out_rf || !out_count || !err) {
618 |         return RUNTIME_FUNCTION_READ_MALFORMED;
619 |     }
620 |     *out_rf = NULL;
621 |     *out_count = 0;
622 |
623 |     if (img->exception_rva == 0 && img->exception_size == 0) {
624 |         return RUNTIME_FUNCTION_READ_OK;
625 |     }
626 |     if (img->exception_rva == 0 || img->exception_size == 0) {
627 |         sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-zero.\n");
628 |         return RUNTIME_FUNCTION_READ_MALFORMED;
629 |     }
630 |     uint32_t off = 0;
631 |     if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {
632 |         sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);
633 |         return RUNTIME_FUNCTION_READ_MALFORMED;
634 |     }
635 |     if ((size_t)off + img->exception_size > b->size) {
636 |         sb_append(err, "Exception directory out of file range.\n");
637 |         return RUNTIME_FUNCTION_READ_MALFORMED;
638 |     }
639 |     if (img->exception_size < 12) {
640 |         sb_append(err, "Exception directory size too small.\n");
641 |         return RUNTIME_FUNCTION_READ_MALFORMED;
642 |     }
643 |     if ((img->exception_size % 12u) != 0) {
644 |         sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");
645 |         return RUNTIME_FUNCTION_READ_MALFORMED;
646 |     }
647 |     size_t n = img->exception_size / 12;
648 |     RuntimeFunction* rf = (RuntimeFunction*)calloc(n, sizeof(RuntimeFunction));
649 |     if (!rf) {
650 |         sb_append(err, "Out of memory.\n");
651 |         return RUNTIME_FUNCTION_READ_OOM;
652 |     }
653 |
654 |     for (size_t i = 0; i < n; i++) {
655 |         size_t e = (size_t)off + i*12;
656 |         uint32_t bva=0, eva=0, uva=0;
657 |         if (!buf_read_u32(b, e + 0, &bva) ||
658 |             !buf_read_u32(b, e + 4, &eva) ||
659 |             !buf_read_u32(b, e + 8, &uva)) {
660 |             free(rf);
661 |             sb_append(err, "Failed to read runtime function entry.\n");
662 |             return RUNTIME_FUNCTION_READ_MALFORMED;
663 |         }
664 |         rf[i].begin_rva = bva;
665 |         rf[i].end_rva = eva;
666 |         rf[i].unwind_rva = uva;
667 |     }
668 |
669 |     *out_rf = rf;
670 |     *out_count = n;
671 |     return RUNTIME_FUNCTION_READ_OK;
672 | }
673 |
674 | bool unwind_read_runtime_functions(const Buf* b,
675 |                                 const PeImage* img,
676 |                                 RuntimeFunction** out_rf,
677 |                                 size_t* out_count,
678 |                                 StrBuf* err) {
679 |     return unwind_read_runtime_functions_ex(
680 |                b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;
681 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "unwind.h"`	Технически подключает `unwind.h`. Логически это дает модулю доступ к: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.
2	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
3	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
5	`const char* unwind_reg_name(uint8_t reg) {`	Технически объявляет/начинает функцию `unwind_reg_name`; она доступная другим модулям через заголовок или линковку. Логически функция переводит номер регистра из UNWIND_INFO в понятное имя.
6	`/* mapping per MS x64 unwind encoding */`	Технически это комментарий исходника. Логически он объясняет разработчику: mapping per MS x64 unwind encoding.
7	`static const char* names[16] = {`	Технически присваивает `static const char* names[16]` значение `{`. Логически это обновляет `static const char* names[16]`: хранит промежуточное значение, которое дальше используется в текущей функции.
8	`"RAX","RCX","RDX","RBX","RSP","RBP","RSI","RDI",`	Технически эта строка является частью C-конструкции `"RAX","RCX","RDX","RBX","RSP","RBP","RSI","RDI",`. Логически она уточняет структуру блока функции `unwind_reg_name` и влияет на то, какие данные будут обработаны.
9	`"R8","R9","R10","R11","R12","R13","R14","R15"`	Технически эта строка является частью C-конструкции `"R8","R9","R10","R11","R12","R13","R14","R15"`. Логически она уточняет структуру блока функции `unwind_reg_name` и влияет на то, какие данные будут обработаны.
10	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
11	`if (reg < 16) return names[reg];`	Технически проверяет условие `reg < 16`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
12	`return "REG?";`	Технически возвращает выражение `"REG?"`. Логически именно это значение становится результатом функции `unwind_reg_name` для вызывающего кода.
13	`}`	Технически завершает область видимости внутри `unwind_reg_name`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
15	`void unwind_flags_to_string(uint8_t flags, char out[64]) {`	Технически объявляет/начинает функцию `unwind_flags_to_string`; она доступная другим модулям через заголовок или линковку. Логически функция переводит флаги UNWIND_INFO в текст.
16	`out[0] = 0;`	Технически присваивает `out[0]` значение `0`. Логически это обновляет `out[0]`: хранит промежуточное значение, которое дальше используется в текущей функции.
17	`bool first = true;`	Технически объявляет `first` в функции `unwind_flags_to_string` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
18	`if (flags & UNW_FLAG_EHANDLER) { strcat(out, "EHANDLER"); first = false; }`	Технически проверяет условие `flags & UNW_FLAG_EHANDLER) { strcat(out, "EHANDLER";` логически отделяет функции с language-specific handler от обычных функций.
19	`if (flags & UNW_FLAG_UHANDLER) { strcat(out, first ? "UHANDLER" : "|UHANDLER"); first = false; }`	Технически проверяет условие `flags & UNW_FLAG_UHANDLER) { strcat(out, first ? "UHANDLER" : "|UHANDLER";` логически отделяет функции с language-specific handler от обычных функций.
20	`if (flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO"); first = false; }`	Технически проверяет условие `flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO";` если оно истинно, выполняется защитная или аналитическая ветка ниже.
21	`if (first) strcpy(out, "0");`	Технически проверяет условие `first) strcpy(out, "0";` если оно истинно, выполняется защитная или аналитическая ветка ниже.
22	`}`	Технически завершает область видимости внутри `unwind_flags_to_string`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
24	`static bool slots_for_code(uint8_t version,`	Технически вызывает `slots_for_code`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `slots_for_code` для выполнения части алгоритма.
25	`uint8_t op,`	Технически эта строка является частью C-конструкции `uint8_t op,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
26	`uint8_t opinfo,`	Технически эта строка является частью C-конструкции `uint8_t opinfo,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
27	`bool v2_seen_epilog,`	Технически эта строка является частью C-конструкции `bool v2_seen_epilog,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
28	`uint8_t* out_slots) {`	Технически эта строка является частью C-конструкции `uint8_t* out_slots) {`. Логически она уточняет структуру блока функции `slots_for_code` и влияет на то, какие данные будут обработаны.
29	`/* based on documented v1 ops + v2 epilog/spare, and common implementations */`	Технически это комментарий исходника. Логически он объясняет разработчику: based on documented v1 ops + v2 epilog/spare, and common implementations.
30	`switch (op) {`	Технически выбирает ветку по значению `op`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.

Стр.	Код	Нормальное объяснение
31	`case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;`	Технически объявляет ветку switch для значения `0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
32	`case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 + opinfo); return true;`	Технически объявляет ветку switch для значения `1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 + opin...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
33	`case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;`	Технически объявляет ветку switch для значения `2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
34	`case 3: /* UWOP_SET_FPREG */ *out_slots = 1; return true;`	Технически объявляет ветку switch для значения `3: /* UWOP_SET_FPREG */ *out_slots = 1; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
35	`case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;`	Технически объявляет ветку switch для значения `4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
36	`case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;`	Технически объявляет ветку switch для значения `5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.

Стр.	Код	Нормальное объяснение
37	`case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */`	Технически объявляет ветку switch для значения `6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
38	`if (version == 2) {`	Технически проверяет условие `version == 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
39	`/* The first long-form epilog descriptor has one operand slot.`	Технически это комментарий исходника. Логически он объясняет разработчику: The first long-form epilog descriptor has one operand slot..
40	`Short-form and subsequent descriptors occupy one slot. */`	Технически эта строка является частью C-конструкции `Short-form and subsequent descriptors occupy one slot. */`. Логически она уточняет структуру блока функции `slots_for_code` и влияет на то, какие данные будут обработаны.
41	`*out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;.
42	`return true;`	Технически возвращает `true` из функции `slots_for_code`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
43	`}`	Технически завершает область видимости внутри `slots_for_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
44	`*out_slots = 2; return true;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_slots = 2; return true;.
45	`case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */`	Технически объявляет ветку switch для значения `7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
46	`if (version == 2) { *out_slots = 1; return true; }`	Технически проверяет условие `version == 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
47	`*out_slots = 3; return true;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_slots = 3; return true;.
48	`case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;`	Технически объявляет ветку switch для значения `8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
49	`case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;`	Технически объявляет ветку switch для значения `9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
50	`case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;`	Технически объявляет ветку switch для значения `10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
51	`default:`	Технически объявляет ветку switch по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
52	`return false;`	Технически возвращает `false` из функции `slots_for_code`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
53	`}`	Технически завершает область видимости внутри `slots_for_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
54	`}`	Технически завершает область видимости внутри `slots_for_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
56	`static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,`	Технически вызывает `parse_unwindinfo_header`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `parse_unwindinfo_header` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
57	`uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {`	Технически эта строка является частью C-конструкции `uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {`. Логически она уточняет структуру блока функции `parse_unwindinfo_header` и влияет на то, какие данные будут обработаны.
58	`uint32_t off = 0;`	Технически объявляет `off` в функции `parse_unwindinfo_header` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
59	`if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {`	Технически проверяет условие `!pe_rva_to_fileoff(img, unwind_rva, &off)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
60	`sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `parse_unwindinfo_header`: добавляет форматированный текст в динамический буфер.
61	`return false;`	Технически возвращает `false` из функции `parse_unwindinfo_header`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
62	`}`	Технически завершает область видимости внутри `parse_unwindinfo_header`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
63	`if ((size_t)off + 4 > b->size) {`	Технически проверяет условие `(size_t)off + 4 > b->size`; логически это защита от чтения за пределами файла на диске.
64	`sb_append(err, "UnwindInfo header out of file range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `parse_unwindinfo_header`: добавляет текст в динамический буфер.
65	`return false;`	Технически возвращает `false` из функции `parse_unwindinfo_header`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
66	`}`	Технически завершает область видимости внутри `parse_unwindinfo_header`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
67	`if (!buf_read(b, off, hdr, 4)) {`	Технически проверяет условие `!buf_read(b, off, hdr, 4)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
68	`sb_append(err, "Failed to read UnwindInfo header bytes.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `parse_unwindinfo_header`: добавляет текст в динамический буфер.
69	`return false;`	Технически возвращает `false` из функции `parse_unwindinfo_header`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
70	`}`	Технически завершает область видимости внутри `parse_unwindinfo_header`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
71	`*out_fileoff = off;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_fileoff = off;.
72	`return true;`	Технически возвращает `true` из функции `parse_unwindinfo_header`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
73	`}`	Технически завершает область видимости внутри `parse_unwindinfo_header`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
75	`static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out) {`	Технически объявляет/начинает функцию `read_u16_slot`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
76	`uint8_t tmp[2];`	Технически объявляет `tmp` в функции `read_u16_slot`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
77	`if (!buf_read(b, off, tmp, 2)) return false;`	Технически проверяет условие `!buf_read(b, off, tmp, 2)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
78	`*out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));`	Технически это комментарий исходника. Логически он объясняет разработчику: out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));.
79	`return true;`	Технически возвращает `true` из функции `read_u16_slot`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
80	`}`	Технически завершает область видимости внутри `read_u16_slot`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
82	`static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi) {`	Технически объявляет/начинает функцию `read_u32_from_slots`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
83	`return (uint32_t)lo | ((uint32_t)hi << 16);`	Технически возвращает выражение `(uint32_t)lo | ((uint32_t)hi << 16)`. Логически именно это значение становится результатом функции `read_u32_from_slots` для вызывающего кода.
84	`}`	Технически завершает область видимости внутри `read_u32_from_slots`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
86	`static bool is_unwindable_gpr(uint8_t reg) {`	Технически объявляет/начинает функцию `is_unwindable_gpr`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
87	`/* Real MSVC, Rust and OpenSSL records also describe volatile registers. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Real MSVC, Rust and OpenSSL records also describe volatile registers..
88	`return reg < 16 && reg != 4;`	Технически возвращает выражение `reg < 16 && reg != 4`. Логически именно это значение становится результатом функции `is_unwindable_gpr` для вызывающего кода.

Стр.	Код	Нормальное объяснение
89	`}`	Технически завершает область видимости внутри `is_unwindable_gpr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
91	`static bool is_unwindable_xmm(uint8_t reg) {`	Технически объявляет/начинает функцию `is_unwindable_xmm`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
92	`return reg < 16;`	Технически возвращает выражение `reg < 16`. Логически именно это значение становится результатом функции `is_unwindable_xmm` для вызывающего кода.
93	`}`	Технически завершает область видимости внутри `is_unwindable_xmm`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
95	`static bool is_abi_nonvolatile_gpr(uint8_t reg) {`	Технически объявляет/начинает функцию `is_abi_nonvolatile_gpr`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
96	`return reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12;`	Технически возвращает выражение `reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12`. Логически именно это значение становится результатом функции `is_abi_nonvolatile_gpr` для вызывающего кода.
97	`}`	Технически завершает область видимости внутри `is_abi_nonvolatile_gpr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
99	`static bool is_abi_nonvolatile_xmm(uint8_t reg) {`	Технически объявляет/начинает функцию `is_abi_nonvolatile_xmm`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
100	`return reg >= 6 && reg < 16;`	Технически возвращает выражение `reg >= 6 && reg < 16`. Логически именно это значение становится результатом функции `is_abi_nonvolatile_xmm` для вызывающего кода.
101	`}`	Технически завершает область видимости внутри `is_abi_nonvolatile_xmm`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
103	`static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies) {`	Технически объявляет/начинает функцию `add_fixed_stack`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
104	`if (!total || !anomalies) return;`	Технически проверяет условие `!total || !anomalies`; логически это защита от NULL-указателя и работы с неинициализированными данными.
105	`if (UINT32_MAX - *total < amount) {`	Технически проверяет условие `UINT32_MAX - *total < amount`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
106	`*total = UINT32_MAX;`	Технически это комментарий исходника. Логически он объясняет разработчику: total = UINT32_MAX;.
107	`*anomalies |= UA_OPERAND_SANITY;`	Технически это комментарий исходника. Логически он объясняет разработчику: anomalies |= UA_OPERAND_SANITY;.
108	`return;`	Технически выходит из функции `add_fixed_stack` без значения. Логически дальнейшие строки функции уже не выполняются.
109	`}`	Технически завершает область видимости внутри `add_fixed_stack`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
110	`*total += amount;`	Технически это комментарий исходника. Логически он объясняет разработчику: total += amount;.

Стр.	Код	Нормальное объяснение
111	`}`	Технически завершает область видимости внутри `add_fixed_stack`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
113	`static bool alloc_large_mode0_suspicious(uint16_t scaled8) {`	Технически объявляет/начинает функцию `alloc_large_mode0_suspicious`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
114	`uint32_t size = (uint32_t)scaled8 * 8u;`	Технически присваивает `uint32_t size` значение `(uint32_t)scaled8 * 8u`. Логически это обновляет `uint32_t size`: хранит промежуточное значение, которое дальше используется в текущей функции.
115	`return size == 0 || size <= 128u || (size % 8u) != 0;`	Технически возвращает выражение `size == 0 || size <= 128u || (size % 8u) != 0`. Логически именно это значение становится результатом функции `alloc_large_mode0_suspicious` для вызывающего кода.
116	`}`	Технически завершает область видимости внутри `alloc_large_mode0_suspicious`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
118	`static bool alloc_large_mode1_suspicious(uint32_t size) {`	Технически объявляет/начинает функцию `alloc_large_mode1_suspicious`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
119	`return size == 0 || (size % 8u) != 0 || size < (512u * 1024u);`	Технически возвращает выражение `size == 0 || (size % 8u) != 0 || size < (512u * 1024u)`. Логически именно это значение становится результатом функции `alloc_large_mode1_suspicious` для вызывающего кода.
120	`}`	Технически завершает область видимости внутри `alloc_large_mode1_suspicious`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
122	`static bool save_nonvol_far_suspicious(uint32_t off) {`	Технически объявляет/начинает функцию `save_nonvol_far_suspicious`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
123	`return off == 0 || (off % 8u) != 0 || off < (512u * 1024u);`	Технически возвращает выражение `off == 0 || (off % 8u) != 0 || off < (512u * 1024u)`. Логически именно это значение становится результатом функции `save_nonvol_far_suspicious` для вызывающего кода.
124	`}`	Технически завершает область видимости внутри `save_nonvol_far_suspicious`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
126	`static bool save_xmm_far_suspicious(uint32_t off) {`	Технически объявляет/начинает функцию `save_xmm_far_suspicious`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
127	`return off == 0 || (off % 16u) != 0 || off < (1024u * 1024u);`	Технически возвращает выражение `off == 0 || (off % 16u) != 0 || off < (1024u * 1024u)`. Логически именно это значение становится результатом функции `save_xmm_far_suspicious` для вызывающего кода.
128	`}`	Технически завершает область видимости внутри `save_xmm_far_suspicious`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
130	`static bool parse_chain_rf(const Buf* b, const PeImage* img, size_t fileoff_after_codes,`	Технически вызывает `parse_chain_rf`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `parse_chain_rf` для выполнения части алгоритма.
131	`RuntimeFunction* out_chain) {`	Технически эта строка является частью С-конструкции `RuntimeFunction* out_chain) {`. Логически она уточняет структуру блока функции `parse_chain_rf` и влияет на то, какие данные будут обработаны.
132	`uint32_t a=0, c=0, u=0;`	Технически объявляет a в функции `parse_chain_rf` и сразу задает начальное значение `0, c=0, u=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
133	`if (!buf_read_u32(b, fileoff_after_codes + 0, &a)) return false;`	Технически проверяет условие `!buf_read_u32(b, fileoff_after_codes + 0, &a)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
134	`if (!buf_read_u32(b, fileoff_after_codes + 4, &c)) return false;`	Технически проверяет условие `!buf_read_u32(b, fileoff_after_codes + 4, &c)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
135	`if (!buf_read_u32(b, fileoff_after_codes + 8, &u)) return false;`	Технически проверяет условие `!buf_read_u32(b, fileoff_after_codes + 8, &u)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
136	`out_chain->begin_rva = a;`	Технически присваивает `out_chain->begin_rva` значение a. Логически это обновляет `begin_rva`: начало функции в координатах RVA.
137	`out_chain->end_rva = c;`	Технически присваивает `out_chain->end_rva` значение c. Логически это обновляет `end_rva`: конец функции в координатах RVA.
138	`out_chain->unwind_rva = u;`	Технически присваивает `out_chain->unwind_rva` значение u. Логически это обновляет `unwind_rva`: RVA структуры UNWIND_INFO для этой функции.
139	`(void)img;`	Технически выполняет оператор `(void)img;`. Логически в функции `parse_chain_rf` он меняет состояние, проверяемые данные или подготовку отчета.
140	`return true;`	Технически возвращает `true` из функции `parse_chain_rf`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
141	`}`	Технически завершает область видимости внутри `parse_chain_rf`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
143	`static bool unwind_parse_fail(UnwindSummary* out,`	Технически вызывает `unwind_parse_fail`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `unwind_parse_fail` для выполнения части алгоритма.
144	`StrBuf* err,`	Технически эта строка является частью С-конструкции `StrBuf* err,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
145	`size_t err_start,`	Технически эта строка является частью С-конструкции `size_t err_start,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
146	`const char* fallback) {`	Технически эта строка является частью С-конструкции `const char* fallback) {`. Логически она уточняет структуру блока функции `unwind_parse_fail` и влияет на то, какие данные будут обработаны.
147	`if (!out) return false;`	Технически проверяет условие `!out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
148	`out->valid = false;`	Технически присваивает `out->valid` значение `false`. Логически это обновляет `valid`: хранит промежуточное значение, которое дальше используется в текущей функции.
150	`const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";`	Технически объявляет `message` в функции `unwind_parse_fail` и сразу задает начальное значение `fallback ? fallback : "UNWIND_INFO parse failed."`. Логически это место для данных: человекочитаемое объяснение finding.

Стр.	Код	Нормальное объяснение
151	`if (err && err->s && err->len > err_start) {`	Технически проверяет условие `err && err->s && err->len > err_start`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
152	`message = err->s + err_start;`	Технически присваивает `message` значение `err->s + err_start`. Логически это обновляет `message`: человекочитаемое объяснение finding.
153	`} else if (err && fallback) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.

Стр.	Код	Нормальное объяснение
154	`sb_appendf(err, "%s\n", fallback);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `unwind_parse_fail`: добавляет форматированный текст в динамический буфер.
155	`}`	Технически завершает область видимости внутри `unwind_parse_fail`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
157	`size_t n = strlen(message);`	Технически присваивает `size_t n` значение `strlen(message)`. Логически это обновляет `size_t n`: хранит размер или количество, поэтому используется в проверках границ.
158	`while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;`	Технически начинает цикл while с условием `n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
159	`if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;`	Технически проверяет условие `n >= sizeof(out->parse_error)) n = sizeof(out->parse_error`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
160	`memcpy(out->parse_error, message, n);`	Технически вызывает `memcpy`. Логически это действие нужно в функции `unwind_parse_fail`: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
161	`out->parse_error[n] = '\0';`	Технически присваивает `out->parse_error[n]` значение `'\0'`. Логически это обновляет `parse_error[n]`: хранит промежуточное значение, которое дальше используется в текущей функции.
162	`return false;`	Технически возвращает `false` из функции `unwind_parse_fail`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
163	`}`	Технически завершает область видимости внутри `unwind_parse_fail`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
165	`bool unwind_parse(const Buf* b,`	Технически вызывает `unwind_parse`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `unwind_parse` для выполнения части алгоритма.
166	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
167	`uint32_t unwind_rva,`	Технически эта строка является частью C-конструкции `uint32_t unwind_rva,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
168	`const UnwindParseOpts* opts,`	Технически эта строка является частью C-конструкции `const UnwindParseOpts* opts,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
169	`UnwindSummary* out,`	Технически эта строка является частью C-конструкции `UnwindSummary* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
170	`StrBuf* ops_text,`	Технически эта строка является частью C-конструкции `StrBuf* ops_text,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
171	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `unwind_parse` и влияет на то, какие данные будут обработаны.
172	`if (!b || !img || !out || !err) return false;`	Технически проверяет условие `!b || !img || !out || !err`; логически это защита от NULL-указателя и работы с неинициализированными данными.
173	`memset(out, 0, sizeof(*out));`	Технически вызывает `memset`. Логически это действие нужно в функции `unwind_parse`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
174	`size_t err_start = err->len;`	Технически объявляет `err_start` в функции `unwind_parse` и сразу задает начальное значение `err->len`. Логически это место для данных: накапливает текст ошибки для вызывающего кода.
175	`if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }`	Технически проверяет условие `ops_text) { ops_text->len = 0; if (ops_text->s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
177	`int max_depth = 8;`	Технически объявляет `max_depth` в функции `unwind_parse` и сразу задает начальное значение `8`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
178	`bool follow_chain = true;`	Технически объявляет `follow_chain` в функции `unwind_parse` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
179	`bool strict_abi = false;`	Технически объявляет `strict_abi` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
180	`if (opts) {`	Технически проверяет условие `opts`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
181	`if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;`	Технически проверяет условие `opts->max_chain_depth > 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
182	`follow_chain = opts->follow_chain;`	Технически присваивает `follow_chain` значение `opts->follow_chain`. Логически это обновляет `follow_chain`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
183	`strict_abi = opts->strict_abi;`	Технически присваивает `strict_abi` значение `opts->strict_abi`. Логически это обновляет `strict_abi`: хранит промежуточное значение, которое дальше используется в текущей функции.
184	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
185	`if (max_depth > 16) max_depth = 16;`	Технически проверяет условие `max_depth > 16`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
187	`uint8_t hdr[4];`	Технически объявляет `hdr` в функции `unwind_parse`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
188	`uint32_t uoff = 0;`	Технически объявляет `uoff` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
189	`if ((unwind_rva & 3u) != 0) {`	Технически проверяет условие `(unwind_rva & 3u) != 0`; логически отбирает записи, где важна корректность UNWIND_INFO.
190	`out->anomalies |= UA_ALIGNMENT;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ALIGNMENT`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
191	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
192	`if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {`	Технически проверяет условие `!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
193	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
194	`return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
195	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
197	`uint8_t ver = (uint8_t)(hdr[0] & 0x7);`	Технически присваивает `uint8_t ver` значение `(uint8_t)(hdr[0] & 0x7)`. Логически это обновляет `uint8_t ver`: хранит промежуточное значение, которое дальше используется в текущей функции.
198	`uint8_t flags = (uint8_t)(hdr[0] >> 3);`	Технически присваивает `uint8_t flags` значение `(uint8_t)(hdr[0] >> 3)`. Логически это обновляет `uint8_t flags`: хранит промежуточное значение, которое дальше используется в текущей функции.
199	`uint8_t prolog = hdr[1];`	Технически объявляет `prolog` в функции `unwind_parse` и сразу задает начальное значение `hdr[1]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
200	`uint8_t count = hdr[2];`	Технически объявляет `count` в функции `unwind_parse` и сразу задает начальное значение `hdr[2]`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
201	`uint8_t fr = (uint8_t)(hdr[3] & 0x0f);`	Технически присваивает `uint8_t fr` значение `(uint8_t)(hdr[3] & 0x0f)`. Логически это обновляет `uint8_t fr`: хранит промежуточное значение, которое дальше используется в текущей функции.
202	`uint8_t foff = (uint8_t)(hdr[3] >> 4);`	Технически присваивает `uint8_t foff` значение `(uint8_t)(hdr[3] >> 4)`. Логически это обновляет `uint8_t foff`: хранит промежуточное значение, которое дальше используется в текущей функции.
204	`out->version = ver;`	Технически присваивает `out->version` значение `ver`. Логически это обновляет `version`: версия UNWIND_INFO; для Windows x64 ожидается версия 1.
205	`out->flags = flags;`	Технически присваивает `out->flags` значение `flags`. Логически это обновляет `flags`: флаги UNWIND_INFO, включая наличие exception/termination handler.
206	`out->prolog_size = prolog;`	Технически присваивает `out->prolog_size` значение `prolog`. Логически это обновляет `prolog_size`: длина пролога функции в байтах.
207	`out->count_codes = count;`	Технически присваивает `out->count_codes` значение `count`. Логически это обновляет `count_codes`: хранит размер или количество, поэтому используется в проверках границ.
208	`out->frame_reg = fr;`	Технически присваивает `out->frame_reg` значение `fr`. Логически это обновляет `frame_reg`: хранит промежуточное значение, которое дальше используется в текущей функции.
209	`out->frame_off = foff;`	Технически присваивает `out->frame_off` значение `foff`. Логически это обновляет `frame_off`: хранит промежуточное значение, которое дальше используется в текущей функции.
211	`if (!(ver == 1 || ver == 2)) {`	Технически проверяет условие `!(ver == 1 || ver == 2)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
212	`out->anomalies |= UA_VERSION_UNKNOWN;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_VERSION_UNKNOWN`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
213	`return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
214	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
215	`if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {`	Технически проверяет условие `flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)`; логически отделяет функции с language-specific handler от обычных функций.
216	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_BAD_OPERAND`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
217	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
218	`if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {`	Технически проверяет условие `(flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))`; логически отделяет функции с language-specific handler от обычных функций.
219	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_BAD_OPERAND`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
220	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
221	`if (fr == 0 && foff != 0) {`	Технически проверяет условие `fr == 0 && foff != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
222	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_BAD_OPERAND`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
223	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
224	`if (fr != 0 && !is_unwindable_gpr(fr)) {`	Технически проверяет условие `fr != 0 && !is_unwindable_gpr(fr)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
225	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_BAD_OPERAND`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
226	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
227	`if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {`	Технически проверяет условие `strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
228	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
229	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
231	`/* total slots stored on disk (aligned to even number) */`	Технически это комментарий исходника. Логически он объясняет разработчику: total slots stored on disk (aligned to even number).
232	`uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);`	Технически присваивает `uint32_t slots_even` значение `(uint32_t)((count + 1u) & ~1u)`. Логически это обновляет `uint32_t slots_even`: хранит промежуточное значение, которое дальше используется в текущей функции.
233	`size_t codes_off = (size_t)uoff + 4;`	Технически присваивает `size_t codes_off` значение `(size_t)uoff + 4`. Логически это обновляет `size_t codes_off`: хранит размер или количество, поэтому используется в проверках границ.
234	`size_t codes_bytes = (size_t)slots_even * 2u;`	Технически присваивает `size_t codes_bytes` значение `(size_t)slots_even * 2u`. Логически это обновляет `size_t codes_bytes`: хранит размер или количество, поэтому используется в проверках границ.
235	`if (codes_off + codes_bytes > b->size) {`	Технически проверяет условие `codes_off + codes_bytes > b->size`; логически это защита от чтения за пределами файла на диске.
236	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
237	`sb_append(err, "Unwind codes array out of file range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_parse`: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
238	`return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file range.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
239	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
241	`uint32_t fixed_stack = 0;`	Технически объявляет `fixed_stack` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
242	`uint32_t pushed = 0;`	Технически объявляет `pushed` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
243	`uint16_t nonvol_mask = 0;`	Технически объявляет `nonvol_mask` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
244	`uint16_t xmm_mask = 0;`	Технически объявляет `xmm_mask` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
245	`bool have_prev_code_offset = false;`	Технически объявляет `have_prev_code_offset` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
246	`uint8_t prev_code_offset = 0;`	Технически объявляет `prev_code_offset` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
247	`bool saw_set_fpreg = false;`	Технически объявляет `saw_set_fpreg` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
248	`bool saw_push_nonvol = false;`	Технически объявляет `saw_push_nonvol` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
249	`bool v2_seen_epilog = false;`	Технически объявляет `v2_seen_epilog` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
251	`/* parse slots 0..count-1 as unwind code starts */`	Технически это комментарий исходника. Логически он объясняет разработчику: parse slots 0..count-1 as unwind code starts.
252	`for (uint32_t i = 0; i < count; ) {`	Технически начинает цикл `uint32_t i = 0; i < count;`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
253	`uint16_t slot = 0;`	Технически объявляет `slot` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
254	`if (!read_u16_slot(b, codes_off + (size_t)i * 2, &slot)) {`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)i * 2, &slot)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
255	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
256	`sb_append(err, "Failed to read unwind slot.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_parse`: добавляет текст в динамический буфер.
257	`return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
258	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
260	`uint8_t code_offset = (uint8_t)(slot & 0xff);`	Технически присваивает `uint8_t code_offset` значение `(uint8_t)(slot & 0xff)`. Логически это обновляет `uint8_t code_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
261	`uint8_t op_byte = (uint8_t)(slot >> 8);`	Технически присваивает `uint8_t op_byte` значение `(uint8_t)(slot >> 8)`. Логически это обновляет `uint8_t op_byte`: хранит промежуточное значение, которое дальше используется в текущей функции.
262	`uint8_t op = (uint8_t)(op_byte & 0x0f);`	Технически присваивает `uint8_t op` значение `(uint8_t)(op_byte & 0x0f)`. Логически это обновляет `uint8_t op`: хранит промежуточное значение, которое дальше используется в текущей функции.
263	`uint8_t opinfo = (uint8_t)(op_byte >> 4);`	Технически присваивает `uint8_t opinfo` значение `(uint8_t)(op_byte >> 4)`. Логически это обновляет `uint8_t opinfo`: хранит промежуточное значение, которое дальше используется в текущей функции.
265	`uint8_t slots = 0;`	Технически объявляет `slots` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
266	`if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {`	Технически проверяет условие `!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
267	`out->anomalies |= UA_BAD_OPCODE;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_BAD_OPCODE`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
268	`/* try to continue safely as 1 slot */`	Технически это комментарий исходника. Логически он объясняет разработчику: try to continue safely as 1 slot.
269	`slots = 1;`	Технически присваивает `slots` значение `1`. Логически это обновляет `slots`: хранит промежуточное значение, которое дальше используется в текущей функции.
270	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
271	`if (i + slots > count) {`	Технически проверяет условие `i + slots > count`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
272	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
273	`sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_parse`: добавляет текст в динамический буфер.
274	`return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond CountOfCodes.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
275	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
276	`bool v2_descriptor = ver == 2 && (op == 6 || op == 7);`	Технически присваивает `bool v2_descriptor` значение `ver == 2 && (op == 6 || op == 7)`. Логически это обновляет `bool v2_descriptor`: хранит промежуточное значение, которое дальше используется в текущей функции.
277	`if (!v2_descriptor) {`	Технически проверяет условие `!v2_descriptor`; логически это защита от NULL-указателя и работы с неинициализированными данными.
278	`if (code_offset > prolog) {`	Технически проверяет условие `code_offset > prolog`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
279	`out->anomalies |= UA_CODEOFFSET_OUTSIDE;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_CODEOFFSET_OUTSIDE`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
280	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
281	`if (have_prev_code_offset && code_offset > prev_code_offset) {`	Технически проверяет условие `have_prev_code_offset && code_offset > prev_code_offset`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
282	`out->anomalies |= UA_CODEOFFSET_ORDER;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_CODEOFFSET_ORDER`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
283	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
284	`have_prev_code_offset = true;`	Технически присваивает `have_prev_code_offset` значение `true`. Логически это обновляет `have_prev_code_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
285	`prev_code_offset = code_offset;`	Технически присваивает `prev_code_offset` значение `code_offset`. Логически это обновляет `prev_code_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
286	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
287	`if (strict_abi && !v2_descriptor) {`	Технически проверяет условие `strict_abi && !v2_descriptor`; логически это защита от NULL-указателя и работы с неинициализированными данными.
288	`if (saw_push_nonvol && op != 0 && op != 10) {`	Технически проверяет условие `saw_push_nonvol && op != 0 && op != 10`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
289	`out->anomalies |= UA_ABI_PUSH_ORDER;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_PUSH_ORDER`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
290	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
291	`if (op == 0) saw_push_nonvol = true;`	Технически проверяет условие `op == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
292	`if ((flags & UNW_FLAG_CHAININFO) && op != 4) {`	Технически проверяет условие `(flags & UNW_FLAG_CHAININFO) && op != 4`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
293	`out->anomalies |= UA_ABI_CHAIN_RESTRICTION;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_CHAIN_RESTRICTION`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
294	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
295	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
297	`if (ops_text) {`	Технически проверяет условие `ops_text`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
298	`/* print one-line per op start */`	Технически это комментарий исходника. Логически он объясняет разработчику: print one-line per op start.
299	`switch (op) {`	Технически выбирает ветку по значению `op`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.
300	`case 0: sb_appendf(ops_text, " @%02x PUSH_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;`	Технически объявляет ветку `switch` для значения `0: sb_appendf(ops_text, " @%02x PUSH_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
301	`case 1: sb_appendf(ops_text, " @%02x ALLOC_LARGE mode=%u\n", code_offset, (unsigned)opinfo); break;`	Технически объявляет ветку `switch` для значения `1: sb_appendf(ops_text, " @%02x ALLOC_LARGE mode=%u\n", code_offset, (unsigned)opinfo);...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
302	`case 2: sb_appendf(ops_text, " @%02x ALLOC_SMALL %u\n", code_offset, (unsigned)(opinfo*8u + 8u)); break;`	Технически объявляет ветку `switch` для значения `2: sb_appendf(ops_text, " @%02x ALLOC_SMALL %u\n", code_offset, (unsigned)(opinfo*8u + ...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
303	`case 3: sb_appendf(ops_text, " @%02x SET_FPREG %s + %u\n", code_offset, unwind_reg_name(fr), (unsigned)(foff*16u)); break;`	Технически объявляет ветку `switch` для значения `3: sb_appendf(ops_text, " @%02x SET_FPREG %s + %u\n", code_offset, unwind_reg_name(fr),...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
304	`case 4: sb_appendf(ops_text, " @%02x SAVE_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;`	Технически объявляет ветку `switch` для значения `4: sb_appendf(ops_text, " @%02x SAVE_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
305	`case 5: sb_appendf(ops_text, " @%02x SAVE_NONVOL_FAR %s\n", code_offset, unwind_reg_name(opinfo)); break;`	Технически объявляет ветку `switch` для значения `5: sb_appendf(ops_text, " @%02x SAVE_NONVOL_FAR %s\n", code_offset, unwind_reg_name(opi...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
306	`case 6:`	Технически объявляет ветку `switch` для значения `6`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
307	`if (ver == 2) {`	Технически проверяет условие `ver == 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
308	`if (!v2_seen_epilog) {`	Технически проверяет условие `!v2_seen_epilog`; логически это защита от NULL-указателя и работы с неинициализированными данными.
309	`if ((opinfo & 1u) == 0u) {`	Технически проверяет условие `(opinfo & 1u) == 0u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
310	`uint16_t epilog_slot = 0;`	Технически объявляет `epilog_slot` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
311	`if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,`	Технически проверяет условие `read_u16_slot(b, codes_off + (size_t)(i + 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
312	`&epilog_slot)) {`	Технически выполняет битовую операцию `&epilog_slot)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
313	`unsigned epilog_offset =`	Технически присваивает `unsigned epilog_offset` значение `` `. Логически это обновляет unsigned epilog_offset` ``: хранит промежуточное значение, которое дальше используется в текущей функции.
314	`(unsigned)(epilog_slot & 0xffu) |`	Технически выполняет битовую операцию `(unsigned)(epilog_slot & 0xffu) |`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
315	`((unsigned)(epilog_slot >> 12) << 8);`	Технически выполняет битовую операцию `((unsigned)(epilog_slot >> 12) << 8);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
316	`sb_appendf(ops_text,`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `unwind_parse`: добавляет форматированный текст в динамический буфер.
317	`" EPILOG size=%u offset_from_end=%u flags=%u (v2)\n",`	Технически присваивает `" EPILOG size` значение `%u offset_from_end=%u flags=%u (v2)\n",`. Логически это обновляет `" EPILOG size`: хранит промежуточное значение, которое дальше используется в текущей функции.
318	`(unsigned)code_offset, epilog_offset,`	Технически эта строка является частью C-конструкции `(unsigned)code_offset, epilog_offset,`. Логически она уточняет структуру блока функции `unwind_parse` и влияет на то, какие данные будут обработаны.
319	`(unsigned)opinfo);`	Технически выполняет оператор `(unsigned)opinfo);`. Логически в функции `unwind_parse` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
320	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
321	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
322	`sb_appendf(ops_text, " EPILOG size=%u flags=%u (v2)\n",`	Технически присваивает `sb_appendf(ops_text, " EPILOG size` значение `%u flags=%u (v2)\n",`. Логически это обновляет `sb_appendf(ops_text, " EPILOG size`: хранит промежуточное значение, которое дальше используется в текущей функции.
323	`(unsigned)code_offset, (unsigned)opinfo);`	Технически выполняет оператор `(unsigned)code_offset, (unsigned)opinfo);`. Логически в функции `unwind_parse` он меняет состояние, проверяемые данные или подготовку отчета.
324	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
325	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
326	`unsigned epilog_offset =`	Технически присваивает `unsigned epilog_offset` значение `` `. Логически это обновляет unsigned epilog_offset` ``: хранит промежуточное значение, которое дальше используется в текущей функции.
327	`(unsigned)code_offset | ((unsigned)opinfo << 8);`	Технически выполняет битовую операцию `(unsigned)code_offset | ((unsigned)opinfo << 8);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
328	`sb_appendf(ops_text, " EPILOG offset_from_end=%u (v2)\n",`	Технически присваивает `sb_appendf(ops_text, " EPILOG offset_from_end` значение `%u (v2)\n",`. Логически это обновляет `sb_appendf(ops_text, " EPILOG offset_from_end`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
329	`epilog_offset);`	Технически выполняет оператор `epilog_offset);`. Логически в функции `unwind_parse` он меняет состояние, проверяемые данные или подготовку отчета.
330	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
331	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
332	`sb_appendf(ops_text, " @%02x SAVE_XMM (undoc/v1)\n", code_offset);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `unwind_parse`: добавляет форматированный текст в динамический буфер.
333	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
334	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
335	`case 7:`	Технически объявляет ветку `switch` для значения `7`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
336	`if (ver == 2) sb_appendf(ops_text, " @%02x SPARE (v2)\n", code_offset);`	Технически проверяет условие `ver == 2) sb_appendf(ops_text, " @%02x SPARE (v2)\n", code_offset`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
337	`else sb_appendf(ops_text, " @%02x SAVE_XMM_FAR (undoc/v1)\n", code_offset);`	Технически объявляет прототип функции `sb_appendf` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет форматированный текст в StrBuf.
338	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
339	`case 8: sb_appendf(ops_text, " @%02x SAVE_XMM128 XMM%u\n", code_offset, (unsigned)opinfo); break;`	Технически объявляет ветку `switch` для значения `8: sb_appendf(ops_text, " @%02x SAVE_XMM128 XMM%u\n", code_offset, (unsigned)opinfo); b...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
340	`case 9: sb_appendf(ops_text, " @%02x SAVE_XMM128_FAR XMM%u\n", code_offset, (unsigned)opinfo); break;`	Технически объявляет ветку `switch` для значения `9: sb_appendf(ops_text, " @%02x SAVE_XMM128_FAR XMM%u\n", code_offset, (unsigned)opinfo...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
341	`case 10: sb_appendf(ops_text, " @%02x PUSH_MACHFRAME err=%u\n", code_offset, (unsigned)opinfo); break;`	Технически объявляет ветку `switch` для значения `10: sb_appendf(ops_text, " @%02x PUSH_MACHFRAME err=%u\n", code_offset, (unsigned)opinf...`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
342	`default: sb_appendf(ops_text, " @%02x OP_%u info=%u\n", code_offset, (unsigned)op, (unsigned)opinfo); break;`	Технически объявляет ветку `switch` по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
343	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
344	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
346	`/* compute effects */`	Технически это комментарий исходника. Логически он объясняет разработчику: compute effects.
347	`switch (op) {`	Технически выбирает ветку по значению `op`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.

Стр.	Код	Нормальное объяснение
348	`case 0: /* push nonvol */`	Технически объявляет ветку switch для значения `0: /* push nonvol */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
349	`add_fixed_stack(&fixed_stack, 8, &out->anomalies);`	Технически вызывает add_fixed_stack. Логически это действие нужно в функции unwind_parse: вызывает функцию add_fixed_stack для выполнения части алгоритма.
350	`pushed++;`	Технически увеличивает pushed на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
351	`if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
352	`if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
353	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_ABI_REGISTER_CLASS. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
354	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
355	`if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
356	`break;`	Технически завершает текущий case или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
357	`case 1: { /* alloc large */`	Технически объявляет ветку switch для значения `1: { /* alloc large */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
358	`if (opinfo == 0) {`	Технически проверяет условие `opinfo == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
359	`uint16_t scaled = 0;`	Технически объявляет scaled в функции unwind_parse и сразу задает начальное значение 0. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
360	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
361	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_TRUNCATED. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
362	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");`	Технически объявляет прототип функции unwind_parse_fail без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
363	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
364	`add_fixed_stack(&fixed_stack, (uint32_t)scaled * 8u, &out->anomalies);`	Технически вызывает add_fixed_stack. Логически это действие нужно в функции unwind_parse: вызывает функцию add_fixed_stack для выполнения части алгоритма.
365	`if (alloc_large_mode0_suspicious(scaled)) out->anomalies |= UA_OPERAND_SANITY;`	Технически проверяет условие `alloc_large_mode0_suspicious(scaled)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
366	`if (ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)((uint32_t)scaled * 8u));`	Технически проверяет условие `ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)((uint32_t)scaled * 8u)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
367	`} else if (opinfo == 1) {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
368	`uint16_t lo=0, hi=0;`	Технически объявляет lo в функции unwind_parse и сразу задает начальное значение `0, hi=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
369	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo`; логически это защита от NULL-указателя и работы с неинициализированными данными.
370	`!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {`	Технически объявляет/начинает функцию read_u16_slot; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
371	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_TRUNCATED. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
372	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");`	Технически объявляет прототип функции unwind_parse_fail без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
373	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
374	`uint32_t size = read_u32_from_slots(lo, hi);`	Технически присваивает `uint32_t size` значение `read_u32_from_slots(lo, hi)`. Логически это обновляет `uint32_t size`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
375	`add_fixed_stack(&fixed_stack, size, &out->anomalies);`	Технически вызывает `add_fixed_stack`. Логически это действие нужно в функции `unwind_parse`: вызывает функцию `add_fixed_stack` для выполнения части алгоритма.
376	`if (alloc_large_mode1_suspicious(size)) out->anomalies |= UA_OPERAND_SANITY;`	Технически проверяет условие `alloc_large_mode1_suspicious(size)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
377	`if (ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)size);`	Технически проверяет условие `ops_text) sb_appendf(ops_text, " size=%u\n", (unsigned)size`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
378	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
379	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_BAD_OPERAND`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
380	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
381	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
382	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
383	`case 2: /* alloc small */`	Технически объявляет ветку `switch` для значения `2: /* alloc small */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
384	`add_fixed_stack(&fixed_stack, (uint32_t)opinfo * 8u + 8u, &out->anomalies);`	Технически вызывает `add_fixed_stack`. Логически это действие нужно в функции `unwind_parse`: вызывает функцию `add_fixed_stack` для выполнения части алгоритма.
385	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
386	`case 3: /* set frame reg */`	Технически объявляет ветку `switch` для значения `3: /* set frame reg */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
387	`if (fr == 0 || saw_set_fpreg) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `fr == 0 || saw_set_fpreg`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
388	`if (strict_abi && opinfo != 0) {`	Технически проверяет условие `strict_abi && opinfo != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
389	`out->anomalies |= UA_ABI_RESERVED_OPINFO;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_RESERVED_OPINFO`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
390	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
391	`saw_set_fpreg = true;`	Технически присваивает `saw_set_fpreg` значение `true`. Логически это обновляет `saw_set_fpreg`: хранит промежуточное значение, которое дальше используется в текущей функции.
392	`/* no fixed stack delta */`	Технически это комментарий исходника. Логически он объясняет разработчику: no fixed stack delta.
393	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
394	`case 4: { /* save nonvol */`	Технически объявляет ветку `switch` для значения `4: { /* save nonvol */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
395	`uint16_t scaled = 0;`	Технически объявляет `scaled` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
396	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
397	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
398	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
399	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
400	`if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
401	`if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
402	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
403	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
404	`if (strict_abi && scaled == 0) {`	Технически проверяет условие `strict_abi && scaled == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
405	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
406	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
407	`if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
408	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
409	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
410	`case 5: { /* save nonvol far */`	Технически объявляет ветку `switch` для значения `5: { /* save nonvol far */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
411	`uint16_t lo=0, hi=0;`	Технически объявляет `lo` в функции `unwind_parse` и сразу задает начальное значение `0, hi=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
412	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo;` логически это защита от NULL-указателя и работы с неинициализированными данными.
413	`!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {`	Технически объявляет/начинает функцию `read_u16_slot`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
414	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
415	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL_FAR operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
416	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
417	`uint32_t save_offset = read_u32_from_slots(lo, hi);`	Технически присваивает `uint32_t save_offset` значение `read_u32_from_slots(lo, hi)`. Логически это обновляет `uint32_t save_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
418	`if (save_nonvol_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;`	Технически проверяет условие `save_nonvol_far_suspicious(save_offset)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
419	`if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
420	`if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_gpr(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
421	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
422	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
423	`if (strict_abi && save_offset == 0) {`	Технически проверяет условие `strict_abi && save_offset == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
424	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
425	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
426	`if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
427	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
428	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
429	`case 6: { /* save xmm (v1, undocumented encoding) or epilog (v2) */`	Технически объявляет ветку switch для значения `6: { /* save xmm (v1, undocumented encoding) or epilog (v2) */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
430	`if (ver != 2) {`	Технически проверяет условие `ver != 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
431	`uint16_t scaled = 0;`	Технически объявляет `scaled` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
432	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
433	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
434	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
435	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
436	`if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
437	`if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
438	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
439	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
440	`if (strict_abi && scaled == 0) {`	Технически проверяет условие `strict_abi && scaled == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
441	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
442	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
443	`if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
444	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
445	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
446	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
447	`case 7: { /* save xmm far (v1, undocumented encoding) or spare (v2) */`	Технически объявляет ветку switch для значения `7: { /* save xmm far (v1, undocumented encoding) or spare (v2) */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
448	`if (ver != 2) {`	Технически проверяет условие `ver != 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
449	`uint16_t lo=0, hi=0;`	Технически объявляет `lo` в функции `unwind_parse` и сразу задает начальное значение `0, hi=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
450	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo`; логически это защита от NULL-указателя и работы с неинициализированными данными.
451	`!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {`	Технически объявляет/начинает функцию `read_u16_slot`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
452	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
453	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM_FAR operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
454	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
455	`uint32_t save_offset = read_u32_from_slots(lo, hi);`	Технически присваивает `uint32_t save_offset` значение `read_u32_from_slots(lo, hi)`. Логически это обновляет `uint32_t save_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
456	`if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;`	Технически проверяет условие `save_xmm_far_suspicious(save_offset)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
457	`if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
458	`if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
459	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
460	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
461	`if (strict_abi && save_offset == 0) {`	Технически проверяет условие `strict_abi && save_offset == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
462	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
463	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
464	`if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
465	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
466	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
467	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
468	`case 8: { /* save xmm128 */`	Технически объявляет ветку `switch` для значения `8: { /* save xmm128 */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
469	`uint16_t scaled = 0;`	Технически объявляет `scaled` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
470	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
471	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
472	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128 operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
473	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
474	`if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
475	`if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
476	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
477	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
478	`if (strict_abi && scaled == 0) {`	Технически проверяет условие `strict_abi && scaled == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
479	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
480	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
481	`if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
482	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
483	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
484	`case 9: { /* save xmm128 far */`	Технически объявляет ветку `switch` для значения `9: { /* save xmm128 far */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
485	`uint16_t lo=0, hi=0;`	Технически объявляет `lo` в функции `unwind_parse` и сразу задает начальное значение `0, hi=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
486	`if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||`	Технически проверяет условие `!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo;` логически это защита от NULL-указателя и работы с неинициализированными данными.
487	`!read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {`	Технически объявляет/начинает функцию `read_u16_slot;` она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
488	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
489	`return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128_FAR operand.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
490	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
491	`uint32_t save_offset = read_u32_from_slots(lo, hi);`	Технически присваивает `uint32_t save_offset` значение `read_u32_from_slots(lo, hi)`. Логически это обновляет `uint32_t save_offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
492	`if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;`	Технически проверяет условие `save_xmm_far_suspicious(save_offset)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
493	`if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `!is_unwindable_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
494	`if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {`	Технически проверяет условие `strict_abi && !is_abi_nonvolatile_xmm(opinfo)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
495	`out->anomalies |= UA_ABI_REGISTER_CLASS;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_REGISTER_CLASS`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
496	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
497	`if (strict_abi && save_offset == 0) {`	Технически проверяет условие `strict_abi && save_offset == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
498	`out->anomalies |= UA_ABI_SAVE_OFFSET;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_ABI_SAVE_OFFSET`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
499	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
500	`if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);`	Технически проверяет условие `opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
501	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
502	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
503	`case 10: /* push machframe */`	Технически объявляет ветку `switch` для значения `10: /* push machframe */`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
504	`if (opinfo > 1) out->anomalies |= UA_BAD_OPERAND;`	Технически проверяет условие `opinfo > 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
505	`add_fixed_stack(&fixed_stack, (opinfo == 0) ? 40u : 48u, &out->anomalies);`	Технически вызывает `add_fixed_stack`. Логически это действие нужно в функции `unwind_parse`: вызывает функцию `add_fixed_stack` для выполнения части алгоритма.
506	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
507	`default:`	Технически объявляет ветку `switch` по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
508	`/* ignore */`	Технически это комментарий исходника. Логически он объясняет разработчику: ignore.

Стр.	Код	Нормальное объяснение
509	`break;`	Технически завершает текущий case или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
510	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
512	`if (ver == 2 && op == 6) v2_seen_epilog = true;`	Технически проверяет условие ver == 2 && op == 6; если оно истинно, выполняется защитная или аналитическая ветка ниже.
513	`i += slots;`	Технически изменяет i оператором += на основе slots. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
514	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
515	`if (fr != 0 && !saw_set_fpreg && !(flags & UNW_FLAG_CHAININFO)) {`	Технически проверяет условие fr != 0 && !saw_set_fpreg && !(flags & UNW_FLAG_CHAININFO); логически это защита от NULL-указателя и работы с неинициализированными данными.
516	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_BAD_OPERAND. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
517	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
519	`/* tail (handler or chained info) */`	Технически это комментарий исходника. Логически он объясняет разработчику: tail (handler or chained info).
520	`size_t after_codes = codes_off + codes_bytes;`	Технически объявляет after_codes в функции unwind_parse и сразу задает начальное значение codes_off + codes_bytes. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
522	`if (flags & UNW_FLAG_CHAININFO) {`	Технически проверяет условие flags & UNW_FLAG_CHAININFO; если оно истинно, выполняется защитная или аналитическая ветка ниже.
523	`out->has_chain = true;`	Технически присваивает out->has_chain значение true. Логически это обновляет has_chain: хранит булев признак, от которого зависит ветка алгоритма.
524	`RuntimeFunction chain = {0};`	Технически объявляет chain в функции unwind_parse и сразу задает начальное значение {0}. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
525	`if (!parse_chain_rf(b, img, after_codes, &chain)) {`	Технически проверяет условие !parse_chain_rf(b, img, after_codes, &chain); логически это защита от NULL-указателя и работы с неинициализированными данными.
526	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_TRUNCATED. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
527	`return unwind_parse_fail(out, err, err_start, "Truncated CHAININFO RUNTIME_FUNCTION.");`	Технически объявляет прототип функции unwind_parse_fail без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
528	`} else {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
529	`out->chain = chain;`	Технически присваивает out->chain значение chain. Логически это обновляет chain: хранит промежуточное значение, которое дальше используется в текущей функции.
530	`if (chain.end_rva <= chain.begin_rva ||`	Технически проверяет условие if (chain.end_rva <= chain.begin_rva ||; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
531	`chain.begin_rva >= img->size_of_image ||`	Технически выполняет битовую операцию chain.begin_rva >= img->size_of_image ||. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
532	`chain.end_rva > img->size_of_image ||`	Технически выполняет битовую операцию chain.end_rva > img->size_of_image ||. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
533	`chain.unwind_rva == 0 ||`	Технически выполняет битовую операцию chain.unwind_rva == 0 ||. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
534	`chain.unwind_rva >= img->size_of_image) {`	Технически эта строка является частью С-конструкции chain.unwind_rva >= img->size_of_image) {. Логически она уточняет структуру блока функции unwind_parse и влияет на то, какие данные будут обработаны.
535	`out->anomalies |= UA_CHAIN_INVALID;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_CHAIN_INVALID. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
536	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
537	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
538	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
540	`if (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)) {`	Технически проверяет условие `flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)`; логически отделяет функции с language-specific handler от обычных функций.
541	`out->has_handler = true;`	Технически присваивает `out->has_handler` значение `true`. Логически это обновляет `has_handler`: показывает, есть ли EHANDLER/UHANDLER.
542	`uint32_t handler_rva = 0;`	Технически объявляет `handler_rva` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: адрес language-specific handler, если он указан в UNWIND_INFO.
543	`if (!buf_read_u32(b, after_codes + 0, &handler_rva)) {`	Технически проверяет условие `!buf_read_u32(b, after_codes + 0, &handler_rva)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
544	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_TRUNCATED`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
545	`return unwind_parse_fail(out, err, err_start, "Truncated exception handler RVA.");`	Технически объявляет прототип функции `unwind_parse_fail` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
546	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
547	`out->handler_rva = handler_rva;`	Технически присваивает `out->handler_rva` значение `handler_rva`. Логически это обновляет `handler_rva`: адрес language-specific handler, если он указан в UNWIND_INFO.
548	`if (handler_rva >= img->size_of_image) {`	Технически проверяет условие `handler_rva >= img->size_of_image`; логически это защита от выхода RVA/диапазона за SizeOfImage.
549	`out->anomalies |= UA_HANDLER_OOB;`	Технически применяет битовую операцию `|=` к `out->anomalies` со значением `UA_HANDLER_OOB`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
550	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
551	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
552	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
554	`out->fixed_stack_bytes = fixed_stack;`	Технически присваивает `out->fixed_stack_bytes` значение `fixed_stack`. Логически это обновляет `fixed_stack_bytes`: хранит промежуточное значение, которое дальше используется в текущей функции.
555	`out->pushed_nonvol_count = pushed;`	Технически присваивает `out->pushed_nonvol_count` значение `pushed`. Логически это обновляет `pushed_nonvol_count`: хранит размер или количество, поэтому используется в проверках границ.
556	`out->saved_nonvol_mask = nonvol_mask;`	Технически присваивает `out->saved_nonvol_mask` значение `nonvol_mask`. Логически это обновляет `saved_nonvol_mask`: хранит промежуточное значение, которое дальше используется в текущей функции.
557	`out->saved_xmm_mask = xmm_mask;`	Технически присваивает `out->saved_xmm_mask` значение `xmm_mask`. Логически это обновляет `saved_xmm_mask`: битовая маска сохраненных nonvolatile XMM регистров.
559	`/* follow chain (lightly): detect loops and depth. */`	Технически это комментарий исходника. Логически он объясняет разработчику: follow chain (lightly): detect loops and depth..
560	`if (follow_chain && out->has_chain) {`	Технически проверяет условие `follow_chain && out->has_chain`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
561	`uint32_t seen[16]; /* depth max 16 */`	Технически эта строка является частью C-конструкции `uint32_t seen[16]; /* depth max 16 */`. Логически она уточняет структуру блока функции `unwind_parse` и влияет на то, какие данные будут обработаны.
562	`int seen_n = 0;`	Технически объявляет `seen_n` в функции `unwind_parse` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
563	`uint32_t cur = out->chain.unwind_rva;`	Технически объявляет `cur` в функции `unwind_parse` и сразу задает начальное значение `out->chain.unwind_rva`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
564	`for (int depth = 0; depth < max_depth; depth++) {`	Технически начинает цикл `int depth = 0; depth < max_depth; depth++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
565	`bool loop = false;`	Технически объявляет `loop` в функции `unwind_parse` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
566	`for (int j = 0; j < seen_n; j++) if (seen[j] == cur) loop = true;`	Технически начинает цикл `int j = 0; j < seen_n; j++) if (seen[j] == cur`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
567	`if (loop) { out->anomalies |= UA_CHAIN_LOOP; break; }`	Технически проверяет условие loop; если оно истинно, выполняется защитная или аналитическая ветка ниже.
568	`if (seen_n < (int)(sizeof(seen)/sizeof(seen[0]))) seen[seen_n++] = cur;`	Технически проверяет условие `seen_n < (int)(sizeof(seen)/sizeof(seen[0]))`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
570	`UnwindSummary tmp;`	Технически объявляет tmp в функции unwind_parse. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
571	`StrBuf tmp_ops = {0};`	Технически объявляет tmp_ops в функции unwind_parse и сразу задает начальное значение {0}. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
572	`StrBuf tmp_err = {0};`	Технически объявляет tmp_err в функции unwind_parse и сразу задает начальное значение {0}. Логически это место для данных: накапливает текст ошибки для вызывающего кода.
573	`sb_init(&tmp_err, 256);`	Технически вызывает sb_init. Логически это действие нужно в функции unwind_parse: создает динамический текстовый буфер.
574	`sb_init(&tmp_ops, 256);`	Технически вызывает sb_init. Логически это действие нужно в функции unwind_parse: создает динамический текстовый буфер.
575	`UnwindParseOpts o2 = {`	Технически присваивает UnwindParseOpts o2 значение {. Логически это обновляет UnwindParseOpts o2: хранит промежуточное значение, которое дальше используется в текущей функции.
576	`.max_chain_depth = 0,`	Технически присваивает .max_chain_depth значение 0,. Логически это обновляет max_chain_depth: хранит промежуточное значение, которое дальше используется в текущей функции.
577	`.follow_chain = false,`	Технически присваивает .follow_chain значение false,. Логически это обновляет follow_chain: хранит промежуточное значение, которое дальше используется в текущей функции.
578	`.strict_abi = strict_abi`	Технически присваивает .strict_abi значение strict_abi. Логически это обновляет strict_abi: хранит промежуточное значение, которое дальше используется в текущей функции.
579	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
580	`if (!unwind_parse(b, img, cur, &o2, &tmp, NULL, &tmp_err)) {`	Технически проверяет условие !unwind_parse(b, img, cur, &o2, &tmp, NULL, &tmp_err); логически это защита от NULL-указателя и работы с неинициализированными данными.
581	`out->anomalies |= UA_TRUNCATED;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_TRUNCATED. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
582	`sb_free(&tmp_err);`	Технически вызывает sb_free. Логически это действие нужно в функции unwind_parse: освобождает память динамического буфера.
583	`sb_free(&tmp_ops);`	Технически вызывает sb_free. Логически это действие нужно в функции unwind_parse: освобождает память динамического буфера.
584	`break;`	Технически завершает текущий case или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
585	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
586	`/* combine fixed stack heuristically */`	Технически это комментарий исходника. Логически он объясняет разработчику: combine fixed stack heuristically.
587	`add_fixed_stack(&out->fixed_stack_bytes, tmp.fixed_stack_bytes, &out->anomalies);`	Технически вызывает add_fixed_stack. Логически это действие нужно в функции unwind_parse: вызывает функцию add_fixed_stack для выполнения части алгоритма.
588	`out->pushed_nonvol_count += tmp.pushed_nonvol_count;`	Технически изменяет out->pushed_nonvol_count оператором += на основе tmp.pushed_nonvol_count. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
589	`out->saved_nonvol_mask |= tmp.saved_nonvol_mask;`	Технически применяет битовую операцию |= к out->saved_nonvol_mask со значением tmp.saved_nonvol_mask. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
590	`out->saved_xmm_mask |= tmp.saved_xmm_mask;`	Технически применяет битовую операцию |= к out->saved_xmm_mask со значением tmp.saved_xmm_mask. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
591	`if (tmp.frame_reg != out->frame_reg || tmp.frame_off != out->frame_off) {`	Технически проверяет условие tmp.frame_reg != out->frame_reg || tmp.frame_off != out->frame_off; если оно истинно, выполняется защитная или аналитическая ветка ниже.
592	`out->anomalies |= UA_BAD_OPERAND;`	Технически применяет битовую операцию |= к out->anomalies со значением UA_BAD_OPERAND. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
593	`}`	Технически завершает область видимости внутри unwind_parse. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
594	`if (!out->has_handler && tmp.has_handler) {`	Технически проверяет условие !out->has_handler && tmp.has_handler; логически это защита от NULL-указателя и работы с неинициализированными данными.
595	`out->has_handler = true;`	Технически присваивает out->has_handler значение true. Логически это обновляет has_handler: показывает, есть ли EHANDLER/UHANDLER.

Стр.	Код	Нормальное объяснение
596	`out->handler_rva = tmp.handler_rva;`	Технически присваивает `out->handler_rva` значение `tmp.handler_rva`. Логически это обновляет `handler_rva`: адрес language-specific handler, если он указан в UNWIND_INFO.
597	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
598	`sb_free(&tmp_err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `unwind_parse`: освобождает память динамического буфера.
599	`sb_free(&tmp_ops);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `unwind_parse`: освобождает память динамического буфера.
601	`if (!tmp.has_chain) break;`	Технически проверяет условие `!tmp.has_chain`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
602	`cur = tmp.chain.unwind_rva;`	Технически присваивает `cur` значение `tmp.chain.unwind_rva`. Логически это обновляет `cur`: хранит промежуточное значение, которое дальше используется в текущей функции.
603	`if (depth == max_depth-1) out->anomalies |= UA_CHAIN_DEPTH_EXCEEDED;`	Технически проверяет условие `depth == max_depth-1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
604	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
605	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
607	`out->valid = true;`	Технически присваивает `out->valid` значение `true`. Логически это обновляет `valid`: хранит промежуточное значение, которое дальше используется в текущей функции.
608	`return true;`	Технически возвращает `true` из функции `unwind_parse`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
609	`}`	Технически завершает область видимости внутри `unwind_parse`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
611	`RuntimeFunctionReadResult unwind_read_runtime_functions_ex(`	Технически вызывает `unwind_read_runtime_functions_ex`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `unwind_read_runtime_functions_ex` для выполнения части алгоритма.
612	`const Buf* b,`	Технически эта строка является частью С-конструкции `const Buf* b,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
613	`const PeImage* img,`	Технически эта строка является частью С-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
614	`RuntimeFunction** out_rf,`	Технически эта строка является частью С-конструкции `RuntimeFunction** out_rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
615	`size_t* out_count,`	Технически эта строка является частью С-конструкции `size_t* out_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
616	`StrBuf* err) {`	Технически эта строка является частью С-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `unwind_read_runtime_functions_ex` и влияет на то, какие данные будут обработаны.
617	`if (!b || !img || !out_rf || !out_count || !err) {`	Технически проверяет условие `!b || !img || !out_rf || !out_count || !err`; логически это защита от NULL-указателя и работы с неинициализированными данными.
618	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
619	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
620	`*out_rf = NULL;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_rf = NULL;.
621	`*out_count = 0;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_count = 0;.
623	`if (img->exception_rva == 0 && img->exception_size == 0) {`	Технически проверяет условие `img->exception_rva == 0 && img->exception_size == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
624	`return RUNTIME_FUNCTION_READ_OK;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_OK`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
625	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
626	`if (img->exception_rva == 0 || img->exception_size == 0) {`	Технически проверяет условие `img->exception_rva == 0 || img->exception_size == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
627	`sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-zero.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
628	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
629	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
630	`uint32_t off = 0;`	Технически объявляет `off` в функции `unwind_read_runtime_functions_ex` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
631	`if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {`	Технически проверяет условие `!pe_rva_to_fileoff(img, img->exception_rva, &off)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
632	`sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет форматированный текст в динамический буфер.
633	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
634	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
635	`if ((size_t)off + img->exception_size > b->size) {`	Технически проверяет условие `(size_t)off + img->exception_size > b->size`; логически это защита от чтения за пределами файла на диске.
636	`sb_append(err, "Exception directory out of file range.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
637	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.

Стр.	Код	Нормальное объяснение
638	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
639	`if (img->exception_size < 12) {`	Технически проверяет условие `img->exception_size < 12`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
640	`sb_append(err, "Exception directory size too small.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
641	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
642	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
643	`if ((img->exception_size % 12u) != 0) {`	Технически проверяет условие `(img->exception_size % 12u) != 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
644	`sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
645	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
646	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
647	`size_t n = img->exception_size / 12;`	Технически объявляет `n` в функции `unwind_read_runtime_functions_ex` и сразу задает начальное значение `img->exception_size / 12`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
648	`RuntimeFunction* rf = (RuntimeFunction*)calloc(n, sizeof(RuntimeFunction));`	Технически присваивает `RuntimeFunction* rf` значение `(RuntimeFunction*)calloc(n, sizeof(RuntimeFunction))`. Логически это обновляет `RuntimeFunction* rf`: хранит промежуточное значение, которое дальше используется в текущей функции.
649	`if (!rf) {`	Технически проверяет условие `!rf`; логически это защита от NULL-указателя и работы с неинициализированными данными.
650	`sb_append(err, "Out of memory.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
651	`return RUNTIME_FUNCTION_READ_OOM;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_OOM`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
652	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
654	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
655	`size_t e = (size_t)off + i*12;`	Технически присваивает `size_t e` значение `(size_t)off + i*12`. Логически это обновляет `size_t e`: хранит размер или количество, поэтому используется в проверках границ.
656	`uint32_t bva=0, eva=0, uva=0;`	Технически объявляет `bva` в функции `unwind_read_runtime_functions_ex` и сразу задает начальное значение `0, eva=0, uva=0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
657	`if (!buf_read_u32(b, e + 0, &bva) ||`	Технически проверяет условие `!buf_read_u32(b, e + 0, &bva`; логически это защита от NULL-указателя и работы с неинициализированными данными.
658	`!buf_read_u32(b, e + 4, &eva) ||`	Технически вызывает `buf_read_u32`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: читает 32-битное little-endian поле из PE.
659	`!buf_read_u32(b, e + 8, &uva)) {`	Технически объявляет/начинает функцию `buf_read_u32`; она доступная другим модулям через заголовок или линковку. Логически функция читает 32-битное little-endian поле PE.
660	`free(rf);`	Технически вызывает `free`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: освобождает память, чтобы в C не было утечки.
661	`sb_append(err, "Failed to read runtime function entry.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `unwind_read_runtime_functions_ex`: добавляет текст в динамический буфер.
662	`return RUNTIME_FUNCTION_READ_MALFORMED;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_MALFORMED`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
663	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
664	`rf[i].begin_rva = bva;`	Технически присваивает `rf[i].begin_rva` значение `bva`. Логически это обновляет `begin_rva`: начало функции в координатах RVA.
665	`rf[i].end_rva = eva;`	Технически присваивает `rf[i].end_rva` значение `eva`. Логически это обновляет `end_rva`: конец функции в координатах RVA.
666	`rf[i].unwind_rva = uva;`	Технически присваивает `rf[i].unwind_rva` значение `uva`. Логически это обновляет `unwind_rva`: RVA структуры UNWIND_INFO для этой функции.
667	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
669	`*out_rf = rf;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_rf = rf;.
670	`*out_count = n;`	Технически это комментарий исходника. Логически он объясняет разработчику: out_count = n;.
671	`return RUNTIME_FUNCTION_READ_OK;`	Технически возвращает выражение `RUNTIME_FUNCTION_READ_OK`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions_ex` для вызывающего кода.
672	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions_ex`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
674	`bool unwind_read_runtime_functions(const Buf* b,`	Технически вызывает `unwind_read_runtime_functions`. Логически это действие нужно в функции `глобальной области файла`: читает массив RUNTIME_FUNCTION из exception directory.
675	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
676	`RuntimeFunction** out_rf,`	Технически эта строка является частью C-конструкции `RuntimeFunction** out_rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
677	`size_t* out_count,`	Технически эта строка является частью C-конструкции `size_t* out_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
678	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `unwind_read_runtime_functions` и влияет на то, какие данные будут обработаны.
679	`return unwind_read_runtime_functions_ex(`	Технически возвращает выражение `unwind_read_runtime_functions_ex(`. Логически именно это значение становится результатом функции `unwind_read_runtime_functions` для вызывающего кода.
680	`b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;`	Технически выполняет оператор `b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;`. Логически в функции `unwind_read_runtime_functions` он меняет состояние, проверяемые данные или подготовку отчета.
681	`}`	Технически завершает область видимости внутри `unwind_read_runtime_functions`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 635. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

6. Признаки, веса, корреляция и verdict

Здесь сырые факты превращаются в 27 признаков, score, confidence и итоговый verdict. Это аналитическое ядро ВКР.

Что сначала выучить в C и теории

- агрегация статистики по массивам
- хеш-таблица для подсчета повторного использования RVA
- пороговые эвристики, score, confidence и mitigation
- почему один слабый сигнал нельзя считать доказательством

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/heuristics.h

Назначение файла: модель finding, mitigation, score, confidence и verdict.

Код, который пишем на этом шаге

```c
#pragma once
/*
  heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata
*/

#include <stddef.h>
#include <stdbool.h>
#include <stdint.h>
#include "pe.h"
#include "unwind.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

enum HeuristicEvidenceFamily {
    HEF_TABLE_INTEGRITY    = 1u << 0,
    HEF_UNWIND_INTEGRITY   = 1u << 1,
    HEF_METADATA_PLACEMENT = 1u << 2,
    HEF_UNWIND_REUSE       = 1u << 3,
    HEF_HANDLER_DOMINANCE  = 1u << 4
};

enum { HEURISTICS_MAX_FINDINGS = 32, HEURISTICS_MAX_MITIGATIONS = 16 };

typedef struct HeuristicFinding {
    char code[40];
    char family[32];
    char severity[16];
    char message[192];
    bool has_location;
    uint32_t representative_rva;
    size_t affected_count;
} HeuristicFinding;

typedef struct HeuristicMitigation {
    char code[40];
    char message[192];
} HeuristicMitigation;

typedef struct HeuristicReport {
    int    score; /* 0..100 (higher => more suspicious) */
    double suspicion;  /* 0..1 derived from score */
    double confidence; /* 0..1 confidence in the static verdict */
    bool   requires_manual_review;
    uint32_t evidence_families_mask;
    char   verdict[48];

    size_t total;

    size_t pe_section_raw_oob;
    size_t pe_section_virtual_oob;
    size_t pe_section_raw_overlaps;
    size_t pe_section_virtual_overlaps;
    size_t pe_section_misaligned;
    bool   pe_exception_directory_oob;

    size_t rf_invalid_ranges;
    size_t rf_oob;
    size_t rf_unwind_oob;
    size_t rf_unsorted;
    size_t rf_overlaps;
    size_t rf_duplicate_begins;

    size_t unwind_parse_fail;
    size_t unwind_anomalies;
    size_t unwind_abi_warnings;

    size_t unwind_unusual_section;
    size_t unwind_exec_section;
    size_t unwind_writable_section;
    size_t handler_nonexec_section;
    size_t handler_writable_section;

    double unwind_reuse_ratio;    /* max_count / total */
```

```
    double unwind_unique_ratio;   /* unique / total */

    double handler_reuse_ratio;   /* max_count / handler_count (if any) */
    size_t handler_count;

    size_t finding_count;
    HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];
    size_t weak_finding_count;
    size_t medium_finding_count;
    size_t strong_finding_count;
    char evidence_level[16];

    size_t mitigation_count;
    HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];

    StrBuf details; /* human-readable findings */
} HeuristicReport;

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err);

void heuristics_free(HeuristicReport* r);
const char* heuristics_evidence_family_name(uint32_t bit);
size_t heuristics_evidence_family_count(uint32_t mask);

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata`	Технически эта строка является частью C-конструкции `heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
4	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
6	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
7	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
8	`#include <stdint.h>`	Технически подключает `stdint.h`. Логически это дает модулю доступ к: точные типы uint8_t/uint32_t/uint64_t, без которых нельзя надежно читать бинарный формат.
9	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.
10	`#include "unwind.h"`	Технически подключает `unwind.h`. Логически это дает модулю доступ к: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.
11	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
13	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
14	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
15	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
17	`enum HeuristicEvidenceFamily {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
18	`HEF_TABLE_INTEGRITY = 1u << 0,`	Технически присваивает `HEF_TABLE_INTEGRITY` значение `1u << 0,`. Логически это обновляет `HEF_TABLE_INTEGRITY`: хранит промежуточное значение, которое дальше используется в текущей функции.
19	`HEF_UNWIND_INTEGRITY = 1u << 1,`	Технически присваивает `HEF_UNWIND_INTEGRITY` значение `1u << 1,`. Логически это обновляет `HEF_UNWIND_INTEGRITY`: хранит промежуточное значение, которое дальше используется в текущей функции.
20	`HEF_METADATA_PLACEMENT = 1u << 2,`	Технически присваивает `HEF_METADATA_PLACEMENT` значение `1u << 2,`. Логически это обновляет `HEF_METADATA_PLACEMENT`: хранит промежуточное значение, которое дальше используется в текущей функции.
21	`HEF_UNWIND_REUSE = 1u << 3,`	Технически присваивает `HEF_UNWIND_REUSE` значение `1u << 3,`. Логически это обновляет `HEF_UNWIND_REUSE`: хранит промежуточное значение, которое дальше используется в текущей функции.
22	`HEF_HANDLER_DOMINANCE = 1u << 4`	Технически присваивает `HEF_HANDLER_DOMINANCE` значение `1u << 4`. Логически это обновляет `HEF_HANDLER_DOMINANCE`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
23	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
25	`enum { HEURISTICS_MAX_FINDINGS = 32, HEURISTICS_MAX_MITIGATIONS = 16 };`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
27	`typedef struct HeuristicFinding {`	Технически начинает описание структуры `HeuristicFinding`. Логически структура собирает связанные поля в одну модель данных проекта.
28	`char code[40];`	Технически объявляет поле `code` внутри структуры. Логически это поле: машинный код finding или mitigation, стабильный для тестов и отчетов.
29	`char family[32];`	Технически объявляет поле `family` внутри структуры. Логически это поле: семейство доказательств, к которому относится finding.
30	`char severity[16];`	Технически объявляет поле `severity` внутри структуры. Логически это поле: уровень серьезности finding: weak/medium/high.
31	`char message[192];`	Технически объявляет поле `message` внутри структуры. Логически это поле: человекочитаемое объяснение finding.
32	`bool has_location;`	Технически объявляет поле `has_location` внутри структуры. Логически это поле: есть ли у finding конкретный representative RVA.
33	`uint32_t representative_rva;`	Технически объявляет поле `representative_rva` внутри структуры. Логически это поле: примерный адрес, где обнаружена проблема.
34	`size_t affected_count;`	Технически объявляет поле `affected_count` внутри структуры. Логически это поле: сколько записей/функций затронуто признаком.
35	`} HeuristicFinding;`	Технически завершает typedef и дает типу имя `HeuristicFinding`. Логически дальше код может объявлять переменные этого типа без `struct`.
37	`typedef struct HeuristicMitigation {`	Технически начинает описание структуры `HeuristicMitigation`. Логически структура собирает связанные поля в одну модель данных проекта.
38	`char code[40];`	Технически объявляет поле `code` внутри структуры. Логически это поле: машинный код finding или mitigation, стабильный для тестов и отчетов.
39	`char message[192];`	Технически объявляет поле `message` внутри структуры. Логически это поле: человекочитаемое объяснение finding.

Стр.	Код	Нормальное объяснение
40	`} HeuristicMitigation;`	Технически завершает typedef и дает типу имя `HeuristicMitigation`. Логически дальше код может объявлять переменные этого типа без `struct`.
42	`typedef struct HeuristicReport {`	Технически начинает описание структуры `HeuristicReport`. Логически структура собирает связанные поля в одну модель данных проекта.
43	`int score; /* 0..100 (higher => more suspicious) */`	Технически присваивает `int score; /* 0..100 (higher` значение `> more suspicious) */`. Логически это обновляет `100 (higher`: хранит промежуточное значение, которое дальше используется в текущей функции.
44	`double suspicion; /* 0..1 derived from score */`	Технически объявляет поле `suspicion` внутри структуры. Логически это поле: score, нормализованный в диапазон 0..1.
45	`double confidence; /* 0..1 confidence in the static verdict */`	Технически объявляет поле `confidence` внутри структуры. Логически это поле: уверенность в выводе с учетом силы и разнообразия доказательств.
46	`bool requires_manual_review;`	Технически объявляет поле `requires_manual_review` внутри структуры. Логически это поле: флаг, что результат надо проверить вручную.
47	`uint32_t evidence_families_mask;`	Технически объявляет поле `evidence_families_mask` внутри структуры. Логически это поле: битовая маска семейств доказательств.
48	`char verdict[48];`	Технически объявляет поле `verdict` внутри структуры. Логически это поле: итоговая текстовая оценка.
50	`size_t total;`	Технически объявляет поле `total` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
52	`size_t pe_section_raw_oob;`	Технически объявляет поле `pe_section_raw_oob` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
53	`size_t pe_section_virtual_oob;`	Технически объявляет поле `pe_section_virtual_oob` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
54	`size_t pe_section_raw_overlaps;`	Технически объявляет поле `pe_section_raw_overlaps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
55	`size_t pe_section_virtual_overlaps;`	Технически объявляет поле `pe_section_virtual_overlaps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
56	`size_t pe_section_misaligned;`	Технически объявляет поле `pe_section_misaligned` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
57	`bool pe_exception_directory_oob;`	Технически объявляет поле `pe_exception_directory_oob` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
59	`size_t rf_invalid_ranges;`	Технически объявляет поле `rf_invalid_ranges` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
60	`size_t rf_oob;`	Технически объявляет поле `rf_oob` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
61	`size_t rf_unwind_oob;`	Технически объявляет поле `rf_unwind_oob` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
62	`size_t rf_unsorted;`	Технически объявляет `rf_unsorted` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
63	`size_t rf_overlaps;`	Технически объявляет `rf_overlaps` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
64	`size_t rf_duplicate_begins;`	Технически объявляет `rf_duplicate_begins` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
66	`size_t unwind_parse_fail;`	Технически объявляет `unwind_parse_fail` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
67	`size_t unwind_anomalies;`	Технически объявляет `unwind_anomalies` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
68	`size_t unwind_abi_warnings;`	Технически объявляет `unwind_abi_warnings` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
70	`size_t unwind_unusual_section;`	Технически объявляет `unwind_unusual_section` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
71	`size_t unwind_exec_section;`	Технически объявляет `unwind_exec_section` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
72	`size_t unwind_writable_section;`	Технически объявляет `unwind_writable_section` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
73	`size_t handler_nonexec_section;`	Технически объявляет `handler_nonexec_section` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
74	`size_t handler_writable_section;`	Технически объявляет `handler_writable_section` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
76	`double unwind_reuse_ratio; /* max_count / total */`	Технически эта строка является частью С-конструкции `double unwind_reuse_ratio; /* max_count / total */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
77	`double unwind_unique_ratio; /* unique / total */`	Технически эта строка является частью С-конструкции `double unwind_unique_ratio; /* unique / total */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
79	`double handler_reuse_ratio; /* max_count / handler_count (if any) */`	Технически вызывает `handler_count`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `handler_count` для выполнения части алгоритма.
80	`size_t handler_count;`	Технически объявляет `handler_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
82	`size_t finding_count;`	Технически объявляет `finding_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
83	`HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];`	Технически объявляет `findings` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
84	`size_t weak_finding_count;`	Технически объявляет `weak_finding_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
85	`size_t medium_finding_count;`	Технически объявляет `medium_finding_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
86	`size_t strong_finding_count;`	Технически объявляет `strong_finding_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
87	`char evidence_level[16];`	Технически объявляет `evidence_level` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
89	`size_t mitigation_count;`	Технически объявляет `mitigation_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
90	`HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];`	Технически объявляет `mitigations` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
92	`StrBuf details; /* human-readable findings */`	Технически эта строка является частью С-конструкции `StrBuf details; /* human-readable findings */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
93	`} HeuristicReport;`	Технически завершает typedef и дает типу имя `HeuristicReport`. Логически дальше код может объявлять переменные этого типа без `struct`.
95	`bool heuristics_analyze(const PeImage* img,`	Технически вызывает `heuristics_analyze`. Логически это действие нужно в функции `глобальной области файла`: главная функция аналитики: считает признаки, веса, confidence, mitigations и verdict.
96	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
97	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
98	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
99	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
100	`HeuristicReport* out,`	Технически эта строка является частью C-конструкции `HeuristicReport* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
101	`StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
103	`void heuristics_free(HeuristicReport* r);`	Технически объявляет прототип функции `heuristics_free` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и освобождает ресурсы эвристического отчета.
104	`const char* heuristics_evidence_family_name(uint32_t bit);`	Технически объявляет прототип функции `heuristics_evidence_family_name` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и переводит бит семейства доказательств в текстовое имя.
105	`size_t heuristics_evidence_family_count(uint32_t mask);`	Технически объявляет прототип функции `heuristics_evidence_family_count` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и считает, сколько разных семейств доказательств сработало.
107	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
108	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
109	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 89. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/heuristics.c

Назначение файла: эвристический движок, который превращает PE/unwind факты в признаки и итоговую оценку.

Код, который пишем на этом шаге

```
 1 | #include "heuristics.h"
 2 | #include <stdlib.h>
 3 | #include <string.h>
 4 | #include <stdio.h>
 5 | #include <stdarg.h>
 6 |
 7 | #ifndef IMAGE_SCN_MEM_EXECUTE
 8 | #define IMAGE_SCN_MEM_EXECUTE 0x20000000u
 9 | #endif
10 | #ifndef IMAGE_SCN_MEM_WRITE
11 | #define IMAGE_SCN_MEM_WRITE 0x80000000u
12 | #endif
13 |
14 | #define UA_ABI_MASK \
15 |     (UA_ABI_REGISTER_CLASS | UA_ABI_RESERVED_OPINFO | UA_ABI_PUSH_ORDER | \
16 |      UA_ABI_CHAIN_RESTRICTION | UA_ABI_SAVE_OFFSET)
17 |
18 |
19 | /*
20 |  * Heuristic calibration notes
21 |  * ---------------------------
22 |  * These constants intentionally form a conservative engineering model rather
23 |  * than a mathematically validated classifier. The weights are grouped by the
24 |  * evidence family discussed in the thesis so the rationale stays stable and
25 |  * reviewable:
26 |  *   - table/unwind integrity violations => strongest static weights
27 |  *   - placement anomalies => medium/high depending on prevalence
28 |  *   - reuse/dominance patterns => moderate unless they are very strong
29 |  *   - mitigations reduce confidence when evidence is isolated or sample size is tiny
30 |  */
31 | static const int SCORE_EMPTY_PDATA = 10;
32 | static const int SCORE_PE_SECTION_RAW_OOB = 25;
33 | static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;
34 | static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;
35 | static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;
36 | static const int SCORE_PE_SECTION_MISALIGNED = 8;
37 | static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;
38 | static const int SCORE_INVALID_RANGE_LOW = 10;
```

```
static const int SCORE_INVALID_RANGE_HIGH = 20;
static const int SCORE_RF_OOB_LOW = 10;
static const int SCORE_RF_OOB_HIGH = 25;
static const int SCORE_UNWIND_RVA_OOB_LOW = 10;
static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;
static const int SCORE_RF_UNSORTED = 15;
static const int SCORE_RF_OVERLAP = 25;
static const int SCORE_DUPLICATE_BEGIN = 10;
static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;
static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;
static const int SCORE_UNWIND_ANOM_LOW = 10;
static const int SCORE_UNWIND_ANOM_HIGH = 20;
static const int SCORE_UNWIND_REUSE_MEDIUM = 10;
static const int SCORE_UNWIND_REUSE_HIGH = 20;
static const int SCORE_HANDLER_DOMINANCE = 15;
static const int SCORE_UNWIND_EXEC_SECTION = 20;
static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;
static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;
static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;
static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;
static const int SCORE_HANDLER_NONEXEC = 15;
static const int SCORE_HANDLER_WRITABLE = 10;
static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;
static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;
static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;
static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;

static const double CONFIDENCE_BASE = 0.15;
static const double CONFIDENCE_ZERO_SCORE_BASE = 0.20;
static const double CONFIDENCE_PER_FAMILY = 0.11;
static const double CONFIDENCE_PER_STRONG = 0.08;
static const double CONFIDENCE_PER_MEDIUM = 0.04;
static const double CONFIDENCE_BONUS_TABLE_HARD = 0.08;
static const double CONFIDENCE_BONUS_UNWIND_HARD = 0.08;
static const double CONFIDENCE_PENALTY_SINGLE_SIGNAL = 0.12;
static const double CONFIDENCE_PENALTY_SMALL_SAMPLE = 0.06;
static const double CONFIDENCE_PENALTY_PLACEMENT_ONLY = 0.10;
static const double CONFIDENCE_PENALTY_SINGLE_FAMILY = 0.08;
static const double CONFIDENCE_MIN = 0.05;
static const double CONFIDENCE_MAX = 0.90;

static const double THRESHOLD_HIGH_PERCENT = 0.01;   /* >1% of entries => not just a one-off glitch */
static const double THRESHOLD_UNWIND_ANOM_HIGH = 0.05; /* >5% anomalies => broad integrity issue */
static const double THRESHOLD_UNWIND_REUSE_MEDIUM = 0.10;
static const double THRESHOLD_UNWIND_REUSE_HIGH = 0.30;
static const double THRESHOLD_HANDLER_DOMINANCE = 0.80;

static bool sec_name_eq(const char* a, const char* b) {
    if (!a || !b) return false;
    while (*a && *b) {
        char ca = *a;
        char cb = *b;
        if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');
        if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');
        if (ca != cb) return false;
        a++; b++;
    }
    return *a == 0 && *b == 0;
}

static bool sec_name_contains_ci(const char* haystack, const char* needle) {
    if (!haystack || !needle || !*needle) return false;
    size_t nlen = strlen(needle);
    for (const char* h = haystack; *h; h++) {
        size_t i = 0;
        while (i < nlen) {
            char ca = h[i];
            char cb = needle[i];
            if (!ca) return false;
            if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');
            if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');
            if (ca != cb) break;
            i++;
        }
        if (i == nlen) return true;
    }
    return false;
}

static bool sec_name_allowed_for_unwind(const char* name8) {
    /* Common placements: .xdata (MSVC), .rdata/.pdata in practice, and toolchain-specific xdata-like sections. */
    return sec_name_eq(name8, ".xdata") || sec_name_eq(name8, ".rdata") || sec_name_eq(name8, ".pdata") ||
           sec_name_eq(name8, ".gnu_xdata") || sec_name_eq(name8, ".mrdata") ||
           sec_name_eq(name8, "xdata") || sec_name_eq(name8, "rdata") || sec_name_eq(name8, "pdata");
}

static bool sec_name_plausibly_unwind_named(const char* name8) {
    return sec_name_contains_ci(name8, "xdata") || sec_name_contains_ci(name8, "pdata") || sec_name_contains_ci(name8, "rdata");
}

static bool sec_name_code_like(const char* name8) {
    return sec_name_eq(name8, ".text") || sec_name_eq(name8, "text") ||
           sec_name_contains_ci(name8, "text") || sec_name_contains_ci(name8, "code") || sec_name_contains_ci(name8, "init");
}

static bool image_has_executable_section(const PeImage* img) {
    if (!img || !img->sections) return false;
    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* section = &img->sections[i];
        if ((section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0 &&
            (section->virtual_size != 0 || section->raw_size != 0)) {
            return true;
        }
    }
    return false;
}

/* Very small u32->count hash map (open addressing, linear probing). */
typedef struct HEntry {
    uint32_t key;
    uint32_t val;
    bool used;
} HEntry;

typedef struct HMap {
```

```
    HEntry* e;
    size_t cap;
    size_t len;
} HMap;

static uint32_t h32(uint32_t x) {
    /* xorshift-ish */
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static bool hmap_init(HMap* m, size_t cap) {
    if (!m) return false;
    m->cap = cap < 64 ? 64 : cap;
    m->len = 0;
    m->e = (HEntry*)calloc(m->cap, sizeof(HEntry));
    return m->e != NULL;
}

static void hmap_free(HMap* m) {
    if (!m) return;
    free(m->e);
    m->e = NULL;
    m->cap = 0;
    m->len = 0;
}

static bool hmap_grow(HMap* m);

static bool hmap_inc(HMap* m, uint32_t key, uint32_t* out_newval) {
    if (!m || !m->e) return false;
    if ((m->len + 1) * 10 >= m->cap * 7) {
        if (!hmap_grow(m)) return false;
    }
    uint32_t hv = h32(key);
    size_t idx = (size_t)hv % m->cap;
    for (size_t step = 0; step < m->cap; step++) {
        HEntry* ent = &m->e[idx];
        if (!ent->used) {
            ent->used = true;
            ent->key = key;
            ent->val = 1;
            m->len++;
            if (out_newval) *out_newval = 1;
            return true;
        }
        if (ent->key == key) {
            ent->val++;
            if (out_newval) *out_newval = ent->val;
            return true;
        }
        idx = (idx + 1) % m->cap;
    }
    return false;
}

static bool hmap_grow(HMap* m) {
    size_t oldcap = m->cap;
    HEntry* old = m->e;
    if (oldcap > (SIZE_MAX - 1u) / 2u) return false;
    size_t newcap = oldcap * 2 + 1;
    HEntry* ne = (HEntry*)calloc(newcap, sizeof(HEntry));
    if (!ne) return false;

    m->e = ne;
    m->cap = newcap;
    m->len = 0;

    for (size_t i = 0; i < oldcap; i++) {
        if (!old[i].used) continue;
        /* reinsert */
        uint32_t key = old[i].key;
        uint32_t val = old[i].val;
        uint32_t hv = h32(key);
        size_t idx = (size_t)hv % m->cap;
        for (;;) {
            HEntry* ent = &m->e[idx];
            if (!ent->used) {
                ent->used = true;
                ent->key = key;
                ent->val = val;
                m->len++;
                break;
            }
            idx = (idx + 1) % m->cap;
        }
    }
    free(old);
    return true;
}

static uint32_t hmap_max_val(const HMap* m) {
    uint32_t mx = 0;
    if (!m || !m->e) return 0;
    for (size_t i = 0; i < m->cap; i++) {
        if (m->e[i].used && m->e[i].val > mx) mx = m->e[i].val;
    }
    return mx;
}

static bool hmap_max_entry(const HMap* m, uint32_t* out_key, uint32_t* out_val) {
    uint32_t best_key = 0;
    uint32_t best_val = 0;
    if (!m || !m->e) return false;
    for (size_t i = 0; i < m->cap; i++) {
        if (!m->e[i].used) continue;
        if (m->e[i].val > best_val) {
            best_key = m->e[i].key;
            best_val = m->e[i].val;
        }
    }
```

```
    if (best_val == 0) return false;
    if (out_key) *out_key = best_key;
    if (out_val) *out_val = best_val;
    return true;
}

static size_t hmap_unique(const HMap* m) { return m ? m->len : 0; }

typedef struct RfIdx {
    uint32_t begin;
    uint32_t end;
} RfIdx;

typedef struct FindingLocation {
    bool set;
    uint32_t rva;
} FindingLocation;

static void remember_location(FindingLocation* location, uint32_t rva) {
    if (!location || location->set) return;
    location->set = true;
    location->rva = rva;
}

enum PeSectionIssue {
    PE_ISSUE_RAW_OOB,
    PE_ISSUE_VIRTUAL_OOB,
    PE_ISSUE_RAW_OVERLAP,
    PE_ISSUE_VIRTUAL_OVERLAP,
    PE_ISSUE_MISALIGNED
};

static FindingLocation first_pe_section_issue(const PeImage* img, enum PeSectionIssue issue) {
    FindingLocation result = {0};
    if (!img || !img->sections) return result;

    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* s = &img->sections[i];
        uint64_t span = s->virtual_size > s->raw_size ? s->virtual_size : s->raw_size;
        bool match = false;
        if (issue == PE_ISSUE_RAW_OOB) {
            match = s->raw_size > 0 &&
                    ((uint64_t)s->raw_ptr + (uint64_t)s->raw_size > (uint64_t)img->file_size);
        } else if (issue == PE_ISSUE_VIRTUAL_OOB) {
            match = span > 0 &&
                    ((uint64_t)s->virtual_address + span > (uint64_t)img->size_of_image);
        } else if (issue == PE_ISSUE_MISALIGNED) {
            match = (img->section_alignment > 0 &&
                     (s->virtual_address % img->section_alignment) != 0) ||
                    (s->raw_size > 0 && img->file_alignment > 0 &&
                     (s->raw_ptr % img->file_alignment) != 0);
        } else {
            for (size_t j = 0; j < i && !match; j++) {
                const PeSection* p = &img->sections[j];
                uint64_t pspan = p->virtual_size > p->raw_size ? p->virtual_size : p->raw_size;
                if (issue == PE_ISSUE_RAW_OVERLAP) {
                    match = s->raw_size > 0 && p->raw_size > 0 &&
                            (uint64_t)s->raw_ptr < (uint64_t)p->raw_ptr + p->raw_size &&
                            (uint64_t)p->raw_ptr < (uint64_t)s->raw_ptr + s->raw_size;
                } else if (issue == PE_ISSUE_VIRTUAL_OVERLAP) {
                    match = span > 0 && pspan > 0 &&
                            (uint64_t)s->virtual_address < (uint64_t)p->virtual_address + pspan &&
                            (uint64_t)p->virtual_address < (uint64_t)s->virtual_address + span;
                }
            }
        }
        if (match) {
            remember_location(&result, s->virtual_address);
            break;
        }
    }
    return result;
}

static int cmp_rfidx(const void* a, const void* b) {
    const RfIdx* x = (const RfIdx*)a;
    const RfIdx* y = (const RfIdx*)b;
    if (x->begin < y->begin) return -1;
    if (x->begin > y->begin) return 1;
    if (x->end < y->end) return -1;
    if (x->end > y->end) return 1;
    return 0;
}

static int clamp_int(int v, int lo, int hi) {
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

static void add_score(int* score, int add) {
    if (!score) return;
    *score = clamp_int(*score + add, 0, 100);
}

static double safe_ratio(double a, double b) { return (b == 0.0) ? 0.0 : (a / b); }


const char* heuristics_evidence_family_name(uint32_t bit) {
    switch (bit) {
        case HEF_TABLE_INTEGRITY: return "table_integrity";
        case HEF_UNWIND_INTEGRITY: return "unwind_integrity";
        case HEF_METADATA_PLACEMENT: return "metadata_placement";
        case HEF_UNWIND_REUSE: return "unwind_reuse";
        case HEF_HANDLER_DOMINANCE: return "handler_dominance";
        default: return "unknown";
    }
}

size_t heuristics_evidence_family_count(uint32_t mask) {
    size_t n = 0;
    while (mask) {
        n += (mask & 1u) ? 1u : 0u;
        mask >>= 1;
    }
```

```
    return n;
}

static uint32_t family_bit_from_name(const char* family) {
    if (!family) return 0;
    if (strcmp(family, "table_integrity") == 0) return HEF_TABLE_INTEGRITY;
    if (strcmp(family, "unwind_integrity") == 0) return HEF_UNWIND_INTEGRITY;
    if (strcmp(family, "metadata_placement") == 0) return HEF_METADATA_PLACEMENT;
    if (strcmp(family, "unwind_reuse") == 0) return HEF_UNWIND_REUSE;
    if (strcmp(family, "handler_dominance") == 0) return HEF_HANDLER_DOMINANCE;
    return 0;
}

static void safe_copy(char* dst, size_t dst_cap, const char* src) {
    if (!dst || dst_cap == 0) return;
    if (!src) src = "";
    snprintf(dst, dst_cap, "%s", src);
}

#if defined(__GNUC__) || defined(__clang__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wformat-nonliteral"
#endif

static HeuristicFinding* add_finding(HeuristicReport* out,
                                     const char* code,
                                     const char* family,
                                     const char* severity,
                                     const char* fmt,
                                     ...) {
    if (!out || !fmt) return NULL;

    char msg[192];
    va_list ap;
    va_start(ap, fmt);
    (void)vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);

    out->evidence_families_mask |= family_bit_from_name(family);

    HeuristicFinding* stored = NULL;
    if (out->finding_count < HEURISTICS_MAX_FINDINGS) {
        HeuristicFinding* f = &out->findings[out->finding_count++];
        memset(f, 0, sizeof(*f));
        safe_copy(f->code, sizeof(f->code), code ? code : "unspecified");
        safe_copy(f->family, sizeof(f->family), family ? family : "unknown");
        safe_copy(f->severity, sizeof(f->severity), severity ? severity : "info");
        safe_copy(f->message, sizeof(f->message), msg);
        stored = f;
    }

    sb_appendf(&out->details, "- [%s/%s] %s\n",
               severity ? severity : "info",
               code ? code : "unspecified",
               msg);
    return stored;
}

static void annotate_finding(HeuristicFinding* finding,
                             FindingLocation location,
                             size_t affected_count) {
    if (!finding) return;
    finding->has_location = location.set;
    finding->representative_rva = location.rva;
    finding->affected_count = affected_count;
}

static void append_note(HeuristicReport* out, const char* fmt, ...) {
    if (!out || !fmt) return;

    char msg[192];
    va_list ap;
    va_start(ap, fmt);
    (void)vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);

    sb_appendf(&out->details, "- [info] %s\n", msg);
}

static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev) {
    size_t n = 0;
    if (!out || !sev) return 0;
    for (size_t i = 0; i < out->finding_count; i++) {
        if (strcmp(out->findings[i].severity, sev) == 0) n++;
    }
    return n;
}

static bool has_finding_code(const HeuristicReport* out, const char* code) {
    if (!out || !code) return false;
    for (size_t i = 0; i < out->finding_count; i++) {
        if (strcmp(out->findings[i].code, code) == 0) return true;
    }
    return false;
}

static void add_mitigation(HeuristicReport* out, const char* code, const char* fmt, ...) {
    if (!out || !fmt) return;
    if (out->mitigation_count >= HEURISTICS_MAX_MITIGATIONS) return;

    char msg[192];
    va_list ap;
    va_start(ap, fmt);
    (void)vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);

    HeuristicMitigation* m = &out->mitigations[out->mitigation_count++];
    memset(m, 0, sizeof(*m));
    safe_copy(m->code, sizeof(m->code), code ? code : "unspecified");
    safe_copy(m->message, sizeof(m->message), msg);

    sb_appendf(&out->details, "  [mitigation/%s] %s\n",
               code ? code : "unspecified",
               msg);
}
```

```
499 |
500 | #if defined(__GNUC__) || defined(__clang__)
501 | #pragma GCC diagnostic pop
502 | #endif
503 |
504 | static const char* evidence_level_from_counts(size_t weak_count,
505 |                                               size_t medium_count,
506 |                                               size_t strong_count,
507 |                                               int score) {
508 |     if (strong_count >= 2 || score >= 70) return "strong";
509 |     if (strong_count >= 1 || medium_count >= 2 || score >= 30) return "medium";
510 |     if (weak_count || medium_count || strong_count || score > 0) return "weak";
511 |     return "none";
512 | }
513 |
514 | static void finalize_verdict(HeuristicReport* out) {
515 |     if (!out) return;
516 |
517 |     out->suspicion = safe_ratio((double)out->score, 100.0);
518 |
519 |     size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);
520 |     size_t strong_count = count_findings_with_severity(out, "high") + count_findings_with_severity(out, "critical");
521 |     size_t medium_count = count_findings_with_severity(out, "medium");
522 |     size_t weak_count = 0;
523 |     if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count - strong_count - medium_count;
524 |
525 |     out->strong_finding_count = strong_count;
526 |     out->medium_finding_count = medium_count;
527 |     out->weak_finding_count = weak_count;
528 |     safe_copy(out->evidence_level, sizeof(out->evidence_level),
529 |               evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));
530 |
531 |     double confidence = CONFIDENCE_BASE;
532 |     confidence += (double)family_count * CONFIDENCE_PER_FAMILY;
533 |     confidence += (double)strong_count * CONFIDENCE_PER_STRONG;
534 |     confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;
535 |
536 |     if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps) confidence += CONFIDENCE_BONUS_TABLE_HARD;
537 |     if (out->unwind_parse_fail || out->unwind_anomalies) confidence += CONFIDENCE_BONUS_UNWIND_HARD;
538 |     if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;
539 |
540 |     /* Weakening rules: lower confidence when only isolated / non-correlated static signals exist. */
541 |     if (out->finding_count == 1 && strong_count == 0) {
542 |         add_mitigation(out, "single_signal_only",
543 |                        "Only one non-strong signal contributed; treat as weak standalone evidence.");
544 |         confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;
545 |     }
546 |     if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {
547 |         add_mitigation(out, "small_sample_size",
548 |                        "Very small .pdata sample can overstate ratios; confidence reduced until more functions are observed.");
549 |         confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;
550 |     }
551 |     if (out->unwind_unusual_section > 0 &&
552 |         out->unwind_exec_section == 0 &&
553 |         out->unwind_writable_section == 0 &&
554 |         out->handler_nonexec_section == 0 &&
555 |         out->handler_writable_section == 0 &&
556 |         out->rf_invalid_ranges == 0 &&
557 |         out->rf_oob == 0 &&
558 |         out->rf_unwind_oob == 0 &&
559 |         out->unwind_parse_fail == 0 &&
560 |         out->unwind_anomalies == 0) {
561 |         add_mitigation(out, "placement_only_signal",
562 |                        "Placement-only anomalies may come from uncommon but benign layouts; manual verification required.");
563 |         confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;
564 |     }
565 |     if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {
566 |         add_mitigation(out, "single_family_only",
567 |                        "Evidence comes from only one family; cross-layer corroboration is recommended before strong conclusions.");
568 |         confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;
569 |     }
570 |
571 |     if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;
572 |     if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;
573 |     out->confidence = confidence;
574 |
575 |     out->requires_manual_review =
576 |         (out->score >= 20) ||
577 |         (out->unwind_exec_section > 0) ||
578 |         (out->unwind_writable_section > 0) ||
579 |         (out->handler_nonexec_section > 0) ||
580 |         (out->handler_writable_section > 0) ||
581 |         (out->unwind_parse_fail > 0) ||
582 |         (out->mitigation_count > 0);
583 |
584 |     if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");
585 |     else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict), "moderate_suspicion");
586 |     else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");
587 |     else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");
588 | }
589 |
590 | bool heuristics_analyze(const PeImage* img,
591 |                         const RuntimeFunction* rf,
592 |                         const UnwindSummary* us,
593 |                         const bool* us_ok,
594 |                         size_t n,
595 |                         HeuristicReport* out,
596 |                         StrBuf* err) {
597 |     if (!img || !out || !err) return false;
598 |     if (n > 0 && (!rf || !us || !us_ok)) return false;
599 |     memset(out, 0, sizeof(*out));
600 |     if (!sb_init(&out->details, 1024)) {
601 |         sb_append(err, "Out of memory (details buffer).\n");
602 |         return false;
603 |     }
604 |     out->total = n;
605 |     out->pe_section_raw_oob = img->section_raw_oob;
606 |     out->pe_section_virtual_oob = img->section_virtual_oob;
607 |     out->pe_section_raw_overlaps = img->section_raw_overlaps;
608 |     out->pe_section_virtual_overlaps = img->section_virtual_overlaps;
609 |     out->pe_section_misaligned = img->section_misaligned;
610 |     out->pe_exception_directory_oob = img->exception_directory_oob;
611 |
612 |     /* RUNTIME_FUNCTION sanity */
613 |     uint32_t prev_begin = 0;
```

```
    bool prev_valid = false;
    size_t unsorted = 0;
    FindingLocation invalid_range_loc = {0};
    FindingLocation rf_oob_loc = {0};
    FindingLocation unwind_oob_loc = {0};
    FindingLocation unsorted_loc = {0};
    FindingLocation overlap_loc = {0};
    FindingLocation duplicate_loc = {0};
    FindingLocation parse_fail_loc = {0};
    FindingLocation anomaly_loc = {0};
    FindingLocation unusual_unwind_loc = {0};
    FindingLocation executable_unwind_loc = {0};
    FindingLocation writable_unwind_loc = {0};
    FindingLocation nonexec_handler_loc = {0};
    FindingLocation writable_handler_loc = {0};

    HMap unwind_counts = {0};
    HMap handler_counts = {0};
    if (n > (SIZE_MAX - 1u) / 2u) {
        sb_append(err, "Runtime function count is too large for reuse analysis.\n");
        sb_free(&out->details);
        return false;
    }
    if (!hmap_init(&unwind_counts, n * 2 + 1u) ||
        !hmap_init(&handler_counts, 128)) {
        sb_append(err, "Out of memory (hash maps).\n");
        hmap_free(&unwind_counts);
        hmap_free(&handler_counts);
        sb_free(&out->details);
        return false;
    }

    for (size_t i = 0; i < n; i++) {
        uint32_t b = rf[i].begin_rva;
        uint32_t e = rf[i].end_rva;
        uint32_t u = rf[i].unwind_rva;

        if (e <= b) {
            out->rf_invalid_ranges++;
            remember_location(&invalid_range_loc, b);
        }
        if (b >= img->size_of_image || e > img->size_of_image) {
            out->rf_oob++;
            remember_location(&rf_oob_loc, b);
        }
        if (u >= img->size_of_image || u == 0) {
            out->rf_unwind_oob++;
            remember_location(&unwind_oob_loc, u);
        }

        if (prev_valid && b < prev_begin) {
            unsorted++;
            remember_location(&unsorted_loc, b);
        }
        prev_begin = b;
        prev_valid = true;

        if (!hmap_inc(&unwind_counts, u, NULL)) {
            sb_append(err, "Out of memory while counting unwind metadata reuse.\n");
            hmap_free(&unwind_counts);
            hmap_free(&handler_counts);
            sb_free(&out->details);
            return false;
        }

        if (us_ok[i] && us[i].has_handler) {
            out->handler_count++;
            if (!hmap_inc(&handler_counts, us[i].handler_rva, NULL)) {
                sb_append(err, "Out of memory while counting handler reuse.\n");
                hmap_free(&unwind_counts);
                hmap_free(&handler_counts);
                sb_free(&out->details);
                return false;
            }
        }

        /* Section placement sanity: unwind info is typically in non-executable data (.xdata/.rdata).
           Placing unwind metadata in executable / unusual sections is a common interception/tampering artifact. */
        if (us_ok[i]) {
            const PeSection* su = pe_find_section_for_rva(img, u);
            if (!su) {
                out->unwind_unusual_section++;
                remember_location(&unusual_unwind_loc, u);
            } else {
                bool allowed = sec_name_allowed_for_unwind(su->name);
                bool plausible = sec_name_plausibly_unwind_named(su->name);
                if (!allowed && !plausible) {
                    out->unwind_unusual_section++;
                    remember_location(&unusual_unwind_loc, u);
                }
                if (su->characteristics & IMAGE_SCN_MEM_EXECUTE) {
                    out->unwind_exec_section++;
                    remember_location(&executable_unwind_loc, u);
                }
                if (su->characteristics & IMAGE_SCN_MEM_WRITE) {
                    out->unwind_writable_section++;
                    remember_location(&writable_unwind_loc, u);
                }
            }

            if (us[i].has_handler) {
                const PeSection* sh = pe_find_section_for_rva(img, us[i].handler_rva);
                if (!sh || !(sh->characteristics & IMAGE_SCN_MEM_EXECUTE)) {
                    out->handler_nonexec_section++;
                    remember_location(&nonexec_handler_loc, us[i].handler_rva);
                }
                if (sh && (sh->characteristics & IMAGE_SCN_MEM_WRITE)) {
                    out->handler_writable_section++;
                    remember_location(&writable_handler_loc, us[i].handler_rva);
                }
            }
        }

        if (!us_ok[i]) {
            out->unwind_parse_fail++;
```

```
            remember_location(&parse_fail_loc, u);
        } else if (us[i].anomalies & (uint32_t)~UA_ABI_MASK) {
            out->unwind_anomalies++;
            remember_location(&anomaly_loc, u);
        }
        if (us_ok[i] && (us[i].anomalies & UA_ABI_MASK)) {
            out->unwind_abi_warnings++;
        }
    }
    out->rf_unsorted = unsorted;

    /* Overlap / duplicates check on sorted array */
    RfIdx* idx = NULL;
    if (n > 0) {
        idx = (RfIdx*)malloc(n * sizeof(RfIdx));
        if (!idx) {
            sb_append(err, "Out of memory (sort array).\n");
            hmap_free(&unwind_counts);
            hmap_free(&handler_counts);
            sb_free(&out->details);
            return false;
        }
        for (size_t i = 0; i < n; i++) { idx[i].begin = rf[i].begin_rva; idx[i].end = rf[i].end_rva; }
        qsort(idx, n, sizeof(RfIdx), cmp_rfidx);

        uint32_t max_end = 0;
        uint32_t sorted_prev_begin = 0;
        bool have_prev = false;
        for (size_t i = 0; i < n; i++) {
            if (have_prev) {
                if (idx[i].begin == sorted_prev_begin) {
                    out->rf_duplicate_begins++;
                    remember_location(&duplicate_loc, idx[i].begin);
                }
                if (idx[i].begin < max_end) {
                    out->rf_overlaps++;
                    remember_location(&overlap_loc, idx[i].begin);
                }
            }
            sorted_prev_begin = idx[i].begin;
            if (!have_prev || idx[i].end > max_end) max_end = idx[i].end;
            have_prev = true;
        }
    }
    free(idx);

    uint32_t unwind_max = hmap_max_val(&unwind_counts);
    uint32_t dominant_unwind_rva = 0;
    (void)hmap_max_entry(&unwind_counts, &dominant_unwind_rva, NULL);
    size_t unwind_unique = hmap_unique(&unwind_counts);
    out->unwind_reuse_ratio = safe_ratio((double)unwind_max, (double)n);
    out->unwind_unique_ratio = safe_ratio((double)unwind_unique, (double)n);

    uint32_t handler_max = hmap_max_val(&handler_counts);
    uint32_t dominant_handler_rva = 0;
    (void)hmap_max_entry(&handler_counts, &dominant_handler_rva, NULL);
    out->handler_reuse_ratio = (out->handler_count > 0) ? safe_ratio((double)handler_max, (double)out->handler_count) : 0.0;

    /* Scoring */
    int score = 0;
    if (out->pe_section_raw_oob) {
        add_score(&score, SCORE_PE_SECTION_RAW_OOB);
        annotate_finding(
            add_finding(out, "pe_section_raw_oob", "table_integrity", "high",
                        "Section raw ranges exceed the input file: %zu", out->pe_section_raw_oob),
            first_pe_section_issue(img, PE_ISSUE_RAW_OOB), out->pe_section_raw_oob);
    }
    if (out->pe_section_virtual_oob) {
        add_score(&score, SCORE_PE_SECTION_VIRTUAL_OOB);
        annotate_finding(
            add_finding(out, "pe_section_virtual_oob", "table_integrity", "high",
                        "Section virtual ranges exceed SizeOfImage: %zu", out->pe_section_virtual_oob),
            first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OOB), out->pe_section_virtual_oob);
    }
    if (out->pe_section_raw_overlaps) {
        add_score(&score, SCORE_PE_SECTION_RAW_OVERLAP);
        annotate_finding(
            add_finding(out, "pe_section_raw_overlap", "table_integrity", "medium",
                        "Overlapping section raw ranges: %zu", out->pe_section_raw_overlaps),
            first_pe_section_issue(img, PE_ISSUE_RAW_OVERLAP), out->pe_section_raw_overlaps);
    }
    if (out->pe_section_virtual_overlaps) {
        add_score(&score, SCORE_PE_SECTION_VIRTUAL_OVERLAP);
        annotate_finding(
            add_finding(out, "pe_section_virtual_overlap", "table_integrity", "high",
                        "Overlapping section virtual ranges: %zu", out->pe_section_virtual_overlaps),
            first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OVERLAP), out->pe_section_virtual_overlaps);
    }
    if (out->pe_section_misaligned) {
            add_score(&score, SCORE_PE_SECTION_MISALIGNED);
        annotate_finding(
            add_finding(out, "pe_section_misaligned", "table_integrity", "medium",
                        "PE alignment/header invariants or section alignment rules are violated: %zu",
                        out->pe_section_misaligned),
            first_pe_section_issue(img, PE_ISSUE_MISALIGNED), out->pe_section_misaligned);
    }
    if (out->pe_exception_directory_oob) {
        FindingLocation location = { true, img->exception_rva };
        add_score(&score, SCORE_PE_EXCEPTION_DIRECTORY_OOB);
        annotate_finding(
            add_finding(out, "pe_exception_directory_oob", "table_integrity", "high",
                        "Exception directory is malformed, unreadable, incomplete, or out of bounds."),
            location, 1);
    }
    if (n == 0 &&
        img->exception_rva == 0 &&
        img->exception_size == 0 &&
        image_has_executable_section(img)) {
        add_score(&score, SCORE_EMPTY_PDATA);
        add_finding(out, "no_runtime_functions", "table_integrity", "medium",
                    "No RUNTIME_FUNCTION entries found; this can be valid for a leaf-only image, so confirm code complexity
  manually.");
    } else if (n == 0 && img->exception_rva == 0 && img->exception_size == 0) {
        append_note(out, "No RUNTIME_FUNCTION entries and no executable sections; this is consistent with a resource-only or
  forwarding image.");
```

```
842 |     }
843 |
844 |     double invalid_ranges_pct = safe_ratio((double)out->rf_invalid_ranges, (double)n);
845 |     double oob_pct = safe_ratio((double)out->rf_oob, (double)n);
846 |     double unwind_oob_pct = safe_ratio((double)out->rf_unwind_oob, (double)n);
847 |     double unwind_fail_pct = safe_ratio((double)out->unwind_parse_fail, (double)n);
848 |     double unwind_anom_pct = safe_ratio((double)out->unwind_anomalies, (double)n);
849 |
850 |     if (out->rf_invalid_ranges) {
851 |         add_score(&score, invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? SCORE_INVALID_RANGE_HIGH : SCORE_INVALID_RANGE_LOW);
852 |         annotate_finding(
853 |             add_finding(out, "invalid_ranges", "table_integrity", invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
854 |                         "Invalid ranges (End<=Begin): %zu (%.2f%%)", out->rf_invalid_ranges, invalid_ranges_pct*100.0),
855 |             invalid_range_loc, out->rf_invalid_ranges);
856 |     }
857 |     if (out->rf_oob) {
858 |         add_score(&score, oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_RF_OOB_HIGH : SCORE_RF_OOB_LOW);
859 |         annotate_finding(
860 |             add_finding(out, "rf_oob", "table_integrity", oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
861 |                         "Begin/End out of SizeOfImage: %zu (%.2f%%)", out->rf_oob, oob_pct*100.0),
862 |             rf_oob_loc, out->rf_oob);
863 |     }
864 |     if (out->rf_unwind_oob) {
865 |         add_score(&score, unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_RVA_OOB_HIGH : SCORE_UNWIND_RVA_OOB_LOW);
866 |         annotate_finding(
867 |             add_finding(out, "unwind_rva_oob", "table_integrity", unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
868 |                         "UnwindInfoRVA missing/out-of-bounds: %zu (%.2f%%)", out->rf_unwind_oob, unwind_oob_pct*100.0),
869 |             unwind_oob_loc, out->rf_unwind_oob);
870 |     }
871 |     if (out->rf_unsorted) {
872 |         add_score(&score, out->rf_unsorted > 0 ? SCORE_RF_UNSORTED : 0);
873 |         annotate_finding(
874 |             add_finding(out, "rf_unsorted", "table_integrity", "medium",
875 |                         "Table not sorted by BeginAddress: %zu inversions", out->rf_unsorted),
876 |             unsorted_loc, out->rf_unsorted);
877 |     }
878 |     if (out->rf_overlaps) {
879 |         add_score(&score, out->rf_overlaps > 0 ? SCORE_RF_OVERLAP : 0);
880 |         annotate_finding(
881 |             add_finding(out, "rf_overlaps", "table_integrity", "high",
882 |                         "Overlapping ranges (after sorting): %zu", out->rf_overlaps),
883 |             overlap_loc, out->rf_overlaps);
884 |     }
885 |     if (out->rf_duplicate_begins) {
886 |         add_score(&score, SCORE_DUPLICATE_BEGIN);
887 |         annotate_finding(
888 |             add_finding(out, "duplicate_begin", "table_integrity", "medium",
889 |                         "Duplicate BeginAddress (after sorting): %zu", out->rf_duplicate_begins),
890 |             duplicate_loc, out->rf_duplicate_begins);
891 |     }
892 |
893 |     if (out->unwind_parse_fail) {
894 |         add_score(&score, unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_PARSE_FAIL_HIGH : SCORE_UNWIND_PARSE_FAIL_LOW);
895 |         annotate_finding(
896 |             add_finding(out, "unwind_parse_fail", "unwind_integrity", unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
897 |                         "UnwindInfo parse failures: %zu (%.2f%%)", out->unwind_parse_fail, unwind_fail_pct*100.0),
898 |             parse_fail_loc, out->unwind_parse_fail);
899 |     }
900 |     if (out->unwind_anomalies) {
901 |         add_score(&score, unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? SCORE_UNWIND_ANOM_HIGH : SCORE_UNWIND_ANOM_LOW);
902 |         annotate_finding(
903 |             add_finding(out, "unwind_anomalies", "unwind_integrity", unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? "high" :
    "medium",
904 |                         "UnwindInfo anomalies (version/opcode/chain/etc): %zu (%.2f%%)", out->unwind_anomalies,
    unwind_anom_pct*100.0),
905 |             anomaly_loc, out->unwind_anomalies);
906 |     }
907 |
908 |     bool moderate_reuse_candidate =
909 |         out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && n > 200;
910 |     if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_HIGH && n > 200) {
911 |         add_score(&score, SCORE_UNWIND_REUSE_HIGH);
912 |         FindingLocation location = { true, dominant_unwind_rva };
913 |         annotate_finding(
914 |             add_finding(out, "unwind_reuse_strong", "unwind_reuse", "high",
915 |                         "Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio=%.2f)",
916 |                         (unsigned)unwind_max, out->unwind_reuse_ratio),
917 |             location, unwind_max);
918 |     }
919 |
920 |     if (out->handler_count > 50 && out->handler_reuse_ratio > THRESHOLD_HANDLER_DOMINANCE) {
921 |         bool has_static_context =
922 |             out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 || out->rf_unsorted > 0 ||
923 |             out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 || out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||
924 |             out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 || out->unwind_writable_section > 0 ||
925 |             out->handler_nonexec_section > 0 || out->handler_writable_section > 0 ||
926 |             moderate_reuse_candidate;
927 |         const PeSection* dominant_sec = dominant_handler_rva ? pe_find_section_for_rva(img, dominant_handler_rva) : NULL;
928 |         bool dominant_runtime_like = dominant_sec &&
929 |                                      (dominant_sec->characteristics & IMAGE_SCN_MEM_EXECUTE) &&
930 |                                      sec_name_code_like(dominant_sec->name) &&
931 |                                      out->handler_nonexec_section == 0;
932 |
933 |         if (!(dominant_runtime_like && !has_static_context)) {
934 |             add_score(&score, SCORE_HANDLER_DOMINANCE);
935 |             FindingLocation location = { true, dominant_handler_rva };
936 |             annotate_finding(
937 |                 add_finding(out, "handler_dominance", "handler_dominance", "medium",
938 |                             "One handler dominates: max %u entries share same HandlerRVA (ratio=%.2f%s%s)",
939 |                             (unsigned)handler_max, out->handler_reuse_ratio,
940 |                             dominant_sec ? ", section=" : "",
941 |                             dominant_sec ? dominant_sec->name : ""),
942 |                 location, handler_max);
943 |         }
944 |     }
945 |
946 |     if (moderate_reuse_candidate &&
947 |         !has_finding_code(out, "unwind_reuse_strong") &&
948 |         (has_finding_code(out, "handler_dominance") ||
949 |          out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 ||
950 |          out->rf_unsorted > 0 || out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 ||
951 |          out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||
952 |          out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 ||
953 |          out->unwind_writable_section > 0 || out->handler_nonexec_section > 0 ||
954 |          out->handler_writable_section > 0)) {
```

```
955 |         add_score(&score, SCORE_UNWIND_REUSE_MEDIUM);
956 |         FindingLocation location = { true, dominant_unwind_rva };
957 |         annotate_finding(
958 |             add_finding(out, "unwind_reuse_moderate", "unwind_reuse", "medium",
959 |                         "Moderate UnwindInfo reuse is corroborated by another static signal: max %u entries share same UnwindInfoRVA
    (ratio=%.2f)",
960 |                         (unsigned)unwind_max, out->unwind_reuse_ratio),
961 |             location, unwind_max);
962 |     }
963 |
964 |     /* Placement heuristics */
965 |     if (out->unwind_exec_section) {
966 |         add_score(&score, SCORE_UNWIND_EXEC_SECTION);
967 |         annotate_finding(
968 |             add_finding(out, "unwind_in_exec_section", "metadata_placement", "high",
969 |                         "UnwindInfo located in executable section(s): %zu", out->unwind_exec_section),
970 |             executable_unwind_loc, out->unwind_exec_section);
971 |     }
972 |     if (out->unwind_unusual_section) {
973 |         int add = (out->unwind_unusual_section > (n / 20)) ? SCORE_UNWIND_UNUSUAL_SECTION_HIGH : SCORE_UNWIND_UNUSUAL_SECTION_LOW; /*
    >5% => broader placement pattern */
974 |         add_score(&score, add);
975 |         annotate_finding(
976 |             add_finding(out, "unwind_unusual_section", "metadata_placement", add >= 15 ? "high" : "medium",
977 |                         "UnwindInfo located in unusual sections (not .xdata/.rdata/.pdata): %zu", out->unwind_unusual_section),
978 |             unusual_unwind_loc, out->unwind_unusual_section);
979 |     }
980 |     if (out->unwind_writable_section) {
981 |         int add = (out->unwind_writable_section > (n / 20)) ? SCORE_UNWIND_WRITABLE_SECTION_HIGH : SCORE_UNWIND_WRITABLE_SECTION_LOW;
982 |         add_score(&score, add);
983 |         annotate_finding(
984 |             add_finding(out, "unwind_writable_section", "metadata_placement", add >= 18 ? "high" : "medium",
985 |                         "UnwindInfo located in writable section(s): %zu", out->unwind_writable_section),
986 |             writable_unwind_loc, out->unwind_writable_section);
987 |     }
988 |     if (out->handler_nonexec_section) {
989 |         add_score(&score, SCORE_HANDLER_NONEXEC);
990 |         annotate_finding(
991 |             add_finding(out, "handler_nonexec_section", "metadata_placement", "high",
992 |                         "Exception handler RVA in non-executable/unknown section: %zu", out->handler_nonexec_section),
993 |             nonexec_handler_loc, out->handler_nonexec_section);
994 |     }
995 |     if (out->handler_writable_section) {
996 |         add_score(&score, SCORE_HANDLER_WRITABLE);
997 |         annotate_finding(
998 |             add_finding(out, "handler_writable_section", "metadata_placement", "medium",
999 |                         "Exception handler RVA in writable section(s): %zu", out->handler_writable_section),
1000 |             writable_handler_loc, out->handler_writable_section);
1001 |     }
1002 |
1003 |     if (out->unwind_exec_section > 0 && out->handler_nonexec_section > 0) {
1004 |         add_score(&score, SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER);
1005 |         add_finding(out, "placement_exec_nonexec_handler_combo", "metadata_placement", "high",
1006 |                     "Unwind metadata in executable section(s) coincides with handler targets in non-executable/unknown section(s);
    the combined placement pattern is much harder to explain benignly.");
1007 |     }
1008 |     if ((out->unwind_exec_section > 0 || out->handler_nonexec_section > 0) &&
1009 |         (out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 || out->rf_unwind_oob > 0)) {
1010 |         add_score(&score, SCORE_COMBO_PLACEMENT_AND_INTEGRITY);
1011 |         add_finding(out, "placement_integrity_combo", "unwind_integrity", "medium",
1012 |                     "Suspicious metadata placement is corroborated by UNWIND_INFO / UnwindInfoRVA integrity problems; treat the
    combination as stronger than either signal alone.");
1013 |     }
1014 |     if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && has_finding_code(out, "handler_dominance")) {
1015 |         add_score(&score, SCORE_COMBO_REUSE_AND_DOMINANCE);
1016 |         add_finding(out, "reuse_dominance_combo", "unwind_reuse", "medium",
1017 |                     "Heavy UnwindInfo reuse coincides with a dominant handler target, which is consistent with centralized
    dispatcher-style metadata reuse.");
1018 |     }
1019 |     if (out->unwind_writable_section > 0 && out->handler_writable_section > 0) {
1020 |         add_score(&score, SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER);
1021 |         add_finding(out, "writable_metadata_handler_combo", "metadata_placement", "high",
1022 |                     "Writable unwind metadata coincides with writable handler targets; this combination is uncommon for normal
    compiler-produced images and deserves close review.");
1023 |     }
1024 |
1025 |     if (score == 0) {
1026 |         append_note(out, "No strong anomalies detected by heuristics.");
1027 |     }
1028 |     out->score = score;
1029 |     finalize_verdict(out);
1030 |
1031 |     hmap_free(&unwind_counts);
1032 |     hmap_free(&handler_counts);
1033 |     if (!sb_ok(&out->details)) {
1034 |         sb_append(err, "Out of memory while building heuristic details.\n");
1035 |         return false;
1036 |     }
1037 |     return true;
1038 | }
1039 |
1040 | void heuristics_free(HeuristicReport* r) {
1041 |     if (!r) return;
1042 |     sb_free(&r->details);
1043 |     memset(r, 0, sizeof(*r));
1044 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "heuristics.h"`	Технически подключает `heuristics.h`. Логически это дает модулю доступ к: модель finding, mitigation, score, confidence и verdict.
2	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
3	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
4	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.

Стр.	Код	Нормальное объяснение
5	`#include <stdarg.h>`	Технически подключает `stdarg.h`. Логически это дает модулю доступ к: va_list для функций вида appendf/printf.
7	`#ifndef IMAGE_SCN_MEM_EXECUTE`	Технически включает код только при условии `#ifndef IMAGE_SCN_MEM_EXECUTE`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
8	`#define IMAGE_SCN_MEM_EXECUTE 0x20000000u`	Технически объявляет макрос `IMAGE_SCN_MEM_EXECUTE` со значением `0x20000000u`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
9	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
10	`#ifndef IMAGE_SCN_MEM_WRITE`	Технически включает код только при условии `#ifndef IMAGE_SCN_MEM_WRITE`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
11	`#define IMAGE_SCN_MEM_WRITE 0x80000000u`	Технически объявляет макрос `IMAGE_SCN_MEM_WRITE` со значением `0x80000000u`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
12	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
14	`#define UA_ABI_MASK \`	Технически объявляет макрос `UA_ABI_MASK` со значением `\`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
15	`(UA_ABI_REGISTER_CLASS | UA_ABI_RESERVED_OPINFO | UA_ABI_PUSH_ORDER | \`	Технически выполняет битовую операцию `(UA_ABI_REGISTER_CLASS | UA_ABI_RESERVED_OPINFO | UA_ABI_PUSH_ORDER | \`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
16	`UA_ABI_CHAIN_RESTRICTION | UA_ABI_SAVE_OFFSET)`	Технически выполняет битовую операцию `UA_ABI_CHAIN_RESTRICTION | UA_ABI_SAVE_OFFSET)`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
19	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
20	`* Heuristic calibration notes`	Технически это комментарий исходника. Логически он объясняет разработчику: Heuristic calibration notes.
21	`* ----------------------------`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.
22	`* These constants intentionally form a conservative engineering model rather`	Технически это комментарий исходника. Логически он объясняет разработчику: These constants intentionally form a conservative engineering model rather.
23	`* than a mathematically validated classifier. The weights are grouped by the`	Технически это комментарий исходника. Логически он объясняет разработчику: than a mathematically validated classifier. The weights are grouped by the.
24	`* evidence family discussed in the thesis so the rationale stays stable and`	Технически это комментарий исходника. Логически он объясняет разработчику: evidence family discussed in the thesis so the rationale stays stable and.
25	`* reviewable:`	Технически это комментарий исходника. Логически он объясняет разработчику: reviewable:.
26	`* - table/unwind integrity violations => strongest static weights`	Технически это комментарий исходника. Логически он объясняет разработчику: - table/unwind integrity violations => strongest static weights.
27	`* - placement anomalies => medium/high depending on prevalence`	Технически это комментарий исходника. Логически он объясняет разработчику: - placement anomalies => medium/high depending on prevalence.
28	`* - reuse/dominance patterns => moderate unless they are very strong`	Технически это комментарий исходника. Логически он объясняет разработчику: - reuse/dominance patterns => moderate unless they are very strong.
29	`* - mitigations reduce confidence when evidence is isolated or sample size is tiny`	Технически это комментарий исходника. Логически он объясняет разработчику: - mitigations reduce confidence when evidence is isolated or sample size is tiny.
30	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
31	`static const int SCORE_EMPTY_PDATA = 10;`	Технически объявляет константу веса `SCORE_EMPTY_PDATA` со значением `10`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
32	`static const int SCORE_PE_SECTION_RAW_OOB = 25;`	Технически объявляет константу веса `SCORE_PE_SECTION_RAW_OOB` со значением `25`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
33	`static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;`	Технически объявляет константу веса `SCORE_PE_SECTION_VIRTUAL_OOB` со значением `20`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
34	`static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;`	Технически объявляет константу веса `SCORE_PE_SECTION_RAW_OVERLAP` со значением `15`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
35	`static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;`	Технически объявляет константу веса `SCORE_PE_SECTION_VIRTUAL_OVERLAP` со значением `20`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
36	`static const int SCORE_PE_SECTION_MISALIGNED = 8;`	Технически объявляет константу веса `SCORE_PE_SECTION_MISALIGNED` со значением `8`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
37	`static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;`	Технически объявляет константу веса `SCORE_PE_EXCEPTION_DIRECTORY_OOB` со значением `25`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.

Стр.	Код	Нормальное объяснение
38	`static const int SCORE_INVALID_RANGE_LOW = 10;`	Технически объявляет константу веса SCORE_INVALID_RANGE_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
39	`static const int SCORE_INVALID_RANGE_HIGH = 20;`	Технически объявляет константу веса SCORE_INVALID_RANGE_HIGH со значением 20. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
40	`static const int SCORE_RF_OOB_LOW = 10;`	Технически объявляет константу веса SCORE_RF_OOB_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
41	`static const int SCORE_RF_OOB_HIGH = 25;`	Технически объявляет константу веса SCORE_RF_OOB_HIGH со значением 25. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
42	`static const int SCORE_UNWIND_RVA_OOB_LOW = 10;`	Технически объявляет константу веса SCORE_UNWIND_RVA_OOB_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
43	`static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;`	Технически объявляет константу веса SCORE_UNWIND_RVA_OOB_HIGH со значением 25. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
44	`static const int SCORE_RF_UNSORTED = 15;`	Технически объявляет константу веса SCORE_RF_UNSORTED со значением 15. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
45	`static const int SCORE_RF_OVERLAP = 25;`	Технически объявляет константу веса SCORE_RF_OVERLAP со значением 25. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
46	`static const int SCORE_DUPLICATE_BEGIN = 10;`	Технически объявляет константу веса SCORE_DUPLICATE_BEGIN со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
47	`static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;`	Технически объявляет константу веса SCORE_UNWIND_PARSE_FAIL_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
48	`static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;`	Технически объявляет константу веса SCORE_UNWIND_PARSE_FAIL_HIGH со значением 25. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
49	`static const int SCORE_UNWIND_ANOM_LOW = 10;`	Технически объявляет константу веса SCORE_UNWIND_ANOM_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
50	`static const int SCORE_UNWIND_ANOM_HIGH = 20;`	Технически объявляет константу веса SCORE_UNWIND_ANOM_HIGH со значением 20. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
51	`static const int SCORE_UNWIND_REUSE_MEDIUM = 10;`	Технически объявляет константу веса SCORE_UNWIND_REUSE_MEDIUM со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
52	`static const int SCORE_UNWIND_REUSE_HIGH = 20;`	Технически объявляет константу веса SCORE_UNWIND_REUSE_HIGH со значением 20. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
53	`static const int SCORE_HANDLER_DOMINANCE = 15;`	Технически объявляет константу веса SCORE_HANDLER_DOMINANCE со значением 15. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
54	`static const int SCORE_UNWIND_EXEC_SECTION = 20;`	Технически объявляет константу веса SCORE_UNWIND_EXEC_SECTION со значением 20. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
55	`static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;`	Технически объявляет константу веса SCORE_UNWIND_UNUSUAL_SECTION_LOW со значением 8. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
56	`static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;`	Технически объявляет константу веса SCORE_UNWIND_UNUSUAL_SECTION_HIGH со значением 15. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
57	`static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;`	Технически объявляет константу веса SCORE_UNWIND_WRITABLE_SECTION_LOW со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
58	`static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;`	Технически объявляет константу веса SCORE_UNWIND_WRITABLE_SECTION_HIGH со значением 18. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
59	`static const int SCORE_HANDLER_NONEXEC = 15;`	Технически объявляет константу веса SCORE_HANDLER_NONEXEC со значением 15. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
60	`static const int SCORE_HANDLER_WRITABLE = 10;`	Технически объявляет константу веса SCORE_HANDLER_WRITABLE со значением 10. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.

Стр.	Код	Нормальное объяснение
61	`static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;`	Технически объявляет константу веса `SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER` со значением `10`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
62	`static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;`	Технически объявляет константу веса `SCORE_COMBO_PLACEMENT_AND_INTEGRITY` со значением `8`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
63	`static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;`	Технически объявляет константу веса `SCORE_COMBO_REUSE_AND_DOMINANCE` со значением `8`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
64	`static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;`	Технически объявляет константу веса `SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER` со значением `10`. Логически это числовой вклад признака в итоговый score: чем прямее нарушение PE/unwind-контракта, тем выше такой вес.
66	`static const double CONFIDENCE_BASE = 0.15;`	Технически объявляет коэффициент confidence `CONFIDENCE_BASE` со значением `0.15`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
67	`static const double CONFIDENCE_ZERO_SCORE_BASE = 0.20;`	Технически объявляет коэффициент confidence `CONFIDENCE_ZERO_SCORE_BASE` со значением `0.20`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
68	`static const double CONFIDENCE_PER_FAMILY = 0.11;`	Технически объявляет коэффициент confidence `CONFIDENCE_PER_FAMILY` со значением `0.11`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
69	`static const double CONFIDENCE_PER_STRONG = 0.08;`	Технически объявляет коэффициент confidence `CONFIDENCE_PER_STRONG` со значением `0.08`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
70	`static const double CONFIDENCE_PER_MEDIUM = 0.04;`	Технически объявляет коэффициент confidence `CONFIDENCE_PER_MEDIUM` со значением `0.04`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
71	`static const double CONFIDENCE_BONUS_TABLE_HARD = 0.08;`	Технически объявляет коэффициент confidence `CONFIDENCE_BONUS_TABLE_HARD` со значением `0.08`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.

Стр.	Код	Нормальное объяснение
72	`static const double CONFIDENCE_BONUS_UNWIND_HARD = 0.08;`	Технически объявляет коэффициент confidence `CONFIDENCE_BONUS_UNWIND_HARD` со значением `0.08`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
73	`static const double CONFIDENCE_PENALTY_SINGLE_SIGNAL = 0.12;`	Технически объявляет коэффициент confidence `CONFIDENCE_PENALTY_SINGLE_SIGNAL` со значением `0.12`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
74	`static const double CONFIDENCE_PENALTY_SMALL_SAMPLE = 0.06;`	Технически объявляет коэффициент confidence `CONFIDENCE_PENALTY_SMALL_SAMPLE` со значением `0.06`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
75	`static const double CONFIDENCE_PENALTY_PLACEMENT_ONLY = 0.10;`	Технически объявляет коэффициент confidence `CONFIDENCE_PENALTY_PLACEMENT_ONLY` со значением `0.10`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
76	`static const double CONFIDENCE_PENALTY_SINGLE_FAMILY = 0.08;`	Технически объявляет коэффициент confidence `CONFIDENCE_PENALTY_SINGLE_FAMILY` со значением `0.08`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
77	`static const double CONFIDENCE_MIN = 0.05;`	Технически объявляет коэффициент confidence `CONFIDENCE_MIN` со значением `0.05`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
78	`static const double CONFIDENCE_MAX = 0.90;`	Технически объявляет коэффициент confidence `CONFIDENCE_MAX` со значением `0.90`. Логически он регулирует уверенность verdict, отдельно от самой подозрительности score.
80	`static const double THRESHOLD_HIGH_PERCENT = 0.01; /* >1% of entries => not just a one-off glitch */`	Технически присваивает `static const double THRESHOLD_HIGH_PERCENT` значение `0.01; /* >1% of entries => not just a one-off glitch */`. Логически это обновляет `static const double THRESHOLD_HIGH_PERCENT`: хранит промежуточное значение, которое дальше используется в текущей функции.
81	`static const double THRESHOLD_UNWIND_ANOM_HIGH = 0.05; /* >5% anomalies => broad integrity issue */`	Технически присваивает `static const double THRESHOLD_UNWIND_ANOM_HIGH` значение `0.05; /* >5% anomalies => broad integrity issue */`. Логически это обновляет `static const double THRESHOLD_UNWIND_ANOM_HIGH`: хранит промежуточное значение, которое дальше используется в текущей функции.
82	`static const double THRESHOLD_UNWIND_REUSE_MEDIUM = 0.10;`	Технически объявляет порог `THRESHOLD_UNWIND_REUSE_MEDIUM` со значением `0.10`. Логически это граница, после которой единичная странность считается устойчивым паттерном, а не случайным шумом.
83	`static const double THRESHOLD_UNWIND_REUSE_HIGH = 0.30;`	Технически объявляет порог `THRESHOLD_UNWIND_REUSE_HIGH` со значением `0.30`. Логически это граница, после которой единичная странность считается устойчивым паттерном, а не случайным шумом.
84	`static const double THRESHOLD_HANDLER_DOMINANCE = 0.80;`	Технически объявляет порог `THRESHOLD_HANDLER_DOMINANCE` со значением `0.80`. Логически это граница, после которой единичная странность считается устойчивым паттерном, а не случайным шумом.
86	`static bool sec_name_eq(const char* a, const char* b) {`	Технически объявляет/начинает функцию `sec_name_eq`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
87	`if (!a || !b) return false;`	Технически проверяет условие `!a || !b`; логически это защита от NULL-указателя и работы с неинициализированными данными.
88	`while (*a && *b) {`	Технически начинает цикл while с условием `*a && *b`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.

Стр.	Код	Нормальное объяснение
89	`char ca = *a;`	Технически объявляет `ca` в функции `sec_name_eq` и сразу задает начальное значение `*a`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
90	`char cb = *b;`	Технически объявляет `cb` в функции `sec_name_eq` и сразу задает начальное значение `*b`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
91	`if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');`	Технически проверяет условие `ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a';` если оно истинно, выполняется защитная или аналитическая ветка ниже.
92	`if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');`	Технически проверяет условие `cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a';` если оно истинно, выполняется защитная или аналитическая ветка ниже.
93	`if (ca != cb) return false;`	Технически проверяет условие `ca != cb;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
94	`a++; b++;`	Технически выполняет оператор `a++; b++;`. Логически в функции `sec_name_eq` он меняет состояние, проверяемые данные или подготовку отчета.
95	`}`	Технически завершает область видимости внутри `sec_name_eq`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
96	`return *a == 0 && *b == 0;`	Технически возвращает выражение `*a == 0 && *b == 0`. Логически именно это значение становится результатом функции `sec_name_eq` для вызывающего кода.
97	`}`	Технически завершает область видимости внутри `sec_name_eq`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
99	`static bool sec_name_contains_ci(const char* haystack, const char* needle) {`	Технически объявляет/начинает функцию `sec_name_contains_ci;` она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
100	`if (!haystack || !needle || !*needle) return false;`	Технически проверяет условие `!haystack || !needle || !*needle;` логически это защита от NULL-указателя и работы с неинициализированными данными.
101	`size_t nlen = strlen(needle);`	Технически присваивает `size_t nlen` значение `strlen(needle)`. Логически это обновляет `size_t nlen`: хранит размер или количество, поэтому используется в проверках границ.
102	`for (const char* h = haystack; *h; h++) {`	Технически начинает цикл `const char* h = haystack; *h; h++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
103	`size_t i = 0;`	Технически объявляет `i` в функции `sec_name_contains_ci` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
104	`while (i < nlen) {`	Технически начинает цикл while с условием `i < nlen`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
105	`char ca = h[i];`	Технически объявляет `ca` в функции `sec_name_contains_ci` и сразу задает начальное значение `h[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
106	`char cb = needle[i];`	Технически объявляет `cb` в функции `sec_name_contains_ci` и сразу задает начальное значение `needle[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
107	`if (!ca) return false;`	Технически проверяет условие `!ca;` логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
108	`if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');`	Технически проверяет условие `ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a';` если оно истинно, выполняется защитная или аналитическая ветка ниже.
109	`if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');`	Технически проверяет условие `cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a';` если оно истинно, выполняется защитная или аналитическая ветка ниже.
110	`if (ca != cb) break;`	Технически проверяет условие `ca != cb;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
111	`i++;`	Технически увеличивает `i` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
112	`}`	Технически завершает область видимости внутри `sec_name_contains_ci`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
113	`if (i == nlen) return true;`	Технически проверяет условие `i == nlen;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
114	`}`	Технически завершает область видимости внутри `sec_name_contains_ci`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
115	`return false;`	Технически возвращает `false` из функции `sec_name_contains_ci`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
116	`}`	Технически завершает область видимости внутри `sec_name_contains_ci`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
118	`static bool sec_name_allowed_for_unwind(const char* name8) {`	Технически объявляет/начинает функцию `sec_name_allowed_for_unwind;` она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
119	`/* Common placements: .xdata (MSVC), .rdata/.pdata in practice, and toolchain-specific xdata-like sections. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Common placements: .xdata (MSVC), .rdata/.pdata in practice, and toolchain-specific xdata-like sections..
120	`return sec_name_eq(name8, ".xdata") || sec_name_eq(name8, ".rdata") || sec_name_eq(name8, ".pdata") ||`	Технически возвращает выражение `sec_name_eq(name8, ".xdata") || sec_name_eq(name8, ".rdata") || sec_name_eq(name8, ".pdata") ||`. Логически именно это значение становится результатом функции `sec_name_allowed_for_unwind` для вызывающего кода.
121	`sec_name_eq(name8, ".gnu_xdata") || sec_name_eq(name8, ".mrdata") ||`	Технически вызывает `sec_name_eq`. Логически это действие нужно в функции `sec_name_allowed_for_unwind`: вызывает функцию `sec_name_eq` для выполнения части алгоритма.
122	`sec_name_eq(name8, "xdata") || sec_name_eq(name8, "rdata") || sec_name_eq(name8, "pdata");`	Технически вызывает `sec_name_eq`. Логически это действие нужно в функции `sec_name_allowed_for_unwind`: вызывает функцию `sec_name_eq` для выполнения части алгоритма.
123	`}`	Технически завершает область видимости внутри `sec_name_allowed_for_unwind`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
125	`static bool sec_name_plausibly_unwind_named(const char* name8) {`	Технически объявляет/начинает функцию `sec_name_plausibly_unwind_named`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
126	`return sec_name_contains_ci(name8, "xdata") || sec_name_contains_ci(name8, "pdata") || sec_name_contains_ci(name8, "rdata");`	Технически объявляет прототип функции `sec_name_contains_ci` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
127	`}`	Технически завершает область видимости внутри `sec_name_plausibly_unwind_named`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
129	`static bool sec_name_code_like(const char* name8) {`	Технически объявляет/начинает функцию `sec_name_code_like`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
130	`return sec_name_eq(name8, ".text") || sec_name_eq(name8, "text") ||`	Технически возвращает выражение `sec_name_eq(name8, ".text") || sec_name_eq(name8, "text") ||`. Логически именно это значение становится результатом функции `sec_name_code_like` для вызывающего кода.
131	`sec_name_contains_ci(name8, "text") || sec_name_contains_ci(name8, "code") || sec_name_contains_ci(name8, "init");`	Технически вызывает `sec_name_contains_ci`. Логически это действие нужно в функции `sec_name_code_like`: вызывает функцию `sec_name_contains_ci` для выполнения части алгоритма.
132	`}`	Технически завершает область видимости внутри `sec_name_code_like`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
134	`static bool image_has_executable_section(const PeImage* img) {`	Технически объявляет/начинает функцию `image_has_executable_section`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
135	`if (!img || !img->sections) return false;`	Технически проверяет условие `!img || !img->sections`; логически это защита от NULL-указателя и работы с неинициализированными данными.
136	`for (size_t i = 0; i < img->num_sections; i++) {`	Технически начинает цикл `size_t i = 0; i < img->num_sections; i++`. Логически он проходит по секциям PE-файла, чтобы проверить каждую запись, а не только первую.
137	`const PeSection* section = &img->sections[i];`	Технически объявляет `section` в функции `image_has_executable_section` и сразу задает начальное значение `&img->sections[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
138	`if ((section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0 &&`	Технически проверяет условие `(section->characteristics & IMAGE_SCN_MEM_EXECUTE`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
139	`(section->virtual_size != 0 || section->raw_size != 0)) {`	Технически выполняет битовую операцию `(section->virtual_size != 0 || section->raw_size != 0)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
140	`return true;`	Технически возвращает `true` из функции `image_has_executable_section`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
141	`}`	Технически завершает область видимости внутри `image_has_executable_section`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
142	`}`	Технически завершает область видимости внутри `image_has_executable_section`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
143	`return false;`	Технически возвращает `false` из функции `image_has_executable_section`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
144	`}`	Технически завершает область видимости внутри `image_has_executable_section`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
146	`/* Very small u32->count hash map (open addressing, linear probing). */`	Технически это комментарий исходника. Логически он объясняет разработчику: Very small u32->count hash map (open addressing, linear probing)..
147	`typedef struct HEntry {`	Технически начинает описание структуры `HEntry`. Логически структура собирает связанные поля в одну модель данных проекта.
148	`uint32_t key;`	Технически объявляет поле `key` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
149	`    uint32_t val;`	Технически объявляет поле `val` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
150	`    bool used;`	Технически объявляет поле `used` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
151	`} HEntry;`	Технически завершает typedef и дает типу имя `HEntry`. Логически дальше код может объявлять переменные этого типа без `struct`.
153	`typedef struct HMap {`	Технически начинает описание структуры `HMap`. Логически структура собирает связанные поля в одну модель данных проекта.
154	`    HEntry* e;`	Технически объявляет поле `e` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
155	`    size_t cap;`	Технически объявляет поле `cap` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
156	`    size_t len;`	Технически объявляет поле `len` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
157	`} HMap;`	Технически завершает typedef и дает типу имя `HMap`. Логически дальше код может объявлять переменные этого типа без `struct`.
159	`static uint32_t h32(uint32_t x) {`	Технически объявляет/начинает функцию `h32`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
160	`    /* xorshift-ish */`	Технически это комментарий исходника. Логически он объясняет разработчику: xorshift-ish.
161	`    x ^= x >> 16;`	Технически применяет битовую операцию `^=` к `x` со значением `x >> 16`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
162	`    x *= 0x7feb352dU;`	Технически изменяет `x` оператором `*=` на основе `0x7feb352dU`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
163	`    x ^= x >> 15;`	Технически применяет битовую операцию `^=` к `x` со значением `x >> 15`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
164	`    x *= 0x846ca68bU;`	Технически изменяет `x` оператором `*=` на основе `0x846ca68bU`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
165	`    x ^= x >> 16;`	Технически применяет битовую операцию `^=` к `x` со значением `x >> 16`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
166	`    return x;`	Технически возвращает выражение `x`. Логически именно это значение становится результатом функции `h32` для вызывающего кода.
167	`}`	Технически завершает область видимости внутри `h32`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
169	`static bool hmap_init(HMap* m, size_t cap) {`	Технически объявляет/начинает функцию `hmap_init`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
170	`    if (!m) return false;`	Технически проверяет условие `!m`; логически это защита от NULL-указателя и работы с неинициализированными данными.
171	`    m->cap = cap < 64 ? 64 : cap;`	Технически объявляет поле `cap` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
172	`    m->len = 0;`	Технически присваивает `m->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
173	`    m->e = (HEntry*)calloc(m->cap, sizeof(HEntry));`	Технически присваивает `m->e` значение `(HEntry*)calloc(m->cap, sizeof(HEntry))`. Логически это обновляет `e`: хранит промежуточное значение, которое дальше используется в текущей функции.
174	`    return m->e != NULL;`	Технически возвращает выражение `m->e != NULL`. Логически именно это значение становится результатом функции `hmap_init` для вызывающего кода.
175	`}`	Технически завершает область видимости внутри `hmap_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
177	`static void hmap_free(HMap* m) {`	Технически объявляет/начинает функцию `hmap_free`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
178	`    if (!m) return;`	Технически проверяет условие `!m`; логически это защита от NULL-указателя и работы с неинициализированными данными.
179	`    free(m->e);`	Технически вызывает `free`. Логически это действие нужно в функции `hmap_free`: освобождает память, чтобы в C не было утечки.
180	`    m->e = NULL;`	Технически присваивает `m->e` значение `NULL`. Логически это обновляет `e`: хранит промежуточное значение, которое дальше используется в текущей функции.
181	`    m->cap = 0;`	Технически присваивает `m->cap` значение `0`. Логически это обновляет `cap`: хранит промежуточное значение, которое дальше используется в текущей функции.
182	`    m->len = 0;`	Технически присваивает `m->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
183	`}`	Технически завершает область видимости внутри `hmap_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
185	`static bool hmap_grow(HMap* m);`	Технически объявляет прототип функции `hmap_grow` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
187	`static bool hmap_inc(HMap* m, uint32_t key, uint32_t* out_newval) {`	Технически объявляет/начинает функцию `hmap_inc`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
188	`if (!m || !m->e) return false;`	Технически проверяет условие `!m || !m->e`; логически это защита от NULL-указателя и работы с неинициализированными данными.
189	`if ((m->len + 1) * 10 >= m->cap * 7) {`	Технически проверяет условие `(m->len + 1) * 10 >= m->cap * 7`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
190	`if (!hmap_grow(m)) return false;`	Технически проверяет условие `!hmap_grow(m)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
191	`}`	Технически завершает область видимости внутри `hmap_inc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
192	`uint32_t hv = h32(key);`	Технически присваивает `uint32_t hv` значение `h32(key)`. Логически это обновляет `uint32_t hv`: хранит промежуточное значение, которое дальше используется в текущей функции.
193	`size_t idx = (size_t)hv % m->cap;`	Технически присваивает `size_t idx` значение `(size_t)hv % m->cap`. Логически это обновляет `size_t idx`: хранит размер или количество, поэтому используется в проверках границ.
194	`for (size_t step = 0; step < m->cap; step++) {`	Технически начинает цикл `size_t step = 0; step < m->cap; step++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
195	`HEntry* ent = &m->e[idx];`	Технически объявляет `ent` в функции `hmap_inc` и сразу задает начальное значение `&m->e[idx]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
196	`if (!ent->used) {`	Технически проверяет условие `!ent->used`; логически это защита от NULL-указателя и работы с неинициализированными данными.
197	`ent->used = true;`	Технически присваивает `ent->used` значение `true`. Логически это обновляет `used`: хранит промежуточное значение, которое дальше используется в текущей функции.
198	`ent->key = key;`	Технически присваивает `ent->key` значение `key`. Логически это обновляет `key`: хранит промежуточное значение, которое дальше используется в текущей функции.
199	`ent->val = 1;`	Технически присваивает `ent->val` значение `1`. Логически это обновляет `val`: хранит промежуточное значение, которое дальше используется в текущей функции.
200	`m->len++;`	Технически увеличивает `m->len` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
201	`if (out_newval) *out_newval = 1;`	Технически проверяет условие `out_newval`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
202	`return true;`	Технически возвращает `true` из функции `hmap_inc`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
203	`}`	Технически завершает область видимости внутри `hmap_inc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
204	`if (ent->key == key) {`	Технически проверяет условие `ent->key == key`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
205	`ent->val++;`	Технически увеличивает `ent->val` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
206	`if (out_newval) *out_newval = ent->val;`	Технически проверяет условие `out_newval`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
207	`return true;`	Технически возвращает `true` из функции `hmap_inc`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
208	`}`	Технически завершает область видимости внутри `hmap_inc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
209	`idx = (idx + 1) % m->cap;`	Технически присваивает `idx` значение `(idx + 1) % m->cap`. Логически это обновляет `idx`: хранит промежуточное значение, которое дальше используется в текущей функции.
210	`}`	Технически завершает область видимости внутри `hmap_inc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
211	`return false;`	Технически возвращает `false` из функции `hmap_inc`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
212	`}`	Технически завершает область видимости внутри `hmap_inc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
214	`static bool hmap_grow(HMap* m) {`	Технически объявляет/начинает функцию `hmap_grow`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
215	`size_t oldcap = m->cap;`	Технически объявляет `oldcap` в функции `hmap_grow` и сразу задает начальное значение `m->cap`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
216	`HEntry* old = m->e;`	Технически объявляет `old` в функции `hmap_grow` и сразу задает начальное значение `m->e`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
217	`if (oldcap > (SIZE_MAX - 1u) / 2u) return false;`	Технически проверяет условие `oldcap > (SIZE_MAX - 1u) / 2u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
218	`size_t newcap = oldcap * 2 + 1;`	Технически объявляет `newcap` в функции `hmap_grow` и сразу задает начальное значение `oldcap * 2 + 1`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
219	`HEntry* ne = (HEntry*)calloc(newcap, sizeof(HEntry));`	Технически присваивает `HEntry* ne` значение `(HEntry*)calloc(newcap, sizeof(HEntry))`. Логически это обновляет `HEntry* ne`: хранит промежуточное значение, которое дальше используется в текущей функции.
220	`if (!ne) return false;`	Технически проверяет условие `!ne`; логически это защита от NULL-указателя и работы с неинициализированными данными.
222	`m->e = ne;`	Технически присваивает `m->e` значение `ne`. Логически это обновляет `e`: хранит промежуточное значение, которое дальше используется в текущей функции.
223	`m->cap = newcap;`	Технически присваивает `m->cap` значение `newcap`. Логически это обновляет `cap`: хранит промежуточное значение, которое дальше используется в текущей функции.
224	`m->len = 0;`	Технически присваивает `m->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
226	`for (size_t i = 0; i < oldcap; i++) {`	Технически начинает цикл `size_t i = 0; i < oldcap; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
227	`if (!old[i].used) continue;`	Технически проверяет условие `!old[i].used`; логически это защита от NULL-указателя и работы с неинициализированными данными.
228	`/* reinsert */`	Технически это комментарий исходника. Логически он объясняет разработчику: reinsert.
229	`uint32_t key = old[i].key;`	Технически объявляет `key` в функции `hmap_grow` и сразу задает начальное значение `old[i].key`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
230	`uint32_t val = old[i].val;`	Технически объявляет `val` в функции `hmap_grow` и сразу задает начальное значение `old[i].val`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
231	`uint32_t hv = h32(key);`	Технически присваивает `uint32_t hv` значение `h32(key)`. Логически это обновляет `uint32_t hv`: хранит промежуточное значение, которое дальше используется в текущей функции.
232	`size_t idx = (size_t)hv % m->cap;`	Технически присваивает `size_t idx` значение `(size_t)hv % m->cap`. Логически это обновляет `size_t idx`: хранит размер или количество, поэтому используется в проверках границ.
233	`for (;;) {`	Технически начинает цикл `;;`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
234	`HEntry* ent = &m->e[idx];`	Технически объявляет `ent` в функции `hmap_grow` и сразу задает начальное значение `&m->e[idx]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
235	`if (!ent->used) {`	Технически проверяет условие `!ent->used`; логически это защита от NULL-указателя и работы с неинициализированными данными.
236	`ent->used = true;`	Технически присваивает `ent->used` значение `true`. Логически это обновляет `used`: хранит промежуточное значение, которое дальше используется в текущей функции.
237	`ent->key = key;`	Технически присваивает `ent->key` значение `key`. Логически это обновляет `key`: хранит промежуточное значение, которое дальше используется в текущей функции.
238	`ent->val = val;`	Технически присваивает `ent->val` значение `val`. Логически это обновляет `val`: хранит промежуточное значение, которое дальше используется в текущей функции.
239	`m->len++;`	Технически увеличивает `m->len` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
240	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
241	`}`	Технически завершает область видимости внутри `hmap_grow`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
242	`idx = (idx + 1) % m->cap;`	Технически присваивает `idx` значение `(idx + 1) % m->cap`. Логически это обновляет `idx`: хранит промежуточное значение, которое дальше используется в текущей функции.
243	`}`	Технически завершает область видимости внутри `hmap_grow`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
244	`}`	Технически завершает область видимости внутри `hmap_grow`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
245	`free(old);`	Технически вызывает `free`. Логически это действие нужно в функции `hmap_grow`: освобождает память, чтобы в C не было утечки.
246	`return true;`	Технически возвращает `true` из функции `hmap_grow`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.

Стр.	Код	Нормальное объяснение
247	`}`	Технически завершает область видимости внутри `hmap_grow`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
249	`static uint32_t hmap_max_val(const HMap* m) {`	Технически объявляет/начинает функцию `hmap_max_val`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
250	`uint32_t mx = 0;`	Технически объявляет `mx` в функции `hmap_max_val` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
251	`if (!m || !m->e) return 0;`	Технически проверяет условие `!m || !m->e`; логически это защита от NULL-указателя и работы с неинициализированными данными.
252	`for (size_t i = 0; i < m->cap; i++) {`	Технически начинает цикл `size_t i = 0; i < m->cap; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
253	`if (m->e[i].used && m->e[i].val > mx) mx = m->e[i].val;`	Технически проверяет условие `m->e[i].used && m->e[i].val > mx`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
254	`}`	Технически завершает область видимости внутри `hmap_max_val`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
255	`return mx;`	Технически возвращает выражение `mx`. Логически именно это значение становится результатом функции `hmap_max_val` для вызывающего кода.
256	`}`	Технически завершает область видимости внутри `hmap_max_val`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
258	`static bool hmap_max_entry(const HMap* m, uint32_t* out_key, uint32_t* out_val) {`	Технически объявляет/начинает функцию `hmap_max_entry`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
259	`uint32_t best_key = 0;`	Технически объявляет `best_key` в функции `hmap_max_entry` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
260	`uint32_t best_val = 0;`	Технически объявляет `best_val` в функции `hmap_max_entry` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
261	`if (!m || !m->e) return false;`	Технически проверяет условие `!m || !m->e`; логически это защита от NULL-указателя и работы с неинициализированными данными.
262	`for (size_t i = 0; i < m->cap; i++) {`	Технически начинает цикл `size_t i = 0; i < m->cap; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
263	`if (!m->e[i].used) continue;`	Технически проверяет условие `!m->e[i].used`; логически это защита от NULL-указателя и работы с неинициализированными данными.
264	`if (m->e[i].val > best_val) {`	Технически проверяет условие `m->e[i].val > best_val`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
265	`best_key = m->e[i].key;`	Технически присваивает `best_key` значение `m->e[i].key`. Логически это обновляет `best_key`: хранит промежуточное значение, которое дальше используется в текущей функции.
266	`best_val = m->e[i].val;`	Технически присваивает `best_val` значение `m->e[i].val`. Логически это обновляет `best_val`: хранит промежуточное значение, которое дальше используется в текущей функции.
267	`}`	Технически завершает область видимости внутри `hmap_max_entry`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
268	`}`	Технически завершает область видимости внутри `hmap_max_entry`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
269	`if (best_val == 0) return false;`	Технически проверяет условие `best_val == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
270	`if (out_key) *out_key = best_key;`	Технически проверяет условие `out_key`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
271	`if (out_val) *out_val = best_val;`	Технически проверяет условие `out_val`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
272	`return true;`	Технически возвращает `true` из функции `hmap_max_entry`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
273	`}`	Технически завершает область видимости внутри `hmap_max_entry`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
275	`static size_t hmap_unique(const HMap* m) { return m ? m->len : 0; }`	Технически вызывает `hmap_unique`. Логически это действие нужно в функции `глобальной области файла`: считает количество уникальных RVA.
277	`typedef struct RfIdx {`	Технически начинает описание структуры `RfIdx`. Логически структура собирает связанные поля в одну модель данных проекта.
278	`uint32_t begin;`	Технически объявляет поле `begin` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
279	`uint32_t end;`	Технически объявляет поле end внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
280	`} RfIdx;`	Технически завершает typedef и дает типу имя `RfIdx`. Логически дальше код может объявлять переменные этого типа без `struct`.
282	`typedef struct FindingLocation {`	Технически начинает описание структуры `FindingLocation`. Логически структура собирает связанные поля в одну модель данных проекта.
283	`bool set;`	Технически объявляет поле `set` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
284	`uint32_t rva;`	Технически объявляет поле `rva` внутри структуры. Логически это поле: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
285	`} FindingLocation;`	Технически завершает typedef и дает типу имя `FindingLocation`. Логически дальше код может объявлять переменные этого типа без `struct`.
287	`static void remember_location(FindingLocation* location, uint32_t rva) {`	Технически объявляет/начинает функцию `remember_location`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
288	`if (!location || location->set) return;`	Технически проверяет условие `!location || location->set`; логически это защита от NULL-указателя и работы с неинициализированными данными.
289	`location->set = true;`	Технически объявляет поле `true` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
290	`location->rva = rva;`	Технически объявляет поле `rva` внутри структуры. Логически это поле: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
291	`}`	Технически завершает область видимости внутри `remember_location`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
293	`enum PeSectionIssue {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.

Стр.	Код	Нормальное объяснение
294	`PE_ISSUE_RAW_OOB,`	Технически эта строка является частью C-конструкции `PE_ISSUE_RAW_OOB,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
295	`PE_ISSUE_VIRTUAL_OOB,`	Технически эта строка является частью C-конструкции `PE_ISSUE_VIRTUAL_OOB,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
296	`PE_ISSUE_RAW_OVERLAP,`	Технически эта строка является частью C-конструкции `PE_ISSUE_RAW_OVERLAP,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
297	`PE_ISSUE_VIRTUAL_OVERLAP,`	Технически эта строка является частью C-конструкции `PE_ISSUE_VIRTUAL_OVERLAP,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
298	`PE_ISSUE_MISALIGNED`	Технически объявляет поле `PE_ISSUE_MISALIGNED` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
299	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
301	`static FindingLocation first_pe_section_issue(const PeImage* img, enum PeSectionIssue issue) {`	Технически объявляет/начинает функцию `first_pe_section_issue`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
302	`FindingLocation result = {0};`	Технически объявляет `result` в функции `first_pe_section_issue` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
303	`if (!img || !img->sections) return result;`	Технически проверяет условие `!img || !img->sections`; логически это защита от NULL-указателя и работы с неинициализированными данными.
305	`for (size_t i = 0; i < img->num_sections; i++) {`	Технически начинает цикл `size_t i = 0; i < img->num_sections; i++`. Логически он проходит по секциям PE-файла, чтобы проверить каждую запись, а не только первую.
306	`const PeSection* s = &img->sections[i];`	Технически объявляет `s` в функции `first_pe_section_issue` и сразу задает начальное значение `&img->sections[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
307	`uint64_t span = s->virtual_size > s->raw_size ? s->virtual_size : s->raw_size;`	Технически объявляет `span` в функции `first_pe_section_issue` и сразу задает начальное значение `s->virtual_size > s->raw_size ? s->virtual_size : s->raw_size`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
308	`bool match = false;`	Технически объявляет `match` в функции `first_pe_section_issue` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
309	`if (issue == PE_ISSUE_RAW_OOB) {`	Технически проверяет условие `issue == PE_ISSUE_RAW_OOB`; логически это защита от чтения за пределами файла на диске.
310	`match = s->raw_size > 0 &&`	Технически присваивает `match` значение `s->raw_size > 0 &&`. Логически это обновляет `match`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
311	((uint64_t)s->raw_ptr + (uint64_t)s->raw_size > (uint64_t)img->file_size);	Технически выполняет оператор ((uint64_t)s->raw_ptr + (uint64_t)s->raw_size > (uint64_t)img->file_size);. Логически в функции first_pe_section_issue он меняет состояние, проверяемые данные или подготовку отчета.
312	} else if (issue == PE_ISSUE_VIRTUAL_OOB) {	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
313	match = span > 0 &&	Технически присваивает match значение span > 0 &&. Логически это обновляет match: хранит промежуточное значение, которое дальше используется в текущей функции.
314	((uint64_t)s->virtual_address + span > (uint64_t)img->size_of_image);	Технически выполняет оператор ((uint64_t)s->virtual_address + span > (uint64_t)img->size_of_image);. Логически в функции first_pe_section_issue он меняет состояние, проверяемые данные или подготовку отчета.
315	} else if (issue == PE_ISSUE_MISALIGNED) {	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
316	match = (img->section_alignment > 0 &&	Технически присваивает match значение (img->section_alignment > 0 &&. Логически это обновляет match: хранит промежуточное значение, которое дальше используется в текущей функции.
317	(s->virtual_address % img->section_alignment) != 0) ||	Технически выполняет битовую операцию (s->virtual_address % img->section_alignment) != 0) ||. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
318	(s->raw_size > 0 && img->file_alignment > 0 &&	Технически выполняет битовую операцию (s->raw_size > 0 && img->file_alignment > 0 &&. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
319	(s->raw_ptr % img->file_alignment) != 0);	Технически выполняет оператор (s->raw_ptr % img->file_alignment) != 0);. Логически в функции first_pe_section_issue он меняет состояние, проверяемые данные или подготовку отчета.
320	} else {	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
321	for (size_t j = 0; j < i && !match; j++) {	Технически начинает цикл size_t j = 0; j < i && !match; j++. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
322	const PeSection* p = &img->sections[j];	Технически объявляет p в функции first_pe_section_issue и сразу задает начальное значение &img->sections[j]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
323	uint64_t pspan = p->virtual_size > p->raw_size ? p->virtual_size : p->raw_size;	Технически объявляет pspan в функции first_pe_section_issue и сразу задает начальное значение p->virtual_size > p->raw_size ? p->virtual_size : p->raw_size. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
324	if (issue == PE_ISSUE_RAW_OVERLAP) {	Технически проверяет условие issue == PE_ISSUE_RAW_OVERLAP; логически это защита от чтения за пределами файла на диске.
325	match = s->raw_size > 0 && p->raw_size > 0 &&	Технически присваивает match значение s->raw_size > 0 && p->raw_size > 0 &&. Логически это обновляет match: хранит промежуточное значение, которое дальше используется в текущей функции.
326	(uint64_t)s->raw_ptr < (uint64_t)p->raw_ptr + p->raw_size &&	Технически выполняет битовую операцию (uint64_t)s->raw_ptr < (uint64_t)p->raw_ptr + p->raw_size &&. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
327	(uint64_t)p->raw_ptr < (uint64_t)s->raw_ptr + s->raw_size;	Технически выполняет оператор (uint64_t)p->raw_ptr < (uint64_t)s->raw_ptr + s->raw_size;. Логически в функции first_pe_section_issue он меняет состояние, проверяемые данные или подготовку отчета.
328	} else if (issue == PE_ISSUE_VIRTUAL_OVERLAP) {	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.

Стр.	Код	Нормальное объяснение
329	match = span > 0 && pspan > 0 &&	Технически присваивает match значение span > 0 && pspan > 0 &&. Логически это обновляет match: хранит промежуточное значение, которое дальше используется в текущей функции.
330	(uint64_t)s->virtual_address < (uint64_t)p->virtual_address + pspan &&	Технически выполняет битовую операцию (uint64_t)s->virtual_address < (uint64_t)p->virtual_address + pspan &&. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
331	(uint64_t)p->virtual_address < (uint64_t)s->virtual_address + span;	Технически выполняет оператор (uint64_t)p->virtual_address < (uint64_t)s->virtual_address + span;. Логически в функции first_pe_section_issue он меняет состояние, проверяемые данные или подготовку отчета.
332	}	Технически завершает область видимости внутри first_pe_section_issue. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
333	}	Технически завершает область видимости внутри first_pe_section_issue. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
334	}	Технически завершает область видимости внутри first_pe_section_issue. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
335	if (match) {	Технически проверяет условие match; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
336	`remember_location(&result, s->virtual_address);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `first_pe_section_issue`: сохраняет первый характерный RVA для finding.
337	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
338	`}`	Технически завершает область видимости внутри `first_pe_section_issue`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
339	`}`	Технически завершает область видимости внутри `first_pe_section_issue`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
340	`return result;`	Технически возвращает выражение `result`. Логически именно это значение становится результатом функции `first_pe_section_issue` для вызывающего кода.
341	`}`	Технически завершает область видимости внутри `first_pe_section_issue`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
343	`static int cmp_rfidx(const void* a, const void* b) {`	Технически объявляет/начинает функцию `cmp_rfidx`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
344	`const RfIdx* x = (const RfIdx*)a;`	Технически присваивает `const RfIdx* x` значение `(const RfIdx*)a`. Логически это обновляет `const RfIdx* x`: хранит промежуточное значение, которое дальше используется в текущей функции.
345	`const RfIdx* y = (const RfIdx*)b;`	Технически присваивает `const RfIdx* y` значение `(const RfIdx*)b`. Логически это обновляет `const RfIdx* y`: хранит промежуточное значение, которое дальше используется в текущей функции.
346	`if (x->begin < y->begin) return -1;`	Технически проверяет условие `x->begin < y->begin`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
347	`if (x->begin > y->begin) return 1;`	Технически проверяет условие `x->begin > y->begin`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
348	`if (x->end < y->end) return -1;`	Технически проверяет условие `x->end < y->end`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
349	`if (x->end > y->end) return 1;`	Технически проверяет условие `x->end > y->end`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
350	`return 0;`	Технически возвращает код 0 из функции `cmp_rfidx`. Логически это успешное завершение программы или callback.
351	`}`	Технически завершает область видимости внутри `cmp_rfidx`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
353	`static int clamp_int(int v, int lo, int hi) {`	Технически объявляет/начинает функцию `clamp_int`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
354	`if (v < lo) return lo;`	Технически проверяет условие `v < lo`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
355	`if (v > hi) return hi;`	Технически проверяет условие `v > hi`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
356	`return v;`	Технически возвращает выражение `v`. Логически именно это значение становится результатом функции `clamp_int` для вызывающего кода.
357	`}`	Технически завершает область видимости внутри `clamp_int`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
359	`static void add_score(int* score, int add) {`	Технически объявляет/начинает функцию `add_score`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
360	`if (!score) return;`	Технически проверяет условие `!score`; логически это защита от NULL-указателя и работы с неинициализированными данными.
361	`*score = clamp_int(*score + add, 0, 100);`	Технически это комментарий исходника. Логически он объясняет разработчику: score = clamp_int(*score + add, 0, 100);.
362	`}`	Технически завершает область видимости внутри `add_score`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
364	`static double safe_ratio(double a, double b) { return (b == 0.0) ? 0.0 : (a / b); }`	Технически вызывает `safe_ratio`. Логически это действие нужно в функции `глобальной области файла`: считает отношение без деления на ноль.
367	`const char* heuristics_evidence_family_name(uint32_t bit) {`	Технически объявляет/начинает функцию `heuristics_evidence_family_name`; она доступная другим модулям через заголовок или линковку. Логически функция переводит бит семейства доказательств в текстовое имя.

Стр.	Код	Нормальное объяснение
368	`switch (bit) {`	Технически выбирает ветку по значению `bit`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.
369	`case HEF_TABLE_INTEGRITY: return "table_integrity";`	Технически объявляет ветку `switch` для значения `HEF_TABLE_INTEGRITY: return "table_integrity";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.

Стр.	Код	Нормальное объяснение
370	`case HEF_UNWIND_INTEGRITY: return "unwind_integrity";`	Технически объявляет ветку switch для значения HEF_UNWIND_INTEGRITY: return "unwind_integrity";. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
371	`case HEF_METADATA_PLACEMENT: return "metadata_placement";`	Технически объявляет ветку switch для значения HEF_METADATA_PLACEMENT: return "metadata_placement";. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
372	`case HEF_UNWIND_REUSE: return "unwind_reuse";`	Технически объявляет ветку switch для значения HEF_UNWIND_REUSE: return "unwind_reuse";. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
373	`case HEF_HANDLER_DOMINANCE: return "handler_dominance";`	Технически объявляет ветку switch для значения HEF_HANDLER_DOMINANCE: return "handler_dominance";. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
374	`default: return "unknown";`	Технически объявляет ветку switch по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
375	`}`	Технически завершает область видимости внутри heuristics_evidence_family_name. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
376	`}`	Технически завершает область видимости внутри heuristics_evidence_family_name. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
378	`size_t heuristics_evidence_family_count(uint32_t mask) {`	Технически объявляет/начинает функцию heuristics_evidence_family_count; она доступная другим модулям через заголовок или линковку. Логически функция считает, сколько разных семейств доказательств сработало.
379	`size_t n = 0;`	Технически объявляет n в функции heuristics_evidence_family_count и сразу задает начальное значение 0. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
380	`while (mask) {`	Технически начинает цикл while с условием mask. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
381	`n += (mask & 1u) ? 1u : 0u;`	Технически изменяет n оператором += на основе (mask & 1u) ? 1u : 0u. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
382	`mask >>= 1;`	Технически применяет битовую операцию >>= к mask со значением 1. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
383	`}`	Технически завершает область видимости внутри heuristics_evidence_family_count. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
384	`return n;`	Технически возвращает выражение n. Логически именно это значение становится результатом функции heuristics_evidence_family_count для вызывающего кода.
385	`}`	Технически завершает область видимости внутри heuristics_evidence_family_count. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
387	`static uint32_t family_bit_from_name(const char* family) {`	Технически объявляет/начинает функцию family_bit_from_name; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
388	`if (!family) return 0;`	Технически проверяет условие !family; логически это защита от NULL-указателя и работы с неинициализированными данными.
389	`if (strcmp(family, "table_integrity") == 0) return HEF_TABLE_INTEGRITY;`	Технически проверяет условие strcmp(family, "table_integrity") == 0; если оно истинно, выполняется защитная или аналитическая ветка ниже.
390	`if (strcmp(family, "unwind_integrity") == 0) return HEF_UNWIND_INTEGRITY;`	Технически проверяет условие strcmp(family, "unwind_integrity") == 0; логически отбирает записи, где важна корректность UNWIND_INFO.
391	`if (strcmp(family, "metadata_placement") == 0) return HEF_METADATA_PLACEMENT;`	Технически проверяет условие strcmp(family, "metadata_placement") == 0; если оно истинно, выполняется защитная или аналитическая ветка ниже.
392	`if (strcmp(family, "unwind_reuse") == 0) return HEF_UNWIND_REUSE;`	Технически проверяет условие strcmp(family, "unwind_reuse") == 0; логически отбирает записи, где важна корректность UNWIND_INFO.
393	`if (strcmp(family, "handler_dominance") == 0) return HEF_HANDLER_DOMINANCE;`	Технически проверяет условие strcmp(family, "handler_dominance") == 0; логически отделяет функции с language-specific handler от обычных функций.
394	`return 0;`	Технически возвращает код 0 из функции family_bit_from_name. Логически это успешное завершение программы или callback.
395	`}`	Технически завершает область видимости внутри family_bit_from_name. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
397	`static void safe_copy(char* dst, size_t dst_cap, const char* src) {`	Технически объявляет/начинает функцию safe_copy; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
398	`if (!dst || dst_cap == 0) return;`	Технически проверяет условие !dst || dst_cap == 0; логически это защита от NULL-указателя и работы с неинициализированными данными.
399	`if (!src) src = "";`	Технически проверяет условие !src; логически это защита от NULL-указателя и работы с неинициализированными данными.
400	`snprintf(dst, dst_cap, "%s", src);`	Технически вызывает snprintf. Логически это действие нужно в функции safe_copy: формирует строку с ограничением размера буфера.

Стр.	Код	Нормальное объяснение
401	`}`	Технически завершает область видимости внутри `safe_copy`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
403	`#if defined(__GNUC__) || defined(__clang__)`	Технически включает код только при условии `#if defined(__GNUC__) || defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
404	`#pragma GCC diagnostic push`	Технически это директива препроцессора `#pragma GCC diagnostic push`. Логически она меняет код еще до компиляции C.

Стр.	Код	Нормальное объяснение
405	`#pragma GCC diagnostic ignored "-Wformat-nonliteral"`	Технически это директива препроцессора `#pragma GCC diagnostic ignored "-Wformat-nonliteral"`. Логически она меняет код еще до компиляции C.
406	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
408	`static HeuristicFinding* add_finding(HeuristicReport* out,`	Технически вызывает `add_finding`. Логически это действие нужно в функции `глобальной области файла`: создает finding, который попадет в отчет.
409	`const char* code,`	Технически эта строка является частью C-конструкции `const char* code,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
410	`const char* family,`	Технически эта строка является частью C-конструкции `const char* family,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
411	`const char* severity,`	Технически эта строка является частью C-конструкции `const char* severity,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
412	`const char* fmt,`	Технически эта строка является частью C-конструкции `const char* fmt,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
413	`...) {`	Технически эта строка является частью C-конструкции `...) {`. Логически она уточняет структуру блока функции `add_finding` и влияет на то, какие данные будут обработаны.
414	`if (!out || !fmt) return NULL;`	Технически проверяет условие `!out || !fmt`; логически это защита от NULL-указателя и работы с неинициализированными данными.
416	`char msg[192];`	Технически объявляет `msg` в функции `add_finding`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
417	`va_list ap;`	Технически выполняет оператор `va_list ap;`. Логически в функции `add_finding` он меняет состояние, проверяемые данные или подготовку отчета.
418	`va_start(ap, fmt);`	Технически вызывает `va_start`. Логически это действие нужно в функции `add_finding`: вызывает функцию `va_start` для выполнения части алгоритма.
419	`(void)vsnprintf(msg, sizeof(msg), fmt, ap);`	Технически вызывает `vsnprintf`. Логически это действие нужно в функции `add_finding`: вызывает функцию `vsnprintf` для выполнения части алгоритма.
420	`va_end(ap);`	Технически вызывает `va_end`. Логически это действие нужно в функции `add_finding`: вызывает функцию `va_end` для выполнения части алгоритма.
422	`out->evidence_families_mask |= family_bit_from_name(family);`	Технически применяет битовую операцию `|=` к `out->evidence_families_mask` со значением `family_bit_from_name(family)`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
424	`HeuristicFinding* stored = NULL;`	Технически объявляет `stored` в функции `add_finding` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
425	`if (out->finding_count < HEURISTICS_MAX_FINDINGS) {`	Технически проверяет условие `out->finding_count < HEURISTICS_MAX_FINDINGS`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
426	`HeuristicFinding* f = &out->findings[out->finding_count++];`	Технически объявляет `f` в функции `add_finding` и сразу задает начальное значение `&out->findings[out->finding_count++]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
427	`memset(f, 0, sizeof(*f));`	Технически вызывает `memset`. Логически это действие нужно в функции `add_finding`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
428	`safe_copy(f->code, sizeof(f->code), code ? code : "unspecified");`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_finding`: копирует строку с гарантированным нулевым окончанием.
429	`safe_copy(f->family, sizeof(f->family), family ? family : "unknown");`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_finding`: копирует строку с гарантированным нулевым окончанием.
430	`safe_copy(f->severity, sizeof(f->severity), severity ? severity : "info");`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_finding`: копирует строку с гарантированным нулевым окончанием.
431	`safe_copy(f->message, sizeof(f->message), msg);`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_finding`: копирует строку с гарантированным нулевым окончанием.
432	`stored = f;`	Технически присваивает `stored` значение `f`. Логически это обновляет `stored`: хранит промежуточное значение, которое дальше используется в текущей функции.
433	`}`	Технически завершает область видимости внутри `add_finding`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
435	`sb_appendf(&out->details, "- [%s/%s] %s\n",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `add_finding`: добавляет форматированный текст в динамический буфер.
436	`severity ? severity : "info",`	Технически эта строка является частью С-конструкции `severity ? severity : "info",`. Логически она уточняет структуру блока функции `add_finding` и влияет на то, какие данные будут обработаны.
437	`code ? code : "unspecified",`	Технически эта строка является частью С-конструкции `code ? code : "unspecified",`. Логически она уточняет структуру блока функции `add_finding` и влияет на то, какие данные будут обработаны.
438	`msg);`	Технически выполняет оператор `msg);`. Логически в функции `add_finding` он меняет состояние, проверяемые данные или подготовку отчета.
439	`return stored;`	Технически возвращает выражение `stored`. Логически именно это значение становится результатом функции `add_finding` для вызывающего кода.
440	`}`	Технически завершает область видимости внутри `add_finding`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
442	`static void annotate_finding(HeuristicFinding* finding,`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `глобальной области файла`: добавляет к finding representative RVA и число затронутых объектов.
443	`FindingLocation location,`	Технически эта строка является частью С-конструкции `FindingLocation location,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
444	`size_t affected_count) {`	Технически эта строка является частью С-конструкции `size_t affected_count) {`. Логически она уточняет структуру блока функции `annotate_finding` и влияет на то, какие данные будут обработаны.
445	`if (!finding) return;`	Технически проверяет условие `!finding`; логически это защита от NULL-указателя и работы с неинициализированными данными.
446	`finding->has_location = location.set;`	Технически присваивает `finding->has_location` значение `location.set`. Логически это обновляет `has_location`: есть ли у finding конкретный representative RVA.
447	`finding->representative_rva = location.rva;`	Технически присваивает `finding->representative_rva` значение `location.rva`. Логически это обновляет `representative_rva`: примерный адрес, где обнаружена проблема.
448	`finding->affected_count = affected_count;`	Технически присваивает `finding->affected_count` значение `affected_count`. Логически это обновляет `affected_count`: сколько записей/функций затронуто признаком.
449	`}`	Технически завершает область видимости внутри `annotate_finding`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
451	`static void append_note(HeuristicReport* out, const char* fmt, ...) {`	Технически объявляет/начинает функцию `append_note`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
452	`if (!out || !fmt) return;`	Технически проверяет условие `!out || !fmt`; логически это защита от NULL-указателя и работы с неинициализированными данными.
454	`char msg[192];`	Технически объявляет `msg` в функции `append_note`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
455	`va_list ap;`	Технически выполняет оператор `va_list ap;`. Логически в функции `append_note` он меняет состояние, проверяемые данные или подготовку отчета.
456	`va_start(ap, fmt);`	Технически вызывает `va_start`. Логически это действие нужно в функции `append_note`: вызывает функцию `va_start` для выполнения части алгоритма.
457	`(void)vsnprintf(msg, sizeof(msg), fmt, ap);`	Технически вызывает `vsnprintf`. Логически это действие нужно в функции `append_note`: вызывает функцию `vsnprintf` для выполнения части алгоритма.
458	`va_end(ap);`	Технически вызывает `va_end`. Логически это действие нужно в функции `append_note`: вызывает функцию `va_end` для выполнения части алгоритма.
460	`sb_appendf(&out->details, "- [info] %s\n", msg);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `append_note`: добавляет форматированный текст в динамический буфер.
461	`}`	Технически завершает область видимости внутри `append_note`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
463	`static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev) {`	Технически объявляет/начинает функцию `count_findings_with_severity`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
464	`size_t n = 0;`	Технически объявляет `n` в функции `count_findings_with_severity` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
465	`if (!out || !sev) return 0;`	Технически проверяет условие `!out || !sev`; логически это защита от NULL-указателя и работы с неинициализированными данными.
466	`for (size_t i = 0; i < out->finding_count; i++) {`	Технически начинает цикл `size_t i = 0; i < out->finding_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
467	`if (strcmp(out->findings[i].severity, sev) == 0) n++;`	Технически проверяет условие `strcmp(out->findings[i].severity, sev) == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
468	`}`	Технически завершает область видимости внутри `count_findings_with_severity`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
469	`return n;`	Технически возвращает выражение `n`. Логически именно это значение становится результатом функции `count_findings_with_severity` для вызывающего кода.
470	`}`	Технически завершает область видимости внутри `count_findings_with_severity`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
472	`static bool has_finding_code(const HeuristicReport* out, const char* code) {`	Технически объявляет/начинает функцию `has_finding_code`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
473	`if (!out || !code) return false;`	Технически проверяет условие `!out || !code`; логически это защита от NULL-указателя и работы с неинициализированными данными.
474	`for (size_t i = 0; i < out->finding_count; i++) {`	Технически начинает цикл `size_t i = 0; i < out->finding_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
475	`if (strcmp(out->findings[i].code, code) == 0) return true;`	Технически проверяет условие `strcmp(out->findings[i].code, code) == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
476	`}`	Технически завершает область видимости внутри `has_finding_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
477	`return false;`	Технически возвращает `false` из функции `has_finding_code`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
478	`}`	Технически завершает область видимости внутри `has_finding_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
480	`static void add_mitigation(HeuristicReport* out, const char* code, const char* fmt, ...) {`	Технически объявляет/начинает функцию `add_mitigation`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
481	`if (!out || !fmt) return;`	Технически проверяет условие `!out || !fmt`; логически это защита от NULL-указателя и работы с неинициализированными данными.
482	`if (out->mitigation_count >= HEURISTICS_MAX_MITIGATIONS) return;`	Технически проверяет условие `out->mitigation_count >= HEURISTICS_MAX_MITIGATIONS`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
484	`char msg[192];`	Технически объявляет `msg` в функции `add_mitigation`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
485	`va_list ap;`	Технически выполняет оператор `va_list ap;`. Логически в функции `add_mitigation` он меняет состояние, проверяемые данные или подготовку отчета.
486	`va_start(ap, fmt);`	Технически вызывает `va_start`. Логически это действие нужно в функции `add_mitigation`: вызывает функцию `va_start` для выполнения части алгоритма.
487	`(void)vsnprintf(msg, sizeof(msg), fmt, ap);`	Технически вызывает `vsnprintf`. Логически это действие нужно в функции `add_mitigation`: вызывает функцию `vsnprintf` для выполнения части алгоритма.
488	`va_end(ap);`	Технически вызывает `va_end`. Логически это действие нужно в функции `add_mitigation`: вызывает функцию `va_end` для выполнения части алгоритма.
490	`HeuristicMitigation* m = &out->mitigations[out->mitigation_count++];`	Технически объявляет `m` в функции `add_mitigation` и сразу задает начальное значение `&out->mitigations[out->mitigation_count++]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
491	`memset(m, 0, sizeof(*m));`	Технически вызывает `memset`. Логически это действие нужно в функции `add_mitigation`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
492	`safe_copy(m->code, sizeof(m->code), code ? code : "unspecified");`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_mitigation`: копирует строку с гарантированным нулевым окончанием.
493	`safe_copy(m->message, sizeof(m->message), msg);`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `add_mitigation`: копирует строку с гарантированным нулевым окончанием.
495	`sb_appendf(&out->details, " [mitigation/%s] %s\n",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `add_mitigation`: добавляет форматированный текст в динамический буфер.
496	`code ? code : "unspecified",`	Технически эта строка является частью С-конструкции `code ? code : "unspecified",`. Логически она уточняет структуру блока функции `add_mitigation` и влияет на то, какие данные будут обработаны.
497	`msg);`	Технически выполняет оператор `msg);`. Логически в функции `add_mitigation` он меняет состояние, проверяемые данные или подготовку отчета.
498	`}`	Технически завершает область видимости внутри `add_mitigation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
500	`#if defined(__GNUC__) || defined(__clang__)`	Технически включает код только при условии `#if defined(__GNUC__) || defined(__clang__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
501	`#pragma GCC diagnostic pop`	Технически это директива препроцессора `#pragma GCC diagnostic pop`. Логически она меняет код еще до компиляции С.
502	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Стр.	Код	Нормальное объяснение
504	`static const char* evidence_level_from_counts(size_t weak_count,`	Технически вызывает `evidence_level_from_counts`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `evidence_level_from_counts` для выполнения части алгоритма.
505	`size_t medium_count,`	Технически эта строка является частью C-конструкции `size_t medium_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
506	`size_t strong_count,`	Технически эта строка является частью C-конструкции `size_t strong_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
507	`int score) {`	Технически эта строка является частью C-конструкции `int score) {`. Логически она уточняет структуру блока функции `evidence_level_from_counts` и влияет на то, какие данные будут обработаны.
508	`if (strong_count >= 2 || score >= 70) return "strong";`	Технически проверяет условие `strong_count >= 2 || score >= 70`; логически выбирает уровень оценки или уверенности.
509	`if (strong_count >= 1 || medium_count >= 2 || score >= 30) return "medium";`	Технически проверяет условие `strong_count >= 1 || medium_count >= 2 || score >= 30`; логически выбирает уровень оценки или уверенности.
510	`if (weak_count || medium_count || strong_count || score > 0) return "weak";`	Технически проверяет условие `weak_count || medium_count || strong_count || score > 0`; логически выбирает уровень оценки или уверенности.
511	`return "none";`	Технически возвращает выражение `"none"`. Логически именно это значение становится результатом функции `evidence_level_from_counts` для вызывающего кода.
512	`}`	Технически завершает область видимости внутри `evidence_level_from_counts`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
514	`static void finalize_verdict(HeuristicReport* out) {`	Технически объявляет/начинает функцию `finalize_verdict`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
515	`if (!out) return;`	Технически проверяет условие `!out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
517	`out->suspicion = safe_ratio((double)out->score, 100.0);`	Технически присваивает `out->suspicion` значение `safe_ratio((double)out->score, 100.0)`. Логически это обновляет `suspicion`: score, нормализованный в диапазон 0..1.
519	`size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);`	Технически присваивает `size_t family_count` значение `heuristics_evidence_family_count(out->evidence_families_mask)`. Логически это обновляет `size_t family_count`: хранит размер или количество, поэтому используется в проверках границ.
520	`size_t strong_count = count_findings_with_severity(out, "high") + count_findings_with_severity(out, "critical");`	Технически присваивает `size_t strong_count` значение `count_findings_with_severity(out, "high") + count_findings_with_severity(out, "critical")`. Логически это обновляет `size_t strong_count`: хранит размер или количество, поэтому используется в проверках границ.
521	`size_t medium_count = count_findings_with_severity(out, "medium");`	Технически присваивает `size_t medium_count` значение `count_findings_with_severity(out, "medium")`. Логически это обновляет `size_t medium_count`: хранит размер или количество, поэтому используется в проверках границ.
522	`size_t weak_count = 0;`	Технически объявляет `weak_count` в функции `finalize_verdict` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
523	`if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count - strong_count - medium_count;`	Технически проверяет условие `out->finding_count >= strong_count + medium_count`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
525	`out->strong_finding_count = strong_count;`	Технически присваивает `out->strong_finding_count` значение `strong_count`. Логически это обновляет `strong_finding_count`: хранит размер или количество, поэтому используется в проверках границ.
526	`out->medium_finding_count = medium_count;`	Технически присваивает `out->medium_finding_count` значение `medium_count`. Логически это обновляет `medium_finding_count`: хранит размер или количество, поэтому используется в проверках границ.
527	`out->weak_finding_count = weak_count;`	Технически присваивает `out->weak_finding_count` значение `weak_count`. Логически это обновляет `weak_finding_count`: хранит размер или количество, поэтому используется в проверках границ.
528	`safe_copy(out->evidence_level, sizeof(out->evidence_level),`	Технически вызывает `safe_copy`. Логически это действие нужно в функции `finalize_verdict`: копирует строку с гарантированным нулевым окончанием.
529	`evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));`	Технически вызывает `evidence_level_from_counts`. Логически это действие нужно в функции `finalize_verdict`: вызывает функцию `evidence_level_from_counts` для выполнения части алгоритма.
531	`double confidence = CONFIDENCE_BASE;`	Технически объявляет `confidence` в функции `finalize_verdict` и сразу задает начальное значение `CONFIDENCE_BASE`. Логически это место для данных: уверенность в выводе с учетом силы и разнообразия доказательств.
532	`confidence += (double)family_count * CONFIDENCE_PER_FAMILY;`	Технически изменяет `confidence` оператором `+=` на основе `(double)family_count * CONFIDENCE_PER_FAMILY`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
533	`confidence += (double)strong_count * CONFIDENCE_PER_STRONG;`	Технически изменяет `confidence` оператором `+=` на основе `(double)strong_count * CONFIDENCE_PER_STRONG`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.

Стр.	Код	Нормальное объяснение
534	`confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;`	Технически изменяет `confidence` оператором += на основе `(double)medium_count * CONFIDENCE_PER_MEDIUM`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
536	`if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps) confidence += CONFIDENCE_BONUS_TABLE_HARD;`	Технически проверяет условие `out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps`; логически отбирает записи, где важна корректность UNWIND_INFO.
537	`if (out->unwind_parse_fail || out->unwind_anomalies) confidence += CONFIDENCE_BONUS_UNWIND_HARD;`	Технически проверяет условие `out->unwind_parse_fail || out->unwind_anomalies`; логически отбирает записи, где важна корректность UNWIND_INFO.
538	`if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;`	Технически проверяет условие `out->score == 0`; логически выбирает уровень оценки или уверенности.
540	`/* Weakening rules: lower confidence when only isolated / non-correlated static signals exist. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Weakening rules: lower confidence when only isolated / non-correlated static signals exist..
541	`if (out->finding_count == 1 && strong_count == 0) {`	Технически проверяет условие `out->finding_count == 1 && strong_count == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
542	`add_mitigation(out, "single_signal_only",`	Технически вызывает `add_mitigation`. Логически это действие нужно в функции `finalize_verdict`: добавляет объяснение, почему confidence надо снизить.
543	`"Only one non-strong signal contributed; treat as weak standalone evidence.");`	Технически выполняет оператор `"Only one non-strong signal contributed; treat as weak standalone evidence.");`. Логически в функции `finalize_verdict` он меняет состояние, проверяемые данные или подготовку отчета.
544	`confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;`	Технически изменяет `confidence` оператором -= на основе `CONFIDENCE_PENALTY_SINGLE_SIGNAL`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
545	`}`	Технически завершает область видимости внутри `finalize_verdict`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
546	`if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {`	Технически проверяет условие `out->total < 32 && strong_count == 0 && out->finding_count > 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
547	`add_mitigation(out, "small_sample_size",`	Технически вызывает `add_mitigation`. Логически это действие нужно в функции `finalize_verdict`: добавляет объяснение, почему confidence надо снизить.
548	`"Very small .pdata sample can overstate ratios; confidence reduced until more functions are observed.");`	Технически выполняет оператор `"Very small .pdata sample can overstate ratios; confidence reduced until more functions are observed.");`. Логически в функции `finalize_verdict` он меняет состояние, проверяемые данные или подготовку отчета.
549	`confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;`	Технически изменяет `confidence` оператором -= на основе `CONFIDENCE_PENALTY_SMALL_SAMPLE`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
550	`}`	Технически завершает область видимости внутри `finalize_verdict`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
551	`if (out->unwind_unusual_section > 0 &&`	Технически проверяет условие `if (out->unwind_unusual_section > 0 &&`; логически отбирает записи, где важна корректность UNWIND_INFO.
552	`out->unwind_exec_section == 0 &&`	Технически выполняет битовую операцию `out->unwind_exec_section == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
553	`out->unwind_writable_section == 0 &&`	Технически выполняет битовую операцию `out->unwind_writable_section == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
554	`out->handler_nonexec_section == 0 &&`	Технически выполняет битовую операцию `out->handler_nonexec_section == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
555	`out->handler_writable_section == 0 &&`	Технически выполняет битовую операцию `out->handler_writable_section == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
556	`out->rf_invalid_ranges == 0 &&`	Технически выполняет битовую операцию `out->rf_invalid_ranges == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
557	`out->rf_oob == 0 &&`	Технически выполняет битовую операцию `out->rf_oob == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
558	`out->rf_unwind_oob == 0 &&`	Технически выполняет битовую операцию `out->rf_unwind_oob == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
559	`out->unwind_parse_fail == 0 &&`	Технически выполняет битовую операцию `out->unwind_parse_fail == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
560	`out->unwind_anomalies == 0) {`	Технически эта строка является частью C-конструкции `out->unwind_anomalies == 0) {`. Логически она уточняет структуру блока функции `finalize_verdict` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
561	`add_mitigation(out, "placement_only_signal",`	Технически вызывает `add_mitigation`. Логически это действие нужно в функции `finalize_verdict`: добавляет объяснение, почему confidence надо снизить.
562	`"Placement-only anomalies may come from uncommon but benign layouts; manual verification required.");`	Технически выполняет оператор `"Placement-only anomalies may come from uncommon but benign layouts; manual verification required.");`. Логически в функции `finalize_verdict` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
563	`confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;`	Технически изменяет `confidence` оператором `-=` на основе `CONFIDENCE_PENALTY_PLACEMENT_ONLY`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
564	`}`	Технически завершает область видимости внутри `finalize_verdict`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
565	`if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {`	Технически проверяет условие `(out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
566	`add_mitigation(out, "single_family_only",`	Технически вызывает `add_mitigation`. Логически это действие нужно в функции `finalize_verdict`: добавляет объяснение, почему confidence надо снизить.
567	`"Evidence comes from only one family; cross-layer corroboration is recommended before strong conclusions.");`	Технически выполняет оператор `"Evidence comes from only one family; cross-layer corroboration is recommended before strong conclusions.");`. Логически в функции `finalize_verdict` он меняет состояние, проверяемые данные или подготовку отчета.
568	`confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;`	Технически изменяет `confidence` оператором `-=` на основе `CONFIDENCE_PENALTY_SINGLE_FAMILY`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
569	`}`	Технически завершает область видимости внутри `finalize_verdict`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
571	`if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;`	Технически проверяет условие `confidence < CONFIDENCE_MIN`; логически выбирает уровень оценки или уверенности.
572	`if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;`	Технически проверяет условие `confidence > CONFIDENCE_MAX`; логически выбирает уровень оценки или уверенности.
573	`out->confidence = confidence;`	Технически присваивает `out->confidence` значение `confidence`. Логически это обновляет `confidence`: уверенность в выводе с учетом силы и разнообразия доказательств.
575	`out->requires_manual_review =`	Технически присваивает `out->requires_manual_review` значение `` `. Логически это обновляет requires_manual_review` ``: флаг, что результат надо проверить вручную.
576	`(out->score >= 20) ||`	Технически выполняет битовую операцию `(out->score >= 20) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
577	`(out->unwind_exec_section > 0) ||`	Технически выполняет битовую операцию `(out->unwind_exec_section > 0) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
578	`(out->unwind_writable_section > 0) ||`	Технически выполняет битовую операцию `(out->unwind_writable_section > 0) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
579	`(out->handler_nonexec_section > 0) ||`	Технически выполняет битовую операцию `(out->handler_nonexec_section > 0) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
580	`(out->handler_writable_section > 0) ||`	Технически выполняет битовую операцию `(out->handler_writable_section > 0) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
581	`(out->unwind_parse_fail > 0) ||`	Технически выполняет битовую операцию `(out->unwind_parse_fail > 0) ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
582	`(out->mitigation_count > 0);`	Технически выполняет оператор `(out->mitigation_count > 0);`. Логически в функции `finalize_verdict` он меняет состояние, проверяемые данные или подготовку отчета.
584	`if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");`	Технически проверяет условие `out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion"`; логически выбирает уровень оценки или уверенности.
585	`else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict), "moderate_suspicion");`	Технически объявляет прототип функции `if` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
586	`else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");`	Технически объявляет прототип функции `if` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
587	`else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");`	Технически объявляет прототип функции `safe_copy` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
588	`}`	Технически завершает область видимости внутри `finalize_verdict`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
590	`bool heuristics_analyze(const PeImage* img,`	Технически вызывает `heuristics_analyze`. Логически это действие нужно в функции `глобальной области файла`: главная функция аналитики: считает признаки, веса, confidence, mitigations и verdict.
591	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
592	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
593	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
594	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
595	`HeuristicReport* out,`	Технически эта строка является частью C-конструкции `HeuristicReport* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
596	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
597	`if (!img || !out || !err) return false;`	Технически проверяет условие `!img || !out || !err`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
598	`if (n > 0 && (!rf || !us || !us_ok)) return false;`	Технически проверяет условие `n > 0 && (!rf || !us || !us_ok)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
599	`memset(out, 0, sizeof(*out));`	Технически вызывает `memset`. Логически это действие нужно в функции `heuristics_analyze`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
600	`if (!sb_init(&out->details, 1024)) {`	Технически проверяет условие `!sb_init(&out->details, 1024)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
601	`sb_append(err, "Out of memory (details buffer).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
602	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
603	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
604	`out->total = n;`	Технически присваивает `out->total` значение `n`. Логически это обновляет `total`: хранит промежуточное значение, которое дальше используется в текущей функции.
605	`out->pe_section_raw_oob = img->section_raw_oob;`	Технически присваивает `out->pe_section_raw_oob` значение `img->section_raw_oob`. Логически это обновляет `pe_section_raw_oob`: хранит промежуточное значение, которое дальше используется в текущей функции.
606	`out->pe_section_virtual_oob = img->section_virtual_oob;`	Технически присваивает `out->pe_section_virtual_oob` значение `img->section_virtual_oob`. Логически это обновляет `pe_section_virtual_oob`: хранит промежуточное значение, которое дальше используется в текущей функции.
607	`out->pe_section_raw_overlaps = img->section_raw_overlaps;`	Технически присваивает `out->pe_section_raw_overlaps` значение `img->section_raw_overlaps`. Логически это обновляет `pe_section_raw_overlaps`: хранит промежуточное значение, которое дальше используется в текущей функции.
608	`out->pe_section_virtual_overlaps = img->section_virtual_overlaps;`	Технически присваивает `out->pe_section_virtual_overlaps` значение `img->section_virtual_overlaps`. Логически это обновляет `pe_section_virtual_overlaps`: хранит промежуточное значение, которое дальше используется в текущей функции.
609	`out->pe_section_misaligned = img->section_misaligned;`	Технически присваивает `out->pe_section_misaligned` значение `img->section_misaligned`. Логически это обновляет `pe_section_misaligned`: хранит промежуточное значение, которое дальше используется в текущей функции.
610	`out->pe_exception_directory_oob = img->exception_directory_oob;`	Технически присваивает `out->pe_exception_directory_oob` значение `img->exception_directory_oob`. Логически это обновляет `pe_exception_directory_oob`: хранит промежуточное значение, которое дальше используется в текущей функции.
612	`/* RUNTIME_FUNCTION sanity */`	Технически это комментарий исходника. Логически он объясняет разработчику: RUNTIME_FUNCTION sanity.
613	`uint32_t prev_begin = 0;`	Технически объявляет `prev_begin` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
614	`bool prev_valid = false;`	Технически объявляет `prev_valid` в функции `heuristics_analyze` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
615	`size_t unsorted = 0;`	Технически объявляет `unsorted` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
616	`FindingLocation invalid_range_loc = {0};`	Технически объявляет `invalid_range_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
617	`FindingLocation rf_oob_loc = {0};`	Технически объявляет `rf_oob_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
618	`FindingLocation unwind_oob_loc = {0};`	Технически объявляет `unwind_oob_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
619	`FindingLocation unsorted_loc = {0};`	Технически объявляет `unsorted_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
620	`FindingLocation overlap_loc = {0};`	Технически объявляет `overlap_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
621	`FindingLocation duplicate_loc = {0};`	Технически объявляет `duplicate_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
622	`FindingLocation parse_fail_loc = {0};`	Технически объявляет `parse_fail_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
623	`FindingLocation anomaly_loc = {0};`	Технически объявляет `anomaly_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
624	`FindingLocation unusual_unwind_loc = {0};`	Технически объявляет `unusual_unwind_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
625	`FindingLocation executable_unwind_loc = {0};`	Технически объявляет `executable_unwind_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
626	`FindingLocation writable_unwind_loc = {0};`	Технически объявляет `writable_unwind_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
627	`FindingLocation nonexec_handler_loc = {0};`	Технически объявляет `nonexec_handler_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
628	`FindingLocation writable_handler_loc = {0};`	Технически объявляет `writable_handler_loc` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
630	`HMap unwind_counts = {0};`	Технически объявляет `unwind_counts` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
631	`HMap handler_counts = {0};`	Технически объявляет `handler_counts` в функции `heuristics_analyze` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
632	`if (n > (SIZE_MAX - 1u) / 2u) {`	Технически проверяет условие `n > (SIZE_MAX - 1u) / 2u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
633	`sb_append(err, "Runtime function count is too large for reuse analysis.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
634	`sb_free(&out->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `heuristics_analyze`: освобождает память динамического буфера.
635	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
636	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
637	`if (!hmap_init(&unwind_counts, n * 2 + 1u) ||`	Технически проверяет условие `!hmap_init(&unwind_counts, n * 2 + 1u`; логически это защита от NULL-указателя и работы с неинициализированными данными.
638	`!hmap_init(&handler_counts, 128)) {`	Технически объявляет/начинает функцию `hmap_init`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
639	`sb_append(err, "Out of memory (hash maps).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
640	`hmap_free(&unwind_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
641	`hmap_free(&handler_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
642	`sb_free(&out->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `heuristics_analyze`: освобождает память динамического буфера.
643	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
644	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
646	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
647	`uint32_t b = rf[i].begin_rva;`	Технически объявляет `b` в функции `heuristics_analyze` и сразу задает начальное значение `rf[i].begin_rva`. Логически это место для данных: хранит буфер, с которым работает текущий модуль.

Стр.	Код	Нормальное объяснение
648	`uint32_t e = rf[i].end_rva;`	Технически объявляет e в функции heuristics_analyze и сразу задает начальное значение rf[i].end_rva. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
649	`uint32_t u = rf[i].unwind_rva;`	Технически объявляет u в функции heuristics_analyze и сразу задает начальное значение rf[i].unwind_rva. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
651	`if (e <= b) {`	Технически проверяет условие e <= b; если оно истинно, выполняется защитная или аналитическая ветка ниже.
652	`out->rf_invalid_ranges++;`	Технически увеличивает out->rf_invalid_ranges на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
653	`remember_location(&invalid_range_loc, b);`	Технически вызывает remember_location. Логически это действие нужно в функции heuristics_analyze: сохраняет первый характерный RVA для finding.
654	`}`	Технически завершает область видимости внутри heuristics_analyze. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
655	`if (b >= img->size_of_image || e > img->size_of_image) {`	Технически проверяет условие b >= img->size_of_image || e > img->size_of_image; логически это защита от выхода RVA/диапазона за SizeOfImage.
656	`out->rf_oob++;`	Технически увеличивает out->rf_oob на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
657	`remember_location(&rf_oob_loc, b);`	Технически вызывает remember_location. Логически это действие нужно в функции heuristics_analyze: сохраняет первый характерный RVA для finding.
658	`}`	Технически завершает область видимости внутри heuristics_analyze. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
659	`if (u >= img->size_of_image || u == 0) {`	Технически проверяет условие u >= img->size_of_image || u == 0; логически это защита от выхода RVA/диапазона за SizeOfImage.
660	`out->rf_unwind_oob++;`	Технически увеличивает out->rf_unwind_oob на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
661	`remember_location(&unwind_oob_loc, u);`	Технически вызывает remember_location. Логически это действие нужно в функции heuristics_analyze: сохраняет первый характерный RVA для finding.
662	`}`	Технически завершает область видимости внутри heuristics_analyze. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
664	`if (prev_valid && b < prev_begin) {`	Технически проверяет условие prev_valid && b < prev_begin; если оно истинно, выполняется защитная или аналитическая ветка ниже.
665	`unsorted++;`	Технически увеличивает unsorted на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
666	`remember_location(&unsorted_loc, b);`	Технически вызывает remember_location. Логически это действие нужно в функции heuristics_analyze: сохраняет первый характерный RVA для finding.
667	`}`	Технически завершает область видимости внутри heuristics_analyze. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
668	`prev_begin = b;`	Технически присваивает prev_begin значение b. Логически это обновляет prev_begin: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
669	`prev_valid = true;`	Технически присваивает prev_valid значение true. Логически это обновляет prev_valid: хранит промежуточное значение, которое дальше используется в текущей функции.
671	`if (!hmap_inc(&unwind_counts, u, NULL)) {`	Технически проверяет условие !hmap_inc(&unwind_counts, u, NULL); логически это защита от NULL-указателя и работы с неинициализированными данными.
672	`sb_append(err, "Out of memory while counting unwind metadata reuse.\n");`	Технически вызывает sb_append. Логически это действие нужно в функции heuristics_analyze: добавляет текст в динамический буфер.
673	`hmap_free(&unwind_counts);`	Технически вызывает hmap_free. Логически это действие нужно в функции heuristics_analyze: вызывает функцию hmap_free для выполнения части алгоритма.
674	`hmap_free(&handler_counts);`	Технически вызывает hmap_free. Логически это действие нужно в функции heuristics_analyze: вызывает функцию hmap_free для выполнения части алгоритма.
675	`sb_free(&out->details);`	Технически вызывает sb_free. Логически это действие нужно в функции heuristics_analyze: освобождает память динамического буфера.
676	`return false;`	Технически возвращает false из функции heuristics_analyze. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
677	`}`	Технически завершает область видимости внутри heuristics_analyze. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
679	`if (us_ok[i] && us[i].has_handler) {`	Технически проверяет условие us_ok[i] && us[i].has_handler; логически отделяет функции с language-specific handler от обычных функций.
680	`out->handler_count++;`	Технически увеличивает out->handler_count на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.

Стр.	Код	Нормальное объяснение
681	`if (!hmap_inc(&handler_counts, us[i].handler_rva, NULL)) {`	Технически проверяет условие `!hmap_inc(&handler_counts, us[i].handler_rva, NULL)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
682	`sb_append(err, "Out of memory while counting handler reuse.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
683	`hmap_free(&unwind_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
684	`hmap_free(&handler_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
685	`sb_free(&out->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `heuristics_analyze`: освобождает память динамического буфера.
686	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
687	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
688	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
690	`/* Section placement sanity: unwind info is typically in non-executable data (.xdata/.rdata).`	Технически это комментарий исходника. Логически он объясняет разработчику: Section placement sanity: unwind info is typically in non-executable data (.xdata/.rdata)..
691	`Placing unwind metadata in executable / unusual sections is a common interception/tampering artifact. */`	Технически эта строка является частью C-конструкции `Placing unwind metadata in executable / unusual sections is a common interception/tampering artifact. */`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
692	`if (us_ok[i]) {`	Технически проверяет условие `us_ok[i]`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
693	`const PeSection* su = pe_find_section_for_rva(img, u);`	Технически присваивает `const PeSection* su` значение `pe_find_section_for_rva(img, u)`. Логически это обновляет `const PeSection* su`: хранит промежуточное значение, которое дальше используется в текущей функции.
694	`if (!su) {`	Технически проверяет условие `!su`; логически это защита от NULL-указателя и работы с неинициализированными данными.
695	`out->unwind_unusual_section++;`	Технически увеличивает `out->unwind_unusual_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
696	`remember_location(&unusual_unwind_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
697	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
698	`bool allowed = sec_name_allowed_for_unwind(su->name);`	Технически присваивает `bool allowed` значение `sec_name_allowed_for_unwind(su->name)`. Логически это обновляет `bool allowed`: хранит промежуточное значение, которое дальше используется в текущей функции.
699	`bool plausible = sec_name_plausibly_unwind_named(su->name);`	Технически присваивает `bool plausible` значение `sec_name_plausibly_unwind_named(su->name)`. Логически это обновляет `bool plausible`: хранит промежуточное значение, которое дальше используется в текущей функции.
700	`if (!allowed && !plausible) {`	Технически проверяет условие `!allowed && !plausible`; логически это защита от NULL-указателя и работы с неинициализированными данными.
701	`out->unwind_unusual_section++;`	Технически увеличивает `out->unwind_unusual_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
702	`remember_location(&unusual_unwind_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
703	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
704	`if (su->characteristics & IMAGE_SCN_MEM_EXECUTE) {`	Технически проверяет условие `su->characteristics & IMAGE_SCN_MEM_EXECUTE`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
705	`out->unwind_exec_section++;`	Технически увеличивает `out->unwind_exec_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
706	`remember_location(&executable_unwind_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
707	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
708	`if (su->characteristics & IMAGE_SCN_MEM_WRITE) {`	Технически проверяет условие `su->characteristics & IMAGE_SCN_MEM_WRITE`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
709	`out->unwind_writable_section++;`	Технически увеличивает `out->unwind_writable_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
710	`remember_location(&writable_unwind_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.

Стр.	Код	Нормальное объяснение
711	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
712	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
714	`if (us[i].has_handler) {`	Технически проверяет условие `us[i].has_handler`; логически отделяет функции с language-specific handler от обычных функций.
715	`const PeSection* sh = pe_find_section_for_rva(img, us[i].handler_rva);`	Технически присваивает `const PeSection* sh` значение `pe_find_section_for_rva(img, us[i].handler_rva)`. Логически это обновляет `const PeSection* sh`: хранит промежуточное значение, которое дальше используется в текущей функции.
716	`if (!sh || !(sh->characteristics & IMAGE_SCN_MEM_EXECUTE)) {`	Технически проверяет условие `!sh || !(sh->characteristics & IMAGE_SCN_MEM_EXECUTE)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
717	`out->handler_nonexec_section++;`	Технически увеличивает `out->handler_nonexec_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
718	`remember_location(&nonexec_handler_loc, us[i].handler_rva);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
719	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
720	`if (sh && (sh->characteristics & IMAGE_SCN_MEM_WRITE)) {`	Технически проверяет условие `sh && (sh->characteristics & IMAGE_SCN_MEM_WRITE)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
721	`out->handler_writable_section++;`	Технически увеличивает `out->handler_writable_section` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
722	`remember_location(&writable_handler_loc, us[i].handler_rva);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
723	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
724	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
725	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
727	`if (!us_ok[i]) {`	Технически проверяет условие `!us_ok[i]`; логически это защита от NULL-указателя и работы с неинициализированными данными.
728	`out->unwind_parse_fail++;`	Технически увеличивает `out->unwind_parse_fail` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
729	`remember_location(&parse_fail_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
730	`} else if (us[i].anomalies & (uint32_t)~UA_ABI_MASK) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
731	`out->unwind_anomalies++;`	Технически увеличивает `out->unwind_anomalies` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
732	`remember_location(&anomaly_loc, u);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
733	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
734	`if (us_ok[i] && (us[i].anomalies & UA_ABI_MASK)) {`	Технически проверяет условие `us_ok[i] && (us[i].anomalies & UA_ABI_MASK)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
735	`out->unwind_abi_warnings++;`	Технически увеличивает `out->unwind_abi_warnings` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
736	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
737	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
738	`out->rf_unsorted = unsorted;`	Технически присваивает `out->rf_unsorted` значение `unsorted`. Логически это обновляет `rf_unsorted`: хранит промежуточное значение, которое дальше используется в текущей функции.
740	`/* Overlap / duplicates check on sorted array */`	Технически это комментарий исходника. Логически он объясняет разработчику: Overlap / duplicates check on sorted array.

Стр.	Код	Нормальное объяснение
741	`RfIdx* idx = NULL;`	Технически объявляет `idx` в функции `heuristics_analyze` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
742	`if (n > 0) {`	Технически проверяет условие `n > 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
743	`idx = (RfIdx*)malloc(n * sizeof(RfIdx));`	Технически присваивает `idx` значение `(RfIdx*)malloc(n * sizeof(RfIdx))`. Логически это обновляет `idx`: хранит промежуточное значение, которое дальше используется в текущей функции.
744	`if (!idx) {`	Технически проверяет условие `!idx`; логически это защита от NULL-указателя и работы с неинициализированными данными.
745	`sb_append(err, "Out of memory (sort array).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
746	`hmap_free(&unwind_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
747	`hmap_free(&handler_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
748	`sb_free(&out->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `heuristics_analyze`: освобождает память динамического буфера.
749	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
750	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
751	`for (size_t i = 0; i < n; i++) { idx[i].begin = rf[i].begin_rva; idx[i].end = rf[i].end_rva; }`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
752	`qsort(idx, n, sizeof(RfIdx), cmp_rfidx);`	Технически вызывает `qsort`. Логически это действие нужно в функции `heuristics_analyze`: сортирует массив, чтобы потом проверить порядок и пересечения.
754	`uint32_t max_end = 0;`	Технически объявляет `max_end` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
755	`uint32_t sorted_prev_begin = 0;`	Технически объявляет `sorted_prev_begin` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
756	`bool have_prev = false;`	Технически объявляет `have_prev` в функции `heuristics_analyze` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
757	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
758	`if (have_prev) {`	Технически проверяет условие `have_prev`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
759	`if (idx[i].begin == sorted_prev_begin) {`	Технически проверяет условие `idx[i].begin == sorted_prev_begin`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
760	`out->rf_duplicate_begins++;`	Технически увеличивает `out->rf_duplicate_begins` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
761	`remember_location(&duplicate_loc, idx[i].begin);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
762	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
763	`if (idx[i].begin < max_end) {`	Технически проверяет условие `idx[i].begin < max_end`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
764	`out->rf_overlaps++;`	Технически увеличивает `out->rf_overlaps` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
765	`remember_location(&overlap_loc, idx[i].begin);`	Технически вызывает `remember_location`. Логически это действие нужно в функции `heuristics_analyze`: сохраняет первый характерный RVA для finding.
766	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
767	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
768	`sorted_prev_begin = idx[i].begin;`	Технически присваивает `sorted_prev_begin` значение `idx[i].begin`. Логически это обновляет `sorted_prev_begin`: хранит промежуточное значение, которое дальше используется в текущей функции.
769	`if (!have_prev || idx[i].end > max_end) max_end = idx[i].end;`	Технически проверяет условие `!have_prev || idx[i].end > max_end`; логически это защита от NULL-указателя и работы с неинициализированными данными.
770	`have_prev = true;`	Технически присваивает `have_prev` значение `true`. Логически это обновляет `have_prev`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
771	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
772	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
773	`free(idx);`	Технически вызывает `free`. Логически это действие нужно в функции `heuristics_analyze`: освобождает память, чтобы в C не было утечки.
775	`uint32_t unwind_max = hmap_max_val(&unwind_counts);`	Технически присваивает `uint32_t unwind_max` значение `hmap_max_val(&unwind_counts)`. Логически это обновляет `uint32_t unwind_max`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
776	`uint32_t dominant_unwind_rva = 0;`	Технически объявляет `dominant_unwind_rva` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
777	`(void)hmap_max_entry(&unwind_counts, &dominant_unwind_rva, NULL);`	Технически вызывает `hmap_max_entry`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_max_entry` для выполнения части алгоритма.
778	`size_t unwind_unique = hmap_unique(&unwind_counts);`	Технически присваивает `size_t unwind_unique` значение `hmap_unique(&unwind_counts)`. Логически это обновляет `size_t unwind_unique`: хранит размер или количество, поэтому используется в проверках границ.
779	`out->unwind_reuse_ratio = safe_ratio((double)unwind_max, (double)n);`	Технически присваивает `out->unwind_reuse_ratio` значение `safe_ratio((double)unwind_max, (double)n)`. Логически это обновляет `unwind_reuse_ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
780	`out->unwind_unique_ratio = safe_ratio((double)unwind_unique, (double)n);`	Технически присваивает `out->unwind_unique_ratio` значение `safe_ratio((double)unwind_unique, (double)n)`. Логически это обновляет `unwind_unique_ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
782	`uint32_t handler_max = hmap_max_val(&handler_counts);`	Технически присваивает `uint32_t handler_max` значение `hmap_max_val(&handler_counts)`. Логически это обновляет `uint32_t handler_max`: хранит промежуточное значение, которое дальше используется в текущей функции.
783	`uint32_t dominant_handler_rva = 0;`	Технически объявляет `dominant_handler_rva` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
784	`(void)hmap_max_entry(&handler_counts, &dominant_handler_rva, NULL);`	Технически вызывает `hmap_max_entry`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_max_entry` для выполнения части алгоритма.
785	`out->handler_reuse_ratio = (out->handler_count > 0) ? safe_ratio((double)handler_max, (double)out->handler_count) : 0.0;`	Технически присваивает `out->handler_reuse_ratio` значение `(out->handler_count > 0) ? safe_ratio((double)handler_max, (double)out->handler_count) : 0.0`. Логически это обновляет `handler_reuse_ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
787	`/* Scoring */`	Технически это комментарий исходника. Логически он объясняет разработчику: Scoring.
788	`int score = 0;`	Технически объявляет `score` в функции `heuristics_analyze` и сразу задает начальное значение `0`. Логически это место для данных: суммарная оценка подозрительности 0..100.
789	`if (out->pe_section_raw_oob) {`	Технически проверяет условие `out->pe_section_raw_oob`; логически это защита от чтения за пределами файла на диске.
790	`add_score(&score, SCORE_PE_SECTION_RAW_OOB);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
791	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
792	`add_finding(out, "pe_section_raw_oob", "table_integrity", "high",`	Технически строка использует код finding `pe_section_raw_oob`. Логически этот finding означает: секция на диске выходит за размер файла; это ломает базовый контракт PE-загрузчика.
793	`"Section raw ranges exceed the input file: %zu", out->pe_section_raw_oob),`	Технически эта строка является частью C-конструкции `"Section raw ranges exceed the input file: %zu", out->pe_section_raw_oob),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
794	`first_pe_section_issue(img, PE_ISSUE_RAW_OOB), out->pe_section_raw_oob);`	Технически вызывает `first_pe_section_issue`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `first_pe_section_issue` для выполнения части алгоритма.
795	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
796	`if (out->pe_section_virtual_oob) {`	Технически проверяет условие `out->pe_section_virtual_oob`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
797	`add_score(&score, SCORE_PE_SECTION_VIRTUAL_OOB);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
798	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.

Стр.	Код	Нормальное объяснение
799	`add_finding(out, "pe_section_virtual_oob", "table_integrity", "high",`	Технически строка использует код finding `pe_section_virtual_oob`. Логически этот finding означает: секция в памяти выходит за SizeOfImage; это ошибка виртуальной координатной системы образа.
800	`"Section virtual ranges exceed SizeOfImage: %zu", out->pe_section_virtual_oob),`	Технически эта строка является частью C-конструкции `"Section virtual ranges exceed SizeOfImage: %zu", out->pe_section_virtual_oob),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
801	`first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OOB), out->pe_section_virtual_oob);`	Технически вызывает `first_pe_section_issue`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `first_pe_section_issue` для выполнения части алгоритма.
802	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
803	`if (out->pe_section_raw_overlaps) {`	Технически проверяет условие `out->pe_section_raw_overlaps`; логически это защита от чтения за пределами файла на диске.
804	`add_score(&score, SCORE_PE_SECTION_RAW_OVERLAP);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
805	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
806	`add_finding(out, "pe_section_raw_overlap", "table_integrity", "medium",`	Технически строка использует код finding `pe_section_raw_overlap`. Логически этот finding означает: raw-диапазоны секций пересекаются; один участок файла заявлен сразу несколькими секциями.
807	`"Overlapping section raw ranges: %zu", out->pe_section_raw_overlaps),`	Технически эта строка является частью C-конструкции `"Overlapping section raw ranges: %zu", out->pe_section_raw_overlaps),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
808	`first_pe_section_issue(img, PE_ISSUE_RAW_OVERLAP), out->pe_section_raw_overlaps);`	Технически вызывает `first_pe_section_issue`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `first_pe_section_issue` для выполнения части алгоритма.
809	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
810	`if (out->pe_section_virtual_overlaps) {`	Технически проверяет условие `out->pe_section_virtual_overlaps`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
811	`add_score(&score, SCORE_PE_SECTION_VIRTUAL_OVERLAP);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
812	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
813	`add_finding(out, "pe_section_virtual_overlap", "table_integrity", "high",`	Технически строка использует код finding `pe_section_virtual_overlap`. Логически этот finding означает: виртуальные диапазоны секций пересекаются; RVA попадает в несколько секций одновременно.
814	`"Overlapping section virtual ranges: %zu", out->pe_section_virtual_overlaps),`	Технически эта строка является частью C-конструкции `"Overlapping section virtual ranges: %zu", out->pe_section_virtual_overlaps),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
815	`first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OVERLAP), out->pe_section_virtual_overlaps);`	Технически вызывает `first_pe_section_issue`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `first_pe_section_issue` для выполнения части алгоритма.
816	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
817	`if (out->pe_section_misaligned) {`	Технически проверяет условие `out->pe_section_misaligned`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
818	`add_score(&score, SCORE_PE_SECTION_MISALIGNED);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
819	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
820	`add_finding(out, "pe_section_misaligned", "table_integrity", "medium",`	Технически строка использует код finding `pe_section_misaligned`. Логически этот finding означает: section table нарушает выравнивание или порядок, поэтому адресация становится ненадежной.
821	`"PE alignment/header invariants or section alignment rules are violated: %zu",`	Технически эта строка является частью C-конструкции `"PE alignment/header invariants or section alignment rules are violated: %zu",`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
822	`out->pe_section_misaligned),`	Технически эта строка является частью C-конструкции `out->pe_section_misaligned),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
823	`first_pe_section_issue(img, PE_ISSUE_MISALIGNED), out->pe_section_misaligned);`	Технически вызывает `first_pe_section_issue`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `first_pe_section_issue` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
824	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
825	`if (out->pe_exception_directory_oob) {`	Технически проверяет условие `out->pe_exception_directory_oob`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
826	`FindingLocation location = { true, img->exception_rva };`	Технически объявляет `location` в функции `heuristics_analyze` и сразу задает начальное значение `{ true, img->exception_rva }`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
827	`add_score(&score, SCORE_PE_EXCEPTION_DIRECTORY_OOB);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
828	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
829	`add_finding(out, "pe_exception_directory_oob", "table_integrity", "high",`	Технически строка использует код finding `pe_exception_directory_oob`. Логически этот finding означает: каталог исключений невозможно безопасно прочитать как таблицу RUNTIME_FUNCTION.
830	`"Exception directory is malformed, unreadable, incomplete, or out of bounds."),`	Технически эта строка является частью C-конструкции `"Exception directory is malformed, unreadable, incomplete, or out of bounds."),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
831	`location, 1);`	Технически выполняет оператор `location, 1);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
832	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
833	`if (n == 0 &&`	Технически проверяет условие `if (n == 0 &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
834	`img->exception_rva == 0 &&`	Технически выполняет битовую операцию `img->exception_rva == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
835	`img->exception_size == 0 &&`	Технически выполняет битовую операцию `img->exception_size == 0 &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
836	`image_has_executable_section(img)) {`	Технически объявляет/начинает функцию `image_has_executable_section`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
837	`add_score(&score, SCORE_EMPTY_PDATA);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
838	`add_finding(out, "no_runtime_functions", "table_integrity", "medium",`	Технически строка использует код finding `no_runtime_functions`. Логически этот finding означает: у исполняемого x64-образа нет .pdata; это бывает легально, но требует ручной проверки.
839	`"No RUNTIME_FUNCTION entries found; this can be valid for a leaf-only image, so confirm code complexity manually.");`	Технически выполняет оператор `"No RUNTIME_FUNCTION entries found; this can be valid for a leaf-only image, so confirm code complexity manually.");`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
840	`} else if (n == 0 && img->exception_rva == 0 && img->exception_size == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
841	`append_note(out, "No RUNTIME_FUNCTION entries and no executable sections; this is consistent with a resource-only or forwarding image.");`	Технически вызывает `append_note`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `append_note` для выполнения части алгоритма.
842	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
844	`double invalid_ranges_pct = safe_ratio((double)out->rf_invalid_ranges, (double)n);`	Технически присваивает `double invalid_ranges_pct` значение `safe_ratio((double)out->rf_invalid_ranges, (double)n)`. Логически это обновляет `double invalid_ranges_pct`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
845	`double oob_pct = safe_ratio((double)out->rf_oob, (double)n);`	Технически присваивает `double oob_pct` значение `safe_ratio((double)out->rf_oob, (double)n)`. Логически это обновляет `double oob_pct`: хранит промежуточное значение, которое дальше используется в текущей функции.
846	`double unwind_oob_pct = safe_ratio((double)out->rf_unwind_oob, (double)n);`	Технически присваивает `double unwind_oob_pct` значение `safe_ratio((double)out->rf_unwind_oob, (double)n)`. Логически это обновляет `double unwind_oob_pct`: хранит промежуточное значение, которое дальше используется в текущей функции.
847	`double unwind_fail_pct = safe_ratio((double)out->unwind_parse_fail, (double)n);`	Технически присваивает `double unwind_fail_pct` значение `safe_ratio((double)out->unwind_parse_fail, (double)n)`. Логически это обновляет `double unwind_fail_pct`: хранит промежуточное значение, которое дальше используется в текущей функции.
848	`double unwind_anom_pct = safe_ratio((double)out->unwind_anomalies, (double)n);`	Технически присваивает `double unwind_anom_pct` значение `safe_ratio((double)out->unwind_anomalies, (double)n)`. Логически это обновляет `double unwind_anom_pct`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
850	`if (out->rf_invalid_ranges) {`	Технически проверяет условие `out->rf_invalid_ranges`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
851	`add_score(&score, invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? SCORE_INVALID_RANGE_HIGH : SCORE_INVALID_RANGE_LOW);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
852	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
853	`add_finding(out, "invalid_ranges", "table_integrity", invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",`	Технически строка использует код finding `invalid_ranges`. Логически этот finding означает: у RUNTIME_FUNCTION EndAddress не больше BeginAddress, значит диапазон функции невозможен.
854	`"Invalid ranges (End<=Begin): %zu (%.2f%%)", out->rf_invalid_ranges, invalid_ranges_pct*100.0),`	Технически вызывает `ranges`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `ranges` для выполнения части алгоритма.
855	`invalid_range_loc, out->rf_invalid_ranges);`	Технически выполняет оператор `invalid_range_loc, out->rf_invalid_ranges);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
856	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
857	`if (out->rf_oob) {`	Технически проверяет условие `out->rf_oob`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
858	`add_score(&score, oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_RF_OOB_HIGH : SCORE_RF_OOB_LOW);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
859	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
860	`add_finding(out, "rf_oob", "table_integrity", oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",`	Технически строка использует код finding `rf_oob`. Логически этот finding означает: Begin/End функции выходят за SizeOfImage.
861	`"Begin/End out of SizeOfImage: %zu (%.2f%%)", out->rf_oob, oob_pct*100.0),`	Технически вызывает `zu`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `zu` для выполнения части алгоритма.
862	`rf_oob_loc, out->rf_oob);`	Технически выполняет оператор `rf_oob_loc, out->rf_oob);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
863	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
864	`if (out->rf_unwind_oob) {`	Технически проверяет условие `out->rf_unwind_oob`; логически отбирает записи, где важна корректность UNWIND_INFO.
865	`add_score(&score, unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_RVA_OOB_HIGH : SCORE_UNWIND_RVA_OOB_LOW);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
866	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
867	`add_finding(out, "unwind_rva_oob", "table_integrity", unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",`	Технически строка использует код finding `unwind_rva_oob`. Логически этот finding означает: UnwindInfoAddress отсутствует или выходит за SizeOfImage.
868	`"UnwindInfoRVA missing/out-of-bounds: %zu (%.2f%%)", out->rf_unwind_oob, unwind_oob_pct*100.0),`	Технически вызывает `zu`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `zu` для выполнения части алгоритма.
869	`unwind_oob_loc, out->rf_unwind_oob);`	Технически выполняет оператор `unwind_oob_loc, out->rf_unwind_oob);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
870	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
871	`if (out->rf_unsorted) {`	Технически проверяет условие `out->rf_unsorted`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
872	`add_score(&score, out->rf_unsorted > 0 ? SCORE_RF_UNSORTED : 0);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
873	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
874	`add_finding(out, "rf_unsorted", "table_integrity", "medium",`	Технически строка использует код finding `rf_unsorted`. Логически этот finding означает: таблица функций не отсортирована по BeginAddress, хотя Windows ожидает упорядоченный поиск.
875	`"Table not sorted by BeginAddress: %zu inversions", out->rf_unsorted),`	Технически эта строка является частью C-конструкции `"Table not sorted by BeginAddress: %zu inversions", out->rf_unsorted),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
876	`unsorted_loc, out->rf_unsorted);`	Технически выполняет оператор `unsorted_loc, out->rf_unsorted);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
877	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
878	`if (out->rf_overlaps) {`	Технически проверяет условие `out->rf_overlaps`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
879	`add_score(&score, out->rf_overlaps > 0 ? SCORE_RF_OVERLAP : 0);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
880	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
881	`add_finding(out, "rf_overlaps", "table_integrity", "high",`	Технически строка использует код finding `rf_overlaps`. Логически этот finding означает: диапазоны функций пересекаются после сортировки.
882	`"Overlapping ranges (after sorting): %zu", out->rf_overlaps),`	Технически вызывает `ranges`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `ranges` для выполнения части алгоритма.
883	`overlap_loc, out->rf_overlaps);`	Технически выполняет оператор `overlap_loc, out->rf_overlaps);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
884	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
885	`if (out->rf_duplicate_begins) {`	Технически проверяет условие `out->rf_duplicate_begins`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
886	`add_score(&score, SCORE_DUPLICATE_BEGIN);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
887	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
888	`add_finding(out, "duplicate_begin", "table_integrity", "medium",`	Технически строка использует код finding `duplicate_begin`. Логически этот finding означает: несколько функций начинаются с одного RVA.
889	`"Duplicate BeginAddress (after sorting): %zu", out->rf_duplicate_begins),`	Технически вызывает `BeginAddress`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `BeginAddress` для выполнения части алгоритма.
890	`duplicate_loc, out->rf_duplicate_begins);`	Технически выполняет оператор `duplicate_loc, out->rf_duplicate_begins);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
891	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
893	`if (out->unwind_parse_fail) {`	Технически проверяет условие `out->unwind_parse_fail`; логически отбирает записи, где важна корректность UNWIND_INFO.
894	`add_score(&score, unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_PARSE_FAIL_HIGH : SCORE_UNWIND_PARSE_FAIL_LOW);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
895	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
896	`add_finding(out, "unwind_parse_fail", "unwind_integrity", unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",`	Технически строка использует код finding `unwind_parse_fail`. Логически этот finding означает: UNWIND_INFO не удалось разобрать безопасно.
897	`"UnwindInfo parse failures: %zu (%.2f%%)", out->unwind_parse_fail, unwind_fail_pct*100.0),`	Технически вызывает `zu`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `zu` для выполнения части алгоритма.
898	`parse_fail_loc, out->unwind_parse_fail);`	Технически выполняет оператор `parse_fail_loc, out->unwind_parse_fail);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
899	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
900	`if (out->unwind_anomalies) {`	Технически проверяет условие `out->unwind_anomalies`; логически отбирает записи, где важна корректность UNWIND_INFO.
901	`add_score(&score, unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? SCORE_UNWIND_ANOM_HIGH : SCORE_UNWIND_ANOM_LOW);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
902	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
903	`add_finding(out, "unwind_anomalies", "unwind_integrity", unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? "high" : "medium",`	Технически строка использует код finding `unwind_anomalies`. Логически этот finding означает: UNWIND_INFO разобран, но содержит странные opcode, version, alignment или chain-связи.
904	`"UnwindInfo anomalies (version/opcode/chain/etc): %zu (%.2f%%)", out->unwind_anomalies, unwind_anom_pct*100.0),`	Технически вызывает `anomalies`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `anomalies` для выполнения части алгоритма.
905	`anomaly_loc, out->unwind_anomalies);`	Технически выполняет оператор `anomaly_loc, out->unwind_anomalies);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
906	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
908	`bool moderate_reuse_candidate =`	Технически присваивает `bool moderate_reuse_candidate` значение `. Логически это обновляет bool moderate_reuse_candidate`: хранит промежуточное значение, которое дальше используется в текущей функции.
909	`out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && n > 200;`	Технически выполняет битовую операцию `out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && n > 200;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
910	`if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_HIGH && n > 200) {`	Технически проверяет условие `out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_HIGH && n > 200`; логически отбирает записи, где важна корректность UNWIND_INFO.
911	`add_score(&score, SCORE_UNWIND_REUSE_HIGH);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
912	`FindingLocation location = { true, dominant_unwind_rva };`	Технически объявляет `location` в функции `heuristics_analyze` и сразу задает начальное значение `{ true, dominant_unwind_rva }`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
913	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.

Стр.	Код	Нормальное объяснение
914	`add_finding(out, "unwind_reuse_strong", "unwind_reuse", "high",`	Технически строка использует код finding `unwind_reuse_strong`. Логически этот finding означает: слишком много функций ссылаются на один UNWIND_INFO.
915	`"Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio=%.2f)",`	Технически присваивает `"Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio значение %.2f)",`. Логически это обновляет `"Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
916	`(unsigned)unwind_max, out->unwind_reuse_ratio),`	Технически эта строка является частью C-конструкции `(unsigned)unwind_max, out->unwind_reuse_ratio),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
917	`location, unwind_max);`	Технически выполняет оператор `location, unwind_max);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
918	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
920	`if (out->handler_count > 50 && out->handler_reuse_ratio > THRESHOLD_HANDLER_DOMINANCE) {`	Технически проверяет условие `out->handler_count > 50 && out->handler_reuse_ratio > THRESHOLD_HANDLER_DOMINANCE`; логически отделяет функции с language-specific handler от обычных функций.
921	`bool has_static_context =`	Технически присваивает `bool has_static_context` значение `` `. Логически это обновляет bool has_static_context` ``: хранит промежуточное значение, которое дальше используется в текущей функции.
922	`out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 || out->rf_unsorted > 0 ||`	Технически выполняет битовую операцию `out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 || out->rf_unsorted > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
923	`out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 || out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||`	Технически выполняет битовую операцию `out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 || out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
924	`out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 || out->unwind_writable_section > 0 ||`	Технически выполняет битовую операцию `out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 || out->unwind_writable_section > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
925	`out->handler_nonexec_section > 0 || out->handler_writable_section > 0 ||`	Технически выполняет битовую операцию `out->handler_nonexec_section > 0 || out->handler_writable_section > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
926	`moderate_reuse_candidate;`	Технически выполняет оператор `moderate_reuse_candidate;`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
927	`const PeSection* dominant_sec = dominant_handler_rva ? pe_find_section_for_rva(img, dominant_handler_rva) : NULL;`	Технически присваивает `const PeSection* dominant_sec` значение `dominant_handler_rva ? pe_find_section_for_rva(img, dominant_handler_rva) : NULL`. Логически это обновляет `const PeSection* dominant_sec`: хранит промежуточное значение, которое дальше используется в текущей функции.
928	`bool dominant_runtime_like = dominant_sec &&`	Технически присваивает `bool dominant_runtime_like` значение `dominant_sec &&`. Логически это обновляет `bool dominant_runtime_like`: хранит промежуточное значение, которое дальше используется в текущей функции.
929	`(dominant_sec->characteristics & IMAGE_SCN_MEM_EXECUTE) &&`	Технически выполняет битовую операцию `(dominant_sec->characteristics & IMAGE_SCN_MEM_EXECUTE) &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
930	`sec_name_code_like(dominant_sec->name) &&`	Технически вызывает `sec_name_code_like`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `sec_name_code_like` для выполнения части алгоритма.
931	`out->handler_nonexec_section == 0;`	Технически выполняет оператор `out->handler_nonexec_section == 0;`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
933	`if (!(dominant_runtime_like && !has_static_context)) {`	Технически проверяет условие `!(dominant_runtime_like && !has_static_context)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
934	`add_score(&score, SCORE_HANDLER_DOMINANCE);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
935	`FindingLocation location = { true, dominant_handler_rva };`	Технически объявляет `location` в функции `heuristics_analyze` и сразу задает начальное значение `{ true, dominant_handler_rva }`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
936	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.

Стр.	Код	Нормальное объяснение
937	`add_finding(out, "handler_dominance", "handler_dominance", "medium",`	Технически строка использует код finding `handler_dominance`. Логически этот finding означает: слишком много функций используют один handler.
938	`"One handler dominates: max %u entries share same HandlerRVA (ratio=%.2f%s%s)",`	Технически присваивает `"One handler dominates: max %u entries share same HandlerRVA (ratio значение %.2f%s%s)",`. Логически это обновляет `"One handler dominates: max %u entries share same HandlerRVA (ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
939	`(unsigned)handler_max, out->handler_reuse_ratio,`	Технически эта строка является частью C-конструкции `(unsigned)handler_max, out->handler_reuse_ratio,`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
940	`dominant_sec ? ", section=" : "",`	Технически присваивает `dominant_sec ? ", section` значение `" : "",`. Логически это обновляет `dominant_sec ? ", section`: хранит промежуточное значение, которое дальше используется в текущей функции.
941	`dominant_sec ? dominant_sec->name : ""),`	Технически эта строка является частью C-конструкции `dominant_sec ? dominant_sec->name : ""),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
942	`location, handler_max);`	Технически выполняет оператор `location, handler_max);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
943	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
944	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
946	`if (moderate_reuse_candidate &&`	Технически проверяет условие `if (moderate_reuse_candidate &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
947	`!has_finding_code(out, "unwind_reuse_strong") &&`	Технически строка использует код finding `unwind_reuse_strong`. Логически этот finding означает: слишком много функций ссылаются на один UNWIND_INFO.
948	`(has_finding_code(out, "handler_dominance") ||`	Технически строка использует код finding `handler_dominance`. Логически этот finding означает: слишком много функций используют один handler.
949	`out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 ||`	Технически выполняет битовую операцию `out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
950	`out->rf_unsorted > 0 || out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 ||`	Технически выполняет битовую операцию `out->rf_unsorted > 0 || out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
951	`out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||`	Технически выполняет битовую операцию `out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
952	`out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 ||`	Технически выполняет битовую операцию `out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
953	`out->unwind_writable_section > 0 || out->handler_nonexec_section > 0 ||`	Технически выполняет битовую операцию `out->unwind_writable_section > 0 || out->handler_nonexec_section > 0 ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
954	`out->handler_writable_section > 0)) {`	Технически эта строка является частью C-конструкции `out->handler_writable_section > 0)) {`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
955	`add_score(&score, SCORE_UNWIND_REUSE_MEDIUM);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
956	`FindingLocation location = { true, dominant_unwind_rva };`	Технически объявляет `location` в функции `heuristics_analyze` и сразу задает начальное значение `{ true, dominant_unwind_rva }`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
957	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
958	`add_finding(out, "unwind_reuse_moderate", "unwind_reuse", "medium",`	Технически строка использует код finding `unwind_reuse_moderate`. Логически этот finding означает: умеренное повторное использование UNWIND_INFO требует проверки контекста.
959	`"Moderate UnwindInfo reuse is corroborated by another static signal: max %u entries share same UnwindInfoRVA (ratio=%.2f)",`	Технически присваивает `"Moderate UnwindInfo reuse is corroborated by another static signal: max %u entries sha...` значение `%.2f)",`. Логически это обновляет ``: хранит промежуточное значение, которое дальше используется в текущей функции.
960	`(unsigned)unwind_max, out->unwind_reuse_ratio),`	Технически эта строка является частью C-конструкции `(unsigned)unwind_max, out->unwind_reuse_ratio),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
961	`location, unwind_max);`	Технически выполняет оператор `location, unwind_max);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
962	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
964	`/* Placement heuristics */`	Технически это комментарий исходника. Логически он объясняет разработчику: Placement heuristics.
965	`if (out->unwind_exec_section) {`	Технически проверяет условие `out->unwind_exec_section`; логически отбирает записи, где важна корректность UNWIND_INFO.
966	`add_score(&score, SCORE_UNWIND_EXEC_SECTION);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
967	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
968	`add_finding(out, "unwind_in_exec_section", " metadata_placement", "high",`	Технически строка использует код finding `unwind_in_exec_section`. Логически этот finding означает: UNWIND_INFO лежит в исполняемой секции, что нетипично для xdata.
969	`"UnwindInfo located in executable section(s): %zu", out->unwind_exec_section),`	Технически вызывает `section`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `section` для выполнения части алгоритма.
970	`executable_unwind_loc, out->unwind_exec_section);`	Технически выполняет оператор `executable_unwind_loc, out->unwind_exec_section);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
971	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
972	`if (out->unwind_unusual_section) {`	Технически проверяет условие `out->unwind_unusual_section`; логически отбирает записи, где важна корректность UNWIND_INFO.
973	`int add = (out->unwind_unusual_section > (n / 20)) ? SCORE_UNWIND_UNUSUAL_SECTION_HIGH : SCORE_UNWIND_UNUSUAL_SECTION_LOW; /* >5% => broader placement pattern */`	Технически присваивает `int add` значение `(out->unwind_unusual_section > (n / 20)) ? SCORE_UNWIND_UNUSUAL_SECTION_HIGH : SCORE_UNWIND_UNUSUAL_SECTION...`. Логически это обновляет `int add`: хранит промежуточное значение, которое дальше используется в текущей функции.
974	`add_score(&score, add);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
975	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
976	`add_finding(out, "unwind_unusual_section", " metadata_placement", add >= 15 ? "high" : "medium",`	Технически строка использует код finding `unwind_unusual_section`. Логически этот finding означает: UNWIND_INFO лежит в секции с нетипичным именем/назначением.
977	`"UnwindInfo located in unusual sections (not .xdata/.rdata/.pdata): %zu", out->unwind_unusual_section),`	Технически вызывает `sections`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `sections` для выполнения части алгоритма.
978	`unusual_unwind_loc, out->unwind_unusual_section);`	Технически выполняет оператор `unusual_unwind_loc, out->unwind_unusual_section);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
979	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
980	`if (out->unwind_writable_section) {`	Технически проверяет условие `out->unwind_writable_section`; логически отбирает записи, где важна корректность UNWIND_INFO.
981	`int add = (out->unwind_writable_section > (n / 20)) ? SCORE_UNWIND_WRITABLE_SECTION_HIGH : SCORE_UNWIND_WRITABLE_SECTION_LOW;`	Технически присваивает `int add` значение `(out->unwind_writable_section > (n / 20)) ? SCORE_UNWIND_WRITABLE_SECTION_HIGH : SCORE_UNWIND_WRITABLE_SECT...`. Логически это обновляет `int add`: хранит промежуточное значение, которое дальше используется в текущей функции.
982	`add_score(&score, add);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
983	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.

Стр.	Код	Нормальное объяснение
984	`add_finding(out, "unwind_writable_section", " metadata_placement", add >= 18 ? "high" : "medium",`	Технически строка использует код finding `unwind_writable_section`. Логически этот finding означает: UNWIND_INFO лежит в writable-секции и может быть изменен во время выполнения.
985	`"UnwindInfo located in writable section(s): %zu", out->unwind_writable_section),`	Технически вызывает `section`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `section` для выполнения части алгоритма.
986	`writable_unwind_loc, out->unwind_writable_section);`	Технически выполняет оператор `writable_unwind_loc, out->unwind_writable_section);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
987	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
988	`if (out->handler_nonexec_section) {`	Технически проверяет условие `out->handler_nonexec_section`; логически отделяет функции с language-specific handler от обычных функций.
989	`add_score(&score, SCORE_HANDLER_NONEXEC);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
990	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
991	`add_finding(out, "handler_nonexec_section", " metadata_placement", "high",`	Технически строка использует код finding `handler_nonexec_section`. Логически этот finding означает: handler указывает в неисполняемую область.

Стр.	Код	Нормальное объяснение
992	`"Exception handler RVA in non-executable/unknown section: %zu", out->handler_nonexec_section),`	Технически эта строка является частью C-конструкции `"Exception handler RVA in non-executable/unknown section: %zu", out->handler_nonexec_section),`. Логически она уточняет структуру блока функции `heuristics_analyze` и влияет на то, какие данные будут обработаны.
993	`nonexec_handler_loc, out->handler_nonexec_section);`	Технически выполняет оператор `nonexec_handler_loc, out->handler_nonexec_section);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
994	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
995	`if (out->handler_writable_section) {`	Технически проверяет условие `out->handler_writable_section`; логически отделяет функции с language-specific handler от обычных функций.
996	`add_score(&score, SCORE_HANDLER_WRITABLE);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
997	`annotate_finding(`	Технически вызывает `annotate_finding`. Логически это действие нужно в функции `heuristics_analyze`: добавляет к finding representative RVA и число затронутых объектов.
998	`add_finding(out, "handler_writable_section", " metadata_placement", "medium",`	Технически строка использует код finding `handler_writable_section`. Логически этот finding означает: handler лежит в writable-секции.
999	`"Exception handler RVA in writable section(s): %zu", out->handler_writable_section),`	Технически вызывает `section`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `section` для выполнения части алгоритма.
1000	`writable_handler_loc, out->handler_writable_section);`	Технически выполняет оператор `writable_handler_loc, out->handler_writable_section);`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
1001	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1003	`if (out->unwind_exec_section > 0 && out->handler_nonexec_section > 0) {`	Технически проверяет условие `out->unwind_exec_section > 0 && out->handler_nonexec_section > 0`; логически отбирает записи, где важна корректность UNWIND_INFO.
1004	`add_score(&score, SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
1005	`add_finding(out, "placement_exec_nonexec_handler_combo", " metadata_placement", "high",`	Технически строка использует код finding `placement_exec_nonexec_handler_combo`. Логически этот finding означает: сочетание исполняемой unwind-секции и неисполняемого handler усиливает подозрение.
1006	`"Unwind metadata in executable section(s) coincides with handler targets in non-executable/unknown section(s); the combined placement pattern is much harder to explain benignly.");`	Технически вызывает `section`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `section` для выполнения части алгоритма.
1007	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1008	`if ((out->unwind_exec_section > 0 || out->handler_nonexec_section > 0) &&`	Технически проверяет условие `(out->unwind_exec_section > 0 || out->handler_nonexec_section > 0`; логически отбирает записи, где важна корректность UNWIND_INFO.
1009	`(out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 || out->rf_unwind_oob > 0)) {`	Технически выполняет битовую операцию `(out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 || out->rf_unwind_oob > 0)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1010	`add_score(&score, SCORE_COMBO_PLACEMENT_AND_INTEGRITY);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
1011	`add_finding(out, "placement_integrity_combo", " unwind_integrity", "medium",`	Технически строка использует код finding `placement_integrity_combo`. Логически этот finding означает: проблемы размещения совпали с проблемами целостности unwind/table.
1012	`"Suspicious metadata placement is corroborated by UNWIND_INFO / UnwindInfoRVA integrity problems; treat the combination as stronger than either signal alone.");`	Технически выполняет оператор `"Suspicious metadata placement is corroborated by UNWIND_INFO / UnwindInfoRVA integrity problems; treat the combination as stro...`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
1013	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1014	`if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && has_finding_code(out, "handler_dominance")) {`	Технически проверяет условие `out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && has_finding_code(out, "handler_dominance");` логически отбирает записи, где важна корректность UNWIND_INFO.
1015	`add_score(&score, SCORE_COMBO_REUSE_AND_DOMINANCE);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
1016	`add_finding(out, "reuse_dominance_combo", "unwind_reuse", " medium",`	Технически строка использует код finding `reuse_dominance_combo`. Логически этот finding означает: повторное использование unwind совпало с доминированием handler.
1017	`"Heavy UnwindInfo reuse coincides with a dominant handler target, which is consistent with centralized dispatcher-style metadata reuse.");`	Технически выполняет оператор `"Heavy UnwindInfo reuse coincides with a dominant handler target, which is consistent with centralized dispatcher-style metadat...`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
1018	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1019	`if (out->unwind_writable_section > 0 && out->handler_writable_section > 0) {`	Технически проверяет условие `out->unwind_writable_section > 0 && out->handler_writable_section > 0`; логически отбирает записи, где важна корректность UNWIND_INFO.
1020	`add_score(&score, SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER);`	Технически вызывает `add_score`. Логически это действие нужно в функции `heuristics_analyze`: увеличивает score с ограничением 0..100.
1021	`add_finding(out, "writable_metadata_handler_combo", "metadata_placement", "high",`	Технически строка использует код finding `writable_metadata_handler_combo`. Логически этот finding означает: и unwind metadata, и handler связаны с writable-областями.
1022	`"Writable unwind metadata coincides with writable handler targets; this combination is uncommon for normal compiler-produced images and deserves close review.");`	Технически выполняет оператор `"Writable unwind metadata coincides with writable handler targets; this combination is uncommon for normal compiler-produced im...`. Логически в функции `heuristics_analyze` он меняет состояние, проверяемые данные или подготовку отчета.
1023	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1025	`if (score == 0) {`	Технически проверяет условие `score == 0`; логически выбирает уровень оценки или уверенности.
1026	`append_note(out, "No strong anomalies detected by heuristics.");`	Технически вызывает `append_note`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `append_note` для выполнения части алгоритма.
1027	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1028	`out->score = score;`	Технически присваивает `out->score` значение `score`. Логически это обновляет `score`: суммарная оценка подозрительности 0..100.
1029	`finalize_verdict(out);`	Технически вызывает `finalize_verdict`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `finalize_verdict` для выполнения части алгоритма.
1031	`hmap_free(&unwind_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
1032	`hmap_free(&handler_counts);`	Технически вызывает `hmap_free`. Логически это действие нужно в функции `heuristics_analyze`: вызывает функцию `hmap_free` для выполнения части алгоритма.
1033	`if (!sb_ok(&out->details)) {`	Технически проверяет условие `!sb_ok(&out->details)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1034	`sb_append(err, "Out of memory while building heuristic details.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `heuristics_analyze`: добавляет текст в динамический буфер.
1035	`return false;`	Технически возвращает `false` из функции `heuristics_analyze`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
1036	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1037	`return true;`	Технически возвращает `true` из функции `heuristics_analyze`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
1038	`}`	Технически завершает область видимости внутри `heuristics_analyze`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1040	`void heuristics_free(HeuristicReport* r) {`	Технически объявляет/начинает функцию `heuristics_free`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает ресурсы эвристического отчета.
1041	`if (!r) return;`	Технически проверяет условие `!r`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1042	`sb_free(&r->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `heuristics_free`: освобождает память динамического буфера.
1043	`memset(r, 0, sizeof(*r));`	Технически вызывает `memset`. Логически это действие нужно в функции `heuristics_free`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
1044	`}`	Технически завершает область видимости внутри `heuristics_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 948. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

7. Динамический слой Windows

Этот модуль проверяет не только файл на диске, но и runtime-поведение: какие API вызываются, где останавливается трасса, есть ли динамические признаки обработки исключений.

Что сначала выучить в C и теории

- условная компиляция под Windows и заглушки для не-Windows
- WinAPI debugging loop
- breakpoints, VEH/VCH/UEF, RtlAddFunctionTable
- почему на MacBook этот слой изучается и собирается через MinGW/Windows VM

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/dynamic.h

Назначение файла: модель динамического отчета и настройки runtime-валидации.

Код, который пишем на этом шаге

```
#pragma once
/*
  dynamic.h - Windows-only dynamic evidence collector (Debug API)

  Defensive intent:
    Collect runtime evidence of exception-handling redirection (e.g., to an
    interceptor or VM dispatcher) by observing:
      - hits of handlers referenced from UNWIND_INFO (x64)
      - registration/hits of vectored/unhandled exception handlers
      - registration of dynamic function tables
      - first-chance vs second-chance exception flow

  This module does NOT attempt to "break" protections or recover hidden data.
*/

#include <stdint.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"
#include "unwind.h"

#ifdef __cplusplus
extern "C" {
#endif

enum { DYNAMIC_MAX_EVENTS = 256 };

typedef struct DynamicEvent {
    char type[40];
    char module[64];
    uint32_t thread_id;
    uint32_t exception_code;
    bool first_chance;
    uint64_t address;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    uint64_t arg3;
} DynamicEvent;

typedef struct DynamicReport {
    bool enabled;
    bool ran;
    bool ok;
    bool timed_out;
    bool target_exited;
    uint32_t target_exit_code;

    uint32_t timeout_ms;
    uint32_t trace_steps;

    bool instrumentation_complete;
    uint32_t modules_observed_peak;
    uint32_t module_refresh_failures;
    uint32_t module_list_truncations;
    uint32_t unwind_handlers_discovered;
    uint32_t breakpoint_set_attempts;
    uint32_t breakpoint_set_failures;
    uint32_t breakpoint_capacity_drops;
    uint32_t pending_rearm_failures;
    uint32_t breakpoint_rearm_failures;

    uint32_t exceptions_seen;
    uint32_t first_chance_seen;
    uint32_t second_chance_seen;

    uint32_t unwind_handler_bp_set;
    uint32_t unwind_handler_hits;

    uint32_t veh_registrations;
    uint32_t veh_hits;
    uint32_t vch_registrations;
    uint32_t vch_hits;
    uint32_t uef_set_calls;
```

```
    uint32_t uef_hits;

    uint32_t rtl_add_function_table_calls;
    uint32_t rtl_install_function_table_cb_calls;

    uint32_t event_count;
    uint32_t dropped_events;
    DynamicEvent events[DYNAMIC_MAX_EVENTS];

    /* Execution trace sampled after first handler hit (RIP addresses). */
    uint32_t trace_count;
    uint64_t trace[256];

    /* Human readable details */
    StrBuf details;
} DynamicReport;

void dynamic_report_init(DynamicReport* dr);
void dynamic_report_free(DynamicReport* dr);

/*
  Run target under a debugger and collect evidence.

  - pe_path: path to target executable
  - child_argv/child_argc: optional args (may be NULL/0)
  - img/rf/us/us_ok: results of static parsing (used to set handler breakpoints)
  - rf_count: number of runtime functions
  - timeout_ms: hard timeout for debug loop
  - trace_steps: number of single-steps to record after handler hit (<=256 recommended)

  Returns true if the dynamic session ran (even if it found no evidence).
*/
bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
                             const UnwindSummary* us,
                             const bool* us_ok,
                             size_t rf_count,
                             uint32_t timeout_ms,
                             uint32_t trace_steps,
                             DynamicReport* out,
                             StrBuf* err);

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`dynamic.h - Windows-only dynamic evidence collector (Debug API)`	Технически объявляет/начинает функцию `collector`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
5	`Defensive intent:`	Технически эта строка является частью C-конструкции `Defensive intent:`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
6	`Collect runtime evidence of exception-handling redirection (e.g., to an`	Технически вызывает `redirection`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `redirection` для выполнения части алгоритма.
7	`interceptor or VM dispatcher) by observing:`	Технически эта строка является частью C-конструкции `interceptor or VM dispatcher) by observing:`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
8	`- hits of handlers referenced from UNWIND_INFO (x64)`	Технически объявляет/начинает функцию `UNWIND_INFO`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
9	`- registration/hits of vectored/unhandled exception handlers`	Технически эта строка является частью C-конструкции `- registration/hits of vectored/unhandled exception handlers`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
10	`- registration of dynamic function tables`	Технически эта строка является частью C-конструкции `- registration of dynamic function tables`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
11	`- first-chance vs second-chance exception flow`	Технически эта строка является частью C-конструкции `- first-chance vs second-chance exception flow`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
13	`This module does NOT attempt to "break" protections or recover hidden data.`	Технически эта строка является частью C-конструкции `This module does NOT attempt to "break" protections or recover hidden data.`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
14	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
16	`#include <stdint.h>`	Технически подключает `stdint.h`. Логически это дает модулю доступ к: точные типы uint8_t/uint32_t/uint64_t, без которых нельзя надежно читать бинарный формат.
17	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
18	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
19	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.

Стр.	Код	Нормальное объяснение
20	`#include "unwind.h"`	Технически подключает `unwind.h`. Логически это дает модулю доступ к: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.
22	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
23	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
24	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
26	`enum { DYNAMIC_MAX_EVENTS = 256 };`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
28	`typedef struct DynamicEvent {`	Технически начинает описание структуры `DynamicEvent`. Логически структура собирает связанные поля в одну модель данных проекта.
29	`char type[40];`	Технически объявляет поле `type` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
30	`char module[64];`	Технически объявляет поле `module` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
31	`uint32_t thread_id;`	Технически объявляет поле `thread_id` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
32	`uint32_t exception_code;`	Технически объявляет поле `exception_code` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
33	`bool first_chance;`	Технически объявляет поле `first_chance` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
34	`uint64_t address;`	Технически объявляет поле `address` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
35	`uint64_t arg0;`	Технически объявляет поле `arg0` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
36	`uint64_t arg1;`	Технически объявляет поле `arg1` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
37	`uint64_t arg2;`	Технически объявляет поле `arg2` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
38	`uint64_t arg3;`	Технически объявляет поле `arg3` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
39	`} DynamicEvent;`	Технически завершает typedef и дает типу имя `DynamicEvent`. Логически дальше код может объявлять переменные этого типа без `struct`.

Стр.	Код	Нормальное объяснение
41	`typedef struct DynamicReport {`	Технически начинает описание структуры `DynamicReport`. Логически структура собирает связанные поля в одну модель данных проекта.
42	`bool enabled;`	Технически объявляет поле `enabled` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
43	`bool ran;`	Технически объявляет поле `ran` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
44	`bool ok;`	Технически объявляет поле `ok` внутри структуры. Логически это поле: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
45	`bool timed_out;`	Технически объявляет поле `timed_out` внутри структуры. Логически это поле: является выходным параметром, куда функция записывает результат.
46	`bool target_exited;`	Технически объявляет поле `target_exited` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
47	`uint32_t target_exit_code;`	Технически объявляет поле `target_exit_code` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
49	`uint32_t timeout_ms;`	Технически объявляет поле `timeout_ms` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
50	`uint32_t trace_steps;`	Технически объявляет поле `trace_steps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
52	`bool instrumentation_complete;`	Технически объявляет поле `instrumentation_complete` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
53	`uint32_t modules_observed_peak;`	Технически объявляет поле `modules_observed_peak` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
54	`uint32_t module_refresh_failures;`	Технически объявляет поле `module_refresh_failures` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
55	`uint32_t module_list_truncations;`	Технически объявляет поле `module_list_truncations` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
56	`uint32_t unwind_handlers_discovered;`	Технически объявляет поле `unwind_handlers_discovered` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
57	`uint32_t breakpoint_set_attempts;`	Технически объявляет поле `breakpoint_set_attempts` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
58	`uint32_t breakpoint_set_failures;`	Технически объявляет поле `breakpoint_set_failures` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
59	`uint32_t breakpoint_capacity_drops;`	Технически объявляет поле `breakpoint_capacity_drops` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
60	`uint32_t pending_rearm_failures;`	Технически объявляет поле `pending_rearm_failures` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
61	`uint32_t breakpoint_rearm_failures;`	Технически объявляет `breakpoint_rearm_failures` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
63	`uint32_t exceptions_seen;`	Технически объявляет `exceptions_seen` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
64	`uint32_t first_chance_seen;`	Технически объявляет `first_chance_seen` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
65	`uint32_t second_chance_seen;`	Технически объявляет `second_chance_seen` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
67	`uint32_t unwind_handler_bp_set;`	Технически объявляет `unwind_handler_bp_set` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
68	`uint32_t unwind_handler_hits;`	Технически объявляет `unwind_handler_hits` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
70	`uint32_t veh_registrations;`	Технически объявляет `veh_registrations` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
71	`uint32_t veh_hits;`	Технически объявляет `veh_hits` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
72	`uint32_t vch_registrations;`	Технически объявляет `vch_registrations` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
73	`uint32_t vch_hits;`	Технически объявляет `vch_hits` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
74	`uint32_t uef_set_calls;`	Технически объявляет `uef_set_calls` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
75	`uint32_t uef_hits;`	Технически объявляет `uef_hits` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
77	`uint32_t rtl_add_function_table_calls;`	Технически объявляет `rtl_add_function_table_calls` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
78	`uint32_t rtl_install_function_table_cb_calls;`	Технически объявляет `rtl_install_function_table_cb_calls` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
80	`uint32_t event_count;`	Технически объявляет `event_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
81	`uint32_t dropped_events;`	Технически объявляет `dropped_events` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
82	`DynamicEvent events[DYNAMIC_MAX_EVENTS];`	Технически выполняет оператор `DynamicEvent events[DYNAMIC_MAX_EVENTS];`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
84	`/* Execution trace sampled after first handler hit (RIP addresses). */`	Технически это комментарий исходника. Логически он объясняет разработчику: Execution trace sampled after first handler hit (RIP addresses)..
85	`uint32_t trace_count;`	Технически объявляет `trace_count` в функции `глобальной области файла`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
86	`uint64_t trace[256];`	Технически объявляет `trace` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
88	`/* Human readable details */`	Технически это комментарий исходника. Логически он объясняет разработчику: Human readable details.
89	`StrBuf details;`	Технически объявляет `details` в функции `глобальной области файла`. Логически это место для данных: накопленный текстовый лог деталей анализа.

Стр.	Код	Нормальное объяснение
90	`} DynamicReport;`	Технически завершает typedef и дает типу имя `DynamicReport`. Логически дальше код может объявлять переменные этого типа без `struct`.
92	`void dynamic_report_init(DynamicReport* dr);`	Технически объявляет прототип функции `dynamic_report_init` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и инициализирует структуру динамического отчета.
93	`void dynamic_report_free(DynamicReport* dr);`	Технически объявляет прототип функции `dynamic_report_free` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и освобождает память внутри динамического отчета.
95	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
96	`Run target under a debugger and collect evidence.`	Технически эта строка является частью C-конструкции `Run target under a debugger and collect evidence.`. Логически она уточняет структуру блока функции `глобальной области` файла и влияет на то, какие данные будут обработаны.
98	`- pe_path: path to target executable`	Технически эта строка является частью C-конструкции `- pe_path: path to target executable`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
99	`- child_argv/child_argc: optional args (may be NULL/0)`	Технически объявляет/начинает функцию `args`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
100	`- img/rf/us/us_ok: results of static parsing (used to set handler breakpoints)`	Технически объявляет/начинает функцию `parsing`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
101	`- rf_count: number of runtime functions`	Технически эта строка является частью C-конструкции `- rf_count: number of runtime functions`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
102	`- timeout_ms: hard timeout for debug loop`	Технически эта строка является частью C-конструкции `- timeout_ms: hard timeout for debug loop`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
103	`- trace_steps: number of single-steps to record after handler hit (<=256 recommended)`	Технически объявляет/начинает функцию `hit`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
105	`Returns true if the dynamic session ran (even if it found no evidence).`	Технически вызывает `ran`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `ran` для выполнения части алгоритма.
106	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
107	`bool dynamic_collect_windows(const char* pe_path,`	Технически вызывает `dynamic_collect_windows`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_collect_windows` для выполнения части алгоритма.
108	`const char* const* child_argv,`	Технически эта строка является частью C-конструкции `const char* const* child_argv,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
109	`int child_argc,`	Технически эта строка является частью C-конструкции `int child_argc,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
110	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
111	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
112	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
113	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
114	`size_t rf_count,`	Технически эта строка является частью C-конструкции `size_t rf_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
115	`uint32_t timeout_ms,`	Технически эта строка является частью C-конструкции `uint32_t timeout_ms,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
116	`uint32_t trace_steps,`	Технически эта строка является частью C-конструкции `uint32_t trace_steps,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
117	`DynamicReport* out,`	Технически эта строка является частью C-конструкции `DynamicReport* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
118	`StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
120	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.

Стр.	Код	Нормальное объяснение
121	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
122	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 101. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/dynamic_win.c

Назначение файла: Windows debug-loop и runtime-сборщик динамических признаков.

Код, который пишем на этом шаге

```
#include "dynamic.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

#ifdef _WIN32

#include <windows.h>
#include <tlhelp32.h>

/* MinGW exposes exception_code as a compatibility macro. */
#ifdef exception_code
#undef exception_code
#endif

/* ------------------------------------------------------------------------- */

typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;

typedef struct ModList {
    ModInfo mods[128];
    size_t count;
    bool truncated;
} ModList;

static void mods_init(ModList* ml) { if (ml) memset(ml, 0, sizeof(*ml)); }

static const char* basename_lower(const char* path) {
    const char* base = path;
    for (const char* p = path; *p; p++) {
        if (*p == '\\' || *p == '/') base = p + 1;
    }
    return base;
}

static void strlower_inplace(char* s) {
    for (; s && *s; s++) {
        if (*s >= 'A' && *s <= 'Z') *s = (char)(*s - 'A' + 'a');
    }
}

static void mods_add(ModList* ml, const char* name, uint64_t base, uint64_t size) {
    if (!ml || !name) return;
    if (ml->count >= sizeof(ml->mods)/sizeof(ml->mods[0])) {
        ml->truncated = true;
        return;
    }
    ModInfo* m = &ml->mods[ml->count++];
    strncpy(m->name, name, sizeof(m->name)-1);
    m->name[sizeof(m->name)-1] = 0;
    strlower_inplace(m->name);
    m->base = base;
    m->size = size;
}

static const ModInfo* mods_find(const ModList* ml, const char* name) {
    if (!ml || !name) return NULL;
    char tmp[MAX_PATH];
    strncpy(tmp, name, sizeof(tmp)-1);
    tmp[sizeof(tmp)-1] = 0;
    strlower_inplace(tmp);
    for (size_t i = 0; i < ml->count; i++) {
        if (strcmp(ml->mods[i].name, tmp) == 0) return &ml->mods[i];
    }
    return NULL;
}

static const ModInfo* mods_find_by_addr(const ModList* ml, uint64_t addr) {
    if (!ml) return NULL;
    for (size_t i = 0; i < ml->count; i++) {
        uint64_t b = ml->mods[i].base;
        uint64_t e = ml->mods[i].size > UINT64_MAX - b
            ? UINT64_MAX
            : b + ml->mods[i].size;
        if (addr >= b && addr < e) return &ml->mods[i];
    }
    return NULL;
}

static bool mods_refresh_toolhelp(ModList* ml, DWORD pid) {
    if (!ml) return false;
    ml->count = 0;
    ml->truncated = false;
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid);
    if (snap == INVALID_HANDLE_VALUE) return false;

```

```
    MODULEENTRY32 me;
    memset(&me, 0, sizeof(me));
    me.dwSize = sizeof(me);
    if (!Module32First(snap, &me)) {
        CloseHandle(snap);
        return false;
    }
    do {
        const char* base = basename_lower(me.szModule);
        mods_add(ml, base, (uint64_t)(uintptr_t)me.modBaseAddr, (uint64_t)me.modBaseSize);
    } while (Module32Next(snap, &me));

    CloseHandle(snap);
    return true;
}

static bool mods_refresh_accounted(ModList* ml, DWORD pid, DynamicReport* dr) {
    if (!mods_refresh_toolhelp(ml, pid)) {
        if (dr) {
            dr->module_refresh_failures++;
            dr->instrumentation_complete = false;
        }
        return false;
    }
    if (dr) {
        if (ml->count > dr->modules_observed_peak) {
            dr->modules_observed_peak = (uint32_t)ml->count;
        }
        if (ml->truncated) {
            dr->module_list_truncations++;
            dr->instrumentation_complete = false;
        }
    }
    return true;
}

/* ------------------------------------------------------------------------ */

typedef enum BpKind {
    BP_UNWIND_HANDLER       = 1u << 0,
    BP_API_ADD_VEH          = 1u << 1,
    BP_API_ADD_VCH          = 1u << 2,
    BP_API_SET_UEF          = 1u << 3,
    BP_API_RTL_ADD_FT       = 1u << 4,
    BP_API_RTL_INSTALL_FTCB = 1u << 5,
    BP_CUSTOM_VEH_HANDLER   = 1u << 6,
    BP_CUSTOM_VCH_HANDLER   = 1u << 7,
    BP_CUSTOM_UEF           = 1u << 8
} BpKind;

typedef struct Breakpoint {
    uint64_t addr;
    uint8_t  orig;
    bool     armed;
    bool     rearm_pending;
    uint32_t kinds;
} Breakpoint;

typedef struct BpList {
    Breakpoint bps[512];
    size_t count;
} BpList;

static void bps_init(BpList* bl) { if (bl) memset(bl, 0, sizeof(*bl)); }

static Breakpoint* bps_find(BpList* bl, uint64_t addr) {
    if (!bl) return NULL;
    for (size_t i = 0; i < bl->count; i++) {
        if (bl->bps[i].addr == addr) return &bl->bps[i];
    }
    return NULL;
}

static bool write_one_byte(HANDLE hProcess, uint64_t addr, uint8_t val, StrBuf* err) {
    SIZE_T nw = 0;
    uint8_t b = val;
    if (WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) && nw == 1) {
        FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
        return true;
    }
    DWORD werr = GetLastError();

    /* Retry with temporary RWX. Some targets map code pages RX. */
    DWORD oldProt = 0;
    if (!VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, PAGE_EXECUTE_READWRITE, &oldProt)) {
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx (err=%lu) and VirtualProtectEx failed (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr, (unsigned long)GetLastError());
        return false;
    }

    nw = 0;
    if (!WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) || nw != 1) {
        DWORD werr2 = GetLastError();
        DWORD tmp = 0;
        VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr2);
        return false;
    }

    FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
    DWORD tmp = 0;
    VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
    return true;
}

static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    SIZE_T nread = 0;
    uint8_t b = 0;
    if (!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread != 1) {
        if (err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu)\n",
                            (unsigned long long)bp->addr, (unsigned long)GetLastError());
        return false;
```

```
    }
    bp->orig = b;
    if (!write_one_byte(hProcess, bp->addr, 0xCC, err)) return false;
    bp->armed = true;
    bp->rearm_pending = false;
    return true;
}

static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    if (!bp || !bp->armed) return true;
    if (!write_one_byte(hProcess, bp->addr, bp->orig, err)) return false;
    bp->armed = false;
    return true;
}

static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf* err) {
    if (!bl) return false;

    Breakpoint* ex = bps_find(bl, addr);
    if (ex) {
        ex->kinds |= (uint32_t)kind;
        if (ex->armed || ex->rearm_pending) return true;
        return bp_set(hProcess, ex, err);
    }

    if (bl->count >= sizeof(bl->bps)/sizeof(bl->bps[0])) return false;
    Breakpoint* bp = &bl->bps[bl->count];
    memset(bp, 0, sizeof(*bp));
    bp->addr = addr;
    bp->kinds = (uint32_t)kind;

    if (!bp_set(hProcess, bp, err)) return false;
    bl->count++;
    return true;
}

/* ------------------------------------------------------------------------ */
/* Self-check helpers: log where breakpoints were actually placed in the target.
   Goal: make it obvious whether a requested breakpoint is (a) placed, (b) placed in the
   expected module, and (c) armed in memory (0xCC). */

static const char* bp_kind_name(BpKind k); /* forward */
static void dr_append(DynamicReport* dr, const char* fmt, ...); /* forward */

static bool bp_peek_byte(HANDLE hProcess, uint64_t addr, uint8_t* outb, StrBuf* err) {
    if (!outb) return false;
    SIZE_T nr = 0;
    uint8_t b = 0;
    if (ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)addr, &b, 1, &nr) && nr == 1) {
        *outb = b;
        return true;
    }
    if (err) sb_appendf(err, "ReadProcessMemory peek failed at 0x%llx (err=%lu)\n",
                        (unsigned long long)addr, (unsigned long)GetLastError());
    return false;
}

static void format_bp_location(const ModList* mods, uint64_t addr, char* out, size_t outsz) {
    if (!out || outsz == 0) return;
    out[0] = 0;
    const ModInfo* mi = mods ? mods_find_by_addr(mods, addr) : NULL;
    if (mi) {
        snprintf(out, outsz, "%s+0x%llx (0x%llx)",
                 mi->name,
                 (unsigned long long)(addr - mi->base),
                 (unsigned long long)addr);
    } else {
        snprintf(out, outsz, "0x%llx", (unsigned long long)addr);
    }
}

/* Place a breakpoint and emit a deterministic self-check line in dr->details.
   To reduce log spam, this logs only on the first attempt to create the bp at addr. */
static bool bp_add_and_arm_selfcheck(BpList* bl,
                                     HANDLE hProcess,
                                     const ModList* mods,
                                     DynamicReport* dr,
                                     uint64_t addr,
                                     BpKind kind,
                                     const char* what,
                                     StrBuf* err) {
    if (!bl) return false;

    Breakpoint* existing = bps_find(bl, addr);
    bool first_time = (existing == NULL);
    bool capacity_exhausted =
        first_time && bl->count >= sizeof(bl->bps) / sizeof(bl->bps[0]);
    if (dr && first_time) dr->breakpoint_set_attempts++;

    StrBuf te; sb_init(&te, 256);
    bool ok = bp_add_and_arm(bl, hProcess, addr, kind, &te);

    if (dr && !ok) {
        dr->breakpoint_set_failures++;
        if (capacity_exhausted) dr->breakpoint_capacity_drops++;
        dr->instrumentation_complete = false;
    }

    if (dr && first_time) {
        char loc[192];
        format_bp_location(mods, addr, loc, sizeof(loc));

        uint8_t cur = 0xFF;
        StrBuf peek_err; sb_init(&peek_err, 128);
        bool peek_ok = bp_peek_byte(hProcess, addr, &cur, &peek_err);

        if (ok) {
            if (peek_ok) {
                dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte=0x%02x)%s\n",
                          bp_kind_name(kind),
                          (what ? what : ""),
                          loc,
                          (unsigned)cur,
                          (cur == 0xCC ? "" : " [WARN: not INT3]"));
            } else {
```

```
                dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (peek failed: %s)\n",
                          bp_kind_name(kind), (what ? what : ""), loc,
                          (peek_err.s ? peek_err.s : "unknown"));
            }
        } else {
            dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : FAIL%s%s",
                      bp_kind_name(kind),
                      (what ? what : ""),
                      loc,
                      (te.len ? " : " : "\n"),
                      (te.len ? te.s : "\n"));
        }

        sb_free(&peek_err);
    }

    if (!ok && err && te.len) sb_append(err, te.s);
    sb_free(&te);
    return ok;
}


/* ------------------------------------------------------------------------ */

/* Pending breakpoint re-arming must be tracked per-thread.
   With multiple threads, a single global pending pointer can cause lost rearms. */
typedef struct PendingRearmEntry {
    DWORD tid;
    Breakpoint* bp;
} PendingRearmEntry;

typedef struct PendingRearmList {
    PendingRearmEntry e[64];
} PendingRearmList;

static void pending_rearm_init(PendingRearmList* pr) {
    if (!pr) return;
    memset(pr, 0, sizeof(*pr));
}

static bool pending_rearm_set(PendingRearmList* pr, DWORD tid, Breakpoint* bp) {
    if (!pr || !bp) return false;
    /* overwrite existing */
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (pr->e[i].bp && pr->e[i].tid == tid) {
            pr->e[i].bp = bp;
            return true;
        }
    }
    /* take a free slot */
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (!pr->e[i].bp) {
            pr->e[i].tid = tid;
            pr->e[i].bp = bp;
            return true;
        }
    }
    return false;
}

static Breakpoint* pending_rearm_take(PendingRearmList* pr, DWORD tid) {
    if (!pr) return NULL;
    for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
        if (pr->e[i].bp && pr->e[i].tid == tid) {
            Breakpoint* bp = pr->e[i].bp;
            pr->e[i].bp = NULL;
            pr->e[i].tid = 0;
            return bp;
        }
    }
    return NULL;
}

static HANDLE open_event_thread(DWORD tid) {
    return OpenThread(THREAD_GET_CONTEXT | THREAD_SET_CONTEXT | THREAD_QUERY_INFORMATION, FALSE, tid);
}

static void close_event_thread(HANDLE h) {
    if (h) CloseHandle(h);
}

static void dr_append(DynamicReport* dr, const char* fmt, ...) {
    if (!dr) return;
    va_list ap;
    va_start(ap, fmt);
    /* Reuse sb_appendf's variadic through vsnprintf into temp */
    char tmp[1024];
    int n = vsnprintf(tmp, sizeof(tmp), fmt, ap);
    va_end(ap);
    if (n <= 0) return;
    tmp[sizeof(tmp)-1] = 0;
    sb_append(&dr->details, tmp);
}

static void dynamic_event_add(DynamicReport* dr,
                              const ModList* mods,
                              const char* type,
                              DWORD thread_id,
                              uint64_t address,
                              uint32_t exception_code,
                              bool first_chance,
                              uint64_t arg0,
                              uint64_t arg1,
                              uint64_t arg2,
                              uint64_t arg3) {
    if (!dr || !type) return;
    if (dr->event_count >= DYNAMIC_MAX_EVENTS) {
        dr->dropped_events++;
        dr->instrumentation_complete = false;
        return;
    }

    DynamicEvent* event = &dr->events[dr->event_count++];
    memset(event, 0, sizeof(*event));
    snprintf(event->type, sizeof(event->type), "%s", type);
```

```
    const ModInfo* module = mods_find_by_addr(mods, address);
    if (module) snprintf(event->module, sizeof(event->module), "%s", module->name);
    event->thread_id = (uint32_t)thread_id;
    event->exception_code = exception_code;
    event->first_chance = first_chance;
    event->address = address;
    event->arg0 = arg0;
    event->arg1 = arg1;
    event->arg2 = arg2;
    event->arg3 = arg3;
}

void dynamic_report_init(DynamicReport* dr) {
    if (!dr) return;
    memset(dr, 0, sizeof(*dr));
    dr->enabled = true;
    dr->instrumentation_complete = true;
    sb_init(&dr->details, 2048);
}

void dynamic_report_free(DynamicReport* dr) {
    if (!dr) return;
    sb_free(&dr->details);
    memset(dr, 0, sizeof(*dr));
}

static bool append_backslashes(StrBuf* out, size_t count) {
    for (size_t i = 0; i < count; i++) {
        if (!sb_appendn(out, "\\", 1)) return false;
    }
    return true;
}

/* Quote one argument according to the Windows C runtime command-line rules. */
static bool append_cmd_arg(StrBuf* out, const char* arg) {
    if (!out || !arg) return false;
    bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;
    if (!quote) return sb_append(out, arg);
    if (!sb_appendn(out, "\"", 1)) return false;

    const char* p = arg;
    while (*p) {
        size_t backslashes = 0;
        while (*p == '\\') {
            backslashes++;
            p++;
        }
        if (*p == '"') {
            if (!append_backslashes(out, backslashes * 2u + 1u)) return false;
            if (!sb_appendn(out, "\"", 1)) return false;
            p++;
        } else if (*p == '\0') {
            if (!append_backslashes(out, backslashes * 2u)) return false;
        } else {
            if (!append_backslashes(out, backslashes)) return false;
            if (!sb_appendn(out, p, 1)) return false;
            p++;
        }
    }
    return sb_appendn(out, "\"", 1);
}

/* Build a command line for CreateProcessA while preserving target arguments. */
static bool build_cmdline(const char* path, const char* const* argv, int argc, StrBuf* out) {
    if (!path || !out) return false;
    out->len = 0;
    if (out->s) out->s[0] = 0;

    if (!append_cmd_arg(out, path)) return false;

    for (int i = 0; i < argc; i++) {
        const char* a = argv[i];
        if (!a || !sb_appendn(out, " ", 1) || !append_cmd_arg(out, a)) return false;
    }
    return true;
}

/* Resolve a symbol to (real module basename, offset) in *this* process.
   Important: many APIs are forwarded (e.g. kernel32 -> kernelbase). */
typedef struct LocalProcLoc {
    char mod_name[MAX_PATH]; /* lower-case basename, e.g. kernelbase.dll */
    uint64_t offset;
} LocalProcLoc;

static bool local_proc_loc(const char* preferred_mod, const char* sym, LocalProcLoc* out) {
    if (!preferred_mod || !sym || !out) return false;
    memset(out, 0, sizeof(*out));

    HMODULE hPref = GetModuleHandleA(preferred_mod);
    if (!hPref) return false;

    FARPROC p = GetProcAddress(hPref, sym);
    if (!p) return false;

    HMODULE hReal = NULL;
#if defined(__GNUC__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wpedantic"
#endif
    if (!GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS | GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,
                            (LPCSTR)p, &hReal) || !hReal) {
        hReal = hPref;
    }
#if defined(__GNUC__)
#pragma GCC diagnostic pop
#endif

    char path[MAX_PATH];
    memset(path, 0, sizeof(path));
    if (!GetModuleFileNameA(hReal, path, (DWORD)sizeof(path))) {
        strncpy(out->mod_name, preferred_mod, sizeof(out->mod_name)-1);
        out->mod_name[sizeof(out->mod_name)-1] = 0;
        strlower_inplace(out->mod_name);
    } else {
        const char* base = basename_lower(path);
```

```
        strncpy(out->mod_name, base, sizeof(out->mod_name)-1);
        out->mod_name[sizeof(out->mod_name)-1] = 0;
        strlower_inplace(out->mod_name);
    }

    out->offset = (uint64_t)(uintptr_t)p - (uint64_t)(uintptr_t)hReal;
    return true;
}

static bool add_api_bps_if_ready(BpList* bl, HANDLE hProcess, const ModList* mods, DynamicReport* dr, StrBuf* err) {
    if (!bl || !mods) return false;

    const struct {
        const char* preferred_mod;
        const char* sym;
        BpKind kind;
    } apis[] = {
        {"kernel32.dll", "AddVectoredExceptionHandler", BP_API_ADD_VEH},
        {"kernel32.dll", "AddVectoredContinueHandler", BP_API_ADD_VCH},
        {"kernel32.dll", "SetUnhandledExceptionFilter", BP_API_SET_UEF},
        {"ntdll.dll",    "RtlAddFunctionTable", BP_API_RTL_ADD_FT},
        {"ntdll.dll",    "RtlInstallFunctionTableCallback", BP_API_RTL_INSTALL_FTCB},
    };

    for (size_t i = 0; i < sizeof(apis)/sizeof(apis[0]); i++) {
        LocalProcLoc loc;
        if (!local_proc_loc(apis[i].preferred_mod, apis[i].sym, &loc)) continue;

        const ModInfo* m = mods_find(mods, loc.mod_name);
        if (!m) {
            if (dr) dr_append(dr, "[Dynamic] API %s resolved to %s but module not present in target yet\n",
                              apis[i].sym, loc.mod_name);
            continue;
        }

        uint64_t bp_addr = m->base + loc.offset;
        {
            char what[320];
            snprintf(what, sizeof(what), "%s!%s", loc.mod_name, apis[i].sym);
            if (!bp_add_and_arm_selfcheck(bl, hProcess, mods, dr, bp_addr, apis[i].kind, what, err)) {
                if (dr) dr_append(dr, "[Dynamic] failed to set API breakpoint %s at 0x%llx\n",
                                  what, (unsigned long long)bp_addr);
            }
        }
    }

    return true;
}

static const char* bp_kind_name(BpKind k) {
    switch (k) {
        case BP_UNWIND_HANDLER: return "unwind_handler";
        case BP_API_ADD_VEH: return "api_add_veh";
        case BP_API_ADD_VCH: return "api_add_vch";
        case BP_API_SET_UEF: return "api_set_uef";
        case BP_API_RTL_ADD_FT: return "api_rtl_add_ft";
        case BP_API_RTL_INSTALL_FTCB: return "api_rtl_install_ftcb";
        case BP_CUSTOM_VEH_HANDLER: return "veh_handler";
        case BP_CUSTOM_VCH_HANDLER: return "vch_handler";
        case BP_CUSTOM_UEF: return "uef_handler";
        default: return "bp";
    }
}

static void add_trace(DynamicReport* dr, uint64_t rip) {
    if (!dr) return;
    if (dr->trace_count >= (uint32_t)(sizeof(dr->trace)/sizeof(dr->trace[0]))) return;
    dr->trace[dr->trace_count++] = rip;
}

static void start_trace(DynamicReport* dr,
                        uint32_t trace_steps,
                        DWORD tid,
                        uint64_t rip,
                        bool* tracing,
                        DWORD* trace_tid,
                        uint32_t* trace_left) {
    if (!dr || !tracing || !trace_tid || !trace_left ||
        trace_steps == 0 || dr->trace_count != 0) {
        return;
    }

    uint32_t budget = trace_steps > 256u ? 256u : trace_steps;
    add_trace(dr, rip);
    *trace_left = budget - 1u;
    *tracing = *trace_left > 0;
    *trace_tid = *tracing ? tid : 0;
}

static void addr_to_loc(const PeImage* img, uint64_t base, uint64_t addr, char* out, size_t outsz) {
    if (!out || outsz == 0) return;
    out[0] = 0;
    if (!img || addr < base) {
        snprintf(out, outsz, "0x%llx", (unsigned long long)addr);
        return;
    }
    uint32_t rva = (uint32_t)(addr - base);
    const PeSection* s = pe_find_section_for_rva(img, rva);
    if (s) snprintf(out, outsz, "0x%llx (rva=0x%08x sec=%s)", (unsigned long long)addr, (unsigned)rva, s->name);
    else snprintf(out, outsz, "0x%llx (rva=0x%08x)", (unsigned long long)addr, (unsigned)rva);
}

/* ------------------------------------------------------------------------- */

bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
                             const UnwindSummary* us,
                             const bool* us_ok,
                             size_t rf_count,
                             uint32_t timeout_ms,
                             uint32_t trace_steps,
```

```
                            DynamicReport* out,
                            StrBuf* err) {
    if (!out || !err || !pe_path || !img) return false;
    if (rf_count > 0 && (!rf || !us || !us_ok)) return false;

    dynamic_report_init(out);
    if (!sb_ok(&out->details)) {
        sb_append(err, "Out of memory (dynamic report buffer).\n");
        return false;
    }
    out->enabled = true;
    out->timeout_ms = timeout_ms;
    out->trace_steps = trace_steps > 256u ? 256u : trace_steps;

    StrBuf cmd;
    if (!sb_init(&cmd, 1024)) {
        sb_append(err, "Out of memory (command line buffer).\n");
        return false;
    }
    if (!build_cmdline(pe_path, child_argv, child_argc, &cmd)) {
        sb_append(err, "Failed to build command line.\n");
        sb_free(&cmd);
        return false;
    }

    STARTUPINFOA si;
    PROCESS_INFORMATION pi;
    memset(&si, 0, sizeof(si));
    memset(&pi, 0, sizeof(pi));
    si.cb = sizeof(si);

    /*
     * Do not use CREATE_SUSPENDED.
     *
     * We want the standard Windows Debug API event stream to start
     * immediately after CreateProcess. On some configurations,
     * creating the debuggee suspended can result in no debug events
     * being delivered (no CREATE_PROCESS_DEBUG_EVENT), which causes
     * a deadlock (the tool waits forever, never resumes the thread).
     */
    DWORD flags = DEBUG_ONLY_THIS_PROCESS;

    if (!CreateProcessA(NULL, cmd.s, NULL, NULL, FALSE, flags, NULL, NULL, &si, &pi)) {
        sb_appendf(err, "CreateProcess failed (err=%lu)\n", GetLastError());
        sb_free(&cmd);
        return false;
    }

    out->ran = true;

    HANDLE hProcess = pi.hProcess;
    DWORD pid = pi.dwProcessId;

    ModList mods;
    mods_init(&mods);

    BpList bps;
    bps_init(&bps);

    /* Collect unique handler RVAs from static parse. */
    uint32_t* handler_rvas = (uint32_t*)calloc(rf_count ? rf_count : 1, sizeof(uint32_t));
    if (!handler_rvas) {
        sb_append(err, "Out of memory (handler RVA list).\n");
        TerminateProcess(hProcess, 0);
        if (pi.hThread) CloseHandle(pi.hThread);
        if (pi.hProcess) CloseHandle(pi.hProcess);
        sb_free(&cmd);
        return false;
    }
    size_t handler_count = 0;
    for (size_t i = 0; i < rf_count; i++) {
        if (!us_ok[i] || !us[i].has_handler) continue;
        uint32_t hr = us[i].handler_rva;
        bool seen = false;
        for (size_t j = 0; j < handler_count; j++) {
            if (handler_rvas[j] == hr) { seen = true; break; }
        }
        if (!seen) handler_rvas[handler_count++] = hr;
    }
    out->unwind_handlers_discovered = (uint32_t)handler_count;

    DEBUG_EVENT ev;
    bool got_create = false;
    bool debug_loop_ok = true;
    bool process_exited = false;
    bool timed_out = false;
    uint64_t base_main = 0;

    bool initial_break_ignored = false;

    uint64_t start = GetTickCount64();

    PendingRearmList pending;
    pending_rearm_init(&pending);
    bool tracing = false;
    uint32_t trace_left = 0;
    DWORD trace_tid = 0;

    /* Let debug loop run */
    for (;;) {
        uint64_t elapsed = GetTickCount64() - start;
        if (elapsed > timeout_ms) {
            dr_append(out, "[Dynamic] timeout reached (%u ms)\n", timeout_ms);
            dynamic_event_add(out, &mods, "timeout", 0, 0, 0, false, timeout_ms, 0, 0, 0);
            timed_out = true;
            break;
        }

        if (!WaitForDebugEvent(&ev, 100)) {
            DWORD werr = GetLastError();
            if (werr == ERROR_SEM_TIMEOUT) {
                continue;
            }
            dr_append(out, "[Dynamic] WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);
            sb_appendf(err, "WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);
```

```
782 |             debug_loop_ok = false;
783 |             break;
784 |         }
785 |
786 |         DWORD cont = DBG_CONTINUE;
787 |         bool finish_after_continue = false;
788 |
789 |         if (ev.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT) {
790 |             got_create = true;
791 |             base_main = (uint64_t)(uintptr_t)ev.u.CreateProcessInfo.lpBaseOfImage;
792 |             dr_append(out, "[Dynamic] CREATE_PROCESS pid=%lu base=0x%llx\n", (unsigned long)pid, (unsigned long long)base_main);
793 |
794 |             /* refresh module list and add API breakpoints */
795 |             (void)mods_refresh_accounted(&mods, pid, out);
796 |             add_api_bps_if_ready(&bps, hProcess, &mods, out, err);
797 |
798 |             /* Set breakpoints on handlers referenced in UNWIND_INFO */
799 |             for (size_t i = 0; i < handler_count; i++) {
800 |                 uint64_t a = base_main + (uint64_t)handler_rvas[i];
801 |                 char what[96];
802 |                 snprintf(what, sizeof(what), "unwind_handler_rva=0x%08lx", (unsigned long)handler_rvas[i]);
803 |                 if (bp_add_and_arm_selfcheck(&bps, hProcess, &mods, out, a, BP_UNWIND_HANDLER, what, err)) {
804 |                     out->unwind_handler_bp_set++;
805 |                 } else {
806 |                     /* best effort: ignore failures (some handlers could be invalid/outside main image) */
807 |                 }
808 |             }
809 |
810 |             /* Process continues after ContinueDebugEvent(). */
811 |             CloseHandle(ev.u.CreateProcessInfo.hFile);
812 |         }
813 |         else if (ev.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT) {
814 |             /* keep module list updated so we can set API breakpoints */
815 |             (void)mods_refresh_accounted(&mods, pid, out);
816 |             add_api_bps_if_ready(&bps, hProcess, &mods, out, err);
817 |             if (ev.u.LoadDll.hFile) CloseHandle(ev.u.LoadDll.hFile);
818 |         }
819 |         else if (ev.dwDebugEventCode == UNLOAD_DLL_DEBUG_EVENT) {
820 |             (void)mods_refresh_accounted(&mods, pid, out);
821 |         }
822 |         else if (ev.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT) {
823 |             dr_append(out, "[Dynamic] EXIT_PROCESS code=%lu\n", (unsigned long)ev.u.ExitProcess.dwExitCode);
824 |             process_exited = true;
825 |             out->target_exited = true;
826 |             out->target_exit_code = (uint32_t)ev.u.ExitProcess.dwExitCode;
827 |             dynamic_event_add(
828 |                 out, &mods, "process_exit", ev.dwThreadId, 0, 0, false,
829 |                 ev.u.ExitProcess.dwExitCode, 0, 0, 0);
830 |             finish_after_continue = true;
831 |         }
832 |         else if (ev.dwDebugEventCode == EXCEPTION_DEBUG_EVENT) {
833 |             DWORD code = ev.u.Exception.ExceptionRecord.ExceptionCode;
834 |             uint64_t addr = (uint64_t)(uintptr_t)ev.u.Exception.ExceptionRecord.ExceptionAddress;
835 |
836 |             if (code != EXCEPTION_BREAKPOINT && code != EXCEPTION_SINGLE_STEP) {
837 |                 bool first_chance = ev.u.Exception.dwFirstChance != 0;
838 |                 out->exceptions_seen++;
839 |                 if (first_chance) out->first_chance_seen++;
840 |                 else out->second_chance_seen++;
841 |                 dr_append(out, "[Dynamic] exception 0x%08lx at 0x%llx (%s chance)\n",
842 |                           (unsigned long)code,
843 |                           (unsigned long long)addr,
844 |                           first_chance ? "first" : "second");
845 |                 dynamic_event_add(
846 |                     out, &mods, "exception", ev.dwThreadId, addr, (uint32_t)code,
847 |                     first_chance, 0, 0, 0, 0);
848 |                 /* Preserve the target's normal exception-dispatch flow. */
849 |                 cont = DBG_EXCEPTION_NOT_HANDLED;
850 |             }
851 |
852 |             if (code == EXCEPTION_BREAKPOINT) {
853 |                 Breakpoint* bp = bps_find(&bps, addr);
854 |                 if (!bp && !initial_break_ignored) {
855 |                     /* Ignore the debugger's system initial breakpoint only when it is not ours. */
856 |                     initial_break_ignored = true;
857 |                     cont = DBG_CONTINUE;
858 |                 } else if (!bp) {
859 |                     cont = DBG_EXCEPTION_NOT_HANDLED;
860 |                 } else {
861 |                     bool bp_prepared = bp_clear(hProcess, bp, err);
862 |                     if (bp_prepared) {
863 |                         bp->rearm_pending = true;
864 |                         bp_prepared = pending_rearm_set(&pending, ev.dwThreadId, bp);
865 |                         if (!bp_prepared) {
866 |                             bp->rearm_pending = false;
867 |                             out->pending_rearm_failures++;
868 |                             out->instrumentation_complete = false;
869 |                         }
870 |                     }
871 |                     if (!bp_prepared) {
872 |                         dr_append(out, "[Dynamic] failed to prepare breakpoint rearm at 0x%llx\n",
873 |                                   (unsigned long long)addr);
874 |                         debug_loop_ok = false;
875 |                         finish_after_continue = true;
876 |                         cont = DBG_CONTINUE;
877 |                     } else {
878 |                         HANDLE hEvThread = open_event_thread(ev.dwThreadId);
879 |                         if (!hEvThread) {
880 |                             dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",
881 |                                       (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
882 |                             debug_loop_ok = false;
883 |                             finish_after_continue = true;
884 |                             cont = DBG_CONTINUE;
885 |                         } else {
886 |                             CONTEXT ctx;
887 |                             memset(&ctx, 0, sizeof(ctx));
888 |                             ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;
889 |
890 |                             if (!GetThreadContext(hEvThread, &ctx)) {
891 |                                 dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",
892 |                                           (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
893 |                                 close_event_thread(hEvThread);
894 |                                 debug_loop_ok = false;
895 |                                 finish_after_continue = true;
896 |                                 cont = DBG_CONTINUE;
```

```
                            } else {
                                /* RIP points to INT3+1; rewind */
                                ctx.Rip = (DWORD64)addr;
                                ctx.EFlags |= 0x100; /* Trap flag */

                                /* handle API breaks: capture handler pointers */
                                if (bp->kinds & BP_API_ADD_VEH) {
                                    uint64_t first = (uint64_t)ctx.Rcx;
                                    uint64_t handler = (uint64_t)ctx.Rdx;
                                    out->veh_registrations++;
                                    dr_append(out, "[Dynamic] AddVectoredExceptionHandler first=%llu handler=0x%llx\n",
                                              (unsigned long long)first, (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "veh_registration", ev.dwThreadId, addr, 0, false,
                                        first, handler, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_VEH_HANDLER, "custom_veh_handler", err);
                                    }
                                }
                                if (bp->kinds & BP_API_ADD_VCH) {
                                    uint64_t first = (uint64_t)ctx.Rcx;
                                    uint64_t handler = (uint64_t)ctx.Rdx;
                                    out->vch_registrations++;
                                    dr_append(out, "[Dynamic] AddVectoredContinueHandler first=%llu handler=0x%llx\n",
                                              (unsigned long long)first, (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "vch_registration", ev.dwThreadId, addr, 0, false,
                                        first, handler, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_VCH_HANDLER, "custom_vch_handler", err);
                                    }
                                }
                                if (bp->kinds & BP_API_SET_UEF) {
                                    out->uef_set_calls++;
                                    uint64_t handler = (uint64_t)ctx.Rcx;
                                    dr_append(out, "[Dynamic] SetUnhandledExceptionFilter handler=0x%llx\n",
                                              (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "uef_registration", ev.dwThreadId, addr, 0, false,
                                        handler, 0, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_UEF, "custom_uef_handler", err);
                                    }
                                }
                                if (bp->kinds & BP_API_RTL_ADD_FT) {
                                    uint64_t table = (uint64_t)ctx.Rcx;
                                    uint64_t count = (uint64_t)ctx.Rdx;
                                    uint64_t baseaddr = (uint64_t)ctx.R8;
                                    out->rtl_add_function_table_calls++;
                                    dr_append(
                                        out,
                                        "[Dynamic] RtlAddFunctionTable table=0x%llx count=%llu base=0x%llx\n",
                                        (unsigned long long)table,
                                        (unsigned long long)count,
                                        (unsigned long long)baseaddr);
                                    dynamic_event_add(
                                        out, &mods, "rtl_add_function_table",
                                        ev.dwThreadId, addr, 0, false,
                                        table, count, baseaddr, 0);
                                }
                                if (bp->kinds & BP_API_RTL_INSTALL_FTCB) {
                                    uint64_t table_id = (uint64_t)ctx.Rcx;
                                    uint64_t baseaddr = (uint64_t)ctx.Rdx;
                                    uint64_t length = (uint64_t)ctx.R8;
                                    uint64_t callback = (uint64_t)ctx.R9;
                                    out->rtl_install_function_table_cb_calls++;
                                    dr_append(
                                        out,
                                        "[Dynamic] RtlInstallFunctionTableCallback id=0x%llx base=0x%llx length=%llu
callback=0x%llx\n",
                                        (unsigned long long)table_id,
                                        (unsigned long long)baseaddr,
                                        (unsigned long long)length,
                                        (unsigned long long)callback);
                                    dynamic_event_add(
                                        out, &mods, "rtl_install_function_table_callback",
                                        ev.dwThreadId, addr, 0, false,
                                        table_id, baseaddr, length, callback);
                                }
                                if (bp->kinds & BP_UNWIND_HANDLER) {
                                    out->unwind_handler_hits++;
                                    char loc[128];
                                    addr_to_loc(img, base_main, addr, loc, sizeof(loc));
                                    dr_append(out, "[Dynamic] hit unwind handler at %s\n", loc);
                                    dynamic_event_add(
                                        out, &mods, "unwind_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_VEH_HANDLER) {
                                    out->veh_hits++;
                                    dr_append(out, "[Dynamic] hit vectored exception handler 0x%llx\n", (unsigned long long)addr);
                                    dynamic_event_add(
                                        out, &mods, "veh_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_VCH_HANDLER) {
                                    out->vch_hits++;
                                    dr_append(out, "[Dynamic] hit vectored continue handler 0x%llx\n", (unsigned long long)addr);
```

```
                                    dynamic_event_add(
                                        out, &mods, "vch_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_UEF) {
                                    out->uef_hits++;
                                    dr_append(out, "[Dynamic] hit unhandled exception filter 0x%llx\n", (unsigned long long)addr);
                                    dynamic_event_add(
                                        out, &mods, "uef_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }

                                if (!SetThreadContext(hEvThread, &ctx)) {
                                    dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",
                                              (unsigned long)ev.dwThreadId,
                                              (unsigned long)GetLastError());
                                    debug_loop_ok = false;
                                    finish_after_continue = true;
                                }
                                close_event_thread(hEvThread);
                                cont = DBG_CONTINUE;
                            }
                        }
                    }
                }
                else if (code == EXCEPTION_SINGLE_STEP) {
                    /* Re-arm the breakpoint after executing original instruction (per-thread). */
                    Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);
                    if (pr_bp) {
                        StrBuf te; sb_init(&te, 128);
                        if (!bp_set(hProcess, pr_bp, &te)) {
                            out->breakpoint_rearm_failures++;
                            out->instrumentation_complete = false;
                            dr_append(out, "[Dynamic] failed to re-arm breakpoint at 0x%llx: %s",
                                      (unsigned long long)pr_bp->addr,
                                      te.s ? te.s : "unknown error\n");
                            debug_loop_ok = false;
                            finish_after_continue = true;
                        }
                        sb_free(&te);
                    }

                    HANDLE hEvThread = open_event_thread(ev.dwThreadId);
                    if (!hEvThread) {
                        dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",
                                  (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                        debug_loop_ok = false;
                        finish_after_continue = true;
                        cont = DBG_CONTINUE;
                    } else {
                        CONTEXT ctx;
                        memset(&ctx, 0, sizeof(ctx));
                        ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;

                        if (!GetThreadContext(hEvThread, &ctx)) {
                            dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",
                                      (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                            close_event_thread(hEvThread);
                            debug_loop_ok = false;
                            finish_after_continue = true;
                            cont = DBG_CONTINUE;
                        } else {
                            if (tracing && trace_left && ev.dwThreadId == trace_tid) {
                                add_trace(out, (uint64_t)ctx.Rip);
                                trace_left--;
                                /* keep trap flag */
                                ctx.EFlags |= 0x100;
                                if (trace_left == 0) {
                                    tracing = false;
                                    trace_tid = 0;
                                    ctx.EFlags &= ~0x100u;
                                    dr_append(out, "[Dynamic] trace collected: %u steps\n", (unsigned)out->trace_count);
                                }
                            } else {
                                /* stop single-stepping */
                                ctx.EFlags &= ~0x100u;
                            }
                            if (!SetThreadContext(hEvThread, &ctx)) {
                                dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",
                                          (unsigned long)ev.dwThreadId,
                                          (unsigned long)GetLastError());
                                debug_loop_ok = false;
                                finish_after_continue = true;
                            }
                            close_event_thread(hEvThread);
                            cont = DBG_CONTINUE;
                        }
                    }
                }

            }
            else if (ev.dwDebugEventCode == EXIT_THREAD_DEBUG_EVENT) {
                Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);
                if (pr_bp) {
                    StrBuf te;
                    sb_init(&te, 128);
                    if (!bp_set(hProcess, pr_bp, &te)) {
                        out->breakpoint_rearm_failures++;
                        out->instrumentation_complete = false;
                        dr_append(out, "[Dynamic] failed to re-arm breakpoint after thread exit: %s",
                                  te.s ? te.s : "unknown error\n");
                        debug_loop_ok = false;
                        finish_after_continue = true;
                    }
                    sb_free(&te);
                }
```

```
            if (tracing && trace_tid == ev.dwThreadId) {
                tracing = false;
                trace_tid = 0;
                trace_left = 0;
            }
        }

        if (!ContinueDebugEvent(ev.dwProcessId, ev.dwThreadId, cont)) {
            DWORD cerr = GetLastError();
            dr_append(out, "[Dynamic] ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);
            sb_appendf(err, "ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);
            debug_loop_ok = false;
            break;
        }
        if (finish_after_continue) break;
    }

    out->timed_out = timed_out;
    out->target_exited = process_exited;
    out->ok = got_create && debug_loop_ok && !timed_out;
    if (!sb_ok(&out->details)) {
        out->ok = false;
        sb_append(err, "Out of memory while building dynamic details.\n");
    }

    /* Cleanup: remove breakpoints */
    for (size_t i = 0; i < bps.count; i++) {
        bp_clear(hProcess, &bps.bps[i], NULL);
    }

    /*
     * Terminate only when the target did not exit by itself. A debuggee may
     * not signal its process handle until the debugger continues the queued
     * exit event, so drain cleanup events instead of waiting a fixed 5 s.
     */
    if (!process_exited && TerminateProcess(hProcess, 0)) {
        uint64_t cleanup_deadline = GetTickCount64() + 1000u;
        while (WaitForSingleObject(hProcess, 0) != WAIT_OBJECT_0 &&
               GetTickCount64() < cleanup_deadline) {
            DEBUG_EVENT cleanup_event;
            if (!WaitForDebugEvent(&cleanup_event, 50)) {
                if (GetLastError() == ERROR_SEM_TIMEOUT) continue;
                break;
            }

            if (cleanup_event.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT &&
                cleanup_event.u.CreateProcessInfo.hFile) {
                CloseHandle(cleanup_event.u.CreateProcessInfo.hFile);
            } else if (cleanup_event.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT &&
                       cleanup_event.u.LoadDll.hFile) {
                CloseHandle(cleanup_event.u.LoadDll.hFile);
            }

            bool cleanup_exited =
                cleanup_event.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT;
            if (!ContinueDebugEvent(
                    cleanup_event.dwProcessId,
                    cleanup_event.dwThreadId,
                    DBG_CONTINUE)) {
                break;
            }
            if (cleanup_exited) break;
        }
        (void)WaitForSingleObject(hProcess, 100);
    }

    if (pi.hThread) CloseHandle(pi.hThread);
    if (pi.hProcess) CloseHandle(pi.hProcess);

    free(handler_rvas);
    sb_free(&cmd);

    if (!out->ok) {
        if (timed_out) {
            sb_appendf(err, "Dynamic session timed out after %u ms.\n", timeout_ms);
        } else if (!got_create) {
            sb_append(err, "Dynamic session failed before CREATE_PROCESS_DEBUG_EVENT.\n");
        } else {
            sb_append(err, "Dynamic session failed inside the debug event loop.\n");
        }
        return false;
    }
    return true;
}

#else /* !_WIN32 */

void dynamic_report_init(DynamicReport* dr) {
    if (!dr) return;
    memset(dr, 0, sizeof(*dr));
    dr->enabled = false;
    sb_init(&dr->details, 256);
    sb_append(&dr->details, "Dynamic mode is supported only on Windows.\n");
}

void dynamic_report_free(DynamicReport* dr) {
    if (!dr) return;
    sb_free(&dr->details);
    memset(dr, 0, sizeof(*dr));
}

bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
                             const UnwindSummary* us,
                             const bool* us_ok,
                             size_t rf_count,
                             uint32_t timeout_ms,
                             uint32_t trace_steps,
                             DynamicReport* out,
                             StrBuf* err) {
    (void)pe_path; (void)child_argv; (void)child_argc; (void)img; (void)rf; (void)us; (void)us_ok; (void)rf_count;
    (void)timeout_ms; (void)trace_steps;
```

```
1241 |     dynamic_report_init(out);
1242 |     if (!out || !sb_ok(&out->details)) {
1243 |         sb_append(err, "Out of memory (dynamic report buffer).\n");
1244 |         return false;
1245 |     }
1246 |     out->enabled = true;
1247 |     out->timeout_ms = timeout_ms;
1248 |     out->trace_steps = trace_steps > 256u ? 256u : trace_steps;
1249 |     sb_append(err, "Dynamic mode is supported only on Windows.\n");
1250 |     return false;
1251 | }
1252 |
1253 | #endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "dynamic.h"`	Технически подключает `dynamic.h`. Логически это дает модулю доступ к: модель динамического отчета и настройки runtime-валидации.
2	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.
3	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
4	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
5	`#include <stdarg.h>`	Технически подключает `stdarg.h`. Логически это дает модулю доступ к: va_list для функций вида appendf/printf.
7	`#ifdef _WIN32`	Технически включает код только при условии `#ifdef _WIN32`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
9	`#include <windows.h>`	Технически подключает `windows.h`. Логически это дает модулю доступ к: WinAPI для динамической проверки под Windows.
10	`#include <tlhelp32.h>`	Технически подключает `tlhelp32.h`. Логически это дает модулю доступ к: перечисление модулей процесса через Toolhelp snapshot.
12	`/* MinGW exposes exception_code as a compatibility macro. */`	Технически это комментарий исходника. Логически он объясняет разработчику: MinGW exposes exception_code as a compatibility macro..
13	`#ifdef exception_code`	Технически включает код только при условии `#ifdef exception_code`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
14	`#undef exception_code`	Технически это директива препроцессора `#undef exception_code`. Логически она меняет код еще до компиляции C.
15	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
17	`/* ---------- */`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.
19	`typedef struct ModInfo {`	Технически начинает описание структуры `ModInfo`. Логически структура собирает связанные поля в одну модель данных проекта.
20	`char name[MAX_PATH];`	Технически объявляет поле `name` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
21	`uint64_t base;`	Технически объявляет поле `base` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
22	`uint64_t size;`	Технически объявляет поле `size` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
23	`} ModInfo;`	Технически завершает typedef и дает типу имя `ModInfo`. Логически дальше код может объявлять переменные этого типа без `struct`.
25	`typedef struct ModList {`	Технически начинает описание структуры `ModList`. Логически структура собирает связанные поля в одну модель данных проекта.
26	`ModInfo mods[128];`	Технически объявляет поле `mods` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
27	`size_t count;`	Технически объявляет поле `count` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
28	`bool truncated;`	Технически объявляет поле `truncated` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
29	`} ModList;`	Технически завершает typedef и дает типу имя `ModList`. Логически дальше код может объявлять переменные этого типа без `struct`.
31	`static void mods_init(ModList* ml) { if (ml) memset(ml, 0, sizeof(*ml)); }`	Технически вызывает `mods_init`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `mods_init` для выполнения части алгоритма.
33	`static const char* basename_lower(const char* path) {`	Технически объявляет/начинает функцию `basename_lower`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
34	`const char* base = path;`	Технически объявляет поле `path` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
35	`for (const char* p = path; *p; p++) {`	Технически начинает цикл `const char* p = path; *p; p++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
36	`if (*p == '\\' || *p == '/') base = p + 1;`	Технически проверяет условие `*p == '\\' || *p == '/'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
37	`}`	Технически завершает область видимости внутри `basename_lower`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
38	`return base;`	Технически возвращает выражение `base`. Логически именно это значение становится результатом функции `basename_lower` для вызывающего кода.
39	`}`	Технически завершает область видимости внутри `basename_lower`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
41	`static void strlower_inplace(char* s) {`	Технически объявляет/начинает функцию `strlower_inplace`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
42	`for (; s && *s; s++) {`	Технически начинает цикл `; s && *s; s++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
43	`if (*s >= 'A' && *s <= 'Z') *s = (char)(*s - 'A' + 'a');`	Технически проверяет условие `*s >= 'A' && *s <= 'Z') *s = (char)(*s - 'A' + 'a';` если оно истинно, выполняется защитная или аналитическая ветка ниже.
44	`}`	Технически завершает область видимости внутри `strlower_inplace`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
45	`}`	Технически завершает область видимости внутри `strlower_inplace`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
47	`static void mods_add(ModList* ml, const char* name, uint64_t base, uint64_t size) {`	Технически объявляет/начинает функцию `mods_add`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
48	`if (!ml || !name) return;`	Технически проверяет условие `!ml || !name`; логически это защита от NULL-указателя и работы с неинициализированными данными.
49	`if (ml->count >= sizeof(ml->mods)/sizeof(ml->mods[0])) {`	Технически проверяет условие `ml->count >= sizeof(ml->mods)/sizeof(ml->mods[0])`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
50	`ml->truncated = true;`	Технически присваивает `ml->truncated` значение `true`. Логически это обновляет `truncated`: хранит промежуточное значение, которое дальше используется в текущей функции.
51	`return;`	Технически выходит из функции `mods_add` без значения. Логически дальнейшие строки функции уже не выполняются.
52	`}`	Технически завершает область видимости внутри `mods_add`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
53	`ModInfo* m = &ml->mods[ml->count++];`	Технически объявляет `m` в функции `mods_add` и сразу задает начальное значение `&ml->mods[ml->count++]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
54	`strncpy(m->name, name, sizeof(m->name)-1);`	Технически вызывает `strncpy`. Логически это действие нужно в функции `mods_add`: вызывает функцию `strncpy` для выполнения части алгоритма.
55	`m->name[sizeof(m->name)-1] = 0;`	Технически присваивает `m->name[sizeof(m->name)-1]` значение `0`. Логически это обновляет `name)-1]`: хранит промежуточное значение, которое дальше используется в текущей функции.
56	`strlower_inplace(m->name);`	Технически вызывает `strlower_inplace`. Логически это действие нужно в функции `mods_add`: вызывает функцию `strlower_inplace` для выполнения части алгоритма.
57	`m->base = base;`	Технически присваивает `m->base` значение `base`. Логически это обновляет `base`: хранит промежуточное значение, которое дальше используется в текущей функции.
58	`m->size = size;`	Технически присваивает `m->size` значение `size`. Логически это обновляет `size`: хранит размер или количество, поэтому используется в проверках границ.
59	`}`	Технически завершает область видимости внутри `mods_add`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
61	`static const ModInfo* mods_find(const ModList* ml, const char* name) {`	Технически объявляет/начинает функцию `mods_find`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
62	`if (!ml || !name) return NULL;`	Технически проверяет условие `!ml || !name`; логически это защита от NULL-указателя и работы с неинициализированными данными.
63	`char tmp[MAX_PATH];`	Технически объявляет `tmp` в функции `mods_find`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
64	`strncpy(tmp, name, sizeof(tmp)-1);`	Технически вызывает `strncpy`. Логически это действие нужно в функции `mods_find`: вызывает функцию `strncpy` для выполнения части алгоритма.
65	`tmp[sizeof(tmp)-1] = 0;`	Технически присваивает `tmp[sizeof(tmp)-1]` значение `0`. Логически это обновляет `tmp[sizeof(tmp)-1]`: хранит промежуточное значение, которое дальше используется в текущей функции.
66	`strlower_inplace(tmp);`	Технически вызывает `strlower_inplace`. Логически это действие нужно в функции `mods_find`: вызывает функцию `strlower_inplace` для выполнения части алгоритма.
67	`for (size_t i = 0; i < ml->count; i++) {`	Технически начинает цикл `size_t i = 0; i < ml->count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
68	`if (strcmp(ml->mods[i].name, tmp) == 0) return &ml->mods[i];`	Технически проверяет условие `strcmp(ml->mods[i].name, tmp) == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
69	`}`	Технически завершает область видимости внутри `mods_find`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
70	`return NULL;`	Технически возвращает выражение `NULL`. Логически именно это значение становится результатом функции `mods_find` для вызывающего кода.
71	`}`	Технически завершает область видимости внутри `mods_find`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
73	`static const ModInfo* mods_find_by_addr(const ModList* ml, uint64_t addr) {`	Технически объявляет/начинает функцию `mods_find_by_addr`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
74	`if (!ml) return NULL;`	Технически проверяет условие `!ml`; логически это защита от NULL-указателя и работы с неинициализированными данными.
75	`for (size_t i = 0; i < ml->count; i++) {`	Технически начинает цикл `size_t i = 0; i < ml->count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
76	`uint64_t b = ml->mods[i].base;`	Технически объявляет `b` в функции `mods_find_by_addr` и сразу задает начальное значение `ml->mods[i].base`. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
77	`uint64_t e = ml->mods[i].size > UINT64_MAX - b`	Технически присваивает `uint64_t e` значение `ml->mods[i].size > UINT64_MAX - b`. Логически это обновляет `uint64_t e`: хранит промежуточное значение, которое дальше используется в текущей функции.
78	`? UINT64_MAX`	Технически эта строка является частью C-конструкции `? UINT64_MAX`. Логически она уточняет структуру блока функции `mods_find_by_addr` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
79	`: b + ml->mods[i].size;`	Технически выполняет оператор `: b + ml->mods[i].size;`. Логически в функции `mods_find_by_addr` он меняет состояние, проверяемые данные или подготовку отчета.
80	`if (addr >= b && addr < e) return &ml->mods[i];`	Технически проверяет условие `addr >= b && addr < e`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
81	`}`	Технически завершает область видимости внутри `mods_find_by_addr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
82	`return NULL;`	Технически возвращает выражение `NULL`. Логически именно это значение становится результатом функции `mods_find_by_addr` для вызывающего кода.
83	`}`	Технически завершает область видимости внутри `mods_find_by_addr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
85	`static bool mods_refresh_toolhelp(ModList* ml, DWORD pid) {`	Технически объявляет/начинает функцию `mods_refresh_toolhelp`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
86	`if (!ml) return false;`	Технически проверяет условие `!ml`; логически это защита от NULL-указателя и работы с неинициализированными данными.
87	`ml->count = 0;`	Технически присваивает `ml->count` значение `0`. Логически это обновляет `count`: хранит размер или количество, поэтому используется в проверках границ.
88	`ml->truncated = false;`	Технически присваивает `ml->truncated` значение `false`. Логически это обновляет `truncated`: хранит промежуточное значение, которое дальше используется в текущей функции.
89	`HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid);`	Технически присваивает `HANDLE snap` значение `CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid)`. Логически это обновляет `HANDLE snap`: хранит промежуточное значение, которое дальше используется в текущей функции.
90	`if (snap == INVALID_HANDLE_VALUE) return false;`	Технически проверяет условие `snap == INVALID_HANDLE_VALUE`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
92	`MODULEENTRY32 me;`	Технически объявляет `me` в функции `mods_refresh_toolhelp`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
93	`memset(&me, 0, sizeof(me));`	Технически вызывает `memset`. Логически это действие нужно в функции `mods_refresh_toolhelp`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
94	`me.dwSize = sizeof(me);`	Технически присваивает `me.dwSize` значение `sizeof(me)`. Логически это обновляет `dwSize`: хранит промежуточное значение, которое дальше используется в текущей функции.
95	`if (!Module32First(snap, &me)) {`	Технически проверяет условие `!Module32First(snap, &me)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
96	`CloseHandle(snap);`	Технически вызывает `CloseHandle`. Логически это действие нужно в функции `mods_refresh_toolhelp`: вызывает функцию `CloseHandle` для выполнения части алгоритма.
97	`return false;`	Технически возвращает `false` из функции `mods_refresh_toolhelp`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
98	`}`	Технически завершает область видимости внутри `mods_refresh_toolhelp`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
99	`do {`	Технически эта строка является частью C-конструкции `do {`. Логически она уточняет структуру блока функции `mods_refresh_toolhelp` и влияет на то, какие данные будут обработаны.
100	`const char* base = basename_lower(me.szModule);`	Технически присваивает `const char* base` значение `basename_lower(me.szModule)`. Логически это обновляет `const char* base`: хранит промежуточное значение, которое дальше используется в текущей функции.
101	`mods_add(ml, base, (uint64_t)(uintptr_t)me.modBaseAddr, (uint64_t)me.modBaseSize);`	Технически вызывает `mods_add`. Логически это действие нужно в функции `mods_refresh_toolhelp`: вызывает функцию `mods_add` для выполнения части алгоритма.
102	`} while (Module32Next(snap, &me));`	Технически вызывает `while`. Логически это действие нужно в функции `mods_refresh_toolhelp`: вызывает функцию `while` для выполнения части алгоритма.
104	`CloseHandle(snap);`	Технически вызывает `CloseHandle`. Логически это действие нужно в функции `mods_refresh_toolhelp`: вызывает функцию `CloseHandle` для выполнения части алгоритма.
105	`return true;`	Технически возвращает `true` из функции `mods_refresh_toolhelp`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
106	`}`	Технически завершает область видимости внутри `mods_refresh_toolhelp`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
108	`static bool mods_refresh_accounted(ModList* ml, DWORD pid, DynamicReport* dr) {`	Технически объявляет/начинает функцию `mods_refresh_accounted`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
109	`if (!mods_refresh_toolhelp(ml, pid)) {`	Технически проверяет условие `!mods_refresh_toolhelp(ml, pid)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
110	`if (dr) {`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
111	`dr->module_refresh_failures++;`	Технически увеличивает `dr->module_refresh_failures` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
112	`dr->instrumentation_complete = false;`	Технически присваивает `dr->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
113	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
114	`return false;`	Технически возвращает `false` из функции `mods_refresh_accounted`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
115	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
116	`if (dr) {`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
117	`if (ml->count > dr->modules_observed_peak) {`	Технически проверяет условие `ml->count > dr->modules_observed_peak`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
118	`dr->modules_observed_peak = (uint32_t)ml->count;`	Технически присваивает `dr->modules_observed_peak` значение `(uint32_t)ml->count`. Логически это обновляет `modules_observed_peak`: хранит промежуточное значение, которое дальше используется в текущей функции.
119	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
120	`if (ml->truncated) {`	Технически проверяет условие `ml->truncated`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
121	`dr->module_list_truncations++;`	Технически увеличивает `dr->module_list_truncations` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
122	`dr->instrumentation_complete = false;`	Технически присваивает `dr->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
123	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
124	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
125	`return true;`	Технически возвращает `true` из функции `mods_refresh_accounted`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
126	`}`	Технически завершает область видимости внутри `mods_refresh_accounted`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
128	`/* ------------------------------------------------------------ */`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.

Стр.	Код	Нормальное объяснение
130	`typedef enum BpKind {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
131	`BP_UNWIND_HANDLER = 1u << 0,`	Технически присваивает `BP_UNWIND_HANDLER` значение `1u << 0,`. Логически это обновляет `BP_UNWIND_HANDLER`: хранит промежуточное значение, которое дальше используется в текущей функции.
132	`BP_API_ADD_VEH = 1u << 1,`	Технически присваивает `BP_API_ADD_VEH` значение `1u << 1,`. Логически это обновляет `BP_API_ADD_VEH`: хранит промежуточное значение, которое дальше используется в текущей функции.
133	`BP_API_ADD_VCH = 1u << 2,`	Технически присваивает `BP_API_ADD_VCH` значение `1u << 2,`. Логически это обновляет `BP_API_ADD_VCH`: хранит промежуточное значение, которое дальше используется в текущей функции.
134	`BP_API_SET_UEF = 1u << 3,`	Технически присваивает `BP_API_SET_UEF` значение `1u << 3,`. Логически это обновляет `BP_API_SET_UEF`: хранит промежуточное значение, которое дальше используется в текущей функции.
135	`BP_API_RTL_ADD_FT = 1u << 4,`	Технически присваивает `BP_API_RTL_ADD_FT` значение `1u << 4,`. Логически это обновляет `BP_API_RTL_ADD_FT`: хранит промежуточное значение, которое дальше используется в текущей функции.
136	`BP_API_RTL_INSTALL_FTCB = 1u << 5,`	Технически присваивает `BP_API_RTL_INSTALL_FTCB` значение `1u << 5,`. Логически это обновляет `BP_API_RTL_INSTALL_FTCB`: хранит промежуточное значение, которое дальше используется в текущей функции.
137	`BP_CUSTOM_VEH_HANDLER = 1u << 6,`	Технически присваивает `BP_CUSTOM_VEH_HANDLER` значение `1u << 6,`. Логически это обновляет `BP_CUSTOM_VEH_HANDLER`: хранит промежуточное значение, которое дальше используется в текущей функции.
138	`BP_CUSTOM_VCH_HANDLER = 1u << 7,`	Технически присваивает `BP_CUSTOM_VCH_HANDLER` значение `1u << 7,`. Логически это обновляет `BP_CUSTOM_VCH_HANDLER`: хранит промежуточное значение, которое дальше используется в текущей функции.
139	`BP_CUSTOM_UEF = 1u << 8`	Технически присваивает `BP_CUSTOM_UEF` значение `1u << 8`. Логически это обновляет `BP_CUSTOM_UEF`: хранит промежуточное значение, которое дальше используется в текущей функции.
140	`} BpKind;`	Технически завершает typedef и дает типу имя `BpKind`. Логически дальше код может объявлять переменные этого типа без `struct`.
142	`typedef struct Breakpoint {`	Технически начинает описание структуры `Breakpoint`. Логически структура собирает связанные поля в одну модель данных проекта.
143	`uint64_t addr;`	Технически объявляет поле `addr` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
144	`uint8_t orig;`	Технически объявляет поле `orig` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
145	`bool armed;`	Технически объявляет поле `armed` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
146	`bool rearm_pending;`	Технически объявляет поле `rearm_pending` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
147	`uint32_t kinds;`	Технически объявляет поле `kinds` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
148	`} Breakpoint;`	Технически завершает typedef и дает типу имя `Breakpoint`. Логически дальше код может объявлять переменные этого типа без `struct`.
150	`typedef struct BpList {`	Технически начинает описание структуры `BpList`. Логически структура собирает связанные поля в одну модель данных проекта.
151	`Breakpoint bps[512];`	Технически объявляет поле `bps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
152	`size_t count;`	Технически объявляет поле `count` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.

Стр.	Код	Нормальное объяснение
153	`} BpList;`	Технически завершает typedef и дает типу имя `BpList`. Логически дальше код может объявлять переменные этого типа без `struct`.
155	`static void bps_init(BpList* bl) { if (bl) memset(bl, 0, sizeof(*bl)); }`	Технически вызывает `bps_init`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bps_init` для выполнения части алгоритма.
157	`static Breakpoint* bps_find(BpList* bl, uint64_t addr) {`	Технически объявляет/начинает функцию `bps_find`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
158	`if (!bl) return NULL;`	Технически проверяет условие `!bl`; логически это защита от NULL-указателя и работы с неинициализированными данными.
159	`for (size_t i = 0; i < bl->count; i++) {`	Технически начинает цикл `size_t i = 0; i < bl->count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
160	`if (bl->bps[i].addr == addr) return &bl->bps[i];`	Технически проверяет условие `bl->bps[i].addr == addr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
161	`}`	Технически завершает область видимости внутри `bps_find`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
162	`return NULL;`	Технически возвращает выражение `NULL`. Логически именно это значение становится результатом функции `bps_find` для вызывающего кода.
163	`}`	Технически завершает область видимости внутри `bps_find`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
165	`static bool write_one_byte(HANDLE hProcess, uint64_t addr, uint8_t val, StrBuf* err) {`	Технически объявляет/начинает функцию `write_one_byte`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
166	`SIZE_T nw = 0;`	Технически присваивает `SIZE_T nw` значение `0`. Логически это обновляет `SIZE_T nw`: хранит промежуточное значение, которое дальше используется в текущей функции.
167	`uint8_t b = val;`	Технически объявляет поле `val` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
168	`if (WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) && nw == 1) {`	Технически проверяет условие `WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) && nw == 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
169	`FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);`	Технически вызывает `FlushInstructionCache`. Логически это действие нужно в функции `write_one_byte`: сбрасывает кэш инструкций после записи breakpoint.
170	`return true;`	Технически возвращает `true` из функции `write_one_byte`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
171	`}`	Технически завершает область видимости внутри `write_one_byte`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
172	`DWORD werr = GetLastError();`	Технически присваивает `DWORD werr` значение `GetLastError()`. Логически это обновляет `DWORD werr`: хранит промежуточное значение, которое дальше используется в текущей функции.
174	`/* Retry with temporary RWX. Some targets map code pages RX. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Retry with temporary RWX. Some targets map code pages RX..
175	`DWORD oldProt = 0;`	Технически объявляет `oldProt` в функции `write_one_byte` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
176	`if (!VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, PAGE_EXECUTE_READWRITE, &oldProt)) {`	Технически проверяет условие `!VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, PAGE_EXECUTE_READWRITE, &oldProt)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
177	`if (err) sb_appendf(err,`	Технически проверяет условие `err`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
178	`"WriteProcessMemory failed at 0x%llx (err=%lu) and VirtualProtectEx failed (err=%lu)\n",`	Технически присваивает `"WriteProcessMemory failed at 0x%llx (err` значение `%lu) and VirtualProtectEx failed (err=%lu)\n",`. Логически это обновляет `"WriteProcessMemory failed at 0x%llx (err`: хранит промежуточное значение, которое дальше используется в текущей функции.
179	`(unsigned long long)addr, (unsigned long)werr, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `write_one_byte`: вызывает функцию `GetLastError` для выполнения части алгоритма.
180	`return false;`	Технически возвращает `false` из функции `write_one_byte`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
181	`}`	Технически завершает область видимости внутри `write_one_byte`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
183	`nw = 0;`	Технически присваивает `nw` значение `0`. Логически это обновляет `nw`: хранит промежуточное значение, которое дальше используется в текущей функции.
184	`if (!WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) || nw != 1) {`	Технически проверяет условие `!WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) || nw != 1`; логически это защита от NULL-указателя и работы с неинициализированными данными.
185	`DWORD werr2 = GetLastError();`	Технически присваивает `DWORD werr2` значение `GetLastError()`. Логически это обновляет `DWORD werr2`: хранит промежуточное значение, которое дальше используется в текущей функции.
186	`DWORD tmp = 0;`	Технически объявляет `tmp` в функции `write_one_byte` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
187	`VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);`	Технически вызывает `VirtualProtectEx`. Логически это действие нужно в функции `write_one_byte`: вызывает функцию `VirtualProtectEx` для выполнения части алгоритма.
188	`if (err) sb_appendf(err,`	Технически проверяет условие `err`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
189	`"WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err=%lu)\n",`	Технически присваивает `"WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err` значение `%lu)\n",`. Логически это обновляет `"WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err`: хранит промежуточное значение, которое дальше используется в текущей функции.
190	`(unsigned long long)addr, (unsigned long)werr2);`	Технически выполняет оператор `(unsigned long long)addr, (unsigned long)werr2);`. Логически в функции `write_one_byte` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
191	`return false;`	Технически возвращает `false` из функции `write_one_byte`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
192	`}`	Технически завершает область видимости внутри `write_one_byte`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
194	`FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);`	Технически вызывает `FlushInstructionCache`. Логически это действие нужно в функции `write_one_byte`: сбрасывает кэш инструкций после записи breakpoint.
195	`DWORD tmp = 0;`	Технически объявляет `tmp` в функции `write_one_byte` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
196	`VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);`	Технически вызывает `VirtualProtectEx`. Логически это действие нужно в функции `write_one_byte`: вызывает функцию `VirtualProtectEx` для выполнения части алгоритма.
197	`return true;`	Технически возвращает `true` из функции `write_one_byte`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
198	`}`	Технически завершает область видимости внутри `write_one_byte`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
200	`static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {`	Технически объявляет/начинает функцию `bp_set`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
201	`SIZE_T nread = 0;`	Технически присваивает `SIZE_T nread` значение `0`. Логически это обновляет `SIZE_T nread`: хранит промежуточное значение, которое дальше используется в текущей функции.
202	`uint8_t b = 0;`	Технически объявляет `b` в функции `bp_set` и сразу задает начальное значение `0`. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
203	`if (!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread != 1) {`	Технически проверяет условие `!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread != 1`; логически это защита от NULL-указателя и работы с неинициализированными данными.
204	`if (err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu)\n",`	Технически проверяет условие `err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
205	`(unsigned long long)bp->addr, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `bp_set`: вызывает функцию `GetLastError` для выполнения части алгоритма.
206	`return false;`	Технически возвращает `false` из функции `bp_set`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
207	`}`	Технически завершает область видимости внутри `bp_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
208	`bp->orig = b;`	Технически объявляет поле `b` внутри структуры. Логически это поле: хранит буфер, с которым работает текущий модуль.
209	`if (!write_one_byte(hProcess, bp->addr, 0xCC, err)) return false;`	Технически проверяет условие `!write_one_byte(hProcess, bp->addr, 0xCC, err)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
210	`bp->armed = true;`	Технически объявляет поле `true` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
211	`bp->rearm_pending = false;`	Технически объявляет поле `false` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
212	`return true;`	Технически возвращает `true` из функции `bp_set`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
213	`}`	Технически завершает область видимости внутри `bp_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
215	`static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {`	Технически объявляет/начинает функцию `bp_clear`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
216	`if (!bp || !bp->armed) return true;`	Технически проверяет условие `!bp || !bp->armed`; логически это защита от NULL-указателя и работы с неинициализированными данными.
217	`if (!write_one_byte(hProcess, bp->addr, bp->orig, err)) return false;`	Технически проверяет условие `!write_one_byte(hProcess, bp->addr, bp->orig, err)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
218	`bp->armed = false;`	Технически присваивает `bp->armed` значение `false`. Логически это обновляет `armed`: хранит промежуточное значение, которое дальше используется в текущей функции.
219	`return true;`	Технически возвращает `true` из функции `bp_clear`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
220	`}`	Технически завершает область видимости внутри `bp_clear`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
222	`static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf* err) {`	Технически объявляет/начинает функцию `bp_add_and_arm`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
223	`if (!bl) return false;`	Технически проверяет условие `!bl`; логически это защита от NULL-указателя и работы с неинициализированными данными.
225	`Breakpoint* ex = bps_find(bl, addr);`	Технически присваивает `Breakpoint* ex` значение `bps_find(bl, addr)`. Логически это обновляет `Breakpoint* ex`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
226	`if (ex) {`	Технически проверяет условие ex; если оно истинно, выполняется защитная или аналитическая ветка ниже.
227	`ex->kinds |= (uint32_t)kind;`	Технически применяет битовую операцию |= к ex->kinds со значением (uint32_t)kind. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
228	`if (ex->armed || ex->rearm_pending) return true;`	Технически проверяет условие ex->armed || ex->rearm_pending; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
229	`return bp_set(hProcess, ex, err);`	Технически объявляет прототип функции bp_set без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
230	`}`	Технически завершает область видимости внутри bp_add_and_arm. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
232	`if (bl->count >= sizeof(bl->bps)/sizeof(bl->bps[0])) return false;`	Технически проверяет условие bl->count >= sizeof(bl->bps)/sizeof(bl->bps[0]); если оно истинно, выполняется защитная или аналитическая ветка ниже.
233	`Breakpoint* bp = &bl->bps[bl->count];`	Технически объявляет bp в функции bp_add_and_arm и сразу задает начальное значение &bl->bps[bl->count]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
234	`memset(bp, 0, sizeof(*bp));`	Технически вызывает memset. Логически это действие нужно в функции bp_add_and_arm: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
235	`bp->addr = addr;`	Технически присваивает bp->addr значение addr. Логически это обновляет addr: хранит промежуточное значение, которое дальше используется в текущей функции.
236	`bp->kinds = (uint32_t)kind;`	Технически присваивает bp->kinds значение (uint32_t)kind. Логически это обновляет kinds: хранит промежуточное значение, которое дальше используется в текущей функции.
238	`if (!bp_set(hProcess, bp, err)) return false;`	Технически проверяет условие !bp_set(hProcess, bp, err); логически это защита от NULL-указателя и работы с неинициализированными данными.
239	`bl->count++;`	Технически увеличивает bl->count на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
240	`return true;`	Технически возвращает true из функции bp_add_and_arm. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
241	`}`	Технически завершает область видимости внутри bp_add_and_arm. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
243	`/* ---------- */`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.
244	`/* Self-check helpers: log where breakpoints were actually placed in the target.`	Технически это комментарий исходника. Логически он объясняет разработчику: Self-check helpers: log where breakpoints were actually placed in the target..
245	`Goal: make it obvious whether a requested breakpoint is (a) placed, (b) placed in the`	Технически вызывает is. Логически это действие нужно в функции глобальной области файла: вызывает функцию is для выполнения части алгоритма.
246	`expected module, and (c) armed in memory (0xCC). */`	Технически вызывает and. Логически это действие нужно в функции глобальной области файла: вызывает функцию and для выполнения части алгоритма.
248	`static const char* bp_kind_name(BpKind k); /* forward */`	Технически объявляет прототип функции bp_kind_name без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
249	`static void dr_append(DynamicReport* dr, const char* fmt, ...); /* forward */`	Технически объявляет прототип функции dr_append без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
251	`static bool bp_peek_byte(HANDLE hProcess, uint64_t addr, uint8_t* outb, StrBuf* err) {`	Технически объявляет/начинает функцию bp_peek_byte; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
252	`if (!outb) return false;`	Технически проверяет условие !outb; логически это защита от NULL-указателя и работы с неинициализированными данными.
253	`SIZE_T nr = 0;`	Технически присваивает SIZE_T nr значение 0. Логически это обновляет SIZE_T nr: хранит промежуточное значение, которое дальше используется в текущей функции.
254	`uint8_t b = 0;`	Технически объявляет b в функции bp_peek_byte и сразу задает начальное значение 0. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
255	`if (ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)addr, &b, 1, &nr) && nr == 1) {`	Технически проверяет условие ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)addr, &b, 1, &nr) && nr == 1; если оно истинно, выполняется защитная или аналитическая ветка ниже.
256	`*outb = b;`	Технически это комментарий исходника. Логически он объясняет разработчику: outb = b;.
257	`return true;`	Технически возвращает true из функции bp_peek_byte. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
258	`}`	Технически завершает область видимости внутри bp_peek_byte. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
259	`if (err) sb_appendf(err, "ReadProcessMemory peek failed at 0x%llx (err=%lu)\n",`	Технически проверяет условие err) sb_appendf(err, "ReadProcessMemory peek failed at 0x%llx (err=%lu; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
260	`(unsigned long long)addr, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `bp_peek_byte`: вызывает функцию `GetLastError` для выполнения части алгоритма.
261	`return false;`	Технически возвращает `false` из функции `bp_peek_byte`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
262	`}`	Технически завершает область видимости внутри `bp_peek_byte`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
264	`static void format_bp_location(const ModList* mods, uint64_t addr, char* out, size_t outsz) {`	Технически объявляет/начинает функцию `format_bp_location`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
265	`if (!out || outsz == 0) return;`	Технически проверяет условие `!out || outsz == 0`; логически это защита от NULL-указателя и работы с неинициализированными данными.
266	`out[0] = 0;`	Технически присваивает `out[0]` значение `0`. Логически это обновляет `out[0]`: хранит промежуточное значение, которое дальше используется в текущей функции.
267	`const ModInfo* mi = mods ? mods_find_by_addr(mods, addr) : NULL;`	Технически присваивает `const ModInfo* mi` значение `mods ? mods_find_by_addr(mods, addr) : NULL`. Логически это обновляет `const ModInfo* mi`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
268	`if (mi) {`	Технически проверяет условие `mi`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
269	`snprintf(out, outsz, "%s+0x%llx (0x%llx)",`	Технически вызывает `snprintf`. Логически это действие нужно в функции `format_bp_location`: формирует строку с ограничением размера буфера.
270	`mi->name,`	Технически эта строка является частью C-конструкции `mi->name,`. Логически она уточняет структуру блока функции `format_bp_location` и влияет на то, какие данные будут обработаны.
271	`(unsigned long long)(addr - mi->base),`	Технически эта строка является частью C-конструкции `(unsigned long long)(addr - mi->base),`. Логически она уточняет структуру блока функции `format_bp_location` и влияет на то, какие данные будут обработаны.
272	`(unsigned long long)addr);`	Технически выполняет оператор `(unsigned long long)addr);`. Логически в функции `format_bp_location` он меняет состояние, проверяемые данные или подготовку отчета.
273	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
274	`snprintf(out, outsz, "0x%llx", (unsigned long long)addr);`	Технически вызывает `snprintf`. Логически это действие нужно в функции `format_bp_location`: формирует строку с ограничением размера буфера.
275	`}`	Технически завершает область видимости внутри `format_bp_location`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
276	`}`	Технически завершает область видимости внутри `format_bp_location`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
278	`/* Place a breakpoint and emit a deterministic self-check line in dr->details.`	Технически это комментарий исходника. Логически он объясняет разработчику: Place a breakpoint and emit a deterministic self-check line in dr->details..
279	`To reduce log spam, this logs only on the first attempt to create the bp at addr. */`	Технически эта строка является частью C-конструкции `To reduce log spam, this logs only on the first attempt to create the bp at addr. */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
280	`static bool bp_add_and_arm_selfcheck(BpList* bl,`	Технически вызывает `bp_add_and_arm_selfcheck`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_add_and_arm_selfcheck` для выполнения части алгоритма.
281	`HANDLE hProcess,`	Технически эта строка является частью C-конструкции `HANDLE hProcess,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
282	`const ModList* mods,`	Технически эта строка является частью C-конструкции `const ModList* mods,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
283	`DynamicReport* dr,`	Технически эта строка является частью C-конструкции `DynamicReport* dr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
284	`uint64_t addr,`	Технически эта строка является частью C-конструкции `uint64_t addr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
285	`BpKind kind,`	Технически эта строка является частью C-конструкции `BpKind kind,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
286	`const char* what,`	Технически эта строка является частью C-конструкции `const char* what,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
287	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
288	`if (!bl) return false;`	Технически проверяет условие `!bl`; логически это защита от NULL-указателя и работы с неинициализированными данными.
290	`Breakpoint* existing = bps_find(bl, addr);`	Технически присваивает `Breakpoint* existing` значение `bps_find(bl, addr)`. Логически это обновляет `Breakpoint* existing`: хранит промежуточное значение, которое дальше используется в текущей функции.
291	`bool first_time = (existing == NULL);`	Технически присваивает `bool first_time` значение `(existing == NULL)`. Логически это обновляет `bool first_time`: хранит промежуточное значение, которое дальше используется в текущей функции.
292	`bool capacity_exhausted =`	Технически присваивает `bool capacity_exhausted` значение `` `. Логически это обновляет bool capacity_exhausted` ``: хранит промежуточное значение, которое дальше используется в текущей функции.
293	`first_time && bl->count >= sizeof(bl->bps) / sizeof(bl->bps[0]);`	Технически вызывает `sizeof`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `sizeof` для выполнения части алгоритма.
294	`if (dr && first_time) dr->breakpoint_set_attempts++;`	Технически проверяет условие `dr && first_time`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
296	`StrBuf te; sb_init(&te, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `глобальной области файла`: создает динамический текстовый буфер.
297	`bool ok = bp_add_and_arm(bl, hProcess, addr, kind, &te);`	Технически присваивает `bool ok` значение `bp_add_and_arm(bl, hProcess, addr, kind, &te)`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
299	`if (dr && !ok) {`	Технически проверяет условие `dr && !ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.
300	`dr->breakpoint_set_failures++;`	Технически увеличивает `dr->breakpoint_set_failures` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
301	`if (capacity_exhausted) dr->breakpoint_capacity_drops++;`	Технически проверяет условие `capacity_exhausted`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
302	`dr->instrumentation_complete = false;`	Технически присваивает `dr->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
303	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
305	`if (dr && first_time) {`	Технически проверяет условие `dr && first_time`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
306	`char loc[192];`	Технически объявляет `loc` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
307	`format_bp_location(mods, addr, loc, sizeof(loc));`	Технически начинает цикл `mods, addr, loc, sizeof(loc)`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
309	`uint8_t cur = 0xFF;`	Технически объявляет `cur` в функции `глобальной области файла` и сразу задает начальное значение `0xFF`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
310	`StrBuf peek_err; sb_init(&peek_err, 128);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `глобальной области файла`: создает динамический текстовый буфер.
311	`bool peek_ok = bp_peek_byte(hProcess, addr, &cur, &peek_err);`	Технически присваивает `bool peek_ok` значение `bp_peek_byte(hProcess, addr, &cur, &peek_err)`. Логически это обновляет `bool peek_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
313	`if (ok) {`	Технически проверяет условие `ok`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
314	`if (peek_ok) {`	Технически проверяет условие `peek_ok`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
315	`dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte=0x%02x)%s\n",`	Технически присваивает `dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte` значение `0x%02x)%s\n",`. Логически это обновляет `dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte`: хранит промежуточное значение, которое дальше используется в текущей функции.
316	`bp_kind_name(kind),`	Технически вызывает `bp_kind_name`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_kind_name` для выполнения части алгоритма.
317	`(what ? what : ""),`	Технически эта строка является частью С-конструкции `(what ? what : ""),`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
318	`loc,`	Технически эта строка является частью С-конструкции `loc,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
319	`(unsigned)cur,`	Технически эта строка является частью С-конструкции `(unsigned)cur,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
320	`(cur == 0xCC ? "" : " [WARN: not INT3]"));`	Технически выполняет оператор `(cur == 0xCC ? "" : " [WARN: not INT3]"));`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
321	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
322	`dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (peek failed: %s)\n",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
323	`bp_kind_name(kind), (what ? what : ""), loc,`	Технически вызывает `bp_kind_name`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_kind_name` для выполнения части алгоритма.
324	`(peek_err.s ? peek_err.s : "unknown"));`	Технически выполняет оператор `(peek_err.s ? peek_err.s : "unknown"));`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
325	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
326	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
327	`dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : FAIL%s%s",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
328	`bp_kind_name(kind),`	Технически вызывает `bp_kind_name`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_kind_name` для выполнения части алгоритма.
329	`(what ? what : ""),`	Технически эта строка является частью C-конструкции `(what ? what : ""),`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
330	`loc,`	Технически эта строка является частью C-конструкции `loc,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
331	`(te.len ? " : " : "\n"),`	Технически эта строка является частью C-конструкции `(te.len ? " : " : "\n"),`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
332	`(te.len ? te.s : "\n"));`	Технически выполняет оператор `(te.len ? te.s : "\n"));`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
333	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
335	`sb_free(&peek_err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
336	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
338	`if (!ok && err && te.len) sb_append(err, te.s);`	Технически проверяет условие `!ok && err && te.len) sb_append(err, te.s;` логически это защита от NULL-указателя и работы с неинициализированными данными.
339	`sb_free(&te);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
340	`return ok;`	Технически возвращает выражение `ok`. Логически именно это значение становится результатом функции `глобальной области файла` для вызывающего кода.
341	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
344	`/* ------------------------------------------------------------ */`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.

Стр.	Код	Нормальное объяснение
346	`/* Pending breakpoint re-arming must be tracked per-thread.`	Технически это комментарий исходника. Логически он объясняет разработчику: Pending breakpoint re-arming must be tracked per-thread..
347	`With multiple threads, a single global pending pointer can cause lost rearms. */`	Технически эта строка является частью C-конструкции `With multiple threads, a single global pending pointer can cause lost rearms. */`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
348	`typedef struct PendingRearmEntry {`	Технически начинает описание структуры `PendingRearmEntry`. Логически структура собирает связанные поля в одну модель данных проекта.
349	`DWORD tid;`	Технически объявляет поле `tid` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
350	`Breakpoint* bp;`	Технически объявляет поле `bp` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
351	`} PendingRearmEntry;`	Технически завершает typedef и дает типу имя `PendingRearmEntry`. Логически дальше код может объявлять переменные этого типа без `struct`.
353	`typedef struct PendingRearmList {`	Технически начинает описание структуры `PendingRearmList`. Логически структура собирает связанные поля в одну модель данных проекта.
354	`PendingRearmEntry e[64];`	Технически объявляет поле `e` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
355	`} PendingRearmList;`	Технически завершает typedef и дает типу имя `PendingRearmList`. Логически дальше код может объявлять переменные этого типа без `struct`.
357	`static void pending_rearm_init(PendingRearmList* pr) {`	Технически объявляет/начинает функцию `pending_rearm_init`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
358	`if (!pr) return;`	Технически проверяет условие `!pr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
359	`memset(pr, 0, sizeof(*pr));`	Технически вызывает `memset`. Логически это действие нужно в функции `pending_rearm_init`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
360	`}`	Технически завершает область видимости внутри `pending_rearm_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
362	`static bool pending_rearm_set(PendingRearmList* pr, DWORD tid, Breakpoint* bp) {`	Технически объявляет/начинает функцию `pending_rearm_set`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
363	`if (!pr || !bp) return false;`	Технически проверяет условие `!pr || !bp`; логически это защита от NULL-указателя и работы с неинициализированными данными.
364	`/* overwrite existing */`	Технически это комментарий исходника. Логически он объясняет разработчику: overwrite existing.
365	`for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
366	`if (pr->e[i].bp && pr->e[i].tid == tid) {`	Технически проверяет условие `pr->e[i].bp && pr->e[i].tid == tid`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
367	`pr->e[i].bp = bp;`	Технически объявляет поле `bp` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
368	`return true;`	Технически возвращает `true` из функции `pending_rearm_set`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
369	`}`	Технически завершает область видимости внутри `pending_rearm_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
370	`}`	Технически завершает область видимости внутри `pending_rearm_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
371	`/* take a free slot */`	Технически это комментарий исходника. Логически он объясняет разработчику: take a free slot.
372	`for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
373	`if (!pr->e[i].bp) {`	Технически проверяет условие `!pr->e[i].bp`; логически это защита от NULL-указателя и работы с неинициализированными данными.
374	`pr->e[i].tid = tid;`	Технически присваивает `pr->e[i].tid` значение `tid`. Логически это обновляет `tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
375	`pr->e[i].bp = bp;`	Технически присваивает `pr->e[i].bp` значение `bp`. Логически это обновляет `bp`: хранит промежуточное значение, которое дальше используется в текущей функции.
376	`return true;`	Технически возвращает `true` из функции `pending_rearm_set`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
377	`}`	Технически завершает область видимости внутри `pending_rearm_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
378	`}`	Технически завершает область видимости внутри `pending_rearm_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
379	`return false;`	Технически возвращает `false` из функции `pending_rearm_set`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
380	`}`	Технически завершает область видимости внутри `pending_rearm_set`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
382	`static Breakpoint* pending_rearm_take(PendingRearmList* pr, DWORD tid) {`	Технически объявляет/начинает функцию `pending_rearm_take`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
383	`if (!pr) return NULL;`	Технически проверяет условие `!pr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
384	`for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
385	`if (pr->e[i].bp && pr->e[i].tid == tid) {`	Технически проверяет условие `pr->e[i].bp && pr->e[i].tid == tid`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
386	`  Breakpoint* bp = pr->e[i].bp;`	Технически объявляет bp в функции pending_rearm_take и сразу задает начальное значение pr->e[i].bp. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
387	`  pr->e[i].bp = NULL;`	Технически присваивает pr->e[i].bp значение NULL. Логически это обновляет bp: хранит промежуточное значение, которое дальше используется в текущей функции.
388	`  pr->e[i].tid = 0;`	Технически присваивает pr->e[i].tid значение 0. Логически это обновляет tid: хранит промежуточное значение, которое дальше используется в текущей функции.
389	`  return bp;`	Технически возвращает выражение bp. Логически именно это значение становится результатом функции pending_rearm_take для вызывающего кода.
390	`  }`	Технически завершает область видимости внутри pending_rearm_take. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
391	`  }`	Технически завершает область видимости внутри pending_rearm_take. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
392	`  return NULL;`	Технически возвращает выражение NULL. Логически именно это значение становится результатом функции pending_rearm_take для вызывающего кода.
393	`}`	Технически завершает область видимости внутри pending_rearm_take. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
395	`static HANDLE open_event_thread(DWORD tid) {`	Технически объявляет/начинает функцию open_event_thread; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
396	`return OpenThread(THREAD_GET_CONTEXT | THREAD_SET_CONTEXT | THREAD_QUERY_INFORMATION, FALSE, tid);`	Технически объявляет прототип функции OpenThread без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
397	`}`	Технически завершает область видимости внутри open_event_thread. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
399	`static void close_event_thread(HANDLE h) {`	Технически объявляет/начинает функцию close_event_thread; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
400	`  if (h) CloseHandle(h);`	Технически проверяет условие h) CloseHandle(h; если оно истинно, выполняется защитная или аналитическая ветка ниже.
401	`}`	Технически завершает область видимости внутри close_event_thread. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
403	`static void dr_append(DynamicReport* dr, const char* fmt, ...) {`	Технически объявляет/начинает функцию dr_append; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
404	`  if (!dr) return;`	Технически проверяет условие !dr; логически это защита от NULL-указателя и работы с неинициализированными данными.
405	`  va_list ap;`	Технически выполняет оператор va_list ap;. Логически в функции dr_append он меняет состояние, проверяемые данные или подготовку отчета.
406	`  va_start(ap, fmt);`	Технически вызывает va_start. Логически это действие нужно в функции dr_append: вызывает функцию va_start для выполнения части алгоритма.
407	`  /* Reuse sb_appendf's variadic through vsnprintf into temp */`	Технически это комментарий исходника. Логически он объясняет разработчику: Reuse sb_appendf's variadic through vsnprintf into temp.
408	`  char tmp[1024];`	Технически объявляет tmp в функции dr_append. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
409	`  int n = vsnprintf(tmp, sizeof(tmp), fmt, ap);`	Технически присваивает int n значение vsnprintf(tmp, sizeof(tmp), fmt, ap). Логически это обновляет int n: хранит промежуточное значение, которое дальше используется в текущей функции.
410	`  va_end(ap);`	Технически вызывает va_end. Логически это действие нужно в функции dr_append: вызывает функцию va_end для выполнения части алгоритма.
411	`  if (n <= 0) return;`	Технически проверяет условие n <= 0; если оно истинно, выполняется защитная или аналитическая ветка ниже.
412	`  tmp[sizeof(tmp)-1] = 0;`	Технически присваивает tmp[sizeof(tmp)-1] значение 0. Логически это обновляет tmp[sizeof(tmp)-1]: хранит промежуточное значение, которое дальше используется в текущей функции.
413	`  sb_append(&dr->details, tmp);`	Технически вызывает sb_append. Логически это действие нужно в функции dr_append: добавляет текст в динамический буфер.
414	`  }`	Технически завершает область видимости внутри dr_append. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
416	`static void dynamic_event_add(DynamicReport* dr,`	Технически вызывает dynamic_event_add. Логически это действие нужно в функции глобальной области файла: вызывает функцию dynamic_event_add для выполнения части алгоритма.
417	`  const ModList* mods,`	Технически эта строка является частью C-конструкции const ModList* mods,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
418	`const char* type,`	Технически эта строка является частью C-конструкции `const char* type,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
419	`DWORD thread_id,`	Технически эта строка является частью C-конструкции `DWORD thread_id,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
420	`uint64_t address,`	Технически эта строка является частью C-конструкции `uint64_t address,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
421	`uint32_t exception_code,`	Технически эта строка является частью C-конструкции `uint32_t exception_code,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
422	`bool first_chance,`	Технически эта строка является частью C-конструкции `bool first_chance,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
423	`uint64_t arg0,`	Технически эта строка является частью C-конструкции `uint64_t arg0,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
424	`uint64_t arg1,`	Технически эта строка является частью C-конструкции `uint64_t arg1,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
425	`uint64_t arg2,`	Технически эта строка является частью C-конструкции `uint64_t arg2,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
426	`uint64_t arg3) {`	Технически эта строка является частью C-конструкции `uint64_t arg3) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
427	`if (!dr || !type) return;`	Технически проверяет условие `!dr || !type`; логически это защита от NULL-указателя и работы с неинициализированными данными.
428	`if (dr->event_count >= DYNAMIC_MAX_EVENTS) {`	Технически проверяет условие `dr->event_count >= DYNAMIC_MAX_EVENTS`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
429	`dr->dropped_events++;`	Технически увеличивает `dr->dropped_events` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
430	`dr->instrumentation_complete = false;`	Технически присваивает `dr->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
431	`return;`	Технически выходит из функции `глобальной области файла` без значения. Логически дальнейшие строки функции уже не выполняются.
432	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
434	`DynamicEvent* event = &dr->events[dr->event_count++];`	Технически присваивает `DynamicEvent* event` значение `&dr->events[dr->event_count++]`. Логически это обновляет `DynamicEvent* event`: хранит промежуточное значение, которое дальше используется в текущей функции.
435	`memset(event, 0, sizeof(*event));`	Технически вызывает `memset`. Логически это действие нужно в функции `глобальной области файла`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
436	`snprintf(event->type, sizeof(event->type), "%s", type);`	Технически вызывает `snprintf`. Логически это действие нужно в функции `глобальной области файла`: формирует строку с ограничением размера буфера.
437	`const ModInfo* module = mods_find_by_addr(mods, address);`	Технически присваивает `const ModInfo* module` значение `mods_find_by_addr(mods, address)`. Логически это обновляет `const ModInfo* module`: хранит промежуточное значение, которое дальше используется в текущей функции.
438	`if (module) snprintf(event->module, sizeof(event->module), "%s", module->name);`	Технически проверяет условие `module) snprintf(event->module, sizeof(event->module), "%s", module->name;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
439	`event->thread_id = (uint32_t)thread_id;`	Технически присваивает `event->thread_id` значение `(uint32_t)thread_id`. Логически это обновляет `thread_id`: хранит промежуточное значение, которое дальше используется в текущей функции.
440	`event->exception_code = exception_code;`	Технически присваивает `event->exception_code` значение `exception_code`. Логически это обновляет `exception_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
441	`event->first_chance = first_chance;`	Технически присваивает `event->first_chance` значение `first_chance`. Логически это обновляет `first_chance`: хранит промежуточное значение, которое дальше используется в текущей функции.
442	`event->address = address;`	Технически присваивает `event->address` значение `address`. Логически это обновляет `address`: хранит промежуточное значение, которое дальше используется в текущей функции.
443	`event->arg0 = arg0;`	Технически присваивает `event->arg0` значение `arg0`. Логически это обновляет `arg0`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
444	`event->arg1 = arg1;`	Технически присваивает `event->arg1` значение `arg1`. Логически это обновляет `arg1`: хранит промежуточное значение, которое дальше используется в текущей функции.
445	`event->arg2 = arg2;`	Технически присваивает `event->arg2` значение `arg2`. Логически это обновляет `arg2`: хранит промежуточное значение, которое дальше используется в текущей функции.
446	`event->arg3 = arg3;`	Технически присваивает `event->arg3` значение `arg3`. Логически это обновляет `arg3`: хранит промежуточное значение, которое дальше используется в текущей функции.
447	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
449	`void dynamic_report_init(DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_report_init`; она доступная другим модулям через заголовок или линковку. Логически функция инициализирует структуру динамического отчета.
450	`if (!dr) return;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
451	`memset(dr, 0, sizeof(*dr));`	Технически вызывает `memset`. Логически это действие нужно в функции `dynamic_report_init`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
452	`dr->enabled = true;`	Технически присваивает `dr->enabled` значение `true`. Логически это обновляет `enabled`: хранит промежуточное значение, которое дальше используется в текущей функции.
453	`dr->instrumentation_complete = true;`	Технически присваивает `dr->instrumentation_complete` значение `true`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
454	`sb_init(&dr->details, 2048);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `dynamic_report_init`: создает динамический текстовый буфер.

Стр.	Код	Нормальное объяснение
455	`}`	Технически завершает область видимости внутри `dynamic_report_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
457	`void dynamic_report_free(DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_report_free`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает память внутри динамического отчета.
458	`if (!dr) return;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
459	`sb_free(&dr->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `dynamic_report_free`: освобождает память динамического буфера.
460	`memset(dr, 0, sizeof(*dr));`	Технически вызывает `memset`. Логически это действие нужно в функции `dynamic_report_free`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
461	`}`	Технически завершает область видимости внутри `dynamic_report_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
463	`static bool append_backslashes(StrBuf* out, size_t count) {`	Технически объявляет/начинает функцию `append_backslashes`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
464	`for (size_t i = 0; i < count; i++) {`	Технически начинает цикл `size_t i = 0; i < count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
465	`if (!sb_appendn(out, "\\", 1)) return false;`	Технически проверяет условие `!sb_appendn(out, "\\", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
466	`}`	Технически завершает область видимости внутри `append_backslashes`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
467	`return true;`	Технически возвращает `true` из функции `append_backslashes`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
468	`}`	Технически завершает область видимости внутри `append_backslashes`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
470	`/* Quote one argument according to the Windows C runtime command-line rules. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Quote one argument according to the Windows C runtime command-line rules..
471	`static bool append_cmd_arg(StrBuf* out, const char* arg) {`	Технически объявляет/начинает функцию `append_cmd_arg`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
472	`if (!out || !arg) return false;`	Технически проверяет условие `!out || !arg`; логически это защита от NULL-указателя и работы с неинициализированными данными.
473	`bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;`	Технически присваивает `bool quote` значение `(*arg == '\0') || strpbrk(arg, " \t\"") != NULL`. Логически это обновляет `bool quote`: хранит промежуточное значение, которое дальше используется в текущей функции.
474	`if (!quote) return sb_append(out, arg);`	Технически проверяет условие `!quote) return sb_append(out, arg`; логически это защита от NULL-указателя и работы с неинициализированными данными.
475	`if (!sb_appendn(out, "\"", 1)) return false;`	Технически проверяет условие `!sb_appendn(out, "\"", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
477	`const char* p = arg;`	Технически объявляет p в функции append_cmd_arg и сразу задает начальное значение arg. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
478	`while (*p) {`	Технически начинает цикл while с условием *p. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
479	`size_t backslashes = 0;`	Технически объявляет backslashes в функции append_cmd_arg и сразу задает начальное значение 0. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
480	`while (*p == '\\') {`	Технически начинает цикл while с условием *p == '\\'. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
481	`backslashes++;`	Технически увеличивает backslashes на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
482	`p++;`	Технически увеличивает p на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
483	`}`	Технически завершает область видимости внутри append_cmd_arg. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
484	`if (*p == '"') {`	Технически проверяет условие *p == '"'; если оно истинно, выполняется защитная или аналитическая ветка ниже.
485	`if (!append_backslashes(out, backslashes * 2u + 1u)) return false;`	Технически проверяет условие !append_backslashes(out, backslashes * 2u + 1u); логически это защита от NULL-указателя и работы с неинициализированными данными.
486	`if (!sb_appendn(out, "\"", 1)) return false;`	Технически проверяет условие !sb_appendn(out, "\"", 1); логически это защита от NULL-указателя и работы с неинициализированными данными.
487	`p++;`	Технически увеличивает p на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
488	`} else if (*p == '\0') {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
489	`if (!append_backslashes(out, backslashes * 2u)) return false;`	Технически проверяет условие !append_backslashes(out, backslashes * 2u); логически это защита от NULL-указателя и работы с неинициализированными данными.
490	`} else {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
491	`if (!append_backslashes(out, backslashes)) return false;`	Технически проверяет условие !append_backslashes(out, backslashes); логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
492	`if (!sb_appendn(out, p, 1)) return false;`	Технически проверяет условие !sb_appendn(out, p, 1); логически это защита от NULL-указателя и работы с неинициализированными данными.
493	`p++;`	Технически увеличивает p на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
494	`}`	Технически завершает область видимости внутри append_cmd_arg. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
495	`}`	Технически завершает область видимости внутри append_cmd_arg. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
496	`return sb_appendn(out, "\"", 1);`	Технически объявляет прототип функции sb_appendn без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет ровно n байтов текста в StrBuf.
497	`}`	Технически завершает область видимости внутри append_cmd_arg. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
499	`/* Build a command line for CreateProcessA while preserving target arguments. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Build a command line for CreateProcessA while preserving target arguments..
500	`static bool build_cmdline(const char* path, const char* const* argv, int argc, StrBuf* out) {`	Технически объявляет/начинает функцию build_cmdline; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
501	`if (!path || !out) return false;`	Технически проверяет условие !path || !out; логически это защита от NULL-указателя и работы с неинициализированными данными.
502	`out->len = 0;`	Технически присваивает out->len значение 0. Логически это обновляет len: хранит размер или количество, поэтому используется в проверках границ.
503	`if (out->s) out->s[0] = 0;`	Технически проверяет условие out->s; если оно истинно, выполняется защитная или аналитическая ветка ниже.
505	`if (!append_cmd_arg(out, path)) return false;`	Технически проверяет условие !append_cmd_arg(out, path); логически это защита от NULL-указателя и работы с неинициализированными данными.
507	`for (int i = 0; i < argc; i++) {`	Технически начинает цикл int i = 0; i < argc; i++. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
508	`const char* a = argv[i];`	Технически объявляет a в функции build_cmdline и сразу задает начальное значение argv[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
509	`if (!a || !sb_appendn(out, " ", 1) || !append_cmd_arg(out, a)) return false;`	Технически проверяет условие `!a || !sb_appendn(out, " ", 1) || !append_cmd_arg(out, a)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
510	`}`	Технически завершает область видимости внутри `build_cmdline`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
511	`return true;`	Технически возвращает `true` из функции `build_cmdline`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
512	`}`	Технически завершает область видимости внутри `build_cmdline`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
514	`/* Resolve a symbol to (real module basename, offset) in *this* process.`	Технически это комментарий исходника. Логически он объясняет разработчику: Resolve a symbol to (real module basename, offset) in *this* process..
515	`Important: many APIs are forwarded (e.g. kernel32 -> kernelbase). */`	Технически вызывает `forwarded`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `forwarded` для выполнения части алгоритма.
516	`typedef struct LocalProcLoc {`	Технически начинает описание структуры `LocalProcLoc`. Логически структура собирает связанные поля в одну модель данных проекта.
517	`char mod_name[MAX_PATH]; /* lower-case basename, e.g. kernelbase.dll */`	Технически объявляет поле `mod_name` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
518	`uint64_t offset;`	Технически объявляет поле `offset` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
519	`} LocalProcLoc;`	Технически завершает typedef и дает типу имя `LocalProcLoc`. Логически дальше код может объявлять переменные этого типа без `struct`.
521	`static bool local_proc_loc(const char* preferred_mod, const char* sym, LocalProcLoc* out) {`	Технически объявляет/начинает функцию `local_proc_loc`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
522	`if (!preferred_mod || !sym || !out) return false;`	Технически проверяет условие `!preferred_mod || !sym || !out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
523	`memset(out, 0, sizeof(*out));`	Технически вызывает `memset`. Логически это действие нужно в функции `local_proc_loc`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
525	`HMODULE hPref = GetModuleHandleA(preferred_mod);`	Технически присваивает `HMODULE hPref` значение `GetModuleHandleA(preferred_mod)`. Логически это обновляет `HMODULE hPref`: хранит промежуточное значение, которое дальше используется в текущей функции.
526	`if (!hPref) return false;`	Технически проверяет условие `!hPref`; логически это защита от NULL-указателя и работы с неинициализированными данными.
528	`FARPROC p = GetProcAddress(hPref, sym);`	Технически присваивает `FARPROC p` значение `GetProcAddress(hPref, sym)`. Логически это обновляет `FARPROC p`: хранит промежуточное значение, которое дальше используется в текущей функции.
529	`if (!p) return false;`	Технически проверяет условие `!p`; логически это защита от NULL-указателя и работы с неинициализированными данными.
531	`HMODULE hReal = NULL;`	Технически объявляет поле `NULL` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
532	`#if defined(__GNUC__)`	Технически включает код только при условии `#if defined(__GNUC__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.

Стр.	Код	Нормальное объяснение
533	`#pragma GCC diagnostic push`	Технически это директива препроцессора `#pragma GCC diagnostic push`. Логически она меняет код еще до компиляции C.
534	`#pragma GCC diagnostic ignored "-Wpedantic"`	Технически это директива препроцессора `#pragma GCC diagnostic ignored "-Wpedantic"`. Логически она меняет код еще до компиляции C.
535	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
536	`if (!GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS | GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,`	Технически проверяет условие `if (!GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS | GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,`; логически это защита от NULL-указателя и работы с неинициализированными данными.
537	`(LPCSTR)p, &hReal) || !hReal) {`	Технически выполняет битовую операцию `(LPCSTR)p, &hReal) || !hReal) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
538	`hReal = hPref;`	Технически присваивает `hReal` значение `hPref`. Логически это обновляет `hReal`: хранит промежуточное значение, которое дальше используется в текущей функции.
539	`}`	Технически завершает область видимости внутри `local_proc_loc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
540	`#if defined(__GNUC__)`	Технически включает код только при условии `#if defined(__GNUC__)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
541	`#pragma GCC diagnostic pop`	Технически это директива препроцессора `#pragma GCC diagnostic pop`. Логически она меняет код еще до компиляции C.
542	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Стр.	Код	Нормальное объяснение
544	`char path[MAX_PATH];`	Технически объявляет `path` в функции `local_proc_loc`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
545	`memset(path, 0, sizeof(path));`	Технически вызывает `memset`. Логически это действие нужно в функции `local_proc_loc`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
546	`if (!GetModuleFileNameA(hReal, path, (DWORD)sizeof(path))) {`	Технически проверяет условие `!GetModuleFileNameA(hReal, path, (DWORD)sizeof(path))`; логически это защита от NULL-указателя и работы с неинициализированными данными.
547	`strncpy(out->mod_name, preferred_mod, sizeof(out->mod_name)-1);`	Технически вызывает `strncpy`. Логически это действие нужно в функции `local_proc_loc`: вызывает функцию `strncpy` для выполнения части алгоритма.
548	`out->mod_name[sizeof(out->mod_name)-1] = 0;`	Технически присваивает `out->mod_name[sizeof(out->mod_name)-1]` значение `0`. Логически это обновляет `mod_name)-1]`: хранит промежуточное значение, которое дальше используется в текущей функции.
549	`strlower_inplace(out->mod_name);`	Технически вызывает `strlower_inplace`. Логически это действие нужно в функции `local_proc_loc`: вызывает функцию `strlower_inplace` для выполнения части алгоритма.
550	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
551	`const char* base = basename_lower(path);`	Технически присваивает `const char* base` значение `basename_lower(path)`. Логически это обновляет `const char* base`: хранит промежуточное значение, которое дальше используется в текущей функции.
552	`strncpy(out->mod_name, base, sizeof(out->mod_name)-1);`	Технически вызывает `strncpy`. Логически это действие нужно в функции `local_proc_loc`: вызывает функцию `strncpy` для выполнения части алгоритма.
553	`out->mod_name[sizeof(out->mod_name)-1] = 0;`	Технически присваивает `out->mod_name[sizeof(out->mod_name)-1]` значение `0`. Логически это обновляет `mod_name)-1]`: хранит промежуточное значение, которое дальше используется в текущей функции.
554	`strlower_inplace(out->mod_name);`	Технически вызывает `strlower_inplace`. Логически это действие нужно в функции `local_proc_loc`: вызывает функцию `strlower_inplace` для выполнения части алгоритма.
555	`}`	Технически завершает область видимости внутри `local_proc_loc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
557	`out->offset = (uint64_t)(uintptr_t)p - (uint64_t)(uintptr_t)hReal;`	Технически присваивает `out->offset` значение `(uint64_t)(uintptr_t)p - (uint64_t)(uintptr_t)hReal`. Логически это обновляет `offset`: хранит промежуточное значение, которое дальше используется в текущей функции.
558	`return true;`	Технически возвращает `true` из функции `local_proc_loc`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
559	`}`	Технически завершает область видимости внутри `local_proc_loc`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
561	`static bool add_api_bps_if_ready(BpList* bl, HANDLE hProcess, const ModList* mods, DynamicReport* dr, StrBuf* err) {`	Технически объявляет/начинает функцию `add_api_bps_if_ready`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
562	`if (!bl || !mods) return false;`	Технически проверяет условие `!bl || !mods`; логически это защита от NULL-указателя и работы с неинициализированными данными.
564	`const struct {`	Технически эта строка является частью C-конструкции `const struct {`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.
565	`const char* preferred_mod;`	Технически объявляет поле `preferred_mod` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
566	`const char* sym;`	Технически объявляет поле `sym` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
567	`BpKind kind;`	Технически объявляет поле `kind` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
568	`} apis[] = {`	Технически присваивает `} apis[]` значение `{`. Логически это обновляет `} apis[]`: хранит промежуточное значение, которое дальше используется в текущей функции.
569	`{"kernel32.dll", "AddVectoredExceptionHandler", BP_API_ADD_VEH},`	Технически эта строка является частью C-конструкции `{"kernel32.dll", "AddVectoredExceptionHandler", BP_API_ADD_VEH},`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
570	`{"kernel32.dll", "AddVectoredContinueHandler", BP_API_ADD_VCH},`	Технически эта строка является частью C-конструкции `{"kernel32.dll", "AddVectoredContinueHandler", BP_API_ADD_VCH},`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.
571	`{"kernel32.dll", "SetUnhandledExceptionFilter", BP_API_SET_UEF},`	Технически эта строка является частью C-конструкции `{"kernel32.dll", "SetUnhandledExceptionFilter", BP_API_SET_UEF},`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.
572	`{"ntdll.dll", "RtlAddFunctionTable", BP_API_RTL_ADD_FT},`	Технически эта строка является частью C-конструкции `{"ntdll.dll", "RtlAddFunctionTable", BP_API_RTL_ADD_FT},`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.
573	`{"ntdll.dll", "RtlInstallFunctionTableCallback", BP_API_RTL_INSTALL_FTCB},`	Технически эта строка является частью C-конструкции `{"ntdll.dll", "RtlInstallFunctionTableCallback", BP_API_RTL_INSTALL_FTCB},`. Логически она уточняет структуру блока функции `add_api_bps_if_ready` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
574	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
576	`for (size_t i = 0; i < sizeof(apis)/sizeof(apis[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(apis)/sizeof(apis[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
577	`LocalProcLoc loc;`	Технически объявляет поле `loc` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
578	`if (!local_proc_loc(apis[i].preferred_mod, apis[i].sym, &loc)) continue;`	Технически проверяет условие `!local_proc_loc(apis[i].preferred_mod, apis[i].sym, &loc)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
580	`const ModInfo* m = mods_find(mods, loc.mod_name);`	Технически присваивает `const ModInfo* m` значение `mods_find(mods, loc.mod_name)`. Логически это обновляет `const ModInfo* m`: хранит промежуточное значение, которое дальше используется в текущей функции.
581	`if (!m) {`	Технически проверяет условие `!m`; логически это защита от NULL-указателя и работы с неинициализированными данными.
582	`if (dr) dr_append(dr, "[Dynamic] API %s resolved to %s but module not present in target yet\n",`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
583	`apis[i].sym, loc.mod_name);`	Технически выполняет оператор `apis[i].sym, loc.mod_name);`. Логически в функции `add_api_bps_if_ready` он меняет состояние, проверяемые данные или подготовку отчета.
584	`continue;`	Технически переходит к следующей итерации цикла. Логически текущая запись уже обработана или не подходит для дальнейших проверок.
585	`}`	Технически завершает область видимости внутри `add_api_bps_if_ready`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
587	`uint64_t bp_addr = m->base + loc.offset;`	Технически объявляет `bp_addr` в функции `add_api_bps_if_ready` и сразу задает начальное значение `m->base + loc.offset`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
588	`{`	Технически открывает тело блока после строки `uint64_t bp_addr = m->base + loc.offset;`. Логически все следующие операции относятся к этому условию, циклу, структуре или функции `add_api_bps_if_ready`.
589	`char what[320];`	Технически объявляет `what` в функции `add_api_bps_if_ready`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
590	`snprintf(what, sizeof(what), "%s!%s", loc.mod_name, apis[i].sym);`	Технически вызывает `snprintf`. Логически это действие нужно в функции `add_api_bps_if_ready`: формирует строку с ограничением размера буфера.
591	`if (!bp_add_and_arm_selfcheck(bl, hProcess, mods, dr, bp_addr, apis[i].kind, what, err)) {`	Технически проверяет условие `!bp_add_and_arm_selfcheck(bl, hProcess, mods, dr, bp_addr, apis[i].kind, what, err)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
592	`if (dr) dr_append(dr, "[Dynamic] failed to set API breakpoint %s at 0x%llx\n",`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
593	`what, (unsigned long long)bp_addr);`	Технически выполняет оператор `what, (unsigned long long)bp_addr);`. Логически в функции `add_api_bps_if_ready` он меняет состояние, проверяемые данные или подготовку отчета.
594	`}`	Технически завершает область видимости внутри `add_api_bps_if_ready`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
595	`}`	Технически завершает область видимости внутри `add_api_bps_if_ready`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
596	`}`	Технически завершает область видимости внутри `add_api_bps_if_ready`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
598	`return true;`	Технически возвращает `true` из функции `add_api_bps_if_ready`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
599	`}`	Технически завершает область видимости внутри `add_api_bps_if_ready`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
602	`static const char* bp_kind_name(BpKind k) {`	Технически объявляет/начинает функцию `bp_kind_name`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
603	`switch (k) {`	Технически выбирает ветку по значению `k`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.
604	`case BP_UNWIND_HANDLER: return "unwind_handler";`	Технически объявляет ветку `switch` для значения `BP_UNWIND_HANDLER: return "unwind_handler";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
605	`case BP_API_ADD_VEH: return "api_add_veh";`	Технически объявляет ветку `switch` для значения `BP_API_ADD_VEH: return "api_add_veh";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
606	`case BP_API_ADD_VCH: return "api_add_vch";`	Технически объявляет ветку `switch` для значения `BP_API_ADD_VCH: return "api_add_vch";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.

Стр.	Код	Нормальное объяснение
607	`case BP_API_SET_UEF: return "api_set_uef";`	Технически объявляет ветку switch для значения `BP_API_SET_UEF: return "api_set_uef";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
608	`case BP_API_RTL_ADD_FT: return "api_rtl_add_ft";`	Технически объявляет ветку switch для значения `BP_API_RTL_ADD_FT: return "api_rtl_add_ft";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.

Стр.	Код	Нормальное объяснение
609	`case BP_API_RTL_INSTALL_FTCB: return "api_rtl_install_ftcb";`	Технически объявляет ветку switch для значения `BP_API_RTL_INSTALL_FTCB: return "api_rtl_install_ftcb";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
610	`case BP_CUSTOM_VEH_HANDLER: return "veh_handler";`	Технически объявляет ветку switch для значения `BP_CUSTOM_VEH_HANDLER: return "veh_handler";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
611	`case BP_CUSTOM_VCH_HANDLER: return "vch_handler";`	Технически объявляет ветку switch для значения `BP_CUSTOM_VCH_HANDLER: return "vch_handler";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
612	`case BP_CUSTOM_UEF: return "uef_handler";`	Технически объявляет ветку switch для значения `BP_CUSTOM_UEF: return "uef_handler";`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
613	`default: return "bp";`	Технически объявляет ветку switch по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
614	`}`	Технически завершает область видимости внутри `bp_kind_name`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
615	`}`	Технически завершает область видимости внутри `bp_kind_name`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
617	`static void add_trace(DynamicReport* dr, uint64_t rip) {`	Технически объявляет/начинает функцию `add_trace`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
618	`if (!dr) return;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
619	`if (dr->trace_count >= (uint32_t)(sizeof(dr->trace)/sizeof(dr->trace[0]))) return;`	Технически проверяет условие `dr->trace_count >= (uint32_t)(sizeof(dr->trace)/sizeof(dr->trace[0]))`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
620	`dr->trace[dr->trace_count++] = rip;`	Технически присваивает `dr->trace[dr->trace_count++]` значение `rip`. Логически это обновляет `trace_count++]`: хранит промежуточное значение, которое дальше используется в текущей функции.
621	`}`	Технически завершает область видимости внутри `add_trace`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
623	`static void start_trace(DynamicReport* dr,`	Технически вызывает `start_trace`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `start_trace` для выполнения части алгоритма.
624	`uint32_t trace_steps,`	Технически эта строка является частью C-конструкции `uint32_t trace_steps,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
625	`DWORD tid,`	Технически эта строка является частью C-конструкции `DWORD tid,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
626	`uint64_t rip,`	Технически эта строка является частью C-конструкции `uint64_t rip,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
627	`bool* tracing,`	Технически эта строка является частью C-конструкции `bool* tracing,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
628	`DWORD* trace_tid,`	Технически эта строка является частью C-конструкции `DWORD* trace_tid,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
629	`uint32_t* trace_left) {`	Технически эта строка является частью C-конструкции `uint32_t* trace_left) {`. Логически она уточняет структуру блока функции `start_trace` и влияет на то, какие данные будут обработаны.
630	`if (!dr || !tracing || !trace_tid || !trace_left ||`	Технически проверяет условие `if (!dr || !tracing || !trace_tid || !trace_left ||`; логически это защита от NULL-указателя и работы с неинициализированными данными.
631	`trace_steps == 0 || dr->trace_count != 0) {`	Технически выполняет битовую операцию `trace_steps == 0 || dr->trace_count != 0) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
632	`return;`	Технически выходит из функции `start_trace` без значения. Логически дальнейшие строки функции уже не выполняются.
633	`}`	Технически завершает область видимости внутри `start_trace`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
635	`uint32_t budget = trace_steps > 256u ? 256u : trace_steps;`	Технически объявляет budget в функции start_trace и сразу задает начальное значение trace_steps > 256u ? 256u : trace_steps. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
636	`add_trace(dr, rip);`	Технически вызывает add_trace. Логически это действие нужно в функции start_trace: вызывает функцию add_trace для выполнения части алгоритма.
637	`*trace_left = budget - 1u;`	Технически это комментарий исходника. Логически он объясняет разработчику: trace_left = budget - 1u;.
638	`*tracing = *trace_left > 0;`	Технически это комментарий исходника. Логически он объясняет разработчику: tracing = *trace_left > 0;.
639	`*trace_tid = *tracing ? tid : 0;`	Технически это комментарий исходника. Логически он объясняет разработчику: trace_tid = *tracing ? tid : 0;.
640	`}`	Технически завершает область видимости внутри start_trace. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
642	`static void addr_to_loc(const PeImage* img, uint64_t base, uint64_t addr, char* out, size_t outsz) {`	Технически объявляет/начинает функцию addr_to_loc; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
643	`if (!out || outsz == 0) return;`	Технически проверяет условие !out || outsz == 0; логически это защита от NULL-указателя и работы с неинициализированными данными.
644	`out[0] = 0;`	Технически присваивает out[0] значение 0. Логически это обновляет out[0]: хранит промежуточное значение, которое дальше используется в текущей функции.
645	`if (!img || addr < base) {`	Технически проверяет условие !img || addr < base; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
646	`snprintf(out, outsz, "0x%llx", (unsigned long long)addr);`	Технически вызывает snprintf. Логически это действие нужно в функции addr_to_loc: формирует строку с ограничением размера буфера.
647	`return;`	Технически выходит из функции addr_to_loc без значения. Логически дальнейшие строки функции уже не выполняются.
648	`}`	Технически завершает область видимости внутри addr_to_loc. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
649	`uint32_t rva = (uint32_t)(addr - base);`	Технически присваивает uint32_t rva значение (uint32_t)(addr - base). Логически это обновляет uint32_t rva: хранит промежуточное значение, которое дальше используется в текущей функции.
650	`const PeSection* s = pe_find_section_for_rva(img, rva);`	Технически присваивает const PeSection* s значение pe_find_section_for_rva(img, rva). Логически это обновляет const PeSection* s: хранит промежуточное значение, которое дальше используется в текущей функции.
651	`if (s) snprintf(out, outsz, "0x%llx (rva=0x%08x sec=%s)", (unsigned long long)addr, (unsigned)rva, s->name);`	Технически проверяет условие s) snprintf(out, outsz, "0x%llx (rva=0x%08x sec=%s)", (unsigned long long)addr, (unsigned)rva, s->name; если оно истинно, выполняется защитная или аналитическая ветка ниже.
652	`else snprintf(out, outsz, "0x%llx (rva=0x%08x)", (unsigned long long)addr, (unsigned)rva);`	Технически объявляет прототип функции snprintf без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
653	`}`	Технически завершает область видимости внутри addr_to_loc. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
655	`/* ------------------------------------------------------------ */`	Технически это комментарий исходника. Логически он объясняет разработчику: ----------.
657	`bool dynamic_collect_windows(const char* pe_path,`	Технически вызывает dynamic_collect_windows. Логически это действие нужно в функции глобальной области файла: вызывает функцию dynamic_collect_windows для выполнения части алгоритма.
658	`const char* const* child_argv,`	Технически эта строка является частью C-конструкции const char* const* child_argv,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
659	`int child_argc,`	Технически эта строка является частью C-конструкции int child_argc,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
660	`const PeImage* img,`	Технически эта строка является частью C-конструкции const PeImage* img,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
661	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции const RuntimeFunction* rf,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
662	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции const UnwindSummary* us,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
663	`const bool* us_ok,`	Технически эта строка является частью C-конструкции const bool* us_ok,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
664	`size_t rf_count,`	Технически эта строка является частью С-конструкции `size_t rf_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
665	`uint32_t timeout_ms,`	Технически эта строка является частью С-конструкции `uint32_t timeout_ms,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
666	`uint32_t trace_steps,`	Технически эта строка является частью С-конструкции `uint32_t trace_steps,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
667	`DynamicReport* out,`	Технически эта строка является частью С-конструкции `DynamicReport* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
668	`StrBuf* err) {`	Технически эта строка является частью С-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
669	`if (!out || !err || !pe_path || !img) return false;`	Технически проверяет условие `!out || !err || !pe_path || !img`; логически это защита от NULL-указателя и работы с неинициализированными данными.
670	`if (rf_count > 0 && (!rf || !us || !us_ok)) return false;`	Технически проверяет условие `rf_count > 0 && (!rf || !us || !us_ok)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
672	`dynamic_report_init(out);`	Технически вызывает `dynamic_report_init`. Логически это действие нужно в функции `глобальной области файла`: инициализирует структуру динамического отчета.
673	`if (!sb_ok(&out->details)) {`	Технически проверяет условие `!sb_ok(&out->details)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
674	`sb_append(err, "Out of memory (dynamic report buffer).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
675	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
676	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
677	`out->enabled = true;`	Технически присваивает `out->enabled` значение `true`. Логически это обновляет `enabled`: хранит промежуточное значение, которое дальше используется в текущей функции.
678	`out->timeout_ms = timeout_ms;`	Технически присваивает `out->timeout_ms` значение `timeout_ms`. Логически это обновляет `timeout_ms`: хранит промежуточное значение, которое дальше используется в текущей функции.
679	`out->trace_steps = trace_steps > 256u ? 256u : trace_steps;`	Технически присваивает `out->trace_steps` значение `trace_steps > 256u ? 256u : trace_steps`. Логически это обновляет `trace_steps`: хранит промежуточное значение, которое дальше используется в текущей функции.
681	`StrBuf cmd;`	Технически объявляет `cmd` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
682	`if (!sb_init(&cmd, 1024)) {`	Технически проверяет условие `!sb_init(&cmd, 1024)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
683	`sb_append(err, "Out of memory (command line buffer).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
684	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
685	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
686	`if (!build_cmdline(pe_path, child_argv, child_argc, &cmd)) {`	Технически проверяет условие `!build_cmdline(pe_path, child_argv, child_argc, &cmd)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
687	`sb_append(err, "Failed to build command line.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
688	`sb_free(&cmd);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
689	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
690	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
692	`STARTUPINFOA si;`	Технически объявляет `si` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
693	`PROCESS_INFORMATION pi;`	Технически объявляет `pi` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
694	`memset(&si, 0, sizeof(si));`	Технически вызывает `memset`. Логически это действие нужно в функции `глобальной области файла`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.

Стр.	Код	Нормальное объяснение
695	`memset(&pi, 0, sizeof(pi));`	Технически вызывает memset. Логически это действие нужно в функции глобальной области файла: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
696	`si.cb = sizeof(si);`	Технически присваивает si.cb значение sizeof(si). Логически это обновляет cb: хранит промежуточное значение, которое дальше используется в текущей функции.
698	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
699	`* Do not use CREATE_SUSPENDED.`	Технически это комментарий исходника. Логически он объясняет разработчику: Do not use CREATE_SUSPENDED..
700	`*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
701	`* We want the standard Windows Debug API event stream to start`	Технически это комментарий исходника. Логически он объясняет разработчику: We want the standard Windows Debug API event stream to start.
702	`* immediately after CreateProcess. On some configurations,`	Технически это комментарий исходника. Логически он объясняет разработчику: immediately after CreateProcess. On some configurations,.
703	`* creating the debuggee suspended can result in no debug events`	Технически это комментарий исходника. Логически он объясняет разработчику: creating the debuggee suspended can result in no debug events.
704	`* being delivered (no CREATE_PROCESS_DEBUG_EVENT), which causes`	Технически это комментарий исходника. Логически он объясняет разработчику: being delivered (no CREATE_PROCESS_DEBUG_EVENT), which causes.
705	`* a deadlock (the tool waits forever, never resumes the thread).`	Технически это комментарий исходника. Логически он объясняет разработчику: a deadlock (the tool waits forever, never resumes the thread)..
706	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
707	`DWORD flags = DEBUG_ONLY_THIS_PROCESS;`	Технически объявляет flags в функции глобальной области файла и сразу задает начальное значение DEBUG_ONLY_THIS_PROCESS. Логически это место для данных: флаги UNWIND_INFO, включая наличие exception/termination handler.
709	`if (!CreateProcessA(NULL, cmd.s, NULL, NULL, FALSE, flags, NULL, NULL, &si, &pi)) {`	Технически проверяет условие !CreateProcessA(NULL, cmd.s, NULL, NULL, FALSE, flags, NULL, NULL, &si, &pi); логически это защита от NULL-указателя и работы с неинициализированными данными.
710	`sb_appendf(err, "CreateProcess failed (err=%lu)\n", GetLastError());`	Технически присваивает sb_appendf(err, "CreateProcess failed (err значение %lu)\n", GetLastError()). Логически это обновляет sb_appendf(err, "CreateProcess failed (err: хранит промежуточное значение, которое дальше используется в текущей функции.
711	`sb_free(&cmd);`	Технически вызывает sb_free. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
712	`return false;`	Технически возвращает false из функции глобальной области файла. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
713	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
715	`out->ran = true;`	Технически присваивает out->ran значение true. Логически это обновляет ran: хранит промежуточное значение, которое дальше используется в текущей функции.
717	`HANDLE hProcess = pi.hProcess;`	Технически объявляет hProcess в функции глобальной области файла и сразу задает начальное значение pi.hProcess. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
718	`DWORD pid = pi.dwProcessId;`	Технически объявляет pid в функции глобальной области файла и сразу задает начальное значение pi.dwProcessId. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
720	`ModList mods;`	Технически объявляет mods в функции глобальной области файла. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
721	`mods_init(&mods);`	Технически вызывает mods_init. Логически это действие нужно в функции глобальной области файла: вызывает функцию mods_init для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
723	`BpList bps;`	Технически объявляет bps в функции глобальной области файла. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
724	`bps_init(&bps);`	Технически вызывает bps_init. Логически это действие нужно в функции глобальной области файла: вызывает функцию bps_init для выполнения части алгоритма.
726	`/* Collect unique handler RVAs from static parse. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Collect unique handler RVAs from static parse..
727	`uint32_t* handler_rvas = (uint32_t*)calloc(rf_count ? rf_count : 1, sizeof(uint32_t));`	Технически присваивает uint32_t* handler_rvas значение (uint32_t*)calloc(rf_count ? rf_count : 1, sizeof(uint32_t)). Логически это обновляет uint32_t* handler_rvas: хранит промежуточное значение, которое дальше используется в текущей функции.
728	`if (!handler_rvas) {`	Технически проверяет условие !handler_rvas; логически это защита от NULL-указателя и работы с неинициализированными данными.
729	`sb_append(err, "Out of memory (handler RVA list).\n");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
730	`TerminateProcess(hProcess, 0);`	Технически вызывает TerminateProcess. Логически это действие нужно в функции глобальной области файла: вызывает функцию TerminateProcess для выполнения части алгоритма.
731	`if (pi.hThread) CloseHandle(pi.hThread);`	Технически проверяет условие pi.hThread) CloseHandle(pi.hThread; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
732	`if (pi.hProcess) CloseHandle(pi.hProcess);`	Технически проверяет условие `pi.hProcess) CloseHandle(pi.hProcess`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
733	`sb_free(&cmd);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
734	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
735	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
736	`size_t handler_count = 0;`	Технически объявляет `handler_count` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
737	`for (size_t i = 0; i < rf_count; i++) {`	Технически начинает цикл `size_t i = 0; i < rf_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
738	`if (!us_ok[i] || !us[i].has_handler) continue;`	Технически проверяет условие `!us_ok[i] || !us[i].has_handler`; логически это защита от NULL-указателя и работы с неинициализированными данными.
739	`uint32_t hr = us[i].handler_rva;`	Технически объявляет `hr` в функции `глобальной области файла` и сразу задает начальное значение `us[i].handler_rva`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
740	`bool seen = false;`	Технически объявляет `seen` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
741	`for (size_t j = 0; j < handler_count; j++) {`	Технически начинает цикл `size_t j = 0; j < handler_count; j++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
742	`if (handler_rvas[j] == hr) { seen = true; break; }`	Технически проверяет условие `handler_rvas[j] == hr`; логически отделяет функции с language-specific handler от обычных функций.
743	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
744	`if (!seen) handler_rvas[handler_count++] = hr;`	Технически проверяет условие `!seen`; логически это защита от NULL-указателя и работы с неинициализированными данными.
745	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
746	`out->unwind_handlers_discovered = (uint32_t)handler_count;`	Технически присваивает `out->unwind_handlers_discovered` значение `(uint32_t)handler_count`. Логически это обновляет `unwind_handlers_discovered`: хранит промежуточное значение, которое дальше используется в текущей функции.
748	`DEBUG_EVENT ev;`	Технически объявляет `ev` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
749	`bool got_create = false;`	Технически объявляет `got_create` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
750	`bool debug_loop_ok = true;`	Технически объявляет `debug_loop_ok` в функции `глобальной области файла` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
751	`bool process_exited = false;`	Технически объявляет `process_exited` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
752	`bool timed_out = false;`	Технически объявляет `timed_out` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: является выходным параметром, куда функция записывает результат.
753	`uint64_t base_main = 0;`	Технически объявляет `base_main` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
755	`bool initial_break_ignored = false;`	Технически объявляет `initial_break_ignored` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
757	`uint64_t start = GetTickCount64();`	Технически присваивает `uint64_t start` значение `GetTickCount64()`. Логически это обновляет `uint64_t start`: хранит промежуточное значение, которое дальше используется в текущей функции.
759	`PendingRearmList pending;`	Технически объявляет `pending` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
760	`pending_rearm_init(&pending);`	Технически вызывает `pending_rearm_init`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `pending_rearm_init` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
761	`bool tracing = false;`	Технически объявляет `tracing` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
762	`uint32_t trace_left = 0;`	Технически объявляет `trace_left` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
763	`DWORD trace_tid = 0;`	Технически объявляет `trace_tid` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
765	`/* Let debug loop run */`	Технически это комментарий исходника. Логически он объясняет разработчику: Let debug loop run.
766	`for (;;) {`	Технически начинает цикл `;;`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
767	`uint64_t elapsed = GetTickCount64() - start;`	Технически присваивает `uint64_t elapsed` значение `GetTickCount64() - start`. Логически это обновляет `uint64_t elapsed`: хранит промежуточное значение, которое дальше используется в текущей функции.
768	`if (elapsed > timeout_ms) {`	Технически проверяет условие `elapsed > timeout_ms`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
769	`dr_append(out, "[Dynamic] timeout reached (%u ms)\n", timeout_ms);`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
770	`dynamic_event_add(out, &mods, "timeout", 0, 0, 0, false, timeout_ms, 0, 0, 0);`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
771	`timed_out = true;`	Технически присваивает `timed_out` значение `true`. Логически это обновляет `timed_out`: является выходным параметром, куда функция записывает результат.
772	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
773	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
775	`if (!WaitForDebugEvent(&ev, 100)) {`	Технически проверяет условие `!WaitForDebugEvent(&ev, 100)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
776	`DWORD werr = GetLastError();`	Технически присваивает `DWORD werr` значение `GetLastError()`. Логически это обновляет `DWORD werr`: хранит промежуточное значение, которое дальше используется в текущей функции.
777	`if (werr == ERROR_SEM_TIMEOUT) {`	Технически проверяет условие `werr == ERROR_SEM_TIMEOUT`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
778	`continue;`	Технически переходит к следующей итерации цикла. Логически текущая запись уже обработана или не подходит для дальнейших проверок.
779	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
780	`dr_append(out, "[Dynamic] WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);`	Технически присваивает `dr_append(out, "[Dynamic] WaitForDebugEvent failed (err` значение `%lu)\n", (unsigned long)werr)`. Логически это обновляет `dr_append(out, "[Dynamic] WaitForDebugEvent failed (err`: хранит промежуточное значение, которое дальше используется в текущей функции.
781	`sb_appendf(err, "WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);`	Технически присваивает `sb_appendf(err, "WaitForDebugEvent failed (err` значение `%lu)\n", (unsigned long)werr)`. Логически это обновляет `sb_appendf(err, "WaitForDebugEvent failed (err`: хранит промежуточное значение, которое дальше используется в текущей функции.
782	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
783	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
784	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
786	`DWORD cont = DBG_CONTINUE;`	Технически объявляет `cont` в функции `глобальной области файла` и сразу задает начальное значение `DBG_CONTINUE`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
787	`bool finish_after_continue = false;`	Технически объявляет `finish_after_continue` в функции `глобальной области файла` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
789	`if (ev.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT) {`	Технически проверяет условие `ev.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
790	`got_create = true;`	Технически присваивает `got_create` значение `true`. Логически это обновляет `got_create`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
791	`base_main = (uint64_t)(uintptr_t)ev.u.CreateProcessInfo.lpBaseOfImage;`	Технически присваивает `base_main` значение `(uint64_t)(uintptr_t)ev.u.CreateProcessInfo.lpBaseOfImage`. Логически это обновляет `base_main`: хранит промежуточное значение, которое дальше используется в текущей функции.
792	`dr_append(out, "[Dynamic] CREATE_PROCESS pid=%lu base=0x%llx\n", (unsigned long)pid, (unsigned long long)base_main);`	Технически присваивает `dr_append(out, "[Dynamic] CREATE_PROCESS pid` значение `%lu base=0x%llx\n", (unsigned long)pid, (unsigned long long)base_main)`. Логически это обновляет `dr_append(out, "[Dynamic] CREATE_PROCESS pid`: хранит промежуточное значение, которое дальше используется в текущей функции.
794	`/* refresh module list and add API breakpoints */`	Технически это комментарий исходника. Логически он объясняет разработчику: refresh module list and add API breakpoints.
795	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
796	`add_api_bps_if_ready(&bps, hProcess, &mods, out, err);`	Технически вызывает `add_api_bps_if_ready`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `add_api_bps_if_ready` для выполнения части алгоритма.
798	`/* Set breakpoints on handlers referenced in UNWIND_INFO */`	Технически это комментарий исходника. Логически он объясняет разработчику: Set breakpoints on handlers referenced in UNWIND_INFO.
799	`for (size_t i = 0; i < handler_count; i++) {`	Технически начинает цикл `size_t i = 0; i < handler_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
800	`uint64_t a = base_main + (uint64_t)handler_rvas[i];`	Технически присваивает `uint64_t a` значение `base_main + (uint64_t)handler_rvas[i]`. Логически это обновляет `uint64_t a`: хранит промежуточное значение, которое дальше используется в текущей функции.
801	`char what[96];`	Технически объявляет `what` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
802	`snprintf(what, sizeof(what), "unwind_handler_rva=0x%08lx", (unsigned long)handler_rvas[i]);`	Технически присваивает `snprintf(what, sizeof(what), "unwind_handler_rva` значение `0x%08lx", (unsigned long)handler_rvas[i])`. Логически это обновляет `snprintf(what, sizeof(what), "unwind_handler_rva`: хранит адрес в координатах RVA, то есть смещение относительно ImageBase.
803	`if (bp_add_and_arm_selfcheck(&bps, hProcess, &mods, out, a, BP_UNWIND_HANDLER, what, err)) {`	Технически проверяет условие `bp_add_and_arm_selfcheck(&bps, hProcess, &mods, out, a, BP_UNWIND_HANDLER, what, err)`; логически отбирает записи, где важна корректность UNWIND_INFO.
804	`out->unwind_handler_bp_set++;`	Технически увеличивает `out->unwind_handler_bp_set` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
805	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
806	`/* best effort: ignore failures (some handlers could be invalid/outside main image) */`	Технически это комментарий исходника. Логически он объясняет разработчику: best effort: ignore failures (some handlers could be invalid/outside main image).
807	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
808	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
810	`/* Process continues after ContinueDebugEvent(). */`	Технически это комментарий исходника. Логически он объясняет разработчику: Process continues after ContinueDebugEvent()..
811	`CloseHandle(ev.u.CreateProcessInfo.hFile);`	Технически вызывает `CloseHandle`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `CloseHandle` для выполнения части алгоритма.
812	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
813	`else if (ev.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
814	`/* keep module list updated so we can set API breakpoints */`	Технически это комментарий исходника. Логически он объясняет разработчику: keep module list updated so we can set API breakpoints.
815	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
816	`add_api_bps_if_ready(&bps, hProcess, &mods, out, err);`	Технически вызывает `add_api_bps_if_ready`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `add_api_bps_if_ready` для выполнения части алгоритма.
817	`if (ev.u.LoadDll.hFile) CloseHandle(ev.u.LoadDll.hFile);`	Технически проверяет условие `ev.u.LoadDll.hFile) CloseHandle(ev.u.LoadDll.hFile`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
818	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
819	`else if (ev.dwDebugEventCode == UNLOAD_DLL_DEBUG_EVENT) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
820	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
821	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
822	`else if (ev.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
823	`dr_append(out, "[Dynamic] EXIT_PROCESS code=%lu\n", (unsigned long)ev.u.ExitProcess.dwExitCode);`	Технически присваивает `dr_append(out, "[Dynamic] EXIT_PROCESS code значение %lu\n", (unsigned long)ev.u.ExitProcess.dwExitCode)`. Логически это обновляет `dr_append(out, "[Dynamic] EXIT_PROCESS code`: хранит промежуточное значение, которое дальше используется в текущей функции.
824	`process_exited = true;`	Технически присваивает `process_exited` значение `true`. Логически это обновляет `process_exited`: хранит промежуточное значение, которое дальше используется в текущей функции.
825	`out->target_exited = true;`	Технически присваивает `out->target_exited` значение `true`. Логически это обновляет `target_exited`: хранит промежуточное значение, которое дальше используется в текущей функции.
826	`out->target_exit_code = (uint32_t)ev.u.ExitProcess.dwExitCode;`	Технически присваивает `out->target_exit_code` значение `(uint32_t)ev.u.ExitProcess.dwExitCode`. Логически это обновляет `target_exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
827	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
828	`out, &mods, "process_exit", ev.dwThreadId, 0, 0, false,`	Технически выполняет битовую операцию `out, &mods, "process_exit", ev.dwThreadId, 0, 0, false,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
829	`ev.u.ExitProcess.dwExitCode, 0, 0, 0);`	Технически выполняет оператор `ev.u.ExitProcess.dwExitCode, 0, 0, 0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
830	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
831	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
832	`else if (ev.dwDebugEventCode == EXCEPTION_DEBUG_EVENT) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
833	`DWORD code = ev.u.Exception.ExceptionRecord.ExceptionCode;`	Технически объявляет `code` в функции `глобальной области файла` и сразу задает начальное значение `ev.u.Exception.ExceptionRecord.ExceptionCode`. Логически это место для данных: машинный код finding или mitigation, стабильный для тестов и отчетов.

Стр.	Код	Нормальное объяснение
834	`uint64_t addr = (uint64_t)(uintptr_t)ev.u.Exception.ExceptionRecord.ExceptionAddress;`	Технически присваивает `uint64_t addr` значение `(uint64_t)(uintptr_t)ev.u.Exception.ExceptionRecord.ExceptionAddress`. Логически это обновляет `uint64_t addr`: хранит промежуточное значение, которое дальше используется в текущей функции.
836	`if (code != EXCEPTION_BREAKPOINT && code != EXCEPTION_SINGLE_STEP) {`	Технически проверяет условие `code != EXCEPTION_BREAKPOINT && code != EXCEPTION_SINGLE_STEP`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
837	`bool first_chance = ev.u.Exception.dwFirstChance != 0;`	Технически объявляет `first_chance` в функции `глобальной области файла` и сразу задает начальное значение `ev.u.Exception.dwFirstChance != 0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
838	`out->exceptions_seen++;`	Технически увеличивает `out->exceptions_seen` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
839	`if (first_chance) out->first_chance_seen++;`	Технически проверяет условие `first_chance`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
840	`else out->second_chance_seen++;`	Технически выполняет оператор `else out->second_chance_seen++;`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
841	`dr_append(out, "[Dynamic] exception 0x%08lx at 0x%llx (%s chance)\n",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
842	`(unsigned long)code,`	Технически эта строка является частью С-конструкции `(unsigned long)code,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
843	`(unsigned long long)addr,`	Технически эта строка является частью С-конструкции `(unsigned long long)addr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
844	`first_chance ? "first" : "second");`	Технически выполняет оператор `first_chance ? "first" : "second");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
845	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
846	`out, &mods, "exception", ev.dwThreadId, addr, (uint32_t)code,`	Технически выполняет битовую операцию `out, &mods, "exception", ev.dwThreadId, addr, (uint32_t)code,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
847	`first_chance, 0, 0, 0, 0);`	Технически выполняет оператор `first_chance, 0, 0, 0, 0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
848	`/* Preserve the target's normal exception-dispatch flow. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Preserve the target's normal exception-dispatch flow..
849	`cont = DBG_EXCEPTION_NOT_HANDLED;`	Технически присваивает `cont` значение `DBG_EXCEPTION_NOT_HANDLED`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
850	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
852	`if (code == EXCEPTION_BREAKPOINT) {`	Технически проверяет условие `code == EXCEPTION_BREAKPOINT`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
853	`Breakpoint* bp = bps_find(&bps, addr);`	Технически присваивает `Breakpoint* bp` значение `bps_find(&bps, addr)`. Логически это обновляет `Breakpoint* bp`: хранит промежуточное значение, которое дальше используется в текущей функции.
854	`if (!bp && !initial_break_ignored) {`	Технически проверяет условие `!bp && !initial_break_ignored`; логически это защита от NULL-указателя и работы с неинициализированными данными.
855	`/* Ignore the debugger's system initial breakpoint only when it is not ours. */`	Технически это комментарий исходника. Логически он объясняет разработчику: Ignore the debugger's system initial breakpoint only when it is not ours..
856	`initial_break_ignored = true;`	Технически присваивает `initial_break_ignored` значение `true`. Логически это обновляет `initial_break_ignored`: хранит промежуточное значение, которое дальше используется в текущей функции.
857	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
858	`} else if (!bp) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
859	`cont = DBG_EXCEPTION_NOT_HANDLED;`	Технически присваивает `cont` значение `DBG_EXCEPTION_NOT_HANDLED`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
860	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
861	`bool bp_prepared = bp_clear(hProcess, bp, err);`	Технически присваивает `bool bp_prepared` значение `bp_clear(hProcess, bp, err)`. Логически это обновляет `bool bp_prepared`: хранит промежуточное значение, которое дальше используется в текущей функции.
862	`if (bp_prepared) {`	Технически проверяет условие `bp_prepared`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
863	`bp->rearm_pending = true;`	Технически присваивает `bp->rearm_pending` значение `true`. Логически это обновляет `rearm_pending`: хранит промежуточное значение, которое дальше используется в текущей функции.
864	`bp_prepared = pending_rearm_set(&pending, ev.dwThreadId, bp);`	Технически присваивает `bp_prepared` значение `pending_rearm_set(&pending, ev.dwThreadId, bp)`. Логически это обновляет `bp_prepared`: хранит промежуточное значение, которое дальше используется в текущей функции.
865	`if (!bp_prepared) {`	Технически проверяет условие `!bp_prepared`; логически это защита от NULL-указателя и работы с неинициализированными данными.
866	`bp->rearm_pending = false;`	Технически присваивает `bp->rearm_pending` значение `false`. Логически это обновляет `rearm_pending`: хранит промежуточное значение, которое дальше используется в текущей функции.
867	`out->pending_rearm_failures++;`	Технически увеличивает `out->pending_rearm_failures` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
868	`out->instrumentation_complete = false;`	Технически присваивает `out->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
869	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
870	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
871	`if (!bp_prepared) {`	Технически проверяет условие `!bp_prepared`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
872	`dr_append(out, "[Dynamic] failed to prepare breakpoint rearm at 0x%llx\n",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
873	`(unsigned long long)addr);`	Технически выполняет оператор `(unsigned long long)addr);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
874	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
875	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
876	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
877	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
878	`HANDLE hEvThread = open_event_thread(ev.dwThreadId);`	Технически присваивает `HANDLE hEvThread` значение `open_event_thread(ev.dwThreadId)`. Логически это обновляет `HANDLE hEvThread`: хранит промежуточное значение, которое дальше используется в текущей функции.
879	`if (!hEvThread) {`	Технически проверяет условие `!hEvThread`; логически это защита от NULL-указателя и работы с неинициализированными данными.
880	`dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] OpenThread failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] OpenThread failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
881	`(unsigned long)ev.dwThreadId, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `GetLastError` для выполнения части алгоритма.
882	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
883	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
884	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
885	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
886	`CONTEXT ctx;`	Технически объявляет `ctx` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
887	`memset(&ctx, 0, sizeof(ctx));`	Технически вызывает `memset`. Логически это действие нужно в функции `глобальной области файла`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
888	`ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;`	Технически присваивает `ctx.ContextFlags` значение `CONTEXT_CONTROL | CONTEXT_INTEGER`. Логически это обновляет `ContextFlags`: хранит промежуточное значение, которое дальше используется в текущей функции.
890	`if (!GetThreadContext(hEvThread, &ctx)) {`	Технически проверяет условие `!GetThreadContext(hEvThread, &ctx)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
891	`dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] GetThreadContext failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] GetThreadContext failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
892	`(unsigned long)ev.dwThreadId, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `GetLastError` для выполнения части алгоритма.
893	`close_event_thread(hEvThread);`	Технически вызывает `close_event_thread`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `close_event_thread` для выполнения части алгоритма.
894	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
895	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
896	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
897	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
898	`/* RIP points to INT3+1; rewind */`	Технически это комментарий исходника. Логически он объясняет разработчику: RIP points to INT3+1; rewind.
899	`ctx.Rip = (DWORD64)addr;`	Технически присваивает `ctx.Rip` значение `(DWORD64)addr`. Логически это обновляет `Rip`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
900	`ctx.EFlags |= 0x100; /* Trap flag */`	Технически выполняет битовую операцию `ctx.EFlags |= 0x100; /* Trap flag */`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
902	`/* handle API breaks: capture handler pointers */`	Технически это комментарий исходника. Логически он объясняет разработчику: handle API breaks: capture handler pointers.
903	`if (bp->kinds & BP_API_ADD_VEH) {`	Технически проверяет условие `bp->kinds & BP_API_ADD_VEH`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
904	`uint64_t first = (uint64_t)ctx.Rcx;`	Технически присваивает `uint64_t first` значение `(uint64_t)ctx.Rcx`. Логически это обновляет `uint64_t first`: хранит промежуточное значение, которое дальше используется в текущей функции.
905	`uint64_t handler = (uint64_t)ctx.Rdx;`	Технически присваивает `uint64_t handler` значение `(uint64_t)ctx.Rdx`. Логически это обновляет `uint64_t handler`: хранит промежуточное значение, которое дальше используется в текущей функции.
906	`out->veh_registrations++;`	Технически увеличивает `out->veh_registrations` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
907	`dr_append(out, "[Dynamic] AddVectoredExceptionHandler first=%llu handler=0x%llx\n",`	Технически присваивает `dr_append(out, "[Dynamic] AddVectoredExceptionHandler first` значение `%llu handler=0x%llx\n",`. Логически это обновляет `dr_append(out, "[Dynamic] AddVectoredExceptionHandler first`: хранит промежуточное значение, которое дальше используется в текущей функции.
908	`(unsigned long long)first, (unsigned long long)handler);`	Технически выполняет оператор `(unsigned long long)first, (unsigned long long)handler);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
909	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
910	`out, &mods, "veh_registration", ev.dwThreadId, addr, 0, false,`	Технически выполняет битовую операцию `out, &mods, "veh_registration", ev.dwThreadId, addr, 0, false,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
911	`first, handler, 0, 0);`	Технически выполняет оператор `first, handler, 0, 0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
912	`if (handler) {`	Технически проверяет условие `handler`; логически отделяет функции с language-specific handler от обычных функций.
913	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
914	`(void)bp_add_and_arm_selfcheck(`	Технически вызывает `bp_add_and_arm_selfcheck`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_add_and_arm_selfcheck` для выполнения части алгоритма.
915	`&bps, hProcess, &mods, out, handler,`	Технически выполняет битовую операцию `&bps, hProcess, &mods, out, handler,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
916	`BP_CUSTOM_VEH_HANDLER, "custom_veh_handler", err);`	Технически выполняет оператор `BP_CUSTOM_VEH_HANDLER, "custom_veh_handler", err);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
917	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
918	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
919	`if (bp->kinds & BP_API_ADD_VCH) {`	Технически проверяет условие `bp->kinds & BP_API_ADD_VCH`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
920	`uint64_t first = (uint64_t)ctx.Rcx;`	Технически присваивает `uint64_t first` значение `(uint64_t)ctx.Rcx`. Логически это обновляет `uint64_t first`: хранит промежуточное значение, которое дальше используется в текущей функции.
921	`uint64_t handler = (uint64_t)ctx.Rdx;`	Технически присваивает `uint64_t handler` значение `(uint64_t)ctx.Rdx`. Логически это обновляет `uint64_t handler`: хранит промежуточное значение, которое дальше используется в текущей функции.
922	`out->vch_registrations++;`	Технически увеличивает `out->vch_registrations` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
923	`dr_append(out, "[Dynamic] AddVectoredContinueHandler first=%llu handler=0x%llx\n",`	Технически присваивает `dr_append(out, "[Dynamic] AddVectoredContinueHandler first` значение `%llu handler=0x%llx\n",`. Логически это обновляет `dr_append(out, "[Dynamic] AddVectoredContinueHandler first`: хранит промежуточное значение, которое дальше используется в текущей функции.
924	`(unsigned long long)first, (unsigned long long)handler);`	Технически выполняет оператор `(unsigned long long)first, (unsigned long long)handler);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
925	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
926	`out, &mods, "vch_registration", ev.dwThreadId, addr, 0, false,`	Технически выполняет битовую операцию `out, &mods, "vch_registration", ev.dwThreadId, addr, 0, false,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
927	`first, handler, 0, 0);`	Технически выполняет оператор `first, handler, 0, 0);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
928	`if (handler) {`	Технически проверяет условие `handler`; логически отделяет функции с language-specific handler от обычных функций.
929	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
930	`(void)bp_add_and_arm_selfcheck(`	Технически вызывает `bp_add_and_arm_selfcheck`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `bp_add_and_arm_selfcheck` для выполнения части алгоритма.
931	`&bps, hProcess, &mods, out, handler,`	Технически выполняет битовую операцию `&bps, hProcess, &mods, out, handler,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
932	`BP_CUSTOM_VCH_HANDLER, "custom_vch_handler", err);`	Технически выполняет оператор `BP_CUSTOM_VCH_HANDLER, "custom_vch_handler", err);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
933	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
934	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
935	`if (bp->kinds & BP_API_SET_UEF) {`	Технически проверяет условие `bp->kinds & BP_API_SET_UEF`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
936	`out->uef_set_calls++;`	Технически увеличивает `out->uef_set_calls` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.

Стр.	Код	Нормальное объяснение
937	`uint64_t handler = (uint64_t)ctx.Rcx;`	Технически присваивает `uint64_t handler` значение `(uint64_t)ctx.Rcx`. Логически это обновляет `uint64_t handler`: хранит промежуточное значение, которое дальше используется в текущей функции.
938	`dr_append(out, "[Dynamic] SetUnhandledExceptionFilter handler=0x%llx\n",`	Технически присваивает `dr_append(out, "[Dynamic] SetUnhandledExceptionFilter handler` значение `0x%llx\n",`. Логически это обновляет `dr_append(out, "[Dynamic] SetUnhandledExceptionFilter handler`: хранит промежуточное значение, которое дальше используется в текущей функции.
939	`(unsigned long long)handler);`	Технически выполняет оператор `(unsigned long long)handler);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
940	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
941	`out, &mods, "uef_registration", ev.dwThreadId, addr, 0, false,`	Технически выполняет битовую операцию `out, &mods, "uef_registration", ev.dwThreadId, addr, 0, false,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
942	`handler, 0, 0, 0);`	Технически выполняет оператор `handler, 0, 0, 0);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
943	`if (handler) {`	Технически проверяет условие `handler`; логически отделяет функции с language-specific handler от обычных функций.
944	`(void)mods_refresh_accounted(&mods, pid, out);`	Технически вызывает `mods_refresh_accounted`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `mods_refresh_accounted` для выполнения части алгоритма.
945	`(void)bp_add_and_arm_selfcheck(`	Технически вызывает `bp_add_and_arm_selfcheck`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `bp_add_and_arm_selfcheck` для выполнения части алгоритма.
946	`&bps, hProcess, &mods, out, handler,`	Технически выполняет битовую операцию `&bps, hProcess, &mods, out, handler,`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
947	`BP_CUSTOM_UEF, "custom_uef_handler", err);`	Технически выполняет оператор `BP_CUSTOM_UEF, "custom_uef_handler", err);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
948	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
949	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
950	`if (bp->kinds & BP_API_RTL_ADD_FT) {`	Технически проверяет условие `bp->kinds & BP_API_RTL_ADD_FT`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
951	`uint64_t table = (uint64_t)ctx.Rcx;`	Технически присваивает `uint64_t table` значение `(uint64_t)ctx.Rcx`. Логически это обновляет `uint64_t table`: хранит промежуточное значение, которое дальше используется в текущей функции.
952	`uint64_t count = (uint64_t)ctx.Rdx;`	Технически присваивает `uint64_t count` значение `(uint64_t)ctx.Rdx`. Логически это обновляет `uint64_t count`: хранит промежуточное значение, которое дальше используется в текущей функции.
953	`uint64_t baseaddr = (uint64_t)ctx.R8;`	Технически присваивает `uint64_t baseaddr` значение `(uint64_t)ctx.R8`. Логически это обновляет `uint64_t baseaddr`: хранит промежуточное значение, которое дальше используется в текущей функции.
954	`out->rtl_add_function_table_calls++;`	Технически увеличивает `out->rtl_add_function_table_calls` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
955	`dr_append(`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
956	`out,`	Технически эта строка является частью С-конструкции `out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
957	`"[Dynamic] RtlAddFunctionTable table=0x%llx count=%llu base=0x%llx\n",`	Технически присваивает `"[Dynamic] RtlAddFunctionTable table` значение `0x%llx count=%llu base=0x%llx\n",`. Логически это обновляет `"[Dynamic] RtlAddFunctionTable table`: хранит промежуточное значение, которое дальше используется в текущей функции.
958	`(unsigned long long)table,`	Технически эта строка является частью С-конструкции `(unsigned long long)table,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
959	`(unsigned long long)count,`	Технически эта строка является частью С-конструкции `(unsigned long long)count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
960	`(unsigned long long)baseaddr);`	Технически выполняет оператор `(unsigned long long)baseaddr);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
961	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
962	`out, &mods, "rtl_add_function_table",`	Технически выполняет битовую операцию `out, &mods, "rtl_add_function_table",`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
963	`ev.dwThreadId, addr, 0, false,`	Технически эта строка является частью С-конструкции `ev.dwThreadId, addr, 0, false,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
964	`table, count, baseaddr, 0);`	Технически выполняет оператор `table, count, baseaddr, 0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
965	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
966	`if (bp->kinds & BP_API_RTL_INSTALL_FTCB) {`	Технически проверяет условие `bp->kinds & BP_API_RTL_INSTALL_FTCB`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
967	`uint64_t table_id = (uint64_t)ctx.Rcx;`	Технически присваивает `uint64_t table_id` значение `(uint64_t)ctx.Rcx`. Логически это обновляет `uint64_t table_id`: хранит промежуточное значение, которое дальше используется в текущей функции.
968	`uint64_t baseaddr = (uint64_t)ctx.Rdx;`	Технически присваивает `uint64_t baseaddr` значение `(uint64_t)ctx.Rdx`. Логически это обновляет `uint64_t baseaddr`: хранит промежуточное значение, которое дальше используется в текущей функции.
969	`uint64_t length = (uint64_t)ctx.R8;`	Технически присваивает `uint64_t length` значение `(uint64_t)ctx.R8`. Логически это обновляет `uint64_t length`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
970	`uint64_t callback = (uint64_t)ctx.R9;`	Технически присваивает `uint64_t callback` значение `(uint64_t)ctx.R9`. Логически это обновляет `uint64_t callback`: хранит промежуточное значение, которое дальше используется в текущей функции.
971	`out->rtl_install_function_table_cb_calls++;`	Технически увеличивает `out->rtl_install_function_table_cb_calls` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
972	`dr_append(`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
973	`out,`	Технически эта строка является частью C-конструкции `out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
974	`"[Dynamic] RtlInstallFunctionTableCallback id=0x%llx base=0x%llx length=%llu callback=0x%llx\n",`	Технически присваивает `"[Dynamic] RtlInstallFunctionTableCallback id` значение `0x%llx base=0x%llx length=%llu callback=0x%llx\n",`. Логически это обновляет `"[Dynamic] RtlInstallFunctionTableCallback id`: хранит промежуточное значение, которое дальше используется в текущей функции.
975	`(unsigned long long)table_id,`	Технически эта строка является частью C-конструкции `(unsigned long long)table_id,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
976	`(unsigned long long)baseaddr,`	Технически эта строка является частью C-конструкции `(unsigned long long)baseaddr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
977	`(unsigned long long)length,`	Технически эта строка является частью C-конструкции `(unsigned long long)length,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
978	`(unsigned long long)callback);`	Технически выполняет оператор `(unsigned long long)callback);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
979	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
980	`out, &mods, "rtl_install_function_table_callback",`	Технически выполняет битовую операцию `out, &mods, "rtl_install_function_table_callback",`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
981	`ev.dwThreadId, addr, 0, false,`	Технически эта строка является частью C-конструкции `ev.dwThreadId, addr, 0, false,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
982	`table_id, baseaddr, length, callback);`	Технически выполняет оператор `table_id, baseaddr, length, callback);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
983	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
984	`if (bp->kinds & BP_UNWIND_HANDLER) {`	Технически проверяет условие `bp->kinds & BP_UNWIND_HANDLER`; логически отбирает записи, где важна корректность UNWIND_INFO.
985	`out->unwind_handler_hits++;`	Технически увеличивает `out->unwind_handler_hits` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
986	`char loc[128];`	Технически объявляет `loc` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
987	`addr_to_loc(img, base_main, addr, loc, sizeof(loc));`	Технически вызывает `addr_to_loc`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `addr_to_loc` для выполнения части алгоритма.
988	`dr_append(out, "[Dynamic] hit unwind handler at %s\n", loc);`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
989	`dynamic_event_add(`	Технически вызывает `dynamic_event_add`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_event_add` для выполнения части алгоритма.
990	`out, &mods, "unwind_handler_hit",`	Технически выполняет битовую операцию `out, &mods, "unwind_handler_hit",`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
991	`ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);`	Технически выполняет оператор `ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
992	`start_trace(`	Технически вызывает `start_trace`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `start_trace` для выполнения части алгоритма.
993	`out, out->trace_steps, ev.dwThreadId,`	Технически эта строка является частью C-конструкции `out, out->trace_steps, ev.dwThreadId,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
994	`(uint64_t)ctx.Rip,`	Технически эта строка является частью C-конструкции `(uint64_t)ctx.Rip,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
995	`&tracing, &trace_tid, &trace_left);`	Технически выполняет битовую операцию `&tracing, &trace_tid, &trace_left);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
996	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
997	`if (bp->kinds & BP_CUSTOM_VEH_HANDLER) {`	Технически проверяет условие `bp->kinds & BP_CUSTOM_VEH_HANDLER`; логически отделяет функции с language-specific handler от обычных функций.
998	`out->veh_hits++;`	Технически увеличивает `out->veh_hits` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.

Стр.	Код	Нормальное объяснение
999	`dr_append(out, "[Dynamic] hit vectored exception handler 0x%llx\n", (unsigned long long)addr);`	Технически вызывает dr_append. Логически это действие нужно в функции глобальной области файла: вызывает функцию dr_append для выполнения части алгоритма.
1000	`dynamic_event_add(`	Технически вызывает dynamic_event_add. Логически это действие нужно в функции глобальной области файла: вызывает функцию dynamic_event_add для выполнения части алгоритма.
1001	`out, &mods, "veh_handler_hit",`	Технически выполняет битовую операцию out, &mods, "veh_handler_hit",. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1002	`ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);`	Технически выполняет оператор ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
1003	`start_trace(`	Технически вызывает start_trace. Логически это действие нужно в функции глобальной области файла: вызывает функцию start_trace для выполнения части алгоритма.
1004	`out, out->trace_steps, ev.dwThreadId,`	Технически эта строка является частью С-конструкции out, out->trace_steps, ev.dwThreadId,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1005	`(uint64_t)ctx.Rip,`	Технически эта строка является частью С-конструкции (uint64_t)ctx.Rip,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1006	`&tracing, &trace_tid, &trace_left);`	Технически выполняет битовую операцию &tracing, &trace_tid, &trace_left);. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1007	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1008	`if (bp->kinds & BP_CUSTOM_VCH_HANDLER) {`	Технически проверяет условие bp->kinds & BP_CUSTOM_VCH_HANDLER; логически отделяет функции с language-specific handler от обычных функций.
1009	`out->vch_hits++;`	Технически увеличивает out->vch_hits на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
1010	`dr_append(out, "[Dynamic] hit vectored continue handler 0x%llx\n", (unsigned long long)addr);`	Технически вызывает dr_append. Логически это действие нужно в функции глобальной области файла: вызывает функцию dr_append для выполнения части алгоритма.
1011	`dynamic_event_add(`	Технически вызывает dynamic_event_add. Логически это действие нужно в функции глобальной области файла: вызывает функцию dynamic_event_add для выполнения части алгоритма.
1012	`out, &mods, "vch_handler_hit",`	Технически выполняет битовую операцию out, &mods, "vch_handler_hit",. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1013	`ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);`	Технически выполняет оператор ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1014	`start_trace(`	Технически вызывает start_trace. Логически это действие нужно в функции глобальной области файла: вызывает функцию start_trace для выполнения части алгоритма.
1015	`out, out->trace_steps, ev.dwThreadId,`	Технически эта строка является частью С-конструкции out, out->trace_steps, ev.dwThreadId,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1016	`(uint64_t)ctx.Rip,`	Технически эта строка является частью С-конструкции (uint64_t)ctx.Rip,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1017	`&tracing, &trace_tid, &trace_left);`	Технически выполняет битовую операцию &tracing, &trace_tid, &trace_left);. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1018	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1019	`if (bp->kinds & BP_CUSTOM_UEF) {`	Технически проверяет условие bp->kinds & BP_CUSTOM_UEF; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1020	`out->uef_hits++;`	Технически увеличивает out->uef_hits на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
1021	`dr_append(out, "[Dynamic] hit unhandled exception filter 0x%llx\n", (unsigned long long)addr);`	Технически вызывает dr_append. Логически это действие нужно в функции глобальной области файла: вызывает функцию dr_append для выполнения части алгоритма.
1022	`dynamic_event_add(`	Технически вызывает dynamic_event_add. Логически это действие нужно в функции глобальной области файла: вызывает функцию dynamic_event_add для выполнения части алгоритма.
1023	`out, &mods, "uef_handler_hit",`	Технически выполняет битовую операцию out, &mods, "uef_handler_hit",. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1024	`ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);`	Технически выполняет оператор ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1025	`start_trace(`	Технически вызывает start_trace. Логически это действие нужно в функции глобальной области файла: вызывает функцию start_trace для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1026	`out, out->trace_steps, ev.dwThreadId,`	Технически эта строка является частью C-конструкции `out, out->trace_steps, ev.dwThreadId,`. Логически она уточняет структуру блока функции `глобальной области` файла и влияет на то, какие данные будут обработаны.
1027	`(uint64_t)ctx.Rip,`	Технически эта строка является частью C-конструкции `(uint64_t)ctx.Rip,`. Логически она уточняет структуру блока функции `глобальной области` файла и влияет на то, какие данные будут обработаны.
1028	`&tracing, &trace_tid, &trace_left);`	Технически выполняет битовую операцию `&tracing, &trace_tid, &trace_left);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1029	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1031	`if (!SetThreadContext(hEvThread, &ctx)) {`	Технически проверяет условие `!SetThreadContext(hEvThread, &ctx)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1032	`dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] SetThreadContext failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] SetThreadContext failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1033	`(unsigned long)ev.dwThreadId,`	Технически эта строка является частью C-конструкции `(unsigned long)ev.dwThreadId,`. Логически она уточняет структуру блока функции `глобальной области` файла и влияет на то, какие данные будут обработаны.
1034	`(unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области` файла: вызывает функцию `GetLastError` для выполнения части алгоритма.
1035	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1036	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1037	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1038	`close_event_thread(hEvThread);`	Технически вызывает `close_event_thread`. Логически это действие нужно в функции `глобальной области` файла: вызывает функцию `close_event_thread` для выполнения части алгоритма.
1039	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
1040	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1041	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1042	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1043	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1044	`}`	Технически завершает область видимости внутри `глобальной области` файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1045	`else if (code == EXCEPTION_SINGLE_STEP) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
1046	`/* Re-arm the breakpoint after executing original instruction (per-thread). */`	Технически это комментарий исходника. Логически он объясняет разработчику: Re-arm the breakpoint after executing original instruction (per-thread)..
1047	`Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);`	Технически присваивает `Breakpoint* pr_bp` значение `pending_rearm_take(&pending, ev.dwThreadId)`. Логически это обновляет `Breakpoint* pr_bp`: хранит промежуточное значение, которое дальше используется в текущей функции.
1048	`if (pr_bp) {`	Технически проверяет условие `pr_bp`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1049	`StrBuf te; sb_init(&te, 128);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `глобальной области` файла: создает динамический текстовый буфер.
1050	`if (!bp_set(hProcess, pr_bp, &te)) {`	Технически проверяет условие `!bp_set(hProcess, pr_bp, &te)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1051	`out->breakpoint_rearm_failures++;`	Технически увеличивает `out->breakpoint_rearm_failures` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.

Стр.	Код	Нормальное объяснение
1052	`out->instrumentation_complete = false;`	Технически объявляет поле `false` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
1053	`dr_append(out, "[Dynamic] failed to re-arm breakpoint at 0x%llx: %s",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
1054	`(unsigned long long)pr_bp->addr,`	Технически эта строка является частью С-конструкции `(unsigned long long)pr_bp->addr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1055	`te.s ? te.s : "unknown error\n");`	Технически выполняет оператор `te.s ? te.s : "unknown error\n");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1056	`debug_loop_ok = false;`	Технически объявляет поле `false` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
1057	`finish_after_continue = true;`	Технически объявляет поле `true` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
1058	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1059	`sb_free(&te);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
1060	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1062	`HANDLE hEvThread = open_event_thread(ev.dwThreadId);`	Технически присваивает `HANDLE hEvThread` значение `open_event_thread(ev.dwThreadId)`. Логически это обновляет `HANDLE hEvThread`: хранит промежуточное значение, которое дальше используется в текущей функции.
1063	`if (!hEvThread) {`	Технически проверяет условие `!hEvThread`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1064	`dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] OpenThread failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] OpenThread failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1065	`(unsigned long)ev.dwThreadId, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `GetLastError` для выполнения части алгоритма.
1066	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1067	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
1068	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
1069	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1070	`CONTEXT ctx;`	Технически объявляет `ctx` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1071	`memset(&ctx, 0, sizeof(ctx));`	Технически вызывает `memset`. Логически это действие нужно в функции `глобальной области файла`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
1072	`ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;`	Технически присваивает `ctx.ContextFlags` значение `CONTEXT_CONTROL | CONTEXT_INTEGER`. Логически это обновляет `ContextFlags`: хранит промежуточное значение, которое дальше используется в текущей функции.
1074	`if (!GetThreadContext(hEvThread, &ctx)) {`	Технически проверяет условие `!GetThreadContext(hEvThread, &ctx)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1075	`dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] GetThreadContext failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] GetThreadContext failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1076	`(unsigned long)ev.dwThreadId, (unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `GetLastError` для выполнения части алгоритма.
1077	`close_event_thread(hEvThread);`	Технически вызывает `close_event_thread`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `close_event_thread` для выполнения части алгоритма.
1078	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1079	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1080	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
1081	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1082	`if (tracing && trace_left && ev.dwThreadId == trace_tid) {`	Технически проверяет условие `tracing && trace_left && ev.dwThreadId == trace_tid`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1083	`add_trace(out, (uint64_t)ctx.Rip);`	Технически вызывает `add_trace`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `add_trace` для выполнения части алгоритма.
1084	`trace_left--;`	Технически уменьшает `trace_left` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
1085	`/* keep trap flag */`	Технически это комментарий исходника. Логически он объясняет разработчику: keep trap flag.
1086	`ctx.EFlags |= 0x100;`	Технически применяет битовую операцию `|=` к `ctx.EFlags` со значением `0x100`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
1087	`if (trace_left == 0) {`	Технически проверяет условие `trace_left == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1088	`tracing = false;`	Технически присваивает `tracing` значение `false`. Логически это обновляет `tracing`: хранит промежуточное значение, которое дальше используется в текущей функции.
1089	`trace_tid = 0;`	Технически присваивает `trace_tid` значение `0`. Логически это обновляет `trace_tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1090	`ctx.EFlags &= ~0x100u;`	Технически применяет битовую операцию `&=` к `ctx.EFlags` со значением `~0x100u`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
1091	`dr_append(out, "[Dynamic] trace collected: %u steps\n", (unsigned)out->trace_count);`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
1092	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1093	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1094	`/* stop single-stepping */`	Технически это комментарий исходника. Логически он объясняет разработчику: stop single-stepping.
1095	`ctx.EFlags &= ~0x100u;`	Технически применяет битовую операцию `&=` к `ctx.EFlags` со значением `~0x100u`. Логически так код сдвигает или обновляет маску флагов, чтобы последовательно проверить биты.
1096	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1097	`if (!SetThreadContext(hEvThread, &ctx)) {`	Технически проверяет условие `!SetThreadContext(hEvThread, &ctx)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1098	`dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",`	Технически присваивает `dr_append(out, "[Dynamic] SetThreadContext failed tid` значение `%lu (err=%lu)\n",`. Логически это обновляет `dr_append(out, "[Dynamic] SetThreadContext failed tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1099	`(unsigned long)ev.dwThreadId,`	Технически эта строка является частью C-конструкции `(unsigned long)ev.dwThreadId,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1100	`(unsigned long)GetLastError());`	Технически вызывает `GetLastError`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `GetLastError` для выполнения части алгоритма.
1101	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1102	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
1103	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1104	`close_event_thread(hEvThread);`	Технически вызывает `close_event_thread`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `close_event_thread` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1105	`cont = DBG_CONTINUE;`	Технически присваивает `cont` значение `DBG_CONTINUE`. Логически это обновляет `cont`: хранит промежуточное значение, которое дальше используется в текущей функции.
1106	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1107	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
1108	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1110	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1111	`else if (ev.dwDebugEventCode == EXIT_THREAD_DEBUG_EVENT) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
1112	`Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);`	Технически присваивает `Breakpoint* pr_bp` значение `pending_rearm_take(&pending, ev.dwThreadId)`. Логически это обновляет `Breakpoint* pr_bp`: хранит промежуточное значение, которое дальше используется в текущей функции.
1113	`if (pr_bp) {`	Технически проверяет условие `pr_bp`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1114	`StrBuf te;`	Технически объявляет `te` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1115	`sb_init(&te, 128);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `глобальной области файла`: создает динамический текстовый буфер.
1116	`if (!bp_set(hProcess, pr_bp, &te)) {`	Технически проверяет условие `!bp_set(hProcess, pr_bp, &te)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1117	`out->breakpoint_rearm_failures++;`	Технически увеличивает `out->breakpoint_rearm_failures` на единицу. Логически это двигает индекс, счетчик найденных объектов или позицию в анализируемой таблице.
1118	`out->instrumentation_complete = false;`	Технически присваивает `out->instrumentation_complete` значение `false`. Логически это обновляет `instrumentation_complete`: хранит промежуточное значение, которое дальше используется в текущей функции.
1119	`dr_append(out, "[Dynamic] failed to re-arm breakpoint after thread exit: %s",`	Технически вызывает `dr_append`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dr_append` для выполнения части алгоритма.
1120	`te.s ? te.s : "unknown error\n");`	Технически выполняет оператор `te.s ? te.s : "unknown error\n");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1121	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1122	`finish_after_continue = true;`	Технически присваивает `finish_after_continue` значение `true`. Логически это обновляет `finish_after_continue`: хранит промежуточное значение, которое дальше используется в текущей функции.
1123	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1124	`sb_free(&te);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
1125	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1126	`if (tracing && trace_tid == ev.dwThreadId) {`	Технически проверяет условие `tracing && trace_tid == ev.dwThreadId`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1127	`tracing = false;`	Технически присваивает `tracing` значение `false`. Логически это обновляет `tracing`: хранит промежуточное значение, которое дальше используется в текущей функции.
1128	`trace_tid = 0;`	Технически присваивает `trace_tid` значение `0`. Логически это обновляет `trace_tid`: хранит промежуточное значение, которое дальше используется в текущей функции.
1129	`trace_left = 0;`	Технически присваивает `trace_left` значение `0`. Логически это обновляет `trace_left`: хранит промежуточное значение, которое дальше используется в текущей функции.
1130	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1131	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1133	`if (!ContinueDebugEvent(ev.dwProcessId, ev.dwThreadId, cont)) {`	Технически проверяет условие `!ContinueDebugEvent(ev.dwProcessId, ev.dwThreadId, cont)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1134	`DWORD cerr = GetLastError();`	Технически присваивает `DWORD cerr` значение `GetLastError()`. Логически это обновляет `DWORD cerr`: хранит промежуточное значение, которое дальше используется в текущей функции.
1135	`dr_append(out, "[Dynamic] ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);`	Технически присваивает `dr_append(out, "[Dynamic] ContinueDebugEvent failed (err` значение `%lu)\n", (unsigned long)cerr)`. Логически это обновляет `dr_append(out, "[Dynamic] ContinueDebugEvent failed (err`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1136	`sb_appendf(err, "ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);`	Технически присваивает `sb_appendf(err, "ContinueDebugEvent failed (err` значение `%lu)\n", (unsigned long)cerr)`. Логически это обновляет `sb_appendf(err, "ContinueDebugEvent failed (err`: хранит промежуточное значение, которое дальше используется в текущей функции.
1137	`debug_loop_ok = false;`	Технически присваивает `debug_loop_ok` значение `false`. Логически это обновляет `debug_loop_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
1138	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
1139	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
1140	`if (finish_after_continue) break;`	Технически проверяет условие `finish_after_continue`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1141	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1143	`out->timed_out = timed_out;`	Технически присваивает `out->timed_out` значение `timed_out`. Логически это обновляет `timed_out`: является выходным параметром, куда функция записывает результат.
1144	`out->target_exited = process_exited;`	Технически присваивает `out->target_exited` значение `process_exited`. Логически это обновляет `target_exited`: хранит промежуточное значение, которое дальше используется в текущей функции.
1145	`out->ok = got_create && debug_loop_ok && !timed_out;`	Технически присваивает `out->ok` значение `got_create && debug_loop_ok && !timed_out`. Логически это обновляет ok: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
1146	`if (!sb_ok(&out->details)) {`	Технически проверяет условие `!sb_ok(&out->details)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1147	`out->ok = false;`	Технически присваивает `out->ok` значение `false`. Логически это обновляет ok: флаг успешного разбора, чтобы другие модули не доверяли неполной модели.
1148	`sb_append(err, "Out of memory while building dynamic details.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1149	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1151	`/* Cleanup: remove breakpoints */`	Технически это комментарий исходника. Логически он объясняет разработчику: Cleanup: remove breakpoints.
1152	`for (size_t i = 0; i < bps.count; i++) {`	Технически начинает цикл `size_t i = 0; i < bps.count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1153	`bp_clear(hProcess, &bps.bps[i], NULL);`	Технически вызывает `bp_clear`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `bp_clear` для выполнения части алгоритма.
1154	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1156	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
1157	`* Terminate only when the target did not exit by itself. A debuggee may`	Технически это комментарий исходника. Логически он объясняет разработчику: Terminate only when the target did not exit by itself. A debuggee may.
1158	`* not signal its process handle until the debugger continues the queued`	Технически это комментарий исходника. Логически он объясняет разработчику: not signal its process handle until the debugger continues the queued.
1159	`* exit event, so drain cleanup events instead of waiting a fixed 5 s.`	Технически это комментарий исходника. Логически он объясняет разработчику: exit event, so drain cleanup events instead of waiting a fixed 5 s..
1160	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
1161	`if (!process_exited && TerminateProcess(hProcess, 0)) {`	Технически проверяет условие `!process_exited && TerminateProcess(hProcess, 0)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1162	`uint64_t cleanup_deadline = GetTickCount64() + 1000u;`	Технически присваивает `uint64_t cleanup_deadline` значение `GetTickCount64() + 1000u`. Логически это обновляет `uint64_t cleanup_deadline`: хранит промежуточное значение, которое дальше используется в текущей функции.
1163	`while (WaitForSingleObject(hProcess, 0) != WAIT_OBJECT_0 &&`	Технически начинает цикл while с условием `WaitForSingleObject(hProcess, 0`. Логически код повторяет обработку, пока поток данных или состояние не исчерпаны.
1164	`GetTickCount64() < cleanup_deadline) {`	Технически объявляет/начинает функцию `GetTickCount64`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
1165	`DEBUG_EVENT cleanup_event;`	Технически объявляет `cleanup_event` в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1166	`if (!WaitForDebugEvent(&cleanup_event, 50)) {`	Технически проверяет условие `!WaitForDebugEvent(&cleanup_event, 50)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
1167	`if (GetLastError() == ERROR_SEM_TIMEOUT) continue;`	Технически проверяет условие `GetLastError() == ERROR_SEM_TIMEOUT`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1168	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
1169	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1171	`if (cleanup_event.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT &&`	Технически проверяет условие `if (cleanup_event.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1172	`cleanup_event.u.CreateProcessInfo.hFile) {`	Технически эта строка является частью С-конструкции `cleanup_event.u.CreateProcessInfo.hFile) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1173	`CloseHandle(cleanup_event.u.CreateProcessInfo.hFile);`	Технически вызывает `CloseHandle`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `CloseHandle` для выполнения части алгоритма.
1174	`} else if (cleanup_event.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT &&`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1175	`cleanup_event.u.LoadDll.hFile) {`	Технически эта строка является частью С-конструкции `cleanup_event.u.LoadDll.hFile) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1176	`CloseHandle(cleanup_event.u.LoadDll.hFile);`	Технически вызывает `CloseHandle`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `CloseHandle` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1177	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1179	`bool cleanup_exited =`	Технически присваивает `bool cleanup_exited` значение `` `. Логически это обновляет bool cleanup_exited` ``: хранит промежуточное значение, которое дальше используется в текущей функции.
1180	`cleanup_event.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT;`	Технически выполняет оператор `cleanup_event.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT;`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1181	`if (!ContinueDebugEvent(`	Технически проверяет условие `if (!ContinueDebugEvent(`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1182	`cleanup_event.dwProcessId,`	Технически эта строка является частью С-конструкции `cleanup_event.dwProcessId,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1183	`cleanup_event.dwThreadId,`	Технически эта строка является частью С-конструкции `cleanup_event.dwThreadId,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1184	`DBG_CONTINUE)) {`	Технически эта строка является частью С-конструкции `DBG_CONTINUE)) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1185	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
1186	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1187	`if (cleanup_exited) break;`	Технически проверяет условие `cleanup_exited`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1188	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1189	`(void)WaitForSingleObject(hProcess, 100);`	Технически вызывает `WaitForSingleObject`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `WaitForSingleObject` для выполнения части алгоритма.
1190	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1192	`if (pi.hThread) CloseHandle(pi.hThread);`	Технически проверяет условие `pi.hThread) CloseHandle(pi.hThread`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1193	`if (pi.hProcess) CloseHandle(pi.hProcess);`	Технически проверяет условие `pi.hProcess) CloseHandle(pi.hProcess`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1195	`free(handler_rvas);`	Технически вызывает `free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память, чтобы в С не было утечки.
1196	`sb_free(&cmd);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `глобальной области файла`: освобождает память динамического буфера.
1198	`if (!out->ok) {`	Технически проверяет условие `!out->ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
1199	`if (timed_out) {`	Технически проверяет условие `timed_out`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1200	`sb_appendf(err, "Dynamic session timed out after %u ms.\n", timeout_ms);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1201	`} else if (!got_create) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1202	`sb_append(err, "Dynamic session failed before CREATE_PROCESS_DEBUG_EVENT.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1203	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1204	`sb_append(err, "Dynamic session failed inside the debug event loop.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1205	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1206	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
1207	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1208	`return true;`	Технически возвращает `true` из функции `глобальной области файла`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
1209	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1211	`#else /* !_WIN32 */`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
1213	`void dynamic_report_init(DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_report_init`; она доступная другим модулям через заголовок или линковку. Логически функция инициализирует структуру динамического отчета.
1214	`if (!dr) return;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1215	`memset(dr, 0, sizeof(*dr));`	Технически вызывает `memset`. Логически это действие нужно в функции `dynamic_report_init`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.

Стр.	Код	Нормальное объяснение
1216	`dr->enabled = false;`	Технически присваивает `dr->enabled` значение `false`. Логически это обновляет `enabled`: хранит промежуточное значение, которое дальше используется в текущей функции.
1217	`sb_init(&dr->details, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `dynamic_report_init`: создает динамический текстовый буфер.
1218	`sb_append(&dr->details, "Dynamic mode is supported only on Windows.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `dynamic_report_init`: добавляет текст в динамический буфер.
1219	`}`	Технически завершает область видимости внутри `dynamic_report_init`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1221	`void dynamic_report_free(DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_report_free`; она доступная другим модулям через заголовок или линковку. Логически функция освобождает память внутри динамического отчета.
1222	`if (!dr) return;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1223	`sb_free(&dr->details);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `dynamic_report_free`: освобождает память динамического буфера.
1224	`memset(dr, 0, sizeof(*dr));`	Технически вызывает `memset`. Логически это действие нужно в функции `dynamic_report_free`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
1225	`}`	Технически завершает область видимости внутри `dynamic_report_free`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1227	`bool dynamic_collect_windows(const char* pe_path,`	Технически вызывает `dynamic_collect_windows`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `dynamic_collect_windows` для выполнения части алгоритма.
1228	`const char* const* child_argv,`	Технически эта строка является частью С-конструкции `const char* const* child_argv,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1229	`int child_argc,`	Технически эта строка является частью С-конструкции `int child_argc,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
1230	`const PeImage* img,`	Технически эта строка является частью С-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1231	`const RuntimeFunction* rf,`	Технически эта строка является частью С-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1232	`const UnwindSummary* us,`	Технически эта строка является частью С-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1233	`const bool* us_ok,`	Технически эта строка является частью С-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1234	`size_t rf_count,`	Технически эта строка является частью С-конструкции `size_t rf_count,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1235	`uint32_t timeout_ms,`	Технически эта строка является частью С-конструкции `uint32_t timeout_ms,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1236	`uint32_t trace_steps,`	Технически эта строка является частью С-конструкции `uint32_t trace_steps,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1237	`DynamicReport* out,`	Технически эта строка является частью С-конструкции `DynamicReport* out,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1238	`StrBuf* err) {`	Технически эта строка является частью С-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1239	`(void)pe_path; (void)child_argv; (void)child_argc; (void)img; (void)rf; (void)us; (void)us_ok; (void)rf_count;`	Технически выполняет оператор `(void)pe_path; (void)child_argv; (void)child_argc; (void)img; (void)rf; (void)us; (void)us_ok; (void)rf_count;`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1240	`(void)timeout_ms; (void)trace_steps;`	Технически выполняет оператор `(void)timeout_ms; (void)trace_steps;`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1241	`dynamic_report_init(out);`	Технически вызывает `dynamic_report_init`. Логически это действие нужно в функции `глобальной области файла`: инициализирует структуру динамического отчета.
1242	`if (!out || !sb_ok(&out->details)) {`	Технически проверяет условие `!out || !sb_ok(&out->details)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1243	`sb_append(err, "Out of memory (dynamic report buffer).\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1244	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
1245	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1246	`out->enabled = true;`	Технически присваивает `out->enabled` значение `true`. Логически это обновляет `enabled`: хранит промежуточное значение, которое дальше используется в текущей функции.
1247	`out->timeout_ms = timeout_ms;`	Технически присваивает `out->timeout_ms` значение `timeout_ms`. Логически это обновляет `timeout_ms`: хранит промежуточное значение, которое дальше используется в текущей функции.
1248	`out->trace_steps = trace_steps > 256u ? 256u : trace_steps;`	Технически присваивает `out->trace_steps` значение `trace_steps > 256u ? 256u : trace_steps`. Логически это обновляет `trace_steps`: хранит промежуточное значение, которое дальше используется в текущей функции.
1249	`sb_append(err, "Dynamic mode is supported only on Windows.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1250	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.

Стр.	Код	Нормальное объяснение
1251	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1253	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 1124. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

8. Отчеты JSON/HTML/CSV

Этот модуль делает результат защищаемым: комиссия, аналитик или тест могут увидеть не только verdict, но и доказательства, уровни, корреляции и рекомендации проверки.

Что сначала выучить в C и теории

- формирование текстовых отчетов без сторонних библиотек
- экранирование JSON, HTML и CSV
- контракт отчета: machine-readable JSON и human-readable HTML
- почему отчет должен объяснять доказательства, а не просто печатать score

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

include/report.h

Назначение файла: интерфейс печати brief/CSV/JSON/HTML отчетов.

Код, который пишем на этом шаге

```
#pragma once
/*
  report.h - output helpers (stdout, CSV, JSON)
*/

#include <stddef.h>
#include <stdbool.h>
#include "pe.h"
#include "unwind.h"
#include "heuristics.h"
#include "dynamic.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
    bool strict_abi;
    bool follow_chain;
    uint32_t max_chain_depth;
} ReportMetadata;

void report_print_brief(const char* path, const PeImage* img, size_t rf_count);

bool report_write_csv(const char* path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      bool ghidra_mode,
                      StrBuf* err);

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
                          const DynamicReport* dr,
                          const ReportMetadata* metadata,
                          StrBuf* err);

bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err);

void report_print_heuristics(const HeuristicReport* hr);

#ifdef __cplusplus
}
#endif
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#pragma once`	Технически это директива включения заголовка только один раз. Логически она защищает проект от повторных объявлений структур и функций при сложной цепочке include.
2	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
3	`report.h - output helpers (stdout, CSV, JSON)`	Технически объявляет/начинает функцию `helpers`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
4	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
6	`#include <stddef.h>`	Технически подключает `stddef.h`. Логически это дает модулю доступ к: size_t и NULL для размеров, индексов и указателей.
7	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
8	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.
9	`#include "unwind.h"`	Технически подключает `unwind.h`. Логически это дает модулю доступ к: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.
10	`#include "heuristics.h"`	Технически подключает `heuristics.h`. Логически это дает модулю доступ к: модель finding, mitigation, score, confidence и verdict.
11	`#include "dynamic.h"`	Технически подключает `dynamic.h`. Логически это дает модулю доступ к: модель динамического отчета и настройки runtime-валидации.
12	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
14	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы C-заголовок можно было безопасно подключить из C++ без искажения имен функций.
15	`extern "C" {`	Технически эта строка является частью C-конструкции `extern "C" {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
16	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
18	`typedef struct ReportMetadata {`	Технически начинает описание структуры `ReportMetadata`. Логически структура собирает связанные поля в одну модель данных проекта.
19	`const char* tool_version;`	Технически объявляет поле `tool_version` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
20	`const char* report_schema_version;`	Технически объявляет поле `report_schema_version` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
21	`const char* generated_at_utc;`	Технически объявляет поле `generated_at_utc` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
22	`const char* input_sha256;`	Технически объявляет поле `input_sha256` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
23	`uint64_t input_size;`	Технически объявляет поле `input_size` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
24	`const char* command_line;`	Технически объявляет поле `command_line` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
25	`const char* build_compiler;`	Технически объявляет поле `build_compiler` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
26	`const char* build_platform;`	Технически объявляет поле `build_platform` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
27	`const char* build_mode;`	Технически объявляет поле `build_mode` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
28	`bool strict_abi;`	Технически объявляет поле `strict_abi` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
29	`bool follow_chain;`	Технически объявляет поле `follow_chain` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
30	`uint32_t max_chain_depth;`	Технически объявляет поле `max_chain_depth` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
31	`} ReportMetadata;`	Технически завершает typedef и дает типу имя `ReportMetadata`. Логически дальше код может объявлять переменные этого типа без `struct`.
33	`void report_print_brief(const char* path, const PeImage* img, size_t rf_count);`	Технически объявляет прототип функции `report_print_brief` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и печатает краткую сводку анализа в консоль.
35	`bool report_write_csv(const char* path,`	Технически вызывает `report_write_csv`. Логически это действие нужно в функции `глобальной области файла`: записывает CSV по функциям и unwind-данным.
36	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
37	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
38	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
39	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
40	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
41	`bool ghidra_mode,`	Технически эта строка является частью C-конструкции `bool ghidra_mode,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
42	`StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
44	`bool report_write_json_ex(const char* path,`	Технически вызывает `report_write_json_ex`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `report_write_json_ex` для выполнения части алгоритма.
45	`const char* input_path,`	Технически эта строка является частью C-конструкции `const char* input_path,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
46	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
47	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
48	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
49	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
50	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
51	`const HeuristicReport* hr,`	Технически эта строка является частью C-конструкции `const HeuristicReport* hr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
52	`const DynamicReport* dr,`	Технически эта строка является частью C-конструкции `const DynamicReport* dr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
53	`const ReportMetadata* metadata,`	Технически эта строка является частью C-конструкции `const ReportMetadata* metadata,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
54	`StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
56	`bool report_write_html(const char* path,`	Технически вызывает `report_write_html`. Логически это действие нужно в функции `глобальной области файла`: записывает HTML-отчет для ручного просмотра.
57	`const char* input_path,`	Технически эта строка является частью C-конструкции `const char* input_path,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
58	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
59	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
60	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
61	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
62	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
63	`  const HeuristicReport* hr,`	Технически эта строка является частью С-конструкции `const HeuristicReport* hr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
64	`  const DynamicReport* dr,`	Технически эта строка является частью С-конструкции `const DynamicReport* dr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
65	`  const ReportMetadata* metadata,`	Технически эта строка является частью С-конструкции `const ReportMetadata* metadata,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
66	`  StrBuf* err);`	Технически объявляет переменную в функции `глобальной области файла`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
68	`void report_print_heuristics(const HeuristicReport* hr);`	Технически объявляет прототип функции `report_print_heuristics` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и печатает найденные эвристики в консоль.
70	`#ifdef __cplusplus`	Технически проверяет компиляцию из C++. Логически это нужно, чтобы С-заголовок можно было безопасно подключить из C++ без искажения имен функций.
71	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
72	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.

Разобрано непустых строк: 63. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

src/report.c

Назначение файла: генератор человекочитаемых и машинночитаемых отчетов.

Код, который пишем на этом шаге

```
 1 | #include "report.h"
 2 | #include "app_info.h"
 3 | #include <stdio.h>
 4 | #include <stdlib.h>
 5 | #include <string.h>
 6 |
 7 | #define IMAGE_SCN_MEM_EXECUTE 0x20000000u
 8 | #define IMAGE_SCN_MEM_READ    0x40000000u
 9 | #define IMAGE_SCN_MEM_WRITE   0x80000000u
10 |
11 | static void append_regmask(StrBuf* sb, uint16_t mask, bool xmm) {
12 |     bool first = true;
13 |     for (uint8_t r = 0; r < 16; r++) {
14 |         if (!(mask & (uint16_t)(1u << r))) continue;
15 |         if (!first) sb_append(sb, ",");
16 |         if (xmm) sb_appendf(sb, "XMM%u", (unsigned)r);
17 |         else sb_append(sb, unwind_reg_name(r));
18 |         first = false;
19 |     }
20 |     if (first) sb_append(sb, "-");
21 | }
22 |
23 | static void anomalies_to_str(StrBuf* sb, uint32_t anom) {
24 |     bool first = true;
25 |     struct { uint32_t bit; const char* name; } m[] = {
26 |         {UA_VERSION_UNKNOWN, "ver"},
27 |         {UA_TRUNCATED, "trunc"},
28 |         {UA_BAD_OPCODE, "op"},
29 |         {UA_BAD_OPERAND, "arg"},
30 |         {UA_CODEOFFSET_OUTSIDE, "off"},
31 |         {UA_CHAIN_LOOP, "chain_loop"},
32 |         {UA_CHAIN_DEPTH_EXCEEDED, "chain_depth"},
33 |         {UA_HANDLER_OOB, "handler_oob"},
34 |         {UA_CODEOFFSET_ORDER, "order"},
35 |         {UA_OPERAND_SANITY, "operand_sanity"},
36 |         {UA_ALIGNMENT, "alignment"},
37 |         {UA_CHAIN_INVALID, "chain_invalid"},
38 |         {UA_ABI_REGISTER_CLASS, "abi_register_class"},
39 |         {UA_ABI_RESERVED_OPINFO, "abi_reserved_opinfo"},
40 |         {UA_ABI_PUSH_ORDER, "abi_push_order"},
41 |         {UA_ABI_CHAIN_RESTRICTION, "abi_chain_restriction"},
42 |         {UA_ABI_SAVE_OFFSET, "abi_save_offset"},
43 |     };
44 |     for (size_t i = 0; i < sizeof(m)/sizeof(m[0]); i++) {
45 |         if (!(anom & m[i].bit)) continue;
46 |         if (!first) sb_append(sb, "|");
47 |         sb_append(sb, m[i].name);
48 |         first = false;
49 |     }
50 |     if (first) sb_append(sb, "0");
51 | }
52 |
53 | static const char* section_name_or_dash(const PeImage* img, uint32_t rva) {
54 |     const PeSection* s = pe_find_section_for_rva(img, rva);
55 |     return s ? s->name : "-";
56 | }
57 |
58 | static uint32_t runtime_function_size(const RuntimeFunction* r) {
59 |     if (!r || r->end_rva < r->begin_rva) return 0;
60 |     return r->end_rva - r->begin_rva;
61 | }
62 |
63 | static bool csv_cell(StrBuf* out, const char* s) {
64 |     if (!out || !s) return false;
```

```
    bool need_q = false;
    for (const char* p = s; *p; p++) {
        if (*p == ',' || *p == '"' || *p == '\n' || *p == '\r') { need_q = true; break; }
    }
    if (!need_q) return sb_append(out, s);

    if (!sb_appendn(out, "\"", 1)) return false;
    for (const char* p = s; *p; p++) {
        if (*p == '"' && !sb_appendn(out, "\"", 1)) return false;
        if (!sb_appendn(out, p, 1)) return false;
    }
    return sb_appendn(out, "\"", 1);
}

static bool build_unwind_comment(const UnwindSummary* u, StrBuf* sb) {
    sb->len = 0;
    if (sb->s) sb->s[0] = 0;

    char fbuf[64];
    unwind_flags_to_string(u->flags, fbuf);

    sb_appendf(sb, "UNWIND v%u flags=%s prolog=%u stack=%uB frame=%s+%u ",
               (unsigned)u->version, fbuf, (unsigned)u->prolog_size,
               (unsigned)u->fixed_stack_bytes,
               (u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),
               (unsigned)(u->frame_off * 16u));

    sb_append(sb, "saved_gp=[");
    append_regmask(sb, u->saved_nonvol_mask, false);
    sb_append(sb, "] saved_xmm=[");
    append_regmask(sb, u->saved_xmm_mask, true);
    sb_append(sb, "]");

    if (u->has_handler) sb_appendf(sb, " handler_rva=0x%08x", (unsigned)u->handler_rva);
    if (u->has_chain) {
        sb_appendf(sb, " chain_to=[0x%08x..0x%08x] u=0x%08x",
                   (unsigned)u->chain.begin_rva,
                   (unsigned)u->chain.end_rva,
                   (unsigned)u->chain.unwind_rva);
    }

    if (u->anomalies) {
        sb_append(sb, " anomalies=");
        anomalies_to_str(sb, u->anomalies);
    }
    return sb_ok(sb);
}

void report_print_brief(const char* path, const PeImage* img, size_t rf_count) {
    printf("File: %s\n", path);
    printf("Machine: 0x%04x (AMD64)\n", (unsigned)img->machine);
    printf("ImageBase: 0x%llx\n", (unsigned long long)img->image_base);
    printf("SizeOfImage: 0x%x\n", (unsigned)img->size_of_image);
    printf("SizeOfHeaders: 0x%x  Sections: %zu\n",
           (unsigned)img->size_of_headers, img->num_sections);
    printf("Exception Directory: RVA=0x%08x Size=0x%x\n", (unsigned)img->exception_rva, (unsigned)img->exception_size);
    printf("RUNTIME_FUNCTION entries: %zu\n", rf_count);
}

bool report_write_csv(const char* path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      bool ghidra_mode,
                      StrBuf* err) {
    (void)img;
    if (!path || (!rf && n) || (!us && n) || (!us_ok && n)) {
        sb_append(err, "Invalid arguments for CSV report.\n");
        return false;
    }

    StrBuf output = {0};
    StrBuf comment = {0};
    if (!sb_init(&output, 8192) || !sb_init(&comment, 512)) {
        sb_free(&comment);
        sb_free(&output);
        sb_append(err, "OOM.\n");
        return false;
    }

    if (ghidra_mode) {
        sb_append(&output, "begin_rva,end_rva,name,comment\n");
        for (size_t i = 0; i < n; i++) {
            const RuntimeFunction* r = &rf[i];
            const UnwindSummary* u = &us[i];

            char name[64];
            snprintf(name, sizeof(name), "sub_%08X", (unsigned)r->begin_rva);

            if (us_ok[i]) build_unwind_comment(u, &comment);
            else {
                comment.len = 0;
                if (comment.s) comment.s[0] = 0;
                sb_append(&comment, "UNWIND parse failed");
                if (u->parse_error[0]) sb_appendf(&comment, ": %s", u->parse_error);
            }

            char a[32], b[32];
            snprintf(a, sizeof(a), "0x%08X", (unsigned)r->begin_rva);
            snprintf(b, sizeof(b), "0x%08X", (unsigned)r->end_rva);

            csv_cell(&output, a); sb_appendn(&output, ",", 1);
            csv_cell(&output, b); sb_appendn(&output, ",", 1);
            csv_cell(&output, name); sb_appendn(&output, ",", 1);
            csv_cell(&output, comment.s ? comment.s : "");
            sb_appendn(&output, "\n", 1);
        }
    } else {
        sb_append(&output,
                  "index,begin_rva,end_rva,size,unwind_rva,unwind_ver,"
                  "unwind_flags,prolog_size,fixed_stack,frame_reg,frame_off,"
                  "handler_rva,has_chain,anomalies,parse_error\n");
        for (size_t i = 0; i < n; i++) {
```

```
            const RuntimeFunction* r = &rf[i];
            const UnwindSummary* u = &us[i];
            char fbuf[64];
            unwind_flags_to_string(u->flags, fbuf);

            sb_appendf(&output, "%zu,0x%08X,0x%08X,%u,0x%08X,",
                       i, (unsigned)r->begin_rva, (unsigned)r->end_rva,
                       (unsigned)runtime_function_size(r),
                       (unsigned)r->unwind_rva);

            if (!us_ok[i]) {
                sb_appendf(&output, "NA,NA,NA,NA,NA,NA,NA,0,0x%08X,",
                           (unsigned)u->anomalies);
                csv_cell(&output, u->parse_error);
                sb_appendn(&output, "\n", 1);
                continue;
            }

            sb_appendf(&output, "%u,%s,%u,%u,%s,%u,0x%08X,%u,0x%08X,",
                       (unsigned)u->version, fbuf, (unsigned)u->prolog_size,
                       (unsigned)u->fixed_stack_bytes,
                       (u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),
                       (unsigned)(u->frame_off * 16u),
                       (unsigned)u->handler_rva,
                       (unsigned)(u->has_chain ? 1u : 0u),
                       (unsigned)u->anomalies);
            csv_cell(&output, "");
            sb_appendn(&output, "\n", 1);
        }
    }

    bool ok = sb_ok(&output) && sb_ok(&comment);
    if (ok) ok = file_write_atomic(path, output.s, output.len);
    sb_free(&comment);
    sb_free(&output);
    if (!ok) sb_appendf(err, "Failed while building or writing CSV: %s\n", path);
    return ok;
}

static size_t valid_utf8_sequence_length(const unsigned char* s, size_t remaining) {
    if (!s || remaining == 0) return 0;
    if (s[0] < 0x80u) return 1;
    if (s[0] >= 0xc2u && s[0] <= 0xdfu) {
        return remaining >= 2 && s[1] >= 0x80u && s[1] <= 0xbfu ? 2 : 0;
    }
    if (s[0] == 0xe0u) {
        return remaining >= 3 && s[1] >= 0xa0u && s[1] <= 0xbfu &&
               s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if ((s[0] >= 0xe1u && s[0] <= 0xecu) || (s[0] >= 0xeeu && s[0] <= 0xefu)) {
        return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0xbfu &&
               s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if (s[0] == 0xedu) {
        return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0x9fu &&
               s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if (s[0] == 0xf0u) {
        return remaining >= 4 && s[1] >= 0x90u && s[1] <= 0xbfu &&
               s[2] >= 0x80u && s[2] <= 0xbfu &&
               s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    if (s[0] >= 0xf1u && s[0] <= 0xf3u) {
        return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0xbfu &&
               s[2] >= 0x80u && s[2] <= 0xbfu &&
               s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    if (s[0] == 0xf4u) {
        return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0x8fu &&
               s[2] >= 0x80u && s[2] <= 0xbfu &&
               s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    return 0;
}

static void json_escape(StrBuf* sb, const char* s) {
    if (!s) return;
    size_t length = strlen(s);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)s[i];
        switch (c) {
            case '\\': sb_append(sb, "\\\\"); break;
            case '"':  sb_append(sb, "\\\""); break;
            case '\n': sb_append(sb, "\\n"); break;
            case '\r': sb_append(sb, "\\r"); break;
            case '\t': sb_append(sb, "\\t"); break;
            default:
                if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);
                else if (c < 0x80) sb_appendn(sb, &s[i], 1);
                else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&s[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &s[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "\\u%04x", (unsigned)c);
                    }
                }
        }
    }
}

static void json_key(StrBuf* sb, const char* k) {
    sb_append(sb, "\"");
    json_escape(sb, k);
    sb_append(sb, "\":");
}

static void json_string(StrBuf* sb, const char* s) {
    sb_append(sb, "\"");
    json_escape(sb, s ? s : "");
    sb_append(sb, "\"");
}

```

```
295 | static void json_string_field(StrBuf* sb, const char* k, const char* v, bool trailing_comma) {
296 |     json_key(sb, k);
297 |     json_string(sb, v);
298 |     if (trailing_comma) sb_append(sb, ",");
299 | }
300 |
301 | static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma) {
302 |     json_key(sb, k);
303 |     sb_append(sb, v ? "true" : "false");
304 |     if (trailing_comma) sb_append(sb, ",");
305 | }
306 |
307 | static void json_heuristic_families(StrBuf* sb, uint32_t mask) {
308 |     sb_append(sb, "[");
309 |     bool first = true;
310 |     uint32_t bits[] = {
311 |         HEF_TABLE_INTEGRITY,
312 |         HEF_UNWIND_INTEGRITY,
313 |         HEF_METADATA_PLACEMENT,
314 |         HEF_UNWIND_REUSE,
315 |         HEF_HANDLER_DOMINANCE
316 |     };
317 |     for (size_t i = 0; i < sizeof(bits)/sizeof(bits[0]); i++) {
318 |         if (!(mask & bits[i])) continue;
319 |         if (!first) sb_append(sb, ",");
320 |         json_string(sb, heuristics_evidence_family_name(bits[i]));
321 |         first = false;
322 |     }
323 |     sb_append(sb, "]");
324 | }
325 |
326 | static unsigned dynamic_signal_count(const DynamicReport* dr) {
327 |     unsigned n = 0;
328 |     if (!dr) return 0;
329 |     if (dr->unwind_handler_hits) n++;
330 |     if (dr->veh_hits || dr->veh_registrations) n++;
331 |     if (dr->vch_hits || dr->vch_registrations) n++;
332 |     if (dr->uef_hits || dr->uef_set_calls) n++;
333 |     if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) n++;
334 |     if (dr->second_chance_seen) n++;
335 |     return n;
336 | }
337 |
338 | static bool dynamic_session_complete(const DynamicReport* dr) {
339 |     return dr &&
340 |            dr->ran &&
341 |            dr->ok &&
342 |            !dr->timed_out &&
343 |            dr->target_exited &&
344 |            dr->instrumentation_complete;
345 | }
346 |
347 | static bool has_static_dynamic_correlation(const HeuristicReport* hr, const DynamicReport* dr) {
348 |     bool suspicious_static_handler_context = false;
349 |     if (!hr || !dynamic_session_complete(dr)) return false;
350 |     suspicious_static_handler_context =
351 |         (hr->handler_nonexec_section > 0) ||
352 |         (hr->handler_writable_section > 0) ||
353 |         ((hr->evidence_families_mask & HEF_HANDLER_DOMINANCE) != 0u);
354 |     if (suspicious_static_handler_context && (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits)) return true;
355 |     if ((hr->unwind_exec_section || hr->unwind_unusual_section || hr->unwind_writable_section ||
356 |          hr->handler_nonexec_section || hr->handler_writable_section) &&
357 |         (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls)) return true;
358 |     return false;
359 | }
360 |
361 |
362 | static const char* runtime_evidence_level(const DynamicReport* dr) {
363 |     if (!dr || !dr->ran) return "none";
364 |     unsigned strong = 0, medium = 0, weak = 0;
365 |     if (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits) strong++;
366 |     if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) medium++;
367 |     if (dr->second_chance_seen) medium++;
368 |     if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls || dr->first_chance_seen) weak++;
369 |     const char* level = "none";
370 |     if (strong >= 1 && medium >= 1) level = "strong";
371 |     else if (strong >= 1 || medium >= 2) level = "medium";
372 |     else if (medium >= 1 || weak >= 1) level = "weak";
373 |     if (!dynamic_session_complete(dr) && strcmp(level, "none") != 0) return "weak";
374 |     return level;
375 | }
376 |
377 | static const char* case_strength(const HeuristicReport* hr, const DynamicReport* dr, bool correlated) {
378 |     const char* static_lvl = (hr && hr->evidence_level[0]) ? hr->evidence_level : "none";
379 |     const char* runtime_lvl = runtime_evidence_level(dr);
380 |     bool static_strong = strcmp(static_lvl, "strong") == 0;
381 |     bool static_medium = strcmp(static_lvl, "medium") == 0;
382 |     bool static_weak = strcmp(static_lvl, "weak") == 0;
383 |     bool runtime_strong = strcmp(runtime_lvl, "strong") == 0;
384 |     bool runtime_medium = strcmp(runtime_lvl, "medium") == 0;
385 |     bool runtime_weak = strcmp(runtime_lvl, "weak") == 0;
386 |
387 |     if (correlated && (static_strong || runtime_strong)) return "strong_correlated_case";
388 |     if (correlated && (static_medium || runtime_medium)) return "medium_correlated_case";
389 |     if (static_strong || runtime_strong) return "strong_unconfirmed_case";
390 |     if (static_medium || runtime_medium) return "medium_case";
391 |     if (static_weak || runtime_weak) return "weak_case";
392 |     return "no_case";
393 | }
394 |
395 | typedef struct AssessmentSummary {
396 |     bool correlated;
397 |     unsigned dynamic_signals;
398 |     double final_confidence;
399 |     const char* final_verdict;
400 |     const char* static_evidence_level;
401 |     const char* runtime_evidence_level;
402 |     const char* strength;
403 | } AssessmentSummary;
404 |
405 | static AssessmentSummary assessment_summary(const HeuristicReport* hr,
406 |                                             const DynamicReport* dr) {
407 |     AssessmentSummary summary;
408 |     summary.correlated = has_static_dynamic_correlation(hr, dr);
409 |     summary.dynamic_signals = dynamic_signal_count(dr);
```

```
    summary.final_confidence = hr ? hr->confidence : 0.10;
    summary.final_verdict = "no_strong_correlated_evidence";
    summary.static_evidence_level = hr ? hr->evidence_level : "none";
    summary.runtime_evidence_level = runtime_evidence_level(dr);
    summary.strength = case_strength(hr, dr, summary.correlated);

    if (dynamic_session_complete(dr) && summary.dynamic_signals) {
        summary.final_confidence += 0.10;
    }
    if (summary.correlated) summary.final_confidence += 0.10;
    if (summary.final_confidence > 0.98) summary.final_confidence = 0.98;

    if (hr) {
        if (hr->score >= 70) {
            summary.final_verdict = summary.correlated
                ? "high_suspicion_correlated"
                : "high_suspicion_static";
        } else if (hr->score >= 40) {
            summary.final_verdict = summary.correlated
                ? "moderate_suspicion_correlated"
                : "moderate_suspicion_static";
        } else if (hr->score >= 15) {
            summary.final_verdict = summary.correlated
                ? "low_suspicion_correlated"
                : "low_suspicion_static";
        } else if (dynamic_session_complete(dr) && summary.dynamic_signals) {
            summary.final_verdict = "runtime_signals_only";
        }
    } else if (dynamic_session_complete(dr) && summary.dynamic_signals) {
        summary.final_verdict = "runtime_signals_only";
    }
    return summary;
}

static bool heuristics_has_finding_code(const HeuristicReport* hr, const char* code) {
    if (!hr || !code) return false;
    for (size_t i = 0; i < hr->finding_count; i++) {
        if (strcmp(hr->findings[i].code, code) == 0) return true;
    }
    return false;
}

static bool heuristics_has_family(const HeuristicReport* hr, const char* family) {
    if (!hr || !family) return false;
    for (size_t i = 0; i < hr->finding_count; i++) {
        if (strcmp(hr->findings[i].family, family) == 0) return true;
    }
    return false;
}

typedef struct ManualValidationSteps {
    const char* items[16];
    size_t count;
} ManualValidationSteps;

static ManualValidationSteps collect_manual_validation_steps(const HeuristicReport* hr,
                                                              const DynamicReport* dr) {
    ManualValidationSteps steps;
    memset(&steps, 0, sizeof(steps));
#define ADD_STEP(txt) do { \
    if (steps.count < sizeof(steps.items) / sizeof(steps.items[0])) { \
        steps.items[steps.count++] = (txt); \
    } \
} while (0)
    if (hr) {
        if (hr->rf_invalid_ranges || hr->rf_oob || hr->rf_unwind_oob || hr->rf_overlaps) {
            ADD_STEP("Validate .pdata range integrity in a PE viewer, sort the RUNTIME_FUNCTION entries manually and confirm that
   begin/end/unwind RVAs stay inside SizeOfImage.");
        }
        if (hr->unwind_parse_fail || hr->unwind_anomalies || heuristics_has_family(hr, "unwind_integrity")) {
            ADD_STEP("Open the suspicious UNWIND_INFO blob in a hex view, verify Version/Flags/CountOfCodes/tail layout, then compare
   the decoded prolog actions with the function prolog in Ghidra.");
        }
        if (hr->unwind_unusual_section || hr->unwind_exec_section || hr->unwind_writable_section || heuristics_has_family(hr,
   "metadata_placement")) {
            ADD_STEP("Inspect the section hosting the unwind metadata, record its RVA/raw offset/permissions and decide whether the
   placement matches a benign compiler layout or looks redirected/manual.");
        }
        if (hr->handler_nonexec_section) {
            ADD_STEP("Disassemble the reported handler target, confirm execute permission and check whether the landing bytes belong
   to a real handler, a thunk or a non-code blob.");
        }
        if (hr->unwind_writable_section || hr->handler_writable_section) {
            ADD_STEP("Check whether writable unwind metadata or writable handler targets are justified by a JIT/runtime scenario; if
   not, treat the placement as suspicious and correlate it with memory protections at runtime.");
        }
        if (heuristics_has_finding_code(hr, "handler_dominance") || heuristics_has_family(hr, "handler_dominance")) {
            ADD_STEP("Open the dominant handler target in Ghidra and compare several referencing functions to decide whether this is
   a normal runtime helper or a suspicious central dispatcher.");
        }
        if (hr->mitigation_count > 0) {
            ADD_STEP("Review weakening rules before making a final conclusion; the current evidence set still has benign explanations
   that must be excluded manually.");
        }
    }
    if (dr) {
        if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) {
            ADD_STEP("Capture the arguments of RtlAddFunctionTable / RtlInstallFunctionTableCallback and inspect the runtime table
   contents together with the surrounding module/section mapping.");
        }
        if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls) {
            ADD_STEP("List each VEH/VCH/UEF registration with its target address, then single-step from the registered routine to
   confirm whether it behaves like a benign runtime hook or an exception interceptor.");
        }
        if (dr->veh_hits || dr->vch_hits || dr->uef_hits) {
            ADD_STEP("Single-step from the handler hit and map module/section transitions immediately after the exception dispatch
   point.");
        }
        if (dr->unwind_handler_hits) {
            ADD_STEP("Verify whether the hit unwind handler corresponds to a known language/runtime handler or to a
   dispatcher/interceptor path referenced by many protected functions.");
        }
    }
    if ((!dr || dynamic_signal_count(dr) == 0) && hr && (hr->score > 0 || hr->finding_count > 0)) {
        ADD_STEP("Run the sample again with --dynamic on a representative exception path to obtain runtime corroboration.");
```

```
    }
#undef ADD_STEP
    return steps;
}

static void json_manual_validation_steps(StrBuf* sb, const HeuristicReport* hr, const DynamicReport* dr) {
    ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);
    sb_append(sb, "[");
    for (size_t i = 0; i < steps.count; i++) {
        if (i) sb_append(sb, ",");
        json_string(sb, steps.items[i]);
    }
    sb_append(sb, "]");
}

void report_print_heuristics(const HeuristicReport* hr) {
    if (!hr) return;
    printf("\n[Heuristic Analysis] Suspicion score: %d / 100\n", hr->score);
    printf("  suspicion=%.3f  confidence=%.3f  verdict=%s  manual_review=%s\n",
           hr->suspicion, hr->confidence,
           hr->verdict[0] ? hr->verdict : "-",
           hr->requires_manual_review ? "yes" : "no");
    printf("  total=%zu  invalid_ranges=%zu  oob=%zu  unwind_oob=%zu\n",
           hr->total, hr->rf_invalid_ranges, hr->rf_oob, hr->rf_unwind_oob);
    printf("  unsorted=%zu  overlaps=%zu  duplicate_begins=%zu  unwind_parse_fail=%zu  unwind_anom=%zu\n",
           hr->rf_unsorted, hr->rf_overlaps, hr->rf_duplicate_begins,
           hr->unwind_parse_fail, hr->unwind_anomalies);
    printf("  strict_abi_warnings=%zu\n", hr->unwind_abi_warnings);
    printf("  unwind_unusual_section=%zu  unwind_exec_section=%zu  unwind_writable_section=%zu\n",
           hr->unwind_unusual_section, hr->unwind_exec_section, hr->unwind_writable_section);
    printf("  handler_nonexec_section=%zu  handler_writable_section=%zu\n",
           hr->handler_nonexec_section, hr->handler_writable_section);
    printf("  unwind_reuse_ratio=%.3f  unwind_unique_ratio=%.3f\n",
           hr->unwind_reuse_ratio, hr->unwind_unique_ratio);
    if (hr->handler_count) {
        printf("  handler_count=%zu  handler_reuse_ratio=%.3f\n",
               hr->handler_count, hr->handler_reuse_ratio);
    }
    printf("  evidence_families=%zu  findings=%zu\n",
           heuristics_evidence_family_count(hr->evidence_families_mask), hr->finding_count);
    printf("Findings:\n%s\n", hr->details.s ? hr->details.s : "");
}

static bool report_metadata_valid(const ReportMetadata* metadata) {
    if (!metadata ||
        !metadata->tool_version || metadata->tool_version[0] == '\0' ||
        !metadata->report_schema_version ||
        strcmp(metadata->report_schema_version, EHUA_REPORT_SCHEMA_VERSION) != 0 ||
        !metadata->generated_at_utc || metadata->generated_at_utc[0] == '\0' ||
        !metadata->input_sha256 || strlen(metadata->input_sha256) != 64u ||
        !metadata->command_line ||
        !metadata->build_compiler || metadata->build_compiler[0] == '\0' ||
        !metadata->build_platform || metadata->build_platform[0] == '\0' ||
        !metadata->build_mode ||
        (strcmp(metadata->build_mode, "debug") != 0 &&
         strcmp(metadata->build_mode, "release") != 0) ||
        metadata->max_chain_depth < 1u ||
        metadata->max_chain_depth > 16u) {
        return false;
    }
    for (size_t i = 0; i < 64u; i++) {
        const char c = metadata->input_sha256[i];
        if (!((c >= '0' && c <= '9') || (c >= 'a' && c <= 'f'))) return false;
    }
    if (strlen(metadata->generated_at_utc) != 20u ||
        metadata->generated_at_utc[4] != '-' ||
        metadata->generated_at_utc[7] != '-' ||
        metadata->generated_at_utc[10] != 'T' ||
        metadata->generated_at_utc[13] != ':' ||
        metadata->generated_at_utc[16] != ':' ||
        metadata->generated_at_utc[19] != 'Z') {
        return false;
    }
    for (size_t i = 0; i < 19u; i++) {
        if (i == 4u || i == 7u || i == 10u || i == 13u || i == 16u) continue;
        if (metadata->generated_at_utc[i] < '0' ||
            metadata->generated_at_utc[i] > '9') {
            return false;
        }
    }
    return true;
}

static bool report_model_valid(const PeImage* img,
                               const HeuristicReport* hr,
                               const DynamicReport* dr) {
    if (!img || (img->num_sections && !img->sections)) return false;
    if (hr && (hr->finding_count > HEURISTICS_MAX_FINDINGS ||
               hr->mitigation_count > HEURISTICS_MAX_MITIGATIONS)) {
        return false;
    }
    if (dr && (dr->event_count > DYNAMIC_MAX_EVENTS ||
               dr->trace_count > sizeof(dr->trace) / sizeof(dr->trace[0]) ||
               dr->trace_steps > sizeof(dr->trace) / sizeof(dr->trace[0]))) {
        return false;
    }
    if (dr && dr->instrumentation_complete &&
        (dr->module_refresh_failures ||
         dr->module_list_truncations ||
         dr->breakpoint_set_failures ||
         dr->breakpoint_capacity_drops ||
         dr->pending_rearm_failures ||
         dr->breakpoint_rearm_failures ||
         dr->dropped_events)) {
        return false;
    }
    return true;
}

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
```

```
628 |                       size_t n,
629 |                       const HeuristicReport* hr,
630 |                       const DynamicReport* dr,
631 |                       const ReportMetadata* metadata,
632 |                       StrBuf* err) {
633 |     if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
634 |         !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
635 |         sb_append(err, "Invalid arguments or metadata for JSON report.\n");
636 |         return false;
637 |     }
638 |
639 |     StrBuf sb;
640 |     if (!sb_init(&sb, 8192)) { sb_append(err, "OOM.\n"); return false; }
641 |     StrBuf comment;
642 |     if (!sb_init(&comment, 512)) { sb_free(&sb); sb_append(err, "OOM.\n"); return false; }
643 |     StrBuf anom;
644 |     if (!sb_init(&anom, 128)) { sb_free(&comment); sb_free(&sb); sb_append(err, "OOM.\n"); return false; }
645 |
646 |     sb_append(&sb, "{");
647 |     json_string_field(&sb, "input", input_path, true);
648 |     json_key(&sb, "report_metadata");
649 |     sb_append(&sb, "{");
650 |     json_string_field(&sb, "schema_version", metadata->report_schema_version, true);
651 |     json_string_field(&sb, "tool_version", metadata->tool_version, true);
652 |     json_string_field(&sb, "generated_at_utc", metadata->generated_at_utc, true);
653 |     json_string_field(&sb, "input_sha256", metadata->input_sha256, true);
654 |     json_key(&sb, "input_size"); sb_appendf(&sb, "%llu,", (unsigned long long)metadata->input_size);
655 |     json_string_field(&sb, "command_line", metadata->command_line, true);
656 |     json_string_field(&sb, "build_compiler", metadata->build_compiler, true);
657 |     json_string_field(&sb, "build_platform", metadata->build_platform, true);
658 |     json_string_field(&sb, "build_mode", metadata->build_mode, true);
659 |     json_bool_field(&sb, "strict_abi", metadata->strict_abi, true);
660 |     json_bool_field(&sb, "follow_chain", metadata->follow_chain, true);
661 |     json_key(&sb, "max_chain_depth");
662 |     sb_appendf(&sb, "%u", (unsigned)metadata->max_chain_depth);
663 |     sb_append(&sb, "},");
664 |     json_key(&sb, "image_base"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)img->image_base);
665 |     json_key(&sb, "size_of_image"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_image);
666 |     json_key(&sb, "size_of_headers"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_headers);
667 |     json_key(&sb, "section_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->section_alignment);
668 |     json_key(&sb, "file_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->file_alignment);
669 |     json_key(&sb, "exception_directory");
670 |     sb_appendf(&sb, "{\"rva\":\"0x%08x\",\"size\":\"0x%x\"}",
671 |                (unsigned)img->exception_rva, (unsigned)img->exception_size);
672 |     sb_append(&sb, ",");
673 |     json_key(&sb, "pe_validation");
674 |     sb_append(&sb, "{");
675 |     json_key(&sb, "section_raw_oob"); sb_appendf(&sb, "%zu,", img->section_raw_oob);
676 |     json_key(&sb, "section_virtual_oob"); sb_appendf(&sb, "%zu,", img->section_virtual_oob);
677 |     json_key(&sb, "section_raw_overlaps"); sb_appendf(&sb, "%zu,", img->section_raw_overlaps);
678 |     json_key(&sb, "section_virtual_overlaps"); sb_appendf(&sb, "%zu,", img->section_virtual_overlaps);
679 |     json_key(&sb, "section_misaligned"); sb_appendf(&sb, "%zu,", img->section_misaligned);
680 |     json_bool_field(&sb, "exception_directory_oob", img->exception_directory_oob, false);
681 |     sb_append(&sb, "},");
682 |     json_key(&sb, "sections");
683 |     sb_append(&sb, "[");
684 |     for (size_t i = 0; i < img->num_sections; i++) {
685 |         const PeSection* section = &img->sections[i];
686 |         if (i) sb_append(&sb, ",");
687 |         sb_append(&sb, "{");
688 |         json_string_field(&sb, "name", section->name, true);
689 |         json_key(&sb, "virtual_address"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->virtual_address);
690 |         json_key(&sb, "virtual_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->virtual_size);
691 |         json_key(&sb, "raw_offset"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_ptr);
692 |         json_key(&sb, "raw_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_size);
693 |         json_key(&sb, "characteristics"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->characteristics);
694 |         json_bool_field(&sb, "readable", (section->characteristics & IMAGE_SCN_MEM_READ) != 0, true);
695 |         json_bool_field(&sb, "writable", (section->characteristics & IMAGE_SCN_MEM_WRITE) != 0, true);
696 |         json_bool_field(&sb, "executable", (section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0, false);
697 |         sb_append(&sb, "}");
698 |     }
699 |     sb_append(&sb, "],");
700 |
701 |     if (hr) {
702 |         json_key(&sb, "heuristic_analysis");
703 |         sb_append(&sb, "{");
704 |         json_key(&sb, "score"); sb_appendf(&sb, "%d,", hr->score);
705 |         json_key(&sb, "suspicion"); sb_appendf(&sb, "%.6f,", hr->suspicion);
706 |         json_key(&sb, "confidence"); sb_appendf(&sb, "%.6f,", hr->confidence);
707 |         json_bool_field(&sb, "requires_manual_review", hr->requires_manual_review, true);
708 |         json_string_field(&sb, "verdict", hr->verdict, true);
709 |         json_string_field(&sb, "evidence_level", hr->evidence_level, true);
710 |         json_key(&sb, "weak_finding_count"); sb_appendf(&sb, "%zu,", hr->weak_finding_count);
711 |         json_key(&sb, "medium_finding_count"); sb_appendf(&sb, "%zu,", hr->medium_finding_count);
712 |         json_key(&sb, "strong_finding_count"); sb_appendf(&sb, "%zu,", hr->strong_finding_count);
713 |         json_key(&sb, "total"); sb_appendf(&sb, "%zu,", hr->total);
714 |         json_key(&sb, "pe_section_raw_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_oob);
715 |         json_key(&sb, "pe_section_virtual_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_oob);
716 |         json_key(&sb, "pe_section_raw_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_overlaps);
717 |         json_key(&sb, "pe_section_virtual_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_overlaps);
718 |         json_key(&sb, "pe_section_misaligned"); sb_appendf(&sb, "%zu,", hr->pe_section_misaligned);
719 |         json_bool_field(&sb, "pe_exception_directory_oob", hr->pe_exception_directory_oob, true);
720 |         json_key(&sb, "rf_invalid_ranges"); sb_appendf(&sb, "%zu,", hr->rf_invalid_ranges);
721 |         json_key(&sb, "rf_oob"); sb_appendf(&sb, "%zu,", hr->rf_oob);
722 |         json_key(&sb, "rf_unwind_oob"); sb_appendf(&sb, "%zu,", hr->rf_unwind_oob);
723 |         json_key(&sb, "rf_unsorted"); sb_appendf(&sb, "%zu,", hr->rf_unsorted);
724 |         json_key(&sb, "rf_overlaps"); sb_appendf(&sb, "%zu,", hr->rf_overlaps);
725 |         json_key(&sb, "rf_duplicate_begins"); sb_appendf(&sb, "%zu,", hr->rf_duplicate_begins);
726 |         json_key(&sb, "unwind_parse_fail"); sb_appendf(&sb, "%zu,", hr->unwind_parse_fail);
727 |         json_key(&sb, "unwind_anomalies"); sb_appendf(&sb, "%zu,", hr->unwind_anomalies);
728 |         json_key(&sb, "unwind_abi_warnings"); sb_appendf(&sb, "%zu,", hr->unwind_abi_warnings);
729 |         json_key(&sb, "unwind_unusual_section"); sb_appendf(&sb, "%zu,", hr->unwind_unusual_section);
730 |         json_key(&sb, "unwind_exec_section"); sb_appendf(&sb, "%zu,", hr->unwind_exec_section);
731 |         json_key(&sb, "unwind_writable_section"); sb_appendf(&sb, "%zu,", hr->unwind_writable_section);
732 |         json_key(&sb, "handler_nonexec_section"); sb_appendf(&sb, "%zu,", hr->handler_nonexec_section);
733 |         json_key(&sb, "handler_writable_section"); sb_appendf(&sb, "%zu,", hr->handler_writable_section);
734 |         json_key(&sb, "unwind_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_reuse_ratio);
735 |         json_key(&sb, "unwind_unique_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_unique_ratio);
736 |         json_key(&sb, "handler_count"); sb_appendf(&sb, "%zu,", hr->handler_count);
737 |         json_key(&sb, "handler_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->handler_reuse_ratio);
738 |         json_key(&sb, "evidence_family_count"); sb_appendf(&sb, "%zu,",
    heuristics_evidence_family_count(hr->evidence_families_mask));
739 |         json_key(&sb, "evidence_families"); json_heuristic_families(&sb, hr->evidence_families_mask); sb_append(&sb, ",");
740 |         json_key(&sb, "findings");
741 |         sb_append(&sb, "[");
```

```
742 |         for (size_t i = 0; i < hr->finding_count; i++) {
743 |             if (i) sb_append(&sb, ",");
744 |             sb_append(&sb, "{");
745 |             json_string_field(&sb, "code", hr->findings[i].code, true);
746 |             json_string_field(&sb, "family", hr->findings[i].family, true);
747 |             json_string_field(&sb, "severity", hr->findings[i].severity, true);
748 |             json_string_field(&sb, "message", hr->findings[i].message, true);
749 |             json_key(&sb, "affected_count"); sb_appendf(&sb, "%zu,", hr->findings[i].affected_count);
750 |             json_key(&sb, "representative_rva");
751 |             if (hr->findings[i].has_location) {
752 |                 sb_appendf(&sb, "\"0x%08x\",", (unsigned)hr->findings[i].representative_rva);
753 |             } else {
754 |                 sb_append(&sb, "null,");
755 |             }
756 |             json_string_field(
757 |                 &sb,
758 |                 "representative_section",
759 |                 hr->findings[i].has_location
760 |                     ? section_name_or_dash(img, hr->findings[i].representative_rva)
761 |                     : "-",
762 |                 false);
763 |             sb_append(&sb, "}");
764 |         }
765 |         sb_append(&sb, "],");
766 |         json_key(&sb, "weakening_rules_applied");
767 |         sb_append(&sb, "[");
768 |         for (size_t i = 0; i < hr->mitigation_count; i++) {
769 |             if (i) sb_append(&sb, ",");
770 |             sb_append(&sb, "{");
771 |             json_string_field(&sb, "code", hr->mitigations[i].code, true);
772 |             json_string_field(&sb, "message", hr->mitigations[i].message, false);
773 |             sb_append(&sb, "}");
774 |         }
775 |         sb_append(&sb, "],");
776 |         json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr->mitigation_count);
777 |         json_string_field(&sb, "details", hr->details.s ? hr->details.s : "", false);
778 |         sb_append(&sb, "},");
779 |     }
780 |
781 |     if (dr) {
782 |         json_key(&sb, "dynamic_validation");
783 |         sb_append(&sb, "{");
784 |         json_bool_field(&sb, "enabled", dr->enabled, true);
785 |         json_bool_field(&sb, "ran", dr->ran, true);
786 |         json_bool_field(&sb, "ok", dr->ok, true);
787 |         json_bool_field(&sb, "timed_out", dr->timed_out, true);
788 |         json_bool_field(&sb, "target_exited", dr->target_exited, true);
789 |         json_key(&sb, "target_exit_code"); sb_appendf(&sb, "%u,", (unsigned)dr->target_exit_code);
790 |         json_key(&sb, "timeout_ms"); sb_appendf(&sb, "%u,", (unsigned)dr->timeout_ms);
791 |         json_key(&sb, "trace_steps"); sb_appendf(&sb, "%u,", (unsigned)dr->trace_steps);
792 |         json_bool_field(&sb, "instrumentation_complete", dr->instrumentation_complete, true);
793 |         json_key(&sb, "modules_observed_peak"); sb_appendf(&sb, "%u,", (unsigned)dr->modules_observed_peak);
794 |         json_key(&sb, "module_refresh_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->module_refresh_failures);
795 |         json_key(&sb, "module_list_truncations"); sb_appendf(&sb, "%u,", (unsigned)dr->module_list_truncations);
796 |         json_key(&sb, "unwind_handlers_discovered"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handlers_discovered);
797 |         json_key(&sb, "breakpoint_set_attempts"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_attempts);
798 |         json_key(&sb, "breakpoint_set_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_failures);
799 |         json_key(&sb, "breakpoint_capacity_drops"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_capacity_drops);
800 |         json_key(&sb, "pending_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->pending_rearm_failures);
801 |         json_key(&sb, "breakpoint_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_rearm_failures);
802 |         json_key(&sb, "exceptions_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->exceptions_seen);
803 |         json_key(&sb, "first_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->first_chance_seen);
804 |         json_key(&sb, "second_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->second_chance_seen);
805 |         json_key(&sb, "unwind_handler_bp_set"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_bp_set);
806 |         json_key(&sb, "unwind_handler_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_hits);
807 |         json_key(&sb, "veh_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_registrations);
808 |         json_key(&sb, "veh_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_hits);
809 |         json_key(&sb, "vch_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_registrations);
810 |         json_key(&sb, "vch_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_hits);
811 |         json_key(&sb, "uef_set_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_set_calls);
812 |         json_key(&sb, "uef_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_hits);
813 |         json_key(&sb, "rtl_add_function_table_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->rtl_add_function_table_calls);
814 |         json_key(&sb, "rtl_install_function_table_cb_calls"); sb_appendf(&sb, "%u,",
    (unsigned)dr->rtl_install_function_table_cb_calls);
815 |         json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", dynamic_signal_count(dr));
816 |         json_key(&sb, "dropped_events"); sb_appendf(&sb, "%u,", (unsigned)dr->dropped_events);
817 |         json_string_field(&sb, "details", dr->details.s ? dr->details.s : "", true);
818 |         json_key(&sb, "events");
819 |         sb_append(&sb, "[");
820 |         for (uint32_t i = 0; i < dr->event_count; i++) {
821 |             const DynamicEvent* event = &dr->events[i];
822 |             if (i) sb_append(&sb, ",");
823 |             sb_append(&sb, "{");
824 |             json_string_field(&sb, "type", event->type, true);
825 |             json_string_field(&sb, "module", event->module, true);
826 |             json_key(&sb, "thread_id"); sb_appendf(&sb, "%u,", (unsigned)event->thread_id);
827 |             json_key(&sb, "exception_code"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)event->exception_code);
828 |             json_bool_field(&sb, "first_chance", event->first_chance, true);
829 |             json_key(&sb, "address"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->address);
830 |             json_key(&sb, "arg0"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg0);
831 |             json_key(&sb, "arg1"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg1);
832 |             json_key(&sb, "arg2"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg2);
833 |             json_key(&sb, "arg3"); sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)event->arg3);
834 |             sb_append(&sb, "}");
835 |         }
836 |         sb_append(&sb, "],");
837 |         json_key(&sb, "trace");
838 |         sb_append(&sb, "[");
839 |         for (uint32_t i = 0; i < dr->trace_count; i++) {
840 |             if (i) sb_append(&sb, ",");
841 |             sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)dr->trace[i]);
842 |         }
843 |         sb_append(&sb, "]");
844 |         sb_append(&sb, "},");
845 |     }
846 |
847 |     if (hr || dr) {
848 |         AssessmentSummary summary = assessment_summary(hr, dr);
849 |
850 |         json_key(&sb, "assessment");
851 |         sb_append(&sb, "{");
852 |         json_string_field(&sb, "final_verdict", summary.final_verdict, true);
853 |         json_key(&sb, "final_confidence"); sb_appendf(&sb, "%.6f,", summary.final_confidence);
854 |         json_string_field(&sb, "static_evidence_level", summary.static_evidence_level, true);
855 |         json_string_field(&sb, "runtime_evidence_level", summary.runtime_evidence_level, true);
```

```
        json_string_field(&sb, "case_strength", summary.strength, true);
        json_bool_field(&sb, "static_dynamic_correlation", summary.correlated, true);
        json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", summary.dynamic_signals);
        json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr ? hr->mitigation_count : 0u);
        json_key(&sb, "manual_validation_steps"); json_manual_validation_steps(&sb, hr, dr);
        sb_append(&sb, "},");
    }

    json_key(&sb, "functions");
    sb_append(&sb, "[");
    for (size_t i = 0; i < n; i++) {
        if (i) sb_append(&sb, ",");
        if (us_ok[i]) build_unwind_comment(&us[i], &comment);
        else {
            comment.len = 0;
            if (comment.s) comment.s[0] = 0;
            sb_append(&comment, "UNWIND parse failed");
            if (us[i].parse_error[0]) sb_appendf(&comment, ": %s", us[i].parse_error);
        }
        anom.len = 0; if (anom.s) anom.s[0] = 0; anomalies_to_str(&anom, us[i].anomalies);

        sb_append(&sb, "{");
        json_key(&sb, "index"); sb_appendf(&sb, "%zu,", i);
        json_key(&sb, "begin_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].begin_rva);
        json_key(&sb, "end_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].end_rva);
        json_key(&sb, "size"); sb_appendf(&sb, "%u,", (unsigned)runtime_function_size(&rf[i]));
        json_key(&sb, "begin_section"); json_string(&sb, section_name_or_dash(img, rf[i].begin_rva)); sb_append(&sb, ",");
        json_key(&sb, "unwind_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].unwind_rva);
        json_key(&sb, "unwind_section"); json_string(&sb, section_name_or_dash(img, rf[i].unwind_rva)); sb_append(&sb, ",");
        json_bool_field(&sb, "unwind_ok", us_ok[i], true);
        json_string_field(&sb, "unwind_error", us[i].parse_error, true);
        json_key(&sb, "unwind");
        sb_append(&sb, "{");
        json_key(&sb, "version"); sb_appendf(&sb, "%u,", (unsigned)us[i].version);
        json_key(&sb, "flags"); sb_appendf(&sb, "%u,", (unsigned)us[i].flags);
        json_key(&sb, "prolog_size"); sb_appendf(&sb, "%u,", (unsigned)us[i].prolog_size);
        json_key(&sb, "count_codes"); sb_appendf(&sb, "%u,", (unsigned)us[i].count_codes);
        json_key(&sb, "fixed_stack_bytes"); sb_appendf(&sb, "%u,", (unsigned)us[i].fixed_stack_bytes);
        json_key(&sb, "frame_reg"); json_string(&sb, (us[i].frame_reg ? unwind_reg_name(us[i].frame_reg) : "-")); sb_append(&sb,
  ",");
        json_key(&sb, "frame_off"); sb_appendf(&sb, "%u,", (unsigned)(us[i].frame_off * 16u));
        json_bool_field(&sb, "has_handler", us[i].has_handler, true);
        json_key(&sb, "handler_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)us[i].handler_rva);
        json_key(&sb, "handler_section"); json_string(&sb, us[i].has_handler ? section_name_or_dash(img, us[i].handler_rva) : "-");
  sb_append(&sb, ",");
        json_bool_field(&sb, "has_chain", us[i].has_chain, true);
        json_key(&sb, "anomalies"); sb_appendf(&sb, "%u,", (unsigned)us[i].anomalies);
        json_string_field(&sb, "anomalies_text", anom.s ? anom.s : "0", true);
        json_key(&sb, "chain");
        if (us[i].has_chain) {
            sb_appendf(&sb, "{\"begin_rva\":\"0x%08x\",\"end_rva\":\"0x%08x\",\"unwind_rva\":\"0x%08x\"}",
                       (unsigned)us[i].chain.begin_rva,
                       (unsigned)us[i].chain.end_rva,
                       (unsigned)us[i].chain.unwind_rva);
        } else {
            sb_append(&sb, "null");
        }
        sb_append(&sb, "},");
        json_string_field(&sb, "comment", comment.s ? comment.s : "", false);
        sb_append(&sb, "}");
    }
    sb_append(&sb, "]}");

    bool ok = sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom);
    if (ok) ok = file_write_atomic(path, sb.s, sb.len);
    if (!ok) {
        sb_appendf(err, "%s: %s\n",
                   sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom)
                       ? "Failed to write JSON"
                       : "Out of memory while building JSON report",
                   path);
    }

    sb_free(&anom);
    sb_free(&comment);
    sb_free(&sb);
    return ok;
}

static void html_escape(StrBuf* sb, const char* text) {
    if (!text) return;
    size_t length = strlen(text);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)text[i];
        switch (c) {
            case '&': sb_append(sb, "&amp;"); break;
            case '<': sb_append(sb, "&lt;"); break;
            case '>': sb_append(sb, "&gt;"); break;
            case '"': sb_append(sb, "&quot;"); break;
            case '\'': sb_append(sb, "&#39;"); break;
            default:
                if (c < 0x80) {
                    sb_appendn(sb, &text[i], 1);
                } else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&text[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &text[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "&#x%02x;", (unsigned)c);
                    }
                }
                break;
        }
    }
}

static void html_text_cell(StrBuf* sb, const char* text) {
    sb_append(sb, "<td>");
    html_escape(sb, text ? text : "");
    sb_append(sb, "</td>");
}

static void html_hex32_cell(StrBuf* sb, uint32_t value) {
```

```
969 |     sb_appendf(sb, "<td><code>0x%08x</code></td>", (unsigned)value);
970 | }
971 |
972 | static void html_hex64_cell(StrBuf* sb, uint64_t value) {
973 |     sb_appendf(sb, "<td><code>0x%llx</code></td>", (unsigned long long)value);
974 | }
975 |
976 | static const char* html_score_class(const HeuristicReport* hr) {
977 |     if (!hr) return "neutral";
978 |     if (hr->score >= 70) return "danger";
979 |     if (hr->score >= 40) return "warn";
980 |     return "ok";
981 | }
982 |
983 | static const char* ru_yes_no(bool value) {
984 |     return value ? "да" : "нет";
985 | }
986 |
987 | static const char* ru_status_text(const DynamicReport* dr) {
988 |     if (!dr) return "не запрашивалась";
989 |     if (dr->timed_out) return "тайм-аут";
990 |     if (!dr->ran) return "не запускалась";
991 |     return dr->ok ? "завершена" : "ошибка";
992 | }
993 |
994 | static const char* ru_level_text(const char* value) {
995 |     if (!value) return "-";
996 |     if (strcmp(value, "strong") == 0) return "сильный";
997 |     if (strcmp(value, "medium") == 0) return "средний";
998 |     if (strcmp(value, "weak") == 0) return "слабый";
999 |     if (strcmp(value, "none") == 0) return "нет";
1000 |     return value;
1001 | }
1002 |
1003 | static const char* ru_severity_text(const char* value) {
1004 |     if (!value) return "-";
1005 |     if (strcmp(value, "critical") == 0) return "критическая";
1006 |     if (strcmp(value, "high") == 0) return "высокая";
1007 |     if (strcmp(value, "medium") == 0) return "средняя";
1008 |     if (strcmp(value, "low") == 0) return "низкая";
1009 |     if (strcmp(value, "info") == 0) return "информационная";
1010 |     return value;
1011 | }
1012 |
1013 | static const char* ru_verdict_text(const char* value) {
1014 |     if (!value) return "-";
1015 |     if (strcmp(value, "high_suspicion") == 0) return "высокая подозрительность";
1016 |     if (strcmp(value, "moderate_suspicion") == 0) return "умеренная подозрительность";
1017 |     if (strcmp(value, "low_suspicion") == 0) return "низкая подозрительность";
1018 |     if (strcmp(value, "no_strong_static_anomalies") == 0) return "сильные статические аномалии не обнаружены";
1019 |     if (strcmp(value, "high_suspicion_correlated") == 0) return "высокая подозрительность с динамическим подтверждением";
1020 |     if (strcmp(value, "high_suspicion_static") == 0) return "высокая статическая подозрительность";
1021 |     if (strcmp(value, "moderate_suspicion_correlated") == 0) return "умеренная подозрительность с динамическим подтверждением";
1022 |     if (strcmp(value, "moderate_suspicion_static") == 0) return "умеренная статическая подозрительность";
1023 |     if (strcmp(value, "low_suspicion_correlated") == 0) return "низкая подозрительность с динамическим подтверждением";
1024 |     if (strcmp(value, "low_suspicion_static") == 0) return "низкая статическая подозрительность";
1025 |     if (strcmp(value, "runtime_signals_only") == 0) return "обнаружены только динамические сигналы";
1026 |     if (strcmp(value, "no_strong_correlated_evidence") == 0) return "сильная корреляция не обнаружена";
1027 |     return value;
1028 | }
1029 |
1030 | static const char* ru_case_strength_text(const char* value) {
1031 |     if (!value) return "-";
1032 |     if (strcmp(value, "strong_correlated_case") == 0) return "сильная, с корреляцией";
1033 |     if (strcmp(value, "medium_correlated_case") == 0) return "средняя, с корреляцией";
1034 |     if (strcmp(value, "strong_unconfirmed_case") == 0) return "сильная, без динамического подтверждения";
1035 |     if (strcmp(value, "medium_case") == 0) return "средняя";
1036 |     if (strcmp(value, "weak_case") == 0) return "слабая";
1037 |     if (strcmp(value, "no_case") == 0) return "значимых признаков нет";
1038 |     return value;
1039 | }
1040 |
1041 | static void html_append_manual_step_ru(StrBuf* sb, const char* step) {
1042 |     if (!step) return;
1043 |     if (strstr(step, "Validate .pdata range integrity")) {
1044 |         sb_append(sb, "Проверьте целостность диапазона .pdata в PE-viewer: вручную отсортируйте записи RUNTIME_FUNCTION и убедитесь,
    что Begin/End/Unwind RVA находятся внутри SizeOfImage.");
1045 |     } else if (strstr(step, "Open the suspicious UNWIND_INFO blob")) {
1046 |         sb_append(sb, "Откройте подозрительный блок UNWIND_INFO в hex-viewer, проверьте Version/Flags/CountOfCodes и tail-layout,
    затем сравните decoded prolog actions с prolog функции в Ghidra.");
1047 |     } else if (strstr(step, "Inspect the section hosting the unwind metadata")) {
1048 |         sb_append(sb, "Проверьте секцию, в которой размещены unwind-метаданные: зафиксируйте RVA, raw offset и права доступа, затем
    решите, похоже ли размещение на обычный compiler layout или на ручное перенаправление.");
1049 |     } else if (strstr(step, "Disassemble the reported handler target")) {
1050 |         sb_append(sb, "Дизассемблируйте указанный handler target, подтвердите execute permission и проверьте, являются ли landing
    bytes настоящим handler, thunk или областью не-кода.");
1051 |     } else if (strstr(step, "Check whether writable unwind metadata")) {
1052 |         sb_append(sb, "Проверьте, оправдано ли наличие writable unwind metadata или writable handler targets сценарием JIT/runtime;
    если нет, считайте размещение подозрительным и сопоставьте его с memory protections во время запуска.");
1053 |     } else if (strstr(step, "Open the dominant handler target")) {
1054 |         sb_append(sb, "Откройте dominant handler target в Ghidra и сравните несколько функций, которые на него ссылаются, чтобы
    отличить обычный runtime helper от подозрительного central dispatcher.");
1055 |     } else if (strstr(step, "Review weakening rules")) {
1056 |         sb_append(sb, "Перед окончательным выводом проверьте правила снижения уверенности: текущий набор признаков еще может иметь
    доброкачественные объяснения, которые нужно исключить вручную.");
1057 |     } else if (strstr(step, "Capture the arguments of RtlAddFunctionTable")) {
1058 |         sb_append(sb, "Зафиксируйте аргументы RtlAddFunctionTable / RtlInstallFunctionTableCallback и проверьте содержимое runtime
    table вместе с module/section mapping.");
1059 |     } else if (strstr(step, "List each VEH/VCH/UEF registration")) {
1060 |         sb_append(sb, "Перечислите каждую регистрацию VEH/VCH/UEF с target address, затем выполните single-step от зарегистрированной
    routine, чтобы отличить benign runtime hook от exception interceptor.");
1061 |     } else if (strstr(step, "Single-step from the handler hit")) {
1062 |         sb_append(sb, "Выполните single-step от handler hit и отобразите переходы между module/section сразу после exception dispatch
    point.");
1063 |     } else if (strstr(step, "Verify whether the hit unwind handler")) {
1064 |         sb_append(sb, "Проверьте, соответствует ли достигнутый unwind handler известному language/runtime handler или
    dispatcher/interceptor path, на который ссылается много защищенных функций.");
1065 |     } else if (strstr(step, "Run the sample again with --dynamic")) {
1066 |         sb_append(sb, "Запустите sample повторно с --dynamic на representative exception path, чтобы получить runtime
    corroboration.");
1067 |     } else {
1068 |         html_escape(sb, step);
1069 |     }
1070 | }
1071 |
```

```
1072 | static void html_append_finding_message_ru(StrBuf* sb, const HeuristicFinding* finding) {
1073 |     if (!finding || !finding->code[0]) return;
1074 |     const char* code = finding->code;
1075 |     size_t n = finding->affected_count;
1076 |     if (strcmp(code, "pe_section_raw_oob") == 0) {
1077 |         sb_append(sb, "Raw-диапазон секции PE выходит за границы файла.");
1078 |     } else if (strcmp(code, "pe_section_virtual_oob") == 0) {
1079 |         sb_append(sb, "Virtual-диапазон секции PE выходит за пределы SizeOfImage.");
1080 |     } else if (strcmp(code, "pe_section_raw_overlap") == 0) {
1081 |         sb_append(sb, "Raw-диапазоны секций PE пересекаются.");
1082 |     } else if (strcmp(code, "pe_section_virtual_overlap") == 0) {
1083 |         sb_append(sb, "Virtual-диапазоны секций PE пересекаются.");
1084 |     } else if (strcmp(code, "pe_section_misaligned") == 0) {
1085 |         sb_append(sb, "Обнаружена секция PE с нарушением выравнивания.");
1086 |     } else if (strcmp(code, "pe_exception_directory_oob") == 0) {
1087 |         sb_append(sb, "Exception directory поврежден, неполон, недоступен для чтения или выходит за допустимые границы.");
1088 |     } else if (strcmp(code, "no_runtime_functions") == 0) {
1089 |         sb_append(sb, "Записи RUNTIME_FUNCTION не найдены; это может быть допустимо для leaf-only image, поэтому сложность кода нужно
    подтвердить вручную.");
1090 |     } else if (strcmp(code, "invalid_ranges") == 0) {
1091 |         sb_appendf(sb, "Обнаружены записи RUNTIME_FUNCTION с некорректным диапазоном Begin/End: %zu.", n);
1092 |     } else if (strcmp(code, "rf_oob") == 0) {
1093 |         sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION выходят за пределы SizeOfImage: %zu.", n);
1094 |     } else if (strcmp(code, "unwind_rva_oob") == 0) {
1095 |         sb_appendf(sb, "UnwindInfoRVA отсутствует или выходит за допустимые границы: %zu.", n);
1096 |     } else if (strcmp(code, "rf_unsorted") == 0) {
1097 |         sb_appendf(sb, "Таблица RUNTIME_FUNCTION не отсортирована по BeginAddress: %zu.", n);
1098 |     } else if (strcmp(code, "rf_overlaps") == 0) {
1099 |         sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION пересекаются: %zu.", n);
1100 |     } else if (strcmp(code, "duplicate_begin") == 0) {
1101 |         sb_appendf(sb, "Несколько записей RUNTIME_FUNCTION используют один и тот же BeginAddress: %zu.", n);
1102 |     } else if (strcmp(code, "unwind_parse_fail") == 0) {
1103 |         sb_appendf(sb, "Ошибки разбора UNWIND_INFO: %zu.", n);
1104 |     } else if (strcmp(code, "unwind_anomalies") == 0) {
1105 |         sb_appendf(sb, "Аномалии UNWIND_INFO (version/opcode/chain/ABI и др.): %zu.", n);
1106 |     } else if (strcmp(code, "unwind_reuse_strong") == 0) {
1107 |         sb_appendf(sb, "Сильное повторное использование UNWIND_INFO: до %zu записей ссылаются на один UnwindInfoRVA.", n);
1108 |     } else if (strcmp(code, "handler_dominance") == 0) {
1109 |         sb_appendf(sb, "Один handler target доминирует: до %zu записей ссылаются на один HandlerRVA.", n);
1110 |     } else if (strcmp(code, "unwind_reuse_moderate") == 0) {
1111 |         sb_appendf(sb, "Умеренное повторное использование UNWIND_INFO подтверждается другим статическим признаком: до %zu записей
    ссылаются на один UnwindInfoRVA.", n);
1112 |     } else if (strcmp(code, "unwind_in_exec_section") == 0) {
1113 |         sb_appendf(sb, "UNWIND_INFO расположен в executable-секции: %zu.", n);
1114 |     } else if (strcmp(code, "unwind_unusual_section") == 0) {
1115 |         sb_appendf(sb, "UNWIND_INFO расположен в необычной секции, отличной от .xdata/.rdata/.pdata: %zu.", n);
1116 |     } else if (strcmp(code, "unwind_writable_section") == 0) {
1117 |         sb_appendf(sb, "UNWIND_INFO расположен в writable-секции: %zu.", n);
1118 |     } else if (strcmp(code, "handler_nonexec_section") == 0) {
1119 |         sb_appendf(sb, "Exception handler RVA указывает в non-executable или неизвестную секцию: %zu.", n);
1120 |     } else if (strcmp(code, "handler_writable_section") == 0) {
1121 |         sb_appendf(sb, "Exception handler RVA указывает в writable-секцию: %zu.", n);
1122 |     } else if (strcmp(code, "placement_exec_nonexec_handler_combo") == 0) {
1123 |         sb_append(sb, "Unwind metadata в executable-секции совпадает с handler targets в non-executable/unknown секции; такую
    комбинацию сложнее объяснить доброкачественно.");
1124 |     } else if (strcmp(code, "placement_integrity_combo") == 0) {
1125 |         sb_append(sb, "Подозрительное размещение metadata подтверждается проблемами целостности UNWIND_INFO / UnwindInfoRVA; вместе
    эти признаки сильнее, чем по отдельности.");
1126 |     } else if (strcmp(code, "reuse_dominance_combo") == 0) {
1127 |         sb_append(sb, "Активное повторное использование UNWIND_INFO совпадает с dominant handler target, что похоже на
    централизованный dispatcher-style metadata reuse.");
1128 |     } else if (strcmp(code, "writable_metadata_handler_combo") == 0) {
1129 |         sb_append(sb, "Writable unwind metadata совпадает с writable handler targets; такая комбинация нетипична для обычных
    compiler-produced images и требует внимательной проверки.");
1130 |     } else {
1131 |         html_escape(sb, finding->message);
1132 |     }
1133 | }
1134 |
1135 | static void html_append_mitigation_message_ru(StrBuf* sb, const char* code, const char* message) {
1136 |     if (!code) {
1137 |         html_escape(sb, message);
1138 |     } else if (strcmp(code, "single_signal_only") == 0) {
1139 |         sb_append(sb, "Вклад дал только один не-сильный признак; как самостоятельное доказательство его нужно считать слабым.");
1140 |     } else if (strcmp(code, "small_sample_size") == 0) {
1141 |         sb_append(sb, "Очень маленькая выборка .pdata может завышать доли; уверенность снижена до появления большего числа
    функций.");
1142 |     } else if (strcmp(code, "placement_only_signal") == 0) {
1143 |         sb_append(sb, "Аномалии только по размещению могут встречаться в необычных, но доброкачественных layout; нужна ручная
    проверка.");
1144 |     } else if (strcmp(code, "single_family_only") == 0) {
1145 |         sb_append(sb, "Признаки относятся только к одному семейству; перед сильным выводом желательно cross-layer corroboration.");
1146 |     } else {
1147 |         html_escape(sb, message);
1148 |     }
1149 | }
1150 |
1151 | bool report_write_html(const char* path,
1152 |                        const char* input_path,
1153 |                        const PeImage* img,
1154 |                        const RuntimeFunction* rf,
1155 |                        const UnwindSummary* us,
1156 |                        const bool* us_ok,
1157 |                        size_t n,
1158 |                        const HeuristicReport* hr,
1159 |                        const DynamicReport* dr,
1160 |                        const ReportMetadata* metadata,
1161 |                        StrBuf* err) {
1162 |     if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
1163 |         !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
1164 |         sb_append(err, "Invalid arguments or metadata for HTML report.\n");
1165 |         return false;
1166 |     }
1167 |
1168 |     StrBuf sb = {0};
1169 |     StrBuf anomalies = {0};
1170 |     if (!sb_init(&sb, 32768) ||
1171 |         !sb_init(&anomalies, 128)) {
1172 |         sb_free(&anomalies);
1173 |         sb_free(&sb);
1174 |         sb_append(err, "OOM.\n");
1175 |         return false;
1176 |     }
1177 |
1178 |     const char* tool_version = metadata->tool_version;
```

```
    const char* generated_at = metadata->generated_at_utc;
    const char* schema_version = metadata->report_schema_version;
    const char* input_hash = metadata->input_sha256;
    const char* build_platform = metadata->build_platform;
    const char* build_compiler = metadata->build_compiler;
    const char* build_mode = metadata->build_mode;
    size_t parse_failures = 0;
    size_t anomaly_count = 0;
    for (size_t i = 0; i < n; i++) {
        if (!us_ok[i]) parse_failures++;
        if (us[i].anomalies) anomaly_count++;
    }

    sb_append(&sb, "<!doctype html><html lang=\"ru\"><head><meta charset=\"utf-8\">");
    sb_append(&sb, "<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
    sb_append(&sb, "<meta http-equiv=\"Content-Security-Policy\" content=\"default-src 'none';"
                   "style-src 'unsafe-inline';script-src 'unsafe-inline';img-src 'none';"
                   "base-uri 'none';form-action 'none'\">");
    sb_append(&sb, "<title>Отчет EH/Unwind Analyzer - ");
    html_escape(&sb, input_path ? input_path : "");
    sb_append(&sb, "</title><style>");
    sb_append(&sb,
        ":root{color-scheme:dark;--bg:#0b1020;--panel:#121a2c;--panel2:#172238;"
        "--text:#e6edf7;--muted:#95a3b8;--line:#2a3851;--accent:#71b7ff;"
        "--ok:#54d99a;--warn:#f7c65d;--danger:#ff718b}"
        "*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--text);"
        "font:14px/1.5 system-ui,-apple-system,Segoe UI,sans-serif}"
        "main{max-width:1500px;margin:auto;padding:28px}h1{margin:0;font-size:28px}"
        "h2{margin:0 0 14px;font-size:19px}p{margin:6px 0}.muted{color:var(--muted)}"
        ".hero,.panel{background:var(--panel);border:1px solid var(--line);border-radius:14px;"
        "padding:20px;margin-bottom:18px}.hero{background:linear-gradient(135deg,#152541,#111827)}"
        ".cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px;"
        "margin:18px 0}.card{background:var(--panel2);border:1px solid var(--line);"
        "border-radius:11px;padding:14px}.card b{display:block;font-size:22px;margin-top:4px}"
        ".ok{color:var(--ok)}.warn{color:var(--warn)}.danger{color:var(--danger)}"
        ".neutral{color:var(--accent)}.badge{display:inline-block;border:1px solid currentColor;"
        "border-radius:999px;padding:2px 8px;font-size:12px}.grid{display:grid;"
        "grid-template-columns:repeat(auto-fit,minmax(320px,1fr));gap:18px}"
        ".table-wrap{overflow:auto;border:1px solid var(--line);border-radius:10px}"
        "table{width:100%;border-collapse:collapse;white-space:nowrap}th,td{padding:9px 11px;"
        "border-bottom:1px solid var(--line);text-align:left;vertical-align:top}"
        "th{position:sticky;top:0;background:#1a2740;color:#bcd0e9;z-index:1}"
        "tr:last-child td{border-bottom:0}tr:hover td{background:#172238}"
        "code{color:#b8dcff}pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#0c1425;"
        "border:1px solid var(--line);border-radius:9px;padding:13px;color:#c8d6e8}"
        ".meta{display:grid;grid-template-columns:max-content 1fr;gap:5px 14px;"
        "overflow-wrap:anywhere}.meta dt{color:var(--muted)}.meta dd{margin:0}"
        ".toolbar{display:flex;gap:10px;align-items:center;margin-bottom:12px;flex-wrap:wrap}"
        "input{min-width:280px;max-width:100%;background:#0c1425;color:var(--text);"
        "border:1px solid var(--line);border-radius:8px;padding:9px 11px}"
        ".empty{color:var(--muted);font-style:italic}.wrap{white-space:normal;min-width:280px}"
        "footer{text-align:center;color:var(--muted);padding:12px}@media(max-width:700px){"
        "main{padding:14px}.hero,.panel{padding:15px}.meta{grid-template-columns:1fr}}");
    sb_append(&sb, "</style></head><body><main>");

    sb_append(&sb, "<section class=\"hero\"><h1>EH/Unwind Analyzer</h1><p class=\"muted\">");
    html_escape(&sb, input_path ? input_path : "");
    sb_append(&sb, "</p><div class=\"cards\">");
    sb_appendf(&sb, "<div class=\"card\">Записи RUNTIME_FUNCTION<b>%zu</b></div>", n);
    sb_appendf(&sb, "<div class=\"card\">Ошибки разбора<b class=\"%s\">%zu</b></div>",
               parse_failures ? "danger" : "ok", parse_failures);
    sb_appendf(&sb, "<div class=\"card\">Аномальные записи<b class=\"%s\">%zu</b></div>",
               anomaly_count ? "warn" : "ok", anomaly_count);
    if (hr) {
        sb_appendf(&sb, "<div class=\"card\">Эвристические признаки<b class=\"%s\">%zu</b></div>",
                   hr->finding_count ? "warn" : "ok", hr->finding_count);
        sb_appendf(&sb, "<div class=\"card\">Оценка подозрительности<b class=\"%s\">%d / 100</b></div>",
                   html_score_class(hr), hr->score);
        sb_append(&sb, "<div class=\"card\">Статический вывод<b class=\"");
        sb_append(&sb, html_score_class(hr));
        sb_append(&sb, "\" style=\"font-size:16px\">");
        html_escape(&sb, ru_verdict_text(hr->verdict));
        sb_append(&sb, "</b></div>");
        sb_appendf(&sb, "<div class=\"card\">Уверенность<b>%.0f%%</b></div>",
                   hr->confidence * 100.0);
    }
    sb_append(&sb, "<div class=\"card\">Динамическая проверка<b class=\"");
    sb_append(&sb, (dr && dr->ran && dr->ok && !dr->timed_out) ? "ok" : "neutral");
    sb_append(&sb, "\" style=\"font-size:16px\">");
    html_escape(&sb, ru_status_text(dr));
    sb_append(&sb, "</b></div></div>");

    sb_append(&sb, "<dl class=\"meta\"><dt>Сформировано</dt><dd>");
    html_escape(&sb, generated_at);
    sb_append(&sb, "</dd><dt>Инструмент / schema</dt><dd>");
    html_escape(&sb, tool_version);
    sb_append(&sb, " / ");
    html_escape(&sb, schema_version);
    sb_append(&sb, "</dd><dt>SHA-256</dt><dd><code>");
    html_escape(&sb, input_hash);
    sb_append(&sb, "</code></dd><dt>Сборка</dt><dd>");
    html_escape(&sb, build_platform);
    sb_append(&sb, " / ");
    html_escape(&sb, build_compiler);
    sb_append(&sb, " / ");
    html_escape(&sb, build_mode);
    sb_append(&sb, "</dd><dt>Параметры анализа</dt><dd>");
    sb_appendf(&sb, "strict ABI: %s; follow chain: %s; max depth: %u",
               ru_yes_no(metadata->strict_abi),
               ru_yes_no(metadata->follow_chain),
               (unsigned)metadata->max_chain_depth);
    sb_appendf(&sb, "</dd><dt>Размер файла</dt><dd>%llu байт</dd><dt>Команда</dt><dd><code>",
               (unsigned long long)metadata->input_size);
    html_escape(&sb, metadata->command_line);
    sb_append(&sb, "</code>");
    sb_append(&sb, "</dd></dl></section>");

    sb_append(&sb, "<div class=\"grid\"><section class=\"panel\"><h2>Образ PE</h2><dl class=\"meta\">");
    sb_appendf(&sb,
        "<dt>ImageBase</dt><dd><code>0x%llx</code></dd>"
        "<dt>SizeOfImage</dt><dd><code>0x%x</code></dd>"
        "<dt>SizeOfHeaders</dt><dd><code>0x%x</code></dd>"
        "<dt>Exception directory</dt><dd><code>0x%08x + 0x%x</code></dd>"
        "<dt>SectionAlignment / FileAlignment</dt><dd><code>0x%x / 0x%x</code></dd></dl></section>",
        (unsigned long long)img->image_base, (unsigned)img->size_of_image,
```

```
        (unsigned)img->size_of_headers, (unsigned)img->exception_rva,
        (unsigned)img->exception_size, (unsigned)img->section_alignment,
        (unsigned)img->file_alignment);
    sb_append(&sb, "<section class=\"panel\"><h2>Структурная проверка</h2><dl class=\"meta\">");
    sb_appendf(&sb,
        "<dt>Raw-диапазоны вне файла</dt><dd>%zu</dd><dt>Virtual-диапазоны вне image</dt><dd>%zu</dd>"
        "<dt>Пересечения raw</dt><dd>%zu</dd><dt>Пересечения virtual</dt><dd>%zu</dd>"
        "<dt>Невыровненные секции</dt><dd>%zu</dd><dt>Exception directory OOB</dt><dd>%s</dd>"
        "</dl></section></div>",
        img->section_raw_oob, img->section_virtual_oob, img->section_raw_overlaps,
        img->section_virtual_overlaps, img->section_misaligned,
        ru_yes_no(img->exception_directory_oob));

    sb_append(&sb, "<section class=\"panel\"><h2>Секции</h2><div class=\"table-wrap\"><table>"
                   "<thead><tr><th>Имя</th><th>VirtualAddress</th><th>VirtualSize</th>"
                   "<th>Raw offset</th><th>Raw size</th><th>Права</th></tr></thead><tbody>");
    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* section = &img->sections[i];
        char permissions[4];
        permissions[0] = (section->characteristics & IMAGE_SCN_MEM_READ) ? 'R' : '-';
        permissions[1] = (section->characteristics & IMAGE_SCN_MEM_WRITE) ? 'W' : '-';
        permissions[2] = (section->characteristics & IMAGE_SCN_MEM_EXECUTE) ? 'X' : '-';
        permissions[3] = '\0';
        sb_append(&sb, "<tr>");
        html_text_cell(&sb, section->name);
        html_hex32_cell(&sb, section->virtual_address);
        html_hex32_cell(&sb, section->virtual_size);
        html_hex32_cell(&sb, section->raw_ptr);
        html_hex32_cell(&sb, section->raw_size);
        html_text_cell(&sb, permissions);
        sb_append(&sb, "</tr>");
    }
    sb_append(&sb, "</tbody></table></div></section>");

    if (hr || dr) {
        AssessmentSummary summary = assessment_summary(hr, dr);
        ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);
        sb_append(&sb, "<section class=\"panel\"><h2>Итоговая оценка</h2><div class=\"cards\">");
        sb_append(&sb, "<div class=\"card\">Вывод<b style=\"font-size:16px\">");
        html_escape(&sb, ru_verdict_text(summary.final_verdict));
        sb_append(&sb, "</b></div><div class=\"card\">Сила доказательств<b style=\"font-size:16px\">");
        html_escape(&sb, ru_case_strength_text(summary.strength));
        sb_appendf(&sb,
            "</b></div><div class=\"card\">Итоговая уверенность<b>%.0f%%</b></div>"
            "<div class=\"card\">Статика / динамика<b style=\"font-size:16px\">",
            summary.final_confidence * 100.0);
        html_escape(&sb, ru_level_text(summary.static_evidence_level));
        sb_append(&sb, " / ");
        html_escape(&sb, ru_level_text(summary.runtime_evidence_level));
        sb_appendf(&sb,
            "</b></div><div class=\"card\">Корреляция слоев<b class=\"%s\" "
            "style=\"font-size:16px\">%s</b></div>"
            "<div class=\"card\">Динамические сигналы<b>%u</b></div></div>",
            summary.correlated ? "ok" : "neutral",
            ru_yes_no(summary.correlated),
            summary.dynamic_signals);
        if (steps.count) {
            sb_append(&sb, "<h2 style=\"margin-top:18px\">План ручной проверки</h2><ol>");
            for (size_t i = 0; i < steps.count; i++) {
                sb_append(&sb, "<li>");
                html_append_manual_step_ru(&sb, steps.items[i]);
                sb_append(&sb, "</li>");
            }
            sb_append(&sb, "</ol>");
        }
        sb_append(&sb, "</section>");
    }

    if (hr) {
        sb_append(&sb, "<section class=\"panel\"><h2>Статические признаки</h2>");
        if (hr->finding_count == 0) {
            sb_append(&sb, "<p class=\"empty\">Эвристические признаки не обнаружены.</p>");
        } else {
            sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Важность</th>"
                           "<th>Код</th><th>Семейство</th><th>Затронуто</th><th>RVA</th>"
                           "<th>Секция</th><th>Описание</th></tr></thead><tbody>");
            for (size_t i = 0; i < hr->finding_count; i++) {
                const HeuristicFinding* finding = &hr->findings[i];
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, ru_severity_text(finding->severity));
                html_text_cell(&sb, finding->code);
                html_text_cell(&sb, finding->family);
                sb_appendf(&sb, "<td>%zu</td>", finding->affected_count);
                if (finding->has_location) html_hex32_cell(&sb, finding->representative_rva);
                else sb_append(&sb, "<td>-</td>");
                html_text_cell(&sb, finding->has_location
                                    ? section_name_or_dash(img, finding->representative_rva)
                                    : "-");
                sb_append(&sb, "<td class=\"wrap\">");
                html_append_finding_message_ru(&sb, finding);
                sb_append(&sb, "</td></tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        }
        if (hr->mitigation_count) {
            sb_append(&sb, "<h2 style=\"margin-top:18px\">Правила снижения уверенности</h2><div class=\"table-wrap\">"
                           "<table><thead><tr><th>Код</th><th>Пояснение</th></tr></thead><tbody>");
            for (size_t i = 0; i < hr->mitigation_count; i++) {
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, hr->mitigations[i].code);
                sb_append(&sb, "<td class=\"wrap\">");
                html_append_mitigation_message_ru(&sb, hr->mitigations[i].code, hr->mitigations[i].message);
                sb_append(&sb, "</td></tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        }
        sb_append(&sb, "<h2 style=\"margin-top:18px\">Детали анализа</h2><pre>");
        html_escape(&sb, hr->details.s ? hr->details.s : "");
        sb_append(&sb, "</pre></section>");
    }

    if (dr) {
        sb_append(&sb, "<section class=\"panel\"><h2>Динамическая проверка</h2><div class=\"cards\">");
        sb_appendf(&sb, "<div class=\"card\">Исключения<b>%u</b></div>",
                   (unsigned)dr->exceptions_seen);
```

```
        sb_appendf(&sb, "<div class=\"card\">Попадания в handler<b>%u</b></div>",
                   (unsigned)(dr->unwind_handler_hits + dr->veh_hits + dr->vch_hits + dr->uef_hits));
        sb_appendf(&sb, "<div class=\"card\">Вызовы runtime-таблиц<b>%u</b></div>",
                   (unsigned)(dr->rtl_add_function_table_calls +
                              dr->rtl_install_function_table_cb_calls));
        sb_appendf(&sb, "<div class=\"card\">Потерянные события<b class=\"%s\">%u</b></div>",
                   dr->dropped_events ? "warn" : "ok", (unsigned)dr->dropped_events);
        sb_appendf(&sb, "<div class=\"card\">Инструментация завершена<b class=\"%s\">%s</b></div>",
                   dr->instrumentation_complete ? "ok" : "warn",
                   ru_yes_no(dr->instrumentation_complete));
        sb_appendf(&sb, "<div class=\"card\">Handlers найдены / включены<b>%u / %u</b></div>",
                   (unsigned)dr->unwind_handlers_discovered,
                   (unsigned)dr->unwind_handler_bp_set);
        sb_appendf(&sb, "<div class=\"card\">Ошибки breakpoint<b class=\"%s\">%u</b></div>",
                   dr->breakpoint_set_failures ? "warn" : "ok",
                   (unsigned)dr->breakpoint_set_failures);
        sb_appendf(&sb, "<div class=\"card\">Покрытие модулей<b class=\"%s\">пик %u, ошибки %u, усечения %u</b></div>",
                   (dr->module_refresh_failures || dr->module_list_truncations) ? "warn" : "ok",
                   (unsigned)dr->modules_observed_peak,
                   (unsigned)dr->module_refresh_failures,
                   (unsigned)dr->module_list_truncations);
        sb_append(&sb, "</div>");
        if (dr->event_count) {
            sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Тип</th><th>Модуль</th>"
                           "<th>Thread</th><th>Exception</th><th>Chance</th><th>Address</th>"
                           "<th>Arg 0</th><th>Arg 1</th><th>Arg 2</th><th>Arg 3</th>"
                           "</tr></thead><tbody>");
            for (uint32_t i = 0; i < dr->event_count; i++) {
                const DynamicEvent* event = &dr->events[i];
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, event->type);
                html_text_cell(&sb, event->module);
                sb_appendf(&sb, "<td>%u</td>", (unsigned)event->thread_id);
                html_hex32_cell(&sb, event->exception_code);
                html_text_cell(&sb, event->first_chance ? "first" : "second / n/a");
                html_hex64_cell(&sb, event->address);
                html_hex64_cell(&sb, event->arg0);
                html_hex64_cell(&sb, event->arg1);
                html_hex64_cell(&sb, event->arg2);
                html_hex64_cell(&sb, event->arg3);
                sb_append(&sb, "</tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        } else {
            sb_append(&sb, "<p class=\"empty\">Динамические события не записаны.</p>");
        }
        if (dr->details.s && dr->details.len) {
            sb_append(&sb, "<h2 style=\"margin-top:18px\">Журнал выполнения</h2><pre>");
            html_escape(&sb, dr->details.s);
            sb_append(&sb, "</pre>");
        }
        sb_append(&sb, "</section>");
    }

    sb_append(&sb, "<section class=\"panel\"><h2>Таблица RUNTIME_FUNCTION</h2><div class=\"toolbar\">"
                   "<input id=\"function-filter\" type=\"search\" placeholder=\"Фильтр по RVA, секции, флагу или ошибке\">"
                   "<span id=\"function-count\" class=\"muted\"></span></div>"
                   "<div class=\"table-wrap\"><table id=\"functions\"><thead><tr><th>#</th>"
                   "<th>Begin</th><th>End</th><th>Size</th><th>Секция кода</th><th>Unwind RVA</th>"
                   "<th>Unwind section</th><th>Статус</th><th>Version / flags</th><th>Stack</th>"
                   "<th>Handler</th><th>Аномалии / ошибка</th></tr></thead><tbody>");
    for (size_t i = 0; i < n; i++) {
        char flags[64];
        unwind_flags_to_string(us[i].flags, flags);
        anomalies.len = 0;
        if (anomalies.s) anomalies.s[0] = '\0';
        anomalies_to_str(&anomalies, us[i].anomalies);
        sb_append(&sb, "<tr>");
        sb_appendf(&sb, "<td>%zu</td>", i);
        html_hex32_cell(&sb, rf[i].begin_rva);
        html_hex32_cell(&sb, rf[i].end_rva);
        sb_appendf(&sb, "<td>%u</td>", (unsigned)runtime_function_size(&rf[i]));
        html_text_cell(&sb, section_name_or_dash(img, rf[i].begin_rva));
        html_hex32_cell(&sb, rf[i].unwind_rva);
        html_text_cell(&sb, section_name_or_dash(img, rf[i].unwind_rva));
        sb_append(&sb, us_ok[i] ? "<td class=\"ok\">OK</td>" : "<td class=\"danger\">Ошибка</td>");
        sb_appendf(&sb, "<td>v%u / ", (unsigned)us[i].version);
        html_escape(&sb, flags);
        sb_append(&sb, "</td>");
        sb_appendf(&sb, "<td>%u B</td>", (unsigned)us[i].fixed_stack_bytes);
        if (us[i].has_handler) html_hex32_cell(&sb, us[i].handler_rva);
        else sb_append(&sb, "<td>-</td>");
        sb_append(&sb, "<td class=\"wrap\">");
        if (us[i].anomalies) html_escape(&sb, anomalies.s);
        if (us[i].anomalies && us[i].parse_error[0]) sb_append(&sb, "; ");
        html_escape(&sb, us[i].parse_error);
        if (!us[i].anomalies && !us[i].parse_error[0]) sb_append(&sb, "-");
        sb_append(&sb, "</td></tr>");
    }
    sb_append(&sb, "</tbody></table></div></section>");

    sb_append(&sb,
        "<footer>Сформировано EH/Unwind Analyzer ");
    html_escape(&sb, tool_version);
    sb_append(&sb,
        "</footer></main><script>"
        "(()=>{const q=document.getElementById('function-filter');"
        "const rows=[...document.querySelectorAll('#functions tbody tr')];"
        "const out=document.getElementById('function-count');"
        "function run(){const needle=q.value.trim().toLowerCase();let shown=0;"
        "for(const row of rows){const visible=!needle||row.textContent.toLowerCase().includes(needle);"
        "row.hidden=!visible;if(visible)shown++;}out.textContent=`${shown} из ${rows.length}`;}"
        "q.addEventListener('input',run);run();})();"
        "</script></body></html>");

    bool ok = sb_ok(&sb) && sb_ok(&anomalies);
    if (ok) ok = file_write_atomic(path, sb.s, sb.len);
    if (!ok) {
        sb_appendf(err, "%s: %s\n",
                   sb_ok(&sb) && sb_ok(&anomalies)
                       ? "Failed to write HTML"
                       : "Out of memory while building HTML report",
                   path);
    }
    sb_free(&anomalies);
```

```
1524 |     sb_free(&sb);
1525 |     return ok;
1526 | }
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include "report.h"`	Технически подключает `report.h`. Логически это дает модулю доступ к: интерфейс печати brief/CSV/JSON/HTML отчетов.
2	`#include "app_info.h"`	Технически подключает `app_info.h`. Логически это дает модулю доступ к: публичный интерфейс метаданных версии и сборки.
3	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.
4	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
5	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
7	`#define IMAGE_SCN_MEM_EXECUTE 0x20000000u`	Технически объявляет макрос `IMAGE_SCN_MEM_EXECUTE` со значением `0x20000000u`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
8	`#define IMAGE_SCN_MEM_READ 0x40000000u`	Технически объявляет макрос `IMAGE_SCN_MEM_READ` со значением `0x40000000u`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
9	`#define IMAGE_SCN_MEM_WRITE 0x80000000u`	Технически объявляет макрос `IMAGE_SCN_MEM_WRITE` со значением `0x80000000u`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
11	`static void append_regmask(StrBuf* sb, uint16_t mask, bool xmm) {`	Технически объявляет/начинает функцию `append_regmask`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
12	`bool first = true;`	Технически объявляет `first` в функции `append_regmask` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
13	`for (uint8_t r = 0; r < 16; r++) {`	Технически начинает цикл `uint8_t r = 0; r < 16; r++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
14	`if (!(mask & (uint16_t)(1u << r))) continue;`	Технически проверяет условие `!(mask & (uint16_t)(1u << r))`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
15	`if (!first) sb_append(sb, ",");`	Технически проверяет условие `!first) sb_append(sb, ","`; логически это защита от NULL-указателя и работы с неинициализированными данными.
16	`if (xmm) sb_appendf(sb, "XMM%u", (unsigned)r);`	Технически проверяет условие `xmm) sb_appendf(sb, "XMM%u", (unsigned)r`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
17	`else sb_append(sb, unwind_reg_name(r));`	Технически объявляет прототип функции `sb_append` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет C-строку в StrBuf.
18	`first = false;`	Технически присваивает `first` значение `false`. Логически это обновляет `first`: хранит промежуточное значение, которое дальше используется в текущей функции.
19	`}`	Технически завершает область видимости внутри `append_regmask`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
20	`if (first) sb_append(sb, "-");`	Технически проверяет условие `first) sb_append(sb, "-"`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
21	`}`	Технически завершает область видимости внутри `append_regmask`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
23	`static void anomalies_to_str(StrBuf* sb, uint32_t anom) {`	Технически объявляет/начинает функцию `anomalies_to_str`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
24	`bool first = true;`	Технически объявляет `first` в функции `anomalies_to_str` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
25	`struct { uint32_t bit; const char* name; } m[] = {`	Технически начинает описание структуры `m`. Логически структура собирает связанные поля в одну модель данных проекта.
26	`{UA_VERSION_UNKNOWN, "ver"},`	Технически эта строка является частью C-конструкции `{UA_VERSION_UNKNOWN, "ver"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
27	`{UA_TRUNCATED, "trunc"},`	Технически эта строка является частью C-конструкции `{UA_TRUNCATED, "trunc"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
28	`{UA_BAD_OPCODE, "op"},`	Технически эта строка является частью C-конструкции `{UA_BAD_OPCODE, "op"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
29	`{UA_BAD_OPERAND, "arg"},`	Технически эта строка является частью C-конструкции `{UA_BAD_OPERAND, "arg"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
30	`{UA_CODEOFFSET_OUTSIDE, "off"},`	Технически эта строка является частью C-конструкции `{UA_CODEOFFSET_OUTSIDE, "off"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
31	`{UA_CHAIN_LOOP, "chain_loop"},`	Технически эта строка является частью C-конструкции `{UA_CHAIN_LOOP, "chain_loop"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
32	`{UA_CHAIN_DEPTH_EXCEEDED, "chain_depth"},`	Технически эта строка является частью C-конструкции `{UA_CHAIN_DEPTH_EXCEEDED, "chain_depth"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
33	`{UA_HANDLER_OOB, "handler_oob"},`	Технически эта строка является частью C-конструкции `{UA_HANDLER_OOB, "handler_oob"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
34	`{UA_CODEOFFSET_ORDER, "order"},`	Технически эта строка является частью C-конструкции `{UA_CODEOFFSET_ORDER, "order"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
35	`{UA_OPERAND_SANITY, "operand_sanity"},`	Технически эта строка является частью C-конструкции `{UA_OPERAND_SANITY, "operand_sanity"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
36	`{UA_ALIGNMENT, "alignment"},`	Технически эта строка является частью C-конструкции `{UA_ALIGNMENT, "alignment"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
37	`{UA_CHAIN_INVALID, "chain_invalid"},`	Технически эта строка является частью C-конструкции `{UA_CHAIN_INVALID, "chain_invalid"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
38	`{UA_ABI_REGISTER_CLASS, "abi_register_class"},`	Технически эта строка является частью C-конструкции `{UA_ABI_REGISTER_CLASS, "abi_register_class"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
39	`{UA_ABI_RESERVED_OPINFO, "abi_reserved_opinfo"},`	Технически эта строка является частью C-конструкции `{UA_ABI_RESERVED_OPINFO, "abi_reserved_opinfo"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
40	`{UA_ABI_PUSH_ORDER, "abi_push_order"},`	Технически эта строка является частью C-конструкции `{UA_ABI_PUSH_ORDER, "abi_push_order"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
41	`{UA_ABI_CHAIN_RESTRICTION, "abi_chain_restriction"},`	Технически эта строка является частью C-конструкции `{UA_ABI_CHAIN_RESTRICTION, "abi_chain_restriction"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
42	`{UA_ABI_SAVE_OFFSET, "abi_save_offset"},`	Технически эта строка является частью C-конструкции `{UA_ABI_SAVE_OFFSET, "abi_save_offset"},`. Логически она уточняет структуру блока функции `anomalies_to_str` и влияет на то, какие данные будут обработаны.
43	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
44	`for (size_t i = 0; i < sizeof(m)/sizeof(m[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(m)/sizeof(m[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
45	`if (!(anom & m[i].bit)) continue;`	Технически проверяет условие `!(anom & m[i].bit)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
46	`if (!first) sb_append(sb, "|");`	Технически проверяет условие `!first) sb_append(sb, "|"`; логически это защита от NULL-указателя и работы с неинициализированными данными.
47	`sb_append(sb, m[i].name);`	Технически вызывает `sb_append`. Логически это действие нужно в функции `anomalies_to_str`: добавляет текст в динамический буфер.
48	`first = false;`	Технически присваивает `first` значение `false`. Логически это обновляет `first`: хранит промежуточное значение, которое дальше используется в текущей функции.
49	`}`	Технически завершает область видимости внутри `anomalies_to_str`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
50	`if (first) sb_append(sb, "0");`	Технически проверяет условие `first) sb_append(sb, "0"`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
51	`}`	Технически завершает область видимости внутри `anomalies_to_str`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
53	`static const char* section_name_or_dash(const PeImage* img, uint32_t rva) {`	Технически объявляет/начинает функцию `section_name_or_dash`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
54	`const PeSection* s = pe_find_section_for_rva(img, rva);`	Технически присваивает `const PeSection* s` значение `pe_find_section_for_rva(img, rva)`. Логически это обновляет `const PeSection* s`: хранит промежуточное значение, которое дальше используется в текущей функции.
55	`return s ? s->name : "-";`	Технически возвращает выражение `s ? s->name : "-"`. Логически именно это значение становится результатом функции `section_name_or_dash` для вызывающего кода.
56	`}`	Технически завершает область видимости внутри `section_name_or_dash`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
58	`static uint32_t runtime_function_size(const RuntimeFunction* r) {`	Технически объявляет/начинает функцию `runtime_function_size`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
59	`if (!r || r->end_rva < r->begin_rva) return 0;`	Технически проверяет условие `!r || r->end_rva < r->begin_rva`; логически это защита от NULL-указателя и работы с неинициализированными данными.
60	`return r->end_rva - r->begin_rva;`	Технически возвращает выражение `r->end_rva - r->begin_rva`. Логически именно это значение становится результатом функции `runtime_function_size` для вызывающего кода.
61	`}`	Технически завершает область видимости внутри `runtime_function_size`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
63	`static bool csv_cell(StrBuf* out, const char* s) {`	Технически объявляет/начинает функцию `csv_cell`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
64	`if (!out || !s) return false;`	Технически проверяет условие `!out || !s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
65	`bool need_q = false;`	Технически объявляет `need_q` в функции `csv_cell` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
66	`for (const char* p = s; *p; p++) {`	Технически начинает цикл `const char* p = s; *p; p++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
67	`if (*p == ',' || *p == '"' || *p == '\n' || *p == '\r') { need_q = true; break; }`	Технически проверяет условие `*p == ',' || *p == '"' || *p == '\n' || *p == '\r'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
68	`}`	Технически завершает область видимости внутри `csv_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
69	`if (!need_q) return sb_append(out, s);`	Технически проверяет условие `!need_q) return sb_append(out, s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
71	`if (!sb_appendn(out, "\"", 1)) return false;`	Технически проверяет условие `!sb_appendn(out, "\"", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
72	`for (const char* p = s; *p; p++) {`	Технически начинает цикл `const char* p = s; *p; p++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
73	`if (*p == '"' && !sb_appendn(out, "\"", 1)) return false;`	Технически проверяет условие `*p == '"' && !sb_appendn(out, "\"", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
74	`if (!sb_appendn(out, p, 1)) return false;`	Технически проверяет условие `!sb_appendn(out, p, 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
75	`}`	Технически завершает область видимости внутри `csv_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
76	`return sb_appendn(out, "\"", 1);`	Технически объявляет прототип функции `sb_appendn` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет ровно n байтов текста в StrBuf.
77	`}`	Технически завершает область видимости внутри `csv_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
79	`static bool build_unwind_comment(const UnwindSummary* u, StrBuf* sb) {`	Технически объявляет/начинает функцию `build_unwind_comment`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
80	`sb->len = 0;`	Технически присваивает `sb->len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
81	`if (sb->s) sb->s[0] = 0;`	Технически проверяет условие `sb->s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
83	`char fbuf[64];`	Технически объявляет `fbuf` в функции `build_unwind_comment`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
84	`unwind_flags_to_string(u->flags, fbuf);`	Технически вызывает `unwind_flags_to_string`. Логически это действие нужно в функции `build_unwind_comment`: переводит флаги UNWIND_INFO в текст.
86	`sb_appendf(sb, "UNWIND v%u flags=%s prolog=%u stack=%uB frame=%s+%u ",`	Технически присваивает `sb_appendf(sb, "UNWIND v%u flags` значение `%s prolog=%u stack=%uB frame=%s+%u ",`. Логически это обновляет `sb_appendf(sb, "UNWIND v%u flags`: хранит промежуточное значение, которое дальше используется в текущей функции.
87	`(unsigned)u->version, fbuf, (unsigned)u->prolog_size,`	Технически эта строка является частью С-конструкции `(unsigned)u->version, fbuf, (unsigned)u->prolog_size,`. Логически она уточняет структуру блока функции `build_unwind_comment` и влияет на то, какие данные будут обработаны.
88	`(unsigned)u->fixed_stack_bytes,`	Технически эта строка является частью С-конструкции `(unsigned)u->fixed_stack_bytes,`. Логически она уточняет структуру блока функции `build_unwind_comment` и влияет на то, какие данные будут обработаны.
89	`(u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),`	Технически вызывает `unwind_reg_name`. Логически это действие нужно в функции `build_unwind_comment`: переводит номер регистра из UNWIND_INFO в понятное имя.
90	`(unsigned)(u->frame_off * 16u));`	Технически выполняет оператор `(unsigned)(u->frame_off * 16u));`. Логически в функции `build_unwind_comment` он меняет состояние, проверяемые данные или подготовку отчета.
92	`sb_append(sb, "saved_gp=[");`	Технически присваивает `sb_append(sb, "saved_gp` значение `[")`. Логически это обновляет `sb_append(sb, "saved_gp`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
93	`append_regmask(sb, u->saved_nonvol_mask, false);`	Технически вызывает append_regmask. Логически это действие нужно в функции build_unwind_comment: вызывает функцию append_regmask для выполнения части алгоритма.
94	`sb_append(sb, "] saved_xmm=[");`	Технически присваивает sb_append(sb, "] saved_xmm значение [("). Логически это обновляет sb_append(sb, "] saved_xmm: хранит промежуточное значение, которое дальше используется в текущей функции.
95	`append_regmask(sb, u->saved_xmm_mask, true);`	Технически вызывает append_regmask. Логически это действие нужно в функции build_unwind_comment: вызывает функцию append_regmask для выполнения части алгоритма.
96	`sb_append(sb, "]");`	Технически вызывает sb_append. Логически это действие нужно в функции build_unwind_comment: добавляет текст в динамический буфер.
98	`if (u->has_handler) sb_appendf(sb, " handler_rva=0x%08x", (unsigned)u->handler_rva);`	Технически проверяет условие u->has_handler) sb_appendf(sb, " handler_rva=0x%08x", (unsigned)u->handler_rva; логически отделяет функции с language-specific handler от обычных функций.
99	`if (u->has_chain) {`	Технически проверяет условие u->has_chain; если оно истинно, выполняется защитная или аналитическая ветка ниже.
100	`sb_appendf(sb, " chain_to=[0x%08x..0x%08x] u=0x%08x",`	Технически присваивает sb_appendf(sb, " chain_to значение [0x%08x..0x%08x] u=0x%08x",. Логически это обновляет sb_appendf(sb, " chain_to: хранит промежуточное значение, которое дальше используется в текущей функции.
101	`(unsigned)u->chain.begin_rva,`	Технически эта строка является частью С-конструкции (unsigned)u->chain.begin_rva,. Логически она уточняет структуру блока функции build_unwind_comment и влияет на то, какие данные будут обработаны.
102	`(unsigned)u->chain.end_rva,`	Технически эта строка является частью С-конструкции (unsigned)u->chain.end_rva,. Логически она уточняет структуру блока функции build_unwind_comment и влияет на то, какие данные будут обработаны.
103	`(unsigned)u->chain.unwind_rva);`	Технически выполняет оператор (unsigned)u->chain.unwind_rva);. Логически в функции build_unwind_comment он меняет состояние, проверяемые данные или подготовку отчета.
104	`}`	Технически завершает область видимости внутри build_unwind_comment. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
106	`if (u->anomalies) {`	Технически проверяет условие u->anomalies; если оно истинно, выполняется защитная или аналитическая ветка ниже.
107	`sb_append(sb, " anomalies=");`	Технически присваивает sb_append(sb, " anomalies значение "). Логически это обновляет sb_append(sb, " anomalies: хранит промежуточное значение, которое дальше используется в текущей функции.
108	`anomalies_to_str(sb, u->anomalies);`	Технически вызывает anomalies_to_str. Логически это действие нужно в функции build_unwind_comment: вызывает функцию anomalies_to_str для выполнения части алгоритма.
109	`}`	Технически завершает область видимости внутри build_unwind_comment. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
110	`return sb_ok(sb);`	Технически объявляет прототип функции sb_ok без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и проверяет, что StrBuf не перешел в состояние ошибки.
111	`}`	Технически завершает область видимости внутри build_unwind_comment. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
113	`void report_print_brief(const char* path, const PeImage* img, size_t rf_count) {`	Технически объявляет/начинает функцию report_print_brief; она доступная другим модулям через заголовок или линковку. Логически функция печатает краткую сводку анализа в консоль.

Стр.	Код	Нормальное объяснение
114	`printf("File: %s\n", path);`	Технически вызывает printf. Логически это действие нужно в функции report_print_brief: выводит информацию пользователю в консоль.
115	`printf("Machine: 0x%04x (AMD64)\n", (unsigned)img->machine);`	Технически вызывает printf. Логически это действие нужно в функции report_print_brief: выводит информацию пользователю в консоль.
116	`printf("ImageBase: 0x%llx\n", (unsigned long long)img->image_base);`	Технически вызывает printf. Логически это действие нужно в функции report_print_brief: выводит информацию пользователю в консоль.
117	`printf("SizeOfImage: 0x%x\n", (unsigned)img->size_of_image);`	Технически вызывает printf. Логически это действие нужно в функции report_print_brief: выводит информацию пользователю в консоль.
118	`printf("SizeOfHeaders: 0x%x Sections: %zu\n",`	Технически вызывает printf. Логически это действие нужно в функции report_print_brief: выводит информацию пользователю в консоль.
119	`(unsigned)img->size_of_headers, img->num_sections);`	Технически выполняет оператор (unsigned)img->size_of_headers, img->num_sections);. Логически в функции report_print_brief он меняет состояние, проверяемые данные или подготовку отчета.
120	`printf("Exception Directory: RVA=0x%08x Size=0x%x\n", (unsigned)img->exception_rva, (unsigned)img->exception_size);`	Технически присваивает printf("Exception Directory: RVA значение 0x%08x Size=0x%x\n", (unsigned)img->exception_rva, (unsigned)img->exception_size). Логически это обновляет printf("Exception Directory: RVA: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
121	`printf("RUNTIME_FUNCTION entries: %zu\n", rf_count);`	Технически вызывает `printf`. Логически это действие нужно в функции `report_print_brief`: выводит информацию пользователю в консоль.
122	`}`	Технически завершает область видимости внутри `report_print_brief`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
124	`bool report_write_csv(const char* path,`	Технически вызывает `report_write_csv`. Логически это действие нужно в функции `глобальной области файла`: записывает CSV по функциям и unwind-данным.
125	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
126	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
127	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
128	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
129	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
130	`bool ghidra_mode,`	Технически эта строка является частью C-конструкции `bool ghidra_mode,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
131	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
132	`(void)img;`	Технически выполняет оператор `(void)img;`. Логически в функции `report_write_csv` он меняет состояние, проверяемые данные или подготовку отчета.
133	`if (!path || (!rf && n) || (!us && n) || (!us_ok && n)) {`	Технически проверяет условие `!path || (!rf && n) || (!us && n) || (!us_ok && n)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
134	`sb_append(err, "Invalid arguments for CSV report.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `report_write_csv`: добавляет текст в динамический буфер.
135	`return false;`	Технически возвращает `false` из функции `report_write_csv`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
136	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
138	`StrBuf output = {0};`	Технически объявляет `output` в функции `report_write_csv` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
139	`StrBuf comment = {0};`	Технически объявляет `comment` в функции `report_write_csv` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
140	`if (!sb_init(&output, 8192) || !sb_init(&comment, 512)) {`	Технически проверяет условие `!sb_init(&output, 8192) || !sb_init(&comment, 512)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
141	`sb_free(&comment);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `report_write_csv`: освобождает память динамического буфера.
142	`sb_free(&output);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `report_write_csv`: освобождает память динамического буфера.
143	`sb_append(err, "OOM.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `report_write_csv`: добавляет текст в динамический буфер.
144	`return false;`	Технически возвращает `false` из функции `report_write_csv`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
145	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
147	`if (ghidra_mode) {`	Технически проверяет условие `ghidra_mode`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
148	`sb_append(&output, "begin_rva,end_rva,name,comment\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `report_write_csv`: добавляет текст в динамический буфер.
149	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
150	`const RuntimeFunction* r = &rf[i];`	Технически объявляет `r` в функции `report_write_csv` и сразу задает начальное значение `&rf[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
151	`const UnwindSummary* u = &us[i];`	Технически объявляет u в функции report_write_csv и сразу задает начальное значение &us[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
153	`char name[64];`	Технически объявляет name в функции report_write_csv. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
154	`snprintf(name, sizeof(name), "sub_%08X", (unsigned)r->begin_rva);`	Технически вызывает snprintf. Логически это действие нужно в функции report_write_csv: формирует строку с ограничением размера буфера.
156	`if (us_ok[i]) build_unwind_comment(u, &comment);`	Технически проверяет условие us_ok[i]) build_unwind_comment(u, &comment; логически отбирает записи, где важна корректность UNWIND_INFO.
157	`else {`	Технически эта строка является частью С-конструкции else {. Логически она уточняет структуру блока функции report_write_csv и влияет на то, какие данные будут обработаны.
158	`comment.len = 0;`	Технически присваивает comment.len значение 0. Логически это обновляет len: хранит размер или количество, поэтому используется в проверках границ.
159	`if (comment.s) comment.s[0] = 0;`	Технически проверяет условие comment.s; если оно истинно, выполняется защитная или аналитическая ветка ниже.
160	`sb_append(&comment, "UNWIND parse failed");`	Технически вызывает sb_append. Логически это действие нужно в функции report_write_csv: добавляет текст в динамический буфер.
161	`if (u->parse_error[0]) sb_appendf(&comment, ": %s", u->parse_error);`	Технически проверяет условие u->parse_error[0]) sb_appendf(&comment, ": %s", u->parse_error; если оно истинно, выполняется защитная или аналитическая ветка ниже.
162	`}`	Технически завершает область видимости внутри report_write_csv. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
164	`char a[32], b[32];`	Технически объявляет b в функции report_write_csv. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
165	`snprintf(a, sizeof(a), "0x%08X", (unsigned)r->begin_rva);`	Технически вызывает snprintf. Логически это действие нужно в функции report_write_csv: формирует строку с ограничением размера буфера.
166	`snprintf(b, sizeof(b), "0x%08X", (unsigned)r->end_rva);`	Технически вызывает snprintf. Логически это действие нужно в функции report_write_csv: формирует строку с ограничением размера буфера.
168	`csv_cell(&output, a); sb_appendn(&output, ",", 1);`	Технически вызывает csv_cell. Логически это действие нужно в функции report_write_csv: вызывает функцию csv_cell для выполнения части алгоритма.
169	`csv_cell(&output, b); sb_appendn(&output, ",", 1);`	Технически вызывает csv_cell. Логически это действие нужно в функции report_write_csv: вызывает функцию csv_cell для выполнения части алгоритма.
170	`csv_cell(&output, name); sb_appendn(&output, ",", 1);`	Технически вызывает csv_cell. Логически это действие нужно в функции report_write_csv: вызывает функцию csv_cell для выполнения части алгоритма.
171	`csv_cell(&output, comment.s ? comment.s : "");`	Технически вызывает csv_cell. Логически это действие нужно в функции report_write_csv: вызывает функцию csv_cell для выполнения части алгоритма.
172	`sb_appendn(&output, "\n", 1);`	Технически вызывает sb_appendn. Логически это действие нужно в функции report_write_csv: добавляет ровно n байтов текста в StrBuf.
173	`}`	Технически завершает область видимости внутри report_write_csv. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
174	`} else {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
175	`sb_append(&output,`	Технически вызывает sb_append. Логически это действие нужно в функции report_write_csv: добавляет текст в динамический буфер.
176	`"index,begin_rva,end_rva,size,unwind_rva,unwind_ver,"`	Технически эта строка является частью С-конструкции "index,begin_rva,end_rva,size,unwind_rva,unwind_ver,". Логически она уточняет структуру блока функции report_write_csv и влияет на то, какие данные будут обработаны.
177	`"unwind_flags,prolog_size,fixed_stack,frame_reg,frame_off,"`	Технически эта строка является частью С-конструкции "unwind_flags,prolog_size,fixed_stack,frame_reg,frame_off,". Логически она уточняет структуру блока функции report_write_csv и влияет на то, какие данные будут обработаны.
178	`"handler_rva,has_chain,anomalies,parse_error\n");`	Технически выполняет оператор "handler_rva,has_chain,anomalies,parse_error\n");. Логически в функции report_write_csv он меняет состояние, проверяемые данные или подготовку отчета.
179	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл size_t i = 0; i < n; i++. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
180	`const RuntimeFunction* r = &rf[i];`	Технически объявляет r в функции report_write_csv и сразу задает начальное значение &rf[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
181	`const UnwindSummary* u = &us[i];`	Технически объявляет u в функции report_write_csv и сразу задает начальное значение &us[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
182	`char fbuf[64];`	Технически объявляет fbuf в функции report_write_csv. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
183	`unwind_flags_to_string(u->flags, fbuf);`	Технически вызывает unwind_flags_to_string. Логически это действие нужно в функции report_write_csv: переводит флаги UNWIND_INFO в текст.
185	`sb_appendf(&output, "%zu,0x%08X,0x%08X,%u,0x%08X,",`	Технически вызывает sb_appendf. Логически это действие нужно в функции report_write_csv: добавляет форматированный текст в динамический буфер.

Стр.	Код	Нормальное объяснение
186	`i, (unsigned)r->begin_rva, (unsigned)r->end_rva,`	Технически эта строка является частью C-конструкции `i, (unsigned)r->begin_rva, (unsigned)r->end_rva,`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
187	`(unsigned)runtime_function_size(r),`	Технически вызывает `runtime_function_size`. Логически это действие нужно в функции `report_write_csv`: вызывает функцию `runtime_function_size` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
188	`(unsigned)r->unwind_rva);`	Технически выполняет оператор `(unsigned)r->unwind_rva);`. Логически в функции `report_write_csv` он меняет состояние, проверяемые данные или подготовку отчета.
190	`if (!us_ok[i]) {`	Технически проверяет условие `!us_ok[i]`; логически это защита от NULL-указателя и работы с неинициализированными данными.
191	`sb_appendf(&output, "NA,NA,NA,NA,NA,NA,0,0x%08X,",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `report_write_csv`: добавляет форматированный текст в динамический буфер.
192	`(unsigned)u->anomalies);`	Технически выполняет оператор `(unsigned)u->anomalies);`. Логически в функции `report_write_csv` он меняет состояние, проверяемые данные или подготовку отчета.
193	`csv_cell(&output, u->parse_error);`	Технически вызывает `csv_cell`. Логически это действие нужно в функции `report_write_csv`: вызывает функцию `csv_cell` для выполнения части алгоритма.
194	`sb_appendn(&output, "\n", 1);`	Технически вызывает `sb_appendn`. Логически это действие нужно в функции `report_write_csv`: добавляет ровно n байтов текста в StrBuf.
195	`continue;`	Технически переходит к следующей итерации цикла. Логически текущая запись уже обработана или не подходит для дальнейших проверок.
196	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
198	`sb_appendf(&output, "%u,%s,%u,%u,%s,%u,0x%08X,%u,0x%08X,",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `report_write_csv`: добавляет форматированный текст в динамический буфер.
199	`(unsigned)u->version, fbuf, (unsigned)u->prolog_size,`	Технически эта строка является частью C-конструкции `(unsigned)u->version, fbuf, (unsigned)u->prolog_size,`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
200	`(unsigned)u->fixed_stack_bytes,`	Технически эта строка является частью C-конструкции `(unsigned)u->fixed_stack_bytes,`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
201	`(u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),`	Технически вызывает `unwind_reg_name`. Логически это действие нужно в функции `report_write_csv`: переводит номер регистра из UNWIND_INFO в понятное имя.
202	`(unsigned)(u->frame_off * 16u),`	Технически эта строка является частью C-конструкции `(unsigned)(u->frame_off * 16u),`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
203	`(unsigned)u->handler_rva,`	Технически эта строка является частью C-конструкции `(unsigned)u->handler_rva,`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
204	`(unsigned)(u->has_chain ? 1u : 0u),`	Технически эта строка является частью C-конструкции `(unsigned)(u->has_chain ? 1u : 0u),`. Логически она уточняет структуру блока функции `report_write_csv` и влияет на то, какие данные будут обработаны.
205	`(unsigned)u->anomalies);`	Технически выполняет оператор `(unsigned)u->anomalies);`. Логически в функции `report_write_csv` он меняет состояние, проверяемые данные или подготовку отчета.
206	`csv_cell(&output, "");`	Технически вызывает `csv_cell`. Логически это действие нужно в функции `report_write_csv`: вызывает функцию `csv_cell` для выполнения части алгоритма.
207	`sb_appendn(&output, "\n", 1);`	Технически вызывает `sb_appendn`. Логически это действие нужно в функции `report_write_csv`: добавляет ровно n байтов текста в StrBuf.
208	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
209	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
211	`bool ok = sb_ok(&output) && sb_ok(&comment);`	Технически присваивает `bool ok` значение `sb_ok(&output) && sb_ok(&comment)`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
212	`if (ok) ok = file_write_atomic(path, output.s, output.len);`	Технически проверяет условие `ok) ok = file_write_atomic(path, output.s, output.len`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
213	`sb_free(&comment);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `report_write_csv`: освобождает память динамического буфера.
214	`sb_free(&output);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `report_write_csv`: освобождает память динамического буфера.
215	`if (!ok) sb_appendf(err, "Failed while building or writing CSV: %s\n", path);`	Технически проверяет условие `!ok) sb_appendf(err, "Failed while building or writing CSV: %s\n", path`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
216	`return ok;`	Технически возвращает выражение ok. Логически именно это значение становится результатом функции `report_write_csv` для вызывающего кода.
217	`}`	Технически завершает область видимости внутри `report_write_csv`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
219	`static size_t valid_utf8_sequence_length(const unsigned char* s, size_t remaining) {`	Технически объявляет/начинает функцию `valid_utf8_sequence_length`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
220	`if (!s || remaining == 0) return 0;`	Технически проверяет условие `!s || remaining == 0`; логически это защита от NULL-указателя и работы с неинициализированными данными.
221	`if (s[0] < 0x80u) return 1;`	Технически проверяет условие `s[0] < 0x80u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
222	`if (s[0] >= 0xc2u && s[0] <= 0xdfu) {`	Технически проверяет условие `s[0] >= 0xc2u && s[0] <= 0xdfu`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
223	`return remaining >= 2 && s[1] >= 0x80u && s[1] <= 0xbfu ? 2 : 0;`	Технически возвращает выражение `remaining >= 2 && s[1] >= 0x80u && s[1] <= 0xbfu ? 2 : 0`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
224	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
225	`if (s[0] == 0xe0u) {`	Технически проверяет условие `s[0] == 0xe0u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
226	`return remaining >= 3 && s[1] >= 0xa0u && s[1] <= 0xbfu &&`	Технически возвращает выражение `remaining >= 3 && s[1] >= 0xa0u && s[1] <= 0xbfu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
227	`s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
228	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
229	`if ((s[0] >= 0xe1u && s[0] <= 0xecu) || (s[0] >= 0xeeu && s[0] <= 0xefu)) {`	Технически проверяет условие `(s[0] >= 0xe1u && s[0] <= 0xecu) || (s[0] >= 0xeeu && s[0] <= 0xefu)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
230	`return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0xbfu &&`	Технически возвращает выражение `remaining >= 3 && s[1] >= 0x80u && s[1] <= 0xbfu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
231	`s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
232	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
233	`if (s[0] == 0xedu) {`	Технически проверяет условие `s[0] == 0xedu`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
234	`return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0x9fu &&`	Технически возвращает выражение `remaining >= 3 && s[1] >= 0x80u && s[1] <= 0x9fu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
235	`s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
236	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
237	`if (s[0] == 0xf0u) {`	Технически проверяет условие `s[0] == 0xf0u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
238	`return remaining >= 4 && s[1] >= 0x90u && s[1] <= 0xbfu &&`	Технически возвращает выражение `remaining >= 4 && s[1] >= 0x90u && s[1] <= 0xbfu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
239	`s[2] >= 0x80u && s[2] <= 0xbfu &&`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
240	`s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`	Технически выполняет битовую операцию `s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
241	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
242	`if (s[0] >= 0xf1u && s[0] <= 0xf3u) {`	Технически проверяет условие `s[0] >= 0xf1u && s[0] <= 0xf3u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
243	`return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0xbfu &&`	Технически возвращает выражение `remaining >= 4 && s[1] >= 0x80u && s[1] <= 0xbfu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
244	`s[2] >= 0x80u && s[2] <= 0xbfu &&`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
245	`s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`	Технически выполняет битовую операцию `s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
246	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
247	`if (s[0] == 0xf4u) {`	Технически проверяет условие `s[0] == 0xf4u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
248	`return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0x8fu &&`	Технически возвращает выражение `remaining >= 4 && s[1] >= 0x80u && s[1] <= 0x8fu &&`. Логически именно это значение становится результатом функции `valid_utf8_sequence_length` для вызывающего кода.
249	`s[2] >= 0x80u && s[2] <= 0xbfu &&`	Технически выполняет битовую операцию `s[2] >= 0x80u && s[2] <= 0xbfu &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
250	`s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`	Технически выполняет битовую операцию `s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
251	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
252	`return 0;`	Технически возвращает код 0 из функции `valid_utf8_sequence_length`. Логически это успешное завершение программы или callback.
253	`}`	Технически завершает область видимости внутри `valid_utf8_sequence_length`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
255	`static void json_escape(StrBuf* sb, const char* s) {`	Технически объявляет/начинает функцию `json_escape`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
256	`if (!s) return;`	Технически проверяет условие `!s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
257	`size_t length = strlen(s);`	Технически присваивает `size_t length` значение `strlen(s)`. Логически это обновляет `size_t length`: хранит размер или количество, поэтому используется в проверках границ.
258	`for (size_t i = 0; i < length; i++) {`	Технически начинает цикл `size_t i = 0; i < length; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.

Стр.	Код	Нормальное объяснение
259	`unsigned char c = (unsigned char)s[i];`	Технически присваивает `unsigned char c` значение `(unsigned char)s[i]`. Логически это обновляет `unsigned char c`: хранит промежуточное значение, которое дальше используется в текущей функции.
260	`switch (c) {`	Технически выбирает ветку по значению `c`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.
261	`case '\\': sb_append(sb, "\\\\"); break;`	Технически объявляет ветку `switch` для значения `'\\': sb_append(sb, "\\\\"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
262	`case '"': sb_append(sb, "\\\""); break;`	Технически объявляет ветку `switch` для значения `'"': sb_append(sb, "\\\""); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
263	`case '\n': sb_append(sb, "\\n"); break;`	Технически объявляет ветку `switch` для значения `'\n': sb_append(sb, "\\n"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
264	`case '\r': sb_append(sb, "\\r"); break;`	Технически объявляет ветку `switch` для значения `'\r': sb_append(sb, "\\r"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
265	`case '\t': sb_append(sb, "\\t"); break;`	Технически объявляет ветку `switch` для значения `'\t': sb_append(sb, "\\t"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
266	`default:`	Технически объявляет ветку `switch` по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
267	`if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);`	Технически проверяет условие `c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
268	`else if (c < 0x80) sb_appendn(sb, &s[i], 1);`	Технически объявляет прототип функции `if` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
269	`else {`	Технически эта строка является частью С-конструкции `else {`. Логически она уточняет структуру блока функции `json_escape` и влияет на то, какие данные будут обработаны.
270	`size_t sequence = valid_utf8_sequence_length(`	Технически присваивает `size_t sequence` значение `valid_utf8_sequence_length(`. Логически это обновляет `size_t sequence`: хранит размер или количество, поэтому используется в проверках границ.
271	`(const unsigned char*)&s[i], length - i);`	Технически выполняет битовую операцию `(const unsigned char*)&s[i], length - i);`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
272	`if (sequence) {`	Технически проверяет условие `sequence`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
273	`sb_appendn(sb, &s[i], sequence);`	Технически вызывает `sb_appendn`. Логически это действие нужно в функции `json_escape`: добавляет ровно n байтов текста в StrBuf.
274	`i += sequence - 1;`	Технически изменяет `i` оператором `+=` на основе `sequence - 1`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
275	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
276	`sb_appendf(sb, "\\u%04x", (unsigned)c);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `json_escape`: добавляет форматированный текст в динамический буфер.
277	`}`	Технически завершает область видимости внутри `json_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
278	`}`	Технически завершает область видимости внутри `json_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
279	`}`	Технически завершает область видимости внутри `json_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
280	`}`	Технически завершает область видимости внутри `json_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
281	`}`	Технически завершает область видимости внутри `json_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
283	`static void json_key(StrBuf* sb, const char* k) {`	Технически объявляет/начинает функцию `json_key`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
284	`sb_append(sb, "\"");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_key`: добавляет текст в динамический буфер.
285	`json_escape(sb, k);`	Технически вызывает `json_escape`. Логически это действие нужно в функции `json_key`: вызывает функцию `json_escape` для выполнения части алгоритма.
286	`sb_append(sb, "\":");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_key`: добавляет текст в динамический буфер.
287	`}`	Технически завершает область видимости внутри `json_key`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
289	`static void json_string(StrBuf* sb, const char* s) {`	Технически объявляет/начинает функцию `json_string`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
290	`sb_append(sb, "\"");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_string`: добавляет текст в динамический буфер.
291	`json_escape(sb, s ? s : "");`	Технически вызывает `json_escape`. Логически это действие нужно в функции `json_string`: вызывает функцию `json_escape` для выполнения части алгоритма.
292	`sb_append(sb, "\"");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_string`: добавляет текст в динамический буфер.
293	`}`	Технически завершает область видимости внутри `json_string`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
295	`static void json_string_field(StrBuf* sb, const char* k, const char* v, bool trailing_comma) {`	Технически объявляет/начинает функцию `json_string_field`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
296	`json_key(sb, k);`	Технически вызывает `json_key`. Логически это действие нужно в функции `json_string_field`: вызывает функцию `json_key` для выполнения части алгоритма.
297	`json_string(sb, v);`	Технически вызывает `json_string`. Логически это действие нужно в функции `json_string_field`: вызывает функцию `json_string` для выполнения части алгоритма.
298	`if (trailing_comma) sb_append(sb, ",");`	Технически проверяет условие `trailing_comma) sb_append(sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
299	`}`	Технически завершает область видимости внутри `json_string_field`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
301	`static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma) {`	Технически объявляет/начинает функцию `json_bool_field`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
302	`json_key(sb, k);`	Технически вызывает `json_key`. Логически это действие нужно в функции `json_bool_field`: вызывает функцию `json_key` для выполнения части алгоритма.
303	`sb_append(sb, v ? "true" : "false");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_bool_field`: добавляет текст в динамический буфер.
304	`if (trailing_comma) sb_append(sb, ",");`	Технически проверяет условие `trailing_comma) sb_append(sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
305	`}`	Технически завершает область видимости внутри `json_bool_field`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
307	`static void json_heuristic_families(StrBuf* sb, uint32_t mask) {`	Технически объявляет/начинает функцию `json_heuristic_families`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
308	`sb_append(sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_heuristic_families`: добавляет текст в динамический буфер.
309	`bool first = true;`	Технически объявляет `first` в функции `json_heuristic_families` и сразу задает начальное значение `true`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
310	`uint32_t bits[] = {`	Технически присваивает `uint32_t bits[]` значение `{`. Логически это обновляет `uint32_t bits[]`: хранит промежуточное значение, которое дальше используется в текущей функции.
311	`HEF_TABLE_INTEGRITY,`	Технически эта строка является частью C-конструкции `HEF_TABLE_INTEGRITY,`. Логически она уточняет структуру блока функции `json_heuristic_families` и влияет на то, какие данные будут обработаны.
312	`HEF_UNWIND_INTEGRITY,`	Технически эта строка является частью C-конструкции `HEF_UNWIND_INTEGRITY,`. Логически она уточняет структуру блока функции `json_heuristic_families` и влияет на то, какие данные будут обработаны.
313	`HEF_METADATA_PLACEMENT,`	Технически эта строка является частью C-конструкции `HEF_METADATA_PLACEMENT,`. Логически она уточняет структуру блока функции `json_heuristic_families` и влияет на то, какие данные будут обработаны.
314	`HEF_UNWIND_REUSE,`	Технически эта строка является частью C-конструкции `HEF_UNWIND_REUSE,`. Логически она уточняет структуру блока функции `json_heuristic_families` и влияет на то, какие данные будут обработаны.
315	`HEF_HANDLER_DOMINANCE`	Технически эта строка является частью C-конструкции `HEF_HANDLER_DOMINANCE`. Логически она уточняет структуру блока функции `json_heuristic_families` и влияет на то, какие данные будут обработаны.
316	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
317	`for (size_t i = 0; i < sizeof(bits)/sizeof(bits[0]); i++) {`	Технически начинает цикл `size_t i = 0; i < sizeof(bits)/sizeof(bits[0]); i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
318	`if (!(mask & bits[i])) continue;`	Технически проверяет условие `!(mask & bits[i])`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
319	`if (!first) sb_append(sb, ",");`	Технически проверяет условие `!first) sb_append(sb, ","`; логически это защита от NULL-указателя и работы с неинициализированными данными.
320	`json_string(sb, heuristics_evidence_family_name(bits[i]));`	Технически вызывает `json_string`. Логически это действие нужно в функции `json_heuristic_families`: вызывает функцию `json_string` для выполнения части алгоритма.
321	`first = false;`	Технически присваивает `first` значение `false`. Логически это обновляет `first`: хранит промежуточное значение, которое дальше используется в текущей функции.
322	`}`	Технически завершает область видимости внутри `json_heuristic_families`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
323	`sb_append(sb, "]");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_heuristic_families`: добавляет текст в динамический буфер.
324	`}`	Технически завершает область видимости внутри `json_heuristic_families`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
326	`static unsigned dynamic_signal_count(const DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_signal_count`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
327	`unsigned n = 0;`	Технически присваивает `unsigned n` значение `0`. Логически это обновляет `unsigned n`: хранит промежуточное значение, которое дальше используется в текущей функции.
328	`if (!dr) return 0;`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
329	`if (dr->unwind_handler_hits) n++;`	Технически проверяет условие `dr->unwind_handler_hits`; логически отбирает записи, где важна корректность UNWIND_INFO.
330	`if (dr->veh_hits || dr->veh_registrations) n++;`	Технически проверяет условие `dr->veh_hits || dr->veh_registrations`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
331	`if (dr->vch_hits || dr->vch_registrations) n++;`	Технически проверяет условие `dr->vch_hits || dr->vch_registrations`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
332	`if (dr->uef_hits || dr->uef_set_calls) n++;`	Технически проверяет условие `dr->uef_hits || dr->uef_set_calls`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
333	`if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) n++;`	Технически проверяет условие `dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
334	`if (dr->second_chance_seen) n++;`	Технически проверяет условие `dr->second_chance_seen`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
335	`return n;`	Технически возвращает выражение `n`. Логически именно это значение становится результатом функции `dynamic_signal_count` для вызывающего кода.
336	`}`	Технически завершает область видимости внутри `dynamic_signal_count`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
338	`static bool dynamic_session_complete(const DynamicReport* dr) {`	Технически объявляет/начинает функцию `dynamic_session_complete`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
339	`return dr &&`	Технически возвращает выражение `dr &&`. Логически именно это значение становится результатом функции `dynamic_session_complete` для вызывающего кода.
340	`dr->ran &&`	Технически выполняет битовую операцию `dr->ran &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
341	`dr->ok &&`	Технически выполняет битовую операцию `dr->ok &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
342	`!dr->timed_out &&`	Технически выполняет битовую операцию `!dr->timed_out &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
343	`dr->target_exited &&`	Технически выполняет битовую операцию `dr->target_exited &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
344	`dr->instrumentation_complete;`	Технически выполняет оператор `dr->instrumentation_complete;`. Логически в функции `dynamic_session_complete` он меняет состояние, проверяемые данные или подготовку отчета.
345	`}`	Технически завершает область видимости внутри `dynamic_session_complete`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
347	`static bool has_static_dynamic_correlation(const HeuristicReport* hr, const DynamicReport* dr) {`	Технически объявляет/начинает функцию `has_static_dynamic_correlation`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
348	`bool suspicious_static_handler_context = false;`	Технически объявляет `suspicious_static_handler_context` в функции `has_static_dynamic_correlation` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
349	`if (!hr || !dynamic_session_complete(dr)) return false;`	Технически проверяет условие `!hr || !dynamic_session_complete(dr)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
350	`suspicious_static_handler_context =`	Технически присваивает `suspicious_static_handler_context` значение `. Логически это обновляет suspicious_static_handler_context`: хранит промежуточное значение, которое дальше используется в текущей функции.
351	`(hr->handler_nonexec_section > 0) ||`	Технически выполняет битовую операцию `(hr->handler_nonexec_section > 0) ||`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
352	`(hr->handler_writable_section > 0) ||`	Технически выполняет битовую операцию `(hr->handler_writable_section > 0) ||`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
353	`((hr->evidence_families_mask & HEF_HANDLER_DOMINANCE) != 0u);`	Технически выполняет битовую операцию `((hr->evidence_families_mask & HEF_HANDLER_DOMINANCE) != 0u);`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
354	`if (suspicious_static_handler_context && (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits)) return true;`	Технически проверяет условие `suspicious_static_handler_context && (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits)`; логически отбирает записи, где важна корректность UNWIND_INFO.
355	`if ((hr->unwind_exec_section || hr->unwind_unusual_section || hr->unwind_writable_section ||`	Технически проверяет условие `if ((hr->unwind_exec_section || hr->unwind_unusual_section || hr->unwind_writable_section ||`; логически отбирает записи, где важна корректность UNWIND_INFO.
356	`hr->handler_nonexec_section || hr->handler_writable_section) &&`	Технически выполняет битовую операцию `hr->handler_nonexec_section || hr->handler_writable_section) &&`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
357	`(dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls)) return true;`	Технически выполняет битовую операцию `(dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls)) return true;`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
358	`return false;`	Технически возвращает `false` из функции `has_static_dynamic_correlation`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
359	`}`	Технически завершает область видимости внутри `has_static_dynamic_correlation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
362	`static const char* runtime_evidence_level(const DynamicReport* dr) {`	Технически объявляет/начинает функцию `runtime_evidence_level`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
363	`if (!dr || !dr->ran) return "none";`	Технически проверяет условие `!dr || !dr->ran`; логически это защита от NULL-указателя и работы с неинициализированными данными.
364	`unsigned strong = 0, medium = 0, weak = 0;`	Технически присваивает `unsigned strong` значение `0, medium = 0, weak = 0`. Логически это обновляет `unsigned strong`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
365	`if (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits) strong++;`	Технически проверяет условие `dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits`; логически отбирает записи, где важна корректность UNWIND_INFO.
366	`if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) medium++;`	Технически проверяет условие `dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
367	`if (dr->second_chance_seen) medium++;`	Технически проверяет условие `dr->second_chance_seen`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
368	`if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls || dr->first_chance_seen) weak++;`	Технически проверяет условие `dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls || dr->first_chance_seen`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
369	`const char* level = "none";`	Технически объявляет `level` в функции `runtime_evidence_level` и сразу задает начальное значение `"none"`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
370	`if (strong >= 1 && medium >= 1) level = "strong";`	Технически проверяет условие `strong >= 1 && medium >= 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
371	`else if (strong >= 1 || medium >= 2) level = "medium";`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strong >= 1 || medium >= 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
372	`else if (medium >= 1 || weak >= 1) level = "weak";`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `medium >= 1 || weak >= 1`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
373	`if (!dynamic_session_complete(dr) && strcmp(level, "none") != 0) return "weak";`	Технически проверяет условие `!dynamic_session_complete(dr) && strcmp(level, "none") != 0`; логически это защита от NULL-указателя и работы с неинициализированными данными.
374	`return level;`	Технически возвращает выражение `level`. Логически именно это значение становится результатом функции `runtime_evidence_level` для вызывающего кода.
375	`}`	Технически завершает область видимости внутри `runtime_evidence_level`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
377	`static const char* case_strength(const HeuristicReport* hr, const DynamicReport* dr, bool correlated) {`	Технически объявляет/начинает функцию `case_strength`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
378	`const char* static_lvl = (hr && hr->evidence_level[0]) ? hr->evidence_level : "none";`	Технически присваивает `const char* static_lvl` значение `(hr && hr->evidence_level[0]) ? hr->evidence_level : "none"`. Логически это обновляет `const char* static_lvl`: хранит промежуточное значение, которое дальше используется в текущей функции.
379	`const char* runtime_lvl = runtime_evidence_level(dr);`	Технически присваивает `const char* runtime_lvl` значение `runtime_evidence_level(dr)`. Логически это обновляет `const char* runtime_lvl`: хранит промежуточное значение, которое дальше используется в текущей функции.
380	`bool static_strong = strcmp(static_lvl, "strong") == 0;`	Технически присваивает `bool static_strong` значение `strcmp(static_lvl, "strong") == 0`. Логически это обновляет `bool static_strong`: хранит промежуточное значение, которое дальше используется в текущей функции.
381	`bool static_medium = strcmp(static_lvl, "medium") == 0;`	Технически присваивает `bool static_medium` значение `strcmp(static_lvl, "medium") == 0`. Логически это обновляет `bool static_medium`: хранит промежуточное значение, которое дальше используется в текущей функции.
382	`bool static_weak = strcmp(static_lvl, "weak") == 0;`	Технически присваивает `bool static_weak` значение `strcmp(static_lvl, "weak") == 0`. Логически это обновляет `bool static_weak`: хранит промежуточное значение, которое дальше используется в текущей функции.
383	`bool runtime_strong = strcmp(runtime_lvl, "strong") == 0;`	Технически присваивает `bool runtime_strong` значение `strcmp(runtime_lvl, "strong") == 0`. Логически это обновляет `bool runtime_strong`: хранит промежуточное значение, которое дальше используется в текущей функции.
384	`bool runtime_medium = strcmp(runtime_lvl, "medium") == 0;`	Технически присваивает `bool runtime_medium` значение `strcmp(runtime_lvl, "medium") == 0`. Логически это обновляет `bool runtime_medium`: хранит промежуточное значение, которое дальше используется в текущей функции.
385	`bool runtime_weak = strcmp(runtime_lvl, "weak") == 0;`	Технически присваивает `bool runtime_weak` значение `strcmp(runtime_lvl, "weak") == 0`. Логически это обновляет `bool runtime_weak`: хранит промежуточное значение, которое дальше используется в текущей функции.
387	`if (correlated && (static_strong || runtime_strong)) return "strong_correlated_case";`	Технически проверяет условие `correlated && (static_strong || runtime_strong)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
388	`if (correlated && (static_medium || runtime_medium)) return "medium_correlated_case";`	Технически проверяет условие `correlated && (static_medium || runtime_medium)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
389	`if (static_strong || runtime_strong) return "strong_unconfirmed_case";`	Технически проверяет условие `static_strong || runtime_strong`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
390	`if (static_medium || runtime_medium) return "medium_case";`	Технически проверяет условие `static_medium || runtime_medium`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
391	`if (static_weak || runtime_weak) return "weak_case";`	Технически проверяет условие `static_weak || runtime_weak`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
392	`return "no_case";`	Технически возвращает выражение `"no_case"`. Логически именно это значение становится результатом функции `case_strength` для вызывающего кода.
393	`}`	Технически завершает область видимости внутри `case_strength`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
395	`typedef struct AssessmentSummary {`	Технически начинает описание структуры `AssessmentSummary`. Логически структура собирает связанные поля в одну модель данных проекта.
396	`bool correlated;`	Технически объявляет поле `correlated` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
397	`unsigned dynamic_signals;`	Технически объявляет поле `dynamic_signals` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
398	`double final_confidence;`	Технически объявляет поле `final_confidence` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
399	`const char* final_verdict;`	Технически объявляет поле `final_verdict` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
400	`const char* static_evidence_level;`	Технически объявляет поле `static_evidence_level` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
401	`const char* runtime_evidence_level;`	Технически объявляет поле `runtime_evidence_level` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
402	`const char* strength;`	Технически объявляет поле `strength` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
403	`} AssessmentSummary;`	Технически завершает typedef и дает типу имя `AssessmentSummary`. Логически дальше код может объявлять переменные этого типа без `struct`.

Стр.	Код	Нормальное объяснение
405	`static AssessmentSummary assessment_summary(const HeuristicReport* hr,`	Технически вызывает `assessment_summary`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `assessment_summary` для выполнения части алгоритма.
406	`const DynamicReport* dr) {`	Технически эта строка является частью C-конструкции `const DynamicReport* dr) {`. Логически она уточняет структуру блока функции `assessment_summary` и влияет на то, какие данные будут обработаны.
407	`AssessmentSummary summary;`	Технически объявляет поле `summary` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
408	`summary.correlated = has_static_dynamic_correlation(hr, dr);`	Технически присваивает `summary.correlated` значение `has_static_dynamic_correlation(hr, dr)`. Логически это обновляет `correlated`: хранит промежуточное значение, которое дальше используется в текущей функции.
409	`summary.dynamic_signals = dynamic_signal_count(dr);`	Технически присваивает `summary.dynamic_signals` значение `dynamic_signal_count(dr)`. Логически это обновляет `dynamic_signals`: хранит промежуточное значение, которое дальше используется в текущей функции.
410	`summary.final_confidence = hr ? hr->confidence : 0.10;`	Технически присваивает `summary.final_confidence` значение `hr ? hr->confidence : 0.10`. Логически это обновляет `final_confidence`: хранит промежуточное значение, которое дальше используется в текущей функции.
411	`summary.final_verdict = "no_strong_correlated_evidence";`	Технически присваивает `summary.final_verdict` значение `"no_strong_correlated_evidence"`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.
412	`summary.static_evidence_level = hr ? hr->evidence_level : "none";`	Технически присваивает `summary.static_evidence_level` значение `hr ? hr->evidence_level : "none"`. Логически это обновляет `static_evidence_level`: хранит промежуточное значение, которое дальше используется в текущей функции.
413	`summary.runtime_evidence_level = runtime_evidence_level(dr);`	Технически присваивает `summary.runtime_evidence_level` значение `runtime_evidence_level(dr)`. Логически это обновляет `runtime_evidence_level`: хранит промежуточное значение, которое дальше используется в текущей функции.
414	`summary.strength = case_strength(hr, dr, summary.correlated);`	Технически присваивает `summary.strength` значение `case_strength(hr, dr, summary.correlated)`. Логически это обновляет `strength`: хранит промежуточное значение, которое дальше используется в текущей функции.
416	`if (dynamic_session_complete(dr) && summary.dynamic_signals) {`	Технически проверяет условие `dynamic_session_complete(dr) && summary.dynamic_signals`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
417	`summary.final_confidence += 0.10;`	Технически изменяет `summary.final_confidence` оператором `+=` на основе `0.10`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
418	`}`	Технически завершает область видимости внутри `assessment_summary`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
419	`if (summary.correlated) summary.final_confidence += 0.10;`	Технически проверяет условие `summary.correlated`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
420	`if (summary.final_confidence > 0.98) summary.final_confidence = 0.98;`	Технически проверяет условие `summary.final_confidence > 0.98`; логически выбирает уровень оценки или уверенности.
422	`if (hr) {`	Технически проверяет условие `hr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
423	`if (hr->score >= 70) {`	Технически проверяет условие `hr->score >= 70`; логически выбирает уровень оценки или уверенности.
424	`summary.final_verdict = summary.correlated`	Технически присваивает `summary.final_verdict` значение `summary.correlated`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.
425	`? "high_suspicion_correlated"`	Технически эта строка является частью C-конструкции `? "high_suspicion_correlated"`. Логически она уточняет структуру блока функции `assessment_summary` и влияет на то, какие данные будут обработаны.
426	`: "high_suspicion_static";`	Технически выполняет оператор `: "high_suspicion_static";`. Логически в функции `assessment_summary` он меняет состояние, проверяемые данные или подготовку отчета.
427	`} else if (hr->score >= 40) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
428	`summary.final_verdict = summary.correlated`	Технически присваивает `summary.final_verdict` значение `summary.correlated`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.
429	`? "moderate_suspicion_correlated"`	Технически эта строка является частью C-конструкции `? "moderate_suspicion_correlated"`. Логически она уточняет структуру блока функции `assessment_summary` и влияет на то, какие данные будут обработаны.
430	`: "moderate_suspicion_static";`	Технически выполняет оператор `: "moderate_suspicion_static";`. Логически в функции `assessment_summary` он меняет состояние, проверяемые данные или подготовку отчета.
431	`} else if (hr->score >= 15) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
432	`summary.final_verdict = summary.correlated`	Технически присваивает `summary.final_verdict` значение `summary.correlated`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.
433	`? "low_suspicion_correlated"`	Технически эта строка является частью C-конструкции `? "low_suspicion_correlated"`. Логически она уточняет структуру блока функции `assessment_summary` и влияет на то, какие данные будут обработаны.
434	`: "low_suspicion_static";`	Технически выполняет оператор `: "low_suspicion_static";`. Логически в функции `assessment_summary` он меняет состояние, проверяемые данные или подготовку отчета.
435	`} else if (dynamic_session_complete(dr) && summary.dynamic_signals) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
436	`summary.final_verdict = "runtime_signals_only";`	Технически присваивает `summary.final_verdict` значение `"runtime_signals_only"`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.
437	`}`	Технически завершает область видимости внутри `assessment_summary`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
438	`} else if (dynamic_session_complete(dr) && summary.dynamic_signals) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
439	`summary.final_verdict = "runtime_signals_only";`	Технически присваивает `summary.final_verdict` значение `"runtime_signals_only"`. Логически это обновляет `final_verdict`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
440	`}`	Технически завершает область видимости внутри `assessment_summary`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
441	`return summary;`	Технически возвращает выражение `summary`. Логически именно это значение становится результатом функции `assessment_summary` для вызывающего кода.
442	`}`	Технически завершает область видимости внутри `assessment_summary`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
444	`static bool heuristics_has_finding_code(const HeuristicReport* hr, const char* code) {`	Технически объявляет/начинает функцию `heuristics_has_finding_code`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
445	`if (!hr || !code) return false;`	Технически проверяет условие `!hr || !code`; логически это защита от NULL-указателя и работы с неинициализированными данными.
446	`for (size_t i = 0; i < hr->finding_count; i++) {`	Технически начинает цикл `size_t i = 0; i < hr->finding_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
447	`if (strcmp(hr->findings[i].code, code) == 0) return true;`	Технически проверяет условие `strcmp(hr->findings[i].code, code) == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
448	`}`	Технически завершает область видимости внутри `heuristics_has_finding_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
449	`return false;`	Технически возвращает `false` из функции `heuristics_has_finding_code`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
450	`}`	Технически завершает область видимости внутри `heuristics_has_finding_code`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
452	`static bool heuristics_has_family(const HeuristicReport* hr, const char* family) {`	Технически объявляет/начинает функцию `heuristics_has_family`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
453	`if (!hr || !family) return false;`	Технически проверяет условие `!hr || !family`; логически это защита от NULL-указателя и работы с неинициализированными данными.
454	`for (size_t i = 0; i < hr->finding_count; i++) {`	Технически начинает цикл `size_t i = 0; i < hr->finding_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
455	`if (strcmp(hr->findings[i].family, family) == 0) return true;`	Технически проверяет условие `strcmp(hr->findings[i].family, family) == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
456	`}`	Технически завершает область видимости внутри `heuristics_has_family`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
457	`return false;`	Технически возвращает `false` из функции `heuristics_has_family`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
458	`}`	Технически завершает область видимости внутри `heuristics_has_family`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
460	`typedef struct ManualValidationSteps {`	Технически начинает описание структуры `ManualValidationSteps`. Логически структура собирает связанные поля в одну модель данных проекта.
461	`const char* items[16];`	Технически объявляет поле `items` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
462	`size_t count;`	Технически объявляет поле `count` внутри структуры. Логически это поле: хранит размер или количество, поэтому используется в проверках границ.
463	`} ManualValidationSteps;`	Технически завершает typedef и дает типу имя `ManualValidationSteps`. Логически дальше код может объявлять переменные этого типа без `struct`.
465	`static ManualValidationSteps collect_manual_validation_steps(const HeuristicReport* hr,`	Технически вызывает `collect_manual_validation_steps`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `collect_manual_validation_steps` для выполнения части алгоритма.
466	`const DynamicReport* dr) {`	Технически эта строка является частью C-конструкции `const DynamicReport* dr) {`. Логически она уточняет структуру блока функции `collect_manual_validation_steps` и влияет на то, какие данные будут обработаны.
467	`ManualValidationSteps steps;`	Технически объявляет поле `steps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
468	`memset(&steps, 0, sizeof(steps));`	Технически вызывает `memset`. Логически это действие нужно в функции `collect_manual_validation_steps`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
469	`#define ADD_STEP(txt) do { \`	Технически объявляет макрос `ADD_STEP` со значением `(txt) do { \`. Логически это именованная константа, чтобы не разбрасывать магические числа по коду.
470	`if (steps.count < sizeof(steps.items) / sizeof(steps.items[0])) { \`	Технически проверяет условие `steps.count < sizeof(steps.items) / sizeof(steps.items[0])`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
471	`steps.items[steps.count++] = (txt); \`	Технически присваивает `steps.items[steps.count++]` значение `(txt); \`. Логически это обновляет `count++]`: хранит промежуточное значение, которое дальше используется в текущей функции.
472	`} \`	Технически эта строка является частью C-конструкции `} \`. Логически она уточняет структуру блока функции `collect_manual_validation_steps` и влияет на то, какие данные будут обработаны.
473	`} while (0)`	Технически объявляет/начинает функцию `while`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
474	`if (hr) {`	Технически проверяет условие `hr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
475	`if (hr->rf_invalid_ranges || hr->rf_oob || hr->rf_unwind_oob || hr->rf_overlaps) {`	Технически проверяет условие `hr->rf_invalid_ranges || hr->rf_oob || hr->rf_unwind_oob || hr->rf_overlaps`; логически отбирает записи, где важна корректность UNWIND_INFO.
476	`ADD_STEP("Validate .pdata range integrity in a PE viewer, sort the RUNTIME_FUNCTION entries manually and confirm that begin/end/unwind RVAs stay inside SizeOfImage.");`	Технически вызывает `ADD_STEP`. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию `ADD_STEP` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
477	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
478	`if (hr->unwind_parse_fail || hr->unwind_anomalies || heuristics_has_family(hr, "unwind_integrity")) {`	Технически проверяет условие `hr->unwind_parse_fail || hr->unwind_anomalies || heuristics_has_family(hr, "unwind_integrity")`; логически отбирает записи, где важна корректность UNWIND_INFO.
479	`ADD_STEP("Open the suspicious UNWIND_INFO blob in a hex view, verify Version/Flags/CountOfCodes/tail layout, then compare the decoded prolog actions with the function prolog in Ghidra.");`	Технически вызывает `ADD_STEP`. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию `ADD_STEP` для выполнения части алгоритма.
480	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
481	`if (hr->unwind_unusual_section || hr->unwind_exec_section || hr->unwind_writable_section || heuristics_has_family(hr, "metadata_placement")) {`	Технически проверяет условие `hr->unwind_unusual_section || hr->unwind_exec_section || hr->unwind_writable_section || heuristics_has_family(hr, "me...`; логически отбирает записи, где важна корректность UNWIND_INFO.
482	`ADD_STEP("Inspect the section hosting the unwind metadata, record its RVA/raw offset/permissions and decide whether the placement matches a benign compiler layout or looks redirected/manual.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
483	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
484	`if (hr->handler_nonexec_section) {`	Технически проверяет условие `hr->handler_nonexec_section`; логически отделяет функции с language-specific handler от обычных функций.
485	`ADD_STEP("Disassemble the reported handler target, confirm execute permission and check whether the landing bytes belong to a real handler, a thunk or a non-code blob.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
486	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
487	`if (hr->unwind_writable_section || hr->handler_writable_section) {`	Технически проверяет условие `hr->unwind_writable_section || hr->handler_writable_section`; логически отбирает записи, где важна корректность UNWIND_INFO.
488	`ADD_STEP("Check whether writable unwind metadata or writable handler targets are justified by a JIT/runtime scenario; if not, treat the placement as suspicious and correlate it with memory protections at runtime.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
489	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
490	`if (heuristics_has_finding_code(hr, "handler_dominance") || heuristics_has_family(hr, "handler_dominance")) {`	Технически проверяет условие `heuristics_has_finding_code(hr, "handler_dominance") || heuristics_has_family(hr, "handler_dominance")`; логически отделяет функции с language-specific handler от обычных функций.
491	`ADD_STEP("Open the dominant handler target in Ghidra and compare several referencing functions to decide whether this is a normal runtime helper or a suspicious central dispatcher.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
492	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
493	`if (hr->mitigation_count > 0) {`	Технически проверяет условие `hr->mitigation_count > 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
494	`ADD_STEP("Review weakening rules before making a final conclusion; the current evidence set still has benign explanations that must be excluded manually.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
495	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
496	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
497	`if (dr) {`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
498	`if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) {`	Технически проверяет условие `dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
499	`ADD_STEP("Capture the arguments of RtlAddFunctionTable / RtlInstallFunctionTableCallback and inspect the runtime table contents together with the surrounding module/section mapping.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
500	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
501	`if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls) {`	Технически проверяет условие `dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
502	`ADD_STEP("List each VEH/VCH/UEF registration with its target address, then single-step from the registered routine to confirm whether it behaves like a benign runtime hook or an exception interceptor.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.
503	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
504	`if (dr->veh_hits || dr->vch_hits || dr->uef_hits) {`	Технически проверяет условие `dr->veh_hits || dr->vch_hits || dr->uef_hits`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
505	`ADD_STEP("Single-step from the handler hit and map module/section transitions immediately after the exception dispatch point.");`	Технически вызывает ADD_STEP. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию ADD_STEP для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
506	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
507	`if (dr->unwind_handler_hits) {`	Технически проверяет условие `dr->unwind_handler_hits`; логически отбирает записи, где важна корректность UNWIND_INFO.
508	`ADD_STEP("Verify whether the hit unwind handler corresponds to a known language/runtime handler or to a dispatcher/interceptor path referenced by many protected functions.");`	Технически вызывает `ADD_STEP`. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию `ADD_STEP` для выполнения части алгоритма.
509	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
510	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
511	`if ((!dr || dynamic_signal_count(dr) == 0) && hr && (hr->score > 0 || hr->finding_count > 0)) {`	Технически проверяет условие `(!dr || dynamic_signal_count(dr) == 0) && hr && (hr->score > 0 || hr->finding_count > 0)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
512	`ADD_STEP("Run the sample again with --dynamic on a representative exception path to obtain runtime corroboration.");`	Технически вызывает `ADD_STEP`. Логически это действие нужно в функции `collect_manual_validation_steps`: вызывает функцию `ADD_STEP` для выполнения части алгоритма.
513	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
514	`#undef ADD_STEP`	Технически это директива препроцессора `#undef ADD_STEP`. Логически она меняет код еще до компиляции C.
515	`return steps;`	Технически возвращает выражение `steps`. Логически именно это значение становится результатом функции `collect_manual_validation_steps` для вызывающего кода.
516	`}`	Технически завершает область видимости внутри `collect_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
518	`static void json_manual_validation_steps(StrBuf* sb, const HeuristicReport* hr, const DynamicReport* dr) {`	Технически объявляет/начинает функцию `json_manual_validation_steps`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
519	`ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);`	Технически присваивает `ManualValidationSteps steps` значение `collect_manual_validation_steps(hr, dr)`. Логически это обновляет `ManualValidationSteps steps`: хранит промежуточное значение, которое дальше используется в текущей функции.
520	`sb_append(sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_manual_validation_steps`: добавляет текст в динамический буфер.
521	`for (size_t i = 0; i < steps.count; i++) {`	Технически начинает цикл `size_t i = 0; i < steps.count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
522	`if (i) sb_append(sb, ",");`	Технически проверяет условие `i) sb_append(sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
523	`json_string(sb, steps.items[i]);`	Технически вызывает `json_string`. Логически это действие нужно в функции `json_manual_validation_steps`: вызывает функцию `json_string` для выполнения части алгоритма.
524	`}`	Технически завершает область видимости внутри `json_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
525	`sb_append(sb, "]");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `json_manual_validation_steps`: добавляет текст в динамический буфер.
526	`}`	Технически завершает область видимости внутри `json_manual_validation_steps`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
528	`void report_print_heuristics(const HeuristicReport* hr) {`	Технически объявляет/начинает функцию `report_print_heuristics`; она доступная другим модулям через заголовок или линковку. Логически функция печатает найденные эвристики в консоль.
529	`if (!hr) return;`	Технически проверяет условие `!hr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
530	`printf("\n[Heuristic Analysis] Suspicion score: %d / 100\n", hr->score);`	Технически вызывает `printf`. Логически это действие нужно в функции `report_print_heuristics`: выводит информацию пользователю в консоль.
531	`printf(" suspicion=%.3f confidence=%.3f verdict=%s manual_review=%s\n",`	Технически присваивает `printf(" suspicion` значение `%.3f confidence=%.3f verdict=%s manual_review=%s\n",`. Логически это обновляет `printf(" suspicion`: хранит промежуточное значение, которое дальше используется в текущей функции.
532	`hr->suspicion, hr->confidence,`	Технически эта строка является частью C-конструкции `hr->suspicion, hr->confidence,`. Логически она уточняет структуру блока функции `report_print_heuristics` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
533	`hr->verdict[0] ? hr->verdict : "-",`	Технически эта строка является частью C-конструкции `hr->verdict[0] ? hr->verdict : "-",`. Логически она уточняет структуру блока функции `report_print_heuristics` и влияет на то, какие данные будут обработаны.
534	`hr->requires_manual_review ? "yes" : "no");`	Технически выполняет оператор `hr->requires_manual_review ? "yes" : "no");`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
535	`printf(" total=%zu invalid_ranges=%zu oob=%zu unwind_oob=%zu\n",`	Технически присваивает `printf(" total` значение `%zu invalid_ranges=%zu oob=%zu unwind_oob=%zu\n",`. Логически это обновляет `printf(" total`: хранит промежуточное значение, которое дальше используется в текущей функции.
536	`hr->total, hr->rf_invalid_ranges, hr->rf_oob, hr->rf_unwind_oob);`	Технически выполняет оператор `hr->total, hr->rf_invalid_ranges, hr->rf_oob, hr->rf_unwind_oob);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
537	`printf(" unsorted=%zu overlaps=%zu duplicate_begins=%zu unwind_parse_fail=%zu unwind_anom=%zu\n",`	Технически присваивает `printf(" unsorted` значение `%zu overlaps=%zu duplicate_begins=%zu unwind_parse_fail=%zu unwind_anom=%zu\n",`. Логически это обновляет `printf(" unsorted`: хранит промежуточное значение, которое дальше используется в текущей функции.
538	`hr->rf_unsorted, hr->rf_overlaps, hr->rf_duplicate_begins,`	Технически эта строка является частью C-конструкции `hr->rf_unsorted, hr->rf_overlaps, hr->rf_duplicate_begins,`. Логически она уточняет структуру блока функции `report_print_heuristics` и влияет на то, какие данные будут обработаны.
539	`hr->unwind_parse_fail, hr->unwind_anomalies);`	Технически выполняет оператор `hr->unwind_parse_fail, hr->unwind_anomalies);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
540	`printf(" strict_abi_warnings=%zu\n", hr->unwind_abi_warnings);`	Технически присваивает `printf(" strict_abi_warnings` значение `%zu\n", hr->unwind_abi_warnings)`. Логически это обновляет `printf(" strict_abi_warnings`: хранит промежуточное значение, которое дальше используется в текущей функции.
541	`printf(" unwind_unusual_section=%zu unwind_exec_section=%zu unwind_writable_section=%zu\n",`	Технически присваивает `printf(" unwind_unusual_section` значение `%zu unwind_exec_section=%zu unwind_writable_section=%zu\n",`. Логически это обновляет `printf(" unwind_unusual_section`: хранит промежуточное значение, которое дальше используется в текущей функции.
542	`hr->unwind_unusual_section, hr->unwind_exec_section, hr->unwind_writable_section);`	Технически выполняет оператор `hr->unwind_unusual_section, hr->unwind_exec_section, hr->unwind_writable_section);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
543	`printf(" handler_nonexec_section=%zu handler_writable_section=%zu\n",`	Технически присваивает `printf(" handler_nonexec_section` значение `%zu handler_writable_section=%zu\n",`. Логически это обновляет `printf(" handler_nonexec_section`: хранит промежуточное значение, которое дальше используется в текущей функции.
544	`hr->handler_nonexec_section, hr->handler_writable_section);`	Технически выполняет оператор `hr->handler_nonexec_section, hr->handler_writable_section);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
545	`printf(" unwind_reuse_ratio=%.3f unwind_unique_ratio=%.3f\n",`	Технически присваивает `printf(" unwind_reuse_ratio` значение `%.3f unwind_unique_ratio=%.3f\n",`. Логически это обновляет `printf(" unwind_reuse_ratio`: хранит промежуточное значение, которое дальше используется в текущей функции.
546	`hr->unwind_reuse_ratio, hr->unwind_unique_ratio);`	Технически выполняет оператор `hr->unwind_reuse_ratio, hr->unwind_unique_ratio);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
547	`if (hr->handler_count) {`	Технически проверяет условие `hr->handler_count`; логически отделяет функции с language-specific handler от обычных функций.
548	`printf(" handler_count=%zu handler_reuse_ratio=%.3f\n",`	Технически присваивает `printf(" handler_count` значение `%zu handler_reuse_ratio=%.3f\n",`. Логически это обновляет `printf(" handler_count`: хранит размер или количество, поэтому используется в проверках границ.
549	`hr->handler_count, hr->handler_reuse_ratio);`	Технически выполняет оператор `hr->handler_count, hr->handler_reuse_ratio);`. Логически в функции `report_print_heuristics` он меняет состояние, проверяемые данные или подготовку отчета.
550	`}`	Технически завершает область видимости внутри `report_print_heuristics`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
551	`printf(" evidence_families=%zu findings=%zu\n",`	Технически присваивает `printf(" evidence_families` значение `%zu findings=%zu\n",`. Логически это обновляет `printf(" evidence_families`: хранит промежуточное значение, которое дальше используется в текущей функции.
552	`heuristics_evidence_family_count(hr->evidence_families_mask), hr->finding_count);`	Технически вызывает `heuristics_evidence_family_count`. Логически это действие нужно в функции `report_print_heuristics`: считает, сколько разных семейств доказательств сработало.
553	`printf("Findings:\n%s\n", hr->details.s ? hr->details.s : "");`	Технически вызывает `printf`. Логически это действие нужно в функции `report_print_heuristics`: выводит информацию пользователю в консоль.
554	`}`	Технически завершает область видимости внутри `report_print_heuristics`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
556	`static bool report_metadata_valid(const ReportMetadata* metadata) {`	Технически объявляет/начинает функцию `report_metadata_valid`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
557	`if (!metadata ||`	Технически проверяет условие `if (!metadata ||`; логически это защита от NULL-указателя и работы с неинициализированными данными.
558	`!metadata->tool_version || metadata->tool_version[0] == '\0' ||`	Технически выполняет битовую операцию `!metadata->tool_version || metadata->tool_version[0] == '\0' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
559	`!metadata->report_schema_version ||`	Технически выполняет битовую операцию `!metadata->report_schema_version ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
560	`strcmp(metadata->report_schema_version, EHUA_REPORT_SCHEMA_VERSION) != 0 ||`	Технически вызывает `strcmp`. Логически это действие нужно в функции `report_metadata_valid`: сравнивает две C-строки.
561	`!metadata->generated_at_utc || metadata->generated_at_utc[0] == '\0' ||`	Технически выполняет битовую операцию `!metadata->generated_at_utc || metadata->generated_at_utc[0] == '\0' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
562	`!metadata->input_sha256 || strlen(metadata->input_sha256) != 64u ||`	Технически вызывает `strlen`. Логически это действие нужно в функции `report_metadata_valid`: измеряет длину C-строки до нулевого байта.
563	`!metadata->command_line ||`	Технически выполняет битовую операцию `!metadata->command_line ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
564	`!metadata->build_compiler || metadata->build_compiler[0] == '\0' ||`	Технически выполняет битовую операцию `!metadata->build_compiler || metadata->build_compiler[0] == '\0' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
565	`!metadata->build_platform || metadata->build_platform[0] == '\0' ||`	Технически выполняет битовую операцию `!metadata->build_platform || metadata->build_platform[0] == '\0' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
566	`!metadata->build_mode ||`	Технически выполняет битовую операцию `!metadata->build_mode ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
567	`(strcmp(metadata->build_mode, "debug") != 0 &&`	Технически вызывает `strcmp`. Логически это действие нужно в функции `report_metadata_valid`: сравнивает две C-строки.
568	`strcmp(metadata->build_mode, "release") != 0) ||`	Технически вызывает `strcmp`. Логически это действие нужно в функции `report_metadata_valid`: сравнивает две C-строки.
569	`metadata->max_chain_depth < 1u ||`	Технически выполняет битовую операцию `metadata->max_chain_depth < 1u ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
570	`metadata->max_chain_depth > 16u) {`	Технически эта строка является частью C-конструкции `metadata->max_chain_depth > 16u) {`. Логически она уточняет структуру блока функции `report_metadata_valid` и влияет на то, какие данные будут обработаны.
571	`return false;`	Технически возвращает `false` из функции `report_metadata_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
572	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
573	`for (size_t i = 0; i < 64u; i++) {`	Технически начинает цикл `size_t i = 0; i < 64u; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
574	`const char c = metadata->input_sha256[i];`	Технически объявляет `c` в функции `report_metadata_valid` и сразу задает начальное значение `metadata->input_sha256[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
575	`if (!((c >= '0' && c <= '9') || (c >= 'a' && c <= 'f'))) return false;`	Технически проверяет условие `!((c >= '0' && c <= '9') || (c >= 'a' && c <= 'f'))`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
576	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
577	`if (strlen(metadata->generated_at_utc) != 20u ||`	Технически проверяет условие `strlen(metadata->generated_at_utc`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
578	`metadata->generated_at_utc[4] != '-' ||`	Технически выполняет битовую операцию `metadata->generated_at_utc[4] != '-' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
579	`metadata->generated_at_utc[7] != '-' ||`	Технически выполняет битовую операцию `metadata->generated_at_utc[7] != '-' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
580	`metadata->generated_at_utc[10] != 'T' ||`	Технически выполняет битовую операцию `metadata->generated_at_utc[10] != 'T' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
581	`metadata->generated_at_utc[13] != ':' ||`	Технически выполняет битовую операцию `metadata->generated_at_utc[13] != ':' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
582	`metadata->generated_at_utc[16] != ':' ||`	Технически выполняет битовую операцию `metadata->generated_at_utc[16] != ':' ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
583	`metadata->generated_at_utc[19] != 'Z') {`	Технически эта строка является частью C-конструкции `metadata->generated_at_utc[19] != 'Z') {`. Логически она уточняет структуру блока функции `report_metadata_valid` и влияет на то, какие данные будут обработаны.
584	`return false;`	Технически возвращает `false` из функции `report_metadata_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
585	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
586	`for (size_t i = 0; i < 19u; i++) {`	Технически начинает цикл `size_t i = 0; i < 19u; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
587	`if (i == 4u || i == 7u || i == 10u || i == 13u || i == 16u) continue;`	Технически проверяет условие `i == 4u || i == 7u || i == 10u || i == 13u || i == 16u`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
588	`if (metadata->generated_at_utc[i] < '0' ||`	Технически проверяет условие `if (metadata->generated_at_utc[i] < '0' ||`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
589	`metadata->generated_at_utc[i] > '9') {`	Технически эта строка является частью C-конструкции `metadata->generated_at_utc[i] > '9') {`. Логически она уточняет структуру блока функции `report_metadata_valid` и влияет на то, какие данные будут обработаны.
590	`return false;`	Технически возвращает `false` из функции `report_metadata_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
591	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
592	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
593	`return true;`	Технически возвращает `true` из функции `report_metadata_valid`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
594	`}`	Технически завершает область видимости внутри `report_metadata_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
596	`static bool report_model_valid(const PeImage* img,`	Технически вызывает `report_model_valid`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `report_model_valid` для выполнения части алгоритма.
597	`const HeuristicReport* hr,`	Технически эта строка является частью C-конструкции `const HeuristicReport* hr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
598	`const DynamicReport* dr) {`	Технически эта строка является частью C-конструкции `const DynamicReport* dr) {`. Логически она уточняет структуру блока функции `report_model_valid` и влияет на то, какие данные будут обработаны.
599	`if (!img || (img->num_sections && !img->sections)) return false;`	Технически проверяет условие `!img || (img->num_sections && !img->sections)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
600	`if (hr && (hr->finding_count > HEURISTICS_MAX_FINDINGS ||`	Технически проверяет условие `if (hr && (hr->finding_count > HEURISTICS_MAX_FINDINGS ||`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
601	`hr->mitigation_count > HEURISTICS_MAX_MITIGATIONS)) {`	Технически эта строка является частью C-конструкции `hr->mitigation_count > HEURISTICS_MAX_MITIGATIONS)) {`. Логически она уточняет структуру блока функции `report_model_valid` и влияет на то, какие данные будут обработаны.
602	`return false;`	Технически возвращает `false` из функции `report_model_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
603	`}`	Технически завершает область видимости внутри `report_model_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
604	`if (dr && (dr->event_count > DYNAMIC_MAX_EVENTS ||`	Технически проверяет условие `if (dr && (dr->event_count > DYNAMIC_MAX_EVENTS ||`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
605	`dr->trace_count > sizeof(dr->trace) / sizeof(dr->trace[0]) ||`	Технически вызывает `sizeof`. Логически это действие нужно в функции `report_model_valid`: вызывает функцию `sizeof` для выполнения части алгоритма.
606	`dr->trace_steps > sizeof(dr->trace) / sizeof(dr->trace[0]))) {`	Технически объявляет/начинает функцию `sizeof`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
607	`return false;`	Технически возвращает `false` из функции `report_model_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
608	`}`	Технически завершает область видимости внутри `report_model_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
609	`if (dr && dr->instrumentation_complete &&`	Технически проверяет условие `if (dr && dr->instrumentation_complete &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
610	`(dr->module_refresh_failures ||`	Технически выполняет битовую операцию `(dr->module_refresh_failures ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
611	`dr->module_list_truncations ||`	Технически выполняет битовую операцию `dr->module_list_truncations ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
612	`dr->breakpoint_set_failures ||`	Технически выполняет битовую операцию `dr->breakpoint_set_failures ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
613	`dr->breakpoint_capacity_drops ||`	Технически выполняет битовую операцию `dr->breakpoint_capacity_drops ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
614	`dr->pending_rearm_failures ||`	Технически выполняет битовую операцию `dr->pending_rearm_failures ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
615	`dr->breakpoint_rearm_failures ||`	Технически выполняет битовую операцию `dr->breakpoint_rearm_failures ||`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
616	`dr->dropped_events)) {`	Технически эта строка является частью C-конструкции `dr->dropped_events)) {`. Логически она уточняет структуру блока функции `report_model_valid` и влияет на то, какие данные будут обработаны.
617	`return false;`	Технически возвращает `false` из функции `report_model_valid`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
618	`}`	Технически завершает область видимости внутри `report_model_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
619	`return true;`	Технически возвращает `true` из функции `report_model_valid`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
620	`}`	Технически завершает область видимости внутри `report_model_valid`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
622	`bool report_write_json_ex(const char* path,`	Технически вызывает `report_write_json_ex`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `report_write_json_ex` для выполнения части алгоритма.
623	`const char* input_path,`	Технически эта строка является частью C-конструкции `const char* input_path,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
624	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
625	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
626	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
627	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
628	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
629	`const HeuristicReport* hr,`	Технически эта строка является частью C-конструкции `const HeuristicReport* hr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
630	`const DynamicReport* dr,`	Технически эта строка является частью C-конструкции `const DynamicReport* dr,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
631	`const ReportMetadata* metadata,`	Технически эта строка является частью C-конструкции `const ReportMetadata* metadata,`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
632	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
633	`if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||`	Технически проверяет условие `!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n;` логически это защита от NULL-указателя и работы с неинициализированными данными.
634	`!report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {`	Технически объявляет/начинает функцию `report_model_valid`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
635	`sb_append(err, "Invalid arguments or metadata for JSON report.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
636	`return false;`	Технически возвращает `false` из функции `глобальной области файла`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
637	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
639	`StrBuf sb;`	Технически объявляет sb в функции глобальной области файла. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
640	`if (!sb_init(&sb, 8192)) { sb_append(err, "OOM.\n"); return false; }`	Технически проверяет условие !sb_init(&sb, 8192)) { sb_append(err, "OOM.\n"; логически это защита от NULL-указателя и работы с неинициализированными данными.
641	`StrBuf comment;`	Технически объявляет comment в функции глобальной области файла. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
642	`if (!sb_init(&comment, 512)) { sb_free(&sb); sb_append(err, "OOM.\n"); return false; }`	Технически проверяет условие !sb_init(&comment, 512)) { sb_free(&sb); sb_append(err, "OOM.\n"; логически это защита от NULL-указателя и работы с неинициализированными данными.
643	`StrBuf anom;`	Технически объявляет anom в функции глобальной области файла. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
644	`if (!sb_init(&anom, 128)) { sb_free(&comment); sb_free(&sb); sb_append(err, "OOM.\n"); return false; }`	Технически проверяет условие !sb_init(&anom, 128)) { sb_free(&comment); sb_free(&sb); sb_append(err, "OOM.\n"; логически это защита от NULL-указателя и работы с неинициализированными данными.
646	`sb_append(&sb, "{");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
647	`json_string_field(&sb, "input", input_path, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
648	`json_key(&sb, "report_metadata");`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
649	`sb_append(&sb, "{");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
650	`json_string_field(&sb, "schema_version", metadata->report_schema_version, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
651	`json_string_field(&sb, "tool_version", metadata->tool_version, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
652	`json_string_field(&sb, "generated_at_utc", metadata->generated_at_utc, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
653	`json_string_field(&sb, "input_sha256", metadata->input_sha256, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
654	`json_key(&sb, "input_size"); sb_appendf(&sb, "%llu,", (unsigned long long)metadata->input_size);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
655	`json_string_field(&sb, "command_line", metadata->command_line, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
656	`json_string_field(&sb, "build_compiler", metadata->build_compiler, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
657	`json_string_field(&sb, "build_platform", metadata->build_platform, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
658	`json_string_field(&sb, "build_mode", metadata->build_mode, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
659	`json_bool_field(&sb, "strict_abi", metadata->strict_abi, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
660	`json_bool_field(&sb, "follow_chain", metadata->follow_chain, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
661	`json_key(&sb, "max_chain_depth");`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
662	`sb_appendf(&sb, "%u", (unsigned)metadata->max_chain_depth);`	Технически вызывает sb_appendf. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
663	`sb_append(&sb, "},");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
664	`json_key(&sb, "image_base"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)img->image_base);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
665	`json_key(&sb, "size_of_image"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_image);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
666	`json_key(&sb, "size_of_headers"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_headers);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
667	`json_key(&sb, "section_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->section_alignment);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
668	`json_key(&sb, "file_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->file_alignment);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
669	`json_key(&sb, "exception_directory");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
670	`sb_appendf(&sb, "{\"rva\":\"0x%08x\",\"size\":\"0x%x\"}",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
671	`(unsigned)img->exception_rva, (unsigned)img->exception_size);`	Технически выполняет оператор `(unsigned)img->exception_rva, (unsigned)img->exception_size);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
672	`sb_append(&sb, ",");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
673	`json_key(&sb, "pe_validation");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
674	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
675	`json_key(&sb, "section_raw_oob"); sb_appendf(&sb, "%zu,", img->section_raw_oob);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
676	`json_key(&sb, "section_virtual_oob"); sb_appendf(&sb, "%zu,", img->section_virtual_oob);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
677	`json_key(&sb, "section_raw_overlaps"); sb_appendf(&sb, "%zu,", img->section_raw_overlaps);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
678	`json_key(&sb, "section_virtual_overlaps"); sb_appendf(&sb, "%zu,", img->section_virtual_overlaps);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
679	`json_key(&sb, "section_misaligned"); sb_appendf(&sb, "%zu,", img->section_misaligned);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
680	`json_bool_field(&sb, "exception_directory_oob", img->exception_directory_oob, false);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
681	`sb_append(&sb, "},");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
682	`json_key(&sb, "sections");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
683	`sb_append(&sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
684	`for (size_t i = 0; i < img->num_sections; i++) {`	Технически начинает цикл `size_t i = 0; i < img->num_sections; i++`. Логически он проходит по секциям PE-файла, чтобы проверить каждую запись, а не только первую.
685	`const PeSection* section = &img->sections[i];`	Технически объявляет `section` в функции `глобальной области файла` и сразу задает начальное значение `&img->sections[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
686	`if (i) sb_append(&sb, ",");`	Технически проверяет условие `i) sb_append(&sb, ",";` если оно истинно, выполняется защитная или аналитическая ветка ниже.
687	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
688	`json_string_field(&sb, "name", section->name, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
689	`json_key(&sb, "virtual_address"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->virtual_address);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
690	`json_key(&sb, "virtual_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->virtual_size);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
691	`json_key(&sb, "raw_offset"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_ptr);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
692	`json_key(&sb, "raw_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_size);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
693	`json_key(&sb, "characteristics"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->characteristics);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
694	`json_bool_field(&sb, "readable", (section->characteristics & IMAGE_SCN_MEM_READ) != 0, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
695	`json_bool_field(&sb, "writable", (section->characteristics & IMAGE_SCN_MEM_WRITE) != 0, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
696	`json_bool_field(&sb, "executable", (section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0, false);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
697	`sb_append(&sb, "}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
698	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
699	`sb_append(&sb, "],");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
701	`if (hr) {`	Технически проверяет условие `hr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
702	`json_key(&sb, "heuristic_analysis");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
703	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
704	`json_key(&sb, "score"); sb_appendf(&sb, "%d,", hr->score);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
705	`json_key(&sb, "suspicion"); sb_appendf(&sb, "%.6f,", hr->suspicion);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
706	`json_key(&sb, "confidence"); sb_appendf(&sb, "%.6f,", hr->confidence);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
707	`json_bool_field(&sb, "requires_manual_review", hr->requires_manual_review, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
708	`json_string_field(&sb, "verdict", hr->verdict, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
709	`json_string_field(&sb, "evidence_level", hr->evidence_level, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
710	`json_key(&sb, "weak_finding_count"); sb_appendf(&sb, "%zu,", hr->weak_finding_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
711	`json_key(&sb, "medium_finding_count"); sb_appendf(&sb, "%zu,", hr->medium_finding_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
712	`json_key(&sb, "strong_finding_count"); sb_appendf(&sb, "%zu,", hr->strong_finding_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
713	`json_key(&sb, "total"); sb_appendf(&sb, "%zu,", hr->total);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
714	`json_key(&sb, "pe_section_raw_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_oob);`	Технически строка использует код finding `pe_section_raw_oob`. Логически этот finding означает: секция на диске выходит за размер файла; это ломает базовый контракт PE-загрузчика.
715	`json_key(&sb, "pe_section_virtual_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_oob);`	Технически строка использует код finding `pe_section_virtual_oob`. Логически этот finding означает: секция в памяти выходит за SizeOfImage; это ошибка виртуальной координатной системы образа.

Стр.	Код	Нормальное объяснение
716	`json_key(&sb, "pe_section_raw_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_overlaps);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
717	`json_key(&sb, "pe_section_virtual_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_overlaps);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
718	`json_key(&sb, "pe_section_misaligned"); sb_appendf(&sb, "%zu,", hr->pe_section_misaligned);`	Технически строка использует код finding `pe_section_misaligned`. Логически этот finding означает: section table нарушает выравнивание или порядок, поэтому адресация становится ненадежной.
719	`json_bool_field(&sb, "pe_exception_directory_oob", hr->pe_exception_directory_oob, true);`	Технически строка использует код finding `pe_exception_directory_oob`. Логически этот finding означает: каталог исключений невозможно безопасно прочитать как таблицу RUNTIME_FUNCTION.
720	`json_key(&sb, "rf_invalid_ranges"); sb_appendf(&sb, "%zu,", hr->rf_invalid_ranges);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
721	`json_key(&sb, "rf_oob"); sb_appendf(&sb, "%zu,", hr->rf_oob);`	Технически строка использует код finding `rf_oob`. Логически этот finding означает: Begin/End функции выходят за SizeOfImage.
722	`json_key(&sb, "rf_unwind_oob"); sb_appendf(&sb, "%zu,", hr->rf_unwind_oob);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
723	`json_key(&sb, "rf_unsorted"); sb_appendf(&sb, "%zu,", hr->rf_unsorted);`	Технически строка использует код finding `rf_unsorted`. Логически этот finding означает: таблица функций не отсортирована по BeginAddress, хотя Windows ожидает упорядоченный поиск.
724	`json_key(&sb, "rf_overlaps"); sb_appendf(&sb, "%zu,", hr->rf_overlaps);`	Технически строка использует код finding `rf_overlaps`. Логически этот finding означает: диапазоны функций пересекаются после сортировки.

Стр.	Код	Нормальное объяснение
725	`json_key(&sb, "rf_duplicate_begins"); sb_appendf(&sb, "%zu,", hr->rf_duplicate_begins);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
726	`json_key(&sb, "unwind_parse_fail"); sb_appendf(&sb, "%zu,", hr->unwind_parse_fail);`	Технически строка использует код finding `unwind_parse_fail`. Логически этот finding означает: UNWIND_INFO не удалось разобрать безопасно.
727	`json_key(&sb, "unwind_anomalies"); sb_appendf(&sb, "%zu,", hr->unwind_anomalies);`	Технически строка использует код finding `unwind_anomalies`. Логически этот finding означает: UNWIND_INFO разобран, но содержит странные opcode, version, alignment или chain-связи.
728	`json_key(&sb, "unwind_abi_warnings"); sb_appendf(&sb, "%zu,", hr->unwind_abi_warnings);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
729	`json_key(&sb, "unwind_unusual_section"); sb_appendf(&sb, "%zu,", hr->unwind_unusual_section);`	Технически строка использует код finding `unwind_unusual_section`. Логически этот finding означает: UNWIND_INFO лежит в секции с нетипичным именем/назначением.
730	`json_key(&sb, "unwind_exec_section"); sb_appendf(&sb, "%zu,", hr->unwind_exec_section);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
731	`json_key(&sb, "unwind_writable_section"); sb_appendf(&sb, "%zu,", hr->unwind_writable_section);`	Технически строка использует код finding `unwind_writable_section`. Логически этот finding означает: UNWIND_INFO лежит в writable-секции и может быть изменен во время выполнения.
732	`json_key(&sb, "handler_nonexec_section"); sb_appendf(&sb, "%zu,", hr->handler_nonexec_section);`	Технически строка использует код finding `handler_nonexec_section`. Логически этот finding означает: handler указывает в неисполняемую область.
733	`json_key(&sb, "handler_writable_section"); sb_appendf(&sb, "%zu,", hr->handler_writable_section);`	Технически строка использует код finding `handler_writable_section`. Логически этот finding означает: handler лежит в writable-секции.
734	`json_key(&sb, "unwind_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_reuse_ratio);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
735	`json_key(&sb, "unwind_unique_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_unique_ratio);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
736	`json_key(&sb, "handler_count"); sb_appendf(&sb, "%zu,", hr->handler_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
737	`json_key(&sb, "handler_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->handler_reuse_ratio);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
738	`json_key(&sb, "evidence_family_count"); sb_appendf(&sb, "%zu,", heuristics_evidence_family_count(hr->evidence_families_mask));`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
739	`json_key(&sb, "evidence_families"); json_heuristic_families(&sb, hr->evidence_families_mask); sb_append(&sb, ",");`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
740	`json_key(&sb, "findings");`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
741	`sb_append(&sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
742	`for (size_t i = 0; i < hr->finding_count; i++) {`	Технически начинает цикл `size_t i = 0; i < hr->finding_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
743	`if (i) sb_append(&sb, ",");`	Технически проверяет условие `i) sb_append(&sb, ",";` если оно истинно, выполняется защитная или аналитическая ветка ниже.
744	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
745	`json_string_field(&sb, "code", hr->findings[i].code, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_string_field` для выполнения части алгоритма.
746	`json_string_field(&sb, "family", hr->findings[i].family, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_string_field` для выполнения части алгоритма.
747	`json_string_field(&sb, "severity", hr->findings[i].severity, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_string_field` для выполнения части алгоритма.
748	`json_string_field(&sb, "message", hr->findings[i].message, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_string_field` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
749	`json_key(&sb, "affected_count"); sb_appendf(&sb, "%zu,", hr->findings[i].affected_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
750	`json_key(&sb, "representative_rva");`	Технически вызывает `json_key`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_key` для выполнения части алгоритма.
751	`if (hr->findings[i].has_location) {`	Технически проверяет условие `hr->findings[i].has_location`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
752	`sb_appendf(&sb, "\"0x%08x\",", (unsigned)hr->findings[i].representative_rva);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
753	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
754	`sb_append(&sb, "null,");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
755	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
756	`json_string_field(`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
757	`&sb,`	Технически выполняет битовую операцию `&sb,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
758	`"representative_section",`	Технически эта строка является частью C-конструкции `"representative_section",`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
759	`hr->findings[i].has_location`	Технически эта строка является частью C-конструкции `hr->findings[i].has_location`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
760	`? section_name_or_dash(img, hr->findings[i].representative_rva)`	Технически объявляет/начинает функцию `section_name_or_dash`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
761	`: "-",`	Технически эта строка является частью C-конструкции `: "-",`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
762	`false);`	Технически выполняет оператор `false);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
763	`sb_append(&sb, "}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
764	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
765	`sb_append(&sb, "],");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
766	`json_key(&sb, "weakening_rules_applied");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
767	`sb_append(&sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
768	`for (size_t i = 0; i < hr->mitigation_count; i++) {`	Технически начинает цикл `size_t i = 0; i < hr->mitigation_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
769	`if (i) sb_append(&sb, ",");`	Технически проверяет условие `i) sb_append(&sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
770	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
771	`json_string_field(&sb, "code", hr->mitigations[i].code, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
772	`json_string_field(&sb, "message", hr->mitigations[i].message, false);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
773	`sb_append(&sb, "}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
774	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
775	`sb_append(&sb, "],");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
776	`json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr->mitigation_count);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
777	`json_string_field(&sb, "details", hr->details.s ? hr->details.s : "", false);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
778	`sb_append(&sb, "},");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
779	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
781	`if (dr) {`	Технически проверяет условие `dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
782	`json_key(&sb, "dynamic_validation");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
783	`sb_append(&sb, "{");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
784	`json_bool_field(&sb, "enabled", dr->enabled, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
785	`json_bool_field(&sb, "ran", dr->ran, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
786	`json_bool_field(&sb, "ok", dr->ok, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
787	`json_bool_field(&sb, "timed_out", dr->timed_out, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
788	`json_bool_field(&sb, "target_exited", dr->target_exited, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
789	`json_key(&sb, "target_exit_code"); sb_appendf(&sb, "%u,", (unsigned)dr->target_exit_code);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
790	`json_key(&sb, "timeout_ms"); sb_appendf(&sb, "%u,", (unsigned)dr->timeout_ms);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
791	`json_key(&sb, "trace_steps"); sb_appendf(&sb, "%u,", (unsigned)dr->trace_steps);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
792	`json_bool_field(&sb, "instrumentation_complete", dr->instrumentation_complete, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
793	`json_key(&sb, "modules_observed_peak"); sb_appendf(&sb, "%u,", (unsigned)dr->modules_observed_peak);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
794	`json_key(&sb, "module_refresh_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->module_refresh_failures);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
795	`json_key(&sb, "module_list_truncations"); sb_appendf(&sb, "%u,", (unsigned)dr->module_list_truncations);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
796	`json_key(&sb, "unwind_handlers_discovered"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handlers_discovered);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
797	`json_key(&sb, "breakpoint_set_attempts"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_attempts);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
798	`json_key(&sb, "breakpoint_set_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_failures);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
799	`json_key(&sb, "breakpoint_capacity_drops"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_capacity_drops);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
800	`json_key(&sb, "pending_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->pending_rearm_failures);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
801	`json_key(&sb, "breakpoint_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_rearm_failures);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
802	`json_key(&sb, "exceptions_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->exceptions_seen);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
803	`json_key(&sb, "first_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->first_chance_seen);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
804	`json_key(&sb, "second_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->second_chance_seen);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
805	`json_key(&sb, "unwind_handler_bp_set"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_bp_set);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
806	`json_key(&sb, "unwind_handler_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_hits);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
807	`json_key(&sb, "veh_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_registrations);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
808	`json_key(&sb, "veh_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_hits);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
809	`json_key(&sb, "vch_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_registrations);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
810	`json_key(&sb, "vch_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_hits);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
811	`json_key(&sb, "uef_set_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_set_calls);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
812	`json_key(&sb, "uef_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_hits);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
813	`json_key(&sb, "rtl_add_function_table_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->rtl_add_function_table_calls);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
814	`json_key(&sb, "rtl_install_function_table_cb_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->rtl_install_function_table_cb_calls);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
815	`json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", dynamic_signal_count(dr));`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
816	`json_key(&sb, "dropped_events"); sb_appendf(&sb, "%u,", (unsigned)dr->dropped_events);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
817	`json_string_field(&sb, "details", dr->details.s ? dr->details.s : "", true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
818	`json_key(&sb, "events");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
819	`sb_append(&sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
820	`for (uint32_t i = 0; i < dr->event_count; i++) {`	Технически начинает цикл `uint32_t i = 0; i < dr->event_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
821	`const DynamicEvent* event = &dr->events[i];`	Технически присваивает `const DynamicEvent* event` значение `&dr->events[i]`. Логически это обновляет `const DynamicEvent* event`: хранит промежуточное значение, которое дальше используется в текущей функции.
822	`if (i) sb_append(&sb, ",");`	Технически проверяет условие `i) sb_append(&sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
823	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
824	`json_string_field(&sb, "type", event->type, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
825	`json_string_field(&sb, "module", event->module, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
826	`json_key(&sb, "thread_id"); sb_appendf(&sb, "%u,", (unsigned)event->thread_id);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
827	`json_key(&sb, "exception_code"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)event->exception_code);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
828	`json_bool_field(&sb, "first_chance", event->first_chance, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
829	`json_key(&sb, "address"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->address);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
830	`json_key(&sb, "arg0"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg0);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
831	`json_key(&sb, "arg1"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg1);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
832	`json_key(&sb, "arg2"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg2);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
833	`json_key(&sb, "arg3"); sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)event->arg3);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
834	`sb_append(&sb, "}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
835	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
836	`sb_append(&sb, "],");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
837	`json_key(&sb, "trace");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
838	`sb_append(&sb, "[");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
839	`for (uint32_t i = 0; i < dr->trace_count; i++) {`	Технически начинает цикл `uint32_t i = 0; i < dr->trace_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
840	`if (i) sb_append(&sb, ",");`	Технически проверяет условие `i) sb_append(&sb, ","`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
841	`sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)dr->trace[i]);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
842	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
843	`sb_append(&sb, "]");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
844	`sb_append(&sb, "},");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
845	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
847	`if (hr || dr) {`	Технически проверяет условие hr || dr; если оно истинно, выполняется защитная или аналитическая ветка ниже.
848	`AssessmentSummary summary = assessment_summary(hr, dr);`	Технически присваивает AssessmentSummary summary значение assessment_summary(hr, dr). Логически это обновляет AssessmentSummary summary: хранит промежуточное значение, которое дальше используется в текущей функции.
850	`json_key(&sb, "assessment");`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
851	`sb_append(&sb, "{");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
852	`json_string_field(&sb, "final_verdict", summary.final_verdict, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
853	`json_key(&sb, "final_confidence"); sb_appendf(&sb, "%.6f,", summary.final_confidence);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
854	`json_string_field(&sb, "static_evidence_level", summary.static_evidence_level, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
855	`json_string_field(&sb, "runtime_evidence_level", summary.runtime_evidence_level, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
856	`json_string_field(&sb, "case_strength", summary.strength, true);`	Технически вызывает json_string_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_string_field для выполнения части алгоритма.
857	`json_bool_field(&sb, "static_dynamic_correlation", summary.correlated, true);`	Технически вызывает json_bool_field. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_bool_field для выполнения части алгоритма.
858	`json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", summary.dynamic_signals);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
859	`json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr ? hr->mitigation_count : 0u);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
860	`json_key(&sb, "manual_validation_steps"); json_manual_validation_steps(&sb, hr, dr);`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
861	`sb_append(&sb, "},");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
862	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
864	`json_key(&sb, "functions");`	Технически вызывает json_key. Логически это действие нужно в функции глобальной области файла: вызывает функцию json_key для выполнения части алгоритма.
865	`sb_append(&sb, "[");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
866	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл size_t i = 0; i < n; i++. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
867	`if (i) sb_append(&sb, ",");`	Технически проверяет условие i) sb_append(&sb, ","; если оно истинно, выполняется защитная или аналитическая ветка ниже.
868	`if (us_ok[i]) build_unwind_comment(&us[i], &comment);`	Технически проверяет условие us_ok[i]) build_unwind_comment(&us[i], &comment; логически отбирает записи, где важна корректность UNWIND_INFO.
869	`else {`	Технически эта строка является частью C-конструкции else {. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
870	`comment.len = 0;`	Технически присваивает comment.len значение 0. Логически это обновляет len: хранит размер или количество, поэтому используется в проверках границ.
871	`if (comment.s) comment.s[0] = 0;`	Технически проверяет условие comment.s; если оно истинно, выполняется защитная или аналитическая ветка ниже.
872	`sb_append(&comment, "UNWIND parse failed");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
873	`if (us[i].parse_error[0]) sb_appendf(&comment, ": %s", us[i].parse_error);`	Технически проверяет условие us[i].parse_error[0]) sb_appendf(&comment, ": %s", us[i].parse_error; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
874	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
875	`anom.len = 0; if (anom.s) anom.s[0] = 0; anomalies_to_str(&anom, us[i].anomalies);`	Технически присваивает `anom.len` значение `0; if (anom.s) anom.s[0] = 0; anomalies_to_str(&anom, us[i].anomalies)`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
877	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
878	`json_key(&sb, "index"); sb_appendf(&sb, "%zu,", i);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
879	`json_key(&sb, "begin_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].begin_rva);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
880	`json_key(&sb, "end_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].end_rva);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
881	`json_key(&sb, "size"); sb_appendf(&sb, "%u,", (unsigned)runtime_function_size(&rf[i]));`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
882	`json_key(&sb, "begin_section"); json_string(&sb, section_name_or_dash(img, rf[i].begin_rva)); sb_append(&sb, ",");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
883	`json_key(&sb, "unwind_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].unwind_rva);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
884	`json_key(&sb, "unwind_section"); json_string(&sb, section_name_or_dash(img, rf[i].unwind_rva)); sb_append(&sb, ",");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
885	`json_bool_field(&sb, "unwind_ok", us_ok[i], true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
886	`json_string_field(&sb, "unwind_error", us[i].parse_error, true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
887	`json_key(&sb, "unwind");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
888	`sb_append(&sb, "{");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
889	`json_key(&sb, "version"); sb_appendf(&sb, "%u,", (unsigned)us[i].version);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
890	`json_key(&sb, "flags"); sb_appendf(&sb, "%u,", (unsigned)us[i].flags);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
891	`json_key(&sb, "prolog_size"); sb_appendf(&sb, "%u,", (unsigned)us[i].prolog_size);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
892	`json_key(&sb, "count_codes"); sb_appendf(&sb, "%u,", (unsigned)us[i].count_codes);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
893	`json_key(&sb, "fixed_stack_bytes"); sb_appendf(&sb, "%u,", (unsigned)us[i].fixed_stack_bytes);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
894	`json_key(&sb, "frame_reg"); json_string(&sb, (us[i].frame_reg ? unwind_reg_name(us[i].frame_reg) : "-")); sb_append(&sb, ",");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
895	`json_key(&sb, "frame_off"); sb_appendf(&sb, "%u,", (unsigned)(us[i].frame_off * 16u));`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
896	`json_bool_field(&sb, "has_handler", us[i].has_handler, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
897	`json_key(&sb, "handler_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)us[i].handler_rva);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
898	`json_key(&sb, "handler_section"); json_string(&sb, us[i].has_handler ? section_name_or_dash(img, us[i].handler_rva) : "-"); sb_append(&sb, ",");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
899	`json_bool_field(&sb, "has_chain", us[i].has_chain, true);`	Технически вызывает `json_bool_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_bool_field` для выполнения части алгоритма.
900	`json_key(&sb, "anomalies"); sb_appendf(&sb, "%u,", (unsigned)us[i].anomalies);`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
901	`json_string_field(&sb, "anomalies_text", anom.s ? anom.s : "0", true);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_string_field` для выполнения части алгоритма.
902	`json_key(&sb, "chain");`	Технически вызывает `json_key`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `json_key` для выполнения части алгоритма.
903	`if (us[i].has_chain) {`	Технически проверяет условие `us[i].has_chain`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
904	`sb_appendf(&sb, "{\"begin_rva\":\"0x%08x\",\"end_rva\":\"0x%08x\",\"unwind_rva\":\"0x%08x\"}",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
905	`(unsigned)us[i].chain.begin_rva,`	Технически эта строка является частью С-конструкции `(unsigned)us[i].chain.begin_rva,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
906	`(unsigned)us[i].chain.end_rva,`	Технически эта строка является частью С-конструкции `(unsigned)us[i].chain.end_rva,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
907	`(unsigned)us[i].chain.unwind_rva);`	Технически выполняет оператор `(unsigned)us[i].chain.unwind_rva);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
908	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
909	`sb_append(&sb, "null");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
910	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
911	`sb_append(&sb, "},");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
912	`json_string_field(&sb, "comment", comment.s ? comment.s : "", false);`	Технически вызывает `json_string_field`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `json_string_field` для выполнения части алгоритма.
913	`sb_append(&sb, "}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
914	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
915	`sb_append(&sb, "]}");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
917	`bool ok = sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom);`	Технически присваивает `bool ok` значение `sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom)`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
918	`if (ok) ok = file_write_atomic(path, sb.s, sb.len);`	Технически проверяет условие `ok) ok = file_write_atomic(path, sb.s, sb.len;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
919	`if (!ok) {`	Технически проверяет условие `!ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.

Стр.	Код	Нормальное объяснение
920	`sb_appendf(err, "%s: %s\n",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
921	`sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom)`	Технически объявляет/начинает функцию `sb_ok`; она доступная другим модулям через заголовок или линковку. Логически функция проверяет, что StrBuf не перешел в состояние ошибки.
922	`? "Failed to write JSON"`	Технически эта строка является частью С-конструкции `? "Failed to write JSON"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
923	`: "Out of memory while building JSON report",`	Технически эта строка является частью С-конструкции `: "Out of memory while building JSON report",`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
924	`path);`	Технически выполняет оператор `path);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
925	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
927	`sb_free(&anom);`	Технически вызывает `sb_free`. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
928	`sb_free(&comment);`	Технически вызывает `sb_free`. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
929	`sb_free(&sb);`	Технически вызывает `sb_free`. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
930	`return ok;`	Технически возвращает выражение `ok`. Логически именно это значение становится результатом функции глобальной области файла для вызывающего кода.
931	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
933	`static void html_escape(StrBuf* sb, const char* text) {`	Технически объявляет/начинает функцию `html_escape`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
934	`if (!text) return;`	Технически проверяет условие `!text`; логически это защита от NULL-указателя и работы с неинициализированными данными.
935	`size_t length = strlen(text);`	Технически присваивает `size_t length` значение `strlen(text)`. Логически это обновляет `size_t length`: хранит размер или количество, поэтому используется в проверках границ.
936	`for (size_t i = 0; i < length; i++) {`	Технически начинает цикл `size_t i = 0; i < length; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
937	`unsigned char c = (unsigned char)text[i];`	Технически присваивает `unsigned char c` значение `(unsigned char)text[i]`. Логически это обновляет `unsigned char c`: хранит промежуточное значение, которое дальше используется в текущей функции.
938	`switch (c) {`	Технически выбирает ветку по значению `c`. Логически это удобно для opcode, флагов, verdict или типа breakpoint.
939	`case '&': sb_append(sb, "&"); break;`	Технически объявляет ветку `switch` для значения `'&': sb_append(sb, "&"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
940	`case '<': sb_append(sb, "<"); break;`	Технически объявляет ветку `switch` для значения `'<': sb_append(sb, "<"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
941	`case '>': sb_append(sb, ">"); break;`	Технически объявляет ветку `switch` для значения `'>': sb_append(sb, ">"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
942	`case '"': sb_append(sb, """); break;`	Технически объявляет ветку `switch` для значения `'"': sb_append(sb, """); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
943	`case '\'': sb_append(sb, "'"); break;`	Технически объявляет ветку `switch` для значения `'\'': sb_append(sb, "'"); break;`. Логически так код отдельно обрабатывает конкретный opcode, флаг или режим.
944	`default:`	Технически объявляет ветку `switch` по умолчанию. Логически сюда попадают неизвестные значения, которые часто становятся anomaly или fallback.
945	`if (c < 0x80) {`	Технически проверяет условие `c < 0x80`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
946	`sb_appendn(sb, &text[i], 1);`	Технически вызывает `sb_appendn`. Логически это действие нужно в функции `html_escape`: добавляет ровно `n` байтов текста в StrBuf.
947	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
948	`size_t sequence = valid_utf8_sequence_length(`	Технически присваивает `size_t sequence` значение `valid_utf8_sequence_length(`. Логически это обновляет `size_t sequence`: хранит размер или количество, поэтому используется в проверках границ.
949	`(const unsigned char*)&text[i], length - i);`	Технически выполняет битовую операцию `(const unsigned char*)&text[i], length - i);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
950	`if (sequence) {`	Технически проверяет условие `sequence`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
951	`sb_appendn(sb, &text[i], sequence);`	Технически вызывает `sb_appendn`. Логически это действие нужно в функции `html_escape`: добавляет ровно `n` байтов текста в StrBuf.
952	`i += sequence - 1;`	Технически изменяет `i` оператором `+=` на основе `sequence - 1`. Логически это обновляет счетчик, размер, score или накопленное состояние без отдельной временной переменной.
953	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
954	`sb_appendf(sb, "&#x%02x;", (unsigned)c);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_escape`: добавляет форматированный текст в динамический буфер.

Стр.	Код	Нормальное объяснение
955	`}`	Технически завершает область видимости внутри `html_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
956	`}`	Технически завершает область видимости внутри `html_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
957	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
958	`}`	Технически завершает область видимости внутри `html_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
959	`}`	Технически завершает область видимости внутри `html_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
960	`}`	Технически завершает область видимости внутри `html_escape`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
962	`static void html_text_cell(StrBuf* sb, const char* text) {`	Технически объявляет/начинает функцию `html_text_cell`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
963	`sb_append(sb, "<td>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_text_cell`: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
964	`    html_escape(sb, text ? text : "");`	Технически вызывает `html_escape`. Логически это действие нужно в функции `html_text_cell`: вызывает функцию `html_escape` для выполнения части алгоритма.
965	`    sb_append(sb, "</td>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_text_cell`: добавляет текст в динамический буфер.
966	`}`	Технически завершает область видимости внутри `html_text_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
968	`static void html_hex32_cell(StrBuf* sb, uint32_t value) {`	Технически объявляет/начинает функцию `html_hex32_cell`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
969	`    sb_appendf(sb, "<td><code>0x%08x</code></td>", (unsigned)value);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_hex32_cell`: добавляет форматированный текст в динамический буфер.
970	`}`	Технически завершает область видимости внутри `html_hex32_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
972	`static void html_hex64_cell(StrBuf* sb, uint64_t value) {`	Технически объявляет/начинает функцию `html_hex64_cell`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
973	`    sb_appendf(sb, "<td><code>0x%llx</code></td>", (unsigned long long)value);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_hex64_cell`: добавляет форматированный текст в динамический буфер.
974	`}`	Технически завершает область видимости внутри `html_hex64_cell`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
976	`static const char* html_score_class(const HeuristicReport* hr) {`	Технически объявляет/начинает функцию `html_score_class`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
977	`    if (!hr) return "neutral";`	Технически проверяет условие `!hr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
978	`    if (hr->score >= 70) return "danger";`	Технически проверяет условие `hr->score >= 70`; логически выбирает уровень оценки или уверенности.
979	`    if (hr->score >= 40) return "warn";`	Технически проверяет условие `hr->score >= 40`; логически выбирает уровень оценки или уверенности.
980	`    return "ok";`	Технически возвращает выражение `"ok"`. Логически именно это значение становится результатом функции `html_score_class` для вызывающего кода.
981	`}`	Технически завершает область видимости внутри `html_score_class`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
983	`static const char* ru_yes_no(bool value) {`	Технически объявляет/начинает функцию `ru_yes_no`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
984	`    return value ? "да" : "нет";`	Технически возвращает выражение `value ? "да" : "нет"`. Логически именно это значение становится результатом функции `ru_yes_no` для вызывающего кода.
985	`}`	Технически завершает область видимости внутри `ru_yes_no`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
987	`static const char* ru_status_text(const DynamicReport* dr) {`	Технически объявляет/начинает функцию `ru_status_text`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
988	`    if (!dr) return "не запрашивалась";`	Технически проверяет условие `!dr`; логически это защита от NULL-указателя и работы с неинициализированными данными.
989	`    if (dr->timed_out) return "тайм-аут";`	Технически проверяет условие `dr->timed_out`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
990	`    if (!dr->ran) return "не запускалась";`	Технически проверяет условие `!dr->ran`; логически это защита от NULL-указателя и работы с неинициализированными данными.
991	`    return dr->ok ? "завершена" : "ошибка";`	Технически возвращает выражение `dr->ok ? "завершена" : "ошибка"`. Логически именно это значение становится результатом функции `ru_status_text` для вызывающего кода.
992	`}`	Технически завершает область видимости внутри `ru_status_text`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
994	`static const char* ru_level_text(const char* value) {`	Технически объявляет/начинает функцию `ru_level_text`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
995	`    if (!value) return "-";`	Технически проверяет условие `!value`; логически это защита от NULL-указателя и работы с неинициализированными данными.
996	`    if (strcmp(value, "strong") == 0) return "сильный";`	Технически проверяет условие `strcmp(value, "strong") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
997	`    if (strcmp(value, "medium") == 0) return "средний";`	Технически проверяет условие `strcmp(value, "medium") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
998	`    if (strcmp(value, "weak") == 0) return "слабый";`	Технически проверяет условие `strcmp(value, "weak") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
999	`    if (strcmp(value, "none") == 0) return "нет";`	Технически проверяет условие `strcmp(value, "none") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
1000	`return value;`	Технически возвращает выражение `value`. Логически именно это значение становится результатом функции `ru_level_text` для вызывающего кода.
1001	`}`	Технически завершает область видимости внутри `ru_level_text`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1003	`static const char* ru_severity_text(const char* value) {`	Технически объявляет/начинает функцию `ru_severity_text`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
1004	`if (!value) return "-";`	Технически проверяет условие `!value`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1005	`if (strcmp(value, "critical") == 0) return "критическая";`	Технически проверяет условие `strcmp(value, "critical") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1006	`if (strcmp(value, "high") == 0) return "высокая";`	Технически проверяет условие `strcmp(value, "high") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1007	`if (strcmp(value, "medium") == 0) return "средняя";`	Технически проверяет условие `strcmp(value, "medium") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1008	`if (strcmp(value, "low") == 0) return "низкая";`	Технически проверяет условие `strcmp(value, "low") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1009	`if (strcmp(value, "info") == 0) return "информационная";`	Технически проверяет условие `strcmp(value, "info") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1010	`return value;`	Технически возвращает выражение `value`. Логически именно это значение становится результатом функции `ru_severity_text` для вызывающего кода.
1011	`}`	Технически завершает область видимости внутри `ru_severity_text`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1013	`static const char* ru_verdict_text(const char* value) {`	Технически объявляет/начинает функцию `ru_verdict_text`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
1014	`if (!value) return "-";`	Технически проверяет условие `!value`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1015	`if (strcmp(value, "high_suspicion") == 0) return "высокая подозрительность";`	Технически проверяет условие `strcmp(value, "high_suspicion") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1016	`if (strcmp(value, "moderate_suspicion") == 0) return " умеренная подозрительность";`	Технически проверяет условие `strcmp(value, "moderate_suspicion") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1017	`if (strcmp(value, "low_suspicion") == 0) return "низкая подозрительность";`	Технически проверяет условие `strcmp(value, "low_suspicion") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1018	`if (strcmp(value, "no_strong_static_anomalies") == 0) return " сильные статические аномалии не обнаружены";`	Технически проверяет условие `strcmp(value, "no_strong_static_anomalies") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1019	`if (strcmp(value, "high_suspicion_correlated") == 0) return " высокая подозрительность с динамическим подтверждением";`	Технически проверяет условие `strcmp(value, "high_suspicion_correlated") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1020	`if (strcmp(value, "high_suspicion_static") == 0) return " высокая статическая подозрительность";`	Технически проверяет условие `strcmp(value, "high_suspicion_static") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1021	`if (strcmp(value, "moderate_suspicion_correlated") == 0) return "умеренная подозрительность с динамическим подтверждением";`	Технически проверяет условие `strcmp(value, "moderate_suspicion_correlated") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1022	`if (strcmp(value, "moderate_suspicion_static") == 0) return " умеренная статическая подозрительность";`	Технически проверяет условие `strcmp(value, "moderate_suspicion_static") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1023	`if (strcmp(value, "low_suspicion_correlated") == 0) return " низкая подозрительность с динамическим подтверждением";`	Технически проверяет условие `strcmp(value, "low_suspicion_correlated") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1024	`if (strcmp(value, "low_suspicion_static") == 0) return " низкая статическая подозрительность";`	Технически проверяет условие `strcmp(value, "low_suspicion_static") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1025	`if (strcmp(value, "runtime_signals_only") == 0) return " обнаружены только динамические сигналы";`	Технически проверяет условие `strcmp(value, "runtime_signals_only") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1026	`if (strcmp(value, "no_strong_correlated_evidence") == 0) return "сильная корреляция не обнаружена";`	Технически проверяет условие `strcmp(value, "no_strong_correlated_evidence") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1027	`return value;`	Технически возвращает выражение `value`. Логически именно это значение становится результатом функции `ru_verdict_text` для вызывающего кода.
1028	`}`	Технически завершает область видимости внутри `ru_verdict_text`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1030	`static const char* ru_case_strength_text(const char* value) {`	Технически объявляет/начинает функцию `ru_case_strength_text`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.

Стр.	Код	Нормальное объяснение
1031	`if (!value) return "-";`	Технически проверяет условие `!value`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1032	`if (strcmp(value, "strong_correlated_case") == 0) return " сильная, с корреляцией";`	Технически проверяет условие `strcmp(value, "strong_correlated_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1033	`if (strcmp(value, "medium_correlated_case") == 0) return " средняя, с корреляцией";`	Технически проверяет условие `strcmp(value, "medium_correlated_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
1034	`if (strcmp(value, "strong_unconfirmed_case") == 0) return "сильная, без динамического подтверждения";`	Технически проверяет условие `strcmp(value, "strong_unconfirmed_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1035	`if (strcmp(value, "medium_case") == 0) return "средняя";`	Технически проверяет условие `strcmp(value, "medium_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1036	`if (strcmp(value, "weak_case") == 0) return "слабая";`	Технически проверяет условие `strcmp(value, "weak_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1037	`if (strcmp(value, "no_case") == 0) return "значимых признаков нет";`	Технически проверяет условие `strcmp(value, "no_case") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1038	`return value;`	Технически возвращает выражение `value`. Логически именно это значение становится результатом функции `ru_case_strength_text` для вызывающего кода.
1039	`}`	Технически завершает область видимости внутри `ru_case_strength_text`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1041	`static void html_append_manual_step_ru(StrBuf* sb, const char* step) {`	Технически объявляет/начинает функцию `html_append_manual_step_ru`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
1042	`if (!step) return;`	Технически проверяет условие `!step`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1043	`if (strstr(step, "Validate .pdata range integrity")) {`	Технически проверяет условие `strstr(step, "Validate .pdata range integrity")`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1044	`sb_append(sb, "Проверьте целостность диапазона .pdata в PE-viewer: вручную отсортируйте записи RUNTIME_FUNCTION и убедитесь, что Begin/End/Unwind RVA находятся внутри SizeOfImage.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1045	`} else if (strstr(step, "Open the suspicious UNWIND_INFO blob")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1046	`sb_append(sb, "Откройте подозрительный блок UNWIND_INFO в hex-viewer, проверьте Version/Flags/CountOfCodes и tail-layout, затем сравните decoded prolog actions с prolog функции в Ghidra.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1047	`} else if (strstr(step, "Inspect the section hosting the unwind metadata")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1048	`sb_append(sb, "Проверьте секцию, в которой размещены unwind-метаданные: зафиксируйте RVA, raw offset и права доступа, затем решите, похоже ли размещение на обычный compiler layout или на ручное перенаправление.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1049	`} else if (strstr(step, "Disassemble the reported handler target")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1050	`sb_append(sb, "Дизассемблируйте указанный handler target, подтвердите execute permission и проверьте, являются ли landing bytes настоящим handler, thunk или областью не-кода.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1051	`} else if (strstr(step, "Check whether writable unwind metadata")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1052	`sb_append(sb, "Проверьте, оправдано ли наличие writable unwind metadata или writable handler targets сценарием JIT/runtime; если нет, считайте размещение подозрительным и сопоставьте его с memory protections во время запуска.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1053	`} else if (strstr(step, "Open the dominant handler target")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1054	`sb_append(sb, "Откройте dominant handler target в Ghidra и сравните несколько функций, которые на него ссылаются, чтобы отличить обычный runtime helper от подозрительного central dispatcher.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1055	`} else if (strstr(step, "Review weakening rules")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1056	`sb_append(sb, "Перед окончательным выводом проверьте правила снижения уверенности: текущий набор признаков еще может иметь доброкачественные объяснения, которые нужно исключить вручную.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1057	`} else if (strstr(step, "Capture the arguments of RtlAddFunctionTable")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1058	`sb_append(sb, "Зафиксируйте аргументы RtlAddFunctionTable / RtlInstallFunctionTableCallback и проверьте содержимое runtime table вместе с module/section mapping.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1059	`} else if (strstr(step, "List each VEH/VCH/UEF registration")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1060	`sb_append(sb, "Перечислите каждую регистрацию VEH/VCH/UEF с target address, затем выполните single-step от зарегистрированной routine, чтобы отличить benign runtime hook от exception interceptor.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1061	`} else if (strstr(step, "Single-step from the handler hit")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1062	`sb_append(sb, "Выполните single-step от handler hit и отобразите переходы между module/section сразу после exception dispatch point.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1063	`} else if (strstr(step, "Verify whether the hit unwind handler")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1064	`sb_append(sb, "Проверьте, соответствует ли достигнутый unwind handler известному language/runtime handler или dispatcher/interceptor path, на который ссылается много защищенных функций.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1065	`} else if (strstr(step, "Run the sample again with --dynamic")) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1066	`sb_append(sb, "Запустите sample повторно с --dynamic на representative exception path, чтобы получить runtime corroboration.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_manual_step_ru`: добавляет текст в динамический буфер.
1067	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1068	`html_escape(sb, step);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `html_append_manual_step_ru`: вызывает функцию `html_escape` для выполнения части алгоритма.
1069	`}`	Технически завершает область видимости внутри `html_append_manual_step_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1070	`}`	Технически завершает область видимости внутри `html_append_manual_step_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1072	`static void html_append_finding_message_ru(StrBuf* sb, const HeuristicFinding* finding) {`	Технически объявляет/начинает функцию `html_append_finding_message_ru`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
1073	`if (!finding || !finding->code[0]) return;`	Технически проверяет условие `!finding || !finding->code[0]`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1074	`const char* code = finding->code;`	Технически объявляет `code` в функции `html_append_finding_message_ru` и сразу задает начальное значение `finding->code`. Логически это место для данных: машинный код finding или mitigation, стабильный для тестов и отчетов.
1075	`size_t n = finding->affected_count;`	Технически объявляет `n` в функции `html_append_finding_message_ru` и сразу задает начальное значение `finding->affected_count`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1076	`if (strcmp(code, "pe_section_raw_oob") == 0) {`	Технически проверяет условие `strcmp(code, "pe_section_raw_oob") == 0`; логически это защита от чтения за пределами файла на диске.
1077	`sb_append(sb, "Raw-диапазон секции PE выходит за границы файла.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1078	`} else if (strcmp(code, "pe_section_virtual_oob") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1079	`sb_append(sb, "Virtual-диапазон секции PE выходит за пределы SizeOfImage.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1080	`} else if (strcmp(code, "pe_section_raw_overlap") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1081	`sb_append(sb, "Raw-диапазоны секций PE пересекаются.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1082	`} else if (strcmp(code, "pe_section_virtual_overlap") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1083	`sb_append(sb, "Virtual-диапазоны секций PE пересекаются.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1084	`} else if (strcmp(code, "pe_section_misaligned") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1085	`sb_append(sb, "Обнаружена секция PE с нарушением выравнивания.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1086	`} else if (strcmp(code, "pe_exception_directory_oob") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1087	`sb_append(sb, "Exception directory поврежден, неполон, недоступен для чтения или выходит за допустимые границы.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1088	`} else if (strcmp(code, "no_runtime_functions") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1089	`sb_append(sb, "Записи RUNTIME_FUNCTION не найдены; это может быть допустимо для leaf-only image, поэтому сложность кода нужно подтвердить вручную.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1090	`} else if (strcmp(code, "invalid_ranges") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1091	`sb_appendf(sb, "Обнаружены записи RUNTIME_FUNCTION с некорректным диапазоном Begin/End: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1092	`} else if (strcmp(code, "rf_oob") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1093	`sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION выходят за пределы SizeOfImage: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1094	`} else if (strcmp(code, "unwind_rva_oob") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.

Стр.	Код	Нормальное объяснение
1095	`sb_appendf(sb, "UnwindInfoRVA отсутствует или выходит за допустимые границы: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1096	`} else if (strcmp(code, "rf_unsorted") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1097	`sb_appendf(sb, "Таблица RUNTIME_FUNCTION не отсортирована по BeginAddress: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1098	`} else if (strcmp(code, "rf_overlaps") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.

Стр.	Код	Нормальное объяснение
1099	`sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION пересекаются: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1100	`} else if (strcmp(code, "duplicate_begin") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1101	`sb_appendf(sb, "Несколько записей RUNTIME_FUNCTION используют один и тот же BeginAddress: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1102	`} else if (strcmp(code, "unwind_parse_fail") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1103	`sb_appendf(sb, "Ошибки разбора UNWIND_INFO: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1104	`} else if (strcmp(code, "unwind_anomalies") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1105	`sb_appendf(sb, "Аномалии UNWIND_INFO (version/opcode/chain/ABI и др.): %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1106	`} else if (strcmp(code, "unwind_reuse_strong") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1107	`sb_appendf(sb, "Сильное повторное использование UNWIND_INFO: до %zu записей ссылаются на один UnwindInfoRVA.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1108	`} else if (strcmp(code, "handler_dominance") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1109	`sb_appendf(sb, "Один handler target доминирует: до %zu записей ссылаются на один HandlerRVA.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1110	`} else if (strcmp(code, "unwind_reuse_moderate") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1111	`sb_appendf(sb, "Умеренное повторное использование UNWIND_INFO подтверждается другим статическим признаком: до %zu записей ссылаются на один UnwindInfoRVA.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1112	`} else if (strcmp(code, "unwind_in_exec_section") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1113	`sb_appendf(sb, "UNWIND_INFO расположен в executable-секции: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1114	`} else if (strcmp(code, "unwind_unusual_section") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1115	`sb_appendf(sb, "UNWIND_INFO расположен в необычной секции, отличной от .xdata/.rdata/.pdata: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1116	`} else if (strcmp(code, "unwind_writable_section") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1117	`sb_appendf(sb, "UNWIND_INFO расположен в writable-секции: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1118	`} else if (strcmp(code, "handler_nonexec_section") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1119	`sb_appendf(sb, "Exception handler RVA указывает в non-executable или неизвестную секцию: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.
1120	`} else if (strcmp(code, "handler_writable_section") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1121	`sb_appendf(sb, "Exception handler RVA указывает в writable-секцию: %zu.", n);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет форматированный текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1122	`} else if (strcmp(code, "placement_exec_nonexec_handler_combo") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1123	`sb_append(sb, "Unwind metadata в executable-секции совпадает с handler targets в non-executable/unknown секции; такую комбинацию сложнее объяснить доброкачественно.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1124	`} else if (strcmp(code, "placement_integrity_combo") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1125	`sb_append(sb, "Подозрительное размещение metadata подтверждается проблемами целостности UNWIND_INFO / UnwindInfoRVA; вместе эти признаки сильнее, чем по отдельности.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1126	`} else if (strcmp(code, "reuse_dominance_combo") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1127	`sb_append(sb, "Активное повторное использование UNWIND_INFO совпадает с dominant handler target, что похоже на централизованный dispatcher-style metadata reuse.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1128	`} else if (strcmp(code, "writable_metadata_handler_combo") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1129	`sb_append(sb, "Writable unwind metadata совпадает с writable handler targets; такая комбинация нетипична для обычных compiler-produced images и требует внимательной проверки.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_finding_message_ru`: добавляет текст в динамический буфер.
1130	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1131	`html_escape(sb, finding->message);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `html_append_finding_message_ru`: вызывает функцию `html_escape` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1132	`}`	Технически завершает область видимости внутри `html_append_finding_message_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1133	`}`	Технически завершает область видимости внутри `html_append_finding_message_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1135	`static void html_append_mitigation_message_ru(StrBuf* sb, const char* code, const char* message) {`	Технически объявляет/начинает функцию `html_append_mitigation_message_ru`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
1136	`if (!code) {`	Технически проверяет условие `!code`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1137	`html_escape(sb, message);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: вызывает функцию `html_escape` для выполнения части алгоритма.
1138	`} else if (strcmp(code, "single_signal_only") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1139	`sb_append(sb, "Вклад дал только один не-сильный признак; как самостоятельное доказательство его нужно считать слабым.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: добавляет текст в динамический буфер.
1140	`} else if (strcmp(code, "small_sample_size") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1141	`sb_append(sb, "Очень маленькая выборка .pdata может завышать доли; уверенность снижена до появления большего числа функций.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: добавляет текст в динамический буфер.
1142	`} else if (strcmp(code, "placement_only_signal") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1143	`sb_append(sb, "Аномалии только по размещению могут встречаться в необычных, но доброкачественных layout; нужна ручная проверка.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: добавляет текст в динамический буфер.
1144	`} else if (strcmp(code, "single_family_only") == 0) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1145	`sb_append(sb, "Признаки относятся только к одному семейству; перед сильным выводом желательно cross-layer corroboration.");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: добавляет текст в динамический буфер.
1146	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1147	`html_escape(sb, message);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `html_append_mitigation_message_ru`: вызывает функцию `html_escape` для выполнения части алгоритма.
1148	`}`	Технически завершает область видимости внутри `html_append_mitigation_message_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1149	`}`	Технически завершает область видимости внутри `html_append_mitigation_message_ru`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1151	`bool report_write_html(const char* path,`	Технически вызывает `report_write_html`. Логически это действие нужно в функции `глобальной области файла`: записывает HTML-отчет для ручного просмотра.

Стр.	Код	Нормальное объяснение
1152	`const char* input_path,`	Технически эта строка является частью C-конструкции `const char* input_path,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1153	`const PeImage* img,`	Технически эта строка является частью C-конструкции `const PeImage* img,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1154	`const RuntimeFunction* rf,`	Технически эта строка является частью C-конструкции `const RuntimeFunction* rf,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1155	`const UnwindSummary* us,`	Технически эта строка является частью C-конструкции `const UnwindSummary* us,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1156	`const bool* us_ok,`	Технически эта строка является частью C-конструкции `const bool* us_ok,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1157	`size_t n,`	Технически эта строка является частью C-конструкции `size_t n,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1158	`const HeuristicReport* hr,`	Технически эта строка является частью C-конструкции `const HeuristicReport* hr,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1159	`const DynamicReport* dr,`	Технически эта строка является частью C-конструкции `const DynamicReport* dr,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1160	`const ReportMetadata* metadata,`	Технически эта строка является частью C-конструкции `const ReportMetadata* metadata,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1161	`StrBuf* err) {`	Технически эта строка является частью C-конструкции `StrBuf* err) {`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1162	`if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||`	Технически проверяет условие `!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n;` логически это защита от NULL-указателя и работы с неинициализированными данными.
1163	`!report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {`	Технически объявляет/начинает функцию `report_model_valid`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
1164	`sb_append(err, "Invalid arguments or metadata for HTML report.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1165	`return false;`	Технически возвращает `false` из функции глобальной области файла. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
1166	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
1168	`StrBuf sb = {0};`	Технически объявляет `sb` в функции глобальной области файла и сразу задает начальное значение `{0}`. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
1169	`StrBuf anomalies = {0};`	Технически объявляет `anomalies` в функции глобальной области файла и сразу задает начальное значение `{0}`. Логически это место для данных: битовая маска обнаруженных проблем декодирования unwind.
1170	`if (!sb_init(&sb, 32768) ||`	Технически проверяет условие `!sb_init(&sb, 32768`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1171	`!sb_init(&anomalies, 128)) {`	Технически объявляет/начинает функцию `sb_init`; она доступная другим модулям через заголовок или линковку. Логически функция инициализирует динамический строковый буфер.
1172	`sb_free(&anomalies);`	Технически вызывает `sb_free`. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
1173	`sb_free(&sb);`	Технически вызывает `sb_free`. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
1174	`sb_append(err, "OOM.\n");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1175	`return false;`	Технически возвращает `false` из функции глобальной области файла. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
1176	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1178	`const char* tool_version = metadata->tool_version;`	Технически объявляет `tool_version` в функции глобальной области файла и сразу задает начальное значение `metadata->tool_version`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1179	`const char* generated_at = metadata->generated_at_utc;`	Технически объявляет `generated_at` в функции `глобальной области файла` и сразу задает начальное значение `metadata->generated_at_utc`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1180	`const char* schema_version = metadata->report_schema_version;`	Технически объявляет `schema_version` в функции `глобальной области файла` и сразу задает начальное значение `metadata->report_schema_version`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1181	`const char* input_hash = metadata->input_sha256;`	Технически объявляет `input_hash` в функции `глобальной области файла` и сразу задает начальное значение `metadata->input_sha256`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1182	`const char* build_platform = metadata->build_platform;`	Технически объявляет `build_platform` в функции `глобальной области файла` и сразу задает начальное значение `metadata->build_platform`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1183	`const char* build_compiler = metadata->build_compiler;`	Технически объявляет `build_compiler` в функции `глобальной области файла` и сразу задает начальное значение `metadata->build_compiler`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1184	`const char* build_mode = metadata->build_mode;`	Технически объявляет `build_mode` в функции `глобальной области файла` и сразу задает начальное значение `metadata->build_mode`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1185	`size_t parse_failures = 0;`	Технически объявляет `parse_failures` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1186	`size_t anomaly_count = 0;`	Технически объявляет `anomaly_count` в функции `глобальной области файла` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
1187	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1188	`if (!us_ok[i]) parse_failures++;`	Технически проверяет условие `!us_ok[i]`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1189	`if (us[i].anomalies) anomaly_count++;`	Технически проверяет условие `us[i].anomalies`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1190	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1192	`sb_append(&sb, "<!doctype html><html lang=\"ru\"><head><meta charset=\"utf-8\">");`	Технически присваивает `sb_append(&sb, "<!doctype html><html lang` значение `\"ru\"><head><meta charset=\"utf-8\">")`. Логически это обновляет `sb_append(&sb, "<!doctype html><html lang`: хранит промежуточное значение, которое дальше используется в текущей функции.
1193	`sb_append(&sb, "<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");`	Технически присваивает `sb_append(&sb, "<meta name` значение `\"viewport\" content=\"width=device-width,initial-scale=1\">")`. Логически это обновляет `sb_append(&sb, "<meta name`: хранит промежуточное значение, которое дальше используется в текущей функции.
1194	`sb_append(&sb, "<meta http-equiv=\"Content-Security-Policy\" content=\"default-src 'none';"`	Технически присваивает `sb_append(&sb, "<meta http-equiv` значение `\"Content-Security-Policy\" content=\"default-src 'none';"`. Логически это обновляет `sb_append(&sb, "<meta http-equiv`: хранит промежуточное значение, которое дальше используется в текущей функции.
1195	`"style-src 'unsafe-inline';script-src 'unsafe-inline';img-src 'none';"`	Технически эта строка является частью С-конструкции `"style-src 'unsafe-inline';script-src 'unsafe-inline';img-src 'none';"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1196	`"base-uri 'none';form-action 'none'\">");`	Технически выполняет оператор `"base-uri 'none';form-action 'none'\">");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1197	`sb_append(&sb, "<title>Отчет EH/Unwind Analyzer - ");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1198	`html_escape(&sb, input_path ? input_path : "");`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1199	`sb_append(&sb, "</title><style>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1200	`sb_append(&sb,`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1201	`":root{color-scheme:dark;--bg:#0b1020;--panel:#121a2c;--panel2:#172238;"`	Технически эта строка является частью С-конструкции `":root{color-scheme:dark;--bg:#0b1020;--panel:#121a2c;--panel2:#172238;"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1202	`"--text:#e6edf7;--muted:#95a3b8;--line:#2a3851;--accent:#71b7ff;"`	Технически эта строка является частью С-конструкции `"--text:#e6edf7;--muted:#95a3b8;--line:#2a3851;--accent:#71b7ff;"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
1203	"--ok:#54d99a;--warn:#f7c65d;--danger:#ff718b}"	Технически эта строка является частью C-конструкции "--ok:#54d99a;--warn:#f7c65d;--danger:#ff718b}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1204	"*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--text);"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1205	"font:14px/1.5 system-ui,-apple-system,Segoe UI,sans-serif}"	Технически эта строка является частью C-конструкции "font:14px/1.5 system-ui,-apple-system,Segoe UI,sans-serif}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1206	"main{max-width:1500px;margin:auto;padding:28px}h1{margin:0;font-size:28px}"	Технически эта строка является частью C-конструкции "main{max-width:1500px;margin:auto;padding:28px}h1{margin:0;font-size:28px}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1207	"h2{margin:0 0 14px;font-size:19px}p{margin:6px 0}.muted{color:var(--muted)}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1208	".hero,.panel{background:var(--panel);border:1px solid var(--line);border-radius:14px;"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1209	"padding:20px;margin-bottom:18px}.hero{background:linear-gradient(135deg,#152541,#111827)}"	Технически вызывает gradient. Логически это действие нужно в функции глобальной области файла: вызывает функцию gradient для выполнения части алгоритма.
1210	".cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px;"	Технически вызывает repeat. Логически это действие нужно в функции глобальной области файла: вызывает функцию repeat для выполнения части алгоритма.
1211	"margin:18px 0}.card{background:var(--panel2);border:1px solid var(--line);"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1212	"border-radius:11px;padding:14px}.card b{display:block;font-size:22px;margin-top:4px}"	Технически эта строка является частью C-конструкции "border-radius:11px;padding:14px}.card b{display:block;font-size:22px;margin-top:4px}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1213	".ok{color:var(--ok)}.warn{color:var(--warn)}.danger{color:var(--danger)}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1214	".neutral{color:var(--accent)}.badge{display:inline-block;border:1px solid currentColor;"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1215	"border-radius:999px;padding:2px 8px;font-size:12px}.grid{display:grid;"	Технически эта строка является частью C-конструкции "border-radius:999px;padding:2px 8px;font-size:12px}.grid{display:grid;". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1216	"grid-template-columns:repeat(auto-fit,minmax(320px,1fr));gap:18px}"	Технически вызывает repeat. Логически это действие нужно в функции глобальной области файла: вызывает функцию repeat для выполнения части алгоритма.
1217	".table-wrap{overflow:auto;border:1px solid var(--line);border-radius:10px}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1218	"table{width:100%;border-collapse:collapse;white-space:nowrap}th,td{padding:9px 11px;"	Технически эта строка является частью C-конструкции "table{width:100%;border-collapse:collapse;white-space:nowrap}th,td{padding:9px 11px;". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1219	"border-bottom:1px solid var(--line);text-align:left;vertical-align:top}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1220	"th{position:sticky;top:0;background:#1a2740;color:#bcd0e9;z-index:1}"	Технически эта строка является частью C-конструкции "th{position:sticky;top:0;background:#1a2740;color:#bcd0e9;z-index:1}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1221	"tr:last-child td{border-bottom:0}tr:hover td{background:#172238}"	Технически эта строка является частью C-конструкции "tr:last-child td{border-bottom:0}tr:hover td{background:#172238}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1222	"code{color:#b8dcff}pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#0c1425;"	Технически эта строка является частью C-конструкции "code{color:#b8dcff}pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#0c1425;". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1223	"border:1px solid var(--line);border-radius:9px;padding:13px;color:#c8d6e8}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1224	".meta{display:grid;grid-template-columns:max-content 1fr;gap:5px 14px;"	Технически эта строка является частью C-конструкции ".meta{display:grid;grid-template-columns:max-content 1fr;gap:5px 14px;". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1225	"overflow-wrap:anywhere}.meta dt{color:var(--muted)}.meta dd{margin:0}"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1226	".toolbar{display:flex;gap:10px;align-items:center;margin-bottom:12px;flex-wrap:wrap}"	Технически эта строка является частью C-конструкции ".toolbar{display:flex;gap:10px;align-items:center;margin-bottom:12px;flex-wrap:wrap}". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1227	"input{min-width:280px;max-width:100%;background:#0c1425;color:var(--text);"	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1228	`"border:1px solid var(--line);border-radius:8px;padding:9px 11px}"`	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1229	`".empty{color:var(--muted);font-style:italic}.wrap{white-space:normal;min-width:280px}"`	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1230	`"footer{text-align:center;color:var(--muted);padding:12px}@media(max-width:700px){"`	Технически вызывает var. Логически это действие нужно в функции глобальной области файла: вызывает функцию var для выполнения части алгоритма.
1231	`"main{padding:14px}.hero,.panel{padding:15px}.meta{grid-template-columns:1fr}}");`	Технически выполняет оператор "main{padding:14px}.hero,.panel{padding:15px}.meta{grid-template-columns:1fr}}");. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1232	`sb_append(&sb, "</style></head><body><main>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1234	`sb_append(&sb, "<section class=\"hero\"><h1>EH/Unwind Analyzer</h1><p class=\"muted\">");`	Технически присваивает sb_append(&sb, "<section class значение \"hero\"> EH/Unwind Analyzer <p class=\"muted\">"). Логически это обновляет sb_append(&sb, "<section class: хранит промежуточное значение, которое дальше используется в текущей функции.
1235	`html_escape(&sb, input_path ? input_path : "");`	Технически вызывает html_escape. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_escape для выполнения части алгоритма.
1236	`sb_append(&sb, "</p><div class=\"cards\">");`	Технически присваивает sb_append(&sb, "</p><div class значение \"cards\">"). Логически это обновляет sb_append(&sb, "</p><div class: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1237	`sb_appendf(&sb, "<div class=\"card\">Записи RUNTIME_FUNCTION<b>%zu</b></div>", n);`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Записи RUNTIME_FUNCTION%zu ", n). Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1238	`sb_appendf(&sb, "<div class=\"card\">Ошибки разбора<b class=\"%s\">%zu</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Ошибки разбора<b class=\"%s\">%zu ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1239	`parse_failures ? "danger" : "ok", parse_failures);`	Технически выполняет оператор parse_failures ? "danger" : "ok", parse_failures);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1240	`sb_appendf(&sb, "<div class=\"card\">Аномальные записи<b class=\"%s\">%zu</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Аномальные записи<b class=\"%s\">%zu ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1241	`anomaly_count ? "warn" : "ok", anomaly_count);`	Технически выполняет оператор anomaly_count ? "warn" : "ok", anomaly_count);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1242	`if (hr) {`	Технически проверяет условие hr; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1243	`sb_appendf(&sb, "<div class=\"card\">Эвристические признаки<b class=\"%s\">%zu</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Эвристические признаки<b class=\"%s\">%zu ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1244	`hr->finding_count ? "warn" : "ok", hr->finding_count);`	Технически выполняет оператор hr->finding_count ? "warn" : "ok", hr->finding_count);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1245	`sb_appendf(&sb, "<div class=\"card\">Оценка подозрительности<b class=\"%s\">%d / 100</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Оценка подозрительности<b class=\"%s\">%d / 100 ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1246	`html_score_class(hr), hr->score);`	Технически вызывает html_score_class. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_score_class для выполнения части алгоритма.
1247	`sb_append(&sb, "<div class=\"card\">Статический вывод<b class=\"");`	Технически присваивает sb_append(&sb, "<div class значение \"card\">Статический вывод<b class=\""). Логически это обновляет sb_append(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1248	`sb_append(&sb, html_score_class(hr));`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1249	`sb_append(&sb, "\" style=\"font-size:16px\">");`	Технически присваивает sb_append(&sb, "\" style значение \"font-size:16px\">"). Логически это обновляет sb_append(&sb, "\" style: хранит промежуточное значение, которое дальше используется в текущей функции.
1250	`html_escape(&sb, ru_verdict_text(hr->verdict));`	Технически вызывает html_escape. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_escape для выполнения части алгоритма.
1251	`sb_append(&sb, "</b></div>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1252	`sb_appendf(&sb, "<div class=\"card\">Уверенность<b>%.0f%%</b></div>",`	Технически присваивает `sb_appendf(&sb, "<div class` значение `\"card\">Уверенность<b>%.0f%%</b></div>",`. Логически это обновляет `sb_appendf(&sb, "<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1253	`hr->confidence * 100.0);`	Технически выполняет оператор `hr->confidence * 100.0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1254	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1255	`sb_append(&sb, "<div class=\"card\">Динамическая проверка<b class=\"");`	Технически присваивает `sb_append(&sb, "<div class` значение `\"card\">Динамическая проверка<b class=\""`). Логически это обновляет `sb_append(&sb, "<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1256	`sb_append(&sb, (dr && dr->ran && dr->ok && !dr->timed_out) ? "ok" : "neutral");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1257	`sb_append(&sb, "\" style=\"font-size:16px\">");`	Технически присваивает `sb_append(&sb, "\" style` значение `\"font-size:16px\">"`). Логически это обновляет `sb_append(&sb, "\" style`: хранит промежуточное значение, которое дальше используется в текущей функции.
1258	`html_escape(&sb, ru_status_text(dr));`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1259	`sb_append(&sb, "</b></div></div>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1261	`sb_append(&sb, "<dl class=\"meta\"><dt>Сформировано</dt><dd>");`	Технически присваивает `sb_append(&sb, "<dl class` значение `\"meta\"><dt>Сформировано</dt><dd>"`). Логически это обновляет `sb_append(&sb, "<dl class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1262	`html_escape(&sb, generated_at);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1263	`sb_append(&sb, "</dd><dt>Инструмент / schema</dt><dd>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1264	`html_escape(&sb, tool_version);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1265	`sb_append(&sb, " / ");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1266	`html_escape(&sb, schema_version);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1267	`sb_append(&sb, "</dd><dt>SHA-256</dt><dd><code>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1268	`html_escape(&sb, input_hash);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1269	`sb_append(&sb, "</code></dd><dt>Сборка</dt><dd>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1270	`html_escape(&sb, build_platform);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1271	`sb_append(&sb, " / ");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1272	`html_escape(&sb, build_compiler);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1273	`sb_append(&sb, " / ");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1274	`html_escape(&sb, build_mode);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1275	`sb_append(&sb, "</dd><dt>Параметры анализа</dt><dd>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1276	`sb_appendf(&sb, "strict ABI: %s; follow chain: %s; max depth: %u",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1277	`ru_yes_no(metadata->strict_abi),`	Технически вызывает `ru_yes_no`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `ru_yes_no` для выполнения части алгоритма.
1278	`ru_yes_no(metadata->follow_chain),`	Технически вызывает `ru_yes_no`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `ru_yes_no` для выполнения части алгоритма.
1279	`(unsigned)metadata->max_chain_depth);`	Технически выполняет оператор `(unsigned)metadata->max_chain_depth);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1280	`sb_appendf(&sb, "</dd><dt>Размер файла</dt><dd>%llu байт</dd><dt>Команда</dt><dd><code>",`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1281	`(unsigned long long)metadata->input_size);`	Технически выполняет оператор `(unsigned long long)metadata->input_size);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
1282	`html_escape(&sb, metadata->command_line);`	Технически вызывает `html_escape`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_escape` для выполнения части алгоритма.
1283	`sb_append(&sb, "</code>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1284	`sb_append(&sb, "</dd></dl></section>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1286	`sb_append(&sb, "<div class=\"grid\"><section class=\"panel\"><h2>Образ PE</h2><dl class=\"meta\">");`	Технически присваивает `sb_append(&sb, "<div class` значение `\"grid\"><section class=\"panel\"><h2>Образ PE</h2><dl class=\"meta\">")`. Логически это обновляет `sb_append(&sb, "<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1287	`sb_appendf(&sb,`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
1288	`"<dt>ImageBase</dt><dd><code>0x%llx</code></dd>"`	Технически эта строка является частью С-конструкции `"<dt>ImageBase</dt><dd><code>0x%llx</code></dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1289	`"<dt>SizeOfImage</dt><dd><code>0x%x</code></dd>"`	Технически эта строка является частью С-конструкции `"<dt>SizeOfImage</dt><dd><code>0x%x</code></dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1290	`"<dt>SizeOfHeaders</dt><dd><code>0x%x</code></dd>"`	Технически эта строка является частью С-конструкции `"<dt>SizeOfHeaders</dt><dd><code>0x%x</code></dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1291	`"<dt>Exception directory</dt><dd><code>0x%08x + 0x%x</code></dd>"`	Технически эта строка является частью С-конструкции `"<dt>Exception directory</dt><dd><code>0x%08x + 0x%x</code></dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1292	`"<dt>SectionAlignment / FileAlignment</dt><dd><code>0x%x / 0x%x</code></dd></dl></section>",`	Технически эта строка является частью С-конструкции `"<dt>SectionAlignment / FileAlignment</dt><dd><code>0x%x / 0x%x</code></dd></dl></section>",`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1293	`(unsigned long long)img->image_base, (unsigned)img->size_of_image,`	Технически эта строка является частью С-конструкции `(unsigned long long)img->image_base, (unsigned)img->size_of_image,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1294	`(unsigned)img->size_of_headers, (unsigned)img->exception_rva,`	Технически эта строка является частью С-конструкции `(unsigned)img->size_of_headers, (unsigned)img->exception_rva,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1295	`(unsigned)img->exception_size, (unsigned)img->section_alignment,`	Технически эта строка является частью С-конструкции `(unsigned)img->exception_size, (unsigned)img->section_alignment,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1296	`(unsigned)img->file_alignment);`	Технически выполняет оператор `(unsigned)img->file_alignment);`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1297	`sb_append(&sb, "<section class=\"panel\"><h2>Структурная проверка</h2><dl class=\"meta\">");`	Технически присваивает `sb_append(&sb, "<section class` значение `\"panel\"><h2>Структурная проверка</h2><dl class=\"meta\">")`. Логически это обновляет `sb_append(&sb, "<section class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1298	`sb_appendf(&sb,`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
1299	`"<dt>Raw-диапазоны вне файла</dt><dd>%zu</dd><dt>Virtual-диапазоны вне image</dt><dd>%zu</dd>"`	Технически эта строка является частью С-конструкции `"<dt>Raw-диапазоны вне файла</dt><dd>%zu</dd><dt>Virtual-диапазоны вне image</dt><dd>%zu</dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1300	`"<dt>Пересечения raw</dt><dd>%zu</dd><dt>Пересечения virtual</dt><dd>%zu</dd>"`	Технически эта строка является частью С-конструкции `"<dt>Пересечения raw</dt><dd>%zu</dd><dt>Пересечения virtual</dt><dd>%zu</dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1301	`"<dt>Невыровненные секции</dt><dd>%zu</dd><dt>Exception directory OOB</dt><dd>%s</dd>"`	Технически эта строка является частью С-конструкции `"<dt>Невыровненные секции</dt><dd>%zu</dd><dt>Exception directory OOB</dt><dd>%s</dd>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1302	`"</dl></section></div>",`	Технически эта строка является частью С-конструкции `"</dl></section></div>",`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1303	`img->section_raw_oob, img->section_virtual_oob, img->section_raw_overlaps,`	Технически эта строка является частью С-конструкции `img->section_raw_oob, img->section_virtual_oob, img->section_raw_overlaps,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1304	`img->section_virtual_overlaps, img->section_misaligned,`	Технически эта строка является частью С-конструкции `img->section_virtual_overlaps, img->section_misaligned,`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.

Стр.	Код	Нормальное объяснение
1305	`ru_yes_no(img->exception_directory_oob));`	Технически вызывает `ru_yes_no`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `ru_yes_no` для выполнения части алгоритма.
1307	`sb_append(&sb, "<section class=\"panel\"><h2>Секции</h2><div class=\"table-wrap\"><table>"`	Технически присваивает `sb_append(&sb, "<section class` значение `\"panel\"><h2>Секции</h2><div class=\"table-wrap\"><table>"`. Логически это обновляет `sb_append(&sb, "<section class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1308	`"<thead><tr><th>Имя</th><th>VirtualAddress</th><th>VirtualSize</th>"`	Технически эта строка является частью C-конструкции `"<thead><tr><th>Имя</th><th>VirtualAddress</th><th>VirtualSize</th>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1309	`"<th>Raw offset</th><th>Raw size</th><th>Права</th></tr></thead><tbody>");`	Технически выполняет оператор `"<th>Raw offset</th><th>Raw size</th><th>Права</th></tr></thead><tbody>");`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1310	`for (size_t i = 0; i < img->num_sections; i++) {`	Технически начинает цикл `size_t i = 0; i < img->num_sections; i++`. Логически он проходит по секциям PE-файла, чтобы проверить каждую запись, а не только первую.
1311	`const PeSection* section = &img->sections[i];`	Технически объявляет `section` в функции глобальной области файла и сразу задает начальное значение `&img->sections[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1312	`char permissions[4];`	Технически объявляет `permissions` в функции глобальной области файла. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1313	`permissions[0] = (section->characteristics & IMAGE_SCN_MEM_READ) ? 'R' : '-';`	Технически присваивает `permissions[0]` значение `(section->characteristics & IMAGE_SCN_MEM_READ) ? 'R' : '-'`. Логически это обновляет `permissions[0]`: хранит промежуточное значение, которое дальше используется в текущей функции.
1314	`permissions[1] = (section->characteristics & IMAGE_SCN_MEM_WRITE) ? 'W' : '-';`	Технически присваивает `permissions[1]` значение `(section->characteristics & IMAGE_SCN_MEM_WRITE) ? 'W' : '-'`. Логически это обновляет `permissions[1]`: хранит промежуточное значение, которое дальше используется в текущей функции.
1315	`permissions[2] = (section->characteristics & IMAGE_SCN_MEM_EXECUTE) ? 'X' : '-';`	Технически присваивает `permissions[2]` значение `(section->characteristics & IMAGE_SCN_MEM_EXECUTE) ? 'X' : '-'`. Логически это обновляет `permissions[2]`: хранит промежуточное значение, которое дальше используется в текущей функции.
1316	`permissions[3] = '\0';`	Технически присваивает `permissions[3]` значение `'\0'`. Логически это обновляет `permissions[3]`: хранит промежуточное значение, которое дальше используется в текущей функции.
1317	`sb_append(&sb, "<tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1318	`html_text_cell(&sb, section->name);`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1319	`html_hex32_cell(&sb, section->virtual_address);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1320	`html_hex32_cell(&sb, section->virtual_size);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1321	`html_hex32_cell(&sb, section->raw_ptr);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1322	`html_hex32_cell(&sb, section->raw_size);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1323	`html_text_cell(&sb, permissions);`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1324	`sb_append(&sb, "</tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1325	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1326	`sb_append(&sb, "</tbody></table></div></section>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1328	`if (hr || dr) {`	Технически проверяет условие `hr || dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1329	`AssessmentSummary summary = assessment_summary(hr, dr);`	Технически присваивает `AssessmentSummary summary` значение `assessment_summary(hr, dr)`. Логически это обновляет `AssessmentSummary summary`: хранит промежуточное значение, которое дальше используется в текущей функции.
1330	`ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);`	Технически присваивает `ManualValidationSteps steps` значение `collect_manual_validation_steps(hr, dr)`. Логически это обновляет `ManualValidationSteps steps`: хранит промежуточное значение, которое дальше используется в текущей функции.
1331	`sb_append(&sb, "<section class=\"panel\"><h2>Итоговая оценка</h2><div class=\"cards\">");`	Технически присваивает `sb_append(&sb, "<section class` значение `\"panel\"><h2>Итоговая оценка</h2><div class=\"cards\">"`. Логически это обновляет `sb_append(&sb, "<section class`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1332	`sb_append(&sb, "<div class=\"card\">Вывод<b style=\"font-size:16px\">");`	Технически присваивает `sb_append(&sb, "<div class` значение `\"card\">Вывод<b style=\"font-size:16px\">"`). Логически это обновляет `sb_append(&sb, "<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1333	`html_escape(&sb, ru_verdict_text(summary.final_verdict));`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1334	`sb_append(&sb, "</b></div><div class=\"card\">Сила доказательств<b style=\"font-size:16px\">");`	Технически присваивает `sb_append(&sb, "</b></div><div class` значение `\"card\">Сила доказательств<b style=\"font-size:16px\">"`). Логически это обновляет `sb_append(&sb, "</b></div><div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1335	`html_escape(&sb, ru_case_strength_text(summary.strength));`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1336	`sb_appendf(&sb,`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1337	`"</b></div><div class=\"card\">Итоговая уверенность<b>%.0f%%</b></div>"`	Технически присваивает `"</b></div><div class` значение `\"card\">Итоговая уверенность<b>%.0f%%</b></div>"`. Логически это обновляет `"</b></div><div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1338	`"<div class=\"card\">Статика / динамика<b style=\"font-size:16px\">",`	Технически присваивает `"<div class` значение `\"card\">Статика / динамика<b style=\"font-size:16px\">",`. Логически это обновляет `"<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1339	`summary.final_confidence * 100.0);`	Технически выполняет оператор `summary.final_confidence * 100.0);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
1340	`html_escape(&sb, ru_level_text(summary.static_evidence_level));`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1341	`sb_append(&sb, " / ");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1342	`html_escape(&sb, ru_level_text(summary.runtime_evidence_level));`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1343	`sb_appendf(&sb,`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1344	`"</b></div><div class=\"card\">Корреляция слоев<b class=\"%s\" "`	Технически присваивает `"</b></div><div class` значение `\"card\">Корреляция слоев<b class=\"%s\" "`. Логически это обновляет `"</b></div><div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1345	`"style=\"font-size:16px\">%s</b></div>"`	Технически присваивает `"style` значение `\"font-size:16px\">%s</b></div>"`. Логически это обновляет `"style`: хранит промежуточное значение, которое дальше используется в текущей функции.
1346	`"<div class=\"card\">Динамические сигналы<b>%u</b></div></div>",`	Технически присваивает `"<div class` значение `\"card\">Динамические сигналы<b>%u</b></div></div>",`. Логически это обновляет `"<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1347	`summary.correlated ? "ok" : "neutral",`	Технически эта строка является частью C-конструкции `summary.correlated ? "ok" : "neutral",`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1348	`ru_yes_no(summary.correlated),`	Технически вызывает `ru_yes_no`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `ru_yes_no` для выполнения части алгоритма.
1349	`summary.dynamic_signals);`	Технически выполняет оператор `summary.dynamic_signals);`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1350	`if (steps.count) {`	Технически проверяет условие `steps.count`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1351	`sb_append(&sb, "<h2 style=\"margin-top:18px\">План ручной проверки</h2><ol>");`	Технически присваивает `sb_append(&sb, "<h2 style` значение `\"margin-top:18px\">План ручной проверки</h2><ol>"`). Логически это обновляет `sb_append(&sb, "<h2 style`: хранит промежуточное значение, которое дальше используется в текущей функции.
1352	`for (size_t i = 0; i < steps.count; i++) {`	Технически начинает цикл `size_t i = 0; i < steps.count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1353	`sb_append(&sb, "<li>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1354	`html_append_manual_step_ru(&sb, steps.items[i]);`	Технически вызывает `html_append_manual_step_ru`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_append_manual_step_ru` для выполнения части алгоритма.
1355	`sb_append(&sb, "</li>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1356	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1357	`sb_append(&sb, "</ol>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1358	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1359	`sb_append(&sb, "</section>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1360	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1362	`if (hr) {`	Технически проверяет условие hr; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1363	`sb_append(&sb, "<section class=\"panel\"><h2>Статические признаки</h2>");`	Технически присваивает sb_append(&sb, "<section class значение \"panel\"> Статические признаки "). Логически это обновляет sb_append(&sb, "<section class: хранит промежуточное значение, которое дальше используется в текущей функции.
1364	`if (hr->finding_count == 0) {`	Технически проверяет условие hr->finding_count == 0; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1365	`sb_append(&sb, "<p class=\"empty\">Эвристические признаки не обнаружены.</p>");`	Технически присваивает sb_append(&sb, "<p class значение \"empty\">Эвристические признаки не обнаружены.</p>"). Логически это обновляет sb_append(&sb, "<p class: хранит промежуточное значение, которое дальше используется в текущей функции.
1366	`} else {`	Технически закрывает предыдущую ветку и открывает else. Логически код ниже выполняется только когда условие из if оказалось ложным.
1367	`sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Важность</th>"`	Технически присваивает sb_append(&sb, "<div class значение \"table-wrap\"><table><thead><tr><th>Важность</th>". Логически это обновляет sb_append(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1368	`"<th>Код</th><th>Семейство</th><th>Затронуто</th><th>RVA</th>"`	Технически эта строка является частью С-конструкции "<th>Код</th><th>Семейство</th><th>Затронуто</th><th>RVA</th>". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1369	`"<th>Секция</th><th>Описание</th></tr></thead><tbody>");`	Технически выполняет оператор "<th>Секция</th><th>Описание</th></tr></thead><tbody>");. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1370	`for (size_t i = 0; i < hr->finding_count; i++) {`	Технически начинает цикл size_t i = 0; i < hr->finding_count; i++. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1371	`const HeuristicFinding* finding = &hr->findings[i];`	Технически объявляет finding в функции глобальной области файла и сразу задает начальное значение &hr->findings[i]. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
1372	`sb_append(&sb, "<tr>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1373	`html_text_cell(&sb, ru_severity_text(finding->severity));`	Технически вызывает html_text_cell. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_text_cell для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1374	`html_text_cell(&sb, finding->code);`	Технически вызывает html_text_cell. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_text_cell для выполнения части алгоритма.
1375	`html_text_cell(&sb, finding->family);`	Технически вызывает html_text_cell. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_text_cell для выполнения части алгоритма.
1376	`sb_appendf(&sb, "<td>%zu</td>", finding->affected_count);`	Технически вызывает sb_appendf. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
1377	`if (finding->has_location) html_hex32_cell(&sb, finding->representative_rva);`	Технически проверяет условие finding->has_location) html_hex32_cell(&sb, finding->representative_rva; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1378	`else sb_append(&sb, "<td>-</td>");`	Технически объявляет прототип функции sb_append без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет С-строку в StrBuf.
1379	`html_text_cell(&sb, finding->has_location`	Технически вызывает html_text_cell. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_text_cell для выполнения части алгоритма.
1380	`? section_name_or_dash(img, finding->representative_rva)`	Технически объявляет/начинает функцию section_name_or_dash; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
1381	`: "-");`	Технически выполняет оператор : "-");. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1382	`sb_append(&sb, "<td class=\"wrap\">");`	Технически присваивает sb_append(&sb, "<td class значение \"wrap\">"). Логически это обновляет sb_append(&sb, "<td class: хранит промежуточное значение, которое дальше используется в текущей функции.
1383	`html_append_finding_message_ru(&sb, finding);`	Технически вызывает html_append_finding_message_ru. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_append_finding_message_ru для выполнения части алгоритма.
1384	`sb_append(&sb, "</td></tr>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1385	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1386	`sb_append(&sb, "</tbody></table></div>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1387	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1388	`if (hr->mitigation_count) {`	Технически проверяет условие hr->mitigation_count; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1389	`sb_append(&sb, "<h2 style=\"margin-top:18px\">Правила снижения уверенности</h2><div class=\"table-wrap\">"`	Технически присваивает sb_append(&sb, "<h2 style значение \"margin-top:18px\">Правила снижения уверенности</h2><div class=\"table-wrap\">". Логически это обновляет sb_append(&sb, "<h2 style: хранит промежуточное значение, которое дальше используется в текущей функции.
1390	`"<table><thead><tr><th>Код</th><th>Пояснение</th></tr></thead><tbody>");`	Технически выполняет оператор "<table><thead><tr><th>Код</th><th>Пояснение</th></tr></thead><tbody>");. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1391	`for (size_t i = 0; i < hr->mitigation_count; i++) {`	Технически начинает цикл size_t i = 0; i < hr->mitigation_count; i++. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1392	`sb_append(&sb, "<tr>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1393	`html_text_cell(&sb, hr->mitigations[i].code);`	Технически вызывает html_text_cell. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_text_cell для выполнения части алгоритма.
1394	`sb_append(&sb, "<td class=\"wrap\">");`	Технически присваивает sb_append(&sb, "<td class значение \"wrap\">"). Логически это обновляет sb_append(&sb, "<td class: хранит промежуточное значение, которое дальше используется в текущей функции.
1395	`html_append_mitigation_message_ru(&sb, hr->mitigations[i].code, hr->mitigations[i].message);`	Технически вызывает html_append_mitigation_message_ru. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_append_mitigation_message_ru для выполнения части алгоритма.
1396	`sb_append(&sb, "</td></tr>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1397	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1398	`sb_append(&sb, "</tbody></table></div>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1399	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1400	`sb_append(&sb, "<h2 style=\"margin-top:18px\">Детали анализа</h2><pre>");`	Технически присваивает sb_append(&sb, "<h2 style значение \"margin-top:18px\">Детали анализа</h2><pre>"). Логически это обновляет sb_append(&sb, "<h2 style: хранит промежуточное значение, которое дальше используется в текущей функции.
1401	`html_escape(&sb, hr->details.s ? hr->details.s : "");`	Технически вызывает html_escape. Логически это действие нужно в функции глобальной области файла: вызывает функцию html_escape для выполнения части алгоритма.
1402	`sb_append(&sb, "</pre></section>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1403	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1405	`if (dr) {`	Технически проверяет условие dr; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1406	`sb_append(&sb, "<section class=\"panel\"><h2>Динамическая проверка</h2><div class=\"cards\">");`	Технически присваивает sb_append(&sb, "<section class значение \"panel\"> Динамическая проверка <div class=\"cards\">"). Логически это обновляет sb_append(&sb, "<section class: хранит промежуточное значение, которое дальше используется в текущей функции.
1407	`sb_appendf(&sb, "<div class=\"card\">Исключения<b>%u</b></div>" ,`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Исключения%u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1408	`(unsigned)dr->exceptions_seen);`	Технически выполняет оператор (unsigned)dr->exceptions_seen);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1409	`sb_appendf(&sb, "<div class=\"card\">Попадания в handler<b>%u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Попадания в handler%u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1410	`(unsigned)(dr->unwind_handler_hits + dr->veh_hits + dr->vch_hits + dr->uef_hits));`	Технически выполняет оператор (unsigned)(dr->unwind_handler_hits + dr->veh_hits + dr->vch_hits + dr->uef_hits));. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.

Стр.	Код	Нормальное объяснение
1411	`sb_appendf(&sb, "<div class=\"card\">Вызовы runtime-таблиц<b>%u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Вызовы runtime-таблиц%u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1412	`(unsigned)(dr->rtl_add_function_table_calls +`	Технически эта строка является частью С-конструкции (unsigned)(dr->rtl_add_function_table_calls +. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1413	`dr->rtl_install_function_table_cb_calls));`	Технически выполняет оператор dr->rtl_install_function_table_cb_calls));. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1414	`sb_appendf(&sb, "<div class=\"card\">Потерянные события<b class=\"%s\">%u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Потерянные события<b class=\"%s\">%u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1415	`dr->dropped_events ? "warn" : "ok", (unsigned)dr->dropped_events);`	Технически выполняет оператор dr->dropped_events ? "warn" : "ok", (unsigned)dr->dropped_events);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1416	`sb_appendf(&sb, "<div class=\"card\">Инструментация завершена<b class=\"%s\">%s</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Инструментация завершена<b class=\"%s\">%s ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1417	`dr->instrumentation_complete ? "ok" : "warn",`	Технически эта строка является частью С-конструкции dr->instrumentation_complete ? "ok" : "warn",. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1418	`ru_yes_no(dr->instrumentation_complete));`	Технически вызывает ru_yes_no. Логически это действие нужно в функции глобальной области файла: вызывает функцию ru_yes_no для выполнения части алгоритма.
1419	`sb_appendf(&sb, "<div class=\"card\">Handlers найдены / включены<b>%u / %u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Handlers найдены / включены%u / %u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1420	`(unsigned)dr->unwind_handlers_discovered,`	Технически эта строка является частью С-конструкции (unsigned)dr->unwind_handlers_discovered,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1421	`(unsigned)dr->unwind_handler_bp_set);`	Технически выполняет оператор (unsigned)dr->unwind_handler_bp_set);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1422	`sb_appendf(&sb, "<div class=\"card\">Ошибки breakpoint<b class=\"%s\">%u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Ошибки breakpoint<b class=\"%s\">%u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1423	`dr->breakpoint_set_failures ? "warn" : "ok",`	Технически эта строка является частью С-конструкции dr->breakpoint_set_failures ? "warn" : "ok",. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1424	`(unsigned)dr->breakpoint_set_failures);`	Технически выполняет оператор (unsigned)dr->breakpoint_set_failures);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1425	`sb_appendf(&sb, "<div class=\"card\">Покрытие модулей<b class=\"%s\">пик %u, ошибки %u, усечения %u</b></div>",`	Технически присваивает sb_appendf(&sb, "<div class значение \"card\">Покрытие модулей<b class=\"%s\">пик %u, ошибки %u, усечения %u ",. Логически это обновляет sb_appendf(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.
1426	`(dr->module_refresh_failures || dr->module_list_truncations) ? "warn" : "ok",`	Технически выполняет битовую операцию (dr->module_refresh_failures || dr->module_list_truncations) ? "warn" : "ok",. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
1427	`(unsigned)dr->modules_observed_peak,`	Технически эта строка является частью С-конструкции (unsigned)dr->modules_observed_peak,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1428	`(unsigned)dr->module_refresh_failures,`	Технически эта строка является частью С-конструкции (unsigned)dr->module_refresh_failures,. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1429	`(unsigned)dr->module_list_truncations);`	Технически выполняет оператор (unsigned)dr->module_list_truncations);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1430	`sb_append(&sb, "</div>");`	Технически вызывает sb_append. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1431	`if (dr->event_count) {`	Технически проверяет условие dr->event_count; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1432	`sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Тип</th><th>Модуль</th>"`	Технически присваивает sb_append(&sb, "<div class значение \"table-wrap\"><table><thead><tr><th>Тип</th><th>Модуль</th>". Логически это обновляет sb_append(&sb, "<div class: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1433	`"<th>Thread</th><th>Exception</th><th>Chance</th><th>Address</th>"`	Технически эта строка является частью C-конструкции `"<th>Thread</th><th>Exception</th><th>Chance</th><th>Address</th>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1434	`"<th>Arg 0</th><th>Arg 1</th><th>Arg 2</th><th>Arg 3</th>"`	Технически эта строка является частью C-конструкции `"<th>Arg 0</th><th>Arg 1</th><th>Arg 2</th><th>Arg 3</th>"`. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1435	`"</tr></thead><tbody>");`	Технически выполняет оператор `"</tr></thead><tbody>");`. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1436	`for (uint32_t i = 0; i < dr->event_count; i++) {`	Технически начинает цикл `uint32_t i = 0; i < dr->event_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1437	`const DynamicEvent* event = &dr->events[i];`	Технически присваивает `const DynamicEvent* event` значение `&dr->events[i]`. Логически это обновляет `const DynamicEvent* event`: хранит промежуточное значение, которое дальше используется в текущей функции.
1438	`sb_append(&sb, "<tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1439	`html_text_cell(&sb, event->type);`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1440	`html_text_cell(&sb, event->module);`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_text_cell` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
1441	`sb_appendf(&sb, "<td>%u</td>", (unsigned)event->thread_id);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
1442	`html_hex32_cell(&sb, event->exception_code);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1443	`html_text_cell(&sb, event->first_chance ? "first" : "second / n/a");`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1444	`html_hex64_cell(&sb, event->address);`	Технически вызывает `html_hex64_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex64_cell` для выполнения части алгоритма.
1445	`html_hex64_cell(&sb, event->arg0);`	Технически вызывает `html_hex64_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex64_cell` для выполнения части алгоритма.
1446	`html_hex64_cell(&sb, event->arg1);`	Технически вызывает `html_hex64_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex64_cell` для выполнения части алгоритма.
1447	`html_hex64_cell(&sb, event->arg2);`	Технически вызывает `html_hex64_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex64_cell` для выполнения части алгоритма.
1448	`html_hex64_cell(&sb, event->arg3);`	Технически вызывает `html_hex64_cell`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_hex64_cell` для выполнения части алгоритма.
1449	`sb_append(&sb, "</tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1450	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1451	`sb_append(&sb, "</tbody></table></div>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.
1452	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
1453	`sb_append(&sb, "<p class=\"empty\">Динамические события не записаны.</p>");`	Технически присваивает `sb_append(&sb, "<p class` значение `\"empty\">Динамические события не записаны.</p>")`. Логически это обновляет `sb_append(&sb, "<p class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1454	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1455	`if (dr->details.s && dr->details.len) {`	Технически проверяет условие `dr->details.s && dr->details.len`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1456	`sb_append(&sb, "<h2 style=\"margin-top:18px\">Журнал выполнения</h2><pre>");`	Технически присваивает `sb_append(&sb, "<h2 style` значение `\"margin-top:18px\">Журнал выполнения</h2><pre>")`. Логически это обновляет `sb_append(&sb, "<h2 style`: хранит промежуточное значение, которое дальше используется в текущей функции.
1457	`html_escape(&sb, dr->details.s);`	Технически вызывает `html_escape`. Логически это действие нужно в функции глобальной области файла: вызывает функцию `html_escape` для выполнения части алгоритма.
1458	`sb_append(&sb, "</pre>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции глобальной области файла: добавляет текст в динамический буфер.

Стр.	Код	Нормальное объяснение
1459	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1460	`sb_append(&sb, "</section>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1461	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1463	`sb_append(&sb, "<section class=\"panel\"><h2>Таблица RUNTIME_FUNCTION</h2><div class=\"toolbar\">"`	Технически присваивает `sb_append(&sb, "<section class` значение `\"panel\"><h2>Таблица RUNTIME_FUNCTION</h2><div class=\"toolbar\">"`. Логически это обновляет `sb_append(&sb, "<section class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1464	`"<input id=\"function-filter\" type=\"search\" placeholder=\"Фильтр по RVA, секции, флагу или ошибке\">"`	Технически присваивает `"<input id` значение `\"function-filter\" type=\"search\" placeholder=\"Фильтр по RVA, секции, флагу или ошибке\">"`. Логически это обновляет `"<input id`: хранит промежуточное значение, которое дальше используется в текущей функции.
1465	`"<span id=\"function-count\" class=\"muted\"></span></div>"`	Технически присваивает `"<span id` значение `\"function-count\" class=\"muted\"></span></div>"`. Логически это обновляет `"<span id`: хранит промежуточное значение, которое дальше используется в текущей функции.
1466	`"<div class=\"table-wrap\"><table id=\"functions\"><thead><tr><th>#</th>"`	Технически присваивает `"<div class` значение `\"table-wrap\"><table id=\"functions\"><thead><tr><th>#</th>"`. Логически это обновляет `"<div class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1467	`"<th>Begin</th><th>End</th><th>Size</th><th>Секция кода</th><th>Unwind RVA</th>"`	Технически эта строка является частью C-конструкции `"<th>Begin</th><th>End</th><th>Size</th><th>Секция кода</th><th>Unwind RVA</th>"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1468	`"<th>Unwind section</th><th>Статус</th><th>Version / flags</th><th>Stack</th>"`	Технически эта строка является частью C-конструкции `"<th>Unwind section</th><th>Статус</th><th>Version / flags</th><th>Stack</th>"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1469	`"<th>Handler</th><th>Аномалии / ошибка</th></tr></thead><tbody>");`	Технически выполняет оператор `"<th>Handler</th><th>Аномалии / ошибка</th></tr></thead><tbody>");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1470	`for (size_t i = 0; i < n; i++) {`	Технически начинает цикл `size_t i = 0; i < n; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
1471	`char flags[64];`	Технически объявляет `flags` в функции `глобальной области файла`. Логически это место для данных: флаги UNWIND_INFO, включая наличие exception/termination handler.
1472	`unwind_flags_to_string(us[i].flags, flags);`	Технически вызывает `unwind_flags_to_string`. Логически это действие нужно в функции `глобальной области файла`: переводит флаги UNWIND_INFO в текст.
1473	`anomalies.len = 0;`	Технически присваивает `anomalies.len` значение `0`. Логически это обновляет `len`: хранит размер или количество, поэтому используется в проверках границ.
1474	`if (anomalies.s) anomalies.s[0] = '\0';`	Технически проверяет условие `anomalies.s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
1475	`anomalies_to_str(&anomalies, us[i].anomalies);`	Технически вызывает `anomalies_to_str`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `anomalies_to_str` для выполнения части алгоритма.
1476	`sb_append(&sb, "<tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1477	`sb_appendf(&sb, "<td>%zu</td>", i);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1478	`html_hex32_cell(&sb, rf[i].begin_rva);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1479	`html_hex32_cell(&sb, rf[i].end_rva);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1480	`sb_appendf(&sb, "<td>%u</td>", (unsigned)runtime_function_size(&rf[i]));`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1481	`html_text_cell(&sb, section_name_or_dash(img, rf[i].begin_rva));`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1482	`html_hex32_cell(&sb, rf[i].unwind_rva);`	Технически вызывает `html_hex32_cell`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_hex32_cell` для выполнения части алгоритма.
1483	`html_text_cell(&sb, section_name_or_dash(img, rf[i].unwind_rva));`	Технически вызывает `html_text_cell`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_text_cell` для выполнения части алгоритма.
1484	`sb_append(&sb, us_ok[i] ? "<td class=\"ok\">OK</td>" : "<td class=\"danger\">Ошибка</td>");`	Технически присваивает `sb_append(&sb, us_ok[i] ? "<td class` значение `\"ok\">OK</td>" : "<td class=\"danger\">Ошибка</td>"`). Логически это обновляет `sb_append(&sb, us_ok[i] ? "<td class`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1485	`sb_appendf(&sb, "<td>v%u / ", (unsigned)us[i].version);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1486	`html_escape(&sb, flags);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1487	`sb_append(&sb, "</td>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1488	`sb_appendf(&sb, "<td>%u B</td>", (unsigned)us[i].fixed_stack_bytes);`	Технически вызывает `sb_appendf`. Логически это действие нужно в функции `глобальной области файла`: добавляет форматированный текст в динамический буфер.
1489	`if (us[i].has_handler) html_hex32_cell(&sb, us[i].handler_rva);`	Технически проверяет условие `us[i].has_handler) html_hex32_cell(&sb, us[i].handler_rva`; логически отделяет функции с language-specific handler от обычных функций.
1490	`else sb_append(&sb, "<td>-</td>");`	Технически объявляет прототип функции `sb_append` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет C-строку в StrBuf.
1491	`sb_append(&sb, "<td class=\"wrap\">");`	Технически присваивает `sb_append(&sb, "<td class` значение `\"wrap\">"`). Логически это обновляет `sb_append(&sb, "<td class`: хранит промежуточное значение, которое дальше используется в текущей функции.
1492	`if (us[i].anomalies) html_escape(&sb, anomalies.s);`	Технически проверяет условие `us[i].anomalies) html_escape(&sb, anomalies.s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1493	`if (us[i].anomalies && us[i].parse_error[0]) sb_append(&sb, "; ");`	Технически проверяет условие `us[i].anomalies && us[i].parse_error[0]) sb_append(&sb, "; "`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1494	`html_escape(&sb, us[i].parse_error);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1495	`if (!us[i].anomalies && !us[i].parse_error[0]) sb_append(&sb, "-");`	Технически проверяет условие `!us[i].anomalies && !us[i].parse_error[0]) sb_append(&sb, "-"`; логически это защита от NULL-указателя и работы с неинициализированными данными.
1496	`sb_append(&sb, "</td></tr>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1497	`}`	Технически завершает область видимости внутри `глобальной области файла`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1498	`sb_append(&sb, "</tbody></table></div></section>");`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1500	`sb_append(&sb,`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1501	`"<footer>Сформировано EH/Unwind Analyzer ");`	Технически выполняет оператор `"<footer>Сформировано EH/Unwind Analyzer ");`. Логически в функции `глобальной области файла` он меняет состояние, проверяемые данные или подготовку отчета.
1502	`html_escape(&sb, tool_version);`	Технически вызывает `html_escape`. Логически это действие нужно в функции `глобальной области файла`: вызывает функцию `html_escape` для выполнения части алгоритма.
1503	`sb_append(&sb,`	Технически вызывает `sb_append`. Логически это действие нужно в функции `глобальной области файла`: добавляет текст в динамический буфер.
1504	`"</footer></main><script>"`	Технически эта строка является частью C-конструкции `"</footer></main><script>"`. Логически она уточняет структуру блока функции `глобальной области файла` и влияет на то, какие данные будут обработаны.
1505	`"(()=>{const q=document.getElementById('function-filter');"`	Технически присваивает `"(()` значение `>{const q=document.getElementById('function-filter');"`. Логически это обновляет `"(()`: хранит промежуточное значение, которое дальше используется в текущей функции.
1506	`"const rows=[...document.querySelectorAll('#functions tbody tr')];"`	Технически присваивает `"const rows` значение `[...document.querySelectorAll('#functions tbody tr')];"`. Логически это обновляет `"const rows`: хранит промежуточное значение, которое дальше используется в текущей функции.
1507	`"const out=document.getElementById('function-count');"`	Технически присваивает `"const out` значение `document.getElementById('function-count');"`. Логически это обновляет `"const out`: хранит промежуточное значение, которое дальше используется в текущей функции.
1508	`"function run(){const needle=q.value.trim().toLowerCase();let shown=0;"`	Технически присваивает `"function run(){const needle` значение `q.value.trim().toLowerCase();let shown=0;"`. Логически это обновляет `"function run(){const needle`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1509	`"for(const row of rows){const visible=!needle||row.textContent.toLowerCase().includes(needle);"`	Технически присваивает `"for(const row of rows){const visible` значение `!needle||row.textContent.toLowerCase().includes(needle);"`. Логически это обновляет `"for(const row of rows){const visible`: хранит промежуточное значение, которое дальше используется в текущей функции.
1510	`"row.hidden=!visible;if(visible)shown++;}out.textContent=`${shown} из ${rows.length}`;}"`	Технически присваивает `"row.hidden` значение `!visible;if(visible)shown++;}out.textContent=${shown} из ${rows.length};}"`. Логически это обновляет `hidden`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
1511	`"q.addEventListener('input',run);run();})();"`	Технически вызывает addEventListener. Логически это действие нужно в функции глобальной области файла: вызывает функцию addEventListener для выполнения части алгоритма.
1512	`"</script></body></html>");`	Технически выполняет оператор "</script></body></html>");. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1514	`bool ok = sb_ok(&sb) && sb_ok(&anomalies);`	Технически присваивает bool ok значение sb_ok(&sb) && sb_ok(&anomalies). Логически это обновляет bool ok: хранит промежуточное значение, которое дальше используется в текущей функции.
1515	`if (ok) ok = file_write_atomic(path, sb.s, sb.len);`	Технически проверяет условие ok) ok = file_write_atomic(path, sb.s, sb.len; если оно истинно, выполняется защитная или аналитическая ветка ниже.
1516	`if (!ok) {`	Технически проверяет условие !ok; логически это защита от NULL-указателя и работы с неинициализированными данными.
1517	`sb_appendf(err, "%s: %s\n",`	Технически вызывает sb_appendf. Логически это действие нужно в функции глобальной области файла: добавляет форматированный текст в динамический буфер.
1518	`sb_ok(&sb) && sb_ok(&anomalies)`	Технически объявляет/начинает функцию sb_ok; она доступная другим модулям через заголовок или линковку. Логически функция проверяет, что StrBuf не перешел в состояние ошибки.
1519	`? "Failed to write HTML"`	Технически эта строка является частью С-конструкции ? "Failed to write HTML". Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1520	`: "Out of memory while building HTML report",`	Технически эта строка является частью С-конструкции : "Out of memory while building HTML report",. Логически она уточняет структуру блока функции глобальной области файла и влияет на то, какие данные будут обработаны.
1521	`path);`	Технически выполняет оператор path);. Логически в функции глобальной области файла он меняет состояние, проверяемые данные или подготовку отчета.
1522	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
1523	`sb_free(&anomalies);`	Технически вызывает sb_free. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
1524	`sb_free(&sb);`	Технически вызывает sb_free. Логически это действие нужно в функции глобальной области файла: освобождает память динамического буфера.
1525	`return ok;`	Технически возвращает выражение ok. Логически именно это значение становится результатом функции глобальной области файла для вызывающего кода.
1526	`}`	Технически завершает область видимости внутри глобальной области файла. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 1436. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

9. CLI и полный жизненный цикл

main.c соединяет все слои: читает аргументы, загружает бинарник, запускает PE/unwind/heuristics/dynamic, пишет отчеты и освобождает ресурсы.

Что сначала выучить в C и теории

- argc/argv, разбор опций и коды возврата
- инициализация, вызовы модулей и cleanup
- разница между ошибкой запуска и аналитическим finding
- как пользователь получает JSON/HTML/CSV из одной команды

Основная мысль шага

Перед тем как переписывать код, пойми роль файла в общей цепочке. ВКР не состоит из случайных функций: каждый модуль принимает данные от предыдущего слоя, проверяет свой контракт и передает наверх уже более осмысленную модель.

src/main.c

Назначение файла: точка входа CLI и связка всех модулей проекта.

Код, который пишем на этом шаге

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <errno.h>
#include <limits.h>

#include "util.h"
#include "pe.h"
#include "unwind.h"
#include "heuristics.h"
#include "report.h"
#include "dynamic.h"
#include "app_info.h"

#if defined(_WIN32)
#include <windows.h>
#endif

typedef struct Args {
    const char* input;
    bool dump;
    bool show_ops;
    bool heuristics;
    const char* csv_path;
    const char* ghidra_csv_path;
    const char* json_path;
    const char* html_path;
    int  dump_limit;        /* -1 => all */
    bool follow_chain;
    bool strict_abi;
    int  max_chain_depth;
    bool dynamic;
    uint32_t timeout_ms;
    uint32_t trace_steps;
    const char** child_argv;
    int child_argc;
} Args;

typedef enum ParseResult {
    PARSE_OK = 0,
    PARSE_HELP = 1,
    PARSE_VERSION = 2,
    PARSE_ERR = 3
} ParseResult;

static char* dup_cstr(const char* s) {
    if (!s) return NULL;
    size_t n = strlen(s);
    char* out = (char*)malloc(n + 1u);
    if (!out) return NULL;
    memcpy(out, s, n + 1u);
    return out;
}

static char* argv_text_to_utf8(const char* s) {
    if (!s) return NULL;
#if defined(_WIN32)
    int wide_len = MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0);
    if (wide_len <= 0) return dup_cstr(s);
    wchar_t* wide = (wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t));
    if (!wide) return NULL;
    if (MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL);
    if (utf8_len <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    char* out = (char*)malloc((size_t)utf8_len);
    if (!out) {
        free(wide);
```

```
        return NULL;
    }
    if (WideCharToMultiByte(CP_UTF8, 0, wide, -1, out, utf8_len, NULL, NULL) <= 0) {
        free(out);
        free(wide);
        return dup_cstr(s);
    }
    free(wide);
    return out;
#else
    return dup_cstr(s);
#endif
}

static void print_version(void) {
    printf("EH/Unwind Analyzer %s\n", app_version());
    printf("Report schema: %s\n", app_report_schema_version());
    printf("Build: %s, %s, %s\n", app_build_compiler(), app_build_platform(), app_build_mode());
}

static void usage(const char* argv0) {
    printf("EH/Unwind Analyzer %s (PE x64)\n", app_version());
    printf("Usage: %s <pefile> [options]\n\n", argv0);
    printf("Options:\n");
    printf("  --dump                Dump all RUNTIME_FUNCTION entries\n");
    printf("  --limit N             Limit dump to first N entries (default: all when --dump)\n");
    printf("  --ops                 With --dump: also print unwind ops list per function\n");
    printf("  --heuristics          Run EH/Unwind anomaly heuristics and print report\n");
    printf("  --csv <path>          Write analysis CSV (full fields)\n");
    printf("  --ghidra-csv <path>   Write CSV for Ghidra import (begin/end/name/comment)\n");
    printf("  --json <path>         Write JSON (functions + heuristic/dynamic assessment)\n");
    printf("  --html <path>         Write a self-contained interactive HTML report\n");
    printf("  --no-follow-chain     Do not follow UNW_FLAG_CHAININFO\n");
    printf("  --strict-abi          Flag formal Microsoft x64 unwind ABI violations\n");
    printf("  --max-chain-depth N   Max chain depth, 1..16 (default 8)\n");
    printf("  --dynamic             Run Windows-only dynamic exception validation (debugger)\n");
    printf("  --timeout-ms N        Dynamic mode: hard timeout, N>0 (default 15000)\n");
    printf("  --trace-steps N       Dynamic mode: single-step samples after handler hit (default 64, range 0..256)\n");
    printf("  --                    Separator: remaining args are passed to the target program\n");
    printf("  --version             Show version and build information\n");
    printf("  --help                Show this help\n");
}

static bool append_invocation_arg(StrBuf* out, const char* arg) {
    if (!out || !arg) return false;
    bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;
    if (!quote) return sb_append(out, arg);
    if (!sb_append(out, "\"")) return false;
    for (const char* p = arg; *p; p++) {
        if (*p == '"' || *p == '\\') {
            if (!sb_appendn(out, "\\", 1)) return false;
        }
        if (!sb_appendn(out, p, 1)) return false;
    }
    return sb_append(out, "\"");
}

static bool build_invocation(int argc, char** argv, StrBuf* out) {
    if (!out) return false;
    for (int i = 0; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            int redacted = argc - i - 1;
            if (i && !sb_appendn(out, " ", 1)) return false;
            if (!sb_appendf(out, "-- [%d target argument%s redacted]",
                            redacted, redacted == 1 ? "" : "s")) {
                return false;
            }
            break;
        }
        if (i && !sb_appendn(out, " ", 1)) return false;
        char* utf8_arg = argv_text_to_utf8(argv[i]);
        if (!utf8_arg) return false;
        bool ok = append_invocation_arg(out, utf8_arg);
        free(utf8_arg);
        if (!ok) return false;
    }
    return true;
}

static bool parse_long_range(const char* text, long min_value, long max_value, long* out) {
    if (!text || !*text || !out) return false;
    errno = 0;
    char* end = NULL;
    long value = strtol(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < min_value || value > max_value) return false;
    *out = value;
    return true;
}

static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value, uint32_t* out) {
    if (!text || !*text || !out || text[0] == '-') return false;
    errno = 0;
    char* end = NULL;
    unsigned long value = strtoul(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < (unsigned long)min_value || value > (unsigned long)max_value) return false;
    *out = (uint32_t)value;
    return true;
}

static ParseResult parse_args(int argc, char** argv, Args* a) {
    memset(a, 0, sizeof(*a));
    a->dump_limit = -1;
    a->follow_chain = true;
    a->max_chain_depth = 8;
    a->timeout_ms = 15000;
    a->trace_steps = 64;

    if (argc < 2) return PARSE_ERR;
    if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;
    if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;
    a->input = argv[1];

    for (int i = 2; i < argc; i++) {
```

```
        const char* s = argv[i];
        if (strcmp(s, "--") == 0) {
            a->child_argc = argc - (i + 1);
            a->child_argv = (const char**)&argv[i + 1];
            break;
        }
        if (strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0) return PARSE_HELP;
        else if (strcmp(s, "--version") == 0) return PARSE_VERSION;
        else if (strcmp(s, "--dump") == 0) a->dump = true;
        else if (strcmp(s, "--ops") == 0) a->show_ops = true;
        else if (strcmp(s, "--heuristics") == 0) a->heuristics = true;
        else if (strcmp(s, "--dynamic") == 0) a->dynamic = true;
        else if (strcmp(s, "--no-follow-chain") == 0) a->follow_chain = false;
        else if (strcmp(s, "--strict-abi") == 0) a->strict_abi = true;
        else if (strcmp(s, "--limit") == 0 && i + 1 < argc) {
            long value = 0;
            if (!parse_long_range(argv[++i], 0, INT_MAX, &value)) {
                fprintf(stderr, "Invalid value for --limit: %s (expected 0..%d)\n", argv[i], INT_MAX);
                return PARSE_ERR;
            }
            a->dump_limit = (int)value;
        }
        else if (strcmp(s, "--max-chain-depth") == 0 && i + 1 < argc) {
            long value = 0;
            if (!parse_long_range(argv[++i], 1, 16, &value)) {
                fprintf(stderr, "Invalid value for --max-chain-depth: %s (expected 1..16)\n", argv[i]);
                return PARSE_ERR;
            }
            a->max_chain_depth = (int)value;
        }
        else if (strcmp(s, "--timeout-ms") == 0 && i + 1 < argc) {
            if (!parse_u32_range(argv[++i], 1, UINT32_MAX, &a->timeout_ms)) {
                fprintf(stderr, "Invalid value for --timeout-ms: %s (expected 1..%u)\n",
                        argv[i], (unsigned)UINT32_MAX);
                return PARSE_ERR;
            }
        }
        else if (strcmp(s, "--trace-steps") == 0 && i + 1 < argc) {
            if (!parse_u32_range(argv[++i], 0, 256, &a->trace_steps)) {
                fprintf(stderr, "Invalid value for --trace-steps: %s (expected 0..256)\n", argv[i]);
                return PARSE_ERR;
            }
        }
        else if (strcmp(s, "--csv") == 0 && i + 1 < argc) { a->csv_path = argv[++i]; }
        else if (strcmp(s, "--ghidra-csv") == 0 && i + 1 < argc) { a->ghidra_csv_path = argv[++i]; }
        else if (strcmp(s, "--json") == 0 && i + 1 < argc) { a->json_path = argv[++i]; }
        else if (strcmp(s, "--html") == 0 && i + 1 < argc) { a->html_path = argv[++i]; }
        else {
            fprintf(stderr, "Unknown/invalid option: %s\n", s);
            return PARSE_ERR;
        }
    }
    return PARSE_OK;
}

static void print_one(size_t idx, const RuntimeFunction* r, const UnwindSummary* u, bool ok) {
    uint32_t size = (r->end_rva >= r->begin_rva) ? (r->end_rva - r->begin_rva) : 0;
    char fbuf[64];
    unwind_flags_to_string(u->flags, fbuf);

    printf("[%zu] 0x%08X..0x%08X  sz=%-6u  unwind=0x%08X  ",
           idx, (unsigned)r->begin_rva, (unsigned)r->end_rva, (unsigned)size, (unsigned)r->unwind_rva);

    if (!ok) {
        printf("UNWIND: <parse failed%s%s>\n",
               u->parse_error[0] ? ": " : "",
               u->parse_error);
        return;
    }

    printf("v%u flags=%s prolog=%u stack=%uB",
           (unsigned)u->version, fbuf, (unsigned)u->prolog_size, (unsigned)u->fixed_stack_bytes);

    if (u->frame_reg) printf(" fp=%s+%u", unwind_reg_name(u->frame_reg), (unsigned)(u->frame_off * 16u));
    if (u->has_handler) printf(" handler=0x%08X", (unsigned)u->handler_rva);
    if (u->has_chain) printf(" chain_u=0x%08X", (unsigned)u->chain.unwind_rva);
    if (u->anomalies) printf(" anom=0x%08X", (unsigned)u->anomalies);
    printf("\n");
}

int main(int argc, char** argv) {
    Args a;
    int exit_code = 0;
    ParseResult pres = parse_args(argc, argv, &a);
    if (pres != PARSE_OK) {
        if (pres == PARSE_VERSION) {
            print_version();
            return 0;
        }
        usage(argv[0]);
        return (pres == PARSE_HELP) ? 0 : 2;
    }

    uint8_t* owned = NULL;
    Buf b;
    if (!buf_init_from_file(&b, a.input, &owned)) {
        fprintf(stderr, "Failed to read file: %s\n", a.input);
        return 1;
    }

    StrBuf err;
    sb_init(&err, 1024);

    PeImage img;
    if (!pe_parse(&img, &b, &err)) {
        fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");
        sb_free(&err);
        buf_free_owned(owned);
        return 1;
    }

    RuntimeFunction* rf = NULL;
    size_t rf_count = 0;
    StrBuf err2;
    sb_init(&err2, 1024);
```

```
305 |
306 |     RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(
307 |         &b, &img, &rf, &rf_count, &err2);
308 |     if (rf_read != RUNTIME_FUNCTION_READ_OK) {
309 |         fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");
310 |         if (rf_read == RUNTIME_FUNCTION_READ_OOM) {
311 |             pe_free(&img);
312 |             sb_free(&err);
313 |             sb_free(&err2);
314 |             buf_free_owned(owned);
315 |             return 1;
316 |         }
317 |         /*
318 |          * Keep the established report field/finding code for compatibility.
319 |          * It now covers every structurally unreadable exception directory.
320 |          */
321 |         img.exception_directory_oob = true;
322 |         fprintf(stderr, "Continuing with a partial structural report.\n");
323 |         exit_code = 1;
324 |     }
325 |
326 |     report_print_brief(a.input, &img, rf_count);
327 |
328 |     UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1, sizeof(UnwindSummary));
329 |     bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool));
330 |     if (!us || !us_ok) {
331 |         fprintf(stderr, "Out of memory.\n");
332 |         free(us);
333 |         free(us_ok);
334 |         free(rf);
335 |         pe_free(&img);
336 |         sb_free(&err);
337 |         sb_free(&err2);
338 |         buf_free_owned(owned);
339 |         return 1;
340 |     }
341 |
342 |     UnwindParseOpts uopts = {
343 |         .max_chain_depth = a.max_chain_depth,
344 |         .follow_chain = a.follow_chain,
345 |         .strict_abi = a.strict_abi
346 |     };
347 |
348 |     StrBuf ops;
349 |     sb_init(&ops, 1024);
350 |
351 |     for (size_t i = 0; i < rf_count; i++) {
352 |         StrBuf perr;
353 |         sb_init(&perr, 256);
354 |         if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }
355 |         bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops : NULL, &perr);
356 |         us_ok[i] = ok;
357 |         if (a.dump && a.show_ops && ok) {
358 |             /* ops will be printed in dump loop below */
359 |         }
360 |         sb_free(&perr);
361 |     }
362 |
363 |     if (a.dump) {
364 |         size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;
365 |         if (limit > rf_count) limit = rf_count;
366 |         printf("\n[Dump] Showing %zu / %zu entries\n", limit, rf_count);
367 |         for (size_t i = 0; i < limit; i++) {
368 |             print_one(i, &rf[i], &us[i], us_ok[i]);
369 |             if (a.show_ops && us_ok[i]) {
370 |                 StrBuf ops2;
371 |                 StrBuf perr;
372 |                 sb_init(&ops2, 1024);
373 |                 sb_init(&perr, 256);
374 |                 (void)unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], &ops2, &perr);
375 |                 if (ops2.s && ops2.len) printf("%s", ops2.s);
376 |                 sb_free(&ops2);
377 |                 sb_free(&perr);
378 |             }
379 |         }
380 |     }
381 |
382 |     HeuristicReport hr;
383 |     HeuristicReport* hr_ptr = NULL;
384 |     DynamicReport dr;
385 |     memset(&dr, 0, sizeof(dr));
386 |     DynamicReport* dr_ptr = NULL;
387 |     bool dr_initialized = false;
388 |     if (a.heuristics) {
389 |         StrBuf perr;
390 |         sb_init(&perr, 512);
391 |         if (heuristics_analyze(&img, rf, us, us_ok, rf_count, &hr, &perr)) {
392 |             report_print_heuristics(&hr);
393 |             hr_ptr = &hr;
394 |         } else {
395 |             fprintf(stderr, "Heuristic analysis failed:\n%s", perr.s ? perr.s : "");
396 |             exit_code = 1;
397 |         }
398 |         sb_free(&perr);
399 |     }
400 |
401 |     if (a.dynamic) {
402 |         StrBuf derr;
403 |         sb_init(&derr, 1024);
404 |         printf("\n[Dynamic] Starting debugger evidence collection (Windows only)...\n");
405 |         bool ran = dynamic_collect_windows(a.input, a.child_argv, a.child_argc,
406 |                                            &img, rf, us, us_ok, rf_count,
407 |                                            a.timeout_ms, a.trace_steps,
408 |                                            &dr, &derr);
409 |         dr_initialized = true;
410 |         dr_ptr = &dr;
411 |         if (!ran) {
412 |             fprintf(stderr, "[Dynamic] Failed: %s", derr.s ? derr.s : "");
413 |             exit_code = 1;
414 |         } else {
415 |             printf("[Dynamic] Done. exceptions_seen=%u unwind_handler_hits=%u veh_hits=%u dyn_ftable_calls=%u\n",
416 |                    (unsigned)dr.exceptions_seen,
417 |                    (unsigned)dr.unwind_handler_hits,
418 |                    (unsigned)dr.veh_hits,
419 |                    (unsigned)dr.rtl_add_function_table_calls);
```

```
            if (dr.details.s && dr.details.len) {
                printf("%s\n", dr.details.s);
            }
        }
        sb_free(&derr);
    }

    if (a.csv_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (!report_write_csv(a.csv_path, &img, rf, us, us_ok, rf_count, false, &e)) {
            fprintf(stderr, "CSV write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else {
            printf("\nWrote CSV: %s\n", a.csv_path);
        }
        sb_free(&e);
    }
    if (a.ghidra_csv_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (!report_write_csv(a.ghidra_csv_path, &img, rf, us, us_ok, rf_count, true, &e)) {
            fprintf(stderr, "Ghidra CSV write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else {
            printf("\nWrote Ghidra CSV: %s\n", a.ghidra_csv_path);
        }
        sb_free(&e);
    }

    StrBuf invocation = {0};
    char timestamp[32] = "";
    char input_sha256[65] = "";
    char* input_display = NULL;
    ReportMetadata metadata;
    memset(&metadata, 0, sizeof(metadata));
    ReportMetadata* metadata_ptr = NULL;
    if (a.json_path || a.html_path) {
        input_display = argv_text_to_utf8(a.input);
        bool metadata_ok = sb_init(&invocation, 256) &&
                           input_display &&
                           build_invocation(argc, argv, &invocation) &&
                           app_utc_timestamp(timestamp, sizeof(timestamp)) &&
                           sha256_hex(b.data, b.size, input_sha256);
        if (!metadata_ok) {
            fprintf(stderr, "Failed to prepare report metadata.\n");
            exit_code = 1;
        } else {
            metadata.tool_version = app_version();
            metadata.report_schema_version = app_report_schema_version();
            metadata.generated_at_utc = timestamp;
            metadata.input_sha256 = input_sha256;
            metadata.input_size = (uint64_t)b.size;
            metadata.command_line = invocation.s ? invocation.s : "";
            metadata.build_compiler = app_build_compiler();
            metadata.build_platform = app_build_platform();
            metadata.build_mode = app_build_mode();
            metadata.strict_abi = a.strict_abi;
            metadata.follow_chain = a.follow_chain;
            metadata.max_chain_depth = (uint32_t)a.max_chain_depth;
            metadata_ptr = &metadata;
        }
    }

    if (a.json_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (metadata_ptr &&
            !report_write_json_ex(a.json_path, input_display ? input_display : a.input,
                                  &img, rf, us, us_ok, rf_count,
                                  hr_ptr, dr_ptr, metadata_ptr, &e)) {
            fprintf(stderr, "JSON write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else if (metadata_ptr) {
            printf("\nWrote JSON: %s\n", a.json_path);
        }
        sb_free(&e);
    }
    if (a.html_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (metadata_ptr &&
            !report_write_html(a.html_path, input_display ? input_display : a.input,
                               &img, rf, us, us_ok, rf_count,
                               hr_ptr, dr_ptr, metadata_ptr, &e)) {
            fprintf(stderr, "HTML write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else if (metadata_ptr) {
            printf("\nWrote HTML: %s\n", a.html_path);
        }
        sb_free(&e);
    }
    sb_free(&invocation);
    free(input_display);

    if (a.heuristics && hr_ptr) heuristics_free(&hr);
    if (dr_initialized) dynamic_report_free(&dr);

    sb_free(&ops);
    free(us_ok);
    free(us);
    free(rf);
    pe_free(&img);
    sb_free(&err);
    sb_free(&err2);
    buf_free_owned(owned);
    return exit_code;
}
```

Построчный разбор без заглушек

Стр.	Код	Нормальное объяснение
1	`#include <stdio.h>`	Технически подключает `stdio.h`. Логически это дает модулю доступ к: файлы и консольный вывод.
2	`#include <stdlib.h>`	Технически подключает `stdlib.h`. Логически это дает модулю доступ к: malloc/free, exit, strtol и общие функции C runtime.
3	`#include <string.h>`	Технически подключает `string.h`. Логически это дает модулю доступ к: strlen, memcpy, memset и сравнение строк.
4	`#include <stdbool.h>`	Технически подключает `stdbool.h`. Логически это дает модулю доступ к: тип bool, чтобы функции возвращали успех/ошибку явно.
5	`#include <errno.h>`	Технически подключает `errno.h`. Логически это дает модулю доступ к: код ошибки C runtime после системных вызовов.
6	`#include <limits.h>`	Технически подключает `limits.h`. Логически это дает модулю доступ к: объявления, которые нужны этому модулю.
8	`#include "util.h"`	Технически подключает `util.h`. Логически это дает модулю доступ к: общие типы и функции безопасного чтения, строкового буфера, атомарной записи и SHA-256.
9	`#include "pe.h"`	Технически подключает `pe.h`. Логически это дает модулю доступ к: структуры и функции PE-парсера.
10	`#include "unwind.h"`	Технически подключает `unwind.h`. Логически это дает модулю доступ к: модель RUNTIME_FUNCTION, UNWIND_INFO, unwind-кодов и аномалий.
11	`#include "heuristics.h"`	Технически подключает `heuristics.h`. Логически это дает модулю доступ к: модель finding, mitigation, score, confidence и verdict.
12	`#include "report.h"`	Технически подключает `report.h`. Логически это дает модулю доступ к: интерфейс печати brief/CSV/JSON/HTML отчетов.
13	`#include "dynamic.h"`	Технически подключает `dynamic.h`. Логически это дает модулю доступ к: модель динамического отчета и настройки runtime-валидации.
14	`#include "app_info.h"`	Технически подключает `app_info.h`. Логически это дает модулю доступ к: публичный интерфейс метаданных версии и сборки.
16	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
17	`#include <windows.h>`	Технически подключает `windows.h`. Логически это дает модулю доступ к: WinAPI для динамической проверки под Windows.
18	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
20	`typedef struct Args {`	Технически начинает описание структуры `Args`. Логически структура собирает связанные поля в одну модель данных проекта.
21	`const char* input;`	Технически объявляет поле `input` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
22	`bool dump;`	Технически объявляет поле `dump` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
23	`bool show_ops;`	Технически объявляет поле `show_ops` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
24	`bool heuristics;`	Технически объявляет поле `heuristics` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
25	`const char* csv_path;`	Технически объявляет поле `csv_path` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
26	`const char* ghidra_csv_path;`	Технически объявляет поле `ghidra_csv_path` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
27	`const char* json_path;`	Технически объявляет поле `json_path` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
28	`const char* html_path;`	Технически объявляет поле `html_path` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
29	`int dump_limit; /* -1 => all */`	Технически объявляет поле `dump_limit` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
30	`bool follow_chain;`	Технически объявляет поле `follow_chain` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
31	`bool strict_abi;`	Технически объявляет поле `strict_abi` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
32	`int max_chain_depth;`	Технически объявляет поле `max_chain_depth` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
33	`bool dynamic;`	Технически объявляет поле `dynamic` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
34	`uint32_t timeout_ms;`	Технически объявляет поле `timeout_ms` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
35	`uint32_t trace_steps;`	Технически объявляет поле `trace_steps` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
36	`const char** child_argv;`	Технически объявляет поле `child_argv` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
37	`  int child_argc;`	Технически объявляет поле `child_argc` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
38	`} Args;`	Технически завершает typedef и дает типу имя `Args`. Логически дальше код может объявлять переменные этого типа без `struct`.
40	`typedef enum ParseResult {`	Технически начинает перечисление enum. Логически проект получает именованные константы вместо неочевидных чисел.
41	`  PARSE_OK = 0,`	Технически присваивает `PARSE_OK` значение `0,`. Логически это обновляет `PARSE_OK`: хранит промежуточное значение, которое дальше используется в текущей функции.
42	`  PARSE_HELP = 1,`	Технически присваивает `PARSE_HELP` значение `1,`. Логически это обновляет `PARSE_HELP`: хранит промежуточное значение, которое дальше используется в текущей функции.
43	`  PARSE_VERSION = 2,`	Технически присваивает `PARSE_VERSION` значение `2,`. Логически это обновляет `PARSE_VERSION`: хранит промежуточное значение, которое дальше используется в текущей функции.
44	`  PARSE_ERR = 3`	Технически присваивает `PARSE_ERR` значение `3`. Логически это обновляет `PARSE_ERR`: хранит промежуточное значение, которое дальше используется в текущей функции.
45	`} ParseResult;`	Технически завершает typedef и дает типу имя `ParseResult`. Логически дальше код может объявлять переменные этого типа без `struct`.
47	`static char* dup_cstr(const char* s) {`	Технически объявляет/начинает функцию `dup_cstr`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
48	`  if (!s) return NULL;`	Технически проверяет условие `!s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
49	`  size_t n = strlen(s);`	Технически присваивает `size_t n` значение `strlen(s)`. Логически это обновляет `size_t n`: хранит размер или количество, поэтому используется в проверках границ.
50	`  char* out = (char*)malloc(n + 1u);`	Технически присваивает `char* out` значение `(char*)malloc(n + 1u)`. Логически это обновляет `char* out`: хранит промежуточное значение, которое дальше используется в текущей функции.
51	`  if (!out) return NULL;`	Технически проверяет условие `!out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
52	`  memcpy(out, s, n + 1u);`	Технически вызывает `memcpy`. Логически это действие нужно в функции `dup_cstr`: копирует байты без интерпретации; в бинарном парсере это безопаснее ручного доступа к unaligned struct.
53	`  return out;`	Технически возвращает выражение `out`. Логически именно это значение становится результатом функции `dup_cstr` для вызывающего кода.
54	`}`	Технически завершает область видимости внутри `dup_cstr`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
56	`static char* argv_text_to_utf8(const char* s) {`	Технически объявляет/начинает функцию `argv_text_to_utf8`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
57	`  if (!s) return NULL;`	Технически проверяет условие `!s`; логически это защита от NULL-указателя и работы с неинициализированными данными.
58	`#if defined(_WIN32)`	Технически включает код только при условии `#if defined(_WIN32)`. Логически так проект выбирает платформенную или компиляторную реализацию на этапе сборки.
59	`  int wide_len = MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0);`	Технически присваивает `int wide_len` значение `MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0)`. Логически это обновляет `int wide_len`: хранит размер или количество, поэтому используется в проверках границ.
60	`  if (wide_len <= 0) return dup_cstr(s);`	Технически проверяет условие `wide_len <= 0) return dup_cstr(s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
61	`wchar_t* wide = (wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t));`	Технически присваивает `wchar_t* wide` значение `(wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t))`. Логически это обновляет `wchar_t* wide`: хранит промежуточное значение, которое дальше используется в текущей функции.
62	`  if (!wide) return NULL;`	Технически проверяет условие `!wide`; логически это защита от NULL-указателя и работы с неинициализированными данными.
63	`  if (MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0) {`	Технически проверяет условие `MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
64	`  free(wide);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.
65	`  return dup_cstr(s);`	Технически объявляет прототип функции `dup_cstr` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
66	`  }`	Технически завершает область видимости внутри `argv_text_to_utf8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
67	`int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL);`	Технически присваивает `int utf8_len` значение `WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL)`. Логически это обновляет `int utf8_len`: хранит размер или количество, поэтому используется в проверках границ.
68	`  if (utf8_len <= 0) {`	Технически проверяет условие `utf8_len <= 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
69	`  free(wide);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.

Стр.	Код	Нормальное объяснение
70	`return dup_cstr(s);`	Технически объявляет прототип функции `dup_cstr` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
71	`}`	Технически завершает область видимости внутри `argv_text_to_utf8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
72	`char* out = (char*)malloc((size_t)utf8_len);`	Технически присваивает `char* out` значение `(char*)malloc((size_t)utf8_len)`. Логически это обновляет `char* out`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
73	`if (!out) {`	Технически проверяет условие `!out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
74	`free(wide);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.
75	`return NULL;`	Технически возвращает выражение `NULL`. Логически именно это значение становится результатом функции `argv_text_to_utf8` для вызывающего кода.
76	`}`	Технически завершает область видимости внутри `argv_text_to_utf8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
77	`if (WideCharToMultiByte(CP_UTF8, 0, wide, -1, out, utf8_len, NULL, NULL) <= 0) {`	Технически проверяет условие `WideCharToMultiByte(CP_UTF8, 0, wide, -1, out, utf8_len, NULL, NULL) <= 0`; логически это защита от NULL-указателя и работы с неинициализированными данными.
78	`free(out);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.
79	`free(wide);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.
80	`return dup_cstr(s);`	Технически объявляет прототип функции `dup_cstr` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
81	`}`	Технически завершает область видимости внутри `argv_text_to_utf8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
82	`free(wide);`	Технически вызывает `free`. Логически это действие нужно в функции `argv_text_to_utf8`: освобождает память, чтобы в C не было утечки.
83	`return out;`	Технически возвращает выражение `out`. Логически именно это значение становится результатом функции `argv_text_to_utf8` для вызывающего кода.
84	`#else`	Технически открывает альтернативную ветку препроцессора. Логически она задает поведение для другой платформы, компилятора или режима сборки.
85	`return dup_cstr(s);`	Технически объявляет прототип функции `dup_cstr` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и выполняет отдельную часть алгоритма проекта.
86	`#endif`	Технически завершает условный блок препроцессора. Логически дальнейший код снова компилируется без этого условия.
87	`}`	Технически завершает область видимости внутри `argv_text_to_utf8`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
89	`static void print_version(void) {`	Технически объявляет/начинает функцию `print_version`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
90	`printf("EH/Unwind Analyzer %s\n", app_version());`	Технически вызывает `printf`. Логически это действие нужно в функции `print_version`: выводит информацию пользователю в консоль.
91	`printf("Report schema: %s\n", app_report_schema_version());`	Технически вызывает `printf`. Логически это действие нужно в функции `print_version`: выводит информацию пользователю в консоль.
92	`printf("Build: %s, %s, %s\n", app_build_compiler(), app_build_platform(), app_build_mode());`	Технически вызывает `printf`. Логически это действие нужно в функции `print_version`: выводит информацию пользователю в консоль.
93	`}`	Технически завершает область видимости внутри `print_version`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
95	`static void usage(const char* argv0) {`	Технически объявляет/начинает функцию `usage`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
96	`printf("EH/Unwind Analyzer %s (PE x64)\n", app_version());`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
97	`printf("Usage: %s <pefile> [options]\n\n", argv0);`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
98	`printf("Options:\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
99	`printf(" --dump Dump all RUNTIME_FUNCTION entries\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.

Стр.	Код	Нормальное объяснение
100	`printf(" --limit N Limit dump to first N entries (default: all when --dump)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
101	`printf(" --ops With --dump: also print unwind ops list per function\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
102	`printf(" --heuristics Run EH/Unwind anomaly heuristics and print report\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
103	`printf(" --csv <path> Write analysis CSV (full fields)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
104	`printf(" --ghidra-csv <path> Write CSV for Ghidra import (begin/end/name/comment)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
105	`printf(" --json <path> Write JSON (functions + heuristic/dynamic assessment)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
106	`printf(" --html <path> Write a self-contained interactive HTML report\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
107	`printf(" --no-follow-chain Do not follow UNW_FLAG_CHAININFO\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.

Стр.	Код	Нормальное объяснение
108	`printf(" --strict-abi Flag formal Microsoft x64 unwind ABI violations\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
109	`printf(" --max-chain-depth N Max chain depth, 1..16 (default 8)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
110	`printf(" --dynamic Run Windows-only dynamic exception validation (debugger)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
111	`printf(" --timeout-ms N Dynamic mode: hard timeout, N>0 (default 15000)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
112	`printf(" --trace-steps N Dynamic mode: single-step samples after handler hit (default 64, range 0..256)\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
113	`printf(" -- Separator: remaining args are passed to the target program\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
114	`printf(" --version Show version and build information\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
115	`printf(" --help Show this help\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `usage`: выводит информацию пользователю в консоль.
116	`}`	Технически завершает область видимости внутри `usage`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
118	`static bool append_invocation_arg(StrBuf* out, const char* arg) {`	Технически объявляет/начинает функцию `append_invocation_arg`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
119	`if (!out || !arg) return false;`	Технически проверяет условие `!out || !arg`; логически это защита от NULL-указателя и работы с неинициализированными данными.
120	`bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;`	Технически присваивает `bool quote` значение `(*arg == '\0') || strpbrk(arg, " \t\"") != NULL`. Логически это обновляет `bool quote`: хранит промежуточное значение, которое дальше используется в текущей функции.
121	`if (!quote) return sb_append(out, arg);`	Технически проверяет условие `!quote) return sb_append(out, arg`; логически это защита от NULL-указателя и работы с неинициализированными данными.
122	`if (!sb_append(out, "\"")) return false;`	Технически проверяет условие `!sb_append(out, "\"")`; логически это защита от NULL-указателя и работы с неинициализированными данными.
123	`for (const char* p = arg; *p; p++) {`	Технически начинает цикл `const char* p = arg; *p; p++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
124	`if (*p == '"' || *p == '\\') {`	Технически проверяет условие `*p == '"' || *p == '\\'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
125	`if (!sb_appendn(out, "\\", 1)) return false;`	Технически проверяет условие `!sb_appendn(out, "\\", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
126	`}`	Технически завершает область видимости внутри `append_invocation_arg`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
127	`if (!sb_appendn(out, p, 1)) return false;`	Технически проверяет условие `!sb_appendn(out, p, 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
128	`}`	Технически завершает область видимости внутри `append_invocation_arg`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
129	`return sb_append(out, "\"");`	Технически объявляет прототип функции `sb_append` без тела. Логически заголовок обещает другим .c-файлам, что такая функция существует и добавляет C-строку в StrBuf.
130	`}`	Технически завершает область видимости внутри `append_invocation_arg`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
132	`static bool build_invocation(int argc, char** argv, StrBuf* out) {`	Технически объявляет/начинает функцию `build_invocation`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
133	`if (!out) return false;`	Технически проверяет условие `!out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
134	`for (int i = 0; i < argc; i++) {`	Технически начинает цикл `int i = 0; i < argc; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
135	`if (strcmp(argv[i], "--") == 0) {`	Технически проверяет условие `strcmp(argv[i], "--") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
136	`int redacted = argc - i - 1;`	Технически объявляет `redacted` в функции `build_invocation` и сразу задает начальное значение `argc - i - 1`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
137	`if (i && !sb_appendn(out, " ", 1)) return false;`	Технически проверяет условие `i && !sb_appendn(out, " ", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
138	`if (!sb_appendf(out, "-- [%d target argument%s redacted]",`	Технически проверяет условие `if (!sb_appendf(out, "-- [%d target argument%s redacted]",`; логически это защита от NULL-указателя и работы с неинициализированными данными.
139	`redacted, redacted == 1 ? "" : "s")) {`	Технически эта строка является частью C-конструкции `redacted, redacted == 1 ? "" : "s")) {`. Логически она уточняет структуру блока функции `build_invocation` и влияет на то, какие данные будут обработаны.
140	`return false;`	Технически возвращает `false` из функции `build_invocation`. Логически вызывающий код узнает, что операция не удалась и нельзя доверять частичному результату.
141	`}`	Технически завершает область видимости внутри `build_invocation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
142	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.

Стр.	Код	Нормальное объяснение
143	`}`	Технически завершает область видимости внутри `build_invocation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
144	`if (i && !sb_appendn(out, " ", 1)) return false;`	Технически проверяет условие `i && !sb_appendn(out, " ", 1)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
145	`char* utf8_arg = argv_text_to_utf8(argv[i]);`	Технически присваивает `char* utf8_arg` значение `argv_text_to_utf8(argv[i])`. Логически это обновляет `char* utf8_arg`: хранит промежуточное значение, которое дальше используется в текущей функции.
146	`if (!utf8_arg) return false;`	Технически проверяет условие `!utf8_arg`; логически это защита от NULL-указателя и работы с неинициализированными данными.
147	`bool ok = append_invocation_arg(out, utf8_arg);`	Технически присваивает `bool ok` значение `append_invocation_arg(out, utf8_arg)`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
148	`free(utf8_arg);`	Технически вызывает `free`. Логически это действие нужно в функции `build_invocation`: освобождает память, чтобы в C не было утечки.
149	`if (!ok) return false;`	Технически проверяет условие `!ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.
150	`}`	Технически завершает область видимости внутри `build_invocation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
151	`return true;`	Технически возвращает `true` из функции `build_invocation`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
152	`}`	Технически завершает область видимости внутри `build_invocation`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
154	`static bool parse_long_range(const char* text, long min_value, long max_value, long* out) {`	Технически объявляет/начинает функцию `parse_long_range`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
155	`if (!text || !*text || !out) return false;`	Технически проверяет условие `!text || !*text || !out`; логически это защита от NULL-указателя и работы с неинициализированными данными.
156	`errno = 0;`	Технически присваивает `errno` значение `0`. Логически это обновляет `errno`: хранит промежуточное значение, которое дальше используется в текущей функции.
157	`char* end = NULL;`	Технически объявляет `end` в функции `parse_long_range` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
158	`long value = strtol(text, &end, 10);`	Технически присваивает `long value` значение `strtol(text, &end, 10)`. Логически это обновляет `long value`: хранит промежуточное значение, которое дальше используется в текущей функции.
159	`if (errno == ERANGE || end == text || *end != '\0') return false;`	Технически проверяет условие `errno == ERANGE || end == text || *end != '\0'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
160	`if (value < min_value || value > max_value) return false;`	Технически проверяет условие `value < min_value || value > max_value`; если оно истинно, выполняется защитная или аналитическая ветка ниже.

Стр.	Код	Нормальное объяснение
161	`*out = value;`	Технически это комментарий исходника. Логически он объясняет разработчику: out = value;.
162	`return true;`	Технически возвращает `true` из функции `parse_long_range`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
163	`}`	Технически завершает область видимости внутри `parse_long_range`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
165	`static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value, uint32_t* out) {`	Технически объявляет/начинает функцию `parse_u32_range`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
166	`if (!text || !*text || !out || text[0] == '-') return false;`	Технически проверяет условие `!text || !*text || !out || text[0] == '-'`; логически это защита от NULL-указателя и работы с неинициализированными данными.
167	`errno = 0;`	Технически присваивает `errno` значение `0`. Логически это обновляет `errno`: хранит промежуточное значение, которое дальше используется в текущей функции.
168	`char* end = NULL;`	Технически объявляет `end` в функции `parse_u32_range` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
169	`unsigned long value = strtoul(text, &end, 10);`	Технически присваивает `unsigned long value` значение `strtoul(text, &end, 10)`. Логически это обновляет `unsigned long value`: хранит промежуточное значение, которое дальше используется в текущей функции.
170	`if (errno == ERANGE || end == text || *end != '\0') return false;`	Технически проверяет условие `errno == ERANGE || end == text || *end != '\0'`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
171	`if (value < (unsigned long)min_value || value > (unsigned long)max_value) return false;`	Технически проверяет условие `value < (unsigned long)min_value || value > (unsigned long)max_value`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
172	`*out = (uint32_t)value;`	Технически это комментарий исходника. Логически он объясняет разработчику: out = (uint32_t)value;.
173	`return true;`	Технически возвращает `true` из функции `parse_u32_range`. Логически функция сообщает, что данные проверены и результат можно использовать дальше.
174	`}`	Технически завершает область видимости внутри `parse_u32_range`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
176	`static ParseResult parse_args(int argc, char** argv, Args* a) {`	Технически объявляет/начинает функцию `parse_args`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
177	`memset(a, 0, sizeof(*a));`	Технически вызывает `memset`. Логически это действие нужно в функции `parse_args`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
178	`a->dump_limit = -1;`	Технически присваивает `a->dump_limit` значение `-1`. Логически это обновляет `dump_limit`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
179	`a->follow_chain = true;`	Технически присваивает `a->follow_chain` значение `true`. Логически это обновляет `follow_chain`: хранит промежуточное значение, которое дальше используется в текущей функции.
180	`a->max_chain_depth = 8;`	Технически присваивает `a->max_chain_depth` значение `8`. Логически это обновляет `max_chain_depth`: хранит промежуточное значение, которое дальше используется в текущей функции.
181	`a->timeout_ms = 15000;`	Технически присваивает `a->timeout_ms` значение `15000`. Логически это обновляет `timeout_ms`: хранит промежуточное значение, которое дальше используется в текущей функции.
182	`a->trace_steps = 64;`	Технически присваивает `a->trace_steps` значение `64`. Логически это обновляет `trace_steps`: хранит промежуточное значение, которое дальше используется в текущей функции.
184	`if (argc < 2) return PARSE_ERR;`	Технически проверяет условие `argc < 2`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
185	`if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;`	Технически проверяет условие `strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
186	`if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;`	Технически проверяет условие `strcmp(argv[1], "--version") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
187	`a->input = argv[1];`	Технически присваивает `a->input` значение `argv[1]`. Логически это обновляет `input`: хранит промежуточное значение, которое дальше используется в текущей функции.
189	`for (int i = 2; i < argc; i++) {`	Технически начинает цикл `int i = 2; i < argc; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
190	`const char* s = argv[i];`	Технически объявляет `s` в функции `parse_args` и сразу задает начальное значение `argv[i]`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
191	`if (strcmp(s, "--") == 0) {`	Технически проверяет условие `strcmp(s, "--") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
192	`a->child_argc = argc - (i + 1);`	Технически присваивает `a->child_argc` значение `argc - (i + 1)`. Логически это обновляет `child_argc`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
193	`a->child_argv = (const char**)&argv[i + 1];`	Технически присваивает `a->child_argv` значение `(const char**)&argv[i + 1]`. Логически это обновляет `child_argv`: хранит промежуточное значение, которое дальше используется в текущей функции.
194	`break;`	Технически завершает текущий `case` или цикл. Логически это не дает выполнению провалиться в следующую ветку и смешать разные случаи.
195	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
196	`if (strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0) return PARSE_HELP;`	Технически проверяет условие `strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
197	`else if (strcmp(s, "--version") == 0) return PARSE_VERSION;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--version") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
198	`else if (strcmp(s, "--dump") == 0) a->dump = true;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--dump") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
199	`else if (strcmp(s, "--ops") == 0) a->show_ops = true;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--ops") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
200	`else if (strcmp(s, "--heuristics") == 0) a->heuristics = true;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--heuristics") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
201	`else if (strcmp(s, "--dynamic") == 0) a->dynamic = true;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--dynamic") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
202	`else if (strcmp(s, "--no-follow-chain") == 0) a->follow_chain = false;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--no-follow-chain") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
203	`else if (strcmp(s, "--strict-abi") == 0) a->strict_abi = true;`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--strict-abi") == 0`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
204	`else if (strcmp(s, "--limit") == 0 && i + 1 < argc) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
205	`long value = 0;`	Технически объявляет `value` в функции `parse_args` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
206	`if (!parse_long_range(argv[++i], 0, INT_MAX, &value)) {`	Технически проверяет условие `!parse_long_range(argv[++i], 0, INT_MAX, &value)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
207	`fprintf(stderr, "Invalid value for --limit: %s (expected 0..%d)\n", argv[i], INT_MAX);`	Технически вызывает `fprintf`. Логически это действие нужно в функции `parse_args`: печатает сообщение в указанный поток, часто stderr.
208	`return PARSE_ERR;`	Технически возвращает выражение `PARSE_ERR`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
209	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
210	`a->dump_limit = (int)value;`	Технически присваивает `a->dump_limit` значение `(int)value`. Логически это обновляет `dump_limit`: хранит промежуточное значение, которое дальше используется в текущей функции.
211	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
212	`else if (strcmp(s, "--max-chain-depth") == 0 && i + 1 < argc) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
213	`long value = 0;`	Технически объявляет `value` в функции `parse_args` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
214	`if (!parse_long_range(argv[++i], 1, 16, &value)) {`	Технически проверяет условие `!parse_long_range(argv[++i], 1, 16, &value)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
215	`fprintf(stderr, "Invalid value for --max-chain-depth: %s (expected 1..16)\n", argv[i]);`	Технически вызывает `fprintf`. Логически это действие нужно в функции `parse_args`: печатает сообщение в указанный поток, часто stderr.
216	`return PARSE_ERR;`	Технически возвращает выражение `PARSE_ERR`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
217	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
218	`a->max_chain_depth = (int)value;`	Технически присваивает `a->max_chain_depth` значение `(int)value`. Логически это обновляет `max_chain_depth`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
219	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
220	`else if (strcmp(s, "--timeout-ms") == 0 && i + 1 < argc) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
221	`if (!parse_u32_range(argv[++i], 1, UINT32_MAX, &a->timeout_ms)) {`	Технически проверяет условие `!parse_u32_range(argv[++i], 1, UINT32_MAX, &a->timeout_ms)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
222	`fprintf(stderr, "Invalid value for --timeout-ms: %s (expected 1..%u)\n",`	Технически вызывает `fprintf`. Логически это действие нужно в функции `parse_args`: печатает сообщение в указанный поток, часто stderr.
223	`argv[i], (unsigned)UINT32_MAX);`	Технически выполняет оператор `argv[i], (unsigned)UINT32_MAX);`. Логически в функции `parse_args` он меняет состояние, проверяемые данные или подготовку отчета.
224	`return PARSE_ERR;`	Технически возвращает выражение `PARSE_ERR`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
225	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
226	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
227	`else if (strcmp(s, "--trace-steps") == 0 && i + 1 < argc) {`	Технически объявляет/начинает функцию `if`; она доступная другим модулям через заголовок или линковку. Логически функция выполняет отдельную часть алгоритма проекта.
228	`if (!parse_u32_range(argv[++i], 0, 256, &a->trace_steps)) {`	Технически проверяет условие `!parse_u32_range(argv[++i], 0, 256, &a->trace_steps)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
229	`fprintf(stderr, "Invalid value for --trace-steps: %s (expected 0..256)\n", argv[i]);`	Технически вызывает `fprintf`. Логически это действие нужно в функции `parse_args`: печатает сообщение в указанный поток, часто stderr.
230	`return PARSE_ERR;`	Технически возвращает выражение `PARSE_ERR`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
231	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
232	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
233	`else if (strcmp(s, "--csv") == 0 && i + 1 < argc) { a->csv_path = argv[++i]; }`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--csv") == 0 && i + 1 < argc`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
234	`else if (strcmp(s, "--ghidra-csv") == 0 && i + 1 < argc) { a->ghidra_csv_path = argv[++i]; }`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--ghidra-csv") == 0 && i + 1 < argc`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
235	`else if (strcmp(s, "--json") == 0 && i + 1 < argc) { a->json_path = argv[++i]; }`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--json") == 0 && i + 1 < argc`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
236	`else if (strcmp(s, "--html") == 0 && i + 1 < argc) { a->html_path = argv[++i]; }`	Технически проверяет альтернативное условие после предыдущего `if`. Логически проверяет условие `strcmp(s, "--html") == 0 && i + 1 < argc`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
237	`else {`	Технически эта строка является частью С-конструкции `else {`. Логически она уточняет структуру блока функции `parse_args` и влияет на то, какие данные будут обработаны.
238	`fprintf(stderr, "Unknown/invalid option: %s\n", s);`	Технически вызывает `fprintf`. Логически это действие нужно в функции `parse_args`: печатает сообщение в указанный поток, часто stderr.
239	`return PARSE_ERR;`	Технически возвращает выражение `PARSE_ERR`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
240	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
241	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
242	`return PARSE_OK;`	Технически возвращает выражение `PARSE_OK`. Логически именно это значение становится результатом функции `parse_args` для вызывающего кода.
243	`}`	Технически завершает область видимости внутри `parse_args`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
245	`static void print_one(size_t idx, const RuntimeFunction* r, const UnwindSummary* u, bool ok) {`	Технически объявляет/начинает функцию `print_one`; она внутренняя для файла. Логически функция выполняет отдельную часть алгоритма проекта.
246	`uint32_t size = (r->end_rva >= r->begin_rva) ? (r->end_rva - r->begin_rva) : 0;`	Технически присваивает `uint32_t size` значение `(r->end_rva >= r->begin_rva) ? (r->end_rva - r->begin_rva) : 0`. Логически это обновляет `uint32_t size`: хранит промежуточное значение, которое дальше используется в текущей функции.
247	`char fbuf[64];`	Технически объявляет `fbuf` в функции `print_one`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
248	`unwind_flags_to_string(u->flags, fbuf);`	Технически вызывает `unwind_flags_to_string`. Логически это действие нужно в функции `print_one`: переводит флаги UNWIND_INFO в текст.
250	`printf("[%zu] 0x%08X..0x%08X sz=%-6u unwind=0x%08X ",`	Технически присваивает `printf("[%zu] 0x%08X..0x%08X sz` значение `%-6u unwind=0x%08X ",`. Логически это обновляет `0x%08X sz`: хранит промежуточное значение, которое дальше используется в текущей функции.
251	`idx, (unsigned)r->begin_rva, (unsigned)r->end_rva, (unsigned)size, (unsigned)r->unwind_rva);`	Технически выполняет оператор `idx, (unsigned)r->begin_rva, (unsigned)r->end_rva, (unsigned)size, (unsigned)r->unwind_rva);`. Логически в функции `print_one` он меняет состояние, проверяемые данные или подготовку отчета.
253	`if (!ok) {`	Технически проверяет условие `!ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.
254	`printf("UNWIND: <parse failed%s%s>\n",`	Технически вызывает `printf`. Логически это действие нужно в функции `print_one`: выводит информацию пользователю в консоль.
255	`u->parse_error[0] ? ": " : "",`	Технически эта строка является частью С-конструкции `u->parse_error[0] ? ": " : "",`. Логически она уточняет структуру блока функции `print_one` и влияет на то, какие данные будут обработаны.
256	`u->parse_error);`	Технически выполняет оператор `u->parse_error);`. Логически в функции `print_one` он меняет состояние, проверяемые данные или подготовку отчета.
257	`return;`	Технически выходит из функции `print_one` без значения. Логически дальнейшие строки функции уже не выполняются.
258	`}`	Технически завершает область видимости внутри `print_one`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
260	`printf("v%u flags=%s prolog=%u stack=%uB",`	Технически присваивает `printf("v%u flags` значение `%s prolog=%u stack=%uB",`. Логически это обновляет `printf("v%u flags`: хранит промежуточное значение, которое дальше используется в текущей функции.
261	`(unsigned)u->version, fbuf, (unsigned)u->prolog_size, (unsigned)u->fixed_stack_bytes);`	Технически выполняет оператор `(unsigned)u->version, fbuf, (unsigned)u->prolog_size, (unsigned)u->fixed_stack_bytes);`. Логически в функции `print_one` он меняет состояние, проверяемые данные или подготовку отчета.
263	`if (u->frame_reg) printf(" fp=%s+%u", unwind_reg_name(u->frame_reg), (unsigned)(u->frame_off * 16u));`	Технически проверяет условие `u->frame_reg) printf(" fp=%s+%u", unwind_reg_name(u->frame_reg), (unsigned)(u->frame_off * 16u);` логически отбирает записи, где важна корректность UNWIND_INFO.
264	`if (u->has_handler) printf(" handler=0x%08X", (unsigned)u->handler_rva);`	Технически проверяет условие `u->has_handler) printf(" handler=0x%08X", (unsigned)u->handler_rva;` логически отделяет функции с language-specific handler от обычных функций.
265	`if (u->has_chain) printf(" chain_u=0x%08X", (unsigned)u->chain.unwind_rva);`	Технически проверяет условие `u->has_chain) printf(" chain_u=0x%08X", (unsigned)u->chain.unwind_rva;` логически отбирает записи, где важна корректность UNWIND_INFO.
266	`if (u->anomalies) printf(" anom=0x%08X", (unsigned)u->anomalies);`	Технически проверяет условие `u->anomalies) printf(" anom=0x%08X", (unsigned)u->anomalies;` если оно истинно, выполняется защитная или аналитическая ветка ниже.
267	`printf("\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `print_one`: выводит информацию пользователю в консоль.
268	`}`	Технически завершает область видимости внутри `print_one`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
270	`int main(int argc, char** argv) {`	Технически объявляет/начинает функцию `main`; она доступная другим модулям через заголовок или линковку. Логически функция запускает CLI, связывает все модули и определяет код завершения программы.
271	`Args a;`	Технически объявляет `a` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
272	`int exit_code = 0;`	Технически объявляет `exit_code` в функции `main` и сразу задает начальное значение `0`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
273	`ParseResult pres = parse_args(argc, argv, &a);`	Технически присваивает `ParseResult pres` значение `parse_args(argc, argv, &a)`. Логически это обновляет `ParseResult pres`: хранит промежуточное значение, которое дальше используется в текущей функции.
274	`if (pres != PARSE_OK) {`	Технически проверяет условие `pres != PARSE_OK`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
275	`if (pres == PARSE_VERSION) {`	Технически проверяет условие `pres == PARSE_VERSION`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
276	`print_version();`	Технически вызывает `print_version`. Логически это действие нужно в функции `main`: вызывает функцию `print_version` для выполнения части алгоритма.
277	`return 0;`	Технически возвращает код 0 из функции `main`. Логически это успешное завершение программы или callback.
278	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
279	`usage(argv[0]);`	Технически вызывает `usage`. Логически это действие нужно в функции `main`: вызывает функцию `usage` для выполнения части алгоритма.

Стр.	Код	Нормальное объяснение
280	`return (pres == PARSE_HELP) ? 0 : 2;`	Технически возвращает выражение `(pres == PARSE_HELP) ? 0 : 2`. Логически именно это значение становится результатом функции `main` для вызывающего кода.
281	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
283	`uint8_t* owned = NULL;`	Технически объявляет `owned` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
284	`Buf b;`	Технически объявляет `b` в функции `main`. Логически это место для данных: хранит буфер, с которым работает текущий модуль.
285	`if (!buf_init_from_file(&b, a.input, &owned)) {`	Технически проверяет условие `!buf_init_from_file(&b, a.input, &owned)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
286	`fprintf(stderr, "Failed to read file: %s\n", a.input);`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.

Стр.	Код	Нормальное объяснение
287	`return 1;`	Технически возвращает выражение `1`. Логически именно это значение становится результатом функции `main` для вызывающего кода.
288	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
290	`StrBuf err;`	Технически объявляет `err` в функции `main`. Логически это место для данных: накапливает текст ошибки для вызывающего кода.
291	`sb_init(&err, 1024);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
293	`PeImage img;`	Технически объявляет `img` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
294	`if (!pe_parse(&img, &b, &err)) {`	Технически проверяет условие `!pe_parse(&img, &b, &err)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
295	`fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
296	`sb_free(&err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
297	`buf_free_owned(owned);`	Технически вызывает `buf_free_owned`. Логически это действие нужно в функции `main`: освобождает память, которой владеет Buf.
298	`return 1;`	Технически возвращает выражение `1`. Логически именно это значение становится результатом функции `main` для вызывающего кода.
299	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
301	`RuntimeFunction* rf = NULL;`	Технически объявляет `rf` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
302	`size_t rf_count = 0;`	Технически объявляет `rf_count` в функции `main` и сразу задает начальное значение `0`. Логически это место для данных: хранит размер или количество, поэтому используется в проверках границ.
303	`StrBuf err2;`	Технически объявляет `err2` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
304	`sb_init(&err2, 1024);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
306	`RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(`	Технически присваивает `RuntimeFunctionReadResult rf_read` значение `unwind_read_runtime_functions_ex(`. Логически это обновляет `RuntimeFunctionReadResult rf_read`: хранит промежуточное значение, которое дальше используется в текущей функции.
307	`&b, &img, &rf, &rf_count, &err2);`	Технически выполняет битовую операцию `&b, &img, &rf, &rf_count, &err2);`. Логически С так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
308	`if (rf_read != RUNTIME_FUNCTION_READ_OK) {`	Технически проверяет условие `rf_read != RUNTIME_FUNCTION_READ_OK`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
309	`fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
310	`if (rf_read == RUNTIME_FUNCTION_READ_OOM) {`	Технически проверяет условие `rf_read == RUNTIME_FUNCTION_READ_OOM`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
311	`pe_free(&img);`	Технически вызывает `pe_free`. Логически это действие нужно в функции `main`: освобождает память, выделенную PE-парсером.
312	`sb_free(&err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
313	`sb_free(&err2);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.

Стр.	Код	Нормальное объяснение
314	`buf_free_owned(owned);`	Технически вызывает `buf_free_owned`. Логически это действие нужно в функции `main`: освобождает память, которой владеет Buf.
315	`return 1;`	Технически возвращает выражение 1. Логически именно это значение становится результатом функции `main` для вызывающего кода.
316	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
317	`/*`	Технически это комментарий исходника. Логически он объясняет разработчику: .
318	`* Keep the established report field/finding code for compatibility.`	Технически это комментарий исходника. Логически он объясняет разработчику: Keep the established report field/finding code for compatibility..
319	`* It now covers every structurally unreadable exception directory.`	Технически это комментарий исходника. Логически он объясняет разработчику: It now covers every structurally unreadable exception directory..
320	`*/`	Технически это комментарий исходника. Логически он объясняет разработчику: .
321	`img.exception_directory_oob = true;`	Технически объявляет поле `true` внутри структуры. Логически это поле: хранит промежуточное значение, которое дальше используется в текущей функции.
322	`fprintf(stderr, "Continuing with a partial structural report.\n");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
323	`exit_code = 1;`	Технически присваивает `exit_code` значение 1. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
324	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
326	`report_print_brief(a.input, &img, rf_count);`	Технически вызывает `report_print_brief`. Логически это действие нужно в функции `main`: печатает краткую сводку анализа в консоль.
328	`UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1, sizeof(UnwindSummary));`	Технически присваивает `UnwindSummary* us` значение `(UnwindSummary*)calloc(rf_count ? rf_count : 1, sizeof(UnwindSummary))`. Логически это обновляет `UnwindSummary* us`: хранит промежуточное значение, которое дальше используется в текущей функции.
329	`bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool));`	Технически присваивает `bool* us_ok` значение `(bool*)calloc(rf_count ? rf_count : 1, sizeof(bool))`. Логически это обновляет `bool* us_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
330	`if (!us || !us_ok) {`	Технически проверяет условие `!us || !us_ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.
331	`fprintf(stderr, "Out of memory.\n");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
332	`free(us);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
333	`free(us_ok);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
334	`free(rf);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
335	`pe_free(&img);`	Технически вызывает `pe_free`. Логически это действие нужно в функции `main`: освобождает память, выделенную PE-парсером.
336	`sb_free(&err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
337	`sb_free(&err2);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
338	`buf_free_owned(owned);`	Технически вызывает `buf_free_owned`. Логически это действие нужно в функции `main`: освобождает память, которой владеет Buf.
339	`return 1;`	Технически возвращает выражение 1. Логически именно это значение становится результатом функции `main` для вызывающего кода.
340	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
342	`UnwindParseOpts uopts = {`	Технически присваивает `UnwindParseOpts uopts` значение `{`. Логически это обновляет `UnwindParseOpts uopts`: хранит промежуточное значение, которое дальше используется в текущей функции.
343	`.max_chain_depth = a.max_chain_depth,`	Технически присваивает `.max_chain_depth` значение `a.max_chain_depth,`. Логически это обновляет `max_chain_depth`: хранит промежуточное значение, которое дальше используется в текущей функции.
344	`.follow_chain = a.follow_chain,`	Технически присваивает `.follow_chain` значение `a.follow_chain,`. Логически это обновляет `follow_chain`: хранит промежуточное значение, которое дальше используется в текущей функции.
345	`.strict_abi = a.strict_abi`	Технически присваивает `.strict_abi` значение `a.strict_abi`. Логически это обновляет `strict_abi`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
346	`};`	Технически закрывает объявление структуры/enum и ставит обязательную точку с запятой. Логически тип теперь полностью определен и может использоваться в других полях и функциях.
348	`StrBuf ops;`	Технически объявляет `ops` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
349	`sb_init(&ops, 1024);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
351	`for (size_t i = 0; i < rf_count; i++) {`	Технически начинает цикл `size_t i = 0; i < rf_count; i++`. Логически он проходит по массиву записей анализа, чтобы проверить каждую запись, а не только первую.
352	`StrBuf perr;`	Технически объявляет `perr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
353	`sb_init(&perr, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
354	`if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }`	Технически проверяет условие `a.show_ops) { ops.len = 0; if (ops.s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
355	`bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops : NULL, &perr);`	Технически присваивает `bool ok` значение `unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops : NULL, &perr)`. Логически это обновляет `bool ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
356	`us_ok[i] = ok;`	Технически присваивает `us_ok[i]` значение `ok`. Логически это обновляет `us_ok[i]`: хранит промежуточное значение, которое дальше используется в текущей функции.
357	`if (a.dump && a.show_ops && ok) {`	Технически проверяет условие `a.dump && a.show_ops && ok`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
358	`/* ops will be printed in dump loop below */`	Технически это комментарий исходника. Логически он объясняет разработчику: ops will be printed in dump loop below.
359	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
360	`sb_free(&perr);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
361	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
363	`if (a.dump) {`	Технически проверяет условие `a.dump`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
364	`size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;`	Технически присваивает `size_t limit` значение `(a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit`. Логически это обновляет `size_t limit`: хранит размер или количество, поэтому используется в проверках границ.
365	`if (limit > rf_count) limit = rf_count;`	Технически проверяет условие `limit > rf_count`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
366	`printf("\n[Dump] Showing %zu / %zu entries\n", limit, rf_count);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
367	`for (size_t i = 0; i < limit; i++) {`	Технически начинает цикл `size_t i = 0; i < limit; i++`. Логически он проходит по диапазону значений, чтобы проверить каждую запись, а не только первую.
368	`print_one(i, &rf[i], &us[i], us_ok[i]);`	Технически вызывает `print_one`. Логически это действие нужно в функции `main`: вызывает функцию `print_one` для выполнения части алгоритма.
369	`if (a.show_ops && us_ok[i]) {`	Технически проверяет условие `a.show_ops && us_ok[i]`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
370	`StrBuf ops2;`	Технически объявляет `ops2` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
371	`StrBuf perr;`	Технически объявляет `perr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
372	`sb_init(&ops2, 1024);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
373	`sb_init(&perr, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
374	`(void)unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], &ops2, &perr);`	Технически вызывает `unwind_parse`. Логически это действие нужно в функции `main`: вызывает функцию `unwind_parse` для выполнения части алгоритма.
375	`if (ops2.s && ops2.len) printf("%s", ops2.s);`	Технически проверяет условие `ops2.s && ops2.len) printf("%s", ops2.s`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
376	`sb_free(&ops2);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
377	`sb_free(&perr);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
378	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
379	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
380	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
382	`HeuristicReport hr;`	Технически объявляет `hr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
383	`HeuristicReport* hr_ptr = NULL;`	Технически объявляет `hr_ptr` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
384	`DynamicReport dr;`	Технически объявляет `dr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
385	`memset(&dr, 0, sizeof(dr));`	Технически вызывает `memset`. Логически это действие нужно в функции `main`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
386	`DynamicReport* dr_ptr = NULL;`	Технически объявляет `dr_ptr` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
387	`bool dr_initialized = false;`	Технически объявляет `dr_initialized` в функции `main` и сразу задает начальное значение `false`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
388	`if (a.heuristics) {`	Технически проверяет условие `a.heuristics`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
389	`StrBuf perr;`	Технически объявляет `perr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
390	`sb_init(&perr, 512);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
391	`if (heuristics_analyze(&img, rf, us, us_ok, rf_count, &hr, &perr)) {`	Технически проверяет условие `heuristics_analyze(&img, rf, us, us_ok, rf_count, &hr, &perr)`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
392	`report_print_heuristics(&hr);`	Технически вызывает `report_print_heuristics`. Логически это действие нужно в функции `main`: печатает найденные эвристики в консоль.
393	`hr_ptr = &hr;`	Технически присваивает `hr_ptr` значение `&hr`. Логически это обновляет `hr_ptr`: хранит промежуточное значение, которое дальше используется в текущей функции.
394	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
395	`fprintf(stderr, "Heuristic analysis failed:\n%s", perr.s ? perr.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
396	`exit_code = 1;`	Технически присваивает `exit_code` значение 1. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
397	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
398	`sb_free(&perr);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
399	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
401	`if (a.dynamic) {`	Технически проверяет условие `a.dynamic`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
402	`StrBuf derr;`	Технически объявляет `derr` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
403	`sb_init(&derr, 1024);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
404	`printf("\n[Dynamic] Starting debugger evidence collection (Windows only)...\n");`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
405	`bool ran = dynamic_collect_windows(a.input, a.child_argv, a.child_argc,`	Технически присваивает `bool ran` значение `dynamic_collect_windows(a.input, a.child_argv, a.child_argc,`. Логически это обновляет `bool ran`: хранит промежуточное значение, которое дальше используется в текущей функции.
406	`&img, rf, us, us_ok, rf_count,`	Технически выполняет битовую операцию `&img, rf, us, us_ok, rf_count,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
407	`a.timeout_ms, a.trace_steps,`	Технически эта строка является частью C-конструкции `a.timeout_ms, a.trace_steps,`. Логически она уточняет структуру блока функции `main` и влияет на то, какие данные будут обработаны.
408	`&dr, &derr);`	Технически выполняет битовую операцию `&dr, &derr);`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.

Стр.	Код	Нормальное объяснение
409	`dr_initialized = true;`	Технически присваивает `dr_initialized` значение `true`. Логически это обновляет `dr_initialized`: хранит промежуточное значение, которое дальше используется в текущей функции.
410	`dr_ptr = &dr;`	Технически присваивает `dr_ptr` значение `&dr`. Логически это обновляет `dr_ptr`: хранит промежуточное значение, которое дальше используется в текущей функции.
411	`if (!ran) {`	Технически проверяет условие `!ran`; логически это защита от NULL-указателя и работы с неинициализированными данными.
412	`fprintf(stderr, "[Dynamic] Failed: %s", derr.s ? derr.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
413	`exit_code = 1;`	Технически присваивает `exit_code` значение `1`. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
414	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
415	`printf("[Dynamic] Done. exceptions_seen=%u unwind_handler_hits=%u veh_hits=%u dyn_ftable_calls=%u\n",`	Технически присваивает `printf("[Dynamic] Done. exceptions_seen` значение `%u unwind_handler_hits=%u veh_hits=%u dyn_ftable_calls=%u\n",`. Логически это обновляет `exceptions_seen`: хранит промежуточное значение, которое дальше используется в текущей функции.
416	`(unsigned)dr.exceptions_seen,`	Технически эта строка является частью С-конструкции `(unsigned)dr.exceptions_seen,`. Логически она уточняет структуру блока функции `main` и влияет на то, какие данные будут обработаны.
417	`(unsigned)dr.unwind_handler_hits,`	Технически эта строка является частью С-конструкции `(unsigned)dr.unwind_handler_hits,`. Логически она уточняет структуру блока функции `main` и влияет на то, какие данные будут обработаны.
418	`(unsigned)dr.veh_hits,`	Технически эта строка является частью С-конструкции `(unsigned)dr.veh_hits,`. Логически она уточняет структуру блока функции `main` и влияет на то, какие данные будут обработаны.
419	`(unsigned)dr.rtl_add_function_table_calls);`	Технически выполняет оператор `(unsigned)dr.rtl_add_function_table_calls);`. Логически в функции `main` он меняет состояние, проверяемые данные или подготовку отчета.
420	`if (dr.details.s && dr.details.len) {`	Технически проверяет условие `dr.details.s && dr.details.len`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
421	`printf("%s\n", dr.details.s);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
422	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
423	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
424	`sb_free(&derr);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
425	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
427	`if (a.csv_path) {`	Технически проверяет условие `a.csv_path`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
428	`StrBuf e;`	Технически объявляет `e` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
429	`sb_init(&e, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
430	`if (!report_write_csv(a.csv_path, &img, rf, us, us_ok, rf_count, false, &e)) {`	Технически проверяет условие `!report_write_csv(a.csv_path, &img, rf, us, us_ok, rf_count, false, &e)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
431	`fprintf(stderr, "CSV write failed:\n%s", e.s ? e.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.

Стр.	Код	Нормальное объяснение
432	`exit_code = 1;`	Технически присваивает `exit_code` значение `1`. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
433	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
434	`printf("\nWrote CSV: %s\n", a.csv_path);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
435	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
436	`sb_free(&e);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
437	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Стр.	Код	Нормальное объяснение
438	`if (a.ghidra_csv_path) {`	Технически проверяет условие `a.ghidra_csv_path`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
439	`StrBuf e;`	Технически объявляет `e` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
440	`sb_init(&e, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
441	`if (!report_write_csv(a.ghidra_csv_path, &img, rf, us, us_ok, rf_count, true, &e)) {`	Технически проверяет условие `!report_write_csv(a.ghidra_csv_path, &img, rf, us, us_ok, rf_count, true, &e)`; логически это защита от NULL-указателя и работы с неинициализированными данными.
442	`fprintf(stderr, "Ghidra CSV write failed:\n%s", e.s ? e.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
443	`exit_code = 1;`	Технически присваивает `exit_code` значение 1. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
444	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
445	`printf("\nWrote Ghidra CSV: %s\n", a.ghidra_csv_path);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
446	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
447	`sb_free(&e);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
448	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
450	`StrBuf invocation = {0};`	Технически объявляет `invocation` в функции `main` и сразу задает начальное значение `{0}`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
451	`char timestamp[32] = "";`	Технически объявляет `timestamp` в функции `main` и сразу задает начальное значение `""`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
452	`char input_sha256[65] = "";`	Технически объявляет `input_sha256` в функции `main` и сразу задает начальное значение `""`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
453	`char* input_display = NULL;`	Технически объявляет `input_display` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
454	`ReportMetadata metadata;`	Технически объявляет `metadata` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
455	`memset(&metadata, 0, sizeof(metadata));`	Технически вызывает `memset`. Логически это действие нужно в функции `main`: обнуляет структуру перед заполнением, чтобы не оставить мусорные поля.
456	`ReportMetadata* metadata_ptr = NULL;`	Технически объявляет `metadata_ptr` в функции `main` и сразу задает начальное значение `NULL`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
457	`if (a.json_path || a.html_path) {`	Технически проверяет условие `a.json_path || a.html_path`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
458	`input_display = argv_text_to_utf8(a.input);`	Технически присваивает `input_display` значение `argv_text_to_utf8(a.input)`. Логически это обновляет `input_display`: хранит промежуточное значение, которое дальше используется в текущей функции.
459	`bool metadata_ok = sb_init(&invocation, 256) &&`	Технически присваивает `bool metadata_ok` значение `sb_init(&invocation, 256) &&`. Логически это обновляет `bool metadata_ok`: хранит промежуточное значение, которое дальше используется в текущей функции.
460	`input_display &&`	Технически выполняет битовую операцию `input_display &&`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
461	`build_invocation(argc, argv, &invocation) &&`	Технически вызывает `build_invocation`. Логически это действие нужно в функции `main`: вызывает функцию `build_invocation` для выполнения части алгоритма.
462	`app_utc_timestamp(timestamp, sizeof(timestamp)) &&`	Технически вызывает `app_utc_timestamp`. Логически это действие нужно в функции `main`: формирует UTC-время для метаданных отчета.
463	`sha256_hex(b.data, b.size, input_sha256);`	Технически вызывает `sha256_hex`. Логически это действие нужно в функции `main`: считает SHA-256 входного бинарника для воспроизводимости отчета.
464	`if (!metadata_ok) {`	Технически проверяет условие `!metadata_ok`; логически это защита от NULL-указателя и работы с неинициализированными данными.
465	`fprintf(stderr, "Failed to prepare report metadata.\n");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.

Стр.	Код	Нормальное объяснение
466	`exit_code = 1;`	Технически присваивает `exit_code` значение 1. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
467	`} else {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
468	`metadata.tool_version = app_version();`	Технически присваивает `metadata.tool_version` значение `app_version()`. Логически это обновляет `tool_version`: хранит промежуточное значение, которое дальше используется в текущей функции.
469	`metadata.report_schema_version = app_report_schema_version();`	Технически присваивает `metadata.report_schema_version` значение `app_report_schema_version()`. Логически это обновляет `report_schema_version`: хранит промежуточное значение, которое дальше используется в текущей функции.
470	`metadata.generated_at_utc = timestamp;`	Технически присваивает `metadata.generated_at_utc` значение `timestamp`. Логически это обновляет `generated_at_utc`: хранит промежуточное значение, которое дальше используется в текущей функции.
471	`metadata.input_sha256 = input_sha256;`	Технически присваивает `metadata.input_sha256` значение `input_sha256`. Логически это обновляет `input_sha256`: хранит промежуточное значение, которое дальше используется в текущей функции.
472	`metadata.input_size = (uint64_t)b.size;`	Технически присваивает `metadata.input_size` значение `(uint64_t)b.size`. Логически это обновляет `input_size`: хранит размер или количество, поэтому используется в проверках границ.
473	`metadata.command_line = invocation.s ? invocation.s : "";`	Технически присваивает `metadata.command_line` значение `invocation.s ? invocation.s : ""`. Логически это обновляет `command_line`: хранит промежуточное значение, которое дальше используется в текущей функции.
474	`metadata.build_compiler = app_build_compiler();`	Технически присваивает `metadata.build_compiler` значение `app_build_compiler()`. Логически это обновляет `build_compiler`: хранит промежуточное значение, которое дальше используется в текущей функции.
475	`metadata.build_platform = app_build_platform();`	Технически присваивает `metadata.build_platform` значение `app_build_platform()`. Логически это обновляет `build_platform`: хранит промежуточное значение, которое дальше используется в текущей функции.
476	`metadata.build_mode = app_build_mode();`	Технически присваивает `metadata.build_mode` значение `app_build_mode()`. Логически это обновляет `build_mode`: хранит промежуточное значение, которое дальше используется в текущей функции.
477	`metadata.strict_abi = a.strict_abi;`	Технически присваивает `metadata.strict_abi` значение `a.strict_abi`. Логически это обновляет `strict_abi`: хранит промежуточное значение, которое дальше используется в текущей функции.
478	`metadata.follow_chain = a.follow_chain;`	Технически присваивает `metadata.follow_chain` значение `a.follow_chain`. Логически это обновляет `follow_chain`: хранит промежуточное значение, которое дальше используется в текущей функции.
479	`metadata.max_chain_depth = (uint32_t)a.max_chain_depth;`	Технически присваивает `metadata.max_chain_depth` значение `(uint32_t)a.max_chain_depth`. Логически это обновляет `max_chain_depth`: хранит промежуточное значение, которое дальше используется в текущей функции.
480	`metadata_ptr = &metadata;`	Технически присваивает `metadata_ptr` значение `&metadata`. Логически это обновляет `metadata_ptr`: хранит промежуточное значение, которое дальше используется в текущей функции.
481	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
482	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
484	`if (a.json_path) {`	Технически проверяет условие `a.json_path`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
485	`StrBuf e;`	Технически объявляет `e` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.
486	`sb_init(&e, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
487	`if (metadata_ptr &&`	Технически проверяет условие `if (metadata_ptr &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
488	`!report_write_json_ex(a.json_path, input_display ? input_display : a.input,`	Технически вызывает `report_write_json_ex`. Логически это действие нужно в функции `main`: вызывает функцию `report_write_json_ex` для выполнения части алгоритма.
489	`&img, rf, us, us_ok, rf_count,`	Технически выполняет битовую операцию `&img, rf, us, us_ok, rf_count,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
490	`hr_ptr, dr_ptr, metadata_ptr, &e)) {`	Технически выполняет битовую операцию `hr_ptr, dr_ptr, metadata_ptr, &e)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
491	`fprintf(stderr, "JSON write failed:\n%s", e.s ? e.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
492	`exit_code = 1;`	Технически присваивает `exit_code` значение 1. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
493	`} else if (metadata_ptr) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
494	`printf("\nWrote JSON: %s\n", a.json_path);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.

Стр.	Код	Нормальное объяснение
495	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
496	`sb_free(&e);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
497	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
498	`if (a.html_path) {`	Технически проверяет условие `a.html_path`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
499	`StrBuf e;`	Технически объявляет `e` в функции `main`. Логически это место для данных: хранит промежуточное значение, которое дальше используется в текущей функции.

Стр.	Код	Нормальное объяснение
500	`sb_init(&e, 256);`	Технически вызывает `sb_init`. Логически это действие нужно в функции `main`: создает динамический текстовый буфер.
501	`if (metadata_ptr &&`	Технически проверяет условие `if (metadata_ptr &&`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
502	`!report_write_html(a.html_path, input_display ? input_display : a.input,`	Технически вызывает `report_write_html`. Логически это действие нужно в функции `main`: записывает HTML-отчет для ручного просмотра.
503	`&img, rf, us, us_ok, rf_count,`	Технически выполняет битовую операцию `&img, rf, us, us_ok, rf_count,`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
504	`hr_ptr, dr_ptr, metadata_ptr, &e)) {`	Технически выполняет битовую операцию `hr_ptr, dr_ptr, metadata_ptr, &e)) {`. Логически C так проверяет или собирает флаги PE/unwind, где каждый бит имеет отдельный смысл.
505	`fprintf(stderr, "HTML write failed:\n%s", e.s ? e.s : "");`	Технически вызывает `fprintf`. Логически это действие нужно в функции `main`: печатает сообщение в указанный поток, часто stderr.
506	`exit_code = 1;`	Технически присваивает `exit_code` значение `1`. Логически это обновляет `exit_code`: хранит промежуточное значение, которое дальше используется в текущей функции.
507	`} else if (metadata_ptr) {`	Технически закрывает предыдущую ветку и открывает `else`. Логически код ниже выполняется только когда условие из `if` оказалось ложным.
508	`printf("\nWrote HTML: %s\n", a.html_path);`	Технически вызывает `printf`. Логически это действие нужно в функции `main`: выводит информацию пользователю в консоль.
509	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
510	`sb_free(&e);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
511	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.
512	`sb_free(&invocation);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
513	`free(input_display);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
515	`if (a.heuristics && hr_ptr) heuristics_free(&hr);`	Технически проверяет условие `a.heuristics && hr_ptr) heuristics_free(&hr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
516	`if (dr_initialized) dynamic_report_free(&dr);`	Технически проверяет условие `dr_initialized) dynamic_report_free(&dr`; если оно истинно, выполняется защитная или аналитическая ветка ниже.
518	`sb_free(&ops);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
519	`free(us_ok);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
520	`free(us);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
521	`free(rf);`	Технически вызывает `free`. Логически это действие нужно в функции `main`: освобождает память, чтобы в C не было утечки.
522	`pe_free(&img);`	Технически вызывает `pe_free`. Логически это действие нужно в функции `main`: освобождает память, выделенную PE-парсером.
523	`sb_free(&err);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
524	`sb_free(&err2);`	Технически вызывает `sb_free`. Логически это действие нужно в функции `main`: освобождает память динамического буфера.
525	`buf_free_owned(owned);`	Технически вызывает `buf_free_owned`. Логически это действие нужно в функции `main`: освобождает память, которой владеет Buf.
526	`return exit_code;`	Технически возвращает выражение `exit_code`. Логически именно это значение становится результатом функции `main` для вызывающего кода.

Стр.	Код	Нормальное объяснение
527	`}`	Технически завершает область видимости внутри `main`. Логически после этой строки локальные переменные завершенной области больше не доступны, а выполнение продолжается с внешней ветки, цикла или с возвратом из функции.

Разобрано непустых строк: 488. Пустые строки исходника намеренно не перечислены: они нужны только для визуального разделения и не выполняют код.

Как делать на MacBook

На MacBook удобно писать и тестировать статическую часть: PE parser, unwind decoder, heuristics, reports и CLI. Динамический Windows debug-loop физически опирается на WinAPI, поэтому его проверяют через Windows VM, отдельный Windows-компьютер или cross-build MinGW с запуском на Windows. Это не недостаток архитектуры: статический анализ переносим, runtime-валидация платформенная по своей природе.

```
xcode-select --install
brew install cmake ninja python mingw-w64

python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements-dev.txt

cmake --preset debug
cmake --build --preset debug
ctest --preset debug --output-on-failure

cmake --preset release
cmake --build --preset release
ctest --preset release --output-on-failure
```

Приложение. Полный код исходников

В этом приложении код повторяется без построчного объяснения, чтобы в конце учебника был чистый эталон для сверки. Построчные объяснения находятся в шагах выше.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.23)

project(
    eh_unwind_analyzer
    VERSION 1.0.0
    DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"
    LANGUAGES C
)

include(CTest)

option(EHUA_WARNINGS_AS_ERRORS "Treat compiler warnings as errors" ON)
option(EHUA_ENABLE_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)
option(EHUA_ENABLE_COVERAGE "Enable GCC/Clang coverage instrumentation" OFF)
option(EHUA_BUILD_FUZZER "Build the Clang libFuzzer parser harness" OFF)

if(EHUA_ENABLE_SANITIZERS AND EHUA_ENABLE_COVERAGE)
    message(FATAL_ERROR "Sanitizer and coverage instrumentation cannot be enabled together")
endif()

if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)
    message(FATAL_ERROR "EH/Unwind Analyzer supports Windows x64 builds only")
endif()

function(ehua_apply_warnings target)
    if(MSVC)
        target_compile_options(${target} PRIVATE /W4 /permissive- /utf-8)
        target_compile_definitions(${target} PRIVATE _CRT_SECURE_NO_WARNINGS=1)
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE /WX)
        endif()
    else()
        target_compile_options(
            ${target}
            PRIVATE
                -Wall
                -Wextra
                -Wpedantic
                -Wconversion
                -Wshadow
                -Wformat=2
        )
        if(CMAKE_C_COMPILER_ID STREQUAL "GNU")
            target_compile_options(${target} PRIVATE -finput-charset=UTF-8 -fexec-charset=UTF-8)
        endif()
        if(EHUA_WARNINGS_AS_ERRORS)
            target_compile_options(${target} PRIVATE -Werror)
        endif()
    endif()
endfunction()

function(ehua_apply_instrumentation target)
    if(EHUA_ENABLE_SANITIZERS)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_SANITIZERS is currently supported with GCC or Clang")
        endif()
        target_compile_options(${target} PRIVATE -fsanitize=address,undefined -fno-omit-frame-pointer)
        target_link_options(${target} PRIVATE -fsanitize=address,undefined)
    elseif(EHUA_ENABLE_COVERAGE)
        if(MSVC)
            message(FATAL_ERROR "EHUA_ENABLE_COVERAGE is currently supported with GCC or Clang")
        endif()
        target_compile_options(${target} PRIVATE --coverage -O0 -g)
        target_link_options(${target} PRIVATE --coverage)
    endif()
endfunction()

set(EHUA_CORE_SOURCES
    src/app_info.c
    src/dynamic_win.c
    src/heuristics.c
    src/pe.c
    src/report.c
    src/unwind.c
    src/util.c
)

add_library(ehua_core STATIC ${EHUA_CORE_SOURCES})
target_include_directories(
    ehua_core
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
)
target_compile_features(ehua_core PUBLIC c_std_17)

if(WIN32)
    target_compile_definitions(ehua_core PUBLIC WIN32_LEAN_AND_MEAN=1)
endif()

if(MINGW)
    target_compile_definitions(ehua_core PUBLIC __USE_MINGW_ANSI_STDIO=1)
endif()
ehua_apply_warnings(ehua_core)

add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
ehua_apply_warnings(eh_unwind_analyzer)
ehua_apply_instrumentation(ehua_core)
ehua_apply_instrumentation(eh_unwind_analyzer)

if(EHUA_BUILD_FUZZER)
    if(NOT CMAKE_C_COMPILER_ID MATCHES "Clang")
        message(FATAL_ERROR "EHUA_BUILD_FUZZER requires Clang")
    endif()
    include(CheckCSourceCompiles)
    set(EHUA_SAVED_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS}")
    set(EHUA_SAVED_REQUIRED_LINK_OPTIONS "${CMAKE_REQUIRED_LINK_OPTIONS}")
    set(CMAKE_REQUIRED_FLAGS "${CMAKE_REQUIRED_FLAGS} -fsanitize=fuzzer,address,undefined")
    set(CMAKE_REQUIRED_LINK_OPTIONS -fsanitize=fuzzer,address,undefined)
    check_c_source_compiles(
```

```
        "#include <stddef.h>
         #include <stdint.h>
         int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
             return (data != 0 && size != 0) ? 0 : 0;
         }"
        EHUA_LIBFUZZER_AVAILABLE
    )
    set(CMAKE_REQUIRED_FLAGS "${EHUA_SAVED_REQUIRED_FLAGS}")
    set(CMAKE_REQUIRED_LINK_OPTIONS "${EHUA_SAVED_REQUIRED_LINK_OPTIONS}")
    if(NOT EHUA_LIBFUZZER_AVAILABLE)
        message(FATAL_ERROR
            "Clang is present, but its libFuzzer runtime is unavailable. "
            "Install a full LLVM/Clang toolchain or leave EHUA_BUILD_FUZZER=OFF.")
    endif()
    add_executable(
        ehua_parser_fuzzer
        tests/fuzz/fuzz_parser.c
        src/heuristics.c
        src/pe.c
        src/unwind.c
        src/util.c
    )
    target_include_directories(ehua_parser_fuzzer PRIVATE "${CMAKE_CURRENT_SOURCE_DIR}/include")
    target_compile_features(ehua_parser_fuzzer PRIVATE c_std_17)
    ehua_apply_warnings(ehua_parser_fuzzer)
    target_compile_options(
        ehua_parser_fuzzer
        PRIVATE
            -fsanitize=fuzzer,address,undefined
            -fno-omit-frame-pointer
    )
    target_link_options(ehua_parser_fuzzer PRIVATE -fsanitize=fuzzer,address,undefined)
endif()

if(BUILD_TESTING)
    function(ehua_add_unit_test target source)
        add_executable(${target} ${source})
        target_link_libraries(${target} PRIVATE ehua_core)
        if(MSVC)
            target_compile_options(${target} PRIVATE /UNDEBUG)
        else()
            target_compile_options(${target} PRIVATE -UNDEBUG)
        endif()
        ehua_apply_warnings(${target})
        ehua_apply_instrumentation(${target})
        set(unit_workdir "${CMAKE_CURRENT_BINARY_DIR}/tests/unit/${target}")
        file(MAKE_DIRECTORY "${unit_workdir}")
        add_test(NAME ${target} COMMAND ${target})
        set_tests_properties(
            ${target}
            PROPERTIES
                WORKING_DIRECTORY "${unit_workdir}"
                TIMEOUT 30
        )
    endfunction()

    ehua_add_unit_test(unit_unwind tests/unit/unit_unwind.c)
    ehua_add_unit_test(unit_heuristics tests/unit/unit_heuristics.c)
    ehua_add_unit_test(unit_heuristics_mitigations tests/unit/unit_heuristics_mitigations.c)
    ehua_add_unit_test(unit_heuristics_context tests/unit/unit_heuristics_context.c)
    ehua_add_unit_test(unit_heuristics_combos tests/unit/unit_heuristics_combos.c)
    ehua_add_unit_test(unit_core_edges tests/unit/unit_core_edges.c)
    ehua_add_unit_test(unit_report_json tests/unit/unit_report_json.c)
    ehua_add_unit_test(unit_report_html tests/unit/unit_report_html.c)
    ehua_add_unit_test(unit_matrix_report tests/unit/unit_matrix_report.c)

    find_package(Python3 COMPONENTS Interpreter)
    if(Python3_Interpreter_FOUND)
        add_test(
            NAME regression_samples
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/run_regression_samples.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(regression_samples PROPERTIES TIMEOUT 120)
        add_test(
            NAME dynamic_validator
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_dynamic_validator.py"
        )
        set_tests_properties(dynamic_validator PROPERTIES TIMEOUT 30)
        add_test(
            NAME report_semantics
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_report_semantics.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(report_semantics PROPERTIES TIMEOUT 30)
        add_test(
            NAME release_metadata
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_metadata.py"
        )
        set_tests_properties(release_metadata PROPERTIES TIMEOUT 30)
        add_test(
            NAME release_hygiene
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_release_hygiene.py"
        )
        set_tests_properties(release_hygiene PROPERTIES TIMEOUT 30)
        add_test(
            NAME cli_contract
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/test_cli_contract.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
        )
        set_tests_properties(cli_contract PROPERTIES TIMEOUT 60)
        add_test(
            NAME mutation_fuzz
```

```
            COMMAND
                "${Python3_EXECUTABLE}"
                "${CMAKE_CURRENT_SOURCE_DIR}/tests/run_mutation_fuzz.py"
                "$<TARGET_FILE:eh_unwind_analyzer>"
                --cases
                120
        )
        set_tests_properties(
            mutation_fuzz
            PROPERTIES
                TIMEOUT 180
                LABELS "fuzz"
        )
        set(EHUA_PYTHON_TEST_ENV
            "PYTHONDONTWRITEBYTECODE=1"
            "PYTHONPATH=${CMAKE_BINARY_DIR}/pydeps"
        )
        set_tests_properties(
            regression_samples
            dynamic_validator
            report_semantics
            release_metadata
            release_hygiene
            cli_contract
            mutation_fuzz
            PROPERTIES
                ENVIRONMENT "${EHUA_PYTHON_TEST_ENV}"
        )
    endif()
endif()

install(TARGETS eh_unwind_analyzer RUNTIME DESTINATION bin)
install(
    DIRECTORY docs/
    DESTINATION share/eh_unwind_analyzer/docs
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
    PATTERN "__pycache__" EXCLUDE
    PATTERN "*.pyc" EXCLUDE
)
install(
    DIRECTORY ghidra/
    DESTINATION share/eh_unwind_analyzer/ghidra
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
    PATTERN "__pycache__" EXCLUDE
    PATTERN "*.pyc" EXCLUDE
)
install(
    DIRECTORY examples/
    DESTINATION share/eh_unwind_analyzer/examples
    PATTERN ".DS_Store" EXCLUDE
    PATTERN "._*" EXCLUDE
)
install(
    FILES
        README.md
        LICENSE
        CHANGELOG.md
        SECURITY.md
        CONTRIBUTING.md
    DESTINATION share/eh_unwind_analyzer
)

set(CPACK_PACKAGE_NAME "EH-Unwind-Analyzer")
set(CPACK_PACKAGE_VENDOR "EH/Unwind Analyzer project")
set(CPACK_PACKAGE_VERSION "1.0.0")
set(CPACK_PACKAGE_FILE_NAME "eh-unwind-analyzer-${CPACK_PACKAGE_VERSION}-${CMAKE_SYSTEM_NAME}-${CMAKE_SYSTEM_PROCESSOR}")
set(CPACK_GENERATOR "ZIP")
include(CPack)
```

CMakePresets.json

```
{
  "version": 6,
  "cmakeMinimumRequired": {
    "major": 3,
    "minor": 23,
    "patch": 0
  },
  "configurePresets": [
    {
      "name": "base",
      "hidden": true,
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/${presetName}",
      "cacheVariables": {
        "BUILD_TESTING": "ON",
        "CMAKE_EXPORT_COMPILE_COMMANDS": "ON",
        "EHUA_WARNINGS_AS_ERRORS": "ON"
      }
    },
    {
      "name": "debug",
      "displayName": "Debug",
      "inherits": "base",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug"
      }
    },
    {
      "name": "release",
      "displayName": "Release",
      "inherits": "base",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    },
    {
      "name": "sanitize",
      "displayName": "ASan + UBSan",
      "inherits": "base",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "EHUA_ENABLE_SANITIZERS": "ON"
      }
    },
    {
      "name": "coverage",
      "displayName": "Coverage",
      "inherits": "base",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "EHUA_ENABLE_COVERAGE": "ON"
      }
    }
  ],
  "buildPresets": [
    {
      "name": "debug",
      "configurePreset": "debug"
    },
    {
      "name": "release",
      "configurePreset": "release"
    },
    {
      "name": "sanitize",
      "configurePreset": "sanitize"
    },
    {
      "name": "coverage",
      "configurePreset": "coverage"
    }
  ],
  "testPresets": [
    {
      "name": "debug",
      "configurePreset": "debug",
      "output": {
        "outputOnFailure": true
      }
    },
    {
      "name": "release",
      "configurePreset": "release",
      "output": {
        "outputOnFailure": true
      }
    },
    {
      "name": "sanitize",
      "configurePreset": "sanitize",
      "output": {
        "outputOnFailure": true
      }
    },
    {
      "name": "coverage",
      "configurePreset": "coverage",
      "output": {
        "outputOnFailure": true
      }
    }
  ],
  "packagePresets": [
    {
      "name": "release",
      "configurePreset": "release",
      "generators": [
        "ZIP"
      ]
    }
  ]
```

```
112 | }
```

requirements-dev.txt

```
jsonschema==4.26.0
gcovr==8.6
```

requirements-validation.txt

```
jsonschema==4.26.0
```

include/app_info.h

```
#pragma once

#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

#define EHUA_VERSION "1.0.0"
#define EHUA_REPORT_SCHEMA_VERSION "1.1.0"

const char* app_version(void);
const char* app_report_schema_version(void);
const char* app_build_compiler(void);
const char* app_build_platform(void);
const char* app_build_mode(void);

/* UTC timestamp in RFC 3339 form: YYYY-MM-DDTHH:MM:SSZ. */
bool app_utc_timestamp(char* out, size_t out_size);

#ifdef __cplusplus
}
#endif
```

src/app_info.c

```c
#include "app_info.h"

#include <stdio.h>
#include <time.h>

const char* app_version(void) {
    return EHUA_VERSION;
}

const char* app_report_schema_version(void) {
    return EHUA_REPORT_SCHEMA_VERSION;
}

const char* app_build_compiler(void) {
#if defined(_MSC_VER)
    static char compiler[48];
    if (compiler[0] == '\0') {
        (void)snprintf(compiler, sizeof(compiler), "MSVC %d", _MSC_VER);
    }
    return compiler;
#elif defined(__clang__)
    return "Clang " __clang_version__;
#elif defined(__GNUC__)
    return "GCC " __VERSION__;
#else
    return "unknown";
#endif
}

const char* app_build_platform(void) {
#if defined(_WIN64)
    return "windows-x64";
#elif defined(_WIN32)
    return "windows-x86";
#elif defined(__APPLE__) && defined(__aarch64__)
    return "macos-arm64";
#elif defined(__APPLE__) && defined(__x86_64__)
    return "macos-x64";
#elif defined(__linux__) && defined(__x86_64__)
    return "linux-x64";
#elif defined(__linux__) && defined(__aarch64__)
    return "linux-arm64";
#else
    return "unknown";
#endif
}

const char* app_build_mode(void) {
#if defined(NDEBUG)
    return "release";
#else
    return "debug";
#endif
}

bool app_utc_timestamp(char* out, size_t out_size) {
    if (!out || out_size < 21) return false;
    time_t now = time(NULL);
    if (now == (time_t)-1) return false;

    struct tm utc;
#if defined(_WIN32)
    if (gmtime_s(&utc, &now) != 0) return false;
#else
    if (!gmtime_r(&now, &utc)) return false;
#endif
    return strftime(out, out_size, "%Y-%m-%dT%H:%M:%SZ", &utc) == 20;
}
```

include/util.h

```h
#pragma once
/*
  util.h - small helpers for safe parsing and dynamic arrays
  Project: EH/Unwind Analyzer (PE x64)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem);
void buf_free_owned(uint8_t* owned_mem);

bool buf_read(const Buf* b, size_t off, void* out, size_t len);
bool buf_read_u8 (const Buf* b, size_t off, uint8_t*  out);
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out);
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out);
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out);

static inline uint16_t le16(const uint8_t* p) {
    return (uint16_t)(p[0] | ((uint16_t)p[1] << 8));
}
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24));
}
static inline uint64_t le64(const uint8_t* p) {
    return (uint64_t)le32(p) | ((uint64_t)le32(p + 4) << 32);
}

typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;

#if defined(__MINGW32__) && (defined(__GNUC__) || defined(__clang__))
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(gnu_printf, format_index, first_arg)))
#elif defined(__GNUC__) || defined(__clang__)
#define EHUA_PRINTF_FORMAT(format_index, first_arg) \
    __attribute__((format(printf, format_index, first_arg)))
#else
#define EHUA_PRINTF_FORMAT(format_index, first_arg)
#endif

bool sb_init(StrBuf* sb, size_t initial_cap);
void sb_free(StrBuf* sb);
bool sb_appendf(StrBuf* sb, const char* fmt, ...)
    EHUA_PRINTF_FORMAT(2, 3);
bool sb_append(StrBuf* sb, const char* s);
bool sb_appendn(StrBuf* sb, const char* s, size_t n);
bool sb_ok(const StrBuf* sb);
bool file_write_atomic(const char* path, const char* data, size_t len);

/* SHA-256 digest encoded as 64 lowercase hexadecimal characters plus NUL. */
bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]);

#undef EHUA_PRINTF_FORMAT

#ifdef __cplusplus
}
#endif
```

src/util.c

```c
#include "util.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

#if defined(_WIN32)
#include <windows.h>
#else
#include <unistd.h>
#endif

bool buf_init_from_file(Buf* out, const char* path, uint8_t** owned_mem) {
    if (!out || !path || !owned_mem) return false;
    *owned_mem = NULL;
    FILE* f = fopen(path, "rb");
    if (!f) return false;

    if (fseek(f, 0, SEEK_END) != 0) { fclose(f); return false; }
    long sz = ftell(f);
    if (sz < 0) { fclose(f); return false; }
    if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); return false; }

    size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
    uint8_t* mem = (uint8_t*)malloc(alloc_size);
    if (!mem) { fclose(f); return false; }

    size_t rd = fread(mem, 1, (size_t)sz, f);
    fclose(f);
    if (rd != (size_t)sz) { free(mem); return false; }

    out->data = mem;
    out->size = (size_t)sz;
    *owned_mem = mem;
    return true;
}

void buf_free_owned(uint8_t* owned_mem) {
    free(owned_mem);
}

bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u8(const Buf* b, size_t off, uint8_t* out) {
    return buf_read(b, off, out, 1);
}
bool buf_read_u16(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = le16(tmp);
    return true;
}
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
bool buf_read_u64(const Buf* b, size_t off, uint64_t* out) {
    uint8_t tmp[8];
    if (!buf_read(b, off, tmp, 8)) return false;
    *out = le64(tmp);
    return true;
}

static bool sb_reserve(StrBuf* sb, size_t need_cap) {
    if (!sb || sb->failed) return false;
    if (need_cap <= sb->cap) return true;
    size_t newcap = sb->cap ? sb->cap : 128;
    while (newcap < need_cap) {
        size_t increment = newcap / 2u + 64u;
        if (increment > SIZE_MAX - newcap) {
            newcap = need_cap;
            break;
        }
        newcap += increment;
    }
    char* p = (char*)realloc(sb->s, newcap);
    if (!p) {
        sb->failed = true;
        return false;
    }
    sb->s = p;
    sb->cap = newcap;
    return true;
}

bool sb_init(StrBuf* sb, size_t initial_cap) {
    if (!sb) return false;
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
    if (initial_cap == 0) initial_cap = 256;
    if (!sb_reserve(sb, initial_cap) || !sb->s) {
        sb->failed = true;
        return false;
    }
    sb->s[0] = 0;
    return true;
}

void sb_free(StrBuf* sb) {
    if (!sb) return;
    free(sb->s);
```

```
    sb->s = NULL;
    sb->len = 0;
    sb->cap = 0;
    sb->failed = false;
}

bool sb_appendn(StrBuf* sb, const char* s, size_t n) {
    if (!sb) return false;
    if (sb->failed || (!s && n != 0)) {
        sb->failed = true;
        return false;
    }
    if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {
        sb->failed = true;
        return false;
    }
    if (!sb_reserve(sb, sb->len + n + 1)) return false;
    if (n != 0) memcpy(sb->s + sb->len, s, n);
    sb->len += n;
    sb->s[sb->len] = 0;
    return true;
}

bool sb_append(StrBuf* sb, const char* s) {
    if (!sb) return false;
    if (!s) {
        sb->failed = true;
        return false;
    }
    return sb_appendn(sb, s, strlen(s));
}

bool sb_appendf(StrBuf* sb, const char* fmt, ...) {
    if (!sb) return false;
    if (sb->failed || !fmt) {
        sb->failed = true;
        return false;
    }
    va_list ap;
    va_start(ap, fmt);
    va_list ap2;
    va_copy(ap2, ap);

#if defined(__GNUC__) || defined(__clang__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wformat-nonliteral"
#endif
    int need = vsnprintf(NULL, 0, fmt, ap);
    va_end(ap);
    if (need < 0) {
        va_end(ap2);
        sb->failed = true;
        return false;
    }

    if (sb->len == SIZE_MAX || (size_t)need > SIZE_MAX - sb->len - 1u) {
        va_end(ap2);
        sb->failed = true;
        return false;
    }
    if (!sb_reserve(sb, sb->len + (size_t)need + 1u)) { va_end(ap2); return false; }
    int wrote = vsnprintf(sb->s + sb->len, sb->cap - sb->len, fmt, ap2);
#if defined(__GNUC__) || defined(__clang__)
#pragma GCC diagnostic pop
#endif
    va_end(ap2);
    if (wrote != need) {
        sb->failed = true;
        return false;
    }
    sb->len += (size_t)wrote;
    return true;
}

bool sb_ok(const StrBuf* sb) {
    return sb && !sb->failed && sb->s && sb->len < sb->cap;
}

bool file_write_atomic(const char* path, const char* data, size_t len) {
    if (!path || (!data && len != 0)) return false;
    size_t path_len = strlen(path);
    if (path_len > SIZE_MAX - 48u) return false;

    size_t temp_size = path_len + 48u;
    char* temp_path = (char*)malloc(temp_size);
    if (!temp_path) return false;
#if defined(_WIN32)
    unsigned long process_id = (unsigned long)GetCurrentProcessId();
#else
    unsigned long process_id = (unsigned long)getpid();
#endif
    int written = snprintf(temp_path, temp_size, "%s.tmp.%lu", path, process_id);
    if (written < 0 || (size_t)written >= temp_size) {
        free(temp_path);
        return false;
    }

    FILE* f = fopen(temp_path, "wb");
    if (!f) {
        free(temp_path);
        return false;
    }
    size_t wr = len ? fwrite(data, 1, len, f) : 0u;
    bool ok = wr == len && ferror(f) == 0 && fflush(f) == 0;
    if (fclose(f) != 0) ok = false;

    if (ok) {
#if defined(_WIN32)
        ok = MoveFileExA(
                 temp_path,
                 path,
                 MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH) != 0;
#else
        ok = rename(temp_path, path) == 0;
#endif
```

```
    }
    if (!ok) (void)remove(temp_path);
    free(temp_path);
    return ok;
}

typedef struct Sha256Ctx {
    uint32_t state[8];
    uint64_t bit_count;
    uint8_t block[64];
    size_t block_len;
} Sha256Ctx;

static uint32_t sha256_rotr(uint32_t value, unsigned count) {
    return (value >> count) | (value << (32u - count));
}

static void sha256_transform(Sha256Ctx* ctx, const uint8_t block[64]) {
    static const uint32_t k[64] = {
        0x428a2f98u,0x71374491u,0xb5c0fbcfu,0xe9b5dba5u,0x3956c25bu,0x59f111f1u,0x923f82a4u,0xab1c5ed5u,
        0xd807aa98u,0x12835b01u,0x243185beu,0x550c7dc3u,0x72be5d74u,0x80deb1feu,0x9bdc06a7u,0xc19bf174u,
        0xe49b69c1u,0xefbe4786u,0x0fc19dc6u,0x240ca1ccu,0x2de92c6fu,0x4a7484aau,0x5cb0a9dcu,0x76f988dau,
        0x983e5152u,0xa831c66du,0xb00327c8u,0xbf597fc7u,0xc6e00bf3u,0xd5a79147u,0x06ca6351u,0x14292967u,
        0x27b70a85u,0x2e1b2138u,0x4d2c6dfcu,0x53380d13u,0x650a7354u,0x766a0abbu,0x81c2c92eu,0x92722c85u,
        0xa2bfe8a1u,0xa81a664bu,0xc24b8b70u,0xc76c51a3u,0xd192e819u,0xd6990624u,0xf40e3585u,0x106aa070u,
        0x19a4c116u,0x1e376c08u,0x2748774cu,0x34b0bcb5u,0x391c0cb3u,0x4ed8aa4au,0x5b9cca4fu,0x682e6ff3u,
        0x748f82eeu,0x78a5636fu,0x84c87814u,0x8cc70208u,0x90befffau,0xa4506cebu,0xbef9a3f7u,0xc67178f2u
    };
    uint32_t w[64];
    for (size_t i = 0; i < 16; i++) {
        size_t j = i * 4;
        w[i] = ((uint32_t)block[j] << 24) |
               ((uint32_t)block[j + 1] << 16) |
               ((uint32_t)block[j + 2] << 8) |
               (uint32_t)block[j + 3];
    }
    for (size_t i = 16; i < 64; i++) {
        uint32_t s0 = sha256_rotr(w[i - 15], 7) ^ sha256_rotr(w[i - 15], 18) ^ (w[i - 15] >> 3);
        uint32_t s1 = sha256_rotr(w[i - 2], 17) ^ sha256_rotr(w[i - 2], 19) ^ (w[i - 2] >> 10);
        w[i] = w[i - 16] + s0 + w[i - 7] + s1;
    }

    uint32_t a = ctx->state[0], b = ctx->state[1], c = ctx->state[2], d = ctx->state[3];
    uint32_t e = ctx->state[4], f = ctx->state[5], g = ctx->state[6], h = ctx->state[7];
    for (size_t i = 0; i < 64; i++) {
        uint32_t s1 = sha256_rotr(e, 6) ^ sha256_rotr(e, 11) ^ sha256_rotr(e, 25);
        uint32_t ch = (e & f) ^ ((~e) & g);
        uint32_t t1 = h + s1 + ch + k[i] + w[i];
        uint32_t s0 = sha256_rotr(a, 2) ^ sha256_rotr(a, 13) ^ sha256_rotr(a, 22);
        uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
        uint32_t t2 = s0 + maj;
        h = g; g = f; f = e; e = d + t1;
        d = c; c = b; b = a; a = t1 + t2;
    }
    ctx->state[0] += a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d;
    ctx->state[4] += e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h;
}

static void sha256_init(Sha256Ctx* ctx) {
    static const uint32_t initial[8] = {
        0x6a09e667u,0xbb67ae85u,0x3c6ef372u,0xa54ff53au,
        0x510e527fu,0x9b05688cu,0x1f83d9abu,0x5be0cd19u
    };
    memcpy(ctx->state, initial, sizeof(initial));
    ctx->bit_count = 0;
    ctx->block_len = 0;
}

static void sha256_update(Sha256Ctx* ctx, const uint8_t* data, size_t size) {
    if (size == 0) return;
    ctx->bit_count += (uint64_t)size * 8u;
    while (size > 0) {
        size_t available = 64u - ctx->block_len;
        size_t take = size < available ? size : available;
        memcpy(ctx->block + ctx->block_len, data, take);
        ctx->block_len += take;
        data += take;
        size -= take;
        if (ctx->block_len == 64u) {
            sha256_transform(ctx, ctx->block);
            ctx->block_len = 0;
        }
    }
}

static void sha256_final(Sha256Ctx* ctx, uint8_t digest[32]) {
    ctx->block[ctx->block_len++] = 0x80;
    if (ctx->block_len > 56u) {
        while (ctx->block_len < 64u) ctx->block[ctx->block_len++] = 0;
        sha256_transform(ctx, ctx->block);
        ctx->block_len = 0;
    }
    while (ctx->block_len < 56u) ctx->block[ctx->block_len++] = 0;
    for (int i = 7; i >= 0; i--) {
        ctx->block[ctx->block_len++] = (uint8_t)(ctx->bit_count >> (unsigned)(i * 8));
    }
    sha256_transform(ctx, ctx->block);

    for (size_t i = 0; i < 8; i++) {
        digest[i * 4] = (uint8_t)(ctx->state[i] >> 24);
        digest[i * 4 + 1] = (uint8_t)(ctx->state[i] >> 16);
        digest[i * 4 + 2] = (uint8_t)(ctx->state[i] >> 8);
        digest[i * 4 + 3] = (uint8_t)ctx->state[i];
    }
}

bool sha256_hex(const uint8_t* data, size_t size, char out_hex[65]) {
    static const char digits[] = "0123456789abcdef";
    if (!out_hex || (size > 0 && !data)) return false;
    if ((uint64_t)size > UINT64_MAX / 8u) return false;
    Sha256Ctx ctx;
    uint8_t digest[32];
    sha256_init(&ctx);
    sha256_update(&ctx, data, size);
    sha256_final(&ctx, digest);
```

```
342 |     for (size_t i = 0; i < sizeof(digest); i++) {
343 |         out_hex[i * 2] = digits[digest[i] >> 4];
344 |         out_hex[i * 2 + 1] = digits[digest[i] & 0x0fu];
345 |     }
346 |     out_hex[64] = '\0';
347 |     return true;
348 | }
```

include/pe.h

```
#pragma once
/*
  pe.h - minimal PE32+/x64 parser (no Windows headers)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

#define PE_DOS_MAGIC 0x5A4D      /* 'MZ' */
#define PE_NT_MAGIC  0x00004550  /* 'PE\0\0' */

#define IMAGE_FILE_MACHINE_AMD64 0x8664
#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b

#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3

typedef struct PeSection {
    char     name[9]; /* null-terminated */
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;

typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections; /* owned */

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
void pe_free(PeImage* img);

/* Map RVA -> file offset. Returns false if not mappable. */
bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off);

/* Find section containing RVA (or NULL). */
const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva);

#ifdef __cplusplus
}
#endif
```

src/pe.c

```
#include "pe.h"
#include <stdlib.h>
#include <string.h>

static uint32_t u32_max(uint32_t a, uint32_t b) { return a > b ? a : b; }

static bool is_power_of_two_u32(uint32_t value) {
    return value != 0 && (value & (value - 1u)) == 0;
}

static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {
    uint16_t mz = 0;
    if (!buf_read_u16(b, 0, &mz)) return false;
    if (mz != PE_DOS_MAGIC) return false;
    uint32_t lfanew = 0;
    if (!buf_read_u32(b, 0x3c, &lfanew)) return false;
    *out_lfanew = lfanew;
    return true;
}

static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
    uint64_t t = (uint64_t)a + (uint64_t)b;
    if (t > 0xffffffffu) return false;
    *out = (uint32_t)t;
    return true;
}

static bool ranges_overlap_u64(uint64_t a_begin,
                               uint64_t a_size,
                               uint64_t b_begin,
                               uint64_t b_size) {
    if (a_size == 0 || b_size == 0) return false;
    return a_begin < b_begin + b_size && b_begin < a_begin + a_size;
}

bool pe_parse(PeImage* out, const Buf* b, StrBuf* err) {
    if (!out || !b || !err) return false;
    memset(out, 0, sizeof(*out));

    uint32_t lfanew = 0;
    if (!read_dos_lfanew(b, &lfanew)) {
        sb_append(err, "Not a valid MZ/PE file (DOS header check failed).\n");
        return false;
    }
    if ((size_t)lfanew + 4 + 20 > b->size) {
        sb_append(err, "PE header offset out of range.\n");
        return false;
    }

    uint32_t sig = 0;
    if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
        sb_append(err, "Invalid PE signature.\n");
        return false;
    }

    size_t filehdr = (size_t)lfanew + 4;
    uint16_t machine = 0;
    uint16_t nsects = 0;
    uint16_t opt_size = 0;
    if (!buf_read_u16(b, filehdr + 0, &machine) ||
        !buf_read_u16(b, filehdr + 2, &nsects) ||
        !buf_read_u16(b, filehdr + 16, &opt_size)) {
        sb_append(err, "Failed to read COFF header.\n");
        return false;
    }

    if (machine != IMAGE_FILE_MACHINE_AMD64) {
        sb_appendf(err, "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
        return false;
    }

    size_t opthdr = filehdr + 20;
    if (opthdr + opt_size > b->size) {
        sb_append(err, "Optional header out of range.\n");
        return false;
    }
    if (opt_size < 0x70u) {
        sb_append(err, "PE32+ optional header is too small for required fields.\n");
        return false;
    }

    uint16_t magic = 0;
    if (!buf_read_u16(b, opthdr + 0, &magic)) {
        sb_append(err, "Failed to read optional header magic.\n");
        return false;
    }
    if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
        sb_appendf(err, "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic);
        return false;
    }

    uint64_t image_base = 0;
    uint32_t size_of_image = 0;
    uint32_t size_of_headers = 0;
    uint32_t section_alignment = 0;
    uint32_t file_alignment = 0;
    uint32_t number_of_rva_and_sizes = 0;

    /* PE32+ offsets */
    if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
        !buf_read_u32(b, opthdr + 0x20, &section_alignment) ||
        !buf_read_u32(b, opthdr + 0x24, &file_alignment) ||
        !buf_read_u32(b, opthdr + 0x38, &size_of_image) ||
        !buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||
        !buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {
        sb_append(err, "Failed to read key fields from optional header.\n");
        return false;
    }
    if (size_of_image == 0) {
        sb_append(err, "SizeOfImage is zero.\n");
        return false;
```

```
    }
    if (size_of_headers == 0 || (uint64_t)size_of_headers > (uint64_t)b->size) {
        sb_append(err, "SizeOfHeaders is zero or exceeds the file size.\n");
        return false;
    }
    if (section_alignment == 0 || file_alignment == 0) {
        sb_append(err, "SectionAlignment/FileAlignment must be non-zero.\n");
        return false;
    }
    if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {
        sb_append(err, "DataDirectory too small.\n");
        return false;
    }

    /* DataDirectory starts at 0x70 */
    size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;
    if (dd_rel + 8u > opt_size) {
        sb_append(err, "Exception data directory lies outside the optional header.\n");
        return false;
    }
    size_t dd = opthdr + dd_rel;
    uint32_t exc_rva = 0, exc_sz = 0;
    if (!buf_read_u32(b, dd + 0, &exc_rva) || !buf_read_u32(b, dd + 4, &exc_sz)) {
        sb_append(err, "Failed to read exception directory.\n");
        return false;
    }

    size_t secthdr = opthdr + opt_size;
    if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) {
        sb_append(err, "Section header size overflow.\n");
        return false;
    }
    size_t needed = secthdr + (size_t)nsects * 40u;
    if (needed > b->size) {
        sb_append(err, "Section headers out of range.\n");
        return false;
    }

    PeSection* secs = (PeSection*)calloc(nsects ? (size_t)nsects : 1, sizeof(PeSection));
    if (!secs) {
        sb_append(err, "Out of memory.\n");
        return false;
    }

    for (uint16_t i = 0; i < nsects; i++) {
        size_t sh = secthdr + (size_t)i * 40;
        uint8_t name8[8];
        if (!buf_read(b, sh + 0, name8, 8)) { free(secs); sb_append(err, "Failed to read section name.\n"); return false; }
        memcpy(secs[i].name, name8, 8);
        secs[i].name[8] = 0;

        if (!buf_read_u32(b, sh + 8, &secs[i].virtual_size) ||
            !buf_read_u32(b, sh + 12, &secs[i].virtual_address) ||
            !buf_read_u32(b, sh + 16, &secs[i].raw_size) ||
            !buf_read_u32(b, sh + 20, &secs[i].raw_ptr) ||
            !buf_read_u32(b, sh + 36, &secs[i].characteristics)) {
            free(secs);
            sb_append(err, "Failed to read section header.\n");
            return false;
        }
    }

    size_t raw_oob = 0;
    size_t virtual_oob = 0;
    size_t raw_overlaps = 0;
    size_t virtual_overlaps = 0;
    size_t misaligned = 0;

    bool compact_alignment =
        section_alignment < 0x1000u && file_alignment == section_alignment;
    bool file_alignment_valid =
        is_power_of_two_u32(file_alignment) &&
        ((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||
         compact_alignment);
    if (!file_alignment_valid) misaligned++;
    if (section_alignment < file_alignment) misaligned++;
    if (section_alignment < 0x1000u && file_alignment != section_alignment) {
        misaligned++;
    }
    if ((size_of_image % section_alignment) != 0) misaligned++;
    if ((size_of_headers % file_alignment) != 0 ||
        (uint64_t)size_of_headers < (uint64_t)needed) {
        misaligned++;
    }
    if ((image_base & 0xffffu) != 0) misaligned++;

    for (size_t i = 0; i < (size_t)nsects; i++) {
        const PeSection* s = &secs[i];
        uint64_t raw_end = (uint64_t)s->raw_ptr + (uint64_t)s->raw_size;
        uint64_t virtual_span = (uint64_t)u32_max(s->virtual_size, s->raw_size);
        uint64_t virtual_end = (uint64_t)s->virtual_address + virtual_span;
        if (s->raw_size > 0 && raw_end > (uint64_t)b->size) raw_oob++;
        if (virtual_span > 0 && virtual_end > (uint64_t)size_of_image) virtual_oob++;
        if ((s->virtual_address % section_alignment) != 0 ||
            (s->raw_size > 0 &&
             ((s->raw_ptr % file_alignment) != 0 ||
              (s->raw_size % file_alignment) != 0))) {
            misaligned++;
        }
        if (i > 0 && s->virtual_address <= secs[i - 1].virtual_address) {
            misaligned++;
        }

        for (size_t j = 0; j < i; j++) {
            const PeSection* p = &secs[j];
            if (ranges_overlap_u64(s->raw_ptr, s->raw_size, p->raw_ptr, p->raw_size)) {
                raw_overlaps++;
            }
            if (ranges_overlap_u64(
                    s->virtual_address,
                    u32_max(s->virtual_size, s->raw_size),
                    p->virtual_address,
                    u32_max(p->virtual_size, p->raw_size))) {
                virtual_overlaps++;
            }
```

```
        }
    }

    bool exception_directory_oob = false;
    if (exc_rva != 0 || exc_sz != 0) {
        uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;
        exception_directory_oob =
            exc_rva == 0 || exc_sz == 0 || exc_end > (uint64_t)size_of_image;
    }

    out->ok = true;
    out->is_pe64 = true;
    out->machine = machine;
    out->image_base = image_base;
    out->size_of_image = size_of_image;
    out->size_of_headers = size_of_headers;
    out->section_alignment = section_alignment;
    out->file_alignment = file_alignment;
    out->exception_rva = exc_rva;
    out->exception_size = exc_sz;
    out->file_size = b->size;
    out->num_sections = nsects;
    out->sections = secs;
    out->section_raw_oob = raw_oob;
    out->section_virtual_oob = virtual_oob;
    out->section_raw_overlaps = raw_overlaps;
    out->section_virtual_overlaps = virtual_overlaps;
    out->section_misaligned = misaligned;
    out->exception_directory_oob = exception_directory_oob;
    return true;
}

void pe_free(PeImage* img) {
    if (!img) return;
    free(img->sections);
    memset(img, 0, sizeof(*img));
}

const PeSection* pe_find_section_for_rva(const PeImage* img, uint32_t rva) {
    if (!img || !img->sections) return NULL;
    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* s = &img->sections[i];
        uint32_t size = u32_max(s->virtual_size, s->raw_size);
        uint64_t start = (uint64_t)s->virtual_address;
        uint64_t end = start + (uint64_t)size;
        if ((uint64_t)rva >= start && (uint64_t)rva < end) return s;
    }
    return NULL;
}

bool pe_rva_to_fileoff(const PeImage* img, uint32_t rva, uint32_t* out_off) {
    if (!img || !out_off) return false;
    const PeSection* s = pe_find_section_for_rva(img, rva);
    if (s) {
        uint32_t delta = rva - s->virtual_address;
        if (delta >= s->raw_size) {
            /* RVA points into virtual tail (zero-fill) - not present in file */
            return false;
        }
        uint32_t off = 0;
        if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
        if (img->file_size > 0 && (uint64_t)off >= (uint64_t)img->file_size) return false;
        *out_off = off;
        return true;
    }

    /* RVAs inside SizeOfHeaders map directly to the corresponding file offset. */
    if (img->size_of_headers > 0 &&
        rva < img->size_of_headers &&
        (img->file_size == 0 || (uint64_t)rva < (uint64_t)img->file_size)) {
        *out_off = rva;
        return true;
    }
    return false;
}
```

include/unwind.h

```
#pragma once
/*
  unwind.h - x64 UNWIND_INFO parsing (PE exception directory)
*/

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;

typedef struct UnwindSummary {
    bool     valid;
    uint8_t  version;
    uint8_t  flags;
    uint8_t  prolog_size;
    uint8_t  count_codes;
    uint8_t  frame_reg;
    uint8_t  frame_off; /* scaled by 16 */
    bool     has_handler;
    uint32_t handler_rva;

    bool     has_chain;
    RuntimeFunction chain;

    uint32_t fixed_stack_bytes;     /* estimated fixed stack frame delta: push+alloc+machframe */
    uint32_t pushed_nonvol_count;
    uint16_t saved_nonvol_mask;     /* bits 0..15 => reg numbers */
    uint16_t saved_xmm_mask;        /* bits 0..15 => XMM reg numbers */

    uint32_t anomalies;             /* bitmask of internal anomalies */
    char     parse_error[160];       /* non-empty when unwind_parse returns false */
} UnwindSummary;

enum UnwindFlags {
    UNW_FLAG_EHANDLER  = 0x01,
    UNW_FLAG_UHANDLER  = 0x02,
    UNW_FLAG_CHAININFO = 0x04
};

/* anomalies flags (for heuristic analysis / diagnostics) */
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
    UA_CHAIN_INVALID         = 1u << 11,
    UA_ABI_REGISTER_CLASS    = 1u << 12,
    UA_ABI_RESERVED_OPINFO   = 1u << 13,
    UA_ABI_PUSH_ORDER        = 1u << 14,
    UA_ABI_CHAIN_RESTRICTION = 1u << 15,
    UA_ABI_SAVE_OFFSET       = 1u << 16
};

typedef struct UnwindParseOpts {
    int  max_chain_depth; /* default 8 */
    bool follow_chain;    /* compute + annotate chain info */
    bool strict_abi;      /* flag formal Microsoft x64 unwind ABI violations */
} UnwindParseOpts;

typedef enum RuntimeFunctionReadResult {
    RUNTIME_FUNCTION_READ_OK = 0,
    RUNTIME_FUNCTION_READ_MALFORMED,
    RUNTIME_FUNCTION_READ_OOM
} RuntimeFunctionReadResult;

/* Parse UNWIND_INFO at unwind_rva. Produces summary + optional text of unwind ops. */
bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err);

/* Parse exception directory (RUNTIME_FUNCTION table) into array.
   Caller owns *out (malloc). */
RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
                                  const Buf* b,
                                  const PeImage* img,
                                  RuntimeFunction** out,
                                  size_t* out_count,
                                  StrBuf* err);

bool unwind_read_runtime_functions(const Buf* b,
                                  const PeImage* img,
                                  RuntimeFunction** out,
                                  size_t* out_count,
                                  StrBuf* err);

/* Pretty helpers */
const char* unwind_reg_name(uint8_t reg);
void unwind_flags_to_string(uint8_t flags, char out[64]);

```

```
112 | #ifdef __cplusplus
113 | }
114 | #endif
```

src/unwind.c

```
#include "unwind.h"
#include <stdlib.h>
#include <string.h>

const char* unwind_reg_name(uint8_t reg) {
    /* mapping per MS x64 unwind encoding */
    static const char* names[16] = {
        "RAX","RCX","RDX","RBX","RSP","RBP","RSI","RDI",
        "R8","R9","R10","R11","R12","R13","R14","R15"
    };
    if (reg < 16) return names[reg];
    return "REG?";
}

void unwind_flags_to_string(uint8_t flags, char out[64]) {
    out[0] = 0;
    bool first = true;
    if (flags & UNW_FLAG_EHANDLER) { strcat(out, "EHANDLER"); first = false; }
    if (flags & UNW_FLAG_UHANDLER) { strcat(out, first ? "UHANDLER" : "|UHANDLER"); first = false; }
    if (flags & UNW_FLAG_CHAININFO){ strcat(out, first ? "CHAININFO" : "|CHAININFO"); first = false; }
    if (first) strcpy(out, "0");
}

static bool slots_for_code(uint8_t version,
                           uint8_t op,
                           uint8_t opinfo,
                           bool v2_seen_epilog,
                           uint8_t* out_slots) {
    /* based on documented v1 ops + v2 epilog/spare, and common implementations */
    switch (op) {
        case 0: /* UWOP_PUSH_NONVOL */ *out_slots = 1; return true;
        case 1: /* UWOP_ALLOC_LARGE */ if (opinfo > 1) return false; *out_slots = (uint8_t)(2 + opinfo); return true;
        case 2: /* UWOP_ALLOC_SMALL */ *out_slots = 1; return true;
        case 3: /* UWOP_SET_FPREG */   *out_slots = 1; return true;
        case 4: /* UWOP_SAVE_NONVOL */ *out_slots = 2; return true;
        case 5: /* UWOP_SAVE_NONVOL_FAR */ *out_slots = 3; return true;
        case 6: /* v2 UWOP_EPILOG, v1 UWOP_SAVE_XMM (undoc) */
            if (version == 2) {
                /* The first long-form epilog descriptor has one operand slot.
                   Short-form and subsequent descriptors occupy one slot. */
                *out_slots = (!v2_seen_epilog && (opinfo & 1u) == 0u) ? 2u : 1u;
                return true;
            }
            *out_slots = 2; return true;
        case 7: /* v2 UWOP_SPARE, v1 UWOP_SAVE_XMM_FAR (undoc) */
            if (version == 2) { *out_slots = 1; return true; }
            *out_slots = 3; return true;
        case 8: /* UWOP_SAVE_XMM128 */ *out_slots = 2; return true;
        case 9: /* UWOP_SAVE_XMM128_FAR */ *out_slots = 3; return true;
        case 10: /* UWOP_PUSH_MACHFRAME */ *out_slots = 1; return true;
        default:
            return false;
    }
}

static bool parse_unwindinfo_header(const Buf* b, const PeImage* img, uint32_t unwind_rva,
                                    uint32_t* out_fileoff, uint8_t hdr[4], StrBuf* err) {
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, unwind_rva, &off)) {
        sb_appendf(err, "UnwindInfo RVA 0x%08x not mappable to file offset.\n", unwind_rva);
        return false;
    }
    if ((size_t)off + 4 > b->size) {
        sb_append(err, "UnwindInfo header out of file range.\n");
        return false;
    }
    if (!buf_read(b, off, hdr, 4)) {
        sb_append(err, "Failed to read UnwindInfo header bytes.\n");
        return false;
    }
    *out_fileoff = off;
    return true;
}

static bool read_u16_slot(const Buf* b, size_t off, uint16_t* out) {
    uint8_t tmp[2];
    if (!buf_read(b, off, tmp, 2)) return false;
    *out = (uint16_t)(tmp[0] | ((uint16_t)tmp[1] << 8));
    return true;
}

static uint32_t read_u32_from_slots(uint16_t lo, uint16_t hi) {
    return (uint32_t)lo | ((uint32_t)hi << 16);
}

static bool is_unwindable_gpr(uint8_t reg) {
    /* Real MSVC, Rust and OpenSSL records also describe volatile registers. */
    return reg < 16 && reg != 4;
}

static bool is_unwindable_xmm(uint8_t reg) {
    return reg < 16;
}

static bool is_abi_nonvolatile_gpr(uint8_t reg) {
    return reg == 3 || reg == 5 || reg == 6 || reg == 7 || reg >= 12;
}

static bool is_abi_nonvolatile_xmm(uint8_t reg) {
    return reg >= 6 && reg < 16;
}

static void add_fixed_stack(uint32_t* total, uint32_t amount, uint32_t* anomalies) {
    if (!total || !anomalies) return;
    if (UINT32_MAX - *total < amount) {
        *total = UINT32_MAX;
        *anomalies |= UA_OPERAND_SANITY;
        return;
    }
    *total += amount;
}
```

```
|
| static bool alloc_large_mode0_suspicious(uint16_t scaled8) {
|     uint32_t size = (uint32_t)scaled8 * 8u;
|     return size == 0 || size <= 128u || (size % 8u) != 0;
| }
|
| static bool alloc_large_mode1_suspicious(uint32_t size) {
|     return size == 0 || (size % 8u) != 0 || size < (512u * 1024u);
| }
|
| static bool save_nonvol_far_suspicious(uint32_t off) {
|     return off == 0 || (off % 8u) != 0 || off < (512u * 1024u);
| }
|
| static bool save_xmm_far_suspicious(uint32_t off) {
|     return off == 0 || (off % 16u) != 0 || off < (1024u * 1024u);
| }
|
| static bool parse_chain_rf(const Buf* b, const PeImage* img, size_t fileoff_after_codes,
|                            RuntimeFunction* out_chain) {
|     uint32_t a=0, c=0, u=0;
|     if (!buf_read_u32(b, fileoff_after_codes + 0, &a)) return false;
|     if (!buf_read_u32(b, fileoff_after_codes + 4, &c)) return false;
|     if (!buf_read_u32(b, fileoff_after_codes + 8, &u)) return false;
|     out_chain->begin_rva = a;
|     out_chain->end_rva = c;
|     out_chain->unwind_rva = u;
|     (void)img;
|     return true;
| }
|
| static bool unwind_parse_fail(UnwindSummary* out,
|                               StrBuf* err,
|                               size_t err_start,
|                               const char* fallback) {
|     if (!out) return false;
|     out->valid = false;
|
|     const char* message = fallback ? fallback : "UNWIND_INFO parse failed.";
|     if (err && err->s && err->len > err_start) {
|         message = err->s + err_start;
|     } else if (err && fallback) {
|         sb_appendf(err, "%s\n", fallback);
|     }
|
|     size_t n = strlen(message);
|     while (n > 0 && (message[n - 1] == '\n' || message[n - 1] == '\r')) n--;
|     if (n >= sizeof(out->parse_error)) n = sizeof(out->parse_error) - 1;
|     memcpy(out->parse_error, message, n);
|     out->parse_error[n] = '\0';
|     return false;
| }
|
| bool unwind_parse(const Buf* b,
|                   const PeImage* img,
|                   uint32_t unwind_rva,
|                   const UnwindParseOpts* opts,
|                   UnwindSummary* out,
|                   StrBuf* ops_text,
|                   StrBuf* err) {
|     if (!b || !img || !out || !err) return false;
|     memset(out, 0, sizeof(*out));
|     size_t err_start = err->len;
|     if (ops_text) { ops_text->len = 0; if (ops_text->s) ops_text->s[0]=0; }
|
|     int max_depth = 8;
|     bool follow_chain = true;
|     bool strict_abi = false;
|     if (opts) {
|         if (opts->max_chain_depth > 0) max_depth = opts->max_chain_depth;
|         follow_chain = opts->follow_chain;
|         strict_abi = opts->strict_abi;
|     }
|     if (max_depth > 16) max_depth = 16;
|
|     uint8_t hdr[4];
|     uint32_t uoff = 0;
|     if ((unwind_rva & 3u) != 0) {
|         out->anomalies |= UA_ALIGNMENT;
|     }
|     if (!parse_unwindinfo_header(b, img, unwind_rva, &uoff, hdr, err)) {
|         out->anomalies |= UA_TRUNCATED;
|         return unwind_parse_fail(out, err, err_start, "Failed to read UNWIND_INFO header.");
|     }
|
|     uint8_t ver = (uint8_t)(hdr[0] & 0x7);
|     uint8_t flags = (uint8_t)(hdr[0] >> 3);
|     uint8_t prolog = hdr[1];
|     uint8_t count = hdr[2];
|     uint8_t fr = (uint8_t)(hdr[3] & 0x0f);
|     uint8_t foff = (uint8_t)(hdr[3] >> 4);
|
|     out->version = ver;
|     out->flags = flags;
|     out->prolog_size = prolog;
|     out->count_codes = count;
|     out->frame_reg = fr;
|     out->frame_off = foff;
|
|     if (!(ver == 1 || ver == 2)) {
|         out->anomalies |= UA_VERSION_UNKNOWN;
|         return unwind_parse_fail(out, err, err_start, "Unsupported UNWIND_INFO version.");
|     }
|     if (flags & (uint8_t)~(UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER | UNW_FLAG_CHAININFO)) {
|         out->anomalies |= UA_BAD_OPERAND;
|     }
|     if ((flags & UNW_FLAG_CHAININFO) && (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER))) {
|         out->anomalies |= UA_BAD_OPERAND;
|     }
|     if (fr == 0 && foff != 0) {
|         out->anomalies |= UA_BAD_OPERAND;
|     }
|     if (fr != 0 && !is_unwindable_gpr(fr)) {
|         out->anomalies |= UA_BAD_OPERAND;
|     }
```

```
227 |     if (strict_abi && fr != 0 && !is_abi_nonvolatile_gpr(fr)) {
228 |         out->anomalies |= UA_ABI_REGISTER_CLASS;
229 |     }
230 |
231 |     /* total slots stored on disk (aligned to even number) */
232 |     uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);
233 |     size_t codes_off = (size_t)uoff + 4;
234 |     size_t codes_bytes = (size_t)slots_even * 2u;
235 |     if (codes_off + codes_bytes > b->size) {
236 |         out->anomalies |= UA_TRUNCATED;
237 |         sb_append(err, "Unwind codes array out of file range.\n");
238 |         return unwind_parse_fail(out, err, err_start, "Unwind codes array out of file range.");
239 |     }
240 |
241 |     uint32_t fixed_stack = 0;
242 |     uint32_t pushed = 0;
243 |     uint16_t nonvol_mask = 0;
244 |     uint16_t xmm_mask = 0;
245 |     bool have_prev_code_offset = false;
246 |     uint8_t prev_code_offset = 0;
247 |     bool saw_set_fpreg = false;
248 |     bool saw_push_nonvol = false;
249 |     bool v2_seen_epilog = false;
250 |
251 |     /* parse slots 0..count-1 as unwind code starts */
252 |     for (uint32_t i = 0; i < count; ) {
253 |         uint16_t slot = 0;
254 |         if (!read_u16_slot(b, codes_off + (size_t)i * 2, &slot)) {
255 |             out->anomalies |= UA_TRUNCATED;
256 |             sb_append(err, "Failed to read unwind slot.\n");
257 |             return unwind_parse_fail(out, err, err_start, "Failed to read unwind slot.");
258 |         }
259 |
260 |         uint8_t code_offset = (uint8_t)(slot & 0xff);
261 |         uint8_t op_byte = (uint8_t)(slot >> 8);
262 |         uint8_t op = (uint8_t)(op_byte & 0x0f);
263 |         uint8_t opinfo = (uint8_t)(op_byte >> 4);
264 |
265 |         uint8_t slots = 0;
266 |         if (!slots_for_code(ver, op, opinfo, v2_seen_epilog, &slots)) {
267 |             out->anomalies |= UA_BAD_OPCODE;
268 |             /* try to continue safely as 1 slot */
269 |             slots = 1;
270 |         }
271 |         if (i + slots > count) {
272 |             out->anomalies |= UA_TRUNCATED;
273 |             sb_append(err, "Unwind opcode spans beyond CountOfCodes.\n");
274 |             return unwind_parse_fail(out, err, err_start, "Unwind opcode spans beyond CountOfCodes.");
275 |         }
276 |         bool v2_descriptor = ver == 2 && (op == 6 || op == 7);
277 |         if (!v2_descriptor) {
278 |             if (code_offset > prolog) {
279 |                 out->anomalies |= UA_CODEOFFSET_OUTSIDE;
280 |             }
281 |             if (have_prev_code_offset && code_offset > prev_code_offset) {
282 |                 out->anomalies |= UA_CODEOFFSET_ORDER;
283 |             }
284 |             have_prev_code_offset = true;
285 |             prev_code_offset = code_offset;
286 |         }
287 |         if (strict_abi && !v2_descriptor) {
288 |             if (saw_push_nonvol && op != 0 && op != 10) {
289 |                 out->anomalies |= UA_ABI_PUSH_ORDER;
290 |             }
291 |             if (op == 0) saw_push_nonvol = true;
292 |             if ((flags & UNW_FLAG_CHAININFO) && op != 4) {
293 |                 out->anomalies |= UA_ABI_CHAIN_RESTRICTION;
294 |             }
295 |         }
296 |
297 |         if (ops_text) {
298 |             /* print one-line per op start */
299 |             switch (op) {
300 |                 case 0: sb_appendf(ops_text, "  @%02x PUSH_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;
301 |                 case 1: sb_appendf(ops_text, "  @%02x ALLOC_LARGE mode=%u\n", code_offset, (unsigned)opinfo); break;
302 |                 case 2: sb_appendf(ops_text, "  @%02x ALLOC_SMALL %u\n", code_offset, (unsigned)(opinfo*8u + 8u)); break;
303 |                 case 3: sb_appendf(ops_text, "  @%02x SET_FPREG %s + %u\n", code_offset, unwind_reg_name(fr), (unsigned)(foff*16u));
    break;
304 |                 case 4: sb_appendf(ops_text, "  @%02x SAVE_NONVOL %s\n", code_offset, unwind_reg_name(opinfo)); break;
305 |                 case 5: sb_appendf(ops_text, "  @%02x SAVE_NONVOL_FAR %s\n", code_offset, unwind_reg_name(opinfo)); break;
306 |                 case 6:
307 |                     if (ver == 2) {
308 |                         if (!v2_seen_epilog) {
309 |                             if ((opinfo & 1u) == 0u) {
310 |                                 uint16_t epilog_slot = 0;
311 |                                 if (read_u16_slot(b, codes_off + (size_t)(i + 1) * 2,
312 |                                                   &epilog_slot)) {
313 |                                     unsigned epilog_offset =
314 |                                         (unsigned)(epilog_slot & 0xffu) |
315 |                                         ((unsigned)(epilog_slot >> 12) << 8);
316 |                                     sb_appendf(ops_text,
317 |                                                "  EPILOG size=%u offset_from_end=%u flags=%u (v2)\n",
318 |                                                (unsigned)code_offset, epilog_offset,
319 |                                                (unsigned)opinfo);
320 |                                 }
321 |                             } else {
322 |                                 sb_appendf(ops_text, "  EPILOG size=%u flags=%u (v2)\n",
323 |                                            (unsigned)code_offset, (unsigned)opinfo);
324 |                             }
325 |                         } else {
326 |                             unsigned epilog_offset =
327 |                                 (unsigned)code_offset | ((unsigned)opinfo << 8);
328 |                             sb_appendf(ops_text, "  EPILOG offset_from_end=%u (v2)\n",
329 |                                        epilog_offset);
330 |                         }
331 |                     } else {
332 |                         sb_appendf(ops_text, "  @%02x SAVE_XMM (undoc/v1)\n", code_offset);
333 |                     }
334 |                     break;
335 |                 case 7:
336 |                     if (ver == 2) sb_appendf(ops_text, "  @%02x SPARE (v2)\n", code_offset);
337 |                     else sb_appendf(ops_text, "  @%02x SAVE_XMM_FAR (undoc/v1)\n", code_offset);
338 |                     break;
339 |                 case 8: sb_appendf(ops_text, "  @%02x SAVE_XMM128 XMM%u\n", code_offset, (unsigned)opinfo); break;
340 |                 case 9: sb_appendf(ops_text, "  @%02x SAVE_XMM128_FAR XMM%u\n", code_offset, (unsigned)opinfo); break;
```

```
                case 10: sb_appendf(ops_text, "  @%02x PUSH_MACHFRAME err=%u\n", code_offset, (unsigned)opinfo); break;
                default: sb_appendf(ops_text, "  @%02x OP_%u info=%u\n", code_offset, (unsigned)op, (unsigned)opinfo); break;
            }
        }

        /* compute effects */
        switch (op) {
            case 0: /* push nonvol */
                add_fixed_stack(&fixed_stack, 8, &out->anomalies);
                pushed++;
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            case 1: { /* alloc large */
                if (opinfo == 0) {
                    uint16_t scaled = 0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");
                    }
                    add_fixed_stack(&fixed_stack, (uint32_t)scaled * 8u, &out->anomalies);
                    if (alloc_large_mode0_suspicious(scaled)) out->anomalies |= UA_OPERAND_SANITY;
                    if (ops_text) sb_appendf(ops_text, "       size=%u\n", (unsigned)((uint32_t)scaled * 8u));
                } else if (opinfo == 1) {
                    uint16_t lo=0, hi=0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                        !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_ALLOC_LARGE operand.");
                    }
                    uint32_t size = read_u32_from_slots(lo, hi);
                    add_fixed_stack(&fixed_stack, size, &out->anomalies);
                    if (alloc_large_mode1_suspicious(size)) out->anomalies |= UA_OPERAND_SANITY;
                    if (ops_text) sb_appendf(ops_text, "       size=%u\n", (unsigned)size);
                } else {
                    out->anomalies |= UA_BAD_OPERAND;
                }
                break;
            }
            case 2: /* alloc small */
                add_fixed_stack(&fixed_stack, (uint32_t)opinfo * 8u + 8u, &out->anomalies);
                break;
            case 3: /* set frame reg */
                if (fr == 0 || saw_set_fpreg) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && opinfo != 0) {
                    out->anomalies |= UA_ABI_RESERVED_OPINFO;
                }
                saw_set_fpreg = true;
                /* no fixed stack delta */
                break;
            case 4: { /* save nonvol */
                uint16_t scaled = 0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL operand.");
                }
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && scaled == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 5: { /* save nonvol far */
                uint16_t lo=0, hi=0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                    !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_NONVOL_FAR operand.");
                }
                uint32_t save_offset = read_u32_from_slots(lo, hi);
                if (save_nonvol_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
                if (!is_unwindable_gpr(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_gpr(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && save_offset == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) nonvol_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 6: { /* save xmm (v1, undocumented encoding) or epilog (v2) */
                if (ver != 2) {
                    uint16_t scaled = 0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM operand.");
                    }
                    if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                    if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                        out->anomalies |= UA_ABI_REGISTER_CLASS;
                    }
                    if (strict_abi && scaled == 0) {
                        out->anomalies |= UA_ABI_SAVE_OFFSET;
                    }
                    if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                }
                break;
            }
            case 7: { /* save xmm far (v1, undocumented encoding) or spare (v2) */
                if (ver != 2) {
                    uint16_t lo=0, hi=0;
                    if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                        !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                        out->anomalies |= UA_TRUNCATED;
                        return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM_FAR operand.");
                    }
                    uint32_t save_offset = read_u32_from_slots(lo, hi);
```

```
                    if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
                    if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                    if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                        out->anomalies |= UA_ABI_REGISTER_CLASS;
                    }
                    if (strict_abi && save_offset == 0) {
                        out->anomalies |= UA_ABI_SAVE_OFFSET;
                    }
                    if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                }
                break;
            }
            case 8: { /* save xmm128 */
                uint16_t scaled = 0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &scaled)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128 operand.");
                }
                if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && scaled == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 9: { /* save xmm128 far */
                uint16_t lo=0, hi=0;
                if (!read_u16_slot(b, codes_off + (size_t)(i+1) * 2, &lo) ||
                    !read_u16_slot(b, codes_off + (size_t)(i+2) * 2, &hi)) {
                    out->anomalies |= UA_TRUNCATED;
                    return unwind_parse_fail(out, err, err_start, "Truncated UWOP_SAVE_XMM128_FAR operand.");
                }
                uint32_t save_offset = read_u32_from_slots(lo, hi);
                if (save_xmm_far_suspicious(save_offset)) out->anomalies |= UA_OPERAND_SANITY;
                if (!is_unwindable_xmm(opinfo)) out->anomalies |= UA_BAD_OPERAND;
                if (strict_abi && !is_abi_nonvolatile_xmm(opinfo)) {
                    out->anomalies |= UA_ABI_REGISTER_CLASS;
                }
                if (strict_abi && save_offset == 0) {
                    out->anomalies |= UA_ABI_SAVE_OFFSET;
                }
                if (opinfo < 16) xmm_mask |= (uint16_t)(1u << opinfo);
                break;
            }
            case 10: /* push machframe */
                if (opinfo > 1) out->anomalies |= UA_BAD_OPERAND;
                add_fixed_stack(&fixed_stack, (opinfo == 0) ? 40u : 48u, &out->anomalies);
                break;
            default:
                /* ignore */
                break;
        }

        if (ver == 2 && op == 6) v2_seen_epilog = true;
        i += slots;
    }
    if (fr != 0 && !saw_set_fpreg && !(flags & UNW_FLAG_CHAININFO)) {
        out->anomalies |= UA_BAD_OPERAND;
    }

    /* tail (handler or chained info) */
    size_t after_codes = codes_off + codes_bytes;

    if (flags & UNW_FLAG_CHAININFO) {
        out->has_chain = true;
        RuntimeFunction chain = {0};
        if (!parse_chain_rf(b, img, after_codes, &chain)) {
            out->anomalies |= UA_TRUNCATED;
            return unwind_parse_fail(out, err, err_start, "Truncated CHAININFO RUNTIME_FUNCTION.");
        } else {
            out->chain = chain;
            if (chain.end_rva <= chain.begin_rva ||
                chain.begin_rva >= img->size_of_image ||
                chain.end_rva > img->size_of_image ||
                chain.unwind_rva == 0 ||
                chain.unwind_rva >= img->size_of_image) {
                out->anomalies |= UA_CHAIN_INVALID;
            }
        }
    }

    if (flags & (UNW_FLAG_EHANDLER | UNW_FLAG_UHANDLER)) {
        out->has_handler = true;
        uint32_t handler_rva = 0;
        if (!buf_read_u32(b, after_codes + 0, &handler_rva)) {
            out->anomalies |= UA_TRUNCATED;
            return unwind_parse_fail(out, err, err_start, "Truncated exception handler RVA.");
        } else {
            out->handler_rva = handler_rva;
            if (handler_rva >= img->size_of_image) {
                out->anomalies |= UA_HANDLER_OOB;
            }
        }
    }

    out->fixed_stack_bytes = fixed_stack;
    out->pushed_nonvol_count = pushed;
    out->saved_nonvol_mask = nonvol_mask;
    out->saved_xmm_mask = xmm_mask;

    /* follow chain (lightly): detect loops and depth. */
    if (follow_chain && out->has_chain) {
        uint32_t seen[16]; /* depth max 16 */
        int seen_n = 0;
        uint32_t cur = out->chain.unwind_rva;
        for (int depth = 0; depth < max_depth; depth++) {
            bool loop = false;
            for (int j = 0; j < seen_n; j++) if (seen[j] == cur) loop = true;
            if (loop) { out->anomalies |= UA_CHAIN_LOOP; break; }
            if (seen_n < (int)(sizeof(seen)/sizeof(seen[0]))) seen[seen_n++] = cur;

            UnwindSummary tmp;
```

```
            StrBuf tmp_ops = {0};
            StrBuf tmp_err = {0};
            sb_init(&tmp_err, 256);
            sb_init(&tmp_ops, 256);
            UnwindParseOpts o2 = {
                .max_chain_depth = 0,
                .follow_chain = false,
                .strict_abi = strict_abi
            };
            if (!unwind_parse(b, img, cur, &o2, &tmp, NULL, &tmp_err)) {
                out->anomalies |= UA_TRUNCATED;
                sb_free(&tmp_err);
                sb_free(&tmp_ops);
                break;
            }
            /* combine fixed stack heuristically */
            add_fixed_stack(&out->fixed_stack_bytes, tmp.fixed_stack_bytes, &out->anomalies);
            out->pushed_nonvol_count += tmp.pushed_nonvol_count;
            out->saved_nonvol_mask |= tmp.saved_nonvol_mask;
            out->saved_xmm_mask |= tmp.saved_xmm_mask;
            if (tmp.frame_reg != out->frame_reg || tmp.frame_off != out->frame_off) {
                out->anomalies |= UA_BAD_OPERAND;
            }
            if (!out->has_handler && tmp.has_handler) {
                out->has_handler = true;
                out->handler_rva = tmp.handler_rva;
            }
            sb_free(&tmp_err);
            sb_free(&tmp_ops);

            if (!tmp.has_chain) break;
            cur = tmp.chain.unwind_rva;
            if (depth == max_depth-1) out->anomalies |= UA_CHAIN_DEPTH_EXCEEDED;
        }
    }

    out->valid = true;
    return true;
}

RuntimeFunctionReadResult unwind_read_runtime_functions_ex(
                                const Buf* b,
                                const PeImage* img,
                                RuntimeFunction** out_rf,
                                size_t* out_count,
                                StrBuf* err) {
    if (!b || !img || !out_rf || !out_count || !err) {
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    *out_rf = NULL;
    *out_count = 0;

    if (img->exception_rva == 0 && img->exception_size == 0) {
        return RUNTIME_FUNCTION_READ_OK;
    }
    if (img->exception_rva == 0 || img->exception_size == 0) {
        sb_append(err, "Malformed exception directory: RVA and size must both be zero or non-zero.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    uint32_t off = 0;
    if (!pe_rva_to_fileoff(img, img->exception_rva, &off)) {
        sb_appendf(err, "Exception directory RVA 0x%08x not mappable.\n", img->exception_rva);
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((size_t)off + img->exception_size > b->size) {
        sb_append(err, "Exception directory out of file range.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if (img->exception_size < 12) {
        sb_append(err, "Exception directory size too small.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    if ((img->exception_size % 12u) != 0) {
        sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");
        return RUNTIME_FUNCTION_READ_MALFORMED;
    }
    size_t n = img->exception_size / 12;
    RuntimeFunction* rf = (RuntimeFunction*)calloc(n, sizeof(RuntimeFunction));
    if (!rf) {
        sb_append(err, "Out of memory.\n");
        return RUNTIME_FUNCTION_READ_OOM;
    }

    for (size_t i = 0; i < n; i++) {
        size_t e = (size_t)off + i*12;
        uint32_t bva=0, eva=0, uva=0;
        if (!buf_read_u32(b, e + 0, &bva) ||
            !buf_read_u32(b, e + 4, &eva) ||
            !buf_read_u32(b, e + 8, &uva)) {
            free(rf);
            sb_append(err, "Failed to read runtime function entry.\n");
            return RUNTIME_FUNCTION_READ_MALFORMED;
        }
        rf[i].begin_rva = bva;
        rf[i].end_rva = eva;
        rf[i].unwind_rva = uva;
    }

    *out_rf = rf;
    *out_count = n;
    return RUNTIME_FUNCTION_READ_OK;
}

bool unwind_read_runtime_functions(const Buf* b,
                                const PeImage* img,
                                RuntimeFunction** out_rf,
                                size_t* out_count,
                                StrBuf* err) {
    return unwind_read_runtime_functions_ex(
                b, img, out_rf, out_count, err) == RUNTIME_FUNCTION_READ_OK;
}
```

include/heuristics.h

```
#pragma once
/*
  heuristics.h - heuristic assessment of broken / shadowed / low-informative unwind metadata
*/

#include <stddef.h>
#include <stdbool.h>
#include <stdint.h>
#include "pe.h"
#include "unwind.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

enum HeuristicEvidenceFamily {
    HEF_TABLE_INTEGRITY    = 1u << 0,
    HEF_UNWIND_INTEGRITY   = 1u << 1,
    HEF_METADATA_PLACEMENT = 1u << 2,
    HEF_UNWIND_REUSE       = 1u << 3,
    HEF_HANDLER_DOMINANCE  = 1u << 4
};

enum { HEURISTICS_MAX_FINDINGS = 32, HEURISTICS_MAX_MITIGATIONS = 16 };

typedef struct HeuristicFinding {
    char code[40];
    char family[32];
    char severity[16];
    char message[192];
    bool has_location;
    uint32_t representative_rva;
    size_t affected_count;
} HeuristicFinding;

typedef struct HeuristicMitigation {
    char code[40];
    char message[192];
} HeuristicMitigation;

typedef struct HeuristicReport {
    int    score; /* 0..100 (higher => more suspicious) */
    double suspicion;  /* 0..1 derived from score */
    double confidence; /* 0..1 confidence in the static verdict */
    bool   requires_manual_review;
    uint32_t evidence_families_mask;
    char   verdict[48];

    size_t total;

    size_t pe_section_raw_oob;
    size_t pe_section_virtual_oob;
    size_t pe_section_raw_overlaps;
    size_t pe_section_virtual_overlaps;
    size_t pe_section_misaligned;
    bool   pe_exception_directory_oob;

    size_t rf_invalid_ranges;
    size_t rf_oob;
    size_t rf_unwind_oob;
    size_t rf_unsorted;
    size_t rf_overlaps;
    size_t rf_duplicate_begins;

    size_t unwind_parse_fail;
    size_t unwind_anomalies;
    size_t unwind_abi_warnings;

    size_t unwind_unusual_section;
    size_t unwind_exec_section;
    size_t unwind_writable_section;
    size_t handler_nonexec_section;
    size_t handler_writable_section;

    double unwind_reuse_ratio;    /* max_count / total */
    double unwind_unique_ratio;   /* unique / total */

    double handler_reuse_ratio;   /* max_count / handler_count (if any) */
    size_t handler_count;

    size_t finding_count;
    HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];
    size_t weak_finding_count;
    size_t medium_finding_count;
    size_t strong_finding_count;
    char evidence_level[16];

    size_t mitigation_count;
    HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];

    StrBuf details; /* human-readable findings */
} HeuristicReport;

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err);

void heuristics_free(HeuristicReport* r);
const char* heuristics_evidence_family_name(uint32_t bit);
size_t heuristics_evidence_family_count(uint32_t mask);

#ifdef __cplusplus
}
#endif
```

src/heuristics.c

```c
#include "heuristics.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <stdarg.h>

#ifndef IMAGE_SCN_MEM_EXECUTE
#define IMAGE_SCN_MEM_EXECUTE 0x20000000u
#endif
#ifndef IMAGE_SCN_MEM_WRITE
#define IMAGE_SCN_MEM_WRITE 0x80000000u
#endif

#define UA_ABI_MASK \
    (UA_ABI_REGISTER_CLASS | UA_ABI_RESERVED_OPINFO | UA_ABI_PUSH_ORDER | \
     UA_ABI_CHAIN_RESTRICTION | UA_ABI_SAVE_OFFSET)


/*
 * Heuristic calibration notes
 * ---------------------------
 * These constants intentionally form a conservative engineering model rather
 * than a mathematically validated classifier. The weights are grouped by the
 * evidence family discussed in the thesis so the rationale stays stable and
 * reviewable:
 *   - table/unwind integrity violations => strongest static weights
 *   - placement anomalies => medium/high depending on prevalence
 *   - reuse/dominance patterns => moderate unless they are very strong
 *   - mitigations reduce confidence when evidence is isolated or sample size is tiny
 */
static const int SCORE_EMPTY_PDATA = 10;
static const int SCORE_PE_SECTION_RAW_OOB = 25;
static const int SCORE_PE_SECTION_VIRTUAL_OOB = 20;
static const int SCORE_PE_SECTION_RAW_OVERLAP = 15;
static const int SCORE_PE_SECTION_VIRTUAL_OVERLAP = 20;
static const int SCORE_PE_SECTION_MISALIGNED = 8;
static const int SCORE_PE_EXCEPTION_DIRECTORY_OOB = 25;
static const int SCORE_INVALID_RANGE_LOW = 10;
static const int SCORE_INVALID_RANGE_HIGH = 20;
static const int SCORE_RF_OOB_LOW = 10;
static const int SCORE_RF_OOB_HIGH = 25;
static const int SCORE_UNWIND_RVA_OOB_LOW = 10;
static const int SCORE_UNWIND_RVA_OOB_HIGH = 25;
static const int SCORE_RF_UNSORTED = 15;
static const int SCORE_RF_OVERLAP = 25;
static const int SCORE_DUPLICATE_BEGIN = 10;
static const int SCORE_UNWIND_PARSE_FAIL_LOW = 10;
static const int SCORE_UNWIND_PARSE_FAIL_HIGH = 25;
static const int SCORE_UNWIND_ANOM_LOW = 10;
static const int SCORE_UNWIND_ANOM_HIGH = 20;
static const int SCORE_UNWIND_REUSE_MEDIUM = 10;
static const int SCORE_UNWIND_REUSE_HIGH = 20;
static const int SCORE_HANDLER_DOMINANCE = 15;
static const int SCORE_UNWIND_EXEC_SECTION = 20;
static const int SCORE_UNWIND_UNUSUAL_SECTION_LOW = 8;
static const int SCORE_UNWIND_UNUSUAL_SECTION_HIGH = 15;
static const int SCORE_UNWIND_WRITABLE_SECTION_LOW = 10;
static const int SCORE_UNWIND_WRITABLE_SECTION_HIGH = 18;
static const int SCORE_HANDLER_NONEXEC = 15;
static const int SCORE_HANDLER_WRITABLE = 10;
static const int SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER = 10;
static const int SCORE_COMBO_PLACEMENT_AND_INTEGRITY = 8;
static const int SCORE_COMBO_REUSE_AND_DOMINANCE = 8;
static const int SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER = 10;

static const double CONFIDENCE_BASE = 0.15;
static const double CONFIDENCE_ZERO_SCORE_BASE = 0.20;
static const double CONFIDENCE_PER_FAMILY = 0.11;
static const double CONFIDENCE_PER_STRONG = 0.08;
static const double CONFIDENCE_PER_MEDIUM = 0.04;
static const double CONFIDENCE_BONUS_TABLE_HARD = 0.08;
static const double CONFIDENCE_BONUS_UNWIND_HARD = 0.08;
static const double CONFIDENCE_PENALTY_SINGLE_SIGNAL = 0.12;
static const double CONFIDENCE_PENALTY_SMALL_SAMPLE = 0.06;
static const double CONFIDENCE_PENALTY_PLACEMENT_ONLY = 0.10;
static const double CONFIDENCE_PENALTY_SINGLE_FAMILY = 0.08;
static const double CONFIDENCE_MIN = 0.05;
static const double CONFIDENCE_MAX = 0.90;

static const double THRESHOLD_HIGH_PERCENT = 0.01;   /* >1% of entries => not just a one-off glitch */
static const double THRESHOLD_UNWIND_ANOM_HIGH = 0.05; /* >5% anomalies => broad integrity issue */
static const double THRESHOLD_UNWIND_REUSE_MEDIUM = 0.10;
static const double THRESHOLD_UNWIND_REUSE_HIGH = 0.30;
static const double THRESHOLD_HANDLER_DOMINANCE = 0.80;

static bool sec_name_eq(const char* a, const char* b) {
    if (!a || !b) return false;
    while (*a && *b) {
        char ca = *a;
        char cb = *b;
        if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');
        if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');
        if (ca != cb) return false;
        a++; b++;
    }
    return *a == 0 && *b == 0;
}

static bool sec_name_contains_ci(const char* haystack, const char* needle) {
    if (!haystack || !needle || !*needle) return false;
    size_t nlen = strlen(needle);
    for (const char* h = haystack; *h; h++) {
        size_t i = 0;
        while (i < nlen) {
            char ca = h[i];
            char cb = needle[i];
            if (!ca) return false;
            if (ca >= 'A' && ca <= 'Z') ca = (char)(ca - 'A' + 'a');
            if (cb >= 'A' && cb <= 'Z') cb = (char)(cb - 'A' + 'a');
            if (ca != cb) break;
            i++;
```

```
        }
        if (i == nlen) return true;
    }
    return false;
}

static bool sec_name_allowed_for_unwind(const char* name8) {
    /* Common placements: .xdata (MSVC), .rdata/.pdata in practice, and toolchain-specific xdata-like sections. */
    return sec_name_eq(name8, ".xdata") || sec_name_eq(name8, ".rdata") || sec_name_eq(name8, ".pdata") ||
           sec_name_eq(name8, ".gnu_xdata") || sec_name_eq(name8, ".mrdata") ||
           sec_name_eq(name8, "xdata") || sec_name_eq(name8, "rdata") || sec_name_eq(name8, "pdata");
}

static bool sec_name_plausibly_unwind_named(const char* name8) {
    return sec_name_contains_ci(name8, "xdata") || sec_name_contains_ci(name8, "pdata") || sec_name_contains_ci(name8, "rdata");
}

static bool sec_name_code_like(const char* name8) {
    return sec_name_eq(name8, ".text") || sec_name_eq(name8, "text") ||
           sec_name_contains_ci(name8, "text") || sec_name_contains_ci(name8, "code") || sec_name_contains_ci(name8, "init");
}

static bool image_has_executable_section(const PeImage* img) {
    if (!img || !img->sections) return false;
    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* section = &img->sections[i];
        if ((section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0 &&
            (section->virtual_size != 0 || section->raw_size != 0)) {
            return true;
        }
    }
    return false;
}

/* Very small u32->count hash map (open addressing, linear probing). */
typedef struct HEntry {
    uint32_t key;
    uint32_t val;
    bool used;
} HEntry;

typedef struct HMap {
    HEntry* e;
    size_t cap;
    size_t len;
} HMap;

static uint32_t h32(uint32_t x) {
    /* xorshift-ish */
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static bool hmap_init(HMap* m, size_t cap) {
    if (!m) return false;
    m->cap = cap < 64 ? 64 : cap;
    m->len = 0;
    m->e = (HEntry*)calloc(m->cap, sizeof(HEntry));
    return m->e != NULL;
}

static void hmap_free(HMap* m) {
    if (!m) return;
    free(m->e);
    m->e = NULL;
    m->cap = 0;
    m->len = 0;
}

static bool hmap_grow(HMap* m);

static bool hmap_inc(HMap* m, uint32_t key, uint32_t* out_newval) {
    if (!m || !m->e) return false;
    if ((m->len + 1) * 10 >= m->cap * 7) {
        if (!hmap_grow(m)) return false;
    }
    uint32_t hv = h32(key);
    size_t idx = (size_t)hv % m->cap;
    for (size_t step = 0; step < m->cap; step++) {
        HEntry* ent = &m->e[idx];
        if (!ent->used) {
            ent->used = true;
            ent->key = key;
            ent->val = 1;
            m->len++;
            if (out_newval) *out_newval = 1;
            return true;
        }
        if (ent->key == key) {
            ent->val++;
            if (out_newval) *out_newval = ent->val;
            return true;
        }
        idx = (idx + 1) % m->cap;
    }
    return false;
}

static bool hmap_grow(HMap* m) {
    size_t oldcap = m->cap;
    HEntry* old = m->e;
    if (oldcap > (SIZE_MAX - 1u) / 2u) return false;
    size_t newcap = oldcap * 2 + 1;
    HEntry* ne = (HEntry*)calloc(newcap, sizeof(HEntry));
    if (!ne) return false;

    m->e = ne;
    m->cap = newcap;
    m->len = 0;

    for (size_t i = 0; i < oldcap; i++) {
```

```
        if (!old[i].used) continue;
        /* reinsert */
        uint32_t key = old[i].key;
        uint32_t val = old[i].val;
        uint32_t hv = h32(key);
        size_t idx = (size_t)hv % m->cap;
        for (;;) {
            HEntry* ent = &m->e[idx];
            if (!ent->used) {
                ent->used = true;
                ent->key = key;
                ent->val = val;
                m->len++;
                break;
            }
            idx = (idx + 1) % m->cap;
        }
    }
    free(old);
    return true;
}

static uint32_t hmap_max_val(const HMap* m) {
    uint32_t mx = 0;
    if (!m || !m->e) return 0;
    for (size_t i = 0; i < m->cap; i++) {
        if (m->e[i].used && m->e[i].val > mx) mx = m->e[i].val;
    }
    return mx;
}

static bool hmap_max_entry(const HMap* m, uint32_t* out_key, uint32_t* out_val) {
    uint32_t best_key = 0;
    uint32_t best_val = 0;
    if (!m || !m->e) return false;
    for (size_t i = 0; i < m->cap; i++) {
        if (!m->e[i].used) continue;
        if (m->e[i].val > best_val) {
            best_key = m->e[i].key;
            best_val = m->e[i].val;
        }
    }
    if (best_val == 0) return false;
    if (out_key) *out_key = best_key;
    if (out_val) *out_val = best_val;
    return true;
}

static size_t hmap_unique(const HMap* m) { return m ? m->len : 0; }

typedef struct RfIdx {
    uint32_t begin;
    uint32_t end;
} RfIdx;

typedef struct FindingLocation {
    bool set;
    uint32_t rva;
} FindingLocation;

static void remember_location(FindingLocation* location, uint32_t rva) {
    if (!location || location->set) return;
    location->set = true;
    location->rva = rva;
}

enum PeSectionIssue {
    PE_ISSUE_RAW_OOB,
    PE_ISSUE_VIRTUAL_OOB,
    PE_ISSUE_RAW_OVERLAP,
    PE_ISSUE_VIRTUAL_OVERLAP,
    PE_ISSUE_MISALIGNED
};

static FindingLocation first_pe_section_issue(const PeImage* img, enum PeSectionIssue issue) {
    FindingLocation result = {0};
    if (!img || !img->sections) return result;

    for (size_t i = 0; i < img->num_sections; i++) {
        const PeSection* s = &img->sections[i];
        uint64_t span = s->virtual_size > s->raw_size ? s->virtual_size : s->raw_size;
        bool match = false;
        if (issue == PE_ISSUE_RAW_OOB) {
            match = s->raw_size > 0 &&
                    ((uint64_t)s->raw_ptr + (uint64_t)s->raw_size > (uint64_t)img->file_size);
        } else if (issue == PE_ISSUE_VIRTUAL_OOB) {
            match = span > 0 &&
                    ((uint64_t)s->virtual_address + span > (uint64_t)img->size_of_image);
        } else if (issue == PE_ISSUE_MISALIGNED) {
            match = (img->section_alignment > 0 &&
                     (s->virtual_address % img->section_alignment) != 0) ||
                    (s->raw_size > 0 && img->file_alignment > 0 &&
                     (s->raw_ptr % img->file_alignment) != 0);
        } else {
            for (size_t j = 0; j < i && !match; j++) {
                const PeSection* p = &img->sections[j];
                uint64_t pspan = p->virtual_size > p->raw_size ? p->virtual_size : p->raw_size;
                if (issue == PE_ISSUE_RAW_OVERLAP) {
                    match = s->raw_size > 0 && p->raw_size > 0 &&
                            (uint64_t)s->raw_ptr < (uint64_t)p->raw_ptr + p->raw_size &&
                            (uint64_t)p->raw_ptr < (uint64_t)s->raw_ptr + s->raw_size;
                } else if (issue == PE_ISSUE_VIRTUAL_OVERLAP) {
                    match = span > 0 && pspan > 0 &&
                            (uint64_t)s->virtual_address < (uint64_t)p->virtual_address + pspan &&
                            (uint64_t)p->virtual_address < (uint64_t)s->virtual_address + span;
                }
            }
        }
        if (match) {
            remember_location(&result, s->virtual_address);
            break;
        }
    }
    return result;
}
```

```
|
| static int cmp_rfidx(const void* a, const void* b) {
|     const RfIdx* x = (const RfIdx*)a;
|     const RfIdx* y = (const RfIdx*)b;
|     if (x->begin < y->begin) return -1;
|     if (x->begin > y->begin) return 1;
|     if (x->end < y->end) return -1;
|     if (x->end > y->end) return 1;
|     return 0;
| }
|
| static int clamp_int(int v, int lo, int hi) {
|     if (v < lo) return lo;
|     if (v > hi) return hi;
|     return v;
| }
|
| static void add_score(int* score, int add) {
|     if (!score) return;
|     *score = clamp_int(*score + add, 0, 100);
| }
|
| static double safe_ratio(double a, double b) { return (b == 0.0) ? 0.0 : (a / b); }
|
|
| const char* heuristics_evidence_family_name(uint32_t bit) {
|     switch (bit) {
|         case HEF_TABLE_INTEGRITY: return "table_integrity";
|         case HEF_UNWIND_INTEGRITY: return "unwind_integrity";
|         case HEF_METADATA_PLACEMENT: return "metadata_placement";
|         case HEF_UNWIND_REUSE: return "unwind_reuse";
|         case HEF_HANDLER_DOMINANCE: return "handler_dominance";
|         default: return "unknown";
|     }
| }
|
| size_t heuristics_evidence_family_count(uint32_t mask) {
|     size_t n = 0;
|     while (mask) {
|         n += (mask & 1u) ? 1u : 0u;
|         mask >>= 1;
|     }
|     return n;
| }
|
| static uint32_t family_bit_from_name(const char* family) {
|     if (!family) return 0;
|     if (strcmp(family, "table_integrity") == 0) return HEF_TABLE_INTEGRITY;
|     if (strcmp(family, "unwind_integrity") == 0) return HEF_UNWIND_INTEGRITY;
|     if (strcmp(family, "metadata_placement") == 0) return HEF_METADATA_PLACEMENT;
|     if (strcmp(family, "unwind_reuse") == 0) return HEF_UNWIND_REUSE;
|     if (strcmp(family, "handler_dominance") == 0) return HEF_HANDLER_DOMINANCE;
|     return 0;
| }
|
| static void safe_copy(char* dst, size_t dst_cap, const char* src) {
|     if (!dst || dst_cap == 0) return;
|     if (!src) src = "";
|     snprintf(dst, dst_cap, "%s", src);
| }
|
| #if defined(__GNUC__) || defined(__clang__)
| #pragma GCC diagnostic push
| #pragma GCC diagnostic ignored "-Wformat-nonliteral"
| #endif
|
| static HeuristicFinding* add_finding(HeuristicReport* out,
|                                      const char* code,
|                                      const char* family,
|                                      const char* severity,
|                                      const char* fmt,
|                                      ...) {
|     if (!out || !fmt) return NULL;
|
|     char msg[192];
|     va_list ap;
|     va_start(ap, fmt);
|     (void)vsnprintf(msg, sizeof(msg), fmt, ap);
|     va_end(ap);
|
|     out->evidence_families_mask |= family_bit_from_name(family);
|
|     HeuristicFinding* stored = NULL;
|     if (out->finding_count < HEURISTICS_MAX_FINDINGS) {
|         HeuristicFinding* f = &out->findings[out->finding_count++];
|         memset(f, 0, sizeof(*f));
|         safe_copy(f->code, sizeof(f->code), code ? code : "unspecified");
|         safe_copy(f->family, sizeof(f->family), family ? family : "unknown");
|         safe_copy(f->severity, sizeof(f->severity), severity ? severity : "info");
|         safe_copy(f->message, sizeof(f->message), msg);
|         stored = f;
|     }
|
|     sb_appendf(&out->details, "- [%s/%s] %s\n",
|                severity ? severity : "info",
|                code ? code : "unspecified",
|                msg);
|     return stored;
| }
|
| static void annotate_finding(HeuristicFinding* finding,
|                              FindingLocation location,
|                              size_t affected_count) {
|     if (!finding) return;
|     finding->has_location = location.set;
|     finding->representative_rva = location.rva;
|     finding->affected_count = affected_count;
| }
|
| static void append_note(HeuristicReport* out, const char* fmt, ...) {
|     if (!out || !fmt) return;
|
|     char msg[192];
|     va_list ap;
|     va_start(ap, fmt);
```

```
457 |     (void)vsnprintf(msg, sizeof(msg), fmt, ap);
458 |     va_end(ap);
459 |
460 |     sb_appendf(&out->details, "- [info] %s\n", msg);
461 | }
462 |
463 | static size_t count_findings_with_severity(const HeuristicReport* out, const char* sev) {
464 |     size_t n = 0;
465 |     if (!out || !sev) return 0;
466 |     for (size_t i = 0; i < out->finding_count; i++) {
467 |         if (strcmp(out->findings[i].severity, sev) == 0) n++;
468 |     }
469 |     return n;
470 | }
471 |
472 | static bool has_finding_code(const HeuristicReport* out, const char* code) {
473 |     if (!out || !code) return false;
474 |     for (size_t i = 0; i < out->finding_count; i++) {
475 |         if (strcmp(out->findings[i].code, code) == 0) return true;
476 |     }
477 |     return false;
478 | }
479 |
480 | static void add_mitigation(HeuristicReport* out, const char* code, const char* fmt, ...) {
481 |     if (!out || !fmt) return;
482 |     if (out->mitigation_count >= HEURISTICS_MAX_MITIGATIONS) return;
483 |
484 |     char msg[192];
485 |     va_list ap;
486 |     va_start(ap, fmt);
487 |     (void)vsnprintf(msg, sizeof(msg), fmt, ap);
488 |     va_end(ap);
489 |
490 |     HeuristicMitigation* m = &out->mitigations[out->mitigation_count++];
491 |     memset(m, 0, sizeof(*m));
492 |     safe_copy(m->code, sizeof(m->code), code ? code : "unspecified");
493 |     safe_copy(m->message, sizeof(m->message), msg);
494 |
495 |     sb_appendf(&out->details, "  [mitigation/%s] %s\n",
496 |                code ? code : "unspecified",
497 |                msg);
498 | }
499 |
500 | #if defined(__GNUC__) || defined(__clang__)
501 | #pragma GCC diagnostic pop
502 | #endif
503 |
504 | static const char* evidence_level_from_counts(size_t weak_count,
505 |                                               size_t medium_count,
506 |                                               size_t strong_count,
507 |                                               int score) {
508 |     if (strong_count >= 2 || score >= 70) return "strong";
509 |     if (strong_count >= 1 || medium_count >= 2 || score >= 30) return "medium";
510 |     if (weak_count || medium_count || strong_count || score > 0) return "weak";
511 |     return "none";
512 | }
513 |
514 | static void finalize_verdict(HeuristicReport* out) {
515 |     if (!out) return;
516 |
517 |     out->suspicion = safe_ratio((double)out->score, 100.0);
518 |
519 |     size_t family_count = heuristics_evidence_family_count(out->evidence_families_mask);
520 |     size_t strong_count = count_findings_with_severity(out, "high") + count_findings_with_severity(out, "critical");
521 |     size_t medium_count = count_findings_with_severity(out, "medium");
522 |     size_t weak_count = 0;
523 |     if (out->finding_count >= strong_count + medium_count) weak_count = out->finding_count - strong_count - medium_count;
524 |
525 |     out->strong_finding_count = strong_count;
526 |     out->medium_finding_count = medium_count;
527 |     out->weak_finding_count = weak_count;
528 |     safe_copy(out->evidence_level, sizeof(out->evidence_level),
529 |               evidence_level_from_counts(weak_count, medium_count, strong_count, out->score));
530 |
531 |     double confidence = CONFIDENCE_BASE;
532 |     confidence += (double)family_count * CONFIDENCE_PER_FAMILY;
533 |     confidence += (double)strong_count * CONFIDENCE_PER_STRONG;
534 |     confidence += (double)medium_count * CONFIDENCE_PER_MEDIUM;
535 |
536 |     if (out->rf_invalid_ranges || out->rf_oob || out->rf_unwind_oob || out->rf_overlaps) confidence += CONFIDENCE_BONUS_TABLE_HARD;
537 |     if (out->unwind_parse_fail || out->unwind_anomalies) confidence += CONFIDENCE_BONUS_UNWIND_HARD;
538 |     if (out->score == 0) confidence = CONFIDENCE_ZERO_SCORE_BASE;
539 |
540 |     /* Weakening rules: lower confidence when only isolated / non-correlated static signals exist. */
541 |     if (out->finding_count == 1 && strong_count == 0) {
542 |         add_mitigation(out, "single_signal_only",
543 |                        "Only one non-strong signal contributed; treat as weak standalone evidence.");
544 |         confidence -= CONFIDENCE_PENALTY_SINGLE_SIGNAL;
545 |     }
546 |     if (out->total < 32 && strong_count == 0 && out->finding_count > 0) {
547 |         add_mitigation(out, "small_sample_size",
548 |                        "Very small .pdata sample can overstate ratios; confidence reduced until more functions are observed.");
549 |         confidence -= CONFIDENCE_PENALTY_SMALL_SAMPLE;
550 |     }
551 |     if (out->unwind_unusual_section > 0 &&
552 |         out->unwind_exec_section == 0 &&
553 |         out->unwind_writable_section == 0 &&
554 |         out->handler_nonexec_section == 0 &&
555 |         out->handler_writable_section == 0 &&
556 |         out->rf_invalid_ranges == 0 &&
557 |         out->rf_oob == 0 &&
558 |         out->rf_unwind_oob == 0 &&
559 |         out->unwind_parse_fail == 0 &&
560 |         out->unwind_anomalies == 0) {
561 |         add_mitigation(out, "placement_only_signal",
562 |                        "Placement-only anomalies may come from uncommon but benign layouts; manual verification required.");
563 |         confidence -= CONFIDENCE_PENALTY_PLACEMENT_ONLY;
564 |     }
565 |     if ((out->strong_finding_count == 0) && (family_count <= 1) && out->finding_count > 0) {
566 |         add_mitigation(out, "single_family_only",
567 |                        "Evidence comes from only one family; cross-layer corroboration is recommended before strong conclusions.");
568 |         confidence -= CONFIDENCE_PENALTY_SINGLE_FAMILY;
569 |     }
570 |
571 |     if (confidence < CONFIDENCE_MIN) confidence = CONFIDENCE_MIN;
```

```
    if (confidence > CONFIDENCE_MAX) confidence = CONFIDENCE_MAX;
    out->confidence = confidence;

    out->requires_manual_review =
        (out->score >= 20) ||
        (out->unwind_exec_section > 0) ||
        (out->unwind_writable_section > 0) ||
        (out->handler_nonexec_section > 0) ||
        (out->handler_writable_section > 0) ||
        (out->unwind_parse_fail > 0) ||
        (out->mitigation_count > 0);

    if (out->score >= 70) safe_copy(out->verdict, sizeof(out->verdict), "high_suspicion");
    else if (out->score >= 40) safe_copy(out->verdict, sizeof(out->verdict), "moderate_suspicion");
    else if (out->score >= 15) safe_copy(out->verdict, sizeof(out->verdict), "low_suspicion");
    else safe_copy(out->verdict, sizeof(out->verdict), "no_strong_static_anomalies");
}

bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err) {
    if (!img || !out || !err) return false;
    if (n > 0 && (!rf || !us || !us_ok)) return false;
    memset(out, 0, sizeof(*out));
    if (!sb_init(&out->details, 1024)) {
        sb_append(err, "Out of memory (details buffer).\n");
        return false;
    }
    out->total = n;
    out->pe_section_raw_oob = img->section_raw_oob;
    out->pe_section_virtual_oob = img->section_virtual_oob;
    out->pe_section_raw_overlaps = img->section_raw_overlaps;
    out->pe_section_virtual_overlaps = img->section_virtual_overlaps;
    out->pe_section_misaligned = img->section_misaligned;
    out->pe_exception_directory_oob = img->exception_directory_oob;

    /* RUNTIME_FUNCTION sanity */
    uint32_t prev_begin = 0;
    bool prev_valid = false;
    size_t unsorted = 0;
    FindingLocation invalid_range_loc = {0};
    FindingLocation rf_oob_loc = {0};
    FindingLocation unwind_oob_loc = {0};
    FindingLocation unsorted_loc = {0};
    FindingLocation overlap_loc = {0};
    FindingLocation duplicate_loc = {0};
    FindingLocation parse_fail_loc = {0};
    FindingLocation anomaly_loc = {0};
    FindingLocation unusual_unwind_loc = {0};
    FindingLocation executable_unwind_loc = {0};
    FindingLocation writable_unwind_loc = {0};
    FindingLocation nonexec_handler_loc = {0};
    FindingLocation writable_handler_loc = {0};

    HMap unwind_counts = {0};
    HMap handler_counts = {0};
    if (n > (SIZE_MAX - 1u) / 2u) {
        sb_append(err, "Runtime function count is too large for reuse analysis.\n");
        sb_free(&out->details);
        return false;
    }
    if (!hmap_init(&unwind_counts, n * 2 + 1u) ||
        !hmap_init(&handler_counts, 128)) {
        sb_append(err, "Out of memory (hash maps).\n");
        hmap_free(&unwind_counts);
        hmap_free(&handler_counts);
        sb_free(&out->details);
        return false;
    }

    for (size_t i = 0; i < n; i++) {
        uint32_t b = rf[i].begin_rva;
        uint32_t e = rf[i].end_rva;
        uint32_t u = rf[i].unwind_rva;

        if (e <= b) {
            out->rf_invalid_ranges++;
            remember_location(&invalid_range_loc, b);
        }
        if (b >= img->size_of_image || e > img->size_of_image) {
            out->rf_oob++;
            remember_location(&rf_oob_loc, b);
        }
        if (u >= img->size_of_image || u == 0) {
            out->rf_unwind_oob++;
            remember_location(&unwind_oob_loc, u);
        }

        if (prev_valid && b < prev_begin) {
            unsorted++;
            remember_location(&unsorted_loc, b);
        }
        prev_begin = b;
        prev_valid = true;

        if (!hmap_inc(&unwind_counts, u, NULL)) {
            sb_append(err, "Out of memory while counting unwind metadata reuse.\n");
            hmap_free(&unwind_counts);
            hmap_free(&handler_counts);
            sb_free(&out->details);
            return false;
        }

        if (us_ok[i] && us[i].has_handler) {
            out->handler_count++;
            if (!hmap_inc(&handler_counts, us[i].handler_rva, NULL)) {
                sb_append(err, "Out of memory while counting handler reuse.\n");
                hmap_free(&unwind_counts);
                hmap_free(&handler_counts);
                sb_free(&out->details);
                return false;
```

```
            }
        }

        /* Section placement sanity: unwind info is typically in non-executable data (.xdata/.rdata).
           Placing unwind metadata in executable / unusual sections is a common interception/tampering artifact. */
        if (us_ok[i]) {
            const PeSection* su = pe_find_section_for_rva(img, u);
            if (!su) {
                out->unwind_unusual_section++;
                remember_location(&unusual_unwind_loc, u);
            } else {
                bool allowed = sec_name_allowed_for_unwind(su->name);
                bool plausible = sec_name_plausibly_unwind_named(su->name);
                if (!allowed && !plausible) {
                    out->unwind_unusual_section++;
                    remember_location(&unusual_unwind_loc, u);
                }
                if (su->characteristics & IMAGE_SCN_MEM_EXECUTE) {
                    out->unwind_exec_section++;
                    remember_location(&executable_unwind_loc, u);
                }
                if (su->characteristics & IMAGE_SCN_MEM_WRITE) {
                    out->unwind_writable_section++;
                    remember_location(&writable_unwind_loc, u);
                }
            }

            if (us[i].has_handler) {
                const PeSection* sh = pe_find_section_for_rva(img, us[i].handler_rva);
                if (!sh || !(sh->characteristics & IMAGE_SCN_MEM_EXECUTE)) {
                    out->handler_nonexec_section++;
                    remember_location(&nonexec_handler_loc, us[i].handler_rva);
                }
                if (sh && (sh->characteristics & IMAGE_SCN_MEM_WRITE)) {
                    out->handler_writable_section++;
                    remember_location(&writable_handler_loc, us[i].handler_rva);
                }
            }
        }

        if (!us_ok[i]) {
            out->unwind_parse_fail++;
            remember_location(&parse_fail_loc, u);
        } else if (us[i].anomalies & (uint32_t)~UA_ABI_MASK) {
            out->unwind_anomalies++;
            remember_location(&anomaly_loc, u);
        }
        if (us_ok[i] && (us[i].anomalies & UA_ABI_MASK)) {
            out->unwind_abi_warnings++;
        }
    }
    out->rf_unsorted = unsorted;

    /* Overlap / duplicates check on sorted array */
    RfIdx* idx = NULL;
    if (n > 0) {
        idx = (RfIdx*)malloc(n * sizeof(RfIdx));
        if (!idx) {
            sb_append(err, "Out of memory (sort array).\n");
            hmap_free(&unwind_counts);
            hmap_free(&handler_counts);
            sb_free(&out->details);
            return false;
        }
        for (size_t i = 0; i < n; i++) { idx[i].begin = rf[i].begin_rva; idx[i].end = rf[i].end_rva; }
        qsort(idx, n, sizeof(RfIdx), cmp_rfidx);

        uint32_t max_end = 0;
        uint32_t sorted_prev_begin = 0;
        bool have_prev = false;
        for (size_t i = 0; i < n; i++) {
            if (have_prev) {
                if (idx[i].begin == sorted_prev_begin) {
                    out->rf_duplicate_begins++;
                    remember_location(&duplicate_loc, idx[i].begin);
                }
                if (idx[i].begin < max_end) {
                    out->rf_overlaps++;
                    remember_location(&overlap_loc, idx[i].begin);
                }
            }
            sorted_prev_begin = idx[i].begin;
            if (!have_prev || idx[i].end > max_end) max_end = idx[i].end;
            have_prev = true;
        }
    }
    free(idx);

    uint32_t unwind_max = hmap_max_val(&unwind_counts);
    uint32_t dominant_unwind_rva = 0;
    (void)hmap_max_entry(&unwind_counts, &dominant_unwind_rva, NULL);
    size_t unwind_unique = hmap_unique(&unwind_counts);
    out->unwind_reuse_ratio = safe_ratio((double)unwind_max, (double)n);
    out->unwind_unique_ratio = safe_ratio((double)unwind_unique, (double)n);

    uint32_t handler_max = hmap_max_val(&handler_counts);
    uint32_t dominant_handler_rva = 0;
    (void)hmap_max_entry(&handler_counts, &dominant_handler_rva, NULL);
    out->handler_reuse_ratio = (out->handler_count > 0) ? safe_ratio((double)handler_max, (double)out->handler_count) : 0.0;

    /* Scoring */
    int score = 0;
    if (out->pe_section_raw_oob) {
        add_score(&score, SCORE_PE_SECTION_RAW_OOB);
        annotate_finding(
            add_finding(out, "pe_section_raw_oob", "table_integrity", "high",
                        "Section raw ranges exceed the input file: %zu", out->pe_section_raw_oob),
            first_pe_section_issue(img, PE_ISSUE_RAW_OOB), out->pe_section_raw_oob);
    }
    if (out->pe_section_virtual_oob) {
        add_score(&score, SCORE_PE_SECTION_VIRTUAL_OOB);
        annotate_finding(
            add_finding(out, "pe_section_virtual_oob", "table_integrity", "high",
                        "Section virtual ranges exceed SizeOfImage: %zu", out->pe_section_virtual_oob),
            first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OOB), out->pe_section_virtual_oob);
```

```
802 |     }
803 |     if (out->pe_section_raw_overlaps) {
804 |         add_score(&score, SCORE_PE_SECTION_RAW_OVERLAP);
805 |         annotate_finding(
806 |             add_finding(out, "pe_section_raw_overlap", "table_integrity", "medium",
807 |                         "Overlapping section raw ranges: %zu", out->pe_section_raw_overlaps),
808 |             first_pe_section_issue(img, PE_ISSUE_RAW_OVERLAP), out->pe_section_raw_overlaps);
809 |     }
810 |     if (out->pe_section_virtual_overlaps) {
811 |         add_score(&score, SCORE_PE_SECTION_VIRTUAL_OVERLAP);
812 |         annotate_finding(
813 |             add_finding(out, "pe_section_virtual_overlap", "table_integrity", "high",
814 |                         "Overlapping section virtual ranges: %zu", out->pe_section_virtual_overlaps),
815 |             first_pe_section_issue(img, PE_ISSUE_VIRTUAL_OVERLAP), out->pe_section_virtual_overlaps);
816 |     }
817 |     if (out->pe_section_misaligned) {
818 |             add_score(&score, SCORE_PE_SECTION_MISALIGNED);
819 |         annotate_finding(
820 |             add_finding(out, "pe_section_misaligned", "table_integrity", "medium",
821 |                         "PE alignment/header invariants or section alignment rules are violated: %zu",
822 |                         out->pe_section_misaligned),
823 |             first_pe_section_issue(img, PE_ISSUE_MISALIGNED), out->pe_section_misaligned);
824 |     }
825 |     if (out->pe_exception_directory_oob) {
826 |         FindingLocation location = { true, img->exception_rva };
827 |         add_score(&score, SCORE_PE_EXCEPTION_DIRECTORY_OOB);
828 |         annotate_finding(
829 |             add_finding(out, "pe_exception_directory_oob", "table_integrity", "high",
830 |                         "Exception directory is malformed, unreadable, incomplete, or out of bounds."),
831 |             location, 1);
832 |     }
833 |     if (n == 0 &&
834 |         img->exception_rva == 0 &&
835 |         img->exception_size == 0 &&
836 |         image_has_executable_section(img)) {
837 |         add_score(&score, SCORE_EMPTY_PDATA);
838 |         add_finding(out, "no_runtime_functions", "table_integrity", "medium",
839 |                     "No RUNTIME_FUNCTION entries found; this can be valid for a leaf-only image, so confirm code complexity
    manually.");
840 |     } else if (n == 0 && img->exception_rva == 0 && img->exception_size == 0) {
841 |         append_note(out, "No RUNTIME_FUNCTION entries and no executable sections; this is consistent with a resource-only or
    forwarding image.");
842 |     }
843 |
844 |     double invalid_ranges_pct = safe_ratio((double)out->rf_invalid_ranges, (double)n);
845 |     double oob_pct = safe_ratio((double)out->rf_oob, (double)n);
846 |     double unwind_oob_pct = safe_ratio((double)out->rf_unwind_oob, (double)n);
847 |     double unwind_fail_pct = safe_ratio((double)out->unwind_parse_fail, (double)n);
848 |     double unwind_anom_pct = safe_ratio((double)out->unwind_anomalies, (double)n);
849 |
850 |     if (out->rf_invalid_ranges) {
851 |         add_score(&score, invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? SCORE_INVALID_RANGE_HIGH : SCORE_INVALID_RANGE_LOW);
852 |         annotate_finding(
853 |             add_finding(out, "invalid_ranges", "table_integrity", invalid_ranges_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
854 |                         "Invalid ranges (End<=Begin): %zu (%.2f%%)", out->rf_invalid_ranges, invalid_ranges_pct*100.0),
855 |             invalid_range_loc, out->rf_invalid_ranges);
856 |     }
857 |     if (out->rf_oob) {
858 |         add_score(&score, oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_RF_OOB_HIGH : SCORE_RF_OOB_LOW);
859 |         annotate_finding(
860 |             add_finding(out, "rf_oob", "table_integrity", oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
861 |                         "Begin/End out of SizeOfImage: %zu (%.2f%%)", out->rf_oob, oob_pct*100.0),
862 |             rf_oob_loc, out->rf_oob);
863 |     }
864 |     if (out->rf_unwind_oob) {
865 |         add_score(&score, unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_RVA_OOB_HIGH : SCORE_UNWIND_RVA_OOB_LOW);
866 |         annotate_finding(
867 |             add_finding(out, "unwind_rva_oob", "table_integrity", unwind_oob_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
868 |                         "UnwindInfoRVA missing/out-of-bounds: %zu (%.2f%%)", out->rf_unwind_oob, unwind_oob_pct*100.0),
869 |             unwind_oob_loc, out->rf_unwind_oob);
870 |     }
871 |     if (out->rf_unsorted) {
872 |         add_score(&score, out->rf_unsorted > 0 ? SCORE_RF_UNSORTED : 0);
873 |         annotate_finding(
874 |             add_finding(out, "rf_unsorted", "table_integrity", "medium",
875 |                         "Table not sorted by BeginAddress: %zu inversions", out->rf_unsorted),
876 |             unsorted_loc, out->rf_unsorted);
877 |     }
878 |     if (out->rf_overlaps) {
879 |         add_score(&score, out->rf_overlaps > 0 ? SCORE_RF_OVERLAP : 0);
880 |         annotate_finding(
881 |             add_finding(out, "rf_overlaps", "table_integrity", "high",
882 |                         "Overlapping ranges (after sorting): %zu", out->rf_overlaps),
883 |             overlap_loc, out->rf_overlaps);
884 |     }
885 |     if (out->rf_duplicate_begins) {
886 |         add_score(&score, SCORE_DUPLICATE_BEGIN);
887 |         annotate_finding(
888 |             add_finding(out, "duplicate_begin", "table_integrity", "medium",
889 |                         "Duplicate BeginAddress (after sorting): %zu", out->rf_duplicate_begins),
890 |             duplicate_loc, out->rf_duplicate_begins);
891 |     }
892 |
893 |     if (out->unwind_parse_fail) {
894 |         add_score(&score, unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? SCORE_UNWIND_PARSE_FAIL_HIGH : SCORE_UNWIND_PARSE_FAIL_LOW);
895 |         annotate_finding(
896 |             add_finding(out, "unwind_parse_fail", "unwind_integrity", unwind_fail_pct > THRESHOLD_HIGH_PERCENT ? "high" : "medium",
897 |                         "UnwindInfo parse failures: %zu (%.2f%%)", out->unwind_parse_fail, unwind_fail_pct*100.0),
898 |             parse_fail_loc, out->unwind_parse_fail);
899 |     }
900 |     if (out->unwind_anomalies) {
901 |         add_score(&score, unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? SCORE_UNWIND_ANOM_HIGH : SCORE_UNWIND_ANOM_LOW);
902 |         annotate_finding(
903 |             add_finding(out, "unwind_anomalies", "unwind_integrity", unwind_anom_pct > THRESHOLD_UNWIND_ANOM_HIGH ? "high" :
    "medium",
904 |                         "UnwindInfo anomalies (version/opcode/chain/etc): %zu (%.2f%%)", out->unwind_anomalies,
    unwind_anom_pct*100.0),
905 |             anomaly_loc, out->unwind_anomalies);
906 |     }
907 |
908 |     bool moderate_reuse_candidate =
909 |         out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && n > 200;
910 |     if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_HIGH && n > 200) {
911 |         add_score(&score, SCORE_UNWIND_REUSE_HIGH);
912 |         FindingLocation location = { true, dominant_unwind_rva };
```

```
913 |         annotate_finding(
914 |             add_finding(out, "unwind_reuse_strong", "unwind_reuse", "high",
915 |                         "Strong UnwindInfo reuse: max %u entries share same UnwindInfoRVA (ratio=%.2f)",
916 |                         (unsigned)unwind_max, out->unwind_reuse_ratio),
917 |             location, unwind_max);
918 |     }
919 |
920 |     if (out->handler_count > 50 && out->handler_reuse_ratio > THRESHOLD_HANDLER_DOMINANCE) {
921 |         bool has_static_context =
922 |             out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 || out->rf_unsorted > 0 ||
923 |             out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 || out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||
924 |             out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 || out->unwind_writable_section > 0 ||
925 |             out->handler_nonexec_section > 0 || out->handler_writable_section > 0 ||
926 |             moderate_reuse_candidate;
927 |         const PeSection* dominant_sec = dominant_handler_rva ? pe_find_section_for_rva(img, dominant_handler_rva) : NULL;
928 |         bool dominant_runtime_like = dominant_sec &&
929 |                                      (dominant_sec->characteristics & IMAGE_SCN_MEM_EXECUTE) &&
930 |                                      sec_name_code_like(dominant_sec->name) &&
931 |                                      out->handler_nonexec_section == 0;
932 |
933 |         if (!(dominant_runtime_like && !has_static_context)) {
934 |             add_score(&score, SCORE_HANDLER_DOMINANCE);
935 |             FindingLocation location = { true, dominant_handler_rva };
936 |             annotate_finding(
937 |                 add_finding(out, "handler_dominance", "handler_dominance", "medium",
938 |                             "One handler dominates: max %u entries share same HandlerRVA (ratio=%.2f%s%s)",
939 |                             (unsigned)handler_max, out->handler_reuse_ratio,
940 |                             dominant_sec ? ", section=" : "",
941 |                             dominant_sec ? dominant_sec->name : ""),
942 |                 location, handler_max);
943 |         }
944 |     }
945 |
946 |     if (moderate_reuse_candidate &&
947 |         !has_finding_code(out, "unwind_reuse_strong") &&
948 |         (has_finding_code(out, "handler_dominance") ||
949 |          out->rf_invalid_ranges > 0 || out->rf_oob > 0 || out->rf_unwind_oob > 0 ||
950 |          out->rf_unsorted > 0 || out->rf_overlaps > 0 || out->rf_duplicate_begins > 0 ||
951 |          out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 ||
952 |          out->unwind_exec_section > 0 || out->unwind_unusual_section > 0 ||
953 |          out->unwind_writable_section > 0 || out->handler_nonexec_section > 0 ||
954 |          out->handler_writable_section > 0)) {
955 |         add_score(&score, SCORE_UNWIND_REUSE_MEDIUM);
956 |         FindingLocation location = { true, dominant_unwind_rva };
957 |         annotate_finding(
958 |             add_finding(out, "unwind_reuse_moderate", "unwind_reuse", "medium",
959 |                         "Moderate UnwindInfo reuse is corroborated by another static signal: max %u entries share same UnwindInfoRVA
    (ratio=%.2f)",
960 |                         (unsigned)unwind_max, out->unwind_reuse_ratio),
961 |             location, unwind_max);
962 |     }
963 |
964 |     /* Placement heuristics */
965 |     if (out->unwind_exec_section) {
966 |         add_score(&score, SCORE_UNWIND_EXEC_SECTION);
967 |         annotate_finding(
968 |             add_finding(out, "unwind_in_exec_section", "metadata_placement", "high",
969 |                         "UnwindInfo located in executable section(s): %zu", out->unwind_exec_section),
970 |             executable_unwind_loc, out->unwind_exec_section);
971 |     }
972 |     if (out->unwind_unusual_section) {
973 |         int add = (out->unwind_unusual_section > (n / 20)) ? SCORE_UNWIND_UNUSUAL_SECTION_HIGH : SCORE_UNWIND_UNUSUAL_SECTION_LOW; /*
    >5% => broader placement pattern */
974 |         add_score(&score, add);
975 |         annotate_finding(
976 |             add_finding(out, "unwind_unusual_section", "metadata_placement", add >= 15 ? "high" : "medium",
977 |                         "UnwindInfo located in unusual sections (not .xdata/.rdata/.pdata): %zu", out->unwind_unusual_section),
978 |             unusual_unwind_loc, out->unwind_unusual_section);
979 |     }
980 |     if (out->unwind_writable_section) {
981 |         int add = (out->unwind_writable_section > (n / 20)) ? SCORE_UNWIND_WRITABLE_SECTION_HIGH : SCORE_UNWIND_WRITABLE_SECTION_LOW;
982 |         add_score(&score, add);
983 |         annotate_finding(
984 |             add_finding(out, "unwind_writable_section", "metadata_placement", add >= 18 ? "high" : "medium",
985 |                         "UnwindInfo located in writable section(s): %zu", out->unwind_writable_section),
986 |             writable_unwind_loc, out->unwind_writable_section);
987 |     }
988 |     if (out->handler_nonexec_section) {
989 |         add_score(&score, SCORE_HANDLER_NONEXEC);
990 |         annotate_finding(
991 |             add_finding(out, "handler_nonexec_section", "metadata_placement", "high",
992 |                         "Exception handler RVA in non-executable/unknown section: %zu", out->handler_nonexec_section),
993 |             nonexec_handler_loc, out->handler_nonexec_section);
994 |     }
995 |     if (out->handler_writable_section) {
996 |         add_score(&score, SCORE_HANDLER_WRITABLE);
997 |         annotate_finding(
998 |             add_finding(out, "handler_writable_section", "metadata_placement", "medium",
999 |                         "Exception handler RVA in writable section(s): %zu", out->handler_writable_section),
1000|             writable_handler_loc, out->handler_writable_section);
1001|     }
1002|
1003|     if (out->unwind_exec_section > 0 && out->handler_nonexec_section > 0) {
1004|         add_score(&score, SCORE_COMBO_PLACEMENT_EXEC_AND_HANDLER);
1005|         add_finding(out, "placement_exec_nonexec_handler_combo", "metadata_placement", "high",
1006|                     "Unwind metadata in executable section(s) coincides with handler targets in non-executable/unknown section(s);
    the combined placement pattern is much harder to explain benignly.");
1007|     }
1008|     if ((out->unwind_exec_section > 0 || out->handler_nonexec_section > 0) &&
1009|         (out->unwind_parse_fail > 0 || out->unwind_anomalies > 0 || out->rf_unwind_oob > 0)) {
1010|         add_score(&score, SCORE_COMBO_PLACEMENT_AND_INTEGRITY);
1011|         add_finding(out, "placement_integrity_combo", "unwind_integrity", "medium",
1012|                     "Suspicious metadata placement is corroborated by UNWIND_INFO / UnwindInfoRVA integrity problems; treat the
    combination as stronger than either signal alone.");
1013|     }
1014|     if (out->unwind_reuse_ratio > THRESHOLD_UNWIND_REUSE_MEDIUM && has_finding_code(out, "handler_dominance")) {
1015|         add_score(&score, SCORE_COMBO_REUSE_AND_DOMINANCE);
1016|         add_finding(out, "reuse_dominance_combo", "unwind_reuse", "medium",
1017|                     "Heavy UnwindInfo reuse coincides with a dominant handler target, which is consistent with centralized
    dispatcher-style metadata reuse.");
1018|     }
1019|     if (out->unwind_writable_section > 0 && out->handler_writable_section > 0) {
1020|         add_score(&score, SCORE_COMBO_WRITABLE_METADATA_AND_HANDLER);
1021|         add_finding(out, "writable_metadata_handler_combo", "metadata_placement", "high",
1022|                     "Writable unwind metadata coincides with writable handler targets; this combination is uncommon for normal
```

```
    compiler-produced images and deserves close review.");
    }

    if (score == 0) {
        append_note(out, "No strong anomalies detected by heuristics.");
    }
    out->score = score;
    finalize_verdict(out);

    hmap_free(&unwind_counts);
    hmap_free(&handler_counts);
    if (!sb_ok(&out->details)) {
        sb_append(err, "Out of memory while building heuristic details.\n");
        return false;
    }
    return true;
}

void heuristics_free(HeuristicReport* r) {
    if (!r) return;
    sb_free(&r->details);
    memset(r, 0, sizeof(*r));
}
```

include/dynamic.h

```
#pragma once
/*
  dynamic.h - Windows-only dynamic evidence collector (Debug API)

  Defensive intent:
    Collect runtime evidence of exception-handling redirection (e.g., to an
    interceptor or VM dispatcher) by observing:
      - hits of handlers referenced from UNWIND_INFO (x64)
      - registration/hits of vectored/unhandled exception handlers
      - registration of dynamic function tables
      - first-chance vs second-chance exception flow

  This module does NOT attempt to "break" protections or recover hidden data.
*/

#include <stdint.h>
#include <stdbool.h>
#include "util.h"
#include "pe.h"
#include "unwind.h"

#ifdef __cplusplus
extern "C" {
#endif

enum { DYNAMIC_MAX_EVENTS = 256 };

typedef struct DynamicEvent {
    char type[40];
    char module[64];
    uint32_t thread_id;
    uint32_t exception_code;
    bool first_chance;
    uint64_t address;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    uint64_t arg3;
} DynamicEvent;

typedef struct DynamicReport {
    bool enabled;
    bool ran;
    bool ok;
    bool timed_out;
    bool target_exited;
    uint32_t target_exit_code;

    uint32_t timeout_ms;
    uint32_t trace_steps;

    bool instrumentation_complete;
    uint32_t modules_observed_peak;
    uint32_t module_refresh_failures;
    uint32_t module_list_truncations;
    uint32_t unwind_handlers_discovered;
    uint32_t breakpoint_set_attempts;
    uint32_t breakpoint_set_failures;
    uint32_t breakpoint_capacity_drops;
    uint32_t pending_rearm_failures;
    uint32_t breakpoint_rearm_failures;

    uint32_t exceptions_seen;
    uint32_t first_chance_seen;
    uint32_t second_chance_seen;

    uint32_t unwind_handler_bp_set;
    uint32_t unwind_handler_hits;

    uint32_t veh_registrations;
    uint32_t veh_hits;
    uint32_t vch_registrations;
    uint32_t vch_hits;
    uint32_t uef_set_calls;
    uint32_t uef_hits;

    uint32_t rtl_add_function_table_calls;
    uint32_t rtl_install_function_table_cb_calls;

    uint32_t event_count;
    uint32_t dropped_events;
    DynamicEvent events[DYNAMIC_MAX_EVENTS];

    /* Execution trace sampled after first handler hit (RIP addresses). */
    uint32_t trace_count;
    uint64_t trace[256];

    /* Human readable details */
    StrBuf details;
} DynamicReport;

void dynamic_report_init(DynamicReport* dr);
void dynamic_report_free(DynamicReport* dr);

/*
  Run target under a debugger and collect evidence.

  - pe_path: path to target executable
  - child_argv/child_argc: optional args (may be NULL/0)
  - img/rf/us/us_ok: results of static parsing (used to set handler breakpoints)
  - rf_count: number of runtime functions
  - timeout_ms: hard timeout for debug loop
  - trace_steps: number of single-steps to record after handler hit (<=256 recommended)

  Returns true if the dynamic session ran (even if it found no evidence).
*/
bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
```

```
112 |                             const UnwindSummary* us,
113 |                             const bool* us_ok,
114 |                             size_t rf_count,
115 |                             uint32_t timeout_ms,
116 |                             uint32_t trace_steps,
117 |                             DynamicReport* out,
118 |                             StrBuf* err);
119 |
120 | #ifdef __cplusplus
121 | }
122 | #endif
```

src/dynamic_win.c

```
#include "dynamic.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

#ifdef _WIN32

#include <windows.h>
#include <tlhelp32.h>

/* MinGW exposes exception_code as a compatibility macro. */
#ifdef exception_code
#undef exception_code
#endif

/* ------------------------------------------------------------------------ */

typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;

typedef struct ModList {
    ModInfo mods[128];
    size_t count;
    bool truncated;
} ModList;

static void mods_init(ModList* ml) { if (ml) memset(ml, 0, sizeof(*ml)); }

static const char* basename_lower(const char* path) {
    const char* base = path;
    for (const char* p = path; *p; p++) {
        if (*p == '\\' || *p == '/') base = p + 1;
    }
    return base;
}

static void strlower_inplace(char* s) {
    for (; s && *s; s++) {
        if (*s >= 'A' && *s <= 'Z') *s = (char)(*s - 'A' + 'a');
    }
}

static void mods_add(ModList* ml, const char* name, uint64_t base, uint64_t size) {
    if (!ml || !name) return;
    if (ml->count >= sizeof(ml->mods)/sizeof(ml->mods[0])) {
        ml->truncated = true;
        return;
    }
    ModInfo* m = &ml->mods[ml->count++];
    strncpy(m->name, name, sizeof(m->name)-1);
    m->name[sizeof(m->name)-1] = 0;
    strlower_inplace(m->name);
    m->base = base;
    m->size = size;
}

static const ModInfo* mods_find(const ModList* ml, const char* name) {
    if (!ml || !name) return NULL;
    char tmp[MAX_PATH];
    strncpy(tmp, name, sizeof(tmp)-1);
    tmp[sizeof(tmp)-1] = 0;
    strlower_inplace(tmp);
    for (size_t i = 0; i < ml->count; i++) {
        if (strcmp(ml->mods[i].name, tmp) == 0) return &ml->mods[i];
    }
    return NULL;
}

static const ModInfo* mods_find_by_addr(const ModList* ml, uint64_t addr) {
    if (!ml) return NULL;
    for (size_t i = 0; i < ml->count; i++) {
        uint64_t b = ml->mods[i].base;
        uint64_t e = ml->mods[i].size > UINT64_MAX - b
            ? UINT64_MAX
            : b + ml->mods[i].size;
        if (addr >= b && addr < e) return &ml->mods[i];
    }
    return NULL;
}

static bool mods_refresh_toolhelp(ModList* ml, DWORD pid) {
    if (!ml) return false;
    ml->count = 0;
    ml->truncated = false;
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid);
    if (snap == INVALID_HANDLE_VALUE) return false;

    MODULEENTRY32 me;
    memset(&me, 0, sizeof(me));
    me.dwSize = sizeof(me);
    if (!Module32First(snap, &me)) {
        CloseHandle(snap);
        return false;
    }
    do {
        const char* base = basename_lower(me.szModule);
        mods_add(ml, base, (uint64_t)(uintptr_t)me.modBaseAddr, (uint64_t)me.modBaseSize);
    } while (Module32Next(snap, &me));

    CloseHandle(snap);
    return true;
}

static bool mods_refresh_accounted(ModList* ml, DWORD pid, DynamicReport* dr) {
    if (!mods_refresh_toolhelp(ml, pid)) {
        if (dr) {
            dr->module_refresh_failures++;
```

```
            dr->instrumentation_complete = false;
        }
        return false;
    }
    if (dr) {
        if (ml->count > dr->modules_observed_peak) {
            dr->modules_observed_peak = (uint32_t)ml->count;
        }
        if (ml->truncated) {
            dr->module_list_truncations++;
            dr->instrumentation_complete = false;
        }
    }
    return true;
}

/* ---------------------------------------------------------------------- */

typedef enum BpKind {
    BP_UNWIND_HANDLER       = 1u << 0,
    BP_API_ADD_VEH          = 1u << 1,
    BP_API_ADD_VCH          = 1u << 2,
    BP_API_SET_UEF          = 1u << 3,
    BP_API_RTL_ADD_FT       = 1u << 4,
    BP_API_RTL_INSTALL_FTCB = 1u << 5,
    BP_CUSTOM_VEH_HANDLER   = 1u << 6,
    BP_CUSTOM_VCH_HANDLER   = 1u << 7,
    BP_CUSTOM_UEF           = 1u << 8
} BpKind;

typedef struct Breakpoint {
    uint64_t addr;
    uint8_t  orig;
    bool     armed;
    bool     rearm_pending;
    uint32_t kinds;
} Breakpoint;

typedef struct BpList {
    Breakpoint bps[512];
    size_t count;
} BpList;

static void bps_init(BpList* bl) { if (bl) memset(bl, 0, sizeof(*bl)); }

static Breakpoint* bps_find(BpList* bl, uint64_t addr) {
    if (!bl) return NULL;
    for (size_t i = 0; i < bl->count; i++) {
        if (bl->bps[i].addr == addr) return &bl->bps[i];
    }
    return NULL;
}

static bool write_one_byte(HANDLE hProcess, uint64_t addr, uint8_t val, StrBuf* err) {
    SIZE_T nw = 0;
    uint8_t b = val;
    if (WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) && nw == 1) {
        FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
        return true;
    }
    DWORD werr = GetLastError();

    /* Retry with temporary RWX. Some targets map code pages RX. */
    DWORD oldProt = 0;
    if (!VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, PAGE_EXECUTE_READWRITE, &oldProt)) {
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx (err=%lu) and VirtualProtectEx failed (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr, (unsigned long)GetLastError());
        return false;
    }

    nw = 0;
    if (!WriteProcessMemory(hProcess, (LPVOID)(uintptr_t)addr, &b, 1, &nw) || nw != 1) {
        DWORD werr2 = GetLastError();
        DWORD tmp = 0;
        VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
        if (err) sb_appendf(err,
            "WriteProcessMemory failed at 0x%llx after VirtualProtectEx (err=%lu)\n",
            (unsigned long long)addr, (unsigned long)werr2);
        return false;
    }

    FlushInstructionCache(hProcess, (LPCVOID)(uintptr_t)addr, 1);
    DWORD tmp = 0;
    VirtualProtectEx(hProcess, (LPVOID)(uintptr_t)addr, 1, oldProt, &tmp);
    return true;
}

static bool bp_set(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    SIZE_T nread = 0;
    uint8_t b = 0;
    if (!ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)bp->addr, &b, 1, &nread) || nread != 1) {
        if (err) sb_appendf(err, "ReadProcessMemory failed at 0x%llx (err=%lu)\n",
                            (unsigned long long)bp->addr, (unsigned long)GetLastError());
        return false;
    }
    bp->orig = b;
    if (!write_one_byte(hProcess, bp->addr, 0xCC, err)) return false;
    bp->armed = true;
    bp->rearm_pending = false;
    return true;
}

static bool bp_clear(HANDLE hProcess, Breakpoint* bp, StrBuf* err) {
    if (!bp || !bp->armed) return true;
    if (!write_one_byte(hProcess, bp->addr, bp->orig, err)) return false;
    bp->armed = false;
    return true;
}

static bool bp_add_and_arm(BpList* bl, HANDLE hProcess, uint64_t addr, BpKind kind, StrBuf* err) {
    if (!bl) return false;

    Breakpoint* ex = bps_find(bl, addr);
    if (ex) {
```

```
227 |         ex->kinds |= (uint32_t)kind;
228 |         if (ex->armed || ex->rearm_pending) return true;
229 |         return bp_set(hProcess, ex, err);
230 |     }
231 |
232 |     if (bl->count >= sizeof(bl->bps)/sizeof(bl->bps[0])) return false;
233 |     Breakpoint* bp = &bl->bps[bl->count];
234 |     memset(bp, 0, sizeof(*bp));
235 |     bp->addr = addr;
236 |     bp->kinds = (uint32_t)kind;
237 |
238 |     if (!bp_set(hProcess, bp, err)) return false;
239 |     bl->count++;
240 |     return true;
241 | }
242 |
243 | /* ------------------------------------------------------------------------ */
244 | /* Self-check helpers: log where breakpoints were actually placed in the target.
245 |    Goal: make it obvious whether a requested breakpoint is (a) placed, (b) placed in the
246 |    expected module, and (c) armed in memory (0xCC). */
247 |
248 | static const char* bp_kind_name(BpKind k); /* forward */
249 | static void dr_append(DynamicReport* dr, const char* fmt, ...); /* forward */
250 |
251 | static bool bp_peek_byte(HANDLE hProcess, uint64_t addr, uint8_t* outb, StrBuf* err) {
252 |     if (!outb) return false;
253 |     SIZE_T nr = 0;
254 |     uint8_t b = 0;
255 |     if (ReadProcessMemory(hProcess, (LPCVOID)(uintptr_t)addr, &b, 1, &nr) && nr == 1) {
256 |         *outb = b;
257 |         return true;
258 |     }
259 |     if (err) sb_appendf(err, "ReadProcessMemory peek failed at 0x%llx (err=%lu)\n",
260 |                         (unsigned long long)addr, (unsigned long)GetLastError());
261 |     return false;
262 | }
263 |
264 | static void format_bp_location(const ModList* mods, uint64_t addr, char* out, size_t outsz) {
265 |     if (!out || outsz == 0) return;
266 |     out[0] = 0;
267 |     const ModInfo* mi = mods ? mods_find_by_addr(mods, addr) : NULL;
268 |     if (mi) {
269 |         snprintf(out, outsz, "%s+0x%llx (0x%llx)",
270 |                  mi->name,
271 |                  (unsigned long long)(addr - mi->base),
272 |                  (unsigned long long)addr);
273 |     } else {
274 |         snprintf(out, outsz, "0x%llx", (unsigned long long)addr);
275 |     }
276 | }
277 |
278 | /* Place a breakpoint and emit a deterministic self-check line in dr->details.
279 |    To reduce log spam, this logs only on the first attempt to create the bp at addr. */
280 | static bool bp_add_and_arm_selfcheck(BpList* bl,
281 |                                      HANDLE hProcess,
282 |                                      const ModList* mods,
283 |                                      DynamicReport* dr,
284 |                                      uint64_t addr,
285 |                                      BpKind kind,
286 |                                      const char* what,
287 |                                      StrBuf* err) {
288 |     if (!bl) return false;
289 |
290 |     Breakpoint* existing = bps_find(bl, addr);
291 |     bool first_time = (existing == NULL);
292 |     bool capacity_exhausted =
293 |         first_time && bl->count >= sizeof(bl->bps) / sizeof(bl->bps[0]);
294 |     if (dr && first_time) dr->breakpoint_set_attempts++;
295 |
296 |     StrBuf te; sb_init(&te, 256);
297 |     bool ok = bp_add_and_arm(bl, hProcess, addr, kind, &te);
298 |
299 |     if (dr && !ok) {
300 |         dr->breakpoint_set_failures++;
301 |         if (capacity_exhausted) dr->breakpoint_capacity_drops++;
302 |         dr->instrumentation_complete = false;
303 |     }
304 |
305 |     if (dr && first_time) {
306 |         char loc[192];
307 |         format_bp_location(mods, addr, loc, sizeof(loc));
308 |
309 |         uint8_t cur = 0xFF;
310 |         StrBuf peek_err; sb_init(&peek_err, 128);
311 |         bool peek_ok = bp_peek_byte(hProcess, addr, &cur, &peek_err);
312 |
313 |         if (ok) {
314 |             if (peek_ok) {
315 |                 dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (byte=0x%02x)%s\n",
316 |                           bp_kind_name(kind),
317 |                           (what ? what : ""),
318 |                           loc,
319 |                           (unsigned)cur,
320 |                           (cur == 0xCC ? "" : " [WARN: not INT3]"));
321 |             } else {
322 |                 dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : OK (peek failed: %s)\n",
323 |                           bp_kind_name(kind), (what ? what : ""), loc,
324 |                           (peek_err.s ? peek_err.s : "unknown"));
325 |             }
326 |         } else {
327 |             dr_append(dr, "[SelfCheck] BP set %-22s %-40s at %s : FAIL%s%s",
328 |                       bp_kind_name(kind),
329 |                       (what ? what : ""),
330 |                       loc,
331 |                       (te.len ? " : " : "\n"),
332 |                       (te.len ? te.s : "\n"));
333 |         }
334 |
335 |         sb_free(&peek_err);
336 |     }
337 |
338 |     if (!ok && err && te.len) sb_append(err, te.s);
339 |     sb_free(&te);
340 |     return ok;
341 | }
```

```
342 |
343 |
344 | /* ------------------------------------------------------------------------ */
345 |
346 | /* Pending breakpoint re-arming must be tracked per-thread.
347 |    With multiple threads, a single global pending pointer can cause lost rearms. */
348 | typedef struct PendingRearmEntry {
349 |     DWORD tid;
350 |     Breakpoint* bp;
351 | } PendingRearmEntry;
352 |
353 | typedef struct PendingRearmList {
354 |     PendingRearmEntry e[64];
355 | } PendingRearmList;
356 |
357 | static void pending_rearm_init(PendingRearmList* pr) {
358 |     if (!pr) return;
359 |     memset(pr, 0, sizeof(*pr));
360 | }
361 |
362 | static bool pending_rearm_set(PendingRearmList* pr, DWORD tid, Breakpoint* bp) {
363 |     if (!pr || !bp) return false;
364 |     /* overwrite existing */
365 |     for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
366 |         if (pr->e[i].bp && pr->e[i].tid == tid) {
367 |             pr->e[i].bp = bp;
368 |             return true;
369 |         }
370 |     }
371 |     /* take a free slot */
372 |     for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
373 |         if (!pr->e[i].bp) {
374 |             pr->e[i].tid = tid;
375 |             pr->e[i].bp = bp;
376 |             return true;
377 |         }
378 |     }
379 |     return false;
380 | }
381 |
382 | static Breakpoint* pending_rearm_take(PendingRearmList* pr, DWORD tid) {
383 |     if (!pr) return NULL;
384 |     for (size_t i = 0; i < sizeof(pr->e)/sizeof(pr->e[0]); i++) {
385 |         if (pr->e[i].bp && pr->e[i].tid == tid) {
386 |             Breakpoint* bp = pr->e[i].bp;
387 |             pr->e[i].bp = NULL;
388 |             pr->e[i].tid = 0;
389 |             return bp;
390 |         }
391 |     }
392 |     return NULL;
393 | }
394 |
395 | static HANDLE open_event_thread(DWORD tid) {
396 |     return OpenThread(THREAD_GET_CONTEXT | THREAD_SET_CONTEXT | THREAD_QUERY_INFORMATION, FALSE, tid);
397 | }
398 |
399 | static void close_event_thread(HANDLE h) {
400 |     if (h) CloseHandle(h);
401 | }
402 |
403 | static void dr_append(DynamicReport* dr, const char* fmt, ...) {
404 |     if (!dr) return;
405 |     va_list ap;
406 |     va_start(ap, fmt);
407 |     /* Reuse sb_appendf's variadic through vsnprintf into temp */
408 |     char tmp[1024];
409 |     int n = vsnprintf(tmp, sizeof(tmp), fmt, ap);
410 |     va_end(ap);
411 |     if (n <= 0) return;
412 |     tmp[sizeof(tmp)-1] = 0;
413 |     sb_append(&dr->details, tmp);
414 | }
415 |
416 | static void dynamic_event_add(DynamicReport* dr,
417 |                               const ModList* mods,
418 |                               const char* type,
419 |                               DWORD thread_id,
420 |                               uint64_t address,
421 |                               uint32_t exception_code,
422 |                               bool first_chance,
423 |                               uint64_t arg0,
424 |                               uint64_t arg1,
425 |                               uint64_t arg2,
426 |                               uint64_t arg3) {
427 |     if (!dr || !type) return;
428 |     if (dr->event_count >= DYNAMIC_MAX_EVENTS) {
429 |         dr->dropped_events++;
430 |         dr->instrumentation_complete = false;
431 |         return;
432 |     }
433 |
434 |     DynamicEvent* event = &dr->events[dr->event_count++];
435 |     memset(event, 0, sizeof(*event));
436 |     snprintf(event->type, sizeof(event->type), "%s", type);
437 |     const ModInfo* module = mods_find_by_addr(mods, address);
438 |     if (module) snprintf(event->module, sizeof(event->module), "%s", module->name);
439 |     event->thread_id = (uint32_t)thread_id;
440 |     event->exception_code = exception_code;
441 |     event->first_chance = first_chance;
442 |     event->address = address;
443 |     event->arg0 = arg0;
444 |     event->arg1 = arg1;
445 |     event->arg2 = arg2;
446 |     event->arg3 = arg3;
447 | }
448 |
449 | void dynamic_report_init(DynamicReport* dr) {
450 |     if (!dr) return;
451 |     memset(dr, 0, sizeof(*dr));
452 |     dr->enabled = true;
453 |     dr->instrumentation_complete = true;
454 |     sb_init(&dr->details, 2048);
455 | }
456 |
```

```
void dynamic_report_free(DynamicReport* dr) {
    if (!dr) return;
    sb_free(&dr->details);
    memset(dr, 0, sizeof(*dr));
}

static bool append_backslashes(StrBuf* out, size_t count) {
    for (size_t i = 0; i < count; i++) {
        if (!sb_appendn(out, "\\", 1)) return false;
    }
    return true;
}

/* Quote one argument according to the Windows C runtime command-line rules. */
static bool append_cmd_arg(StrBuf* out, const char* arg) {
    if (!out || !arg) return false;
    bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;
    if (!quote) return sb_append(out, arg);
    if (!sb_appendn(out, "\"", 1)) return false;

    const char* p = arg;
    while (*p) {
        size_t backslashes = 0;
        while (*p == '\\') {
            backslashes++;
            p++;
        }
        if (*p == '"') {
            if (!append_backslashes(out, backslashes * 2u + 1u)) return false;
            if (!sb_appendn(out, "\"", 1)) return false;
            p++;
        } else if (*p == '\0') {
            if (!append_backslashes(out, backslashes * 2u)) return false;
        } else {
            if (!append_backslashes(out, backslashes)) return false;
            if (!sb_appendn(out, p, 1)) return false;
            p++;
        }
    }
    return sb_appendn(out, "\"", 1);
}

/* Build a command line for CreateProcessA while preserving target arguments. */
static bool build_cmdline(const char* path, const char* const* argv, int argc, StrBuf* out) {
    if (!path || !out) return false;
    out->len = 0;
    if (out->s) out->s[0] = 0;

    if (!append_cmd_arg(out, path)) return false;

    for (int i = 0; i < argc; i++) {
        const char* a = argv[i];
        if (!a || !sb_appendn(out, " ", 1) || !append_cmd_arg(out, a)) return false;
    }
    return true;
}

/* Resolve a symbol to (real module basename, offset) in *this* process.
   Important: many APIs are forwarded (e.g. kernel32 -> kernelbase). */
typedef struct LocalProcLoc {
    char mod_name[MAX_PATH]; /* lower-case basename, e.g. kernelbase.dll */
    uint64_t offset;
} LocalProcLoc;

static bool local_proc_loc(const char* preferred_mod, const char* sym, LocalProcLoc* out) {
    if (!preferred_mod || !sym || !out) return false;
    memset(out, 0, sizeof(*out));

    HMODULE hPref = GetModuleHandleA(preferred_mod);
    if (!hPref) return false;

    FARPROC p = GetProcAddress(hPref, sym);
    if (!p) return false;

    HMODULE hReal = NULL;
#if defined(__GNUC__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wpedantic"
#endif
    if (!GetModuleHandleExA(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS | GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,
                            (LPCSTR)p, &hReal) || !hReal) {
        hReal = hPref;
    }
#if defined(__GNUC__)
#pragma GCC diagnostic pop
#endif

    char path[MAX_PATH];
    memset(path, 0, sizeof(path));
    if (!GetModuleFileNameA(hReal, path, (DWORD)sizeof(path))) {
        strncpy(out->mod_name, preferred_mod, sizeof(out->mod_name)-1);
        out->mod_name[sizeof(out->mod_name)-1] = 0;
        strlower_inplace(out->mod_name);
    } else {
        const char* base = basename_lower(path);
        strncpy(out->mod_name, base, sizeof(out->mod_name)-1);
        out->mod_name[sizeof(out->mod_name)-1] = 0;
        strlower_inplace(out->mod_name);
    }

    out->offset = (uint64_t)(uintptr_t)p - (uint64_t)(uintptr_t)hReal;
    return true;
}

static bool add_api_bps_if_ready(BpList* bl, HANDLE hProcess, const ModList* mods, DynamicReport* dr, StrBuf* err) {
    if (!bl || !mods) return false;

    const struct {
        const char* preferred_mod;
        const char* sym;
        BpKind kind;
    } apis[] = {
        {"kernel32.dll", "AddVectoredExceptionHandler", BP_API_ADD_VEH},
        {"kernel32.dll", "AddVectoredContinueHandler", BP_API_ADD_VCH},
        {"kernel32.dll", "SetUnhandledExceptionFilter", BP_API_SET_UEF},
```

```
572 |         {"ntdll.dll",    "RtlAddFunctionTable", BP_API_RTL_ADD_FT},
573 |         {"ntdll.dll",    "RtlInstallFunctionTableCallback", BP_API_RTL_INSTALL_FTCB},
574 |     };
575 |
576 |     for (size_t i = 0; i < sizeof(apis)/sizeof(apis[0]); i++) {
577 |         LocalProcLoc loc;
578 |         if (!local_proc_loc(apis[i].preferred_mod, apis[i].sym, &loc)) continue;
579 |
580 |         const ModInfo* m = mods_find(mods, loc.mod_name);
581 |         if (!m) {
582 |             if (dr) dr_append(dr, "[Dynamic] API %s resolved to %s but module not present in target yet\n",
583 |                               apis[i].sym, loc.mod_name);
584 |             continue;
585 |         }
586 |
587 |         uint64_t bp_addr = m->base + loc.offset;
588 |         {
589 |             char what[320];
590 |             snprintf(what, sizeof(what), "%s!%s", loc.mod_name, apis[i].sym);
591 |             if (!bp_add_and_arm_selfcheck(bl, hProcess, mods, dr, bp_addr, apis[i].kind, what, err)) {
592 |                 if (dr) dr_append(dr, "[Dynamic] failed to set API breakpoint %s at 0x%llx\n",
593 |                                   what, (unsigned long long)bp_addr);
594 |             }
595 |         }
596 |     }
597 |
598 |     return true;
599 | }
600 |
601 |
602 | static const char* bp_kind_name(BpKind k) {
603 |     switch (k) {
604 |         case BP_UNWIND_HANDLER: return "unwind_handler";
605 |         case BP_API_ADD_VEH: return "api_add_veh";
606 |         case BP_API_ADD_VCH: return "api_add_vch";
607 |         case BP_API_SET_UEF: return "api_set_uef";
608 |         case BP_API_RTL_ADD_FT: return "api_rtl_add_ft";
609 |         case BP_API_RTL_INSTALL_FTCB: return "api_rtl_install_ftcb";
610 |         case BP_CUSTOM_VEH_HANDLER: return "veh_handler";
611 |         case BP_CUSTOM_VCH_HANDLER: return "vch_handler";
612 |         case BP_CUSTOM_UEF: return "uef_handler";
613 |         default: return "bp";
614 |     }
615 | }
616 |
617 | static void add_trace(DynamicReport* dr, uint64_t rip) {
618 |     if (!dr) return;
619 |     if (dr->trace_count >= (uint32_t)(sizeof(dr->trace)/sizeof(dr->trace[0]))) return;
620 |     dr->trace[dr->trace_count++] = rip;
621 | }
622 |
623 | static void start_trace(DynamicReport* dr,
624 |                         uint32_t trace_steps,
625 |                         DWORD tid,
626 |                         uint64_t rip,
627 |                         bool* tracing,
628 |                         DWORD* trace_tid,
629 |                         uint32_t* trace_left) {
630 |     if (!dr || !tracing || !trace_tid || !trace_left ||
631 |         trace_steps == 0 || dr->trace_count != 0) {
632 |         return;
633 |     }
634 |
635 |     uint32_t budget = trace_steps > 256u ? 256u : trace_steps;
636 |     add_trace(dr, rip);
637 |     *trace_left = budget - 1u;
638 |     *tracing = *trace_left > 0;
639 |     *trace_tid = *tracing ? tid : 0;
640 | }
641 |
642 | static void addr_to_loc(const PeImage* img, uint64_t base, uint64_t addr, char* out, size_t outsz) {
643 |     if (!out || outsz == 0) return;
644 |     out[0] = 0;
645 |     if (!img || addr < base) {
646 |         snprintf(out, outsz, "0x%llx", (unsigned long long)addr);
647 |         return;
648 |     }
649 |     uint32_t rva = (uint32_t)(addr - base);
650 |     const PeSection* s = pe_find_section_for_rva(img, rva);
651 |     if (s) snprintf(out, outsz, "0x%llx (rva=0x%08x sec=%s)", (unsigned long long)addr, (unsigned)rva, s->name);
652 |     else snprintf(out, outsz, "0x%llx (rva=0x%08x)", (unsigned long long)addr, (unsigned)rva);
653 | }
654 |
655 | /* ---------------------------------------------------------------------- */
656 |
657 | bool dynamic_collect_windows(const char* pe_path,
658 |                              const char* const* child_argv,
659 |                              int child_argc,
660 |                              const PeImage* img,
661 |                              const RuntimeFunction* rf,
662 |                              const UnwindSummary* us,
663 |                              const bool* us_ok,
664 |                              size_t rf_count,
665 |                              uint32_t timeout_ms,
666 |                              uint32_t trace_steps,
667 |                              DynamicReport* out,
668 |                              StrBuf* err) {
669 |     if (!out || !err || !pe_path || !img) return false;
670 |     if (rf_count > 0 && (!rf || !us || !us_ok)) return false;
671 |
672 |     dynamic_report_init(out);
673 |     if (!sb_ok(&out->details)) {
674 |         sb_append(err, "Out of memory (dynamic report buffer).\n");
675 |         return false;
676 |     }
677 |     out->enabled = true;
678 |     out->timeout_ms = timeout_ms;
679 |     out->trace_steps = trace_steps > 256u ? 256u : trace_steps;
680 |
681 |     StrBuf cmd;
682 |     if (!sb_init(&cmd, 1024)) {
683 |         sb_append(err, "Out of memory (command line buffer).\n");
684 |         return false;
685 |     }
686 |     if (!build_cmdline(pe_path, child_argv, child_argc, &cmd)) {
```

```
687 |         sb_append(err, "Failed to build command line.\n");
688 |         sb_free(&cmd);
689 |         return false;
690 |     }
691 |
692 |     STARTUPINFOA si;
693 |     PROCESS_INFORMATION pi;
694 |     memset(&si, 0, sizeof(si));
695 |     memset(&pi, 0, sizeof(pi));
696 |     si.cb = sizeof(si);
697 |
698 |     /*
699 |      * Do not use CREATE_SUSPENDED.
700 |      *
701 |      * We want the standard Windows Debug API event stream to start
702 |      * immediately after CreateProcess. On some configurations,
703 |      * creating the debuggee suspended can result in no debug events
704 |      * being delivered (no CREATE_PROCESS_DEBUG_EVENT), which causes
705 |      * a deadlock (the tool waits forever, never resumes the thread).
706 |      */
707 |     DWORD flags = DEBUG_ONLY_THIS_PROCESS;
708 |
709 |     if (!CreateProcessA(NULL, cmd.s, NULL, NULL, FALSE, flags, NULL, NULL, &si, &pi)) {
710 |         sb_appendf(err, "CreateProcess failed (err=%lu)\n", GetLastError());
711 |         sb_free(&cmd);
712 |         return false;
713 |     }
714 |
715 |     out->ran = true;
716 |
717 |     HANDLE hProcess = pi.hProcess;
718 |     DWORD pid = pi.dwProcessId;
719 |
720 |     ModList mods;
721 |     mods_init(&mods);
722 |
723 |     BpList bps;
724 |     bps_init(&bps);
725 |
726 |     /* Collect unique handler RVAs from static parse. */
727 |     uint32_t* handler_rvas = (uint32_t*)calloc(rf_count ? rf_count : 1, sizeof(uint32_t));
728 |     if (!handler_rvas) {
729 |         sb_append(err, "Out of memory (handler RVA list).\n");
730 |         TerminateProcess(hProcess, 0);
731 |         if (pi.hThread) CloseHandle(pi.hThread);
732 |         if (pi.hProcess) CloseHandle(pi.hProcess);
733 |         sb_free(&cmd);
734 |         return false;
735 |     }
736 |     size_t handler_count = 0;
737 |     for (size_t i = 0; i < rf_count; i++) {
738 |         if (!us_ok[i] || !us[i].has_handler) continue;
739 |         uint32_t hr = us[i].handler_rva;
740 |         bool seen = false;
741 |         for (size_t j = 0; j < handler_count; j++) {
742 |             if (handler_rvas[j] == hr) { seen = true; break; }
743 |         }
744 |         if (!seen) handler_rvas[handler_count++] = hr;
745 |     }
746 |     out->unwind_handlers_discovered = (uint32_t)handler_count;
747 |
748 |     DEBUG_EVENT ev;
749 |     bool got_create = false;
750 |     bool debug_loop_ok = true;
751 |     bool process_exited = false;
752 |     bool timed_out = false;
753 |     uint64_t base_main = 0;
754 |
755 |     bool initial_break_ignored = false;
756 |
757 |     uint64_t start = GetTickCount64();
758 |
759 |     PendingRearmList pending;
760 |     pending_rearm_init(&pending);
761 |     bool tracing = false;
762 |     uint32_t trace_left = 0;
763 |     DWORD trace_tid = 0;
764 |
765 |     /* Let debug loop run */
766 |     for (;;) {
767 |         uint64_t elapsed = GetTickCount64() - start;
768 |         if (elapsed > timeout_ms) {
769 |             dr_append(out, "[Dynamic] timeout reached (%u ms)\n", timeout_ms);
770 |             dynamic_event_add(out, &mods, "timeout", 0, 0, 0, false, timeout_ms, 0, 0, 0);
771 |             timed_out = true;
772 |             break;
773 |         }
774 |
775 |         if (!WaitForDebugEvent(&ev, 100)) {
776 |             DWORD werr = GetLastError();
777 |             if (werr == ERROR_SEM_TIMEOUT) {
778 |                 continue;
779 |             }
780 |             dr_append(out, "[Dynamic] WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);
781 |             sb_appendf(err, "WaitForDebugEvent failed (err=%lu)\n", (unsigned long)werr);
782 |             debug_loop_ok = false;
783 |             break;
784 |         }
785 |
786 |         DWORD cont = DBG_CONTINUE;
787 |         bool finish_after_continue = false;
788 |
789 |         if (ev.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT) {
790 |             got_create = true;
791 |             base_main = (uint64_t)(uintptr_t)ev.u.CreateProcessInfo.lpBaseOfImage;
792 |             dr_append(out, "[Dynamic] CREATE_PROCESS pid=%lu base=0x%llx\n", (unsigned long)pid, (unsigned long long)base_main);
793 |
794 |             /* refresh module list and add API breakpoints */
795 |             (void)mods_refresh_accounted(&mods, pid, out);
796 |             add_api_bps_if_ready(&bps, hProcess, &mods, out, err);
797 |
798 |             /* Set breakpoints on handlers referenced in UNWIND_INFO */
799 |             for (size_t i = 0; i < handler_count; i++) {
800 |                 uint64_t a = base_main + (uint64_t)handler_rvas[i];
801 |                 char what[96];
```

```
                snprintf(what, sizeof(what), "unwind_handler_rva=0x%08lx", (unsigned long)handler_rvas[i]);
                if (bp_add_and_arm_selfcheck(&bps, hProcess, &mods, out, a, BP_UNWIND_HANDLER, what, err)) {
                    out->unwind_handler_bp_set++;
                } else {
                    /* best effort: ignore failures (some handlers could be invalid/outside main image) */
                }
            }

            /* Process continues after ContinueDebugEvent(). */
            CloseHandle(ev.u.CreateProcessInfo.hFile);
        }
        else if (ev.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT) {
            /* keep module list updated so we can set API breakpoints */
            (void)mods_refresh_accounted(&mods, pid, out);
            add_api_bps_if_ready(&bps, hProcess, &mods, out, err);
            if (ev.u.LoadDll.hFile) CloseHandle(ev.u.LoadDll.hFile);
        }
        else if (ev.dwDebugEventCode == UNLOAD_DLL_DEBUG_EVENT) {
            (void)mods_refresh_accounted(&mods, pid, out);
        }
        else if (ev.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT) {
            dr_append(out, "[Dynamic] EXIT_PROCESS code=%lu\n", (unsigned long)ev.u.ExitProcess.dwExitCode);
            process_exited = true;
            out->target_exited = true;
            out->target_exit_code = (uint32_t)ev.u.ExitProcess.dwExitCode;
            dynamic_event_add(
                out, &mods, "process_exit", ev.dwThreadId, 0, 0, false,
                ev.u.ExitProcess.dwExitCode, 0, 0, 0);
            finish_after_continue = true;
        }
        else if (ev.dwDebugEventCode == EXCEPTION_DEBUG_EVENT) {
            DWORD code = ev.u.Exception.ExceptionRecord.ExceptionCode;
            uint64_t addr = (uint64_t)(uintptr_t)ev.u.Exception.ExceptionRecord.ExceptionAddress;

            if (code != EXCEPTION_BREAKPOINT && code != EXCEPTION_SINGLE_STEP) {
                bool first_chance = ev.u.Exception.dwFirstChance != 0;
                out->exceptions_seen++;
                if (first_chance) out->first_chance_seen++;
                else out->second_chance_seen++;
                dr_append(out, "[Dynamic] exception 0x%08lx at 0x%llx (%s chance)\n",
                          (unsigned long)code,
                          (unsigned long long)addr,
                          first_chance ? "first" : "second");
                dynamic_event_add(
                    out, &mods, "exception", ev.dwThreadId, addr, (uint32_t)code,
                    first_chance, 0, 0, 0, 0);
                /* Preserve the target's normal exception-dispatch flow. */
                cont = DBG_EXCEPTION_NOT_HANDLED;
            }

            if (code == EXCEPTION_BREAKPOINT) {
                Breakpoint* bp = bps_find(&bps, addr);
                if (!bp && !initial_break_ignored) {
                    /* Ignore the debugger's system initial breakpoint only when it is not ours. */
                    initial_break_ignored = true;
                    cont = DBG_CONTINUE;
                } else if (!bp) {
                    cont = DBG_EXCEPTION_NOT_HANDLED;
                } else {
                    bool bp_prepared = bp_clear(hProcess, bp, err);
                    if (bp_prepared) {
                        bp->rearm_pending = true;
                        bp_prepared = pending_rearm_set(&pending, ev.dwThreadId, bp);
                        if (!bp_prepared) {
                            bp->rearm_pending = false;
                            out->pending_rearm_failures++;
                            out->instrumentation_complete = false;
                        }
                    }
                    if (!bp_prepared) {
                        dr_append(out, "[Dynamic] failed to prepare breakpoint rearm at 0x%llx\n",
                                  (unsigned long long)addr);
                        debug_loop_ok = false;
                        finish_after_continue = true;
                        cont = DBG_CONTINUE;
                    } else {
                        HANDLE hEvThread = open_event_thread(ev.dwThreadId);
                        if (!hEvThread) {
                            dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",
                                      (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                            debug_loop_ok = false;
                            finish_after_continue = true;
                            cont = DBG_CONTINUE;
                        } else {
                            CONTEXT ctx;
                            memset(&ctx, 0, sizeof(ctx));
                            ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;

                            if (!GetThreadContext(hEvThread, &ctx)) {
                                dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",
                                          (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                                close_event_thread(hEvThread);
                                debug_loop_ok = false;
                                finish_after_continue = true;
                                cont = DBG_CONTINUE;
                            } else {
                                /* RIP points to INT3+1; rewind */
                                ctx.Rip = (DWORD64)addr;
                                ctx.EFlags |= 0x100; /* Trap flag */

                                /* handle API breaks: capture handler pointers */
                                if (bp->kinds & BP_API_ADD_VEH) {
                                    uint64_t first = (uint64_t)ctx.Rcx;
                                    uint64_t handler = (uint64_t)ctx.Rdx;
                                    out->veh_registrations++;
                                    dr_append(out, "[Dynamic] AddVectoredExceptionHandler first=%llu handler=0x%llx\n",
                                              (unsigned long long)first, (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "veh_registration", ev.dwThreadId, addr, 0, false,
                                        first, handler, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_VEH_HANDLER, "custom_veh_handler", err);
```

```
                                    }
                                }
                                if (bp->kinds & BP_API_ADD_VCH) {
                                    uint64_t first = (uint64_t)ctx.Rcx;
                                    uint64_t handler = (uint64_t)ctx.Rdx;
                                    out->vch_registrations++;
                                    dr_append(out, "[Dynamic] AddVectoredContinueHandler first=%llu handler=0x%llx\n",
                                              (unsigned long long)first, (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "vch_registration", ev.dwThreadId, addr, 0, false,
                                        first, handler, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_VCH_HANDLER, "custom_vch_handler", err);
                                    }
                                }
                                if (bp->kinds & BP_API_SET_UEF) {
                                    out->uef_set_calls++;
                                    uint64_t handler = (uint64_t)ctx.Rcx;
                                    dr_append(out, "[Dynamic] SetUnhandledExceptionFilter handler=0x%llx\n",
                                              (unsigned long long)handler);
                                    dynamic_event_add(
                                        out, &mods, "uef_registration", ev.dwThreadId, addr, 0, false,
                                        handler, 0, 0, 0);
                                    if (handler) {
                                        (void)mods_refresh_accounted(&mods, pid, out);
                                        (void)bp_add_and_arm_selfcheck(
                                            &bps, hProcess, &mods, out, handler,
                                            BP_CUSTOM_UEF, "custom_uef_handler", err);
                                    }
                                }
                                if (bp->kinds & BP_API_RTL_ADD_FT) {
                                    uint64_t table = (uint64_t)ctx.Rcx;
                                    uint64_t count = (uint64_t)ctx.Rdx;
                                    uint64_t baseaddr = (uint64_t)ctx.R8;
                                    out->rtl_add_function_table_calls++;
                                    dr_append(
                                        out,
                                        "[Dynamic] RtlAddFunctionTable table=0x%llx count=%llu base=0x%llx\n",
                                        (unsigned long long)table,
                                        (unsigned long long)count,
                                        (unsigned long long)baseaddr);
                                    dynamic_event_add(
                                        out, &mods, "rtl_add_function_table",
                                        ev.dwThreadId, addr, 0, false,
                                        table, count, baseaddr, 0);
                                }
                                if (bp->kinds & BP_API_RTL_INSTALL_FTCB) {
                                    uint64_t table_id = (uint64_t)ctx.Rcx;
                                    uint64_t baseaddr = (uint64_t)ctx.Rdx;
                                    uint64_t length = (uint64_t)ctx.R8;
                                    uint64_t callback = (uint64_t)ctx.R9;
                                    out->rtl_install_function_table_cb_calls++;
                                    dr_append(
                                        out,
                                        "[Dynamic] RtlInstallFunctionTableCallback id=0x%llx base=0x%llx length=%llu
callback=0x%llx\n",
                                        (unsigned long long)table_id,
                                        (unsigned long long)baseaddr,
                                        (unsigned long long)length,
                                        (unsigned long long)callback);
                                    dynamic_event_add(
                                        out, &mods, "rtl_install_function_table_callback",
                                        ev.dwThreadId, addr, 0, false,
                                        table_id, baseaddr, length, callback);
                                }
                                if (bp->kinds & BP_UNWIND_HANDLER) {
                                    out->unwind_handler_hits++;
                                    char loc[128];
                                    addr_to_loc(img, base_main, addr, loc, sizeof(loc));
                                    dr_append(out, "[Dynamic] hit unwind handler at %s\n", loc);
                                    dynamic_event_add(
                                        out, &mods, "unwind_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_VEH_HANDLER) {
                                    out->veh_hits++;
                                    dr_append(out, "[Dynamic] hit vectored exception handler 0x%llx\n", (unsigned long long)addr);
                                    dynamic_event_add(
                                        out, &mods, "veh_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_VCH_HANDLER) {
                                    out->vch_hits++;
                                    dr_append(out, "[Dynamic] hit vectored continue handler 0x%llx\n", (unsigned long long)addr);
                                    dynamic_event_add(
                                        out, &mods, "vch_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }
                                if (bp->kinds & BP_CUSTOM_UEF) {
                                    out->uef_hits++;
                                    dr_append(out, "[Dynamic] hit unhandled exception filter 0x%llx\n", (unsigned long long)addr);
                                    dynamic_event_add(
                                        out, &mods, "uef_handler_hit",
                                        ev.dwThreadId, addr, 0, false, 0, 0, 0, 0);
                                    start_trace(
                                        out, out->trace_steps, ev.dwThreadId,
                                        (uint64_t)ctx.Rip,
                                        &tracing, &trace_tid, &trace_left);
                                }

```

```
                                if (!SetThreadContext(hEvThread, &ctx)) {
                                    dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",
                                              (unsigned long)ev.dwThreadId,
                                              (unsigned long)GetLastError());
                                    debug_loop_ok = false;
                                    finish_after_continue = true;
                                }
                                close_event_thread(hEvThread);
                                cont = DBG_CONTINUE;
                            }
                        }
                    }
                }
            }
            else if (code == EXCEPTION_SINGLE_STEP) {
                /* Re-arm the breakpoint after executing original instruction (per-thread). */
                Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);
                if (pr_bp) {
                    StrBuf te; sb_init(&te, 128);
                    if (!bp_set(hProcess, pr_bp, &te)) {
                        out->breakpoint_rearm_failures++;
                        out->instrumentation_complete = false;
                        dr_append(out, "[Dynamic] failed to re-arm breakpoint at 0x%llx: %s",
                                  (unsigned long long)pr_bp->addr,
                                  te.s ? te.s : "unknown error\n");
                        debug_loop_ok = false;
                        finish_after_continue = true;
                    }
                    sb_free(&te);
                }

                HANDLE hEvThread = open_event_thread(ev.dwThreadId);
                if (!hEvThread) {
                    dr_append(out, "[Dynamic] OpenThread failed tid=%lu (err=%lu)\n",
                              (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                    debug_loop_ok = false;
                    finish_after_continue = true;
                    cont = DBG_CONTINUE;
                } else {
                    CONTEXT ctx;
                    memset(&ctx, 0, sizeof(ctx));
                    ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;

                    if (!GetThreadContext(hEvThread, &ctx)) {
                        dr_append(out, "[Dynamic] GetThreadContext failed tid=%lu (err=%lu)\n",
                                  (unsigned long)ev.dwThreadId, (unsigned long)GetLastError());
                        close_event_thread(hEvThread);
                        debug_loop_ok = false;
                        finish_after_continue = true;
                        cont = DBG_CONTINUE;
                    } else {
                        if (tracing && trace_left && ev.dwThreadId == trace_tid) {
                            add_trace(out, (uint64_t)ctx.Rip);
                            trace_left--;
                            /* keep trap flag */
                            ctx.EFlags |= 0x100;
                            if (trace_left == 0) {
                                tracing = false;
                                trace_tid = 0;
                                ctx.EFlags &= ~0x100u;
                                dr_append(out, "[Dynamic] trace collected: %u steps\n", (unsigned)out->trace_count);
                            }
                        } else {
                            /* stop single-stepping */
                            ctx.EFlags &= ~0x100u;
                        }
                        if (!SetThreadContext(hEvThread, &ctx)) {
                            dr_append(out, "[Dynamic] SetThreadContext failed tid=%lu (err=%lu)\n",
                                      (unsigned long)ev.dwThreadId,
                                      (unsigned long)GetLastError());
                            debug_loop_ok = false;
                            finish_after_continue = true;
                        }
                        close_event_thread(hEvThread);
                        cont = DBG_CONTINUE;
                    }
                }
            }

        }
        else if (ev.dwDebugEventCode == EXIT_THREAD_DEBUG_EVENT) {
            Breakpoint* pr_bp = pending_rearm_take(&pending, ev.dwThreadId);
            if (pr_bp) {
                StrBuf te;
                sb_init(&te, 128);
                if (!bp_set(hProcess, pr_bp, &te)) {
                    out->breakpoint_rearm_failures++;
                    out->instrumentation_complete = false;
                    dr_append(out, "[Dynamic] failed to re-arm breakpoint after thread exit: %s",
                              te.s ? te.s : "unknown error\n");
                    debug_loop_ok = false;
                    finish_after_continue = true;
                }
                sb_free(&te);
            }
            if (tracing && trace_tid == ev.dwThreadId) {
                tracing = false;
                trace_tid = 0;
                trace_left = 0;
            }
        }

        if (!ContinueDebugEvent(ev.dwProcessId, ev.dwThreadId, cont)) {
            DWORD cerr = GetLastError();
            dr_append(out, "[Dynamic] ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);
            sb_appendf(err, "ContinueDebugEvent failed (err=%lu)\n", (unsigned long)cerr);
            debug_loop_ok = false;
            break;
        }
        if (finish_after_continue) break;
    }

    out->timed_out = timed_out;
    out->target_exited = process_exited;
    out->ok = got_create && debug_loop_ok && !timed_out;
```

```
    if (!sb_ok(&out->details)) {
        out->ok = false;
        sb_append(err, "Out of memory while building dynamic details.\n");
    }

    /* Cleanup: remove breakpoints */
    for (size_t i = 0; i < bps.count; i++) {
        bp_clear(hProcess, &bps.bps[i], NULL);
    }

    /*
     * Terminate only when the target did not exit by itself. A debuggee may
     * not signal its process handle until the debugger continues the queued
     * exit event, so drain cleanup events instead of waiting a fixed 5 s.
     */
    if (!process_exited && TerminateProcess(hProcess, 0)) {
        uint64_t cleanup_deadline = GetTickCount64() + 1000u;
        while (WaitForSingleObject(hProcess, 0) != WAIT_OBJECT_0 &&
               GetTickCount64() < cleanup_deadline) {
            DEBUG_EVENT cleanup_event;
            if (!WaitForDebugEvent(&cleanup_event, 50)) {
                if (GetLastError() == ERROR_SEM_TIMEOUT) continue;
                break;
            }

            if (cleanup_event.dwDebugEventCode == CREATE_PROCESS_DEBUG_EVENT &&
                cleanup_event.u.CreateProcessInfo.hFile) {
                CloseHandle(cleanup_event.u.CreateProcessInfo.hFile);
            } else if (cleanup_event.dwDebugEventCode == LOAD_DLL_DEBUG_EVENT &&
                       cleanup_event.u.LoadDll.hFile) {
                CloseHandle(cleanup_event.u.LoadDll.hFile);
            }

            bool cleanup_exited =
                cleanup_event.dwDebugEventCode == EXIT_PROCESS_DEBUG_EVENT;
            if (!ContinueDebugEvent(
                    cleanup_event.dwProcessId,
                    cleanup_event.dwThreadId,
                    DBG_CONTINUE)) {
                break;
            }
            if (cleanup_exited) break;
        }
        (void)WaitForSingleObject(hProcess, 100);
    }

    if (pi.hThread) CloseHandle(pi.hThread);
    if (pi.hProcess) CloseHandle(pi.hProcess);

    free(handler_rvas);
    sb_free(&cmd);

    if (!out->ok) {
        if (timed_out) {
            sb_appendf(err, "Dynamic session timed out after %u ms.\n", timeout_ms);
        } else if (!got_create) {
            sb_append(err, "Dynamic session failed before CREATE_PROCESS_DEBUG_EVENT.\n");
        } else {
            sb_append(err, "Dynamic session failed inside the debug event loop.\n");
        }
        return false;
    }
    return true;
}

#else /* !_WIN32 */

void dynamic_report_init(DynamicReport* dr) {
    if (!dr) return;
    memset(dr, 0, sizeof(*dr));
    dr->enabled = false;
    sb_init(&dr->details, 256);
    sb_append(&dr->details, "Dynamic mode is supported only on Windows.\n");
}

void dynamic_report_free(DynamicReport* dr) {
    if (!dr) return;
    sb_free(&dr->details);
    memset(dr, 0, sizeof(*dr));
}

bool dynamic_collect_windows(const char* pe_path,
                             const char* const* child_argv,
                             int child_argc,
                             const PeImage* img,
                             const RuntimeFunction* rf,
                             const UnwindSummary* us,
                             const bool* us_ok,
                             size_t rf_count,
                             uint32_t timeout_ms,
                             uint32_t trace_steps,
                             DynamicReport* out,
                             StrBuf* err) {
    (void)pe_path; (void)child_argv; (void)child_argc; (void)img; (void)rf; (void)us; (void)us_ok; (void)rf_count;
    (void)timeout_ms; (void)trace_steps;
    dynamic_report_init(out);
    if (!out || !sb_ok(&out->details)) {
        sb_append(err, "Out of memory (dynamic report buffer).\n");
        return false;
    }
    out->enabled = true;
    out->timeout_ms = timeout_ms;
    out->trace_steps = trace_steps > 256u ? 256u : trace_steps;
    sb_append(err, "Dynamic mode is supported only on Windows.\n");
    return false;
}

#endif
```

include/report.h

```
#pragma once
/*
  report.h - output helpers (stdout, CSV, JSON)
*/

#include <stddef.h>
#include <stdbool.h>
#include "pe.h"
#include "unwind.h"
#include "heuristics.h"
#include "dynamic.h"
#include "util.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
    bool strict_abi;
    bool follow_chain;
    uint32_t max_chain_depth;
} ReportMetadata;

void report_print_brief(const char* path, const PeImage* img, size_t rf_count);

bool report_write_csv(const char* path,
                      const PeImage* img,
                      const RuntimeFunction* rf,
                      const UnwindSummary* us,
                      const bool* us_ok,
                      size_t n,
                      bool ghidra_mode,
                      StrBuf* err);

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
                          const DynamicReport* dr,
                          const ReportMetadata* metadata,
                          StrBuf* err);

bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err);

void report_print_heuristics(const HeuristicReport* hr);

#ifdef __cplusplus
}
#endif
```

src/report.c

```
#include "report.h"
#include "app_info.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define IMAGE_SCN_MEM_EXECUTE 0x20000000u
#define IMAGE_SCN_MEM_READ    0x40000000u
#define IMAGE_SCN_MEM_WRITE   0x80000000u

static void append_regmask(StrBuf* sb, uint16_t mask, bool xmm) {
    bool first = true;
    for (uint8_t r = 0; r < 16; r++) {
        if (!(mask & (uint16_t)(1u << r))) continue;
        if (!first) sb_append(sb, ",");
        if (xmm) sb_appendf(sb, "XMM%u", (unsigned)r);
        else sb_append(sb, unwind_reg_name(r));
        first = false;
    }
    if (first) sb_append(sb, "-");
}

static void anomalies_to_str(StrBuf* sb, uint32_t anom) {
    bool first = true;
    struct { uint32_t bit; const char* name; } m[] = {
        {UA_VERSION_UNKNOWN, "ver"},
        {UA_TRUNCATED, "trunc"},
        {UA_BAD_OPCODE, "op"},
        {UA_BAD_OPERAND, "arg"},
        {UA_CODEOFFSET_OUTSIDE, "off"},
        {UA_CHAIN_LOOP, "chain_loop"},
        {UA_CHAIN_DEPTH_EXCEEDED, "chain_depth"},
        {UA_HANDLER_OOB, "handler_oob"},
        {UA_CODEOFFSET_ORDER, "order"},
        {UA_OPERAND_SANITY, "operand_sanity"},
        {UA_ALIGNMENT, "alignment"},
        {UA_CHAIN_INVALID, "chain_invalid"},
        {UA_ABI_REGISTER_CLASS, "abi_register_class"},
        {UA_ABI_RESERVED_OPINFO, "abi_reserved_opinfo"},
        {UA_ABI_PUSH_ORDER, "abi_push_order"},
        {UA_ABI_CHAIN_RESTRICTION, "abi_chain_restriction"},
        {UA_ABI_SAVE_OFFSET, "abi_save_offset"},
    };
    for (size_t i = 0; i < sizeof(m)/sizeof(m[0]); i++) {
        if (!(anom & m[i].bit)) continue;
        if (!first) sb_append(sb, "|");
        sb_append(sb, m[i].name);
        first = false;
    }
    if (first) sb_append(sb, "0");
}

static const char* section_name_or_dash(const PeImage* img, uint32_t rva) {
    const PeSection* s = pe_find_section_for_rva(img, rva);
    return s ? s->name : "-";
}

static uint32_t runtime_function_size(const RuntimeFunction* r) {
    if (!r || r->end_rva < r->begin_rva) return 0;
    return r->end_rva - r->begin_rva;
}

static bool csv_cell(StrBuf* out, const char* s) {
    if (!out || !s) return false;
    bool need_q = false;
    for (const char* p = s; *p; p++) {
        if (*p == ',' || *p == '"' || *p == '\n' || *p == '\r') { need_q = true; break; }
    }
    if (!need_q) return sb_append(out, s);

    if (!sb_appendn(out, "\"", 1)) return false;
    for (const char* p = s; *p; p++) {
        if (*p == '"' && !sb_appendn(out, "\"", 1)) return false;
        if (!sb_appendn(out, p, 1)) return false;
    }
    return sb_appendn(out, "\"", 1);
}

static bool build_unwind_comment(const UnwindSummary* u, StrBuf* sb) {
    sb->len = 0;
    if (sb->s) sb->s[0] = 0;

    char fbuf[64];
    unwind_flags_to_string(u->flags, fbuf);

    sb_appendf(sb, "UNWIND v%u flags=%s prolog=%u stack=%uB frame=%s+%u ",
               (unsigned)u->version, fbuf, (unsigned)u->prolog_size,
               (unsigned)u->fixed_stack_bytes,
               (u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),
               (unsigned)(u->frame_off * 16u));

    sb_append(sb, "saved_gp=[");
    append_regmask(sb, u->saved_nonvol_mask, false);
    sb_append(sb, "] saved_xmm=[");
    append_regmask(sb, u->saved_xmm_mask, true);
    sb_append(sb, "]");

    if (u->has_handler) sb_appendf(sb, " handler_rva=0x%08x", (unsigned)u->handler_rva);
    if (u->has_chain) {
        sb_appendf(sb, " chain_to=[0x%08x..0x%08x] u=0x%08x",
                   (unsigned)u->chain.begin_rva,
                   (unsigned)u->chain.end_rva,
                   (unsigned)u->chain.unwind_rva);
    }

    if (u->anomalies) {
        sb_append(sb, " anomalies=");
        anomalies_to_str(sb, u->anomalies);
    }
    return sb_ok(sb);
}
```

```
112 |
113 | void report_print_brief(const char* path, const PeImage* img, size_t rf_count) {
114 |     printf("File: %s\n", path);
115 |     printf("Machine: 0x%04x (AMD64)\n", (unsigned)img->machine);
116 |     printf("ImageBase: 0x%llx\n", (unsigned long long)img->image_base);
117 |     printf("SizeOfImage: 0x%x\n", (unsigned)img->size_of_image);
118 |     printf("SizeOfHeaders: 0x%x  Sections: %zu\n",
119 |            (unsigned)img->size_of_headers, img->num_sections);
120 |     printf("Exception Directory: RVA=0x%08x Size=0x%x\n", (unsigned)img->exception_rva, (unsigned)img->exception_size);
121 |     printf("RUNTIME_FUNCTION entries: %zu\n", rf_count);
122 | }
123 |
124 | bool report_write_csv(const char* path,
125 |                       const PeImage* img,
126 |                       const RuntimeFunction* rf,
127 |                       const UnwindSummary* us,
128 |                       const bool* us_ok,
129 |                       size_t n,
130 |                       bool ghidra_mode,
131 |                       StrBuf* err) {
132 |     (void)img;
133 |     if (!path || (!rf && n) || (!us && n) || (!us_ok && n)) {
134 |         sb_append(err, "Invalid arguments for CSV report.\n");
135 |         return false;
136 |     }
137 |
138 |     StrBuf output = {0};
139 |     StrBuf comment = {0};
140 |     if (!sb_init(&output, 8192) || !sb_init(&comment, 512)) {
141 |         sb_free(&comment);
142 |         sb_free(&output);
143 |         sb_append(err, "OOM.\n");
144 |         return false;
145 |     }
146 |
147 |     if (ghidra_mode) {
148 |         sb_append(&output, "begin_rva,end_rva,name,comment\n");
149 |         for (size_t i = 0; i < n; i++) {
150 |             const RuntimeFunction* r = &rf[i];
151 |             const UnwindSummary* u = &us[i];
152 |
153 |             char name[64];
154 |             snprintf(name, sizeof(name), "sub_%08X", (unsigned)r->begin_rva);
155 |
156 |             if (us_ok[i]) build_unwind_comment(u, &comment);
157 |             else {
158 |                 comment.len = 0;
159 |                 if (comment.s) comment.s[0] = 0;
160 |                 sb_append(&comment, "UNWIND parse failed");
161 |                 if (u->parse_error[0]) sb_appendf(&comment, ": %s", u->parse_error);
162 |             }
163 |
164 |             char a[32], b[32];
165 |             snprintf(a, sizeof(a), "0x%08X", (unsigned)r->begin_rva);
166 |             snprintf(b, sizeof(b), "0x%08X", (unsigned)r->end_rva);
167 |
168 |             csv_cell(&output, a); sb_appendn(&output, ",", 1);
169 |             csv_cell(&output, b); sb_appendn(&output, ",", 1);
170 |             csv_cell(&output, name); sb_appendn(&output, ",", 1);
171 |             csv_cell(&output, comment.s ? comment.s : "");
172 |             sb_appendn(&output, "\n", 1);
173 |         }
174 |     } else {
175 |         sb_append(&output,
176 |                   "index,begin_rva,end_rva,size,unwind_rva,unwind_ver,"
177 |                   "unwind_flags,prolog_size,fixed_stack,frame_reg,frame_off,"
178 |                   "handler_rva,has_chain,anomalies,parse_error\n");
179 |         for (size_t i = 0; i < n; i++) {
180 |             const RuntimeFunction* r = &rf[i];
181 |             const UnwindSummary* u = &us[i];
182 |             char fbuf[64];
183 |             unwind_flags_to_string(u->flags, fbuf);
184 |
185 |             sb_appendf(&output, "%zu,0x%08X,0x%08X,%u,0x%08X,",
186 |                        i, (unsigned)r->begin_rva, (unsigned)r->end_rva,
187 |                        (unsigned)runtime_function_size(r),
188 |                        (unsigned)r->unwind_rva);
189 |
190 |             if (!us_ok[i]) {
191 |                 sb_appendf(&output, "NA,NA,NA,NA,NA,NA,NA,0,0x%08X,",
192 |                            (unsigned)u->anomalies);
193 |                 csv_cell(&output, u->parse_error);
194 |                 sb_appendn(&output, "\n", 1);
195 |                 continue;
196 |             }
197 |
198 |             sb_appendf(&output, "%u,%s,%u,%u,%s,%u,0x%08X,%u,0x%08X,",
199 |                        (unsigned)u->version, fbuf, (unsigned)u->prolog_size,
200 |                        (unsigned)u->fixed_stack_bytes,
201 |                        (u->frame_reg ? unwind_reg_name(u->frame_reg) : "-"),
202 |                        (unsigned)(u->frame_off * 16u),
203 |                        (unsigned)u->handler_rva,
204 |                        (unsigned)(u->has_chain ? 1u : 0u),
205 |                        (unsigned)u->anomalies);
206 |             csv_cell(&output, "");
207 |             sb_appendn(&output, "\n", 1);
208 |         }
209 |     }
210 |
211 |     bool ok = sb_ok(&output) && sb_ok(&comment);
212 |     if (ok) ok = file_write_atomic(path, output.s, output.len);
213 |     sb_free(&comment);
214 |     sb_free(&output);
215 |     if (!ok) sb_appendf(err, "Failed while building or writing CSV: %s\n", path);
216 |     return ok;
217 | }
218 |
219 | static size_t valid_utf8_sequence_length(const unsigned char* s, size_t remaining) {
220 |     if (!s || remaining == 0) return 0;
221 |     if (s[0] < 0x80u) return 1;
222 |     if (s[0] >= 0xc2u && s[0] <= 0xdfu) {
223 |         return remaining >= 2 && s[1] >= 0x80u && s[1] <= 0xbfu ? 2 : 0;
224 |     }
225 |     if (s[0] == 0xe0u) {
226 |         return remaining >= 3 && s[1] >= 0xa0u && s[1] <= 0xbfu &&
```

```
                s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if ((s[0] >= 0xe1u && s[0] <= 0xecu) || (s[0] >= 0xeeu && s[0] <= 0xefu)) {
        return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0xbfu &&
                s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if (s[0] == 0xedu) {
        return remaining >= 3 && s[1] >= 0x80u && s[1] <= 0x9fu &&
                s[2] >= 0x80u && s[2] <= 0xbfu ? 3 : 0;
    }
    if (s[0] == 0xf0u) {
        return remaining >= 4 && s[1] >= 0x90u && s[1] <= 0xbfu &&
                s[2] >= 0x80u && s[2] <= 0xbfu &&
                s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    if (s[0] >= 0xf1u && s[0] <= 0xf3u) {
        return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0xbfu &&
                s[2] >= 0x80u && s[2] <= 0xbfu &&
                s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    if (s[0] == 0xf4u) {
        return remaining >= 4 && s[1] >= 0x80u && s[1] <= 0x8fu &&
                s[2] >= 0x80u && s[2] <= 0xbfu &&
                s[3] >= 0x80u && s[3] <= 0xbfu ? 4 : 0;
    }
    return 0;
}

static void json_escape(StrBuf* sb, const char* s) {
    if (!s) return;
    size_t length = strlen(s);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)s[i];
        switch (c) {
            case '\\': sb_append(sb, "\\\\"); break;
            case '"':  sb_append(sb, "\\\""); break;
            case '\n': sb_append(sb, "\\n"); break;
            case '\r': sb_append(sb, "\\r"); break;
            case '\t': sb_append(sb, "\\t"); break;
            default:
                if (c < 0x20) sb_appendf(sb, "\\u%04x", (unsigned)c);
                else if (c < 0x80) sb_appendn(sb, &s[i], 1);
                else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&s[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &s[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "\\u%04x", (unsigned)c);
                    }
                }
        }
    }
}

static void json_key(StrBuf* sb, const char* k) {
    sb_append(sb, "\"");
    json_escape(sb, k);
    sb_append(sb, "\":");
}

static void json_string(StrBuf* sb, const char* s) {
    sb_append(sb, "\"");
    json_escape(sb, s ? s : "");
    sb_append(sb, "\"");
}

static void json_string_field(StrBuf* sb, const char* k, const char* v, bool trailing_comma) {
    json_key(sb, k);
    json_string(sb, v);
    if (trailing_comma) sb_append(sb, ",");
}

static void json_bool_field(StrBuf* sb, const char* k, bool v, bool trailing_comma) {
    json_key(sb, k);
    sb_append(sb, v ? "true" : "false");
    if (trailing_comma) sb_append(sb, ",");
}

static void json_heuristic_families(StrBuf* sb, uint32_t mask) {
    sb_append(sb, "[");
    bool first = true;
    uint32_t bits[] = {
        HEF_TABLE_INTEGRITY,
        HEF_UNWIND_INTEGRITY,
        HEF_METADATA_PLACEMENT,
        HEF_UNWIND_REUSE,
        HEF_HANDLER_DOMINANCE
    };
    for (size_t i = 0; i < sizeof(bits)/sizeof(bits[0]); i++) {
        if (!(mask & bits[i])) continue;
        if (!first) sb_append(sb, ",");
        json_string(sb, heuristics_evidence_family_name(bits[i]));
        first = false;
    }
    sb_append(sb, "]");
}

static unsigned dynamic_signal_count(const DynamicReport* dr) {
    unsigned n = 0;
    if (!dr) return 0;
    if (dr->unwind_handler_hits) n++;
    if (dr->veh_hits || dr->veh_registrations) n++;
    if (dr->vch_hits || dr->vch_registrations) n++;
    if (dr->uef_hits || dr->uef_set_calls) n++;
    if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) n++;
    if (dr->second_chance_seen) n++;
    return n;
}

static bool dynamic_session_complete(const DynamicReport* dr) {
    return dr &&
           dr->ran &&
           dr->ok &&
```

```
            !dr->timed_out &&
            dr->target_exited &&
            dr->instrumentation_complete;
}

static bool has_static_dynamic_correlation(const HeuristicReport* hr, const DynamicReport* dr) {
    bool suspicious_static_handler_context = false;
    if (!hr || !dynamic_session_complete(dr)) return false;
    suspicious_static_handler_context =
        (hr->handler_nonexec_section > 0) ||
        (hr->handler_writable_section > 0) ||
        ((hr->evidence_families_mask & HEF_HANDLER_DOMINANCE) != 0u);
    if (suspicious_static_handler_context && (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits)) return true;
    if ((hr->unwind_exec_section || hr->unwind_unusual_section || hr->unwind_writable_section ||
         hr->handler_nonexec_section || hr->handler_writable_section) &&
        (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls)) return true;
    return false;
}


static const char* runtime_evidence_level(const DynamicReport* dr) {
    if (!dr || !dr->ran) return "none";
    unsigned strong = 0, medium = 0, weak = 0;
    if (dr->unwind_handler_hits || dr->veh_hits || dr->vch_hits || dr->uef_hits) strong++;
    if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) medium++;
    if (dr->second_chance_seen) medium++;
    if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls || dr->first_chance_seen) weak++;
    const char* level = "none";
    if (strong >= 1 && medium >= 1) level = "strong";
    else if (strong >= 1 || medium >= 2) level = "medium";
    else if (medium >= 1 || weak >= 1) level = "weak";
    if (!dynamic_session_complete(dr) && strcmp(level, "none") != 0) return "weak";
    return level;
}

static const char* case_strength(const HeuristicReport* hr, const DynamicReport* dr, bool correlated) {
    const char* static_lvl = (hr && hr->evidence_level[0]) ? hr->evidence_level : "none";
    const char* runtime_lvl = runtime_evidence_level(dr);
    bool static_strong = strcmp(static_lvl, "strong") == 0;
    bool static_medium = strcmp(static_lvl, "medium") == 0;
    bool static_weak = strcmp(static_lvl, "weak") == 0;
    bool runtime_strong = strcmp(runtime_lvl, "strong") == 0;
    bool runtime_medium = strcmp(runtime_lvl, "medium") == 0;
    bool runtime_weak = strcmp(runtime_lvl, "weak") == 0;

    if (correlated && (static_strong || runtime_strong)) return "strong_correlated_case";
    if (correlated && (static_medium || runtime_medium)) return "medium_correlated_case";
    if (static_strong || runtime_strong) return "strong_unconfirmed_case";
    if (static_medium || runtime_medium) return "medium_case";
    if (static_weak || runtime_weak) return "weak_case";
    return "no_case";
}

typedef struct AssessmentSummary {
    bool correlated;
    unsigned dynamic_signals;
    double final_confidence;
    const char* final_verdict;
    const char* static_evidence_level;
    const char* runtime_evidence_level;
    const char* strength;
} AssessmentSummary;

static AssessmentSummary assessment_summary(const HeuristicReport* hr,
                                            const DynamicReport* dr) {
    AssessmentSummary summary;
    summary.correlated = has_static_dynamic_correlation(hr, dr);
    summary.dynamic_signals = dynamic_signal_count(dr);
    summary.final_confidence = hr ? hr->confidence : 0.10;
    summary.final_verdict = "no_strong_correlated_evidence";
    summary.static_evidence_level = hr ? hr->evidence_level : "none";
    summary.runtime_evidence_level = runtime_evidence_level(dr);
    summary.strength = case_strength(hr, dr, summary.correlated);

    if (dynamic_session_complete(dr) && summary.dynamic_signals) {
        summary.final_confidence += 0.10;
    }
    if (summary.correlated) summary.final_confidence += 0.10;
    if (summary.final_confidence > 0.98) summary.final_confidence = 0.98;

    if (hr) {
        if (hr->score >= 70) {
            summary.final_verdict = summary.correlated
                ? "high_suspicion_correlated"
                : "high_suspicion_static";
        } else if (hr->score >= 40) {
            summary.final_verdict = summary.correlated
                ? "moderate_suspicion_correlated"
                : "moderate_suspicion_static";
        } else if (hr->score >= 15) {
            summary.final_verdict = summary.correlated
                ? "low_suspicion_correlated"
                : "low_suspicion_static";
        } else if (dynamic_session_complete(dr) && summary.dynamic_signals) {
            summary.final_verdict = "runtime_signals_only";
        }
    } else if (dynamic_session_complete(dr) && summary.dynamic_signals) {
        summary.final_verdict = "runtime_signals_only";
    }
    return summary;
}

static bool heuristics_has_finding_code(const HeuristicReport* hr, const char* code) {
    if (!hr || !code) return false;
    for (size_t i = 0; i < hr->finding_count; i++) {
        if (strcmp(hr->findings[i].code, code) == 0) return true;
    }
    return false;
}

static bool heuristics_has_family(const HeuristicReport* hr, const char* family) {
    if (!hr || !family) return false;
    for (size_t i = 0; i < hr->finding_count; i++) {
        if (strcmp(hr->findings[i].family, family) == 0) return true;
    }
```

```
    return false;
}

typedef struct ManualValidationSteps {
    const char* items[16];
    size_t count;
} ManualValidationSteps;

static ManualValidationSteps collect_manual_validation_steps(const HeuristicReport* hr,
                                                              const DynamicReport* dr) {
    ManualValidationSteps steps;
    memset(&steps, 0, sizeof(steps));
#define ADD_STEP(txt) do { \
    if (steps.count < sizeof(steps.items) / sizeof(steps.items[0])) { \
        steps.items[steps.count++] = (txt); \
    } \
} while (0)
    if (hr) {
        if (hr->rf_invalid_ranges || hr->rf_oob || hr->rf_unwind_oob || hr->rf_overlaps) {
            ADD_STEP("Validate .pdata range integrity in a PE viewer, sort the RUNTIME_FUNCTION entries manually and confirm that begin/end/unwind RVAs stay inside SizeOfImage.");
        }
        if (hr->unwind_parse_fail || hr->unwind_anomalies || heuristics_has_family(hr, "unwind_integrity")) {
            ADD_STEP("Open the suspicious UNWIND_INFO blob in a hex view, verify Version/Flags/CountOfCodes/tail layout, then compare the decoded prolog actions with the function prolog in Ghidra.");
        }
        if (hr->unwind_unusual_section || hr->unwind_exec_section || hr->unwind_writable_section || heuristics_has_family(hr, "metadata_placement")) {
            ADD_STEP("Inspect the section hosting the unwind metadata, record its RVA/raw offset/permissions and decide whether the placement matches a benign compiler layout or looks redirected/manual.");
        }
        if (hr->handler_nonexec_section) {
            ADD_STEP("Disassemble the reported handler target, confirm execute permission and check whether the landing bytes belong to a real handler, a thunk or a non-code blob.");
        }
        if (hr->unwind_writable_section || hr->handler_writable_section) {
            ADD_STEP("Check whether writable unwind metadata or writable handler targets are justified by a JIT/runtime scenario; if not, treat the placement as suspicious and correlate it with memory protections at runtime.");
        }
        if (heuristics_has_finding_code(hr, "handler_dominance") || heuristics_has_family(hr, "handler_dominance")) {
            ADD_STEP("Open the dominant handler target in Ghidra and compare several referencing functions to decide whether this is a normal runtime helper or a suspicious central dispatcher.");
        }
        if (hr->mitigation_count > 0) {
            ADD_STEP("Review weakening rules before making a final conclusion; the current evidence set still has benign explanations that must be excluded manually.");
        }
    }
    if (dr) {
        if (dr->rtl_add_function_table_calls || dr->rtl_install_function_table_cb_calls) {
            ADD_STEP("Capture the arguments of RtlAddFunctionTable / RtlInstallFunctionTableCallback and inspect the runtime table contents together with the surrounding module/section mapping.");
        }
        if (dr->veh_registrations || dr->vch_registrations || dr->uef_set_calls) {
            ADD_STEP("List each VEH/VCH/UEF registration with its target address, then single-step from the registered routine to confirm whether it behaves like a benign runtime hook or an exception interceptor.");
        }
        if (dr->veh_hits || dr->vch_hits || dr->uef_hits) {
            ADD_STEP("Single-step from the handler hit and map module/section transitions immediately after the exception dispatch point.");
        }
        if (dr->unwind_handler_hits) {
            ADD_STEP("Verify whether the hit unwind handler corresponds to a known language/runtime handler or to a dispatcher/interceptor path referenced by many protected functions.");
        }
    }
    if ((!dr || dynamic_signal_count(dr) == 0) && hr && (hr->score > 0 || hr->finding_count > 0)) {
        ADD_STEP("Run the sample again with --dynamic on a representative exception path to obtain runtime corroboration.");
    }
#undef ADD_STEP
    return steps;
}

static void json_manual_validation_steps(StrBuf* sb, const HeuristicReport* hr, const DynamicReport* dr) {
    ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);
    sb_append(sb, "[");
    for (size_t i = 0; i < steps.count; i++) {
        if (i) sb_append(sb, ",");
        json_string(sb, steps.items[i]);
    }
    sb_append(sb, "]");
}

void report_print_heuristics(const HeuristicReport* hr) {
    if (!hr) return;
    printf("\n[Heuristic Analysis] Suspicion score: %d / 100\n", hr->score);
    printf("  suspicion=%.3f  confidence=%.3f  verdict=%s  manual_review=%s\n",
           hr->suspicion, hr->confidence,
           hr->verdict[0] ? hr->verdict : "-",
           hr->requires_manual_review ? "yes" : "no");
    printf("  total=%zu  invalid_ranges=%zu  oob=%zu  unwind_oob=%zu\n",
           hr->total, hr->rf_invalid_ranges, hr->rf_oob, hr->rf_unwind_oob);
    printf("  unsorted=%zu  overlaps=%zu  duplicate_begins=%zu  unwind_parse_fail=%zu  unwind_anom=%zu\n",
           hr->rf_unsorted, hr->rf_overlaps, hr->rf_duplicate_begins,
           hr->unwind_parse_fail, hr->unwind_anomalies);
    printf("  strict_abi_warnings=%zu\n", hr->unwind_abi_warnings);
    printf("  unwind_unusual_section=%zu  unwind_exec_section=%zu  unwind_writable_section=%zu\n",
           hr->unwind_unusual_section, hr->unwind_exec_section, hr->unwind_writable_section);
    printf("  handler_nonexec_section=%zu  handler_writable_section=%zu\n",
           hr->handler_nonexec_section, hr->handler_writable_section);
    printf("  unwind_reuse_ratio=%.3f  unwind_unique_ratio=%.3f\n",
           hr->unwind_reuse_ratio, hr->unwind_unique_ratio);
    if (hr->handler_count) {
        printf("  handler_count=%zu  handler_reuse_ratio=%.3f\n",
               hr->handler_count, hr->handler_reuse_ratio);
    }
    printf("  evidence_families=%zu  findings=%zu\n",
           heuristics_evidence_family_count(hr->evidence_families_mask), hr->finding_count);
    printf("Findings:\n%s\n", hr->details.s ? hr->details.s : "");
}

static bool report_metadata_valid(const ReportMetadata* metadata) {
    if (!metadata ||
        !metadata->tool_version || metadata->tool_version[0] == '\0' ||
        !metadata->report_schema_version ||
```

```
        strcmp(metadata->report_schema_version, EHUA_REPORT_SCHEMA_VERSION) != 0 ||
        !metadata->generated_at_utc || metadata->generated_at_utc[0] == '\0' ||
        !metadata->input_sha256 || strlen(metadata->input_sha256) != 64u ||
        !metadata->command_line ||
        !metadata->build_compiler || metadata->build_compiler[0] == '\0' ||
        !metadata->build_platform || metadata->build_platform[0] == '\0' ||
        !metadata->build_mode ||
        (strcmp(metadata->build_mode, "debug") != 0 &&
         strcmp(metadata->build_mode, "release") != 0) ||
        metadata->max_chain_depth < 1u ||
        metadata->max_chain_depth > 16u) {
        return false;
    }
    for (size_t i = 0; i < 64u; i++) {
        const char c = metadata->input_sha256[i];
        if (!((c >= '0' && c <= '9') || (c >= 'a' && c <= 'f'))) return false;
    }
    if (strlen(metadata->generated_at_utc) != 20u ||
        metadata->generated_at_utc[4] != '-' ||
        metadata->generated_at_utc[7] != '-' ||
        metadata->generated_at_utc[10] != 'T' ||
        metadata->generated_at_utc[13] != ':' ||
        metadata->generated_at_utc[16] != ':' ||
        metadata->generated_at_utc[19] != 'Z') {
        return false;
    }
    for (size_t i = 0; i < 19u; i++) {
        if (i == 4u || i == 7u || i == 10u || i == 13u || i == 16u) continue;
        if (metadata->generated_at_utc[i] < '0' ||
            metadata->generated_at_utc[i] > '9') {
            return false;
        }
    }
    return true;
}

static bool report_model_valid(const PeImage* img,
                               const HeuristicReport* hr,
                               const DynamicReport* dr) {
    if (!img || (img->num_sections && !img->sections)) return false;
    if (hr && (hr->finding_count > HEURISTICS_MAX_FINDINGS ||
               hr->mitigation_count > HEURISTICS_MAX_MITIGATIONS)) {
        return false;
    }
    if (dr && (dr->event_count > DYNAMIC_MAX_EVENTS ||
               dr->trace_count > sizeof(dr->trace) / sizeof(dr->trace[0]) ||
               dr->trace_steps > sizeof(dr->trace) / sizeof(dr->trace[0]))) {
        return false;
    }
    if (dr && dr->instrumentation_complete &&
        (dr->module_refresh_failures ||
         dr->module_list_truncations ||
         dr->breakpoint_set_failures ||
         dr->breakpoint_capacity_drops ||
         dr->pending_rearm_failures ||
         dr->breakpoint_rearm_failures ||
         dr->dropped_events)) {
        return false;
    }
    return true;
}

bool report_write_json_ex(const char* path,
                          const char* input_path,
                          const PeImage* img,
                          const RuntimeFunction* rf,
                          const UnwindSummary* us,
                          const bool* us_ok,
                          size_t n,
                          const HeuristicReport* hr,
                          const DynamicReport* dr,
                          const ReportMetadata* metadata,
                          StrBuf* err) {
    if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
        !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
        sb_append(err, "Invalid arguments or metadata for JSON report.\n");
        return false;
    }

    StrBuf sb;
    if (!sb_init(&sb, 8192)) { sb_append(err, "OOM.\n"); return false; }
    StrBuf comment;
    if (!sb_init(&comment, 512)) { sb_free(&sb); sb_append(err, "OOM.\n"); return false; }
    StrBuf anom;
    if (!sb_init(&anom, 128)) { sb_free(&comment); sb_free(&sb); sb_append(err, "OOM.\n"); return false; }

    sb_append(&sb, "{");
    json_string_field(&sb, "input", input_path, true);
    json_key(&sb, "report_metadata");
    sb_append(&sb, "{");
    json_string_field(&sb, "schema_version", metadata->report_schema_version, true);
    json_string_field(&sb, "tool_version", metadata->tool_version, true);
    json_string_field(&sb, "generated_at_utc", metadata->generated_at_utc, true);
    json_string_field(&sb, "input_sha256", metadata->input_sha256, true);
    json_key(&sb, "input_size"); sb_appendf(&sb, "%llu,", (unsigned long long)metadata->input_size);
    json_string_field(&sb, "command_line", metadata->command_line, true);
    json_string_field(&sb, "build_compiler", metadata->build_compiler, true);
    json_string_field(&sb, "build_platform", metadata->build_platform, true);
    json_string_field(&sb, "build_mode", metadata->build_mode, true);
    json_bool_field(&sb, "strict_abi", metadata->strict_abi, true);
    json_bool_field(&sb, "follow_chain", metadata->follow_chain, true);
    json_key(&sb, "max_chain_depth");
    sb_appendf(&sb, "%u", (unsigned)metadata->max_chain_depth);
    sb_append(&sb, "},");
    json_key(&sb, "image_base"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)img->image_base);
    json_key(&sb, "size_of_image"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_image);
    json_key(&sb, "size_of_headers"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->size_of_headers);
    json_key(&sb, "section_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->section_alignment);
    json_key(&sb, "file_alignment"); sb_appendf(&sb, "\"0x%x\",", (unsigned)img->file_alignment);
    json_key(&sb, "exception_directory");
    sb_appendf(&sb, "{\"rva\":\"0x%08x\",\"size\":\"0x%x\"}",
               (unsigned)img->exception_rva, (unsigned)img->exception_size);
    sb_append(&sb, ",");
    json_key(&sb, "pe_validation");
    sb_append(&sb, "{");
```

```
675 |     json_key(&sb, "section_raw_oob"); sb_appendf(&sb, "%zu,", img->section_raw_oob);
676 |     json_key(&sb, "section_virtual_oob"); sb_appendf(&sb, "%zu,", img->section_virtual_oob);
677 |     json_key(&sb, "section_raw_overlaps"); sb_appendf(&sb, "%zu,", img->section_raw_overlaps);
678 |     json_key(&sb, "section_virtual_overlaps"); sb_appendf(&sb, "%zu,", img->section_virtual_overlaps);
679 |     json_key(&sb, "section_misaligned"); sb_appendf(&sb, "%zu,", img->section_misaligned);
680 |     json_bool_field(&sb, "exception_directory_oob", img->exception_directory_oob, false);
681 |     sb_append(&sb, "},");
682 |     json_key(&sb, "sections");
683 |     sb_append(&sb, "[");
684 |     for (size_t i = 0; i < img->num_sections; i++) {
685 |         const PeSection* section = &img->sections[i];
686 |         if (i) sb_append(&sb, ",");
687 |         sb_append(&sb, "{");
688 |         json_string_field(&sb, "name", section->name, true);
689 |         json_key(&sb, "virtual_address"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->virtual_address);
690 |         json_key(&sb, "virtual_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->virtual_size);
691 |         json_key(&sb, "raw_offset"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_ptr);
692 |         json_key(&sb, "raw_size"); sb_appendf(&sb, "\"0x%x\",", (unsigned)section->raw_size);
693 |         json_key(&sb, "characteristics"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)section->characteristics);
694 |         json_bool_field(&sb, "readable", (section->characteristics & IMAGE_SCN_MEM_READ) != 0, true);
695 |         json_bool_field(&sb, "writable", (section->characteristics & IMAGE_SCN_MEM_WRITE) != 0, true);
696 |         json_bool_field(&sb, "executable", (section->characteristics & IMAGE_SCN_MEM_EXECUTE) != 0, false);
697 |         sb_append(&sb, "}");
698 |     }
699 |     sb_append(&sb, "],");
700 |
701 |     if (hr) {
702 |         json_key(&sb, "heuristic_analysis");
703 |         sb_append(&sb, "{");
704 |         json_key(&sb, "score"); sb_appendf(&sb, "%d,", hr->score);
705 |         json_key(&sb, "suspicion"); sb_appendf(&sb, "%.6f,", hr->suspicion);
706 |         json_key(&sb, "confidence"); sb_appendf(&sb, "%.6f,", hr->confidence);
707 |         json_bool_field(&sb, "requires_manual_review", hr->requires_manual_review, true);
708 |         json_string_field(&sb, "verdict", hr->verdict, true);
709 |         json_string_field(&sb, "evidence_level", hr->evidence_level, true);
710 |         json_key(&sb, "weak_finding_count"); sb_appendf(&sb, "%zu,", hr->weak_finding_count);
711 |         json_key(&sb, "medium_finding_count"); sb_appendf(&sb, "%zu,", hr->medium_finding_count);
712 |         json_key(&sb, "strong_finding_count"); sb_appendf(&sb, "%zu,", hr->strong_finding_count);
713 |         json_key(&sb, "total"); sb_appendf(&sb, "%zu,", hr->total);
714 |         json_key(&sb, "pe_section_raw_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_oob);
715 |         json_key(&sb, "pe_section_virtual_oob"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_oob);
716 |         json_key(&sb, "pe_section_raw_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_raw_overlaps);
717 |         json_key(&sb, "pe_section_virtual_overlaps"); sb_appendf(&sb, "%zu,", hr->pe_section_virtual_overlaps);
718 |         json_key(&sb, "pe_section_misaligned"); sb_appendf(&sb, "%zu,", hr->pe_section_misaligned);
719 |         json_bool_field(&sb, "pe_exception_directory_oob", hr->pe_exception_directory_oob, true);
720 |         json_key(&sb, "rf_invalid_ranges"); sb_appendf(&sb, "%zu,", hr->rf_invalid_ranges);
721 |         json_key(&sb, "rf_oob"); sb_appendf(&sb, "%zu,", hr->rf_oob);
722 |         json_key(&sb, "rf_unwind_oob"); sb_appendf(&sb, "%zu,", hr->rf_unwind_oob);
723 |         json_key(&sb, "rf_unsorted"); sb_appendf(&sb, "%zu,", hr->rf_unsorted);
724 |         json_key(&sb, "rf_overlaps"); sb_appendf(&sb, "%zu,", hr->rf_overlaps);
725 |         json_key(&sb, "rf_duplicate_begins"); sb_appendf(&sb, "%zu,", hr->rf_duplicate_begins);
726 |         json_key(&sb, "unwind_parse_fail"); sb_appendf(&sb, "%zu,", hr->unwind_parse_fail);
727 |         json_key(&sb, "unwind_anomalies"); sb_appendf(&sb, "%zu,", hr->unwind_anomalies);
728 |         json_key(&sb, "unwind_abi_warnings"); sb_appendf(&sb, "%zu,", hr->unwind_abi_warnings);
729 |         json_key(&sb, "unwind_unusual_section"); sb_appendf(&sb, "%zu,", hr->unwind_unusual_section);
730 |         json_key(&sb, "unwind_exec_section"); sb_appendf(&sb, "%zu,", hr->unwind_exec_section);
731 |         json_key(&sb, "unwind_writable_section"); sb_appendf(&sb, "%zu,", hr->unwind_writable_section);
732 |         json_key(&sb, "handler_nonexec_section"); sb_appendf(&sb, "%zu,", hr->handler_nonexec_section);
733 |         json_key(&sb, "handler_writable_section"); sb_appendf(&sb, "%zu,", hr->handler_writable_section);
734 |         json_key(&sb, "unwind_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_reuse_ratio);
735 |         json_key(&sb, "unwind_unique_ratio"); sb_appendf(&sb, "%.6f,", hr->unwind_unique_ratio);
736 |         json_key(&sb, "handler_count"); sb_appendf(&sb, "%zu,", hr->handler_count);
737 |         json_key(&sb, "handler_reuse_ratio"); sb_appendf(&sb, "%.6f,", hr->handler_reuse_ratio);
738 |         json_key(&sb, "evidence_family_count"); sb_appendf(&sb, "%zu,",
    heuristics_evidence_family_count(hr->evidence_families_mask));
739 |         json_key(&sb, "evidence_families"); json_heuristic_families(&sb, hr->evidence_families_mask); sb_append(&sb, ",");
740 |         json_key(&sb, "findings");
741 |         sb_append(&sb, "[");
742 |         for (size_t i = 0; i < hr->finding_count; i++) {
743 |             if (i) sb_append(&sb, ",");
744 |             sb_append(&sb, "{");
745 |             json_string_field(&sb, "code", hr->findings[i].code, true);
746 |             json_string_field(&sb, "family", hr->findings[i].family, true);
747 |             json_string_field(&sb, "severity", hr->findings[i].severity, true);
748 |             json_string_field(&sb, "message", hr->findings[i].message, true);
749 |             json_key(&sb, "affected_count"); sb_appendf(&sb, "%zu,", hr->findings[i].affected_count);
750 |             json_key(&sb, "representative_rva");
751 |             if (hr->findings[i].has_location) {
752 |                 sb_appendf(&sb, "\"0x%08x\",", (unsigned)hr->findings[i].representative_rva);
753 |             } else {
754 |                 sb_append(&sb, "null,");
755 |             }
756 |             json_string_field(
757 |                 &sb,
758 |                 "representative_section",
759 |                 hr->findings[i].has_location
760 |                     ? section_name_or_dash(img, hr->findings[i].representative_rva)
761 |                     : "-",
762 |                 false);
763 |             sb_append(&sb, "}");
764 |         }
765 |         sb_append(&sb, "],");
766 |         json_key(&sb, "weakening_rules_applied");
767 |         sb_append(&sb, "[");
768 |         for (size_t i = 0; i < hr->mitigation_count; i++) {
769 |             if (i) sb_append(&sb, ",");
770 |             sb_append(&sb, "{");
771 |             json_string_field(&sb, "code", hr->mitigations[i].code, true);
772 |             json_string_field(&sb, "message", hr->mitigations[i].message, false);
773 |             sb_append(&sb, "}");
774 |         }
775 |         sb_append(&sb, "],");
776 |         json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr->mitigation_count);
777 |         json_string_field(&sb, "details", hr->details.s ? hr->details.s : "", false);
778 |         sb_append(&sb, "},");
779 |     }
780 |
781 |     if (dr) {
782 |         json_key(&sb, "dynamic_validation");
783 |         sb_append(&sb, "{");
784 |         json_bool_field(&sb, "enabled", dr->enabled, true);
785 |         json_bool_field(&sb, "ran", dr->ran, true);
786 |         json_bool_field(&sb, "ok", dr->ok, true);
787 |         json_bool_field(&sb, "timed_out", dr->timed_out, true);
788 |         json_bool_field(&sb, "target_exited", dr->target_exited, true);
```

```
789 |         json_key(&sb, "target_exit_code"); sb_appendf(&sb, "%u,", (unsigned)dr->target_exit_code);
790 |         json_key(&sb, "timeout_ms"); sb_appendf(&sb, "%u,", (unsigned)dr->timeout_ms);
791 |         json_key(&sb, "trace_steps"); sb_appendf(&sb, "%u,", (unsigned)dr->trace_steps);
792 |         json_bool_field(&sb, "instrumentation_complete", dr->instrumentation_complete, true);
793 |         json_key(&sb, "modules_observed_peak"); sb_appendf(&sb, "%u,", (unsigned)dr->modules_observed_peak);
794 |         json_key(&sb, "module_refresh_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->module_refresh_failures);
795 |         json_key(&sb, "module_list_truncations"); sb_appendf(&sb, "%u,", (unsigned)dr->module_list_truncations);
796 |         json_key(&sb, "unwind_handlers_discovered"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handlers_discovered);
797 |         json_key(&sb, "breakpoint_set_attempts"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_attempts);
798 |         json_key(&sb, "breakpoint_set_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_set_failures);
799 |         json_key(&sb, "breakpoint_capacity_drops"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_capacity_drops);
800 |         json_key(&sb, "pending_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->pending_rearm_failures);
801 |         json_key(&sb, "breakpoint_rearm_failures"); sb_appendf(&sb, "%u,", (unsigned)dr->breakpoint_rearm_failures);
802 |         json_key(&sb, "exceptions_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->exceptions_seen);
803 |         json_key(&sb, "first_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->first_chance_seen);
804 |         json_key(&sb, "second_chance_seen"); sb_appendf(&sb, "%u,", (unsigned)dr->second_chance_seen);
805 |         json_key(&sb, "unwind_handler_bp_set"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_bp_set);
806 |         json_key(&sb, "unwind_handler_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->unwind_handler_hits);
807 |         json_key(&sb, "veh_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_registrations);
808 |         json_key(&sb, "veh_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->veh_hits);
809 |         json_key(&sb, "vch_registrations"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_registrations);
810 |         json_key(&sb, "vch_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->vch_hits);
811 |         json_key(&sb, "uef_set_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_set_calls);
812 |         json_key(&sb, "uef_hits"); sb_appendf(&sb, "%u,", (unsigned)dr->uef_hits);
813 |         json_key(&sb, "rtl_add_function_table_calls"); sb_appendf(&sb, "%u,", (unsigned)dr->rtl_add_function_table_calls);
814 |         json_key(&sb, "rtl_install_function_table_cb_calls"); sb_appendf(&sb, "%u,",
    (unsigned)dr->rtl_install_function_table_cb_calls);
815 |         json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", dynamic_signal_count(dr));
816 |         json_key(&sb, "dropped_events"); sb_appendf(&sb, "%u,", (unsigned)dr->dropped_events);
817 |         json_string_field(&sb, "details", dr->details.s ? dr->details.s : "", true);
818 |         json_key(&sb, "events");
819 |         sb_append(&sb, "[");
820 |         for (uint32_t i = 0; i < dr->event_count; i++) {
821 |             const DynamicEvent* event = &dr->events[i];
822 |             if (i) sb_append(&sb, ",");
823 |             sb_append(&sb, "{");
824 |             json_string_field(&sb, "type", event->type, true);
825 |             json_string_field(&sb, "module", event->module, true);
826 |             json_key(&sb, "thread_id"); sb_appendf(&sb, "%u,", (unsigned)event->thread_id);
827 |             json_key(&sb, "exception_code"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)event->exception_code);
828 |             json_bool_field(&sb, "first_chance", event->first_chance, true);
829 |             json_key(&sb, "address"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->address);
830 |             json_key(&sb, "arg0"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg0);
831 |             json_key(&sb, "arg1"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg1);
832 |             json_key(&sb, "arg2"); sb_appendf(&sb, "\"0x%llx\",", (unsigned long long)event->arg2);
833 |             json_key(&sb, "arg3"); sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)event->arg3);
834 |             sb_append(&sb, "}");
835 |         }
836 |         sb_append(&sb, "],");
837 |         json_key(&sb, "trace");
838 |         sb_append(&sb, "[");
839 |         for (uint32_t i = 0; i < dr->trace_count; i++) {
840 |             if (i) sb_append(&sb, ",");
841 |             sb_appendf(&sb, "\"0x%llx\"", (unsigned long long)dr->trace[i]);
842 |         }
843 |         sb_append(&sb, "]");
844 |         sb_append(&sb, "},");
845 |     }
846 |
847 |     if (hr || dr) {
848 |         AssessmentSummary summary = assessment_summary(hr, dr);
849 |
850 |         json_key(&sb, "assessment");
851 |         sb_append(&sb, "{");
852 |         json_string_field(&sb, "final_verdict", summary.final_verdict, true);
853 |         json_key(&sb, "final_confidence"); sb_appendf(&sb, "%.6f,", summary.final_confidence);
854 |         json_string_field(&sb, "static_evidence_level", summary.static_evidence_level, true);
855 |         json_string_field(&sb, "runtime_evidence_level", summary.runtime_evidence_level, true);
856 |         json_string_field(&sb, "case_strength", summary.strength, true);
857 |         json_bool_field(&sb, "static_dynamic_correlation", summary.correlated, true);
858 |         json_key(&sb, "dynamic_signal_count"); sb_appendf(&sb, "%u,", summary.dynamic_signals);
859 |         json_key(&sb, "weakening_rule_count"); sb_appendf(&sb, "%zu,", hr ? hr->mitigation_count : 0u);
860 |         json_key(&sb, "manual_validation_steps"); json_manual_validation_steps(&sb, hr, dr);
861 |         sb_append(&sb, "},");
862 |     }
863 |
864 |     json_key(&sb, "functions");
865 |     sb_append(&sb, "[");
866 |     for (size_t i = 0; i < n; i++) {
867 |         if (i) sb_append(&sb, ",");
868 |         if (us_ok[i]) build_unwind_comment(&us[i], &comment);
869 |         else {
870 |             comment.len = 0;
871 |             if (comment.s) comment.s[0] = 0;
872 |             sb_append(&comment, "UNWIND parse failed");
873 |             if (us[i].parse_error[0]) sb_appendf(&comment, ": %s", us[i].parse_error);
874 |         }
875 |         anom.len = 0; if (anom.s) anom.s[0] = 0; anomalies_to_str(&anom, us[i].anomalies);
876 |
877 |         sb_append(&sb, "{");
878 |         json_key(&sb, "index"); sb_appendf(&sb, "%zu,", i);
879 |         json_key(&sb, "begin_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].begin_rva);
880 |         json_key(&sb, "end_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].end_rva);
881 |         json_key(&sb, "size"); sb_appendf(&sb, "%u,", (unsigned)runtime_function_size(&rf[i]));
882 |         json_key(&sb, "begin_section"); json_string(&sb, section_name_or_dash(img, rf[i].begin_rva)); sb_append(&sb, ",");
883 |         json_key(&sb, "unwind_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)rf[i].unwind_rva);
884 |         json_key(&sb, "unwind_section"); json_string(&sb, section_name_or_dash(img, rf[i].unwind_rva)); sb_append(&sb, ",");
885 |         json_bool_field(&sb, "unwind_ok", us_ok[i], true);
886 |         json_string_field(&sb, "unwind_error", us[i].parse_error, true);
887 |         json_key(&sb, "unwind");
888 |         sb_append(&sb, "{");
889 |         json_key(&sb, "version"); sb_appendf(&sb, "%u,", (unsigned)us[i].version);
890 |         json_key(&sb, "flags"); sb_appendf(&sb, "%u,", (unsigned)us[i].flags);
891 |         json_key(&sb, "prolog_size"); sb_appendf(&sb, "%u,", (unsigned)us[i].prolog_size);
892 |         json_key(&sb, "count_codes"); sb_appendf(&sb, "%u,", (unsigned)us[i].count_codes);
893 |         json_key(&sb, "fixed_stack_bytes"); sb_appendf(&sb, "%u,", (unsigned)us[i].fixed_stack_bytes);
894 |         json_key(&sb, "frame_reg"); json_string(&sb, (us[i].frame_reg ? unwind_reg_name(us[i].frame_reg) : "-")); sb_append(&sb,
    ",");
895 |         json_key(&sb, "frame_off"); sb_appendf(&sb, "%u,", (unsigned)(us[i].frame_off * 16u));
896 |         json_bool_field(&sb, "has_handler", us[i].has_handler, true);
897 |         json_key(&sb, "handler_rva"); sb_appendf(&sb, "\"0x%08x\",", (unsigned)us[i].handler_rva);
898 |         json_key(&sb, "handler_section"); json_string(&sb, us[i].has_handler ? section_name_or_dash(img, us[i].handler_rva) : "-");
    sb_append(&sb, ",");
899 |         json_bool_field(&sb, "has_chain", us[i].has_chain, true);
900 |         json_key(&sb, "anomalies"); sb_appendf(&sb, "%u,", (unsigned)us[i].anomalies);
```

```
        json_string_field(&sb, "anomalies_text", anom.s ? anom.s : "0", true);
        json_key(&sb, "chain");
        if (us[i].has_chain) {
            sb_appendf(&sb, "{\"begin_rva\":\"0x%08x\",\"end_rva\":\"0x%08x\",\"unwind_rva\":\"0x%08x\"}",
                       (unsigned)us[i].chain.begin_rva,
                       (unsigned)us[i].chain.end_rva,
                       (unsigned)us[i].chain.unwind_rva);
        } else {
            sb_append(&sb, "null");
        }
        sb_append(&sb, "},");
        json_string_field(&sb, "comment", comment.s ? comment.s : "", false);
        sb_append(&sb, "}");
    }
    sb_append(&sb, "]}");

    bool ok = sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom);
    if (ok) ok = file_write_atomic(path, sb.s, sb.len);
    if (!ok) {
        sb_appendf(err, "%s: %s\n",
                   sb_ok(&sb) && sb_ok(&comment) && sb_ok(&anom)
                       ? "Failed to write JSON"
                       : "Out of memory while building JSON report",
                   path);
    }

    sb_free(&anom);
    sb_free(&comment);
    sb_free(&sb);
    return ok;
}

static void html_escape(StrBuf* sb, const char* text) {
    if (!text) return;
    size_t length = strlen(text);
    for (size_t i = 0; i < length; i++) {
        unsigned char c = (unsigned char)text[i];
        switch (c) {
            case '&': sb_append(sb, "&amp;"); break;
            case '<': sb_append(sb, "&lt;"); break;
            case '>': sb_append(sb, "&gt;"); break;
            case '"': sb_append(sb, "&quot;"); break;
            case '\'': sb_append(sb, "&#39;"); break;
            default:
                if (c < 0x80) {
                    sb_appendn(sb, &text[i], 1);
                } else {
                    size_t sequence = valid_utf8_sequence_length(
                        (const unsigned char*)&text[i], length - i);
                    if (sequence) {
                        sb_appendn(sb, &text[i], sequence);
                        i += sequence - 1;
                    } else {
                        sb_appendf(sb, "&#x%02x;", (unsigned)c);
                    }
                }
                break;
        }
    }
}

static void html_text_cell(StrBuf* sb, const char* text) {
    sb_append(sb, "<td>");
    html_escape(sb, text ? text : "");
    sb_append(sb, "</td>");
}

static void html_hex32_cell(StrBuf* sb, uint32_t value) {
    sb_appendf(sb, "<td><code>0x%08x</code></td>", (unsigned)value);
}

static void html_hex64_cell(StrBuf* sb, uint64_t value) {
    sb_appendf(sb, "<td><code>0x%llx</code></td>", (unsigned long long)value);
}

static const char* html_score_class(const HeuristicReport* hr) {
    if (!hr) return "neutral";
    if (hr->score >= 70) return "danger";
    if (hr->score >= 40) return "warn";
    return "ok";
}

static const char* ru_yes_no(bool value) {
    return value ? "да" : "нет";
}

static const char* ru_status_text(const DynamicReport* dr) {
    if (!dr) return "не запрашивалась";
    if (dr->timed_out) return "тайм-аут";
    if (!dr->ran) return "не запускалась";
    return dr->ok ? "завершена" : "ошибка";
}

static const char* ru_level_text(const char* value) {
    if (!value) return "-";
    if (strcmp(value, "strong") == 0) return "сильный";
    if (strcmp(value, "medium") == 0) return "средний";
    if (strcmp(value, "weak") == 0) return "слабый";
    if (strcmp(value, "none") == 0) return "нет";
    return value;
}

static const char* ru_severity_text(const char* value) {
    if (!value) return "-";
    if (strcmp(value, "critical") == 0) return "критическая";
    if (strcmp(value, "high") == 0) return "высокая";
    if (strcmp(value, "medium") == 0) return "средняя";
    if (strcmp(value, "low") == 0) return "низкая";
    if (strcmp(value, "info") == 0) return "информационная";
    return value;
}

static const char* ru_verdict_text(const char* value) {
    if (!value) return "-";
    if (strcmp(value, "high_suspicion") == 0) return "высокая подозрительность";
```

```
    if (strcmp(value, "moderate_suspicion") == 0) return "умеренная подозрительность";
    if (strcmp(value, "low_suspicion") == 0) return "низкая подозрительность";
    if (strcmp(value, "no_strong_static_anomalies") == 0) return "сильные статические аномалии не обнаружены";
    if (strcmp(value, "high_suspicion_correlated") == 0) return "высокая подозрительность с динамическим подтверждением";
    if (strcmp(value, "high_suspicion_static") == 0) return "высокая статическая подозрительность";
    if (strcmp(value, "moderate_suspicion_correlated") == 0) return "умеренная подозрительность с динамическим подтверждением";
    if (strcmp(value, "moderate_suspicion_static") == 0) return "умеренная статическая подозрительность";
    if (strcmp(value, "low_suspicion_correlated") == 0) return "низкая подозрительность с динамическим подтверждением";
    if (strcmp(value, "low_suspicion_static") == 0) return "низкая статическая подозрительность";
    if (strcmp(value, "runtime_signals_only") == 0) return "обнаружены только динамические сигналы";
    if (strcmp(value, "no_strong_correlated_evidence") == 0) return "сильная корреляция не обнаружена";
    return value;
}

static const char* ru_case_strength_text(const char* value) {
    if (!value) return "-";
    if (strcmp(value, "strong_correlated_case") == 0) return "сильная, с корреляцией";
    if (strcmp(value, "medium_correlated_case") == 0) return "средняя, с корреляцией";
    if (strcmp(value, "strong_unconfirmed_case") == 0) return "сильная, без динамического подтверждения";
    if (strcmp(value, "medium_case") == 0) return "средняя";
    if (strcmp(value, "weak_case") == 0) return "слабая";
    if (strcmp(value, "no_case") == 0) return "значимых признаков нет";
    return value;
}

static void html_append_manual_step_ru(StrBuf* sb, const char* step) {
    if (!step) return;
    if (strstr(step, "Validate .pdata range integrity")) {
        sb_append(sb, "Проверьте целостность диапазона .pdata в PE-viewer: вручную отсортируйте записи RUNTIME_FUNCTION и убедитесь, что Begin/End/Unwind RVA находятся внутри SizeOfImage.");
    } else if (strstr(step, "Open the suspicious UNWIND_INFO blob")) {
        sb_append(sb, "Откройте подозрительный блок UNWIND_INFO в hex-viewer, проверьте Version/Flags/CountOfCodes и tail-layout, затем сравните decoded prolog actions с prolog функции в Ghidra.");
    } else if (strstr(step, "Inspect the section hosting the unwind metadata")) {
        sb_append(sb, "Проверьте секцию, в которой размещены unwind-метаданные: зафиксируйте RVA, raw offset и права доступа, затем решите, похоже ли размещение на обычный compiler layout или на ручное перенаправление.");
    } else if (strstr(step, "Disassemble the reported handler target")) {
        sb_append(sb, "Дизассемблируйте указанный handler target, подтвердите execute permission и проверьте, являются ли landing bytes настоящим handler, thunk или областью не-кода.");
    } else if (strstr(step, "Check whether writable unwind metadata")) {
        sb_append(sb, "Проверьте, оправдано ли наличие writable unwind metadata или writable handler targets сценарием JIT/runtime; если нет, считайте размещение подозрительным и сопоставьте его с memory protections во время запуска.");
    } else if (strstr(step, "Open the dominant handler target")) {
        sb_append(sb, "Откройте dominant handler target в Ghidra и сравните несколько функций, которые на него ссылаются, чтобы отличить обычный runtime helper от подозрительного central dispatcher.");
    } else if (strstr(step, "Review weakening rules")) {
        sb_append(sb, "Перед окончательным выводом проверьте правила снижения уверенности: текущий набор признаков еще может иметь доброкачественные объяснения, которые нужно исключить вручную.");
    } else if (strstr(step, "Capture the arguments of RtlAddFunctionTable")) {
        sb_append(sb, "Зафиксируйте аргументы RtlAddFunctionTable / RtlInstallFunctionTableCallback и проверьте содержимое runtime table вместе с module/section mapping.");
    } else if (strstr(step, "List each VEH/VCH/UEF registration")) {
        sb_append(sb, "Перечислите каждую регистрацию VEH/VCH/UEF с target address, затем выполните single-step от зарегистрированной routine, чтобы отличить benign runtime hook от exception interceptor.");
    } else if (strstr(step, "Single-step from the handler hit")) {
        sb_append(sb, "Выполните single-step от handler hit и отобразите переходы между module/section сразу после exception dispatch point.");
    } else if (strstr(step, "Verify whether the hit unwind handler")) {
        sb_append(sb, "Проверьте, соответствует ли достигнутый unwind handler известному language/runtime handler или dispatcher/interceptor path, на который ссылается много защищенных функций.");
    } else if (strstr(step, "Run the sample again with --dynamic")) {
        sb_append(sb, "Запустите sample повторно с --dynamic на representative exception path, чтобы получить runtime corroboration.");
    } else {
        html_escape(sb, step);
    }
}

static void html_append_finding_message_ru(StrBuf* sb, const HeuristicFinding* finding) {
    if (!finding || !finding->code[0]) return;
    const char* code = finding->code;
    size_t n = finding->affected_count;
    if (strcmp(code, "pe_section_raw_oob") == 0) {
        sb_append(sb, "Raw-диапазон секции PE выходит за границы файла.");
    } else if (strcmp(code, "pe_section_virtual_oob") == 0) {
        sb_append(sb, "Virtual-диапазон секции PE выходит за пределы SizeOfImage.");
    } else if (strcmp(code, "pe_section_raw_overlap") == 0) {
        sb_append(sb, "Raw-диапазоны секций PE пересекаются.");
    } else if (strcmp(code, "pe_section_virtual_overlap") == 0) {
        sb_append(sb, "Virtual-диапазоны секций PE пересекаются.");
    } else if (strcmp(code, "pe_section_misaligned") == 0) {
        sb_append(sb, "Обнаружена секция PE с нарушением выравнивания.");
    } else if (strcmp(code, "pe_exception_directory_oob") == 0) {
        sb_append(sb, "Exception directory поврежден, неполон, недоступен для чтения или выходит за допустимые границы.");
    } else if (strcmp(code, "no_runtime_functions") == 0) {
        sb_append(sb, "Записи RUNTIME_FUNCTION не найдены; это может быть допустимо для leaf-only image, поэтому сложность кода нужно подтвердить вручную.");
    } else if (strcmp(code, "invalid_ranges") == 0) {
        sb_appendf(sb, "Обнаружены записи RUNTIME_FUNCTION с некорректным диапазоном Begin/End: %zu.", n);
    } else if (strcmp(code, "rf_oob") == 0) {
        sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION выходят за пределы SizeOfImage: %zu.", n);
    } else if (strcmp(code, "unwind_rva_oob") == 0) {
        sb_appendf(sb, "UnwindInfoRVA отсутствует или выходит за допустимые границы: %zu.", n);
    } else if (strcmp(code, "rf_unsorted") == 0) {
        sb_appendf(sb, "Таблица RUNTIME_FUNCTION не отсортирована по BeginAddress: %zu.", n);
    } else if (strcmp(code, "rf_overlaps") == 0) {
        sb_appendf(sb, "Диапазоны RUNTIME_FUNCTION пересекаются: %zu.", n);
    } else if (strcmp(code, "duplicate_begin") == 0) {
        sb_appendf(sb, "Несколько записей RUNTIME_FUNCTION используют один и тот же BeginAddress: %zu.", n);
    } else if (strcmp(code, "unwind_parse_fail") == 0) {
        sb_appendf(sb, "Ошибки разбора UNWIND_INFO: %zu.", n);
    } else if (strcmp(code, "unwind_anomalies") == 0) {
        sb_appendf(sb, "Аномалии UNWIND_INFO (version/opcode/chain/ABI и др.): %zu.", n);
    } else if (strcmp(code, "unwind_reuse_strong") == 0) {
        sb_appendf(sb, "Сильное повторное использование UNWIND_INFO: до %zu записей ссылаются на один UnwindInfoRVA.", n);
    } else if (strcmp(code, "handler_dominance") == 0) {
        sb_appendf(sb, "Один handler target доминирует: до %zu записей ссылаются на один HandlerRVA.", n);
    } else if (strcmp(code, "unwind_reuse_moderate") == 0) {
        sb_appendf(sb, "Умеренное повторное использование UNWIND_INFO подтверждается другим статическим признаком: до %zu записей ссылаются на один UnwindInfoRVA.", n);
    } else if (strcmp(code, "unwind_in_exec_section") == 0) {
        sb_appendf(sb, "UNWIND_INFO расположен в executable-секции: %zu.", n);
    } else if (strcmp(code, "unwind_unusual_section") == 0) {
        sb_appendf(sb, "UNWIND_INFO расположен в необычной секции, отличной от .xdata/.rdata/.pdata: %zu.", n);
    } else if (strcmp(code, "unwind_writable_section") == 0) {
```

```
        sb_appendf(sb, "UNWIND_INFO расположен в writable-секции: %zu.", n);
    } else if (strcmp(code, "handler_nonexec_section") == 0) {
        sb_appendf(sb, "Exception handler RVA указывает в non-executable или неизвестную секцию: %zu.", n);
    } else if (strcmp(code, "handler_writable_section") == 0) {
        sb_appendf(sb, "Exception handler RVA указывает в writable-секцию: %zu.", n);
    } else if (strcmp(code, "placement_exec_nonexec_handler_combo") == 0) {
        sb_append(sb, "Unwind metadata в executable-секции совпадает с handler targets в non-executable/unknown секции; такую комбинацию сложнее объяснить доброкачественно.");
    } else if (strcmp(code, "placement_integrity_combo") == 0) {
        sb_append(sb, "Подозрительное размещение metadata подтверждается проблемами целостности UNWIND_INFO / UnwindInfoRVA; вместе эти признаки сильнее, чем по отдельности.");
    } else if (strcmp(code, "reuse_dominance_combo") == 0) {
        sb_append(sb, "Активное повторное использование UNWIND_INFO совпадает с dominant handler target, что похоже на централизованный dispatcher-style metadata reuse.");
    } else if (strcmp(code, "writable_metadata_handler_combo") == 0) {
        sb_append(sb, "Writable unwind metadata совпадает с writable handler targets; такая комбинация нетипична для обычных compiler-produced images и требует внимательной проверки.");
    } else {
        html_escape(sb, finding->message);
    }
}

static void html_append_mitigation_message_ru(StrBuf* sb, const char* code, const char* message) {
    if (!code) {
        html_escape(sb, message);
    } else if (strcmp(code, "single_signal_only") == 0) {
        sb_append(sb, "Вклад дал только один не-сильный признак; как самостоятельное доказательство его нужно считать слабым.");
    } else if (strcmp(code, "small_sample_size") == 0) {
        sb_append(sb, "Очень маленькая выборка .pdata может завышать доли; уверенность снижена до появления большего числа функций.");
    } else if (strcmp(code, "placement_only_signal") == 0) {
        sb_append(sb, "Аномалии только по размещению могут встречаться в необычных, но доброкачественных layout; нужна ручная проверка.");
    } else if (strcmp(code, "single_family_only") == 0) {
        sb_append(sb, "Признаки относятся только к одному семейству; перед сильным выводом желательно cross-layer corroboration.");
    } else {
        html_escape(sb, message);
    }
}

bool report_write_html(const char* path,
                       const char* input_path,
                       const PeImage* img,
                       const RuntimeFunction* rf,
                       const UnwindSummary* us,
                       const bool* us_ok,
                       size_t n,
                       const HeuristicReport* hr,
                       const DynamicReport* dr,
                       const ReportMetadata* metadata,
                       StrBuf* err) {
    if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
        !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
        sb_append(err, "Invalid arguments or metadata for HTML report.\n");
        return false;
    }

    StrBuf sb = {0};
    StrBuf anomalies = {0};
    if (!sb_init(&sb, 32768) ||
        !sb_init(&anomalies, 128)) {
        sb_free(&anomalies);
        sb_free(&sb);
        sb_append(err, "OOM.\n");
        return false;
    }

    const char* tool_version = metadata->tool_version;
    const char* generated_at = metadata->generated_at_utc;
    const char* schema_version = metadata->report_schema_version;
    const char* input_hash = metadata->input_sha256;
    const char* build_platform = metadata->build_platform;
    const char* build_compiler = metadata->build_compiler;
    const char* build_mode = metadata->build_mode;
    size_t parse_failures = 0;
    size_t anomaly_count = 0;
    for (size_t i = 0; i < n; i++) {
        if (!us_ok[i]) parse_failures++;
        if (us[i].anomalies) anomaly_count++;
    }

    sb_append(&sb, "<!doctype html><html lang=\"ru\"><head><meta charset=\"utf-8\">");
    sb_append(&sb, "<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
    sb_append(&sb, "<meta http-equiv=\"Content-Security-Policy\" content=\"default-src 'none';"
                   "style-src 'unsafe-inline';script-src 'unsafe-inline';img-src 'none';"
                   "base-uri 'none';form-action 'none'\">");
    sb_append(&sb, "<title>Отчет EH/Unwind Analyzer - ");
    html_escape(&sb, input_path ? input_path : "");
    sb_append(&sb, "</title><style>");
    sb_append(&sb,
        ":root{color-scheme:dark;--bg:#0b1020;--panel:#121a2c;--panel2:#172238;"
        "--text:#e6edf7;--muted:#95a3b8;--line:#2a3851;--accent:#71b7ff;"
        "--ok:#54d99a;--warn:#f7c65d;--danger:#ff718b}"
        "*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--text);"
        "font:14px/1.5 system-ui,-apple-system,Segoe UI,sans-serif}"
        "main{max-width:1500px;margin:auto;padding:28px}h1{margin:0;font-size:28px}"
        "h2{margin:0 0 14px;font-size:19px}p{margin:6px 0}.muted{color:var(--muted)}"
        ".hero,.panel{background:var(--panel);border:1px solid var(--line);border-radius:14px;"
        "padding:20px;margin-bottom:18px}.hero{background:linear-gradient(135deg,#152541,#111827)}"
        ".cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:12px;"
        "margin:18px 0}.card{background:var(--panel2);border:1px solid var(--line);"
        "border-radius:11px;padding:14px}.card b{display:block;font-size:22px;margin-top:4px}"
        ".ok{color:var(--ok)}.warn{color:var(--warn)}.danger{color:var(--danger)}"
        ".neutral{color:var(--accent)}.badge{display:inline-block;border:1px solid currentColor;"
        "border-radius:999px;padding:2px 8px;font-size:12px}.grid{display:grid;"
        "grid-template-columns:repeat(auto-fit,minmax(320px,1fr));gap:18px}"
        ".table-wrap{overflow:auto;border:1px solid var(--line);border-radius:10px}"
        "table{width:100%;border-collapse:collapse;white-space:nowrap}th,td{padding:9px 11px;"
        "border-bottom:1px solid var(--line);text-align:left;vertical-align:top}"
        "th{position:sticky;top:0;background:#1a2740;color:#bcd0e9;z-index:1}"
        "tr:last-child td{border-bottom:0}tr:hover td{background:#172238}"
        "code{color:#b8dcff}pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#0c1425;"
        "border:1px solid var(--line);border-radius:9px;padding:13px;color:#c8d6e8}"
        ".meta{display:grid;grid-template-columns:max-content 1fr;gap:5px 14px;"
        "overflow-wrap:anywhere}.meta dt{color:var(--muted)}.meta dd{margin:0}"
```

```
1226 |          ".toolbar{display:flex;gap:10px;align-items:center;margin-bottom:12px;flex-wrap:wrap}"
1227 |          "input{min-width:280px;max-width:100%;background:#0c1425;color:var(--text);"
1228 |          "border:1px solid var(--line);border-radius:8px;padding:9px 11px}"
1229 |          ".empty{color:var(--muted);font-style:italic}.wrap{white-space:normal;min-width:280px}"
1230 |          "footer{text-align:center;color:var(--muted);padding:12px}@media(max-width:700px){"
1231 |          "main{padding:14px}.hero,.panel{padding:15px}.meta{grid-template-columns:1fr}}");
1232 |      sb_append(&sb, "</style></head><body><main>");
1233 |
1234 |      sb_append(&sb, "<section class=\"hero\"><h1>EH/Unwind Analyzer</h1><p class=\"muted\">");
1235 |      html_escape(&sb, input_path ? input_path : "");
1236 |      sb_append(&sb, "</p><div class=\"cards\">");
1237 |      sb_appendf(&sb, "<div class=\"card\">Записи RUNTIME_FUNCTION<b>%zu</b></div>", n);
1238 |      sb_appendf(&sb, "<div class=\"card\">Ошибки разбора<b class=\"%s\">%zu</b></div>",
1239 |                 parse_failures ? "danger" : "ok", parse_failures);
1240 |      sb_appendf(&sb, "<div class=\"card\">Аномальные записи<b class=\"%s\">%zu</b></div>",
1241 |                 anomaly_count ? "warn" : "ok", anomaly_count);
1242 |      if (hr) {
1243 |          sb_appendf(&sb, "<div class=\"card\">Эвристические признаки<b class=\"%s\">%zu</b></div>",
1244 |                     hr->finding_count ? "warn" : "ok", hr->finding_count);
1245 |          sb_appendf(&sb, "<div class=\"card\">Оценка подозрительности<b class=\"%s\">%d / 100</b></div>",
1246 |                     html_score_class(hr), hr->score);
1247 |          sb_append(&sb, "<div class=\"card\">Статический вывод<b class=\"");
1248 |          sb_append(&sb, html_score_class(hr));
1249 |          sb_append(&sb, "\" style=\"font-size:16px\">");
1250 |          html_escape(&sb, ru_verdict_text(hr->verdict));
1251 |          sb_append(&sb, "</b></div>");
1252 |          sb_appendf(&sb, "<div class=\"card\">Уверенность<b>%.0f%%</b></div>",
1253 |                     hr->confidence * 100.0);
1254 |      }
1255 |      sb_append(&sb, "<div class=\"card\">Динамическая проверка<b class=\"");
1256 |      sb_append(&sb, (dr && dr->ran && dr->ok && !dr->timed_out) ? "ok" : "neutral");
1257 |      sb_append(&sb, "\" style=\"font-size:16px\">");
1258 |      html_escape(&sb, ru_status_text(dr));
1259 |      sb_append(&sb, "</b></div></div>");
1260 |
1261 |      sb_append(&sb, "<dl class=\"meta\"><dt>Сформировано</dt><dd>");
1262 |      html_escape(&sb, generated_at);
1263 |      sb_append(&sb, "</dd><dt>Инструмент / schema</dt><dd>");
1264 |      html_escape(&sb, tool_version);
1265 |      sb_append(&sb, " / ");
1266 |      html_escape(&sb, schema_version);
1267 |      sb_append(&sb, "</dd><dt>SHA-256</dt><dd><code>");
1268 |      html_escape(&sb, input_hash);
1269 |      sb_append(&sb, "</code></dd><dt>Сборка</dt><dd>");
1270 |      html_escape(&sb, build_platform);
1271 |      sb_append(&sb, " / ");
1272 |      html_escape(&sb, build_compiler);
1273 |      sb_append(&sb, " / ");
1274 |      html_escape(&sb, build_mode);
1275 |      sb_append(&sb, "</dd><dt>Параметры анализа</dt><dd>");
1276 |      sb_appendf(&sb, "strict ABI: %s; follow chain: %s; max depth: %u",
1277 |                 ru_yes_no(metadata->strict_abi),
1278 |                 ru_yes_no(metadata->follow_chain),
1279 |                 (unsigned)metadata->max_chain_depth);
1280 |      sb_appendf(&sb, "</dd><dt>Размер файла</dt><dd>%llu байт</dd><dt>Команда</dt><dd><code>",
1281 |                 (unsigned long long)metadata->input_size);
1282 |      html_escape(&sb, metadata->command_line);
1283 |      sb_append(&sb, "</code>");
1284 |      sb_append(&sb, "</dd></dl></section>");
1285 |
1286 |      sb_append(&sb, "<div class=\"grid\"><section class=\"panel\"><h2>Образ PE</h2><dl class=\"meta\">");
1287 |      sb_appendf(&sb,
1288 |          "<dt>ImageBase</dt><dd><code>0x%llx</code></dd>"
1289 |          "<dt>SizeOfImage</dt><dd><code>0x%x</code></dd>"
1290 |          "<dt>SizeOfHeaders</dt><dd><code>0x%x</code></dd>"
1291 |          "<dt>Exception directory</dt><dd><code>0x%08x + 0x%x</code></dd>"
1292 |          "<dt>SectionAlignment / FileAlignment</dt><dd><code>0x%x / 0x%x</code></dd></dl></section>",
1293 |          (unsigned long long)img->image_base, (unsigned)img->size_of_image,
1294 |          (unsigned)img->size_of_headers, (unsigned)img->exception_rva,
1295 |          (unsigned)img->exception_size, (unsigned)img->section_alignment,
1296 |          (unsigned)img->file_alignment);
1297 |      sb_append(&sb, "<section class=\"panel\"><h2>Структурная проверка</h2><dl class=\"meta\">");
1298 |      sb_appendf(&sb,
1299 |          "<dt>Raw-диапазоны вне файла</dt><dd>%zu</dd><dt>Virtual-диапазоны вне image</dt><dd>%zu</dd>"
1300 |          "<dt>Пересечения raw</dt><dd>%zu</dd><dt>Пересечения virtual</dt><dd>%zu</dd>"
1301 |          "<dt>Невыровненные секции</dt><dd>%zu</dd><dt>Exception directory OOB</dt><dd>%s</dd>"
1302 |          "</dl></section></div>",
1303 |          img->section_raw_oob, img->section_virtual_oob, img->section_raw_overlaps,
1304 |          img->section_virtual_overlaps, img->section_misaligned,
1305 |          ru_yes_no(img->exception_directory_oob));
1306 |
1307 |      sb_append(&sb, "<section class=\"panel\"><h2>Секции</h2><div class=\"table-wrap\"><table>"
1308 |                     "<thead><tr><th>Имя</th><th>VirtualAddress</th><th>VirtualSize</th>"
1309 |                     "<th>Raw offset</th><th>Raw size</th><th>Права</th></tr></thead><tbody>");
1310 |      for (size_t i = 0; i < img->num_sections; i++) {
1311 |          const PeSection* section = &img->sections[i];
1312 |          char permissions[4];
1313 |          permissions[0] = (section->characteristics & IMAGE_SCN_MEM_READ) ? 'R' : '-';
1314 |          permissions[1] = (section->characteristics & IMAGE_SCN_MEM_WRITE) ? 'W' : '-';
1315 |          permissions[2] = (section->characteristics & IMAGE_SCN_MEM_EXECUTE) ? 'X' : '-';
1316 |          permissions[3] = '\0';
1317 |          sb_append(&sb, "<tr>");
1318 |          html_text_cell(&sb, section->name);
1319 |          html_hex32_cell(&sb, section->virtual_address);
1320 |          html_hex32_cell(&sb, section->virtual_size);
1321 |          html_hex32_cell(&sb, section->raw_ptr);
1322 |          html_hex32_cell(&sb, section->raw_size);
1323 |          html_text_cell(&sb, permissions);
1324 |          sb_append(&sb, "</tr>");
1325 |      }
1326 |      sb_append(&sb, "</tbody></table></div></section>");
1327 |
1328 |      if (hr || dr) {
1329 |          AssessmentSummary summary = assessment_summary(hr, dr);
1330 |          ManualValidationSteps steps = collect_manual_validation_steps(hr, dr);
1331 |          sb_append(&sb, "<section class=\"panel\"><h2>Итоговая оценка</h2><div class=\"cards\">");
1332 |          sb_append(&sb, "<div class=\"card\">Вывод<b style=\"font-size:16px\">");
1333 |          html_escape(&sb, ru_verdict_text(summary.final_verdict));
1334 |          sb_append(&sb, "</b></div><div class=\"card\">Сила доказательств<b style=\"font-size:16px\">");
1335 |          html_escape(&sb, ru_case_strength_text(summary.strength));
1336 |          sb_appendf(&sb,
1337 |              "</b></div><div class=\"card\">Итоговая уверенность<b>%.0f%%</b></div>"
1338 |              "<div class=\"card\">Статика / динамика<b style=\"font-size:16px\">",
1339 |              summary.final_confidence * 100.0);
1340 |          html_escape(&sb, ru_level_text(summary.static_evidence_level));
```

```
        sb_append(&sb, " / ");
        html_escape(&sb, ru_level_text(summary.runtime_evidence_level));
        sb_appendf(&sb,
            "</b></div><div class=\"card\">Корреляция слоев<b class=\"%s\" "
            "style=\"font-size:16px\">%s</b></div>"
            "<div class=\"card\">Динамические сигналы<b>%u</b></div></div>",
            summary.correlated ? "ok" : "neutral",
            ru_yes_no(summary.correlated),
            summary.dynamic_signals);
        if (steps.count) {
            sb_append(&sb, "<h2 style=\"margin-top:18px\">План ручной проверки</h2><ol>");
            for (size_t i = 0; i < steps.count; i++) {
                sb_append(&sb, "<li>");
                html_append_manual_step_ru(&sb, steps.items[i]);
                sb_append(&sb, "</li>");
            }
            sb_append(&sb, "</ol>");
        }
        sb_append(&sb, "</section>");
    }

    if (hr) {
        sb_append(&sb, "<section class=\"panel\"><h2>Статические признаки</h2>");
        if (hr->finding_count == 0) {
            sb_append(&sb, "<p class=\"empty\">Эвристические признаки не обнаружены.</p>");
        } else {
            sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Важность</th>"
                           "<th>Код</th><th>Семейство</th><th>Затронуто</th><th>RVA</th>"
                           "<th>Секция</th><th>Описание</th></tr></thead><tbody>");
            for (size_t i = 0; i < hr->finding_count; i++) {
                const HeuristicFinding* finding = &hr->findings[i];
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, ru_severity_text(finding->severity));
                html_text_cell(&sb, finding->code);
                html_text_cell(&sb, finding->family);
                sb_appendf(&sb, "<td>%zu</td>", finding->affected_count);
                if (finding->has_location) html_hex32_cell(&sb, finding->representative_rva);
                else sb_append(&sb, "<td>-</td>");
                html_text_cell(&sb, finding->has_location
                                    ? section_name_or_dash(img, finding->representative_rva)
                                    : "-");
                sb_append(&sb, "<td class=\"wrap\">");
                html_append_finding_message_ru(&sb, finding);
                sb_append(&sb, "</td></tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        }
        if (hr->mitigation_count) {
            sb_append(&sb, "<h2 style=\"margin-top:18px\">Правила снижения уверенности</h2><div class=\"table-wrap\">"
                           "<table><thead><tr><th>Код</th><th>Пояснение</th></tr></thead><tbody>");
            for (size_t i = 0; i < hr->mitigation_count; i++) {
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, hr->mitigations[i].code);
                sb_append(&sb, "<td class=\"wrap\">");
                html_append_mitigation_message_ru(&sb, hr->mitigations[i].code, hr->mitigations[i].message);
                sb_append(&sb, "</td></tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        }
        sb_append(&sb, "<h2 style=\"margin-top:18px\">Детали анализа</h2><pre>");
        html_escape(&sb, hr->details.s ? hr->details.s : "");
        sb_append(&sb, "</pre></section>");
    }

    if (dr) {
        sb_append(&sb, "<section class=\"panel\"><h2>Динамическая проверка</h2><div class=\"cards\">");
        sb_appendf(&sb, "<div class=\"card\">Исключения<b>%u</b></div>",
                   (unsigned)dr->exceptions_seen);
        sb_appendf(&sb, "<div class=\"card\">Попадания в handler<b>%u</b></div>",
                   (unsigned)(dr->unwind_handler_hits + dr->veh_hits + dr->vch_hits + dr->uef_hits));
        sb_appendf(&sb, "<div class=\"card\">Вызовы runtime-таблиц<b>%u</b></div>",
                   (unsigned)(dr->rtl_add_function_table_calls +
                              dr->rtl_install_function_table_cb_calls));
        sb_appendf(&sb, "<div class=\"card\">Потерянные события<b class=\"%s\">%u</b></div>",
                   dr->dropped_events ? "warn" : "ok", (unsigned)dr->dropped_events);
        sb_appendf(&sb, "<div class=\"card\">Инструментация завершена<b class=\"%s\">%s</b></div>",
                   dr->instrumentation_complete ? "ok" : "warn",
                   ru_yes_no(dr->instrumentation_complete));
        sb_appendf(&sb, "<div class=\"card\">Handlers найдены / включены<b>%u / %u</b></div>",
                   (unsigned)dr->unwind_handlers_discovered,
                   (unsigned)dr->unwind_handler_bp_set);
        sb_appendf(&sb, "<div class=\"card\">Ошибки breakpoint<b class=\"%s\">%u</b></div>",
                   dr->breakpoint_set_failures ? "warn" : "ok",
                   (unsigned)dr->breakpoint_set_failures);
        sb_appendf(&sb, "<div class=\"card\">Покрытие модулей<b class=\"%s\">пик %u, ошибки %u, усечения %u</b></div>",
                   (dr->module_refresh_failures || dr->module_list_truncations) ? "warn" : "ok",
                   (unsigned)dr->modules_observed_peak,
                   (unsigned)dr->module_refresh_failures,
                   (unsigned)dr->module_list_truncations);
        sb_append(&sb, "</div>");
        if (dr->event_count) {
            sb_append(&sb, "<div class=\"table-wrap\"><table><thead><tr><th>Тип</th><th>Модуль</th>"
                           "<th>Thread</th><th>Exception</th><th>Chance</th><th>Address</th>"
                           "<th>Arg 0</th><th>Arg 1</th><th>Arg 2</th><th>Arg 3</th>"
                           "</tr></thead><tbody>");
            for (uint32_t i = 0; i < dr->event_count; i++) {
                const DynamicEvent* event = &dr->events[i];
                sb_append(&sb, "<tr>");
                html_text_cell(&sb, event->type);
                html_text_cell(&sb, event->module);
                sb_appendf(&sb, "<td>%u</td>", (unsigned)event->thread_id);
                html_hex32_cell(&sb, event->exception_code);
                html_text_cell(&sb, event->first_chance ? "first" : "second / n/a");
                html_hex64_cell(&sb, event->address);
                html_hex64_cell(&sb, event->arg0);
                html_hex64_cell(&sb, event->arg1);
                html_hex64_cell(&sb, event->arg2);
                html_hex64_cell(&sb, event->arg3);
                sb_append(&sb, "</tr>");
            }
            sb_append(&sb, "</tbody></table></div>");
        } else {
            sb_append(&sb, "<p class=\"empty\">Динамические события не записаны.</p>");
        }
        if (dr->details.s && dr->details.len) {
```

```
            sb_append(&sb, "<h2 style=\"margin-top:18px\">Журнал выполнения</h2><pre>");
            html_escape(&sb, dr->details.s);
            sb_append(&sb, "</pre>");
        }
        sb_append(&sb, "</section>");
    }

    sb_append(&sb, "<section class=\"panel\"><h2>Таблица RUNTIME_FUNCTION</h2><div class=\"toolbar\">"
                   "<input id=\"function-filter\" type=\"search\" placeholder=\"Фильтр по RVA, секции, флагу или ошибке\">"
                   "<span id=\"function-count\" class=\"muted\"></span></div>"
                   "<div class=\"table-wrap\"><table id=\"functions\"><thead><tr><th>#</th>"
                   "<th>Begin</th><th>End</th><th>Size</th><th>Секция кода</th><th>Unwind RVA</th>"
                   "<th>Unwind section</th><th>Статус</th><th>Version / flags</th><th>Stack</th>"
                   "<th>Handler</th><th>Аномалии / ошибка</th></tr></thead><tbody>");
    for (size_t i = 0; i < n; i++) {
        char flags[64];
        unwind_flags_to_string(us[i].flags, flags);
        anomalies.len = 0;
        if (anomalies.s) anomalies.s[0] = '\0';
        anomalies_to_str(&anomalies, us[i].anomalies);
        sb_append(&sb, "<tr>");
        sb_appendf(&sb, "<td>%zu</td>", i);
        html_hex32_cell(&sb, rf[i].begin_rva);
        html_hex32_cell(&sb, rf[i].end_rva);
        sb_appendf(&sb, "<td>%u</td>", (unsigned)runtime_function_size(&rf[i]));
        html_text_cell(&sb, section_name_or_dash(img, rf[i].begin_rva));
        html_hex32_cell(&sb, rf[i].unwind_rva);
        html_text_cell(&sb, section_name_or_dash(img, rf[i].unwind_rva));
        sb_append(&sb, us_ok[i] ? "<td class=\"ok\">OK</td>" : "<td class=\"danger\">Ошибка</td>");
        sb_appendf(&sb, "<td>v%u / ", (unsigned)us[i].version);
        html_escape(&sb, flags);
        sb_append(&sb, "</td>");
        sb_appendf(&sb, "<td>%u B</td>", (unsigned)us[i].fixed_stack_bytes);
        if (us[i].has_handler) html_hex32_cell(&sb, us[i].handler_rva);
        else sb_append(&sb, "<td>-</td>");
        sb_append(&sb, "<td class=\"wrap\">");
        if (us[i].anomalies) html_escape(&sb, anomalies.s);
        if (us[i].anomalies && us[i].parse_error[0]) sb_append(&sb, "; ");
        html_escape(&sb, us[i].parse_error);
        if (!us[i].anomalies && !us[i].parse_error[0]) sb_append(&sb, "-");
        sb_append(&sb, "</td></tr>");
    }
    sb_append(&sb, "</tbody></table></div></section>");

    sb_append(&sb,
        "<footer>Сформировано EH/Unwind Analyzer ");
    html_escape(&sb, tool_version);
    sb_append(&sb,
        "</footer></main><script>"
        "(()=>{const q=document.getElementById('function-filter');"
        "const rows=[...document.querySelectorAll('#functions tbody tr')];"
        "const out=document.getElementById('function-count');"
        "function run(){const needle=q.value.trim().toLowerCase();let shown=0;"
        "for(const row of rows){const visible=!needle||row.textContent.toLowerCase().includes(needle);"
        "row.hidden=!visible;if(visible)shown++;}out.textContent=`${shown} из ${rows.length}`;}"
        "q.addEventListener('input',run);run();})();"
        "</script></body></html>");

    bool ok = sb_ok(&sb) && sb_ok(&anomalies);
    if (ok) ok = file_write_atomic(path, sb.s, sb.len);
    if (!ok) {
        sb_appendf(err, "%s: %s\n",
                   sb_ok(&sb) && sb_ok(&anomalies)
                       ? "Failed to write HTML"
                       : "Out of memory while building HTML report",
                   path);
    }
    sb_free(&anomalies);
    sb_free(&sb);
    return ok;
}
```

src/main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <errno.h>
#include <limits.h>

#include "util.h"
#include "pe.h"
#include "unwind.h"
#include "heuristics.h"
#include "report.h"
#include "dynamic.h"
#include "app_info.h"

#if defined(_WIN32)
#include <windows.h>
#endif

typedef struct Args {
    const char* input;
    bool dump;
    bool show_ops;
    bool heuristics;
    const char* csv_path;
    const char* ghidra_csv_path;
    const char* json_path;
    const char* html_path;
    int  dump_limit;       /* -1 => all */
    bool follow_chain;
    bool strict_abi;
    int  max_chain_depth;
    bool dynamic;
    uint32_t timeout_ms;
    uint32_t trace_steps;
    const char** child_argv;
    int child_argc;
} Args;

typedef enum ParseResult {
    PARSE_OK = 0,
    PARSE_HELP = 1,
    PARSE_VERSION = 2,
    PARSE_ERR = 3
} ParseResult;

static char* dup_cstr(const char* s) {
    if (!s) return NULL;
    size_t n = strlen(s);
    char* out = (char*)malloc(n + 1u);
    if (!out) return NULL;
    memcpy(out, s, n + 1u);
    return out;
}

static char* argv_text_to_utf8(const char* s) {
    if (!s) return NULL;
#if defined(_WIN32)
    int wide_len = MultiByteToWideChar(CP_ACP, 0, s, -1, NULL, 0);
    if (wide_len <= 0) return dup_cstr(s);
    wchar_t* wide = (wchar_t*)calloc((size_t)wide_len, sizeof(wchar_t));
    if (!wide) return NULL;
    if (MultiByteToWideChar(CP_ACP, 0, s, -1, wide, wide_len) <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wide, -1, NULL, 0, NULL, NULL);
    if (utf8_len <= 0) {
        free(wide);
        return dup_cstr(s);
    }
    char* out = (char*)malloc((size_t)utf8_len);
    if (!out) {
        free(wide);
        return NULL;
    }
    if (WideCharToMultiByte(CP_UTF8, 0, wide, -1, out, utf8_len, NULL, NULL) <= 0) {
        free(out);
        free(wide);
        return dup_cstr(s);
    }
    free(wide);
    return out;
#else
    return dup_cstr(s);
#endif
}

static void print_version(void) {
    printf("EH/Unwind Analyzer %s\n", app_version());
    printf("Report schema: %s\n", app_report_schema_version());
    printf("Build: %s, %s, %s\n", app_build_compiler(), app_build_platform(), app_build_mode());
}

static void usage(const char* argv0) {
    printf("EH/Unwind Analyzer %s (PE x64)\n", app_version());
    printf("Usage: %s <pefile> [options]\n\n", argv0);
    printf("Options:\n");
    printf("  --dump               Dump all RUNTIME_FUNCTION entries\n");
    printf("  --limit N            Limit dump to first N entries (default: all when --dump)\n");
    printf("  --ops                With --dump: also print unwind ops list per function\n");
    printf("  --heuristics         Run EH/Unwind anomaly heuristics and print report\n");
    printf("  --csv <path>         Write analysis CSV (full fields)\n");
    printf("  --ghidra-csv <path>  Write CSV for Ghidra import (begin/end/name/comment)\n");
    printf("  --json <path>        Write JSON (functions + heuristic/dynamic assessment)\n");
    printf("  --html <path>        Write a self-contained interactive HTML report\n");
    printf("  --no-follow-chain    Do not follow UNW_FLAG_CHAININFO\n");
    printf("  --strict-abi         Flag formal Microsoft x64 unwind ABI violations\n");
    printf("  --max-chain-depth N  Max chain depth, 1..16 (default 8)\n");
    printf("  --dynamic            Run Windows-only dynamic exception validation (debugger)\n");
    printf("  --timeout-ms N       Dynamic mode: hard timeout, N>0 (default 15000)\n");
```

```
    printf("  --trace-steps N       Dynamic mode: single-step samples after handler hit (default 64, range 0..256)\n");
    printf("  --                    Separator: remaining args are passed to the target program\n");
    printf("  --version             Show version and build information\n");
    printf("  --help                Show this help\n");
}

static bool append_invocation_arg(StrBuf* out, const char* arg) {
    if (!out || !arg) return false;
    bool quote = (*arg == '\0') || strpbrk(arg, " \t\"") != NULL;
    if (!quote) return sb_append(out, arg);
    if (!sb_append(out, "\"")) return false;
    for (const char* p = arg; *p; p++) {
        if (*p == '"' || *p == '\\') {
            if (!sb_appendn(out, "\\", 1)) return false;
        }
        if (!sb_appendn(out, p, 1)) return false;
    }
    return sb_append(out, "\"");
}

static bool build_invocation(int argc, char** argv, StrBuf* out) {
    if (!out) return false;
    for (int i = 0; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            int redacted = argc - i - 1;
            if (i && !sb_appendn(out, " ", 1)) return false;
            if (!sb_appendf(out, "-- [%d target argument%s redacted]",
                            redacted, redacted == 1 ? "" : "s")) {
                return false;
            }
            break;
        }
        if (i && !sb_appendn(out, " ", 1)) return false;
        char* utf8_arg = argv_text_to_utf8(argv[i]);
        if (!utf8_arg) return false;
        bool ok = append_invocation_arg(out, utf8_arg);
        free(utf8_arg);
        if (!ok) return false;
    }
    return true;
}

static bool parse_long_range(const char* text, long min_value, long max_value, long* out) {
    if (!text || !*text || !out) return false;
    errno = 0;
    char* end = NULL;
    long value = strtol(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < min_value || value > max_value) return false;
    *out = value;
    return true;
}

static bool parse_u32_range(const char* text, uint32_t min_value, uint32_t max_value, uint32_t* out) {
    if (!text || !*text || !out || text[0] == '-') return false;
    errno = 0;
    char* end = NULL;
    unsigned long value = strtoul(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '\0') return false;
    if (value < (unsigned long)min_value || value > (unsigned long)max_value) return false;
    *out = (uint32_t)value;
    return true;
}

static ParseResult parse_args(int argc, char** argv, Args* a) {
    memset(a, 0, sizeof(*a));
    a->dump_limit = -1;
    a->follow_chain = true;
    a->max_chain_depth = 8;
    a->timeout_ms = 15000;
    a->trace_steps = 64;

    if (argc < 2) return PARSE_ERR;
    if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) return PARSE_HELP;
    if (strcmp(argv[1], "--version") == 0) return PARSE_VERSION;
    a->input = argv[1];

    for (int i = 2; i < argc; i++) {
        const char* s = argv[i];
        if (strcmp(s, "--") == 0) {
            a->child_argc = argc - (i + 1);
            a->child_argv = (const char**)&argv[i + 1];
            break;
        }
        if (strcmp(s, "--help") == 0 || strcmp(s, "-h") == 0) return PARSE_HELP;
        else if (strcmp(s, "--version") == 0) return PARSE_VERSION;
        else if (strcmp(s, "--dump") == 0) a->dump = true;
        else if (strcmp(s, "--ops") == 0) a->show_ops = true;
        else if (strcmp(s, "--heuristics") == 0) a->heuristics = true;
        else if (strcmp(s, "--dynamic") == 0) a->dynamic = true;
        else if (strcmp(s, "--no-follow-chain") == 0) a->follow_chain = false;
        else if (strcmp(s, "--strict-abi") == 0) a->strict_abi = true;
        else if (strcmp(s, "--limit") == 0 && i + 1 < argc) {
            long value = 0;
            if (!parse_long_range(argv[++i], 0, INT_MAX, &value)) {
                fprintf(stderr, "Invalid value for --limit: %s (expected 0..%d)\n", argv[i], INT_MAX);
                return PARSE_ERR;
            }
            a->dump_limit = (int)value;
        }
        else if (strcmp(s, "--max-chain-depth") == 0 && i + 1 < argc) {
            long value = 0;
            if (!parse_long_range(argv[++i], 1, 16, &value)) {
                fprintf(stderr, "Invalid value for --max-chain-depth: %s (expected 1..16)\n", argv[i]);
                return PARSE_ERR;
            }
            a->max_chain_depth = (int)value;
        }
        else if (strcmp(s, "--timeout-ms") == 0 && i + 1 < argc) {
            if (!parse_u32_range(argv[++i], 1, UINT32_MAX, &a->timeout_ms)) {
                fprintf(stderr, "Invalid value for --timeout-ms: %s (expected 1..%u)\n",
                        argv[i], (unsigned)UINT32_MAX);
                return PARSE_ERR;
            }
        }
```

```
        else if (strcmp(s, "--trace-steps") == 0 && i + 1 < argc) {
            if (!parse_u32_range(argv[++i], 0, 256, &a->trace_steps)) {
                fprintf(stderr, "Invalid value for --trace-steps: %s (expected 0..256)\n", argv[i]);
                return PARSE_ERR;
            }
        }
        else if (strcmp(s, "--csv") == 0 && i + 1 < argc) { a->csv_path = argv[++i]; }
        else if (strcmp(s, "--ghidra-csv") == 0 && i + 1 < argc) { a->ghidra_csv_path = argv[++i]; }
        else if (strcmp(s, "--json") == 0 && i + 1 < argc) { a->json_path = argv[++i]; }
        else if (strcmp(s, "--html") == 0 && i + 1 < argc) { a->html_path = argv[++i]; }
        else {
            fprintf(stderr, "Unknown/invalid option: %s\n", s);
            return PARSE_ERR;
        }
    }
    return PARSE_OK;
}

static void print_one(size_t idx, const RuntimeFunction* r, const UnwindSummary* u, bool ok) {
    uint32_t size = (r->end_rva >= r->begin_rva) ? (r->end_rva - r->begin_rva) : 0;
    char fbuf[64];
    unwind_flags_to_string(u->flags, fbuf);

    printf("[%zu] 0x%08X..0x%08X  sz=%-6u  unwind=0x%08X  ",
           idx, (unsigned)r->begin_rva, (unsigned)r->end_rva, (unsigned)size, (unsigned)r->unwind_rva);

    if (!ok) {
        printf("UNWIND: <parse failed%s%s>\n",
               u->parse_error[0] ? ": " : "",
               u->parse_error);
        return;
    }

    printf("v%u flags=%s prolog=%u stack=%uB",
           (unsigned)u->version, fbuf, (unsigned)u->prolog_size, (unsigned)u->fixed_stack_bytes);

    if (u->frame_reg) printf(" fp=%s+%u", unwind_reg_name(u->frame_reg), (unsigned)(u->frame_off * 16u));
    if (u->has_handler) printf(" handler=0x%08X", (unsigned)u->handler_rva);
    if (u->has_chain) printf(" chain_u=0x%08X", (unsigned)u->chain.unwind_rva);
    if (u->anomalies) printf(" anom=0x%08X", (unsigned)u->anomalies);
    printf("\n");
}

int main(int argc, char** argv) {
    Args a;
    int exit_code = 0;
    ParseResult pres = parse_args(argc, argv, &a);
    if (pres != PARSE_OK) {
        if (pres == PARSE_VERSION) {
            print_version();
            return 0;
        }
        usage(argv[0]);
        return (pres == PARSE_HELP) ? 0 : 2;
    }

    uint8_t* owned = NULL;
    Buf b;
    if (!buf_init_from_file(&b, a.input, &owned)) {
        fprintf(stderr, "Failed to read file: %s\n", a.input);
        return 1;
    }

    StrBuf err;
    sb_init(&err, 1024);

    PeImage img;
    if (!pe_parse(&img, &b, &err)) {
        fprintf(stderr, "PE parse error:\n%s", err.s ? err.s : "");
        sb_free(&err);
        buf_free_owned(owned);
        return 1;
    }

    RuntimeFunction* rf = NULL;
    size_t rf_count = 0;
    StrBuf err2;
    sb_init(&err2, 1024);

    RuntimeFunctionReadResult rf_read = unwind_read_runtime_functions_ex(
        &b, &img, &rf, &rf_count, &err2);
    if (rf_read != RUNTIME_FUNCTION_READ_OK) {
        fprintf(stderr, "Failed to read exception directory:\n%s", err2.s ? err2.s : "");
        if (rf_read == RUNTIME_FUNCTION_READ_OOM) {
            pe_free(&img);
            sb_free(&err);
            sb_free(&err2);
            buf_free_owned(owned);
            return 1;
        }
        /*
         * Keep the established report field/finding code for compatibility.
         * It now covers every structurally unreadable exception directory.
         */
        img.exception_directory_oob = true;
        fprintf(stderr, "Continuing with a partial structural report.\n");
        exit_code = 1;
    }

    report_print_brief(a.input, &img, rf_count);

    UnwindSummary* us = (UnwindSummary*)calloc(rf_count ? rf_count : 1, sizeof(UnwindSummary));
    bool* us_ok = (bool*)calloc(rf_count ? rf_count : 1, sizeof(bool));
    if (!us || !us_ok) {
        fprintf(stderr, "Out of memory.\n");
        free(us);
        free(us_ok);
        free(rf);
        pe_free(&img);
        sb_free(&err);
        sb_free(&err2);
        buf_free_owned(owned);
        return 1;
    }

```

```
    UnwindParseOpts uopts = {
        .max_chain_depth = a.max_chain_depth,
        .follow_chain = a.follow_chain,
        .strict_abi = a.strict_abi
    };

    StrBuf ops;
    sb_init(&ops, 1024);

    for (size_t i = 0; i < rf_count; i++) {
        StrBuf perr;
        sb_init(&perr, 256);
        if (a.show_ops) { ops.len = 0; if (ops.s) ops.s[0]=0; }
        bool ok = unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], a.show_ops ? &ops : NULL, &perr);
        us_ok[i] = ok;
        if (a.dump && a.show_ops && ok) {
            /* ops will be printed in dump loop below */
        }
        sb_free(&perr);
    }

    if (a.dump) {
        size_t limit = (a.dump_limit < 0) ? rf_count : (size_t)a.dump_limit;
        if (limit > rf_count) limit = rf_count;
        printf("\n[Dump] Showing %zu / %zu entries\n", limit, rf_count);
        for (size_t i = 0; i < limit; i++) {
            print_one(i, &rf[i], &us[i], us_ok[i]);
            if (a.show_ops && us_ok[i]) {
                StrBuf ops2;
                StrBuf perr;
                sb_init(&ops2, 1024);
                sb_init(&perr, 256);
                (void)unwind_parse(&b, &img, rf[i].unwind_rva, &uopts, &us[i], &ops2, &perr);
                if (ops2.s && ops2.len) printf("%s", ops2.s);
                sb_free(&ops2);
                sb_free(&perr);
            }
        }
    }

    HeuristicReport hr;
    HeuristicReport* hr_ptr = NULL;
    DynamicReport dr;
    memset(&dr, 0, sizeof(dr));
    DynamicReport* dr_ptr = NULL;
    bool dr_initialized = false;
    if (a.heuristics) {
        StrBuf perr;
        sb_init(&perr, 512);
        if (heuristics_analyze(&img, rf, us, us_ok, rf_count, &hr, &perr)) {
            report_print_heuristics(&hr);
            hr_ptr = &hr;
        } else {
            fprintf(stderr, "Heuristic analysis failed:\n%s", perr.s ? perr.s : "");
            exit_code = 1;
        }
        sb_free(&perr);
    }

    if (a.dynamic) {
        StrBuf derr;
        sb_init(&derr, 1024);
        printf("\n[Dynamic] Starting debugger evidence collection (Windows only)...\n");
        bool ran = dynamic_collect_windows(a.input, a.child_argv, a.child_argc,
                                           &img, rf, us, us_ok, rf_count,
                                           a.timeout_ms, a.trace_steps,
                                           &dr, &derr);
        dr_initialized = true;
        dr_ptr = &dr;
        if (!ran) {
            fprintf(stderr, "[Dynamic] Failed: %s", derr.s ? derr.s : "");
            exit_code = 1;
        } else {
            printf("[Dynamic] Done. exceptions_seen=%u unwind_handler_hits=%u veh_hits=%u dyn_ftable_calls=%u\n",
                   (unsigned)dr.exceptions_seen,
                   (unsigned)dr.unwind_handler_hits,
                   (unsigned)dr.veh_hits,
                   (unsigned)dr.rtl_add_function_table_calls);
            if (dr.details.s && dr.details.len) {
                printf("%s\n", dr.details.s);
            }
        }
        sb_free(&derr);
    }

    if (a.csv_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (!report_write_csv(a.csv_path, &img, rf, us, us_ok, rf_count, false, &e)) {
            fprintf(stderr, "CSV write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else {
            printf("\nWrote CSV: %s\n", a.csv_path);
        }
        sb_free(&e);
    }
    if (a.ghidra_csv_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (!report_write_csv(a.ghidra_csv_path, &img, rf, us, us_ok, rf_count, true, &e)) {
            fprintf(stderr, "Ghidra CSV write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else {
            printf("\nWrote Ghidra CSV: %s\n", a.ghidra_csv_path);
        }
        sb_free(&e);
    }

    StrBuf invocation = {0};
    char timestamp[32] = "";
    char input_sha256[65] = "";
    char* input_display = NULL;
    ReportMetadata metadata;
    memset(&metadata, 0, sizeof(metadata));
    ReportMetadata* metadata_ptr = NULL;
```

```
    if (a.json_path || a.html_path) {
        input_display = argv_text_to_utf8(a.input);
        bool metadata_ok = sb_init(&invocation, 256) &&
                           input_display &&
                           build_invocation(argc, argv, &invocation) &&
                           app_utc_timestamp(timestamp, sizeof(timestamp)) &&
                           sha256_hex(b.data, b.size, input_sha256);
        if (!metadata_ok) {
            fprintf(stderr, "Failed to prepare report metadata.\n");
            exit_code = 1;
        } else {
            metadata.tool_version = app_version();
            metadata.report_schema_version = app_report_schema_version();
            metadata.generated_at_utc = timestamp;
            metadata.input_sha256 = input_sha256;
            metadata.input_size = (uint64_t)b.size;
            metadata.command_line = invocation.s ? invocation.s : "";
            metadata.build_compiler = app_build_compiler();
            metadata.build_platform = app_build_platform();
            metadata.build_mode = app_build_mode();
            metadata.strict_abi = a.strict_abi;
            metadata.follow_chain = a.follow_chain;
            metadata.max_chain_depth = (uint32_t)a.max_chain_depth;
            metadata_ptr = &metadata;
        }
    }

    if (a.json_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (metadata_ptr &&
            !report_write_json_ex(a.json_path, input_display ? input_display : a.input,
                                  &img, rf, us, us_ok, rf_count,
                                  hr_ptr, dr_ptr, metadata_ptr, &e)) {
            fprintf(stderr, "JSON write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else if (metadata_ptr) {
            printf("\nWrote JSON: %s\n", a.json_path);
        }
        sb_free(&e);
    }
    if (a.html_path) {
        StrBuf e;
        sb_init(&e, 256);
        if (metadata_ptr &&
            !report_write_html(a.html_path, input_display ? input_display : a.input,
                               &img, rf, us, us_ok, rf_count,
                               hr_ptr, dr_ptr, metadata_ptr, &e)) {
            fprintf(stderr, "HTML write failed:\n%s", e.s ? e.s : "");
            exit_code = 1;
        } else if (metadata_ptr) {
            printf("\nWrote HTML: %s\n", a.html_path);
        }
        sb_free(&e);
    }
    sb_free(&invocation);
    free(input_display);

    if (a.heuristics && hr_ptr) heuristics_free(&hr);
    if (dr_initialized) dynamic_report_free(&dr);

    sb_free(&ops);
    free(us_ok);
    free(us);
    free(rf);
    pe_free(&img);
    sb_free(&err);
    sb_free(&err2);
    buf_free_owned(owned);
    return exit_code;
}
```